



CompTIA

CertyIQ

Premium exam material

Get certification quickly with the CertyIQ Premium exam material.

Everything you need to prepare, learn & pass your certification exam easily. Lifetime free updates

First attempt guaranteed success.

<https://www.CertyIQ.com>

About CertyIQ

We here at CertyIQ eventually got enough of the industry's greedy exam paid for. Our team of IT professionals comes with years of experience in the IT industry Prior to training CertIQ we worked in test areas where we observed the horrors of the paywall exam preparation system.

The misuse of the preparation system has left our team disillusioned. And for that reason, we decided it was time to make a difference. We had to make In this way, CertyIQ was created to provide quality materials without stealing from everyday people who are trying to make a living.

Doubt Support

We have developed a very scalable solution using which we are able to solve 400+ doubts every single day with an average rating of 4.8 out of 5.

<https://www.certyiq.com>

Mail us on - certyiqofficial@gmail.com



Lifetime Free Updates

We provide lifetime free updates to our customers. To make life easier for our valued customers and fulfill their needs



Free Exam PDF

You are sure to pass the exam completely free of charge



Money Back Guarantee

We Provide 100% money back guarantee to our customer in case of any failure

John

October 19, 2022



Thanks you so much for your help. I scored 972 in my exam today. More than 90% were from your PDFs!

Dana

September 04, 2022



Thanks a lot for this updated AZ-900 Q&A. I just passed my exam and got 974, I followed both of your Az-900 videos and the 6 PDF, the PDFs are very much valid, all answers are correct. Could you please create a similar video/PDF for DP900, your content/PDF's is really awesome. The team did a really good job. Thank You 😊.

Ahamed Shibly

2 months ago



Customer support is really fast and helpful, I just finished my exam and this video along with the 6 PDF helped me pass! Definitely recommend getting the PDFs. Thank you!

October 22, 2022



Passed my exam today with 891 marks. Out of 52 questions, 51 were from certyiq PDFs including Contoso case study. Thank You certyiq team!

Henry Rome

2 months ago



These questions are real and 100 % valid. Thank you so much for your efforts, also your 4 PDFs are awesome, I passed the DP900 exam on 1 Sept. With 968 marks. Thanks a lot, buddy!

Esmaria

2 months ago



Simple easy to understand explanations. To anyone out there wanting to write AZ900, I highly recommend 6 PDF's. Thank you so much, appreciate all your hard work in having such great content. Passed my exam Today -3 September with 942 score.

Google

(Professional Data Engineer)

Professional Data Engineer on Google Cloud Platform

Total: **469 Questions**

Link: <https://certyiq.com/papers/google/professional-data-engineer>

Question: 1

CertyIQ

Your company built a TensorFlow neural-network model with a large number of neurons and layers. The model fits well for the training data. However, when tested against new data, it performs poorly. What method can you employ to address this?

- A. Threading
- B. Serialization
- C. Dropout Methods
- D. Dimensionality Reduction

Answer: C

Explanation:

The scenario describes a classic overfitting problem in machine learning: a model performs well on training data but poorly on unseen data. The key to addressing overfitting is to introduce techniques that prevent the model from memorizing the training set's specific patterns and instead encourage it to generalize to new data.

Dropout methods are specifically designed to combat overfitting in neural networks. They work by randomly "dropping out" (deactivating) a proportion of neurons during each training iteration. This forces the network to learn redundant representations, preventing reliance on specific neurons and encouraging more robust feature extraction. The random deactivation prevents complex co-adaptations between neurons, promoting simpler models. Consequently, the model is less likely to memorize the training data and more capable of generalizing to unseen data. Therefore, option C, dropout methods, is the most effective choice to reduce overfitting.

Threading (option A) deals with parallelizing processes within a system, serialization (option B) involves converting an object into a data stream for storage or transmission, and dimensionality reduction (option D) aims to reduce the number of variables in a dataset. These options are not directly related to the problem of overfitting. They address other performance and data handling needs but not model generalization.

For further information:

Dropout: <https://jmlr.org/papers/v15/srivastava14a.html> (Original Dropout paper)

Overfitting and Regularization: <https://developers.google.com/machine-learning/crash-course/generalization/peril-of-overfitting> (Google's Machine Learning Crash Course)

Question: 2

CertyIQ

You are building a model to make clothing recommendations. You know a user's fashion preference is likely to change over time, so you build a data pipeline to stream new data back to the model as it becomes available. How should you use this data to train the model?

- A. Continuously retrain the model on just the new data.
- B. Continuously retrain the model on a combination of existing data and the new data.
- C. Train on the existing data while using the new data as your test set.
- D. Train on the new data while using the existing data as your test set.

Answer: B

Explanation:

The correct approach, option B, involves continuously retraining the model using a combination of both existing and new data. This strategy addresses the dynamic nature of user fashion preferences as indicated in

the scenario. Continuously retraining solely on new data (option A) would lead to catastrophic forgetting, where the model loses its knowledge of past preferences, impacting its overall performance. Treating new data as a test set (option C) or vice versa (option D) wouldn't allow the model to learn from the most recent user behavior and adapt over time. Combining existing and new data enables incremental learning and maintains the model's accuracy. It allows the model to retain knowledge of past trends while incorporating the latest shifts in fashion preferences. This avoids the model becoming overly sensitive to short-term, potentially volatile trends. This strategy aligns with the concept of continual learning in machine learning. It ensures the model remains relevant and accurate over time by incorporating up-to-date data. The implementation might involve regular updates, batch processing of new data, or using streaming data pipelines with retraining triggers. This retraining would likely require version control to track model changes and ensure reproducibility. Google Cloud Platform (GCP) offers services like Vertex AI and Dataflow that are well-suited for such iterative model training pipelines. <https://cloud.google.com/vertex-ai> and <https://cloud.google.com/dataflow> provide details about these services. These links provide resources to understand and apply these services for machine learning pipelines.

Question: 3

CertyIQ

You designed a database for patient records as a pilot project to cover a few hundred patients in three clinics. Your design used a single database table to represent all patients and their visits, and you used self-joins to generate reports. The server resource utilization was at 50%. Since then, the scope of the project has expanded. The database must now store 100 times more patient records. You can no longer run the reports, because they either take too long or they encounter errors with insufficient compute resources. How should you adjust the database design?

- A. Add capacity (memory and disk space) to the database server by the order of 200.
- B. Shard the tables into smaller ones based on date ranges, and only generate reports with prespecified date ranges.
- C. Normalize the master patient-record table into the patient table and the visits table, and create other necessary tables to avoid self-join.
- D. Partition the table into smaller tables, with one for each clinic. Run queries against the smaller table pairs, and use unions for consolidated reports.

Answer: C

Explanation:

The correct answer is **C. Normalize the master patient-record table into the patient table and the visits table, and create other necessary tables to avoid self-join.** Here's why:

Option A, adding massive resources, is a short-term fix and not scalable. Increasing resources by 200 times is costly and doesn't address the fundamental design problem. Option B, sharding by date, can improve query performance for specific date ranges but doesn't tackle the core issue of the inefficient data model. Option D, partitioning by clinic, also provides some performance improvement by focusing queries on specific clinics but will still struggle with the self-joins when generating cross-clinic reports and does not address the data model flaw.

Option C directly addresses the root cause: a poorly designed denormalized database. The original single table with patient and visit data leads to massive redundancy and slow queries. By normalizing the data into separate patient and visit tables, we eliminate redundant data and avoid the costly self-joins. The patient table would store patient-specific information, and the visit table would contain details of each visit, referencing the patient table through a foreign key relationship. This allows for more efficient storage and retrieval of data, including faster, less resource-intensive reporting. Normalizing the data reduces data duplication, and simplifies the overall structure, allowing for a much more scalable solution. Creating additional tables to support the patient-visit relationship will also improve query speed and resource usage.

Here are some authoritative links for further reading:

Database Normalization: https://en.wikipedia.org/wiki/Database_normalization

Database Design Principles: <https://cloud.google.com/architecture/database-design-principles>

Relational Databases on Google Cloud: <https://cloud.google.com/sql/>

Normalization for Scalability: <https://www.geeksforgeeks.org/database-normalization-in-dbms/>

In summary, while some performance enhancements might be gained by sharding or partitioning, only normalization provides a sustainable solution and efficient foundation for future growth and reduces data redundancy.

Question: 4

CertyIQ

You create an important report for your large team in Google Data Studio 360. The report uses Google BigQuery as its data source. You notice that visualizations are not showing data that is less than 1 hour old. What should you do?

- A. Disable caching by editing the report settings.
- B. Disable caching in BigQuery by editing table details.
- C. Refresh your browser tab showing the visualizations.
- D. Clear your browser history for the past hour then reload the tab showing the virtualizations.

Answer: A

Explanation:

The correct answer is **A. Disable caching by editing the report settings.**

Here's why: Google Data Studio 360, by default, caches data from its data sources, including BigQuery, to improve performance and reduce query costs. This caching mechanism means that when a report is loaded, Data Studio might not fetch the latest data from BigQuery, leading to a delay in seeing fresh results, as indicated by the visualizations not showing data less than 1 hour old.

Options B, C, and D are incorrect. Option B is incorrect because BigQuery caching does not affect Data Studio's own caching behavior. Option C, refreshing the browser tab, can help with minor browser issues but will still show the cached data from Data Studio, not the fresh data from BigQuery. Option D, clearing browser history, only impacts the client's local cache and does not interact with Data Studio's data caching layer.

Disabling caching in Data Studio will force it to query BigQuery each time a report is viewed, ensuring the visualizations reflect the most up-to-date data. This setting is typically found within the data source connection settings within the Data Studio report itself. While this might increase query costs and loading times, it is crucial when real-time data visibility is essential for reports.

It's important to note that disabling caching is often only necessary when data changes frequently and timely updates are critical. If data changes less frequently, or real-time data is not required, using caching may be a preferred approach.

Authoritative links for further research:

Data Studio Data Freshness: <https://support.google.com/datastudio/answer/7020804?hl=en> - This Google Support article provides detailed information on how Data Studio handles data freshness and caching.

BigQuery Caching: <https://cloud.google.com/bigquery/docs/query-caching> - This Google Cloud documentation explains BigQuery's query caching mechanism, which is separate from Data Studio's caching.

Question: 5

An external customer provides you with a daily dump of data from their database. The data flows into Google Cloud Storage GCS as comma-separated values (CSV) files. You want to analyze this data in Google BigQuery, but the data could have rows that are formatted incorrectly or corrupted. How should you build this pipeline?

- A. Use federated data sources, and check data in the SQL query.
- B. Enable BigQuery monitoring in Google Stackdriver and create an alert.
- C. Import the data into BigQuery using the gcloud CLI and set max_bad_records to 0.
- D. Run a Google Cloud Dataflow batch pipeline to import the data into BigQuery, and push errors to another dead-letter table for analysis.

Answer: D

Explanation:

The correct answer is D because it offers a robust and scalable solution for handling potentially corrupted data within a BigQuery data ingestion pipeline. Dataflow is designed for ETL (Extract, Transform, Load) operations, making it ideal for processing the CSV data before it lands in BigQuery. By using a Dataflow pipeline, we can implement error handling and data validation logic. The pipeline can be configured to parse each CSV row, identify invalid or corrupted records, and instead of failing the entire import, write the bad records to a separate "dead-letter" table. This allows for detailed analysis of data quality issues without disrupting the ingestion of valid data.

Option A (federated data sources) is not suitable because while it allows querying data residing in GCS directly, it doesn't address data cleansing or error handling. Queries will fail upon encountering incorrectly formatted rows.

Option B (BigQuery monitoring) is helpful for identifying issues post-ingestion but does not prevent bad data from entering BigQuery in the first place. It's reactive, not proactive.

Option C (gcloud CLI with max_bad_records=0) is the opposite of what we want. Setting max_bad_records=0 will cause the entire import to fail if even a single bad record is encountered.

Dataflow's ability to handle errors and route them to a dead-letter queue provides a graceful and informative mechanism for managing imperfect data sources. This approach minimizes data loss and allows for continuous loading of usable data. It provides a pathway for analyzing and rectifying data quality issues in the incoming CSV files.

For further research:

Google Cloud Dataflow: <https://cloud.google.com/dataflow/docs>

BigQuery data loading: <https://cloud.google.com/bigquery/docs/loading-data>

Dataflow Error Handling: <https://cloud.google.com/dataflow/docs/guides/managing-your-pipeline#handling-errors>

Question: 6

Your weather app queries a database every 15 minutes to get the current temperature. The frontend is powered by Google App Engine and server millions of users. How should you design the frontend to respond to a database failure?

- A. Issue a command to restart the database servers.
- B. Retry the query with exponential backoff, up to a cap of 15 minutes.
- C. Retry the query every second until it comes back online to minimize staleness of data.

D. Reduce the query frequency to once every hour until the database comes back online.

Answer: B

Explanation:

The correct answer is B, retrying the query with exponential backoff up to a 15-minute cap. This strategy addresses database failures gracefully while minimizing the load on the failing system and ensuring application responsiveness.

Option A, restarting database servers, is an operational task, not a frontend concern. The frontend should handle failure scenarios without needing to manage infrastructure. Option C, retrying every second, would create a "thundering herd" problem, overwhelming the failing database with excessive requests. This could exacerbate the issue and prolong the outage. Option D, reducing the frequency to once an hour, significantly degrades user experience by providing stale data for extended periods.

Exponential backoff, on the other hand, starts with short retry intervals (e.g., seconds) and increases them gradually. This avoids overloading the database when it's struggling, allowing it time to recover. Limiting the backoff to 15 minutes ensures the application doesn't retry indefinitely and that data becomes available within the expected refresh window of 15 minutes once the database recovers. This approach aligns with principles of fault tolerance and resilience in cloud applications. It minimizes the impact of a failure on the user experience, keeps resources in check, and is generally considered best practice for handling transient failures in distributed systems.

Here are authoritative resources for further reading:

Google Cloud's documentation on error handling and retry strategies:

<https://cloud.google.com/architecture/best-practices-for-handling-errors>

AWS documentation on exponential backoff: <https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/> (While AWS-specific, the principles apply broadly)

Microsoft's documentation on retry patterns: <https://learn.microsoft.com/en-us/azure/architecture/patterns/retry> (Again, while Azure-specific, the retry principles apply broadly)

Question: 7

CertyIQ

You are creating a model to predict housing prices. Due to budget constraints, you must run it on a single resource-constrained virtual machine. Which learning algorithm should you use?

- A. Linear regression
- B. Logistic classification
- C. Recurrent neural network
- D. Feedforward neural network

Answer: A

Explanation:

The correct answer is A, Linear Regression. Linear regression is computationally inexpensive, requiring minimal memory and processing power, making it suitable for resource-constrained environments like a single virtual machine with limited resources. It's a simple algorithm that establishes a linear relationship between input features and the target variable (housing price), making it efficient to train and deploy. Unlike complex models like Recurrent Neural Networks (RNNs) and Feedforward Neural Networks (FFNNs) which require significant computational resources for training and inference, linear regression can achieve acceptable performance without demanding a lot of resources. Logistic classification (B) is primarily used for binary

classification problems, not regression like predicting housing prices. Therefore, A, Linear Regression is the most appropriate algorithm given the budget and resource constraints. Its efficiency outweighs the potential gains of more complex models, especially in this situation.

For more information on linear regression and its resource efficiency, refer to these resources:

Google Cloud AI Platform Training Concepts: <https://cloud.google.com/ai-platform/training/docs/concepts/training-overview> (While not directly about algorithms, it discusses efficient training)

Scikit-learn's documentation on Linear Regression: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html (Provides detailed insights into the algorithm and its practical implementation)

Understanding Resource Constraints in Machine Learning: <https://towardsdatascience.com/machine-learning-with-limited-resources-5c65b7bb38f1> (General article on efficient ML strategies)

Question: 8

CertyIQ

You are building new real-time data warehouse for your company and will use Google BigQuery streaming inserts. There is no guarantee that data will only be sent in once but you do have a unique ID for each row of data and an event timestamp. You want to ensure that duplicates are not included while interactively querying data. Which query type should you use?

- A. Include ORDER BY DESK on timestamp column and LIMIT to 1.
- B. Use GROUP BY on the unique ID column and timestamp column and SUM on the values.
- C. Use the LAG window function with PARTITION by unique ID along with WHERE LAG IS NOT NULL.
- D. Use the ROW_NUMBER window function with PARTITION by unique ID along with WHERE row equals 1.

Answer: D

Explanation:

Here's a detailed justification for why option D is the correct choice for deduplicating data in BigQuery using streaming inserts, along with explanations of why other options are less suitable:

Why Option D is Correct: Using ROW_NUMBER()

Option D utilizes the ROW_NUMBER() window function, which is ideally suited for deduplication in this scenario. The core idea is to partition the data based on the unique ID column. This essentially groups all rows with the same unique ID together. Then, ROW_NUMBER() assigns a sequential number to each row within each partition, ordered by the timestamp (though not explicitly stated in the query, this is usually the intent). Since the question indicates an event timestamp, we use this to determine which record is the most recent, and thus, should be kept. By filtering the result using WHERE row equals 1, we are retaining only the first row (the latest record) for each unique ID. This effectively removes all but the first row, which corresponds to the most recent occurrence of that unique ID, thus eliminating duplicates. This method is efficient for interactive queries as it performs the deduplication as part of the query itself and is easy to implement.

Why Other Options Are Incorrect:

Option A: ORDER BY and LIMIT: This approach might retrieve a single row but it would not give you one unique row per unique ID. The ORDER BY clause applied to a timestamp, combined with LIMIT 1 across the entire dataset, would simply pick the most recent entry overall. It fails to address deduplication based on unique ID.

Option B: GROUP BY and SUM: While GROUP BY can help organize data, grouping by both unique ID and timestamp would not lead to deduplication; instead it will aggregate values based on those two fields. Then

using sum, we would be aggregating all the values together instead of filtering the rows. This option is appropriate for aggregation and not deduplication.

Option C: LAG with PARTITION BY: The LAG function retrieves data from a previous row in the partition. When paired with WHERE LAG IS NOT NULL, it essentially checks if a previous record exists. However, this does not remove duplicates. It can be used to identify if a previous record exists in each partition, but not filter out the duplicate records.

Cloud Computing Concepts:

Window Functions: ROW_NUMBER() is a window function, which performs calculations across a set of table rows that are related to the current row. This makes it effective for tasks like ranking, deduplication, and moving averages.

BigQuery Streaming Inserts: BigQuery supports streaming inserts for real-time data ingestion. Because of the near real-time nature, there is not an enforcement of uniqueness at the time of the insert, so developers must use queries such as this one to handle the duplicates in the data.

Data Warehousing: The core principle of data warehousing is to have a single source of truth. The goal of deduplication is to ensure that each unique record is represented only once and is an important aspect of ensuring that the data is correct.

Authoritative Links for Further Research:

BigQuery Window Functions: <https://cloud.google.com/bigquery/docs/reference/standard-sql/window-functions>

BigQuery Streaming Inserts: <https://cloud.google.com/bigquery/streaming-data-into-bigquery>

BigQuery Deduplication: <https://towardsdatascience.com/how-to-deduplicate-data-in-google-bigquery-2905a00d918>

Question: 9

CertyIQ

Your company is using WILDCARD tables to query data across multiple tables with similar names. The SQL statement is currently failing with the following error:

```
# Syntax error : Expected end of statement but got "-" at [4:11]
SELECT age
FROM
    bigquery-public-data.noaa_gsod.gsod
WHERE
    age != 99
    AND_TABLE_SUFFIX = '1929'
ORDER BY
    age DESC
```

Which table name will make the SQL statement work correctly?

- A. 'bigquery-public-data.noaa_gsod.gsod'
- B. bigquery-public-data.noaa_gsod.gsod*
- C. 'bigquery-public-data.noaa_gsod.gsod'*
- D. 'bigquery-public-data.noaa_gsod.gsod*'

Answer: D

Explanation:

Reference:

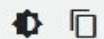
<https://cloud.google.com/bigquery/docs/wildcard-tables>
" target="_blank">

Filtering selected tables using _TABLE_SUFFIX

To restrict a query so that it scans only a specified set of tables, use the `_TABLE_SUFFIX` pseudo column in a `WHERE` clause with a condition that is a constant expression.

The `_TABLE_SUFFIX` pseudo column contains the values matched by the table wildcard. For example, the previous sample query, which scans all tables from the 1940s, uses a table wildcard to represent the last digit of the year:

```
FROM  
  `bigquery-public-data.noaa_gsod.gsod194*`
```



Question: 10

CertyIQ

Your company is in a highly regulated industry. One of your requirements is to ensure individual users have access only to the minimum amount of information required to do their jobs. You want to enforce this requirement with Google BigQuery. Which three approaches can you take? (Choose three.)

- A. Disable writes to certain tables.
- B. Restrict access to tables by role.
- C. Ensure that the data is encrypted at all times.
- D. Restrict BigQuery API access to approved users.
- E. Segregate data across multiple tables or databases.
- F. Use Google Stackdriver Audit Logging to determine policy violations.

Answer: BDE

Explanation:

The correct options for enforcing the principle of least privilege in BigQuery are **B. Restrict access to tables by role**, **D. Restrict BigQuery API access to approved users**, and **E. Segregate data across multiple tables or datasets**.

Option B aligns directly with role-based access control (RBAC), a fundamental security principle. By assigning roles with specific permissions (e.g., "read-only" on certain tables) to users, you grant only the necessary access for their function. <https://cloud.google.com/bigquery/docs/access-control>

Option D reinforces access control at the API level. By limiting which users can interact with the BigQuery API, you prevent unauthorized data access or manipulation, regardless of table permissions. This ensures that only validated application or service accounts can interact with BigQuery.
<https://cloud.google.com/iam/docs/overview>

Option E addresses data exposure by separating sensitive information into different tables or datasets. This allows you to grant granular permissions, ensuring that users only see the data necessary for their work. Different access control policies can then be applied to each table or dataset, minimizing the impact of any security breach. <https://cloud.google.com/bigquery/docs/datasets>

Option A, disabling writes, only addresses modification and does not restrict read access, which is essential

for least privilege. Option C, encryption, is important for data security but doesn't enforce access control based on user roles. Option F, auditing, helps identify violations but does not implement access control mechanisms. Together, options B, D, and E provide a comprehensive approach to limiting data exposure and implementing least privilege principles in BigQuery.

Question: 11

CertyIQ

You are designing a basket abandonment system for an ecommerce company. The system will send a message to a user based on these rules:

- ⇒ No interaction by the user on the site for 1 hour
- Has added more than \$30 worth of products to the basket

-
- ⇒ Has not completed a transaction

You use Google Cloud Dataflow to process the data and decide if a message should be sent. How should you design the pipeline?

- A. Use a fixed-time window with a duration of 60 minutes.
- B. Use a sliding time window with a duration of 60 minutes.
- C. Use a session window with a gap time duration of 60 minutes.
- D. Use a global window with a time based trigger with a delay of 60 minutes.

Answer: C

Explanation:

Use a session window with a gap time duration of 60 minutes.

Session windows group elements that are close to each other in time, with a "gap time" defining how long of a period of inactivity signals the end of a session. If no new data arrives for a specified gap duration, the session is considered complete and processed.

You would define the "gap time" as 60 minutes. As long as the user is actively making progress towards a transaction (e.g., adding items to a cart, entering shipping details), new events will keep the session window open. If 60 minutes pass without any transaction-related activity, the session window will close, and you can then detect that a transaction was not completed and trigger a message. This directly addresses the "has not completed a transaction" scenario.

Question: 12

CertyIQ

Your company handles data processing for a number of different clients. Each client prefers to use their own suite of analytics tools, with some allowing direct query access via Google BigQuery. You need to secure the data so that clients cannot see each other's data. You want to ensure appropriate access to the data.

Which three steps should you take? (Choose three.)

- A. Load data into different partitions.
- B. Load data into a different dataset for each client.
- C. Put each client's BigQuery dataset into a different table.
- D. Restrict a client's dataset to approved users.
- E. Only allow a service account to access the datasets.
- F. Use the appropriate identity and access management (IAM) roles for each client's users.

Answer: BDF

Explanation:

The correct answer is BDF because it directly addresses the requirements of data isolation and secure access. Loading data into different datasets for each client (B) is fundamental for achieving logical separation. This ensures that one client's data is completely isolated from another's at the BigQuery level. Restricting a client's dataset to approved users (D) using IAM policies is crucial for granting access only to authorized individuals. This prevents unauthorized access even if users exist within the same organization. Finally, leveraging appropriate IAM roles for each client's users (F) allows for granular control over what actions specific users can perform within their designated datasets. This follows the principle of least privilege, granting only necessary permissions to each user. Options A and C are incorrect because they don't provide the necessary data isolation or are inefficient for organizing and securing data for multiple clients. Option E, while a security practice, isn't sufficient on its own. Service accounts would need further restriction through IAM to limit access to specific datasets, making option F more directly relevant to the given scenario.

Relevant links for further research:

Google Cloud IAM Overview: <https://cloud.google.com/iam/docs/overview>

BigQuery Access Control: <https://cloud.google.com/bigquery/docs/access-control-overview>

BigQuery Datasets: <https://cloud.google.com/bigquery/docs/datasets>

Question: 13**CertyIQ**

You want to process payment transactions in a point-of-sale application that will run on Google Cloud Platform. Your user base could grow exponentially, but you do not want to manage infrastructure scaling. Which Google database service should you use?

- A. Cloud SQL
- B. BigQuery
- C. Cloud Bigtable
- D. Cloud Datastore

Answer: D**Explanation:**

The correct answer is **D. Cloud Datastore**.

Cloud Datastore is a fully managed, schemaless, NoSQL document database designed for automatic scaling and high availability. This directly addresses the requirement of not wanting to manage infrastructure scaling as the user base grows exponentially. Its serverless nature means Google handles the underlying infrastructure management, allowing you to focus on application development. Unlike relational databases, Datastore's schemaless nature accommodates evolving data structures easily, which is often beneficial in rapidly changing applications.

Cloud SQL (A) is a managed relational database service, while useful, requires more management regarding scaling and doesn't scale as seamlessly as Datastore. BigQuery (B) is an analytical data warehouse primarily designed for large-scale data analysis, not for transactional processing within a point-of-sale system. Cloud Bigtable (C) is a high-performance, scalable NoSQL wide-column store ideal for large datasets and high-throughput applications but is more complex to manage and not the most suitable for document-style transactions.

Therefore, Cloud Datastore's combination of automatic scaling, high availability, and a simple data model make it the most appropriate choice for the given scenario, where the user base is expected to grow dramatically without manual infrastructure intervention.

Authoritative Links:

1. **Cloud Datastore Overview:** <https://cloud.google.com/datastore/docs/concepts/overview>
2. **Cloud Datastore vs. Cloud SQL:** <https://cloud.google.com/datastore/docs/concepts/choosing-database>
3. **Cloud Datastore Scalability:** <https://cloud.google.com/datastore/docs/concepts/scaling-and-planning>

Question: 14

CertyIQ

You want to use a database of information about tissue samples to classify future tissue samples as either normal or mutated. You are evaluating an unsupervised anomaly detection method for classifying the tissue samples. Which two characteristic support this method? (Choose two.)

- A. There are very few occurrences of mutations relative to normal samples.
- B. There are roughly equal occurrences of both normal and mutated samples in the database.
- C. You expect future mutations to have different features from the mutated samples in the database.
- D. You expect future mutations to have similar features to the mutated samples in the database.
- E. You already have labels for which samples are mutated and which are normal in the database.

Answer: AC

Explanation:

The correct answer is AC because unsupervised anomaly detection is most effective when dealing with imbalanced datasets and novel anomalies. Option A is correct because the scarcity of mutation samples relative to normal samples makes anomaly detection a suitable approach. Since normal samples are prevalent, the algorithm learns their patterns well, making deviations (mutations) easily identifiable as outliers. In contrast, Option B, with roughly equal samples, suggests a supervised learning scenario where you could train a model to directly classify normal and mutated samples. Option C aligns with the core principle of anomaly detection, where the model aims to identify data points that deviate from learned patterns. Anomaly detection is designed to detect novel, previously unseen, features unlike the seen mutated samples. Option D contradicts this principle; if future mutations had similar features, supervised classification would be more appropriate. Option E is incorrect because labeled data indicates supervised learning not unsupervised anomaly detection. Therefore, options A and C perfectly describe scenarios where an unsupervised approach makes sense for your specific classification task.

Supporting Concepts and Resources:

Unsupervised Learning: This technique learns from unlabeled data, discovering underlying patterns or structures. Anomaly detection is a form of unsupervised learning.

[Google Cloud AI Platform: Unsupervised Learning](#)

Anomaly Detection: Identifying data points that differ significantly from the majority of the dataset. Useful in situations with imbalanced datasets or novel anomalies.

[Wikipedia: Anomaly Detection](#)

Imbalanced Datasets: A class distribution where one class dominates the other. Anomaly detection can handle such situations.

[Towards Data Science: Handling Imbalanced Datasets in Machine Learning](#)

Question: 15

You need to store and analyze social media postings in Google BigQuery at a rate of 10,000 messages per minute in near real-time. Initially, design the application to use streaming inserts for individual postings. Your application also performs data aggregations right after the streaming inserts. You discover that the queries after streaming inserts do not exhibit strong consistency, and reports from the queries might miss in-flight data. How can you adjust your application design?

- A. Re-write the application to load accumulated data every 2 minutes.
- B. Convert the streaming insert code to batch load for individual messages.
- C. Load the original message to Google Cloud SQL, and export the table every hour to BigQuery via streaming inserts.
- D. Estimate the average latency for data availability after streaming inserts, and always run queries after waiting twice as long.

Answer: D

Explanation:

The correct answer is **D. Estimate the average latency for data availability after streaming inserts, and always run queries after waiting twice as long.**

Here's why:

BigQuery streaming inserts offer near real-time data ingestion, but data isn't immediately available for querying due to internal processing. This "eventual consistency" means there's a latency period between insertion and query availability. Option D directly addresses this by acknowledging and accounting for this latency. By estimating the average delay and then waiting twice as long, the application increases the likelihood that all recently streamed data is available for consistent querying. This is a practical approach for handling eventual consistency in a system requiring near real-time analysis.

Option A, reloading data every 2 minutes, changes the architecture to a micro-batch approach, deviating from the initial near real-time requirement and potentially introducing complexity without addressing the core consistency issue within the micro-batches themselves. Option B, converting to batch loading for individual messages, is inefficient as it still doesn't guarantee consistency when doing analysis right after the load and defeats the purpose of streaming inserts. Option C, using Cloud SQL as an intermediate and hourly exports, introduces significant latency and overhead and is not suitable for near real-time analysis. It also shifts the initial goal from streaming to batch ingestion.

The chosen answer focuses on managing the inherent eventual consistency of BigQuery streaming inserts without fundamentally altering the streaming approach, ensuring near real-time analysis as much as possible. It represents a pragmatic solution that allows the application to remain true to the initial design while addressing the observed consistency problem.

For further reading on BigQuery streaming inserts and their consistency properties, consult the official Google Cloud documentation:

Streaming data into BigQuery: <https://cloud.google.com/bigquery/docs/stream-data-into-bigquery>
BigQuery Data consistency: <https://cloud.google.com/bigquery/docs/data-consistency>

Question: 16

Your startup has never implemented a formal security policy. Currently, everyone in the company has access to the datasets stored in Google BigQuery. Teams have freedom to use the service as they see fit, and they have not documented their use cases. You have been asked to secure the data warehouse. You need to discover what everyone is doing. What should you do first?

- A. Use Google Stackdriver Audit Logs to review data access.
- B. Get the identity and access management (IAM) policy of each table
- C. Use Stackdriver Monitoring to see the usage of BigQuery query slots.
- D. Use the Google Cloud Billing API to see what account the warehouse is being billed to.

Answer: A

Explanation:

The most critical first step in securing the data warehouse, given the current situation of unrestricted access and undocumented usage, is to understand who is accessing what data and when. Option A, using Google Stackdriver Audit Logs to review data access, directly addresses this need. Audit logs capture detailed records of user actions, including data reads and modifications, providing a clear picture of current access patterns. This insight is crucial for identifying potential security vulnerabilities and establishing a baseline for access control.

Option B, while important eventually, is premature. Obtaining the IAM policy of each table only shows who could access the data, not who is actually doing so. Similarly, option C (Stackdriver Monitoring) focuses on resource consumption (query slots) rather than user activity. Option D (Billing API) is useful for cost management, not access monitoring.

Therefore, auditing current access behavior is the logical priority before attempting to define access policies. This enables informed decision-making and minimizes disruption during the security implementation process. Using audit logs allows you to identify user roles, their usage patterns, and the specific datasets being accessed, which is fundamental for designing appropriate access control measures. Without this information, any security policy would be based on conjecture rather than evidence.

Relevant Google Cloud Documentation:

Cloud Logging (formerly Stackdriver Logging): <https://cloud.google.com/logging> - Provides comprehensive information on audit logs and their capabilities.

Audit logging: <https://cloud.google.com/logging/docs/audit/> - Details different types of audit logs available on GCP.

BigQuery Audit Logs: <https://cloud.google.com/bigquery/docs/audit-logs> - Specific documentation on BigQuery's audit logging capabilities.

Question: 17

CertyIQ

Your company is migrating their 30-node Apache Hadoop cluster to the cloud. They want to re-use Hadoop jobs they have already created and minimize the management of the cluster as much as possible. They also want to be able to persist data beyond the life of the cluster. What should you do?

- A. Create a Google Cloud Dataflow job to process the data.
- B. Create a Google Cloud Dataproc cluster that uses persistent disks for HDFS.
- C. Create a Hadoop cluster on Google Compute Engine that uses persistent disks.
- D. Create a Cloud Dataproc cluster that uses the Google Cloud Storage connector.
- E. Create a Hadoop cluster on Google Compute Engine that uses Local SSD disks.

Answer: D

Explanation:

The most appropriate solution is **D. Create a Cloud Dataproc cluster that uses the Google Cloud Storage connector**. Here's why:

Reusing Hadoop Jobs: Cloud Dataproc is a managed service designed to run Hadoop and Spark jobs, allowing you to leverage your existing Hadoop code with minimal modifications. This satisfies the requirement of reusing existing jobs.

Minimizing Cluster Management: Dataproc is a fully managed service, abstracting away much of the underlying infrastructure management. This means you don't have to deal with tasks like cluster setup, scaling, and patching, meeting the requirement to minimize management.

Persisting Data Beyond Cluster Lifespan: The Google Cloud Storage (GCS) connector integrates seamlessly with Dataproc. By storing data in GCS, data persists independently of the Dataproc cluster. This ensures data availability even if the cluster is stopped or scaled down.

Alternatives A, B, C, and E: Option A (Dataflow) requires rewriting Hadoop jobs, violating a key requirement. Option B (persistent disks on Dataproc) is not cost-effective and doesn't address data persistence beyond the cluster's lifespan. Option C (Hadoop on Compute Engine with persistent disks) introduces significant management overhead. Option E (Hadoop on Compute Engine with local SSD) does not provide data persistence beyond the cluster's lifespan and is not cost-effective for data storage.

Cloud Dataproc Benefits: Dataproc enables faster cluster deployment, easier job submission, and automatic cluster scaling. It also integrates with other Google Cloud services.

Google Cloud Storage Benefits: GCS is a cost-effective, durable, and highly available object storage solution. It is the recommended storage solution for use with Dataproc when data persistence is needed.

Authoritative links:

Cloud Dataproc Documentation: <https://cloud.google.com/dataproc/docs>

Google Cloud Storage Documentation: <https://cloud.google.com/storage/docs>

Dataproc and Cloud Storage Connector:

<https://cloud.google.com/dataproc/docs/concepts/connectors/cloud-storage>

In conclusion, option D best aligns with the requirements for reusing existing Hadoop jobs, minimizing management, and ensuring persistent data storage beyond the cluster's lifespan through the use of managed Cloud Dataproc and the Cloud Storage connector.

Question: 18

CertyIQ

Business owners at your company have given you a database of bank transactions. Each row contains the user ID, transaction type, transaction location, and transaction amount. They ask you to investigate what type of machine learning can be applied to the data. Which three machine learning applications can you use? (Choose three.)

- A. Supervised learning to determine which transactions are most likely to be fraudulent.
- B. Unsupervised learning to determine which transactions are most likely to be fraudulent.
- C. Clustering to divide the transactions into N categories based on feature similarity.
- D. Supervised learning to predict the location of a transaction.
- E. Reinforcement learning to predict the location of a transaction.
- F. Unsupervised learning to predict the location of a transaction.

Answer: BCD

Explanation:

The correct machine learning applications for analyzing bank transaction data are B, C, and D. Option B, unsupervised learning, is suitable for detecting fraudulent transactions because fraud is often rare and unlabeled in datasets. Algorithms like anomaly detection can identify unusual patterns without needing predefined fraudulent examples. Option C, clustering, allows grouping similar transactions based on characteristics, potentially revealing distinct spending habits or geographic patterns that can inform business insights or fraud detection efforts. Option D, supervised learning, is applicable when the location of a

transaction needs to be predicted based on other transaction features, provided labeled historical data with known locations is available.

Option A, supervised learning for fraud detection, might seem viable, but given that fraud is typically infrequent and difficult to label comprehensively, relying solely on supervised methods could be challenging and less effective initially than unsupervised approaches. Reinforcement learning (Option E) is not appropriate for this use case because it deals with learning through interaction with an environment, not directly from labeled or unlabeled historical data, and there is no environment for this learning task. Option F, using unsupervised learning to predict location is also not well-suited, since location prediction is inherently a supervised learning problem which requires labeled historical transaction data with known locations.

Unsupervised techniques reveal hidden structures in the data without labels, whereas supervised methods learn from labeled data to make predictions. Clustering is an unsupervised technique specifically designed to group similar data points. These methods are all valuable for analyzing transaction data.

Authoritative Links for Further Research:

Google Cloud AI/ML Documentation: <https://cloud.google.com/ai-platform/docs>

Supervised Learning: <https://developers.google.com/machine-learning/crash-course/ml-intro/supervised-learning>

Unsupervised Learning: <https://developers.google.com/machine-learning/crash-course/ml-intro/unsupervised-learning>

Clustering: <https://scikit-learn.org/stable/modules/clustering.html> (Scikit-learn documentation, a popular Python library for ML)

Anomaly Detection: https://scikit-learn.org/stable/modules/outlier_detection.html (Scikit-learn documentation)

Question: 19

CertyIQ

Your company's on-premises Apache Hadoop servers are approaching end-of-life, and IT has decided to migrate the cluster to Google Cloud Dataproc. A like-for-like migration of the cluster would require 50 TB of Google Persistent Disk per node. The CIO is concerned about the cost of using that much block storage. You want to minimize the storage cost of the migration. What should you do?

- A. Put the data into Google Cloud Storage.
- B. Use preemptible virtual machines (VMs) for the Cloud Dataproc cluster.
- C. Tune the Cloud Dataproc cluster so that there is just enough disk for all data.
- D. Migrate some of the cold data into Google Cloud Storage, and keep only the hot data in Persistent Disk.

Answer: A

Explanation:

The optimal solution is to store the data in Google Cloud Storage (GCS) (Option A). Here's why: GCS is a cost-effective object storage service ideal for storing large datasets. Unlike Persistent Disk, which is block storage attached to VMs, GCS decouples storage from compute, leading to significant cost savings. Dataproc clusters can directly access data in GCS, allowing computation without needing large, expensive Persistent Disks on each node. This approach eliminates the need for 50 TB of disk space per node. Option B, using preemptible VMs, lowers compute costs but does not address the high storage costs. Option C, tuning disk space, doesn't fundamentally solve the issue of expensive block storage for large datasets. Option D, while helpful, doesn't go far enough; it is a partial implementation of option A. GCS offers a scalable and durable storage solution with various tiers to further optimize costs for different data access frequencies. In short, leveraging GCS for data storage significantly reduces infrastructure expenses compared to using large amounts of Persistent Disk.

Further Research:

Google Cloud Storage: <https://cloud.google.com/storage>

Google Cloud Dataproc: <https://cloud.google.com/dataproc>

Cost optimization with GCS: <https://cloud.google.com/storage/docs/cost-optimization>

Question: 20

CertyIQ

You work for a car manufacturer and have set up a data pipeline using Google Cloud Pub/Sub to capture anomalous sensor events. You are using a push subscription in Cloud Pub/Sub that calls a custom HTTPS endpoint that you have created to take action of these anomalous events as they occur. Your custom HTTPS endpoint keeps getting an inordinate amount of duplicate messages. What is the most likely cause of these duplicate messages?

- A. The message body for the sensor event is too large.
- B. Your custom endpoint has an out-of-date SSL certificate.
- C. The Cloud Pub/Sub topic has too many messages published to it.
- D. Your custom endpoint is not acknowledging messages within the acknowledgement deadline.

Answer: D

Explanation:

The correct answer is **D: Your custom endpoint is not acknowledging messages within the acknowledgment deadline**. Here's why:

Cloud Pub/Sub's push subscriptions rely on acknowledgments (ACKs) from the subscriber endpoint to confirm successful message delivery. If a push endpoint fails to acknowledge a message within the configured acknowledgment deadline, Pub/Sub assumes the message wasn't processed and retries delivery. This retry mechanism results in duplicate messages being sent to the endpoint. Option A is incorrect as message size limits typically lead to message rejections, not duplicates. Option B is incorrect as SSL certificate issues would likely cause connection failures and no messages processed at all, not duplicates. Option C is also incorrect, as the number of messages on the topic doesn't directly cause duplicate deliveries to push subscriptions. The most common cause for the described duplicate message issue is the push endpoint failing to send an ACK within the deadline. It is critical to ensure that the endpoint can process the messages and respond with an acknowledgement within the Pub/Sub deadline to avoid duplicate messages. This behavior is part of the "at-least-once" delivery guarantee provided by Pub/Sub, which prioritizes reliability over absolute message uniqueness.

Authoritative Links:

Google Cloud Pub/Sub Push Subscriptions Documentation: <https://cloud.google.com/pubsub/docs/push>

Google Cloud Pub/Sub Acknowledgment and Ack Deadlines:

https://cloud.google.com/pubsub/docs/subscriber#ack_deadlines

Google Cloud Pub/Sub Delivery Guarantees:

https://cloud.google.com/pubsub/docs/overview#delivery_guarantees

Question: 21

CertyIQ

Your company uses a proprietary system to send inventory data every 6 hours to a data ingestion service in the cloud. Transmitted data includes a payload of several fields and the timestamp of the transmission. If there are any concerns about a transmission, the system re-transmits the data. How should you deduplicate the data most efficiently?

- A. Assign global unique identifiers (GUID) to each data entry.
- B. Compute the hash value of each data entry, and compare it with all historical data.
- C. Store each data entry as the primary key in a separate database and apply an index.
- D. Maintain a database table to store the hash value and other metadata for each data entry.

Answer: A

Explanation:

The most efficient way to deduplicate the inventory data is by assigning Global Unique Identifiers (GUIDs) to each entry (Option A). GUIDs, also known as UUIDs, are statistically unique 128-bit identifiers that can be generated independently without requiring a central authority. This eliminates the need to compare new data with historical data, as each entry has a unique identifier. Upon data ingestion, checking for pre-existing GUIDs is a computationally lightweight operation. This approach is highly scalable and performs well even with large volumes of data, due to the minimal lookup complexity. Hash value comparison (Option B) is less efficient, requiring hashing each entry and comparing it to a potential large set of historical hashes, incurring significant computational cost and storage. Storing entire data entries as primary keys in a database with indexing (Option C) is computationally expensive and storage intensive, further complicated by the need for large tables and complicated indexing. Maintaining a database table storing hashes and metadata (Option D) involves the same computationally intensive comparison process like option B and also involves more data storage, thus it will be more complex and less efficient. GUIDs eliminate the need to search across historical datasets by assigning a unique identifier upfront. This translates into lower processing times and lower infrastructure costs for data deduplication. The process becomes simple: check if a GUID exists, if not, process the data, if yes, discard the data. Therefore, using GUIDs aligns with best practices in handling duplicate data in data engineering and efficient data ingestion, making Option A the most effective solution.

Authoritative Links for Further Research:

UUIDs/GUIDs: https://en.wikipedia.org/wiki/Universally_unique_identifier

Data Deduplication: <https://cloud.google.com/architecture/data-deduplication-options>

Scalable Data Processing: <https://cloud.google.com/solutions/scalable-data-processing>

Question: 22

CertyIQ

Your company has hired a new data scientist who wants to perform complicated analyses across very large datasets stored in Google Cloud Storage and in a Cassandra cluster on Google Compute Engine. The scientist primarily wants to create labelled data sets for machine learning projects, along with some visualization tasks. She reports that her laptop is not powerful enough to perform her tasks and it is slowing her down. You want to help her perform her tasks. What should you do?

- A. Run a local version of Jupiter on the laptop.
- B. Grant the user access to Google Cloud Shell.
- C. Host a visualization tool on a VM on Google Compute Engine.
- D. Deploy Google Cloud Datalab to a virtual machine (VM) on Google Compute Engine.

Answer: D

Explanation:

Here's a detailed justification for choosing option D, deploying Google Cloud Datalab, as the best solution:

The data scientist needs a powerful environment to analyze large datasets from both Google Cloud Storage and a Cassandra cluster, create labeled datasets for machine learning, and perform visualizations, all of which

exceeds the capacity of their laptop. Option A, running a local Jupyter notebook, simply replicates the current underpowered setup. Option B, Google Cloud Shell, provides a command-line interface but lacks the necessary interactive notebook environment for data exploration and visualization. Option C, hosting a visualization tool on a Compute Engine VM, addresses a single aspect (visualization) but doesn't address the entire workflow involving data access, processing, and labeling.

Google Cloud Datalab (now superseded by Vertex AI Workbench, but Datalab concepts still apply) provides a fully managed and scalable Jupyter notebook environment hosted on a Google Compute Engine VM. This addresses the scientist's needs by offering: a powerful compute resource for processing large datasets, pre-installed data science libraries, direct access to Google Cloud Storage and other Google Cloud resources, support for various data connectors including Cassandra, and visualization capabilities within the notebook environment. It facilitates collaborative work and enables version control using Git. The data scientist can easily perform interactive data exploration, analysis, and machine learning tasks without being constrained by local laptop limitations.

The solution offers scalability, allowing the data scientist to process data efficiently and meet their machine learning needs. It also handles the infrastructure management behind the scenes, allowing the scientist to focus on data analysis.

Relevant Links for further research:

[Google Cloud Datalab \(archived documentation\)](#): This provides an overview of the service and its functionalities. Note that Datalab has been replaced by Vertex AI Workbench.

[Vertex AI Workbench](#): This is the successor to Datalab, offering similar capabilities with enhanced features. Understanding Workbench helps understand Datalab principles.

[Jupyter Notebooks](#): This gives background on the interactive notebook environment used by Datalab and Vertex AI Workbench.

Question: 23

CertyIQ

You are deploying 10,000 new Internet of Things devices to collect temperature data in your warehouses globally. You need to process, store and analyze these very large datasets in real time. What should you do?

- A. Send the data to Google Cloud Datastore and then export to BigQuery.
- B. Send the data to Google Cloud Pub/Sub, stream Cloud Pub/Sub to Google Cloud Dataflow, and store the data in Google BigQuery.
- C. Send the data to Cloud Storage and then spin up an Apache Hadoop cluster as needed in Google Cloud Dataproc whenever analysis is required.
- D. Export logs in batch to Google Cloud Storage and then spin up a Google Cloud SQL instance, import the data from Cloud Storage, and run an analysis as needed.

Answer: B

Explanation:

Option B is the most appropriate solution for real-time processing, storage, and analysis of data from 10,000 IoT devices. Google Cloud Pub/Sub is a messaging service that allows for asynchronous data ingestion, handling a high volume of messages from various sources like IoT devices. This aligns with the need to manage streaming temperature data. Cloud Dataflow is a fully managed, serverless stream and batch data processing service, ideal for real-time analytics on the streaming Pub/Sub data. It can perform transformations and computations on the incoming data stream. Finally, BigQuery, a fully-managed data warehouse, provides scalable storage and enables rapid analysis of large datasets. The streamed and processed data from Dataflow can be stored directly in BigQuery for analysis, such as trend identification and anomaly detection. Option A suggests using Cloud Datastore, which is not optimized for the analytical

workloads necessary for this scenario, and exporting to BigQuery introduces an unnecessary delay. Option C involves batch processing with Cloud Storage and Dataproc, which contradicts the real-time analysis requirement. Option D is entirely batch-oriented, using Cloud Storage, Cloud SQL, and import/analysis, thus, is unsuitable for real-time analysis.

Here are some authoritative links for further research:

Google Cloud Pub/Sub: <https://cloud.google.com/pubsub/docs>

Google Cloud Dataflow: <https://cloud.google.com/dataflow/docs>

Google BigQuery: <https://cloud.google.com/bigquery/docs>

Question: 24

CertyIQ

You have spent a few days loading data from comma-separated values (CSV) files into the Google BigQuery table `CLICK_STREAM`. The column `DT` stores the epoch time of click events. For convenience, you chose a simple schema where every field is treated as the `STRING` type. Now, you want to compute web session durations of users who visit your site, and you want to change its data type to the `TIMESTAMP`. You want to minimize the migration effort without making future queries computationally expensive. What should you do?

- A. Delete the table `CLICK_STREAM`, and then re-create it such that the column `DT` is of the `TIMESTAMP` type. Reload the data.
- B. Add a column `TS` of the `TIMESTAMP` type to the table `CLICK_STREAM`, and populate the numeric values from the column `TS` for each row. Reference the column `TS` instead of the column `DT` from now on.
- C. Create a view `CLICK_STREAM_V`, where strings from the column `DT` are cast into `TIMESTAMP` values. Reference the view `CLICK_STREAM_V` instead of the table `CLICK_STREAM` from now on.
- D. Add two columns to the table `CLICK_STREAM`: `TS` of the `TIMESTAMP` type and `IS_NEW` of the `BOOLEAN` type. Reload all data in append mode. For each appended row, set the value of `IS_NEW` to true. For future queries, reference the column `TS` instead of the column `DT`, with the `WHERE` clause ensuring that the value of `IS_NEW` must be true.
- E. Construct a query to return every row of the table `CLICK_STREAM`, while using the built-in function to cast strings from the column `DT` into `TIMESTAMP` values. Run the query into a destination table `NEW_CLICK_STREAM`, in which the column `TS` is the `TIMESTAMP` type. Reference the table `NEW_CLICK_STREAM` instead of the table `CLICK_STREAM` from now on. In the future, new data is loaded into the table `NEW_CLICK_STREAM`.

Answer: E

Explanation:

The most efficient approach to convert the `DT` column from `STRING` to `TIMESTAMP` in BigQuery while minimizing migration effort and computational overhead is option E. Creating a new table, `NEW_CLICK_STREAM`, with the correct `TIMESTAMP` data type using a query that casts the `DT` strings avoids altering the original table and preserves the original data for potential historical analysis. This avoids the complexity and downtime of deleting and recreating the entire table (A) or loading data into additional columns on the existing table (B, D). While creating a view (C) is a valid way to represent data with the correct data type, it performs the casting operation every time it's queried, adding computational overhead. Using a destination table with the correct data type is more computationally performant long term. Moreover, this approach aligns well with the practice of creating data pipelines. By making the new table the point of reference going forward, this avoids complex conditional clauses on older data, and keeps the processing simple. Moving forward new data will be loaded directly into the new schema. Option E strikes a balance between avoiding costly reloads and ensuring performance without unnecessary complexity or introducing performance bottlenecks. Furthermore, new data is loaded into the new table eliminating the need for complicated conditions to differentiate between existing data and new data.

Authoritative links:

BigQuery documentation on data type conversion:

https://cloud.google.com/bigquery/docs/reference/standard-sql/conversion_functions

BigQuery documentation on creating tables from queries: https://cloud.google.com/bigquery/docs/writing-results#writing_query_results_to_a_new_table

Question: 25

CertyIQ

You want to use Google Stackdriver Logging to monitor Google BigQuery usage. You need an instant notification to be sent to your monitoring tool when new data is appended to a certain table using an insert job, but you do not want to receive notifications for other tables. What should you do?

- A. Make a call to the Stackdriver API to list all logs, and apply an advanced filter.
- B. In the Stackdriver logging admin interface, and enable a log sink export to BigQuery.
- C. In the Stackdriver logging admin interface, enable a log sink export to Google Cloud Pub/Sub, and subscribe to the topic from your monitoring tool.
- D. Using the Stackdriver API, create a project sink with advanced log filter to export to Pub/Sub, and subscribe to the topic from your monitoring tool.

Answer: D

Explanation:

The correct answer is D because it leverages the appropriate Google Cloud tools for targeted log filtering and real-time notifications. Stackdriver (now Cloud Logging) provides powerful capabilities for managing and analyzing logs. To achieve the specific requirement of receiving notifications only for insert jobs into a particular BigQuery table, a project sink with an advanced log filter is needed.

Here's why the other options are unsuitable:

A: Listing all logs and then applying a filter on the client side is inefficient and not real-time. This approach is not suitable for instant notifications.

B: Exporting all logs to BigQuery would incur unnecessary storage costs and introduce complexity when filtering for the specific table and insert jobs, which does not satisfy the requirement of instant notification to monitoring tools.

C: While exporting to Pub/Sub is a good first step, filtering and scoping log messages is needed via log sink with advanced filtering.

Option D, by using the Cloud Logging API to create a project sink:

1. **Targeted Filtering:** It allows defining a specific filter (an advanced filter using the Cloud Logging query language) to isolate log entries related to BigQuery insert jobs on a designated table. This ensures no other table updates cause notifications, satisfying the question's requirement.
2. **Real-time Export:** The project sink can export filtered logs to Cloud Pub/Sub, a messaging service, in real-time.
3. **Real-time Notification:** The monitoring tool can subscribe to the Pub/Sub topic, allowing it to receive instant notifications whenever a matching log entry is generated.

This approach combines the filtering capability of Cloud Logging sinks with the real-time notification ability of Pub/Sub, providing an efficient and targeted solution.

Relevant Links for Further Research:

Cloud Logging Overview: <https://cloud.google.com/logging/docs/overview>

Cloud Logging Sinks: https://cloud.google.com/logging/docs/export/configure_export

Cloud Logging Advanced Filters: <https://cloud.google.com/logging/docs/view/advanced-queries>

Question: 26**CertyIQ**

You are working on a sensitive project involving private user data. You have set up a project on Google Cloud Platform to house your work internally. An external consultant is going to assist with coding a complex transformation in a Google Cloud Dataflow pipeline for your project. How should you maintain users' privacy?

- A. Grant the consultant the Viewer role on the project.
- B. Grant the consultant the Cloud Dataflow Developer role on the project.
- C. Create a service account and allow the consultant to log on with it.
- D. Create an anonymized sample of the data for the consultant to work with in a different project.

Answer: D**Explanation:**

The most secure approach to protect user privacy when working with an external consultant is option D, creating an anonymized sample dataset. Granting the consultant direct access to the production project (options A, B, C) presents significant privacy risks, even with restricted roles. Option A, Viewer role, while read-only, still exposes the consultant to potentially sensitive metadata and schema details. Option B, Cloud Dataflow Developer, gives write access to Dataflow pipelines and indirectly to data, which is unacceptable for sensitive data. Option C, a shared service account, blurs the audit trail and doesn't prevent the consultant from accessing production data. Anonymized data in a separate project (D) completely removes the privacy risk. The consultant can still develop and test the Dataflow transformation, as it is a scaled-down and de-identified version of the original data. This approach ensures compliance with data privacy regulations, prevents unintended data exposure, and allows the consultant to work effectively without compromising user privacy. This methodology adheres to the principle of least privilege, ensuring data exposure is minimized. <https://cloud.google.com/data-loss-prevention/docs/concepts-deid-strategies> and <https://cloud.google.com/docs/security/best-practices/data-privacy-best-practices> provide further guidance on data anonymization and privacy practices on Google Cloud.

Question: 27**CertyIQ**

You are building a model to predict whether or not it will rain on a given day. You have thousands of input features and want to see if you can improve training speed by removing some features while having a minimum effect on model accuracy. What can you do?

- A. Eliminate features that are highly correlated to the output labels.
- B. Combine highly co-dependent features into one representative feature.
- C. Instead of feeding in each feature individually, average their values in batches of 3.
- D. Remove the features that have null values for more than 50% of the training records.

Answer: B**Explanation:**

The most effective strategy to reduce training time while minimizing accuracy loss when dealing with numerous input features, some of which are highly correlated, is to combine those highly co-dependent features into a single, representative feature (option B). This process, often referred to as feature engineering or feature aggregation, reduces the dimensionality of the input data. High dimensionality can increase training complexity and time due to the curse of dimensionality, where models struggle to generalize from

sparsely populated high-dimensional spaces. Option A, eliminating features correlated with the output, is counterproductive as those features are likely crucial for accurate prediction. Averaging features in batches (option C) doesn't address the core issue of feature redundancy. Removing features based on missing values exceeding 50% (option D) can eliminate potentially valuable information if those missing values don't consistently bias the model. Option B is the optimal choice as it addresses redundancy while retaining information, thus leading to faster training without a significant loss in accuracy. Feature aggregation techniques include Principal Component Analysis (PCA), which creates new, uncorrelated features by combining existing ones, and simple averaging or weighted averaging of related features. These can substantially speed up training by reducing the number of parameters the model needs to learn, while keeping key information that allows high model performance.

Supporting Links:

Feature Selection: https://scikit-learn.org/stable/modules/feature_selection.html (Though not GCP specific, it covers key concepts applicable to any machine learning project)

Curse of Dimensionality: https://en.wikipedia.org/wiki/Curse_of_dimensionality

Feature Engineering: <https://developers.google.com/machine-learning/guides/feature-engineering> (Google's own ML guide)

Principal Component Analysis (PCA): <https://towardsdatascience.com/a-one-stop-shop-for-principal-component-analysis-50b11811d191> (Provides in-depth explanation of PCA, a common feature aggregation method.)

Question: 28

CertyIQ

Your company is performing data preprocessing for a learning algorithm in Google Cloud Dataflow. Numerous data logs are being generated during this step, and the team wants to analyze them. Due to the dynamic nature of the campaign, the data is growing exponentially every hour.

The data scientists have written the following code to read the data for a new key features in the logs.

```
BigQueryIO.Read
    .named("ReadLogData")
    .from("clouddataflow-readonly:samples.log_data")
```

You want to improve the performance of this data read. What should you do?

- A. Specify the TableReference object in the code.
- B. Use .fromQuery operation to read specific fields from the table.
- C. Use of both the Google BigQuery TableSchema and TableFieldSchema classes.
- D. Call a transform that returns TableRow objects, where each element in the PCollection represents a single row in the table.

Answer: B

Explanation:

Use .fromQuery operation to read specific fields from the table. Big Query IO. read. from() directly reads the whole table from Big Query. This function exports the whole table to temporary files in Google Cloud Storage, where it will later be read from. This requires almost no computation, as it only performs an export job, and later Dataflow reads from GCS (not from Big Query).

Big Query IO .read .fromQuery() executes a query and then reads the results received after the query execution. Therefore, this function is more time-consuming, given that it requires that a query is first executed

(which will incur in the corresponding economic and computational costs).

Question: 29

CertyIQ

Your company is streaming real-time sensor data from their factory floor into Bigtable and they have noticed extremely poor performance. How should the row key be redesigned to improve Bigtable performance on queries that populate real-time dashboards?

- A. Use a row key of the form <timestamp>.
- B. Use a row key of the form <sensorid>.
- C. Use a row key of the form <timestamp>#<sensorid>.
- D. Use a row key of the form >#<sensorid>#<timestamp>.

Answer: D

Explanation:

Here's a detailed justification for why option D is the most suitable row key design for improving Bigtable performance in this scenario:

Bigtable, a NoSQL wide-column store, is optimized for reads based on the lexicographical order of row keys. Using a timestamp as the sole row key (option A) leads to a "hotspotting" problem. Since timestamps are constantly increasing, all new writes would target a single server, creating a bottleneck and causing poor performance. Similarly, using only the sensor ID (option B) would concentrate writes for the same sensor on the same server. Option C, #, also doesn't fully mitigate hotspots since the timestamp is still the primary sorting key.

Option D, <sensorid>#<timestamp>, offers the best approach by distributing writes across multiple servers based on sensor IDs, preventing hotspotting. The sensor ID provides initial distribution and timestamp ensures a logical order and efficient retrieval of time series data. When retrieving sensor data for dashboards, the queries can leverage a prefix scan on sensor IDs, which is highly efficient. Using a timestamp within the row key also supports efficient querying for a specific time range for a given sensor. This design balances write distribution with query efficiency, therefore improving overall Bigtable performance. By properly designing the row key to consider the access patterns, you can significantly reduce latency and improve resource utilization.

Relevant Links:

Google Cloud Bigtable Schema Design: <https://cloud.google.com/bigtable/docs/schema-design>

Bigtable Key Design Best Practices: <https://cloud.google.com/blog/products/data-analytics/cloud-bigtable-key-visualizer-lets-you-optimize-performance>

Bigtable Performance Considerations: <https://cloud.google.com/bigtable/docs/performance>

Question: 30

CertyIQ

Your company's customer and order databases are often under heavy load. This makes performing analytics against them difficult without harming operations.

The databases are in a MySQL cluster, with nightly backups taken using mysqldump. You want to perform analytics with minimal impact on operations. What should you do?

- A. Add a node to the MySQL cluster and build an OLAP cube there.
- B. Use an ETL tool to load the data from MySQL into Google BigQuery.
- C. Connect an on-premises Apache Hadoop cluster to MySQL and perform ETL.

D. Mount the backups to Google Cloud SQL, and then process the data using Google Cloud Dataproc.

Answer: B

Explanation:

The optimal solution is to use an ETL tool to load data from MySQL into Google BigQuery (Option B). This approach addresses the core issue: performing analytics without impacting the operational MySQL database. BigQuery is a fully managed, serverless data warehouse designed for large-scale analytics. By moving data to BigQuery, you isolate analytical workloads from the transactional MySQL cluster, preventing performance degradation during periods of heavy load. ETL tools like Dataflow or Cloud Data Fusion can efficiently extract data from MySQL, transform it as needed, and load it into BigQuery. This process ensures that analytical queries run against a data warehouse optimized for such operations, rather than the operational database. Option A, adding a node to MySQL and building an OLAP cube there, would likely exacerbate the load issues as it still places analytical processing on the same infrastructure. Option C introduces unnecessary complexity by involving on-premises resources and a Hadoop cluster. Option D, mounting backups to Cloud SQL, adds latency with restoration and doesn't provide the analytical performance of BigQuery. Hence, B provides the best balance of scalability, performance, and operational safety, aligning with best practices for separating OLTP and OLAP workloads on cloud platforms.

Further Research:

BigQuery Overview: <https://cloud.google.com/bigquery/docs/introduction>

ETL on Google Cloud: <https://cloud.google.com/solutions/etl>

OLTP vs OLAP: <https://www.ibm.com/topics/olap-vs-oltp>

Question: 31

CertyIQ

You have Google Cloud Dataflow streaming pipeline running with a Google Cloud Pub/Sub subscription as the source. You need to make an update to the code that will make the new Cloud Dataflow pipeline incompatible with the current version. You do not want to lose any data when making this update. What should you do?

- A. Update the current pipeline and use the drain flag.
- B. Update the current pipeline and provide the transform mapping JSON object.
- C. Create a new pipeline that has the same Cloud Pub/Sub subscription and cancel the old pipeline.
- D. Create a new pipeline that has a new Cloud Pub/Sub subscription and cancel the old pipeline.

Answer: A

Explanation:

The correct answer is A: Update the current pipeline and use the drain flag. Here's why:

When updating a streaming Dataflow pipeline that introduces incompatibility, a simple update without proper handling can lead to data loss. The drain flag is crucial in this scenario. When a pipeline is drained, Dataflow stops accepting new data, but it continues processing all data already in-flight through the existing pipeline. This ensures that no messages from the Pub/Sub subscription are lost or discarded during the update process. It allows the current version to finish processing all the messages it has already consumed. After the drain operation completes, the pipeline can be safely updated with the new, incompatible code. Once the new version is deployed, it can start consuming from the existing Pub/Sub subscription without any data loss. Options C and D involve creating new pipelines and potentially cancelling the old one, which might cause some messages that are still in transit to be lost. Option B is incorrect because transform mapping is used during pipeline updates when there are schema changes between versions but is not applicable here for incompatible changes requiring draining. The drain mechanism is designed for graceful shutdowns,

guaranteeing data safety. This is a standard practice in streaming data processing to avoid data loss during upgrades.<https://cloud.google.com/dataflow/docs/guides/updating-a-pipeline><https://cloud.google.com/dataflow/docs/concepts/pipeline-lifecycle>

Question: 32

CertyIQ

Your company is running their first dynamic campaign, serving different offers by analyzing real-time data during the holiday season. The data scientists are collecting terabytes of data that rapidly grows every hour during their 30-day campaign. They are using Google Cloud Dataflow to preprocess the data and collect the feature (signals) data that is needed for the machine learning model in Google Cloud Bigtable. The team is observing suboptimal performance with reads and writes of their initial load of 10 TB of data. They want to improve this performance while minimizing cost. What should they do?

- A. Redefine the schema by evenly distributing reads and writes across the row space of the table.
- B. The performance issue should be resolved over time as the size of the Bigtable cluster is increased.
- C. Redesign the schema to use a single row key to identify values that need to be updated frequently in the cluster.
- D. Redesign the schema to use row keys based on numeric IDs that increase sequentially per user viewing the offers.

Answer: A

Explanation:

The correct answer is **A. Redefine the schema by evenly distributing reads and writes across the row space of the table.**

Here's why:

Bigtable performance heavily relies on efficient row key design. When reads and writes are concentrated in a small range of row keys (a "hotspot"), the load isn't distributed across the cluster's tablets, leading to performance bottlenecks. This scenario often arises from sequentially increasing IDs or timestamp-based row keys, which results in all new data landing on the same tablet until it splits. Option A directly addresses this issue by advocating for a schema that distributes reads and writes evenly. This can be achieved through strategies like:

Hashing: Applying a hash function to user IDs or other identifying data to spread writes across the row space.

Salting: Adding a prefix to row keys that distributes them over different tablet servers.

Option B is incorrect because merely increasing cluster size won't fix a poorly designed schema; it simply buys time until the bigger cluster also bottlenecks in the same way. Option C promotes using a single row key for frequent updates, which is the opposite of good Bigtable schema design; it will only exacerbate the hot-spotting issue. Option D is also incorrect because sequentially increasing user IDs will cause the same hot-spotting problem. A good row key design needs to consider both reads and writes, and distribute the data evenly across the entire cluster.

In summary, a well-designed Bigtable schema requires distributing data across the row space. Hashing or salting row keys is crucial for achieving optimal read and write performance.

Authoritative Links:

Google Cloud Bigtable Schema Design: <https://cloud.google.com/bigtable/docs/schema-design>

Bigtable Performance Considerations: <https://cloud.google.com/bigtable/docs/performance>

Key Visualizer in Bigtable: <https://cloud.google.com/bigtable/docs/key-visualizer>

Question: 33

Your software uses a simple JSON format for all messages. These messages are published to Google Cloud Pub/Sub, then processed with Google Cloud Dataflow to create a real-time dashboard for the CFO. During testing, you notice that some messages are missing in the dashboard. You check the logs, and all messages are being published to Cloud Pub/Sub successfully. What should you do next?

- A. Check the dashboard application to see if it is not displaying correctly.
- B. Run a fixed dataset through the Cloud Dataflow pipeline and analyze the output.
- C. Use Google Stackdriver Monitoring on Cloud Pub/Sub to find the missing messages.
- D. Switch Cloud Dataflow to pull messages from Cloud Pub/Sub instead of Cloud Pub/Sub pushing messages to Cloud Dataflow.

Answer: B

Explanation:

The correct answer is **B. Run a fixed dataset through the Cloud Dataflow pipeline and analyze the output.**

Here's the justification:

The problem states messages are successfully published to Pub/Sub, but some are missing in the final dashboard. This indicates a problem after Pub/Sub. Option A, while potentially a cause, doesn't directly address a pipeline issue. Option C is not ideal because it doesn't provide an immediate root cause; it's more for monitoring system health rather than pinpointing data transformation issues within Dataflow. Option D isn't a solution but a different way to integrate, which doesn't directly solve the issue at hand if messages are getting lost.

Running a fixed dataset (a known set of messages) through Dataflow allows you to isolate the problem. You can analyze the output at each stage of the pipeline to see exactly where and why messages are being dropped or transformed incorrectly. This approach facilitates debugging and provides actionable insights into Dataflow pipeline logic. It allows you to verify if the Dataflow transformations are properly parsing the JSON messages, and handling any edge cases. If a known message disappears, you've narrowed down your search to the Dataflow code. This method is far more efficient for diagnosing data processing errors than methods based on monitoring raw messages, as the issue appears to lie in the data pipeline logic. This approach leverages Dataflow's ability to ingest and process the known dataset, allowing a step-by-step analysis for data lineage and pinpointing transformation errors.

Further research:

Google Cloud Dataflow Debugging: <https://cloud.google.com/dataflow/docs/guides/debugging-pipelines>

This official Google documentation provides guidance on debugging Dataflow pipelines, including testing with fixed datasets.

Dataflow Basics: <https://cloud.google.com/dataflow/docs/concepts/dataflow-model> Provides a strong base on how Dataflow works.

Pub/Sub Monitoring: <https://cloud.google.com/pubsub/docs/monitoring> Details how to monitor Pub/Sub topics, while useful it is less relevant to the core issue here.

Question: 34

Flowlogistic Case Study -

Company Overview -

Flowlogistic is a leading logistics and supply chain provider. They help businesses throughout the world manage their resources and transport them to their final destination. The company has grown rapidly, expanding their

offerings to include rail, truck, aircraft, and oceanic shipping.

Company Background -

The company started as a regional trucking company, and then expanded into other logistics market. Because they have not updated their infrastructure, managing and tracking orders and shipments has become a bottleneck. To improve operations, Flowlogistic developed proprietary technology for tracking shipments in real time at the parcel level. However, they are unable to deploy it because their technology stack, based on Apache Kafka, cannot support the processing volume. In addition, Flowlogistic wants to further analyze their orders and shipments to determine how best to deploy their resources.

Solution Concept -

Flowlogistic wants to implement two concepts using the cloud:

- ⇒ Use their proprietary technology in a real-time inventory-tracking system that indicates the location of their loads
- ⇒ Perform analytics on all their orders and shipment logs, which contain both structured and unstructured data, to determine how best to deploy resources, which markets to expand info. They also want to use predictive analytics to learn earlier when a shipment will be delayed.

Existing Technical Environment -

Flowlogistic architecture resides in a single data center:

- ⇒ Databases
- 8 physical servers in 2 clusters
- SQL Server "" user data, inventory, static data
- 3 physical servers
- Cassandra "" metadata, tracking messages
- 10 Kafka servers "" tracking message aggregation and batch insert
- ⇒ Application servers "" customer front end, middleware for order/customs
- 60 virtual machines across 20 physical servers
- Tomcat "" Java services
- Nginx "" static content
- Batch servers
- ⇒ Storage appliances
- iSCSI for virtual machine (VM) hosts
- Fibre Channel storage area network (FC SAN) "" SQL server storage
- Network-attached storage (NAS) image storage, logs, backups
- ⇒ 10 Apache Hadoop /Spark servers
- Core Data Lake
- Data analysis workloads
- ⇒ 20 miscellaneous servers
- Jenkins, monitoring, bastion hosts,

Business Requirements -

Build a reliable and reproducible environment with scaled parity of production.

-
- ⇒ Aggregate data in a centralized Data Lake for analysis
- ⇒ Use historical data to perform predictive analytics on future shipments
- ⇒ Accurately track every shipment worldwide using proprietary technology
- ⇒ Improve business agility and speed of innovation through rapid provisioning of new resources
- ⇒ Analyze and optimize architecture for performance in the cloud
- ⇒ Migrate fully to the cloud if all other requirements are met

Technical Requirements -

- ⇒ Handle both streaming and batch data
- ⇒ Migrate existing Hadoop workloads
- ⇒ Ensure architecture is scalable and elastic to meet the changing demands of the company.
- ⇒ Use managed services whenever possible
- ⇒ Encrypt data flight and at rest
- ⇒ Connect a VPN between the production data center and cloud environment

SEO Statement -

We have grown so quickly that our inability to upgrade our infrastructure is really hampering further growth and efficiency. We are efficient at moving shipments around the world, but we are inefficient at moving data around. We need to organize our information so we can more easily understand where our customers are and what they are shipping.

CTO Statement -

IT has never been a priority for us, so as our data has grown, we have not invested enough in our technology. I have

a good staff to manage IT, but they are so busy managing our infrastructure that I cannot get them to do the things that really matter, such as organizing our data, building the analytics, and figuring out how to implement the CFO's tracking technology.

CFO Statement -

Part of our competitive advantage is that we penalize ourselves for late shipments and deliveries. Knowing where our shipments are at all times has a direct correlation to our bottom line and profitability. Additionally, I don't want to commit capital to building out a server environment.

Flowlogistic wants to use Google BigQuery as their primary analysis system, but they still have Apache Hadoop and Spark workloads that they cannot move to

BigQuery. Flowlogistic does not know how to store the data that is common to both workloads. What should they do?

- A. Store the common data in BigQuery as partitioned tables.
- B. Store the common data in BigQuery and expose authorized views.
- C. Store the common data encoded as Avro in Google Cloud Storage.
- D. Store the common data in the HDFS storage for a Google Cloud Dataproc cluster.

Answer: C

Explanation:

C. Store the common data encoded as Avro in Google Cloud Storage.

Flowlogistic wants to:

Use Google BigQuery for analytics.

Continue running some workloads on Apache Hadoop and Spark.

Avoid committing capital to on-premise servers.

Store common data that can be used by both BigQuery and Spark/Hadoop.

Google Cloud Storage (GCS) is a central, cost-effective cloud storage solution.

Avro is a splittable, compressed, and schema-based file format that is:

Supported by BigQuery (via federated queries or ingestion).

Supported by Apache Hadoop and Spark (through built-in readers).

Storing data in Avro format in GCS makes it accessible and efficient for both BigQuery and Spark/Hadoop, meeting the goal of shared data access without on-premise infrastructure.

Why the other options are not Correct:

A. Store in BigQuery as partitioned tables:

This helps only BigQuery and does not support Spark/Hadoop directly.

B. Expose authorized views in BigQuery:

Views help limit access within BigQuery but do not help Hadoop/Spark workloads access the data.

D. Store in HDFS on a Dataproc cluster:

Requires maintaining Dataproc cluster VMs (contradicts goal to avoid capital commitment to servers).

Less efficient than GCS for shared access and more costly over time.

Question: 35

Flowlogistic Case Study -

Company Overview -

Flowlogistic is a leading logistics and supply chain provider. They help businesses throughout the world manage their resources and transport them to their final destination. The company has grown rapidly, expanding their offerings to include rail, truck, aircraft, and oceanic shipping.

Company Background -

The company started as a regional trucking company, and then expanded into other logistics market. Because they have not updated their infrastructure, managing and tracking orders and shipments has become a bottleneck. To improve operations, Flowlogistic developed proprietary technology for tracking shipments in real time at the parcel level. However, they are unable to deploy it because their technology stack, based on Apache Kafka, cannot support the processing volume. In addition, Flowlogistic wants to further analyze their orders and shipments to determine how best to deploy their resources.

Solution Concept -

Flowlogistic wants to implement two concepts using the cloud:

- Use their proprietary technology in a real-time inventory-tracking system that indicates the location of their loads
- Perform analytics on all their orders and shipment logs, which contain both structured and unstructured data, to determine how best to deploy resources, which markets to expand into. They also want to use predictive analytics to learn earlier when a shipment will be delayed.

Existing Technical Environment -

Flowlogistic architecture resides in a single data center:

- Databases
 - 8 physical servers in 2 clusters
 - SQL Server "" user data, inventory, static data
 - 3 physical servers
 - Cassandra "" metadata, tracking messages
 - 10 Kafka servers "" tracking message aggregation and batch insert
- Application servers "" customer front end, middleware for order/customs
- 60 virtual machines across 20 physical servers
 - Tomcat "" Java services
 - Nginx "" static content
 - Batch servers
- Storage appliances
 - iSCSI for virtual machine (VM) hosts
 - Fibre Channel storage area network (FC SAN) "" SQL server storage
 - Network-attached storage (NAS) image storage, logs, backups
- 10 Apache Hadoop /Spark servers
 - Core Data Lake
 - Data analysis workloads
- 20 miscellaneous servers
 - Jenkins, monitoring, bastion hosts,

Business Requirements -

- Build a reliable and reproducible environment with scaled capacity of production.
- Aggregate data in a centralized Data Lake for analysis
- Use historical data to perform predictive analytics on future shipments
- Accurately track every shipment worldwide using proprietary technology
- Improve business agility and speed of innovation through rapid provisioning of new resources
- Analyze and optimize architecture for performance in the cloud
- Migrate fully to the cloud if all other requirements are met

Technical Requirements -

- Handle both streaming and batch data
- Migrate existing Hadoop workloads
- Ensure architecture is scalable and elastic to meet the changing demands of the company.
- Use managed services whenever possible
- Encrypt data in flight and at rest

⇒ Connect a VPN between the production data center and cloud environment

SEO Statement -

We have grown so quickly that our inability to upgrade our infrastructure is really hampering further growth and efficiency. We are efficient at moving shipments around the world, but we are inefficient at moving data around. We need to organize our information so we can more easily understand where our customers are and what they are shipping.

CTO Statement -

IT has never been a priority for us, so as our data has grown, we have not invested enough in our technology. I have a good staff to manage IT, but they are so busy managing our infrastructure that I cannot get them to do the things that really matter, such as organizing our data, building the analytics, and figuring out how to implement the CFO's tracking technology.

CFO Statement -

Part of our competitive advantage is that we penalize ourselves for late shipments and deliveries. Knowing where our shipments are at all times has a direct correlation to our bottom line and profitability. Additionally, I don't want to commit capital to building out a server environment.

Flowlogistic's management has determined that the current Apache Kafka servers cannot handle the data volume for their real-time inventory tracking system.

You need to build a new system on Google Cloud Platform (GCP) that will feed the proprietary tracking software. The system must be able to ingest data from a variety of global sources, process and query in real-time, and store the data reliably. Which combination of GCP products should you choose?

- A. Cloud Pub/Sub, Cloud Dataflow, and Cloud Storage
- B. Cloud Pub/Sub, Cloud Dataflow, and Local SSD
- C. Cloud Pub/Sub, Cloud SQL, and Cloud Storage
- D. Cloud Load Balancing, Cloud Dataflow, and Cloud Storage

Answer: A

Explanation:

The correct answer is **A. Cloud Pub/Sub, Cloud Dataflow, and Cloud Storage**. Here's why:

Flowlogistic needs a system that can handle real-time ingestion, processing, and storage of tracking data from various global sources. Let's break down how each chosen service addresses specific needs:

Cloud Pub/Sub: This is a fully managed, scalable messaging service that enables real-time data ingestion from global sources. It decouples the data producers (tracking devices) from the data consumers (processing pipelines). Pub/Sub is designed to handle high-throughput streaming data, which is perfect for the real-time tracking use case, replacing their inadequate Kafka setup. <https://cloud.google.com/pubsub/docs/overview>

Cloud Dataflow: This is a serverless data processing service capable of both stream and batch processing. It's ideal for real-time processing of the tracking data coming from Pub/Sub. Dataflow can perform transformations, aggregations, and enrichments on the data stream before storing it.

<https://cloud.google.com/dataflow/docs/concepts/what-is-dataflow>

Cloud Storage: This is a highly scalable, durable, and cost-effective object storage service. It serves as a reliable repository for storing the processed tracking data after Dataflow has completed its work. This will provide a centralized data lake for analytics and predictive modeling, addressing another business requirement mentioned in the case study. <https://cloud.google.com/storage/docs/overview>

Option B uses Local SSD which is not suitable for data lake storage. It is ephemeral and associated with a specific virtual machine. Option C proposes Cloud SQL, which is a relational database, less suitable for a time series or message type data. Cloud Load Balancing in option D is good for distributing web traffic, not for data ingestion. Therefore, **Cloud Pub/Sub, Cloud Dataflow, and Cloud Storage** is the optimal combination for meeting Flowlogistic's real-time tracking system requirements.

Question: 36

Flowlogistic Case Study -

Company Overview -

Flowlogistic is a leading logistics and supply chain provider. They help businesses throughout the world manage their resources and transport them to their final destination. The company has grown rapidly, expanding their offerings to include rail, truck, aircraft, and oceanic shipping.

Company Background -

The company started as a regional trucking company, and then expanded into other logistics market. Because they have not updated their infrastructure, managing and tracking orders and shipments has become a bottleneck. To improve operations, Flowlogistic developed proprietary technology for tracking shipments in real time at the parcel level. However, they are unable to deploy it because their technology stack, based on Apache Kafka, cannot support the processing volume. In addition, Flowlogistic wants to further analyze their orders and shipments to determine how best to deploy their resources.

Solution Concept -

Flowlogistic wants to implement two concepts using the cloud:

■ Use their proprietary technology in a real-time inventory-tracking system that indicates the location of their loads

▫ Perform analytics on all their orders and shipment logs, which contain both structured and unstructured data, to determine how best to deploy resources, which markets to expand info. They also want to use predictive analytics to learn earlier when a shipment will be delayed.

Existing Technical Environment -

Flowlogistic architecture resides in a single data center:

- Databases
- 8 physical servers in 2 clusters
 - SQL Server "" user data, inventory, static data
- 3 physical servers
 - Cassandra "" metadata, tracking messages
- 10 Kafka servers "" tracking message aggregation and batch insert
 - Application servers "" customer front end, middleware for order/customs
- 60 virtual machines across 20 physical servers
 - Tomcat "" Java services
 - Nginx "" static content
 - Batch servers
 - Storage appliances
 - iSCSI for virtual machine (VM) hosts
 - Fibre Channel storage area network (FC SAN) "" SQL server storage
 - Network-attached storage (NAS) image storage, logs, backups
 - 10 Apache Hadoop /Spark servers
 - Core Data Lake
 - Data analysis workloads
 - 20 miscellaneous servers
 - Jenkins, monitoring, bastion hosts,

Business Requirements -

- Build a reliable and reproducible environment with scaled parity of production.
- Aggregate data in a centralized Data Lake for analysis
- Use historical data to perform predictive analytics on future shipments
- Accurately track every shipment worldwide using proprietary technology
- Improve business agility and speed of innovation through rapid provisioning of new resources
- Analyze and optimize architecture for performance in the cloud
- Migrate fully to the cloud if all other requirements are met

Technical Requirements -

■ Handle both streaming and batch data

- Migrate existing Hadoop workloads
- Ensure architecture is scalable and elastic to meet the changing demands of the company.
- Use managed services whenever possible
- Encrypt data flight and at rest
- Connect a VPN between the production data center and cloud environment

SEO Statement -

We have grown so quickly that our inability to upgrade our infrastructure is really hampering further growth and efficiency. We are efficient at moving shipments around the world, but we are inefficient at moving data around. We need to organize our information so we can more easily understand where our customers are and what they are shipping.

CTO Statement -

IT has never been a priority for us, so as our data has grown, we have not invested enough in our technology. I have a good staff to manage IT, but they are so busy managing our infrastructure that I cannot get them to do the things that really matter, such as organizing our data, building the analytics, and figuring out how to implement the CFO's tracking technology.

CFO Statement -

Part of our competitive advantage is that we penalize ourselves for late shipments and deliveries. Knowing where our shipments are at all times has a direct correlation to our bottom line and profitability. Additionally, I don't want to commit capital to building out a server environment.

Flowlogistic's CEO wants to gain rapid insight into their customer base so his sales team can be better informed in the field. This team is not very technical, so they've purchased a visualization tool to simplify the creation of BigQuery reports. However, they've been overwhelmed by all the data in the table, and are spending a lot of money on queries trying to find the data they need. You want to solve their problem in the most cost-effective way. What should you do?

- A. Export the data into a Google Sheet for virtualization.
- B. Create an additional table with only the necessary columns.
- C. Create a view on the table to present to the virtualization tool.
- D. Create identity and access management (IAM) roles on the appropriate columns, so only they appear in a query.

Answer: C

Explanation:

A logical view can be created with only the required columns which is required for visualization. B is not the right option as you will create a table and make it static. What happens when the original data is updated. This new table will not have the latest data and hence view is the best possible option here.

Question: 37

CertyIQ

Flowlogistic Case Study -

Company Overview -

Flowlogistic is a leading logistics and supply chain provider. They help businesses throughout the world manage their resources and transport them to their final destination. The company has grown rapidly, expanding their offerings to include rail, truck, aircraft, and oceanic shipping.

Company Background -

The company started as a regional trucking company, and then expanded into other logistics market. Because they have not updated their infrastructure, managing and tracking orders and shipments has become a bottleneck. To improve operations, Flowlogistic developed proprietary technology for tracking shipments in real time at the parcel level. However, they are unable to deploy it because their technology stack, based on Apache Kafka, cannot support the processing volume. In addition, Flowlogistic wants to further analyze their orders and shipments to determine how best to deploy their resources.

Solution Concept -

Flowlogistic wants to implement two concepts using the cloud:

- Use their proprietary technology in a real-time inventory-tracking system that indicates the location of their loads
- Perform analytics on all their orders and shipment logs, which contain both structured and unstructured data, to determine how best to deploy resources, which markets to expand into. They also want to use predictive analytics to learn earlier when a shipment will be delayed.

Existing Technical Environment -

Flowlogistic architecture resides in a single data center:

- ⇒ Databases
- 8 physical servers in 2 clusters
- SQL Server "" user data, inventory, static data
- 3 physical servers
- Cassandra "" metadata, tracking messages
- 10 Kafka servers "" tracking message aggregation and batch insert
- ⇒ Application servers "" customer front end, middleware for order/customs
- 60 virtual machines across 20 physical servers
- Tomcat "" Java services
- Nginx "" static content
- Batch servers
- ⇒ Storage appliances
- iSCSI for virtual machine (VM) hosts
- Fibre Channel storage area network (FC SAN) "" SQL server storage
- Network-attached storage (NAS) image storage, logs, backups
- ⇒ 10 Apache Hadoop /Spark servers
- Core Data Lake
- Data analysis workloads
- ⇒ 20 miscellaneous servers
- Jenkins, monitoring, bastion hosts,

Business Requirements -

- ⇒ Build a reliable and reproducible environment with scaled parity of production.
- ⇒ Aggregate data in a centralized Data Lake for analysis
- ⇒ Use historical data to perform predictive analytics on future shipments
- ⇒ Accurately track every shipment worldwide using proprietary technology
- ⇒ Improve business agility and speed of innovation through rapid provisioning of new resources
- ⇒ Analyze and optimize architecture for performance in the cloud
- ⇒ Migrate fully to the cloud if all other requirements are met

Technical Requirements -

- ⇒ Handle both streaming and batch data
- ⇒ Migrate existing Hadoop workloads
- ⇒ Ensure architecture is scalable and elastic to meet the changing demands of the company.
- ⇒ Use managed services whenever possible
- ⇒ Encrypt data flight and at rest
- ⇒ Connect a VPN between the production data center and cloud environment

SEO Statement -

We have grown so quickly that our inability to upgrade our infrastructure is really hampering further growth and efficiency. We are efficient at moving shipments around the world, but we are inefficient at moving data around. We need to organize our information so we can more easily understand where our customers are and what they are shipping.

CTO Statement -

IT has never been a priority for us, so as our data has grown, we have not invested enough in our technology. I have a good staff to manage IT, but they are so busy managing our infrastructure that I cannot get them to do the things that really matter, such as organizing our data, building the analytics, and figuring out how to implement the CFO's tracking technology.

CFO Statement -

Part of our competitive advantage is that we penalize ourselves for late shipments and deliveries. Knowing where our shipments are at all times has a direct correlation to our bottom line and profitability. Additionally, I don't want to commit capital to building out a server environment.

Flowlogistic is rolling out their real-time inventory tracking system. The tracking devices will all send package-tracking messages, which will now go to a single

Google Cloud Pub/Sub topic instead of the Apache Kafka cluster. A subscriber application will then process the messages for real-time reporting and store them in

Google BigQuery for historical analysis. You want to ensure the package data can be analyzed over time.

Which approach should you take?

A. Attach the timestamp on each message in the Cloud Pub/Sub subscriber application as they are received.

B. Attach the timestamp and Package ID on the outbound message from each publisher device as they are sent to Cloud Pub/Sub.

- C. Use the NOW () function in BigQuery to record the event's time.
- D. Use the automatically generated timestamp from Cloud Pub/Sub to order the data.

Answer: B

Explanation:

Okay, let's break down why option B is the correct approach for capturing timestamps in this scenario, and why the others are less suitable.

Justification for Option B (Correct):

Accuracy and Precision: Option B advocates attaching the timestamp at the source (the tracking device) along with the Package ID. This ensures the most accurate reflection of when the event (package movement) actually occurred. This is vital for reliable historical analysis as timestamps added later may not reflect the actual time of the event, which can be due to delays in the system.

Consistent Time Source: Each tracking device has its own clock. If the timestamp is added at the publisher (tracking device) then differences in time among devices can be addressed at the source.

Data Integrity: Embedding the Package ID alongside the timestamp guarantees that each time record is uniquely identified and connected to the correct package. This is crucial for precise tracking and analysis.

Immutable Data: Once the timestamp is attached at the source and sent via pub/sub, the data is considered more immutable. This prevents alterations or inconsistencies during later stages of processing.

Flexibility: This method enables using timestamps for various purposes beyond just ordering, such as calculating transit times or identifying anomalies.

Why Other Options Are Not Ideal:

Option A (Timestamp in Subscriber Application): Adding the timestamp within the subscriber application introduces delays based on network latency, processing time, and the subscriber's clock. This creates inaccuracies and defeats the purpose of a reliable timeline. Also, the publisher event time cannot be properly captured by the subscriber application.

Option C (BigQuery NOW() function): Using BigQuery's NOW() function only captures the time the record is inserted into BigQuery, not when the event actually occurred, making it unsuitable for historical analysis and package transit time calculations.

Option D (Cloud Pub/Sub Timestamp): While Cloud Pub/Sub automatically generates timestamps when a message is received, these timestamps reflect when the message hit the Pub/Sub service and not when the tracking device generated the event. Thus it does not capture the required event timeline accurately. While these automatically generated timestamps can be used for ordering data received by pub/sub, the question specifically asks about tracking over time and not ordering messages within pub/sub.

In Summary:

The most reliable and accurate way to ensure that package data can be analyzed over time is to attach the timestamp, along with the package ID at the publisher source (tracking device). This approach ensures integrity, accuracy, and proper data management and facilitates various analytics operations. This approach adheres to the principle of capturing the most accurate representation of events as close to their source as possible.

Authoritative Links:

Cloud Pub/Sub Message Attributes: https://cloud.google.com/pubsub/docs/publisher#message_attributes - Learn how message attributes can be used to carry metadata like timestamps.

Google BigQuery Data Types: <https://cloud.google.com/bigquery/docs/reference/standard-sql/data-types> - Details on how BigQuery stores and handles different data types, including timestamps.

Best Practices for Time Series Data: Although not specifically from Google Cloud, understanding time-series data management is valuable, resources like "Designing Data-Intensive Applications" by Martin Kleppmann

Question: 38

MJTelco Case Study -

Company Overview -

MJTelco is a startup that plans to build networks in rapidly growing, underserved markets around the world. The company has patents for innovative optical communications hardware. Based on these patents, they can create many reliable, high-speed backbone links with inexpensive hardware.

Company Background -

Founded by experienced telecom executives, MJTelco uses technologies originally developed to overcome communications challenges in space. Fundamental to their operation, they need to create a distributed data infrastructure that drives real-time analysis and incorporates machine learning to continuously optimize their topologies. Because their hardware is inexpensive, they plan to overdeploy the network allowing them to account for the impact of dynamic regional politics on location availability and cost. Their management and operations teams are situated all around the globe creating many-to-many relationship between data consumers and provides in their system. After careful consideration, they decided public cloud is the perfect environment to support their needs.

Solution Concept -

MJTelco is running a successful proof-of-concept (PoC) project in its labs. They have two primary needs:

- Scale and harden their PoC to support significantly more data flows generated when they ramp to more than 50,000 installations.
- Refine their machine-learning cycles to verify and improve the dynamic models they use to control topology definition.

MJTelco will also use three separate operating environments " development/test, staging, and production " to meet the needs of running experiments, deploying new features, and serving production customers.

Business Requirements -

- Scale up their production environment with minimal cost, instantiating resources when and where needed in an unpredictable, distributed telecom user community.
- Ensure security of their proprietary data to protect their leading-edge machine learning and analysis.
- Provide reliable and timely access to data for analysis from distributed research workers
- Maintain isolated environments that support rapid iteration of their machine-learning models without affecting their customers.

Technical Requirements -

- Ensure secure and efficient transport and storage of telemetry data
- Rapidly scale instances to support between 10,000 and 100,000 data providers with multiple flows each.
- Allow analysis and presentation against data tables tracking up to 2 years of data storing approximately 100m records/day
- Support rapid iteration of monitoring infrastructure focused on awareness of data pipeline problems both in telemetry flows and in production learning cycles.

CEO Statement -

Our business model relies on our patents, analytics and dynamic machine learning. Our inexpensive hardware is organized to be highly reliable, which gives us cost advantages. We need to quickly stabilize our large distributed data pipelines to meet our reliability and capacity commitments.

CTO Statement -

Our public cloud services must operate as advertised. We need resources that scale and keep our data secure. We also need environments in which our data scientists can carefully study and quickly adapt our models. Because we rely on automation to process our data, we also need our development and test environments to work as we iterate.

CFO Statement -

The project is too large for us to maintain the hardware and software required for the data and analysis. Also, we cannot afford to staff an operations team to monitor so many data feeds, so we will rely on automation and infrastructure. Google Cloud's machine learning will allow our quantitative researchers to work on our high-value problems instead of problems with our data pipelines.

MJTelco's Google Cloud Dataflow pipeline is now ready to start receiving data from the 50,000 installations. You want to allow Cloud Dataflow to scale its compute power up as required. Which Cloud Dataflow pipeline

configuration setting should you update?

- A. The zone
- B. The number of workers
- C. The disk size per worker
- D. The maximum number of workers

Answer: D

Explanation:

The correct answer is **D. The maximum number of workers**.

Here's a detailed justification:

MJTelco needs its Cloud Dataflow pipeline to automatically scale its compute resources based on the incoming data volume from its 50,000 installations. They need the pipeline to be able to handle spikes in data flow without manual intervention. The crucial concept is autoscaling, and within Cloud Dataflow, the number of workers is the primary factor for horizontal scaling of resources.

Option A: The zone specifies the geographical location of resources. While important, it doesn't directly impact the dynamic scaling of compute resources based on data volume.

Option B: The number of workers refers to the initial number of workers requested when the pipeline starts. It doesn't enable automatic scaling.

Option C: The disk size per worker relates to storage per worker, which doesn't control the number of workers or compute resources for processing.

By setting a **maximum number of workers (D)**, Cloud Dataflow is authorized to automatically increase the number of worker instances as the data load increases, up to the configured limit. Conversely, when the data flow decreases, Dataflow can reduce the number of workers to optimize costs. This dynamic scaling aligns perfectly with MJTelco's need to handle fluctuating data volumes from a large number of providers, especially as they grow beyond 50,000 installations. This configuration lets Dataflow intelligently manage resource allocation, fulfilling their requirement for scalability and reliability while ensuring efficient resource use. It enables them to meet their reliability and capacity commitments and avoid manual intervention for resource scaling.

Here are authoritative links for further research:

Cloud Dataflow Autoscaling: <https://cloud.google.com/dataflow/docs/concepts/autoscaling>

Cloud Dataflow Pipeline Options: <https://cloud.google.com/dataflow/docs/reference/pipeline-options>

Autoscaling Concepts: <https://cloud.google.com/compute/docs/autoscaler/>

Question: 39

CertyIQ

MJTelco Case Study -

Company Overview -

MJTelco is a startup that plans to build networks in rapidly growing, underserved markets around the world. The company has patents for innovative optical communications hardware. Based on these patents, they can create many reliable, high-speed backbone links with inexpensive hardware.

Company Background -

Founded by experienced telecom executives, MJTelco uses technologies originally developed to overcome communications challenges in space. Fundamental to their operation, they need to create a distributed data infrastructure that drives real-time analysis and incorporates machine learning to continuously optimize their topologies. Because their hardware is inexpensive, they plan to overdeploy the network allowing them to account for the impact of dynamic regional politics on location availability and cost.

Their management and operations teams are situated all around the globe creating many-to-many relationship between data consumers and provides in their system. After careful consideration, they decided public cloud is the perfect environment to support their needs.

Solution Concept -

MJTelco is running a successful proof-of-concept (PoC) project in its labs. They have two primary needs:

- Scale and harden their PoC to support significantly more data flows generated when they ramp to more than 50,000 installations.
- Refine their machine-learning cycles to verify and improve the dynamic models they use to control topology definition.

MJTelco will also use three separate operating environments `` development/test, staging, and production `` to meet the needs of running experiments, deploying new features, and serving production customers.

Business Requirements -

- Scale up their production environment with minimal cost, instantiating resources when and where needed in an unpredictable, distributed telecom user community.
 - Ensure security of their proprietary data to protect their leading-edge machine learning and analysis.
 - Provide reliable and timely access to data for analysis from distributed research workers
- Maintain isolated environments that support rapid iteration of their machine-learning models without affecting their customers.

Technical Requirements -

- Ensure secure and efficient transport and storage of telemetry data
- Rapidly scale instances to support between 10,000 and 100,000 data providers with multiple flows each.
- Allow analysis and presentation against data tables tracking up to 2 years of data storing approximately 100m records/day
- Support rapid iteration of monitoring infrastructure focused on awareness of data pipeline problems both in telemetry flows and in production learning cycles.

CEO Statement -

Our business model relies on our patents, analytics and dynamic machine learning. Our inexpensive hardware is organized to be highly reliable, which gives us cost advantages. We need to quickly stabilize our large distributed data pipelines to meet our reliability and capacity commitments.

CTO Statement -

Our public cloud services must operate as advertised. We need resources that scale and keep our data secure. We also need environments in which our data scientists can carefully study and quickly adapt our models. Because we rely on automation to process our data, we also need our development and test environments to work as we iterate.

CFO Statement -

The project is too large for us to maintain the hardware and software required for the data and analysis. Also, we cannot afford to staff an operations team to monitor so many data feeds, so we will rely on automation and infrastructure. Google Cloud's machine learning will allow our quantitative researchers to work on our high-value problems instead of problems with our data pipelines.

You need to compose visualizations for operations teams with the following requirements:

- The report must include telemetry data from all 50,000 installations for the most recent 6 weeks (sampling once every minute).
- The report must not be more than 3 hours delayed from live data.
- The actionable report should only show suboptimal links.
- Most suboptimal links should be sorted to the top.
- Suboptimal links can be grouped and filtered by regional geography.
- User response time to load the report must be <5 seconds.

Which approach meets the requirements?

A. Load the data into Google Sheets, use formulas to calculate a metric, and use filters/sorting to show only suboptimal links in a table.

B. Load the data into Google BigQuery tables, write Google Apps Script that queries the data, calculates the metric, and shows only suboptimal rows in a table in Google Sheets.

C. Load the data into Google Cloud Datastore tables, write a Google App Engine Application that queries all rows, applies a function to derive the metric, and then renders results in a table using the Google charts and visualization API.

D. Load the data into Google BigQuery tables, write a Google Data Studio 360 report that connects to your data, calculates a metric, and then uses a filter expression to show only suboptimal rows in a table.

Answer: D

Explanation:

D. Load the data into Google BigQuery tables, write a Google Data Studio 360 report that connects to your data, calculates a metric, and then uses a filter expression to show only suboptimal rows in a table.

User response time must be <5 seconds when loading the report.

Let's evaluate the best approach for performance, scalability, and usability.

BigQuery is designed for fast, scalable, analytical queries over large datasets.

Looker Studio (formerly Data Studio 360) can natively connect to BigQuery, leveraging its compute engine.

Filter expressions and calculated fields in Data Studio can be pushed down to BigQuery as SQL queries, optimizing performance.

Resulting dashboards are interactive, and load times are minimal when optimized properly.

Meets both the performance (<5s) and functionality requirements.

Why the other options are not Correct:

A. Google Sheets with formulas:

Not scalable — performance degrades significantly with large datasets.

Calculations/formulas on large data sets will not meet the <5s load time.

Poor option for near real-time reporting or advanced metrics.

B. BigQuery + Google Apps Script + Google Sheets:

Apps Script is not performant for real-time data processing — it's single-threaded and has execution time limits.

Introduces lag between querying, processing, and rendering in Sheets.

Won't consistently meet <5s requirement.

C. Cloud Datastore + App Engine + Google Charts:

Requires custom app development, which adds latency.

Google Charts rely on client-side rendering, and querying + processing can take longer than 5s, especially on large data.

Overengineered compared to native analytics solutions.

Question: 40

CertyIQ

MJTelco Case Study -

Company Overview -

MJTelco is a startup that plans to build networks in rapidly growing, underserved markets around the world. The company has patents for innovative optical communications hardware. Based on these patents, they can create many reliable, high-speed backbone links with inexpensive hardware.

Company Background -

Founded by experienced telecom executives, MJTelco uses technologies originally developed to overcome communications challenges in space. Fundamental to their operation, they need to create a distributed data infrastructure that drives real-time analysis and incorporates machine learning to continuously optimize their topologies. Because their hardware is inexpensive, they plan to overdeploy the network allowing them to account for the impact of dynamic regional politics on location availability and cost.

Their management and operations teams are situated all around the globe creating many-to-many relationship between data consumers and provides in their system. After careful consideration, they decided public cloud is the perfect environment to support their needs.

Solution Concept -

MJTelco is running a successful proof-of-concept (PoC) project in its labs. They have two primary needs:

- Scale and harden their PoC to support significantly more data flows generated when they ramp to more than 50,000 installations.
- Refine their machine-learning cycles to verify and improve the dynamic models they use to control topology definition.

MJTelco will also use three separate operating environments " development/test, staging, and production " to meet the needs of running experiments, deploying new features, and serving production customers.

Business Requirements -

- Scale up their production environment with minimal cost, instantiating resources when and where needed in an unpredictable, distributed telecom user community.
- Ensure security of their proprietary data to protect their leading-edge machine learning and analysis.

Provide reliable and timely access to data for analysis from distributed research workers

- Maintain isolated environments that support rapid iteration of their machine-learning models without affecting their customers.

Technical Requirements -

- Ensure secure and efficient transport and storage of telemetry data
- Rapidly scale instances to support between 10,000 and 100,000 data providers with multiple flows each.
- Allow analysis and presentation against data tables tracking up to 2 years of data storing approximately 100m records/day
- Support rapid iteration of monitoring infrastructure focused on awareness of data pipeline problems both in telemetry flows and in production learning cycles.

CEO Statement -

Our business model relies on our patents, analytics and dynamic machine learning. Our inexpensive hardware is organized to be highly reliable, which gives us cost advantages. We need to quickly stabilize our large distributed data pipelines to meet our reliability and capacity commitments.

CTO Statement -

Our public cloud services must operate as advertised. We need resources that scale and keep our data secure. We also need environments in which our data scientists can carefully study and quickly adapt our models. Because we rely on automation to process our data, we also need our development and test environments to work as we iterate.

CFO Statement -

The project is too large for us to maintain the hardware and software required for the data and analysis. Also, we cannot afford to staff an operations team to monitor so many data feeds, so we will rely on automation and infrastructure. Google Cloud's machine learning will allow our quantitative researchers to work on our high-value problems instead of problems with our data pipelines.

You create a new report for your large team in Google Data Studio 360. The report uses Google BigQuery as its data source. It is company policy to ensure employees can view only the data associated with their region, so you create and populate a table for each region. You need to enforce the regional access policy to the data.

Which two actions should you take? (Choose two.)

- A. Ensure all the tables are included in global dataset.
- B. Ensure each table is included in a dataset for a region.
- C. Adjust the settings for each table to allow a related region-based security group view access.
- D. Adjust the settings for each view to allow a related region-based security group view access.
- E. Adjust the settings for each dataset to allow a related region-based security group view access.

Answer: BE

Explanation:

Even if now BQ offers table level access control, since the number of tables can be expected to be high, controlling access at the dataset level would ease operations. That is why I would still go for E instead of C.

Question: 41

CertyIQ

MJTelco Case Study -

Company Overview -

MJTelco is a startup that plans to build networks in rapidly growing, underserved markets around the world. The company has patents for innovative optical communications hardware. Based on these patents, they can create many reliable, high-speed backbone links with inexpensive hardware.

Company Background -

Founded by experienced telecom executives, MJTelco uses technologies originally developed to overcome communications challenges in space. Fundamental to their operation, they need to create a distributed data infrastructure that drives real-time analysis and incorporates machine learning to continuously optimize their topologies. Because their hardware is inexpensive, they plan to overdeploy the network allowing them to account for the impact of dynamic regional politics on location availability and cost. Their management and operations teams are situated all around the globe creating many-to-many relationship between data consumers and provides in their system. After careful consideration, they decided public cloud is the perfect environment to support their needs.

Solution Concept -

MJTelco is running a successful proof-of-concept (PoC) project in its labs. They have two primary needs:

- ⇒ Scale and harden their PoC to support significantly more data flows generated when they ramp to more than 50,000 installations.
- ⇒ Refine their machine-learning cycles to verify and improve the dynamic models they use to control topology definition.

MJTelco will also use three separate operating environments `` development/test, staging, and production `` to meet the needs of running experiments, deploying new features, and serving production customers.

Business Requirements -

- ⇒ Scale up their production environment with minimal cost, instantiating resources when and where needed in an unpredictable, distributed telecom user community.
- ⇒ Ensure security of their proprietary data to protect their leading-edge machine learning and analysis.
- ⇒ Provide reliable and timely access to data for analysis from distributed research workers
- ⇒ Maintain isolated environments that support rapid iteration of their machine-learning models without affecting their customers.

Technical Requirements -

■ Ensure secure and efficient transport and storage of telemetry data

- ⇒ Rapidly scale instances to support between 10,000 and 100,000 data providers with multiple flows each.
- ⇒ Allow analysis and presentation against data tables tracking up to 2 years of data storing approximately 100m records/day
- ⇒ Support rapid iteration of monitoring infrastructure focused on awareness of data pipeline problems both in telemetry flows and in production learning cycles.

CEO Statement -

Our business model relies on our patents, analytics and dynamic machine learning. Our inexpensive hardware is organized to be highly reliable, which gives us cost advantages. We need to quickly stabilize our large distributed data pipelines to meet our reliability and capacity commitments.

CTO Statement -

Our public cloud services must operate as advertised. We need resources that scale and keep our data secure. We also need environments in which our data scientists can carefully study and quickly adapt our models. Because we rely on automation to process our data, we also need our development and test environments to work as we iterate.

CFO Statement -

The project is too large for us to maintain the hardware and software required for the data and analysis. Also, we cannot afford to staff an operations team to monitor so many data feeds, so we will rely on automation and infrastructure. Google Cloud's machine learning will allow our quantitative researchers to work on our high-value problems instead of problems with our data pipelines.

MJTelco needs you to create a schema in Google Bigtable that will allow for the historical analysis of the last 2 years of records. Each record that comes in is sent every 15 minutes, and contains a unique identifier of the device and a data record. The most common query is for all the data for a given device for a given day.

Which schema should you use?

- A. Rowkey: date#device_id Column data: data_point
- B. Rowkey: date Column data: device_id, data_point
- C. Rowkey: device_id Column data: date, data_point
- D. Rowkey: data_point Column data: device_id, date
- E. Rowkey: date#data_point Column data: device_id

Answer: A

Explanation:

A. Rowkey: date#device_id Column data: data_point

Rowkey combines date and device_id.

Query for "device + date" becomes a direct lookup → very efficient.

Question: 42

CertyIQ

Your company has recently grown rapidly and now ingesting data at a significantly higher rate than it was previously. You manage the daily batch MapReduce analytics jobs in Apache Hadoop. However, the recent increase in data has meant the batch jobs are falling behind. You were asked to recommend ways the development team could increase the responsiveness of the analytics without increasing costs. What should you recommend they do?

- A. Rewrite the job in Pig.
- B. Rewrite the job in Apache Spark.
- C. Increase the size of the Hadoop cluster.
- D. Decrease the size of the Hadoop cluster but also rewrite the job in Hive.

Answer: B

Explanation:

Rewriting the MapReduce job in Apache Spark (Option B) is the most cost-effective solution for improving responsiveness. Here's why:

Spark's in-memory processing capabilities offer significantly faster performance compared to Hadoop's disk-based MapReduce. Spark can store intermediate data in memory, reducing the need for costly disk I/O operations that slow down Hadoop jobs. This leads to quicker processing times for the same amount of data.

While increasing the Hadoop cluster size (Option C) might seem like a direct solution, it involves higher infrastructure costs (more nodes, storage, and network resources). Given the constraint of not increasing costs, this is not the best approach.

Rewriting the job in Pig (Option A) or Hive (Option D) might simplify development, but they ultimately translate to MapReduce jobs under the hood. These high-level languages don't inherently improve performance beyond

what can be achieved with optimized MapReduce code. Furthermore, rewriting in Hive and decreasing the cluster size is counterintuitive if the goal is to increase responsiveness. Reducing the cluster size while simply changing the query language exacerbates the problem.

Spark's ability to handle data transformations efficiently, especially in a distributed environment, makes it ideal for large datasets. It also supports various data formats and integrations with other data processing tools. The performance gain from using Spark can outweigh the development effort required to rewrite the existing MapReduce job, while keeping costs constant. Spark's DataFrame API, in particular, provides an abstraction layer that enables optimization for processing, further improving overall speed.

[Apache Spark Documentation](#)[Understanding Spark Performance](#)

Question: 43

CertyIQ

You work for a large fast food restaurant chain with over 400,000 employees. You store employee information in Google BigQuery in a Users table consisting of a FirstName field and a LastName field. A member of IT is building an application and asks you to modify the schema and data in BigQuery so the application can query a FullName field consisting of the value of the FirstName field concatenated with a space, followed by the value of the LastName field for each employee. How can you make that data available while minimizing cost?

- A. Create a view in BigQuery that concatenates the FirstName and LastName field values to produce the FullName.
- B. Add a new column called FullName to the Users table. Run an UPDATE statement that updates the FullName column for each user with the concatenation of the FirstName and LastName values.
- C. Create a Google Cloud Dataflow job that queries BigQuery for the entire Users table, concatenates the FirstName value and LastName value for each user, and loads the proper values for FirstName, LastName, and FullName into a new table in BigQuery.
- D. Use BigQuery to export the data for the table to a CSV file. Create a Google Cloud Dataproc job to process the CSV file and output a new CSV file containing the proper values for FirstName, LastName and FullName. Run a BigQuery load job to load the new CSV file into BigQuery.

Answer: A

Explanation:

The best approach is to create a BigQuery view that concatenates the FirstName and LastName fields. This minimizes cost because a view doesn't store any data itself. It's a virtual table defined by a query. When the application queries the view, the concatenation happens on-the-fly, using BigQuery's compute resources only when the query is executed.

Option B, adding a new column and updating the table, incurs storage costs for the new column and processing costs for the UPDATE statement. While it might seem efficient in terms of query performance, it adds unnecessary storage and modifies the existing table structure.

Option C, using Dataflow, involves significant overhead. It requires setting up and running a Dataflow pipeline, which consumes more resources and thus costs more than a simple view. It also creates a new table, increasing storage costs. Dataflow is better suited for complex data transformations and real-time processing, not a simple concatenation.

Option D, exporting to CSV, using Dataproc, and reloading, is the most expensive and complex option. It involves exporting the data, setting up a Dataproc cluster, writing and executing a Spark job (or similar) for processing, and then reloading the data into BigQuery. This incurs costs for storage, Dataproc cluster resources, and BigQuery load jobs. It's also unnecessary given the simplicity of the required transformation.

Views provide a cost-effective and efficient way to present transformed data without storing redundant information. In this scenario, where a simple concatenation is needed, a BigQuery view is the ideal solution.

Further Reading:

BigQuery Views: <https://cloud.google.com/bigquery/docs/views>

BigQuery Pricing: <https://cloud.google.com/bigquery/pricing>

Question: 44

CertyIQ

You are deploying a new storage system for your mobile application, which is a media streaming service. You decide the best fit is Google Cloud Datastore. You have entities with multiple properties, some of which can take on multiple values. For example, in the entity 'Movie' the property 'actors' and the property 'tags' have multiple values but the property 'date released' does not. A typical query would ask for all movies with actor=<actorname> ordered by date_released or all movies with tag=Comedy ordered by date_released. How should you avoid a combinatorial explosion in the number of indexes?

A. Manually configure the index in your index config as follows:

```
Indexes:
-kind: Movie
  Properties:
    -name: actors
    name: date_released
-kind: Movie
  Properties:
    -name: tags
    name: date_released
```

B. Manually configure the index in your index config as follows:

```
Indexes:
-kind: Movie
  Properties:
    -name: actors
    -name: tags
    -name: date_published
```

C. Set the following in your entity options: exclude_from_indexes = 'actors, tags'

D. Set the following in your entity options: exclude_from_indexes = 'date_published'

Answer: A

Explanation:

From Google cloud documentation

The rows of an index table are sorted first by ancestor and then by property values, in the order specified in the index definition. The perfect index for a query, which allows the query to be executed most efficiently, is defined on the following properties, in order:

Properties used in equality filters

Property used in an inequality filter (of which there can be no more than one)

Properties used in sort orders

Properties used in projections (that are not already included in sort orders)

Question: 45

CertyIQ

You work for a manufacturing plant that batches application log files together into a single log file once a day at 2:00 AM. You have written a Google Cloud Dataflow job to process that log file. You need to make sure the log file is processed once per day as inexpensively as possible. What should you do?

- A. Change the processing job to use Google Cloud Dataproc instead.
- B. Manually start the Cloud Dataflow job each morning when you get into the office.
- C. Create a cron job with Google App Engine Cron Service to run the Cloud Dataflow job.
- D. Configure the Cloud Dataflow job as a streaming job so that it processes the log data immediately.

Answer: C

Explanation:

The correct answer is **C. Create a cron job with Google App Engine Cron Service to run the Cloud Dataflow job**. Here's why:

The problem requires processing a daily batch log file at a specific time (2:00 AM) in the most cost-effective way. Option A, using Google Cloud Dataproc, is less suitable for this scenario. Dataproc is designed for large-scale data processing using Hadoop and Spark, adding unnecessary complexity and cost for a simple daily batch job. Option B, manually starting the job, is inefficient and prone to human error, making it unreliable. Option D, configuring the job as a streaming job, is inappropriate as the data arrives in a batch once a day, not as a continuous stream.

Google App Engine Cron Service, as suggested in option C, perfectly addresses the need. It allows scheduling tasks to run automatically at specific times. You can configure a cron job to trigger your Cloud Dataflow job every day at 2:00 AM, ensuring timely and automatic execution. This method leverages a fully managed serverless service, reducing operational overhead and cost compared to maintaining a dedicated compute instance. Moreover, cron jobs are designed for such scheduled executions, making them a perfect fit for this requirement. This approach ensures cost-effectiveness as you only pay for the cron service and Dataflow execution during its scheduled run, without having a compute instance running continuously. This adheres to the principle of least cost while ensuring timely job execution.

For further information on Google App Engine Cron Service, please refer to:

<https://cloud.google.com/appengine/docs/standard/python/config/cron> and

<https://cloud.google.com/scheduler/docs> and for Cloud Dataflow: <https://cloud.google.com/dataflow/docs/>

Question: 46

CertyIQ

You work for an economic consulting firm that helps companies identify economic trends as they happen. As part of your analysis, you use Google BigQuery to correlate customer data with the average prices of the 100 most common goods sold, including bread, gasoline, milk, and others. The average prices of these goods are updated every 30 minutes. You want to make sure this data stays up to date so you can combine it with other data in BigQuery as cheaply as possible. What should you do?

- A. Load the data every 30 minutes into a new partitioned table in BigQuery.
- B. Store and update the data in a regional Google Cloud Storage bucket and create a federated data source in BigQuery
- C. Store the data in Google Cloud Datastore. Use Google Cloud Dataflow to query BigQuery and combine the data programmatically with the data stored in Cloud Datastore
- D. Store the data in a file in a regional Google Cloud Storage bucket. Use Cloud Dataflow to query BigQuery and combine the data programmatically with the data stored in Google Cloud Storage.

Answer: B

Explanation:

The correct answer is **B. Store and update the data in a regional Google Cloud Storage bucket and create a federated data source in BigQuery.**

Here's why:

Cost-Effectiveness: The prompt explicitly states the need to combine data in BigQuery as cheaply as possible. Storing the frequently updated price data in Google Cloud Storage (GCS) and then querying it as an external table using BigQuery Federated Queries is generally more cost-effective than loading the data into BigQuery itself every 30 minutes. BigQuery charges for storage and queries, and frequent loading incurs both costs.

Near Real-Time Access: Federated queries allow you to query data directly from GCS without loading it into BigQuery. This allows for near real-time access to the price data, which is crucial for identifying economic trends "as they happen."

Simplicity: This approach avoids the complexity of data processing pipelines involving Cloud Dataflow (options C and D). While Dataflow is powerful, it's overkill for simply combining data from GCS with BigQuery. It also reduces operational overhead.

Partitioning in BigQuery (Option A): While partitioning could help query performance if the data was already in BigQuery, loading data every 30 minutes into a new partitioned table is inefficient and would lead to a proliferation of tables and significant storage and query costs. Furthermore, partitioning will not be triggered by data changes.

Cloud Datastore (Option C): Cloud Datastore is a NoSQL database. While it can store frequently updated data, querying Datastore with BigQuery using Dataflow adds complexity and cost compared to a federated query with GCS. Datastore is also not designed for analytical queries like those you'd typically perform in BigQuery for economic trend analysis.

Cloud Dataflow with GCS (Option D): Similar to option C, using Dataflow to combine GCS data with BigQuery adds unnecessary complexity and cost. Dataflow is suitable for complex data transformations, but not for directly querying data stored in GCS and combining it.

In Summary: Option B offers a cost-effective, near real-time, and simple solution that meets the stated requirements. It leverages the strengths of both GCS and BigQuery without introducing unnecessary complexity.

Authoritative Links:

BigQuery Federated Queries: <https://cloud.google.com/bigquery/docs/external-data-cloud-storage>

BigQuery Pricing: <https://cloud.google.com/bigquery/pricing>

Google Cloud Storage Pricing: <https://cloud.google.com/storage/pricing>

Question: 47

CertyIQ

You are designing the database schema for a machine learning-based food ordering service that will predict what users want to eat. Here is some of the information you need to store:

- The user profile: What the user likes and doesn't like to eat
- The user account information: Name, address, preferred meal times
- The order information: When orders are made, from where, to whom

The database will be used to store all the transactional data of the product. You want to optimize the data schema. Which Google Cloud Platform product should you use?

- A. BigQuery
- B. Cloud SQL
- C. Cloud Bigtable
- D. Cloud Datastore

Answer: A

Explanation:

The most appropriate Google Cloud Platform product for this scenario is BigQuery (A). The question describes a machine learning-based food ordering service that requires analysis of user profiles, account information, and order details. This inherently suggests a need for analytical capabilities and the ability to perform complex queries across large datasets. BigQuery is a fully-managed, serverless data warehouse designed for precisely these purposes. It excels at handling massive datasets, offering SQL-based querying, and enabling integration with machine learning services for model training and prediction. Unlike Cloud SQL (B), a relational database service, BigQuery isn't designed for transactional data, but rather for analytical workloads. Cloud Bigtable (C), a NoSQL wide-column store, and Cloud Datastore (D), a NoSQL document database, are better suited for high-throughput, low-latency operations rather than complex analytical processing. While these NoSQL options could store the data, they would be less efficient for analytical tasks needed to predict user preferences. BigQuery facilitates efficient data warehousing and allows for optimized analysis which is essential for the machine learning model. Furthermore, BigQuery's serverless architecture simplifies infrastructure management, allowing the focus to remain on developing and training the prediction model.

Authoritative Links:

BigQuery Documentation: <https://cloud.google.com/bigquery/docs>

Choosing a storage option: <https://cloud.google.com/learn/what-cloud-storage-option>

Question: 48

CertyIQ

Your company is loading comma-separated values (CSV) files into Google BigQuery. The data is fully imported successfully; however, the imported data is not matching byte-to-byte to the source file. What is the most likely cause of this problem?

- A. The CSV data loaded in BigQuery is not flagged as CSV.
- B. The CSV data has invalid rows that were skipped on import.
- C. The CSV data loaded in BigQuery is not using BigQuery's default encoding.
- D. The CSV data has not gone through an ETL phase before loading into BigQuery.

Answer: C

Explanation:

The correct answer is C, "The CSV data loaded in BigQuery is not using BigQuery's default encoding." Here's why:

BigQuery, like many data processing systems, defaults to UTF-8 encoding for text data. CSV files, however, can use various encodings (e.g., ASCII, Latin-1, UTF-16, etc.). If the source CSV file uses an encoding other than UTF-8, and this is not specified during the BigQuery load process, BigQuery will interpret the data incorrectly. This misinterpretation leads to byte-level differences compared to the original file, even if the data visually appears to load without error. The issue isn't about the data being identified as CSV (option A) as BigQuery knows this by its loading process. Skipped rows (option B), while a valid concern, don't typically change the byte-to-byte representation of successfully imported rows. Similarly, lack of ETL (option D) may affect data transformation but does not alter fundamental byte representation if the loading process works correctly. Therefore, the encoding mismatch is the prime suspect when byte-level discrepancies exist despite successful loading. Correctly specifying the source encoding using the encoding option during a load job is crucial for ensuring data integrity and accurate byte representation in BigQuery. This ensures that character representations in the CSV files are correctly translated within BigQuery.

Further information on how to specify encoding can be found in the official Google Cloud documentation:

[Loading data from Cloud Storage](#)

[BigQuery data types](#) for how encoding impacts data representation.

Question: 49

CertyIQ

Your company produces 20,000 files every hour. Each data file is formatted as a comma separated values (CSV) file that is less than 4 KB. All files must be ingested on Google Cloud Platform before they can be processed. Your company site has a 200 ms latency to Google Cloud, and your Internet connection bandwidth is limited as 50 Mbps. You currently deploy a secure FTP (SFTP) server on a virtual machine in Google Compute Engine as the data ingestion point. A local SFTP client runs on a dedicated machine to transmit the CSV files as is. The goal is to make reports with data from the previous day available to the executives by 10:00 a.m. each day. This design is barely able to keep up with the current volume, even though the bandwidth utilization is rather low. You are told that due to seasonality, your company expects the number of files to double for the next three months. Which two actions should you take? (Choose two.)

- A. Introduce data compression for each file to increase the rate file of file transfer.
- B. Contact your internet service provider (ISP) to increase your maximum bandwidth to at least 100 Mbps.
- C. Redesign the data ingestion process to use gsutil tool to send the CSV files to a storage bucket in parallel.
- D. Assemble 1,000 files into a tape archive (TAR) file. Transmit the TAR files instead, and disassemble the CSV files in the cloud upon receiving them.
- E. Create an S3-compatible storage endpoint in your network, and use Google Cloud Storage Transfer Service to transfer on-premises data to the designated storage bucket.

Answer: CD

Explanation:

The correct actions are **C and D**.

Here's why:

C. Redesign the data ingestion process to use gsutil tool to send the CSV files to a storage bucket in parallel.

Problem: The current SFTP approach is likely bottlenecked by the latency and sequential file transfers, even with low bandwidth utilization. SFTP inherently performs one-to-one transfers, leading to inefficiency with many small files.

Solution: gsutil is a command-line tool that leverages Google Cloud Storage's (GCS) capabilities for parallel uploads. This allows for multiple files to be transferred concurrently, significantly reducing the overall transfer time. Parallel uploads minimize the impact of network latency by keeping the connection consistently utilized.

Efficiency: This approach aligns with best practices for ingesting large amounts of data into GCS.

Documentation: <https://cloud.google.com/storage/docs/gsutil>

D. Assemble 1,000 files into a tape archive (TAR) file. Transmit the TAR files instead, and disassemble the CSV files in the cloud upon receiving them.

Problem: 20,000 small files/hour creates significant overhead due to the initiation of individual network connections and request handshakes.

Solution: By combining files into a smaller number of larger TAR files, you reduce the number of individual network transfers, diminishing overhead and optimizing throughput. The slight overhead of packing and unpacking is much lower than the network transfer overhead with large number of files.

Efficiency: Moving larger files reduces overheads related to network requests, enhancing efficiency during transfer.

Optimization: This consolidates numerous small file operations into few large ones.

Context: This practice is very common when dealing with transferring large number of small files across a network, be it on-prem or cloud.

Why other options are not correct:

A. Introduce data compression: While compression can reduce the data volume, the files are already small (4KB). The gain will likely be minimal. The problem is more with the sheer number of files.

B. Increase bandwidth: While helpful, the main issue is the latency and the inefficient handling of multiple, small files through SFTP. Increased bandwidth alone won't solve the efficiency problems of handling a large volume of small files, and is not the most efficient way to solve the problem.

E. S3-compatible storage and transfer service: This solution introduces a new piece of infrastructure, adds complexity, and doesn't solve the core issue of sending many small files efficiently. Using gsutil directly to GCS is a better approach.

In summary, parallel uploads (C) and file consolidation (D) are the most effective actions for handling the increased file volume and meeting the reporting deadline given network latency and limited bandwidth.

Question: 50

CertyIQ

You are choosing a NoSQL database to handle telemetry data submitted from millions of Internet-of-Things (IoT) devices. The volume of data is growing at 100 TB per year, and each data entry has about 100 attributes. The data processing pipeline does not require atomicity, consistency, isolation, and durability (ACID).

However, high availability and low latency are required.

You need to analyze the data by querying against individual fields. Which three databases meet your requirements? (Choose three.)

- A. Redis
- B. HBase
- C. MySQL
- D. MongoDB
- E. Cassandra

Answer: BDE**Explanation:**

The problem requires a NoSQL database for high-volume IoT telemetry data, prioritizing high availability and low latency while sacrificing ACID properties. The need to query against individual fields also dictates a specific type of NoSQL solution. Let's evaluate each option:

Redis (A): While Redis is a fast, in-memory data store, it primarily functions as a key-value or data structure server. It lacks the necessary features for querying 100 attributes per entry in an efficient manner, and is not designed for storing and retrieving data at the scale of 100TB per year. Thus, Redis is not a good choice.

HBase (B): HBase is a column-oriented, NoSQL database designed for handling large datasets, making it suitable for the scale requirements. It offers high availability and is optimized for low latency reads and writes. It's particularly well-suited for querying based on fields/columns, aligning with the requirements.

[<https://hbase.apache.org/>]

MySQL (C): MySQL is a relational database (SQL) that enforces ACID properties, which the problem states are not required. It might also struggle with the sheer volume of data and low latency writes demanded by the IoT data stream. Relational databases are not designed for such high scale as quickly as NoSQL databases.

MongoDB (D): MongoDB is a document-oriented database. It is capable of handling large data volumes, and its flexible schema makes it suitable for handling the 100 attributes per entry. Furthermore, MongoDB can provide high availability and low latency reads and writes, meeting the requirements.

[<https://www.mongodb.com/>]

Cassandra (E): Cassandra is another column-oriented NoSQL database that excels at handling large-scale data with high availability and low latency. It is built for write-heavy workloads, like telemetry data, and can efficiently query data using the fields defined within the columns, fitting the use case.

[<https://cassandra.apache.org/>]

HDFS with Hive (F): HDFS is a distributed file system designed for batch processing and large data storage, not real-time data access and low-latency reads. Hive is a query engine that operates on top of HDFS, further adding to the latency, and it's not well-suited for the low latency requirement of this use case.

Therefore, the three databases that best meet the requirements are **HBase (B), MongoDB (D), and Cassandra (E)**. They are NoSQL solutions that support high availability, low latency, large volumes of data, and efficient querying based on fields.

Question: 51

You are training a spam classifier. You notice that you are overfitting the training data. Which three actions can you take to resolve this problem? (Choose three.)

- A. Get more training examples
- B. Reduce the number of training examples
- C. Use a smaller set of features
- D. Use a larger set of features
- E. Increase the regularization parameters
- F. Decrease the regularization parameters

Answer: ACE**Explanation:**

Overfitting occurs when a model learns the training data too well, capturing noise and specific patterns that

don't generalize to unseen data. This results in high accuracy on training data but poor performance on new data. To combat overfitting, we need to make the model less sensitive to the nuances of the training set and more focused on underlying, generalizable patterns.

Option A, "**Get more training examples**," helps by providing the model with a wider variety of data points. This exposes the model to more diverse scenarios, making it less likely to memorize the specifics of a small training dataset and improving generalization. This aligns with the core principles of statistical learning, where more data typically leads to better model generalization.

Option B, "**Reduce the number of training examples**," would worsen overfitting, therefore it's incorrect. Reducing training data makes it more likely for a model to memorize the noise of the limited dataset.

Option C, "**Use a smaller set of features**," is effective in reducing overfitting. Models trained on datasets with too many features might learn relationships specific to those features in the training set that might not exist outside. Feature selection, or reducing the feature space, prevents the model from overfitting to irrelevant details, enabling it to focus on more important patterns.

Option D, "**Use a larger set of features**," would worsen overfitting, therefore it's incorrect. Adding more features increase the chances of the model learning the spurious correlations present in training data.

Option E, "**Increase the regularization parameters**," is another method to mitigate overfitting. Regularization techniques, such as L1 or L2 regularization, add penalties to the model's complexity, discouraging it from learning overly intricate patterns that generalize poorly. Increasing regularization makes it harder to model noise or outliers. This concept is explained in detail on Google Cloud documentation related to ML model training.

Option F, "**Decrease the regularization parameters**," would worsen overfitting, therefore it's incorrect.

In summary, by obtaining more data (A), simplifying the input features (C), and reducing model complexity through regularization (E), we force the model to focus on generalizable patterns rather than memorizing the training set, effectively reducing overfitting.

Authoritative Links:

Google Cloud AI Platform documentation on training ML models: <https://cloud.google.com/ai-platform/docs/ml-solutions/overfitting>

Google Machine Learning Crash Course on Regularization: <https://developers.google.com/machine-learning/crash-course/regularization-for-simplicity/video-lecture>

Question: 52

CertyIQ

You are implementing security best practices on your data pipeline. Currently, you are manually executing jobs as the Project Owner. You want to automate these jobs by taking nightly batch files containing non-public information from Google Cloud Storage, processing them with a Spark Scala job on a Google Cloud Dataproc cluster, and depositing the results into Google BigQuery. How should you securely run this workload?

- A. Restrict the Google Cloud Storage bucket so only you can see the files
- B. Grant the Project Owner role to a service account, and run the job with it
- C. Use a service account with the ability to read the batch files and to write to BigQuery
- D. Use a user account with the Project Viewer role on the Cloud Dataproc cluster to read the batch files and write to BigQuery

Answer: C

Explanation:

The correct answer is **C. Use a service account with the ability to read the batch files and to write to BigQuery**. Here's why:

Option C embodies the principle of least privilege, a core security best practice. Service accounts are designed for applications and services, not human users. By granting a service account only the necessary permissions (read access to Cloud Storage and write access to BigQuery), we minimize the risk associated with overly permissive access. If the service account is compromised, the potential damage is limited.

Option A, restricting the Cloud Storage bucket solely to the Project Owner, is insufficient. While good practice generally, this doesn't automate job execution. You would still have to manually initiate the Spark job, which contradicts the requirement for automated nightly runs.

Option B, granting the Project Owner role to a service account, is overly permissive. The Project Owner role provides broad access, far exceeding what's needed for the job, increasing the attack surface. It violates the principle of least privilege and poses a significant security risk.

Option D, using a user account, even with Project Viewer, is a poor security practice for automated processes. User accounts are tied to individuals and not appropriate for application-level access, they also lack the security benefit of tightly scoped permissions.

Therefore, using a dedicated service account with specific, restricted permissions is the most secure and efficient way to automate the data pipeline. This follows cloud security best practices and minimizes the potential for unauthorized access or data breaches. This allows for automated operation and enhanced security.

For further research, refer to the Google Cloud documentation on:

Service Accounts: <https://cloud.google.com/iam/docs/service-accounts>

Principle of Least Privilege: <https://cloud.google.com/security/gdpr/principles/least-privilege>

IAM Best Practices: <https://cloud.google.com/iam/docs/best-practices>

Question: 53

CertyIQ

You are using Google BigQuery as your data warehouse. Your users report that the following simple query is running very slowly, no matter when they run the query:

```
SELECT country, state, city FROM [myproject:mydataset.mytable] GROUP BY country
```

You check the query plan for the query and see the following output in the Read section of Stage:1:



What is the most likely cause of the delay for this query?

- A. Users are running too many concurrent queries in the system
- B. The [myproject:mydataset.mytable] table has too many partitions
- C. Either the state or the city columns in the [myproject:mydataset.mytable] table have too many NULL values
- D. Most rows in the [myproject:mydataset.mytable] table have the same value in the country column, causing data skew

Answer: D

Explanation:

D Image says that average(dark) and maximum(light) have difference in few times, this it is a skew The color indicators show the relative timings for all steps across all stages. For example, the COMPUTE step of Stage 00 shows a bar whose shaded fraction is 21/30 since 30ms is the maximum time spent in a single step of any stage. The parallel input information shows that each stage required only a single worker, so there's no variance between average and slowest timings.

Reference:

<https://cloud.google.com/bigquery/query-plan>

<https://cloud.google.com/bigquery/docs/best-practices-performance-patterns>

Question: 54

CertyIQ

Your globally distributed auction application allows users to bid on items. Occasionally, users place identical bids at nearly identical times, and different application servers process those bids. Each bid event contains the item, amount, user, and timestamp. You want to collate those bid events into a single location in real time to determine which user bid first. What should you do?

- A. Create a file on a shared file and have the application servers write all bid events to that file. Process the file with Apache Hadoop to identify which user bid first.
- B. Have each application server write the bid events to Cloud Pub/Sub as they occur. Push the events from Cloud Pub/Sub to a custom endpoint that writes the bid event information into Cloud SQL.
- C. Set up a MySQL database for each application server to write bid events into. Periodically query each of those distributed MySQL databases and update a master MySQL database with bid event information.
- D. Have each application server write the bid events to Google Cloud Pub/Sub as they occur. Use a pull subscription to pull the bid events using Google Cloud Dataflow. Give the bid for each item to the user in the bid event that is processed first.

Answer: B

Explanation:

The optimal solution is **B**, utilizing Cloud Pub/Sub and Cloud SQL. Here's why:

Option **A** introduces a shared file system for bid events, which suffers from concurrency issues and potential data loss. Processing with Hadoop adds latency, hindering real-time analysis crucial for determining the first bid. File systems are not designed for high-throughput writes from multiple sources concurrently, making this solution unsuitable.

Option **C**, using distributed MySQL databases, introduces complexity in querying and synchronizing data. Regularly querying and updating a master database creates delays and is not ideal for real-time scenarios. Managing multiple MySQL instances and reconciliation is also operationally complex.

Option **B** leverages Cloud Pub/Sub, a messaging service that enables decoupled and scalable real-time data ingestion. This ensures that bid events are reliably captured as they occur. Dataflow offers a managed service for stream and batch data processing. Although Dataflow is mentioned in Option D, the critical difference is the data landing location. Option B uses a custom endpoint that lands into Cloud SQL, where data is stored in a structured manner. While both use Pub/Sub, option D implies processing within Dataflow itself, not landing the data for later analysis, which is needed in this scenario. Cloud SQL is a managed, relational database for structured data storage, making it perfect for quickly identifying the first bid. Push subscriptions in Pub/Sub ensure immediate event delivery to the custom endpoint, enabling timely analysis and resolution of concurrent bids. The architecture in Option B allows real-time analysis within milliseconds.

Here are some links for further research:

Cloud Pub/Sub: <https://cloud.google.com/pubsub/docs>

Cloud SQL: <https://cloud.google.com/sql/docs>

Cloud Dataflow: <https://cloud.google.com/dataflow/docs>

Push vs Pull Subscriptions in Pub/Sub: https://cloud.google.com/pubsub/docs/subscriber#push_and_pull

Therefore, option **B** provides a robust, scalable, and real-time solution for processing and analyzing bid

events.

Question: 55

CertyIQ

Your organization has been collecting and analyzing data in Google BigQuery for 6 months. The majority of the data analyzed is placed in a time-partitioned table named `events_partitioned`. To reduce the cost of queries, your organization created a view called `events`, which queries only the last 14 days of data. The view is described in legacy SQL. Next month, existing applications will be connecting to BigQuery to read the events data via an ODBC connection. You need to ensure the applications can connect. Which two actions should you take? (Choose two.)

- A. Create a new view over `events` using standard SQL
- B. Create a new partitioned table using a standard SQL query
- C. Create a new view over `events_partitioned` using standard SQL
- D. Create a service account for the ODBC connection to use for authentication
- E. Create a Google Cloud Identity and Access Management (Cloud IAM) role for the ODBC connection and shared events

Answer: CD

Explanation:

Here's a detailed justification for choosing options C and D as the correct actions:

Option C: Create a new view over `events_partitioned` using standard SQL.

The original view `events` uses legacy SQL, which is deprecated and not compatible with ODBC connections using the BigQuery API. To enable ODBC connectivity, you must rewrite the view in standard SQL. Creating a new view directly on the time-partitioned table `events_partitioned` ensures that the view will continue to only query the data needed by applying filters. This improves efficiency and reduces cost compared to querying the whole table. Also, standard SQL has enhanced functionality and compatibility and is the recommended query language for BigQuery.

Option D: Create a service account for the ODBC connection to use for authentication.

Applications connecting to BigQuery via ODBC require a mechanism to authenticate and authorize access. Service accounts provide a secure way to do this. Service accounts are special Google accounts that do not require human interaction and are designed for applications and automated processes to access Google Cloud resources. Instead of using individual user credentials for each application, you create one service account per application/system and grant that account the appropriate permissions to access the relevant BigQuery datasets. This is more secure and manageable than user-based authentication for systems.

Why other options are incorrect:

Option A: Creating a new view over `events` does not resolve the issue of legacy SQL. The core issue is not that `events` needs to be overwritten, it is that a legacy view is being used.

Option B: Creating a new partitioned table is unnecessary for the given scenario. The question indicates that `events_partitioned` is suitable and is already partitioned.

Option E: While creating a Cloud IAM role is necessary for granting permissions, it is done in addition to creating a service account and not in place of it. The service account is what is associated with the role for authentication purposes.

Authoritative Links:

BigQuery standard SQL: <https://cloud.google.com/bigquery/docs/reference/standard-sql/>

BigQuery ODBC driver: <https://cloud.google.com/bigquery/docs/reference/odbc-jdbc-drivers>

Service accounts: <https://cloud.google.com/iam/docs/service-accounts>

IAM roles: <https://cloud.google.com/iam/docs/understanding-roles>

Question: 56

CertyIQ

You have enabled the free integration between Firebase Analytics and Google BigQuery. Firebase now automatically creates a new table daily in BigQuery in the format `app_events_YYYYMMDD`. You want to query all of the tables for the past 30 days in legacy SQL. What should you do?

- A. Use the `TABLE_DATE_RANGE` function
- B. Use the `WHERE_PARTITIONTIME` pseudo column
- C. Use `WHERE date BETWEEN YYYY-MM-DD AND YYYY-MM-DD`
- D. Use `SELECT IF.(date >= YYYY-MM-DD AND date <= YYYY-MM-DD`

Answer: A

Explanation:

The correct answer is **A. Use the `TABLE_DATE_RANGE` function.**

Here's why:

Firebase Analytics integration with BigQuery automatically creates daily tables named `app_events_YYYYMMDD`. To query a range of these tables in Legacy SQL, you need a mechanism to address these date-suffixed table names programmatically. The `TABLE_DATE_RANGE` function in Legacy SQL is designed specifically for querying multiple tables that follow a date-based naming convention. It allows you to specify a table name prefix and a start and end date, enabling BigQuery to access only the relevant tables within the given date range. It's crucial for efficiency as it avoids unnecessary scanning of older, irrelevant data.

Options B, C, and D are not appropriate in Legacy SQL for querying date-suffixed tables created by Firebase Analytics integration:

B. Use the `WHERE_PARTITIONTIME` pseudo column: `_PARTITIONTIME` is used for querying partitioned tables, and while daily tables can be partitioned, Firebase Analytics tables aren't automatically configured as such and using it would only work on such tables. The provided context states the name format `app_events_YYYYMMDD`, implying separate tables instead of a single partitioned table.

C. Use `WHERE date BETWEEN YYYY-MM-DD AND YYYY-MM-DD`: This option is for filtering data within a single table based on a 'date' column. It doesn't address querying multiple tables based on date-suffixed names. It assumes a date column within the data, which is not part of Firebase Analytics' automatically created tables.

D. Use `SELECT IF.(date >= YYYY-MM-DD AND date <= YYYY-MM-DD`: This is an incomplete and syntactically incorrect approach in Legacy SQL and addresses filtering data within a single table, not querying multiple date-suffixed tables. The `IF` function requires a `THEN` and `ELSE` clause. Also it doesn't handle multiple tables.

Therefore, `TABLE_DATE_RANGE` is the only function specifically designed for querying a range of tables with date-based naming patterns in Legacy SQL, making it the most suitable solution for this scenario.

Reference:

[Legacy SQL `TABLE\_DATE\_RANGE` Function](#) (although Legacy SQL is deprecated, it's relevant to the

Question: 57

Your company is currently setting up data pipelines for their campaign. For all the Google Cloud Pub/Sub streaming data, one of the important business requirements is to be able to periodically identify the inputs and their timings during their campaign. Engineers have decided to use windowing and transformation in Google Cloud Dataflow for this purpose. However, when testing this feature, they find that the Cloud Dataflow job fails for the all streaming insert. What is the most likely cause of this problem?

- A. They have not assigned the timestamp, which causes the job to fail
- B. They have not set the triggers to accommodate the data coming in late, which causes the job to fail
- C. They have not applied a global windowing function, which causes the job to fail when the pipeline is created
- D. They have not applied a non-global windowing function, which causes the job to fail when the pipeline is created

Answer: D**Explanation:**

The most likely cause of the Cloud Dataflow job failure for streaming inserts when using windowing and transformation is the absence of a non-global windowing function. Here's why:

Dataflow pipelines, especially those processing unbounded (streaming) data, require a mechanism to group incoming elements into finite chunks for processing. This is the fundamental role of windowing. Without a windowing function, Dataflow doesn't know how to group elements for aggregations or other window-specific operations.

Global windows, while technically a windowing strategy, aggregate all data into a single window. This isn't useful for periodic analysis of streaming data because it waits for the entire stream to finish before processing. Streaming jobs are inherently endless, making global windows impractical in most streaming scenarios intended for real-time or near-real-time insights.

Dataflow throws an error if you try to use transformations that require windowing (like aggregations, joins, etc.) without explicitly defining a windowing strategy that breaks the stream into smaller chunks. The absence of a non-global windowing function means the pipeline tries to operate on the unbounded stream as if it were a single unit, which is incompatible with common windowed transformations. The error often arises when the pipeline is created because the validity of the windowing strategy is checked early in the process.

Options A and B are related to common challenges in streaming data processing, such as handling out-of-order data (late data) and timestamp extraction. But those are generally considered after the data has been properly windowed. While important for data accuracy and completeness, those problems would not produce this specific pipeline creation failure.

For further research, consult the official Google Cloud Dataflow documentation on windowing:

Dataflow Windowing: <https://cloud.google.com/dataflow/docs/concepts/windowing>

Apache Beam Windowing: <https://beam.apache.org/documentation/programming-guide/#windowing>

(Dataflow is based on the Apache Beam model)

These resources explain the role of windowing, different windowing types (fixed, sliding, session), and how to apply them in your Dataflow pipelines. They also cover timestamp assignment and late data handling.

Question: 58

You architect a system to analyze seismic data. Your extract, transform, and load (ETL) process runs as a series of MapReduce jobs on an Apache Hadoop cluster. The ETL process takes days to process a data set because some steps are computationally expensive. Then you discover that a sensor calibration step has been omitted. How should you change your ETL process to carry out sensor calibration systematically in the future?

- A. Modify the transform MapReduce jobs to apply sensor calibration before they do anything else.
- B. Introduce a new MapReduce job to apply sensor calibration to raw data, and ensure all other MapReduce jobs are chained after this.
- C. Add sensor calibration data to the output of the ETL process, and document that all users need to apply sensor calibration themselves.
- D. Develop an algorithm through simulation to predict variance of data output from the last MapReduce job based on calibration factors, and apply the correction to all data.

Answer: B

Explanation:

The most effective approach to incorporate the missing sensor calibration step into the existing MapReduce-based ETL process is option B: introduce a new MapReduce job dedicated to sensor calibration and chain all subsequent MapReduce jobs after it.

Here's why:

Data Integrity and Consistency: By applying calibration to the raw data before any other transformations, you ensure that all subsequent processing steps operate on corrected, reliable data. This is crucial for accurate analysis and prevents the propagation of errors.

Centralized Calibration: Having a dedicated calibration job provides a centralized and maintainable location for the calibration logic. Changes to the calibration process only need to be made in one place, minimizing the risk of inconsistencies.

Clear Data Provenance: Introducing a separate job makes it clear that the data has been calibrated, improving data provenance and auditability.

Modularity and Reusability: A dedicated calibration job can be easily reused for future data sets or even for other analysis pipelines that use the same sensor data.

Maintain Existing Structure: Introducing a new MapReduce job is minimally invasive and preserves the existing structure of the ETL process.

Option A, modifying existing transform jobs, would scatter the calibration logic across multiple jobs, making maintenance and updates difficult. Option C, adding calibration data to the output and relying on users to apply the calibration themselves, is unacceptable because it introduces the potential for human error and inconsistencies. Data analysis should aim to produce clean, corrected data. Option D, predicting variance and applying a correction to the final output, is complex, may introduce inaccuracies, and doesn't address the fundamental problem of working with uncalibrated data from the start.

Relevant Cloud Computing Concepts:

Data Pipelines: This scenario focuses on designing and managing a data pipeline for ETL processing.

MapReduce: Understanding how MapReduce distributes data processing across a cluster is essential.

Data Quality: Sensor calibration directly impacts the quality of the input data.

Data Governance: A well-defined and auditable ETL process is important for data governance.

Authoritative Links for Further Research:

Google Cloud Dataflow: A fully-managed, scalable, and reliable service for building data pipelines. Though the question uses MapReduce on Hadoop, Dataflow is a modern alternative on Google Cloud.

<https://cloud.google.com/dataflow/docs>

Apache Hadoop MapReduce: The fundamental concept behind the original question's architecture.
<https://hadoop.apache.org/docs/current/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html>

Question: 59

CertyIQ

An online retailer has built their current application on Google App Engine. A new initiative at the company mandates that they extend their application to allow their customers to transact directly via the application. They need to manage their shopping transactions and analyze combined data from multiple datasets using a business intelligence (BI) tool. They want to use only a single database for this purpose. Which Google Cloud database should they choose?

- A. BigQuery
- B. Cloud SQL
- C. Cloud BigTable
- D. Cloud Datastore

Answer: B

Explanation:

The correct answer is **B. Cloud SQL**. Here's why:

The scenario requires a database that supports transactional workloads (managing shopping transactions) and analytical workloads (BI using combined datasets). While BigQuery (A) excels at analytics, it's not designed for handling real-time transactional data from an online shopping cart. Cloud Bigtable (C) is a NoSQL database best suited for high-volume, low-latency data storage and retrieval, but it lacks strong transactional capabilities and complex querying required for BI. Cloud Datastore (D), now known as Firestore in Datastore mode, is a NoSQL document database suitable for application data but not ideal for complex analytics or transactional integrity required for financial transactions.

Cloud SQL, on the other hand, is a fully managed relational database service that offers MySQL, PostgreSQL, and SQL Server options. Relational databases are inherently designed for ACID (Atomicity, Consistency, Isolation, Durability) compliance, which is crucial for maintaining the integrity of financial transactions. Furthermore, relational databases readily integrate with BI tools through standard SQL queries, making them suitable for analyzing combined datasets. While it might require some schema design and optimization, Cloud SQL offers the best balance between transactional needs of an e-commerce platform built using Google App Engine and the analytical requirements of the business intelligence initiative. It provides familiar SQL interface and supports the necessary features for both transactional and analytical processing needed by the retailer. This single database approach reduces complexity and cost compared to managing separate transactional and analytical databases. Cloud SQL's scalability also makes it a suitable long-term solution as the business grows.

Authoritative Links:

Cloud SQL: <https://cloud.google.com/sql>

ACID Properties: <https://en.wikipedia.org/wiki/ACID>

BigQuery: <https://cloud.google.com/bigquery>

Question: 60

CertyIQ

You launched a new gaming app almost three years ago. You have been uploading log files from the previous day to a separate Google BigQuery table with the table name format LOGS_yyyymmdd. You have been using table wildcard functions to generate daily and monthly reports for all time ranges. Recently, you discovered that some

queries that cover long date ranges are exceeding the limit of 1,000 tables and failing. How can you resolve this issue?

- A. Convert all daily log tables into date-partitioned tables
- B. Convert the sharded tables into a single partitioned table
- C. Enable query caching so you can cache data from previous months
- D. Create separate views to cover each month, and query from these views

Answer: B

Explanation:

The correct answer is **B. Convert the sharded tables into a single partitioned table.**

Here's a detailed justification:

The primary problem is the limit of 1,000 tables that can be scanned within a single BigQuery query when using table wildcard functions. Currently, the log data is sharded into many small tables (LOGS_YYYYMMDD). Wildcard tables are useful when dealing with lots of data, but querying over many such tables will easily hit BigQuery's table limit.

Partitioning, on the other hand, addresses this limit directly. By converting all the daily sharded tables into a single date-partitioned table, you consolidate the data into one logical table. BigQuery then leverages the partitioning scheme (in this case, date) to only scan the relevant partitions that match the query's date range, dramatically reducing the number of tables scanned and circumventing the 1,000-table limit. This allows you to efficiently query large date ranges.

Option A is incorrect because converting all daily log tables into date-partitioned tables wouldn't solve the table count issue if you still maintained a multitude of separate partitioned tables. The benefit is gained by having a single partitioned table.

Option C, enabling query caching, will only help with repeated queries of the exact same data. It won't reduce the number of tables scanned in the initial query. Cache only helps with a repeated identical query. If the data changes or if even one parameter changes, cache will not be used. It does not address the underlying limit on table scanning.

Option D, creating separate views for each month, only adds another layer of abstraction and does not solve the table scanning issue. Each view still effectively queries potentially many tables within that month. This maintains the same limitation on table scanning that the current wildcard table setup suffers from. It also would be extremely tedious to maintain.

Therefore, converting the sharded tables into a single date-partitioned table is the optimal solution because it directly addresses the 1,000-table limit while maintaining efficient querying capabilities.

For further research:

BigQuery Partitioned Tables: <https://cloud.google.com/bigquery/docs/partitioned-tables>

BigQuery Table Wildcards: <https://cloud.google.com/bigquery/docs/querying-wildcard-tables> (Note the limitations mentioned regarding table limits)

BigQuery Query Caching: <https://cloud.google.com/bigquery/docs/cached-queries>

Question: 61

CertyIQ

Your analytics team wants to build a simple statistical model to determine which customers are most likely to work with your company again, based on a few different metrics. They want to run the model on Apache Spark, using data housed in Google Cloud Storage, and you have recommended using Google Cloud

Dataproc to execute this job. Testing has shown that this workload can run in approximately 30 minutes on a 15-node cluster, outputting the results into Google BigQuery. The plan is to run this workload weekly. How should you optimize the cluster for cost?

- A. Migrate the workload to Google Cloud Dataflow
- B. Use pre-emptible virtual machines (VMs) for the cluster
- C. Use a higher-memory node so that the job runs faster
- D. Use SSDs on the worker nodes so that the job can run faster

Answer: B

Explanation:

The most cost-effective approach for a weekly, predictable Dataproc job is to utilize preemptible VMs (option B). Preemptible VMs offer a significant discount compared to regular VMs, making them ideal for fault-tolerant workloads like Spark jobs. Since the job runs weekly, a preemption event does not drastically disrupt the process as the job can be restarted. The 30-minute runtime on a 15-node cluster suggests a manageable duration for recovery if preemption occurs. Option A, migrating to Dataflow, introduces a different paradigm that might require a significant rewrite of existing Spark code and doesn't directly address cost optimization within a Dataproc context. Options C and D, using higher-memory or SSDs, will likely increase the per-node cost, offsetting the potential runtime gains; this doesn't tackle the root of the cost optimization goal. Preemptible VMs, on the other hand, directly reduce the hourly cost. The nature of the workload makes it suitable for handling potential interruptions. Further research on preemptible VMs for Dataproc can be found here: <https://cloud.google.com/dataproc/docs/concepts/preemptible-vms>

Question: 62

CertyIQ

Your company receives both batch- and stream-based event data. You want to process the data using Google Cloud Dataflow over a predictable time period. However, you realize that in some instances data can arrive late or out of order. How should you design your Cloud Dataflow pipeline to handle data that is late or out of order?

- A. Set a single global window to capture all the data.
- B. Set sliding windows to capture all the lagged data.
- C. Use watermarks and timestamps to capture the lagged data.
- D. Ensure every datasource type (stream or batch) has a timestamp, and use the timestamps to define the logic for lagged data.

Answer: C

Explanation:

The correct answer is C: Use watermarks and timestamps to capture the lagged data. This approach is fundamental to handling late or out-of-order data in streaming data processing with Dataflow.

Dataflow, unlike traditional batch processing, operates on unbounded data streams. Therefore, it needs mechanisms to determine when it has processed all data for a specific window or period, even when data arrives late. Watermarks provide this mechanism. A watermark is a system's understanding of when all data up to a certain point in time is expected to have arrived. It's an approximation, not a guarantee. Data arriving after the watermark for a window has passed is considered late data.

Timestamps are essential because Dataflow uses them to assign data to the correct processing window and to calculate the watermark. Each event in the stream must have an associated timestamp. If events do not have a timestamp, it's necessary to create or assign one during pipeline processing. By using timestamps and

watermarks, Dataflow can handle out-of-order data efficiently by correctly grouping them into corresponding windows based on their timestamps, even if they arrive later than anticipated. This approach ensures data is processed correctly, even with inherent delays or irregularities in the data stream.

Option A (single global window) would capture all data but wouldn't allow for time-based analysis. Option B (sliding windows) is useful for continuous analytics over shifting time ranges, but does not inherently address the handling of late data. Option D mentions using timestamps, which is important but doesn't address how these timestamps relate to late data processing, and doesn't mention watermarks which is a core concept for handling late data. Option C incorporates both timestamps and watermarks which are critical when dealing with both batch and stream-based data sources to handle late-arriving data properly.

Therefore, using watermarks and timestamps allows Dataflow to manage late data correctly by knowing when to consider a window complete even with out-of-order and late data arrivals.

Authoritative Links:

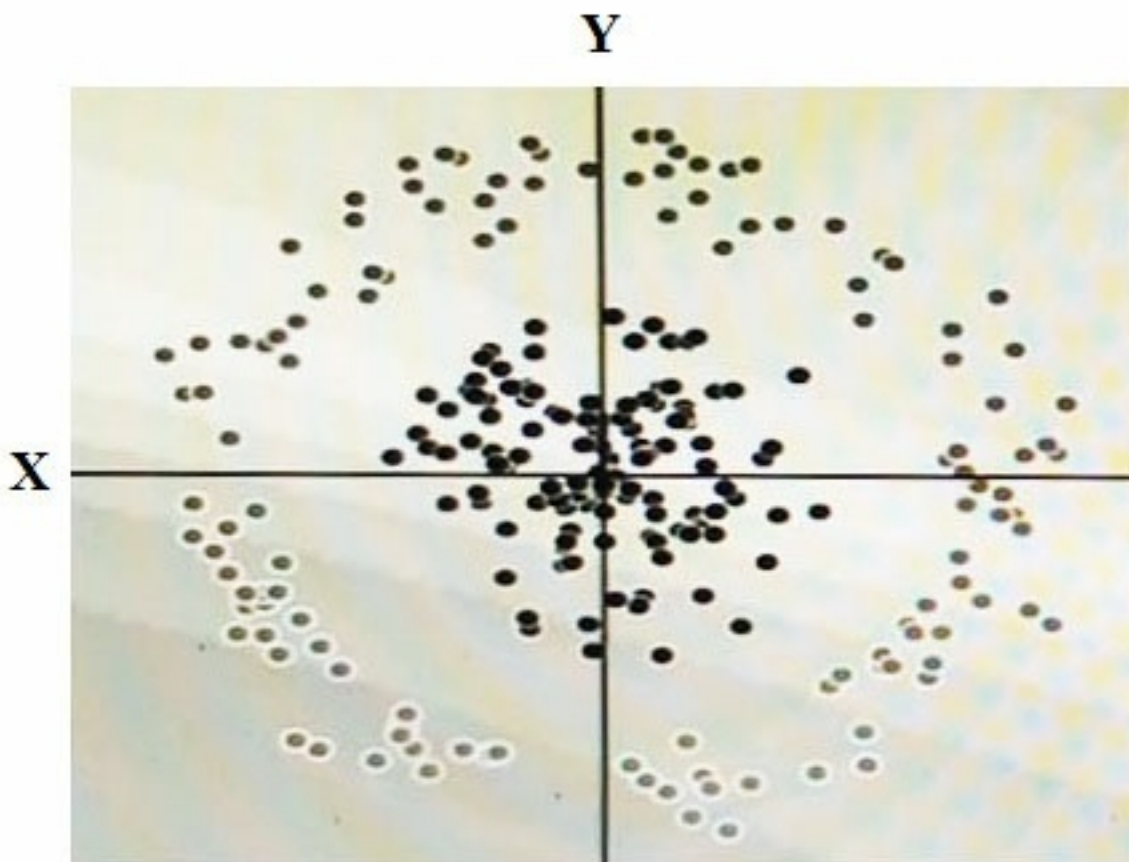
[Apache Beam Programming Guide - Watermarks and Late Data](#): This is an excellent resource detailing how watermarks function in Beam (Dataflow's programming model) and the concept of late data.

[Google Cloud Dataflow documentation - Time and Windows](#): This is Google's Dataflow specific documentation that discusses time and windowing, essential to handling data ordering and late data.

Question: 63

CertyIQ

You have some data, which is shown in the graphic below. The two dimensions are X and Y, and the shade of each dot represents what class it is. You want to classify this data accurately using a linear algorithm. To do this you need to add a synthetic feature. What should the value of that feature be?



- A. $X^2 + Y^2$
- B. X^2
- C. Y^2

D. $\cos(X)$

Answer: A

Explanation:

For fitting a linear classifier when the data is in a circle use A.

Question: 64

CertyIQ

You are integrating one of your internal IT applications and Google BigQuery, so users can query BigQuery from the application's interface. You do not want individual users to authenticate to BigQuery and you do not want to give them access to the dataset. You need to securely access BigQuery from your IT application. What should you do?

- A. Create groups for your users and give those groups access to the dataset
- B. Integrate with a single sign-on (SSO) platform, and pass each user's credentials along with the query request
- C. Create a service account and grant dataset access to that account. Use the service account's private key to access the dataset
- D. Create a dummy user and grant dataset access to that user. Store the username and password for that user in a file on the files system, and use those credentials to access the BigQuery dataset

Answer: C

Explanation:

The correct approach is to utilize a service account (Option C). Service accounts are non-human accounts specifically designed for applications and services to interact with Google Cloud resources. This addresses the requirement of not having individual users authenticate directly to BigQuery. By granting the service account access to the BigQuery dataset, the application can access the data without exposing user credentials or giving them direct data access. The application can use the service account's private key to authenticate and authorize API calls to BigQuery, as opposed to storing sensitive credentials like usernames and passwords. This method ensures that only the application, through the service account, has access, which is a best practice for security. Sharing credentials across multiple applications or using dummy accounts, as suggested in options B and D, introduces significant security risks. Using groups for user-based access, as in option A, defeats the requirement of not giving individual users access to the dataset. Therefore, the service account method, as described in option C, offers the most secure and appropriate solution for accessing BigQuery from an application without direct user authentication.

Authoritative links:

Service accounts: Official Google Cloud documentation on service accounts.

Best practices for using service accounts: Guidance on using service accounts securely and effectively.

BigQuery Authentication: Documentation on authenticating to BigQuery, covering service accounts and other methods.

Question: 65

CertyIQ

You are building a data pipeline on Google Cloud. You need to prepare data using a casual method for a machine-learning process. You want to support a logistic regression model. You also need to monitor and adjust for null values, which must remain real-valued and cannot be removed. What should you do?

- A. Use Cloud Dataprep to find null values in sample source data. Convert all nulls to 'none' using a Cloud Dataproc job.

- B. Use Cloud Dataprep to find null values in sample source data. Convert all nulls to 0 using a Cloud Dataprep job.
- C. Use Cloud Dataflow to find null values in sample source data. Convert all nulls to 'none' using a Cloud Dataprep job.
- D. Use Cloud Dataflow to find null values in sample source data. Convert all nulls to 0 using a custom script.

Answer: B

Explanation:

The correct answer is B. Here's why:

The problem requires handling null values in a dataset intended for a logistic regression model. Logistic regression, being a numerical algorithm, cannot directly process non-numeric values like "none". Therefore, converting null values to a string, like in options A and C, is not suitable.

Option D suggests using a custom script within Cloud Dataflow. While this approach is technically viable, it introduces unnecessary complexity compared to the simplicity offered by Cloud Dataprep, which is explicitly designed for casual data preparation tasks as the question requires. Dataprep's visual interface allows for simpler and faster handling of common data transformations.

Option B leverages Cloud Dataprep's strength in data profiling to identify null values. Crucially, it also uses Dataprep's capability to easily convert these nulls to '0', fulfilling the need to retain real-valued data as required by the question. Substituting null values with 0 is a common, legitimate approach when working with numerical data for machine learning models, assuming that zero is a sensible placeholder in the context of the data. This makes the data immediately suitable for logistic regression without requiring code development.

Therefore, option B effectively addresses the problem's requirements, balancing efficacy with the desired ease of use in the preparation process.

Authoritative Links:

Cloud Dataprep Documentation: <https://cloud.google.com/dataprep/docs> - Provides comprehensive information on Dataprep's features.

Logistic Regression: A quick search for "logistic regression data preprocessing null values" will show the common practice of replacing nulls with reasonable numerical values.

Cloud Dataflow Documentation: <https://cloud.google.com/dataflow/docs> - Can be used as a reference for comparison but is not the optimal solution for the scenario presented.

Question: 66

CertyIQ

You set up a streaming data insert into a Redis cluster via a Kafka cluster. Both clusters are running on Compute Engine instances. You need to encrypt data at rest with encryption keys that you can create, rotate, and destroy as needed. What should you do?

- A. Create a dedicated service account, and use encryption at rest to reference your data stored in your Compute Engine cluster instances as part of your API service calls.
- B. Create encryption keys in Cloud Key Management Service. Use those keys to encrypt your data in all of the Compute Engine cluster instances.
- C. Create encryption keys locally. Upload your encryption keys to Cloud Key Management Service. Use those keys to encrypt your data in all of the Compute Engine cluster instances.
- D. Create encryption keys in Cloud Key Management Service. Reference those keys in your API service calls when accessing the data in your Compute Engine cluster instances.

Answer: B

Explanation:

The correct answer is **B. Create encryption keys in Cloud Key Management Service. Use those keys to encrypt your data in all of the Compute Engine cluster instances.**

Here's a breakdown of why:

The primary requirement is to encrypt data at rest in the Redis and Kafka clusters, which are hosted on Compute Engine instances. We also need full control over key creation, rotation, and destruction. Google Cloud Key Management Service (Cloud KMS) is the ideal service for this. Cloud KMS allows you to manage encryption keys centrally, enabling you to create, rotate, and destroy keys as required. Option B directly uses Cloud KMS to generate the necessary keys. These keys are then used to encrypt data stored on the Compute Engine instances, specifically the Redis and Kafka persistent storage. The encryption takes place at the storage layer of these instances.

Option A is incorrect because it focuses on encryption during API service calls. While important, this addresses encryption in transit, not at rest. Additionally, relying on the instances' default encryption is not the same as providing user-managed keys which are required in this scenario. Option C is not recommended because creating keys locally and uploading them to Cloud KMS introduces an unnecessary security risk and complexity. Local key management can be cumbersome and vulnerable to compromise. Cloud KMS is designed to securely manage key generation from the start. Finally, option D incorrectly suggests encryption is handled through API calls. The goal here is data at rest encryption, and it should not be performed via API calls, but at the storage level of the Compute Engines.

By using Cloud KMS and applying encryption at the storage layer of the Compute Engines, we satisfy the need for user-controlled encryption at rest and gain the flexibility to manage keys securely and efficiently using a central service.

Relevant Cloud Computing Concepts:

Encryption at Rest: Protecting data when it's stored on a persistent medium (like a hard drive).

Cloud Key Management Service (Cloud KMS): A managed service for creating, using, rotating, and destroying cryptographic keys.

Compute Engine: Google Cloud's Infrastructure-as-a-Service, which provides virtual machines.

Authoritative Links for further research:

[Cloud KMS Documentation](#)

[Encryption at rest in Compute Engine](#)

Question: 67

CertyIQ

You are developing an application that uses a recommendation engine on Google Cloud. Your solution should display new videos to customers based on past views. Your solution needs to generate labels for the entities in videos that the customer has viewed. Your design must be able to provide very fast filtering suggestions based on data from other customer preferences on several TB of data. What should you do?

- A. Build and train a complex classification model with Spark MLlib to generate labels and filter the results. Deploy the models using Cloud Dataproc. Call the model from your application.
- B. Build and train a classification model with Spark MLlib to generate labels. Build and train a second classification model with Spark MLlib to filter results to match customer preferences. Deploy the models using Cloud Dataproc. Call the models from your application.
- C. Build an application that calls the Cloud Video Intelligence API to generate labels. Store data in Cloud Bigtable, and filter the predicted labels to match the user's viewing history to generate preferences.
- D. Build an application that calls the Cloud Video Intelligence API to generate labels. Store data in Cloud SQL, and join and filter the predicted labels to match the user's viewing history to generate preferences.

Answer: C

Explanation:

Option C is the most appropriate solution for building a recommendation engine that efficiently handles large datasets and provides fast filtering. Cloud Video Intelligence API readily extracts labels from video content, eliminating the need for custom model training for label generation, which is time-consuming and requires significant expertise (as suggested in options A and B). This accelerates development. Cloud Bigtable, a NoSQL wide-column database, is designed to handle massive amounts of data with low latency, making it ideal for storing and querying user viewing history and video labels at scale, crucial for fast filtering. Unlike Cloud SQL (as in option D) which is a relational database, Bigtable is specifically optimized for the high read/write throughput requirements of this recommendation use case. Bigtable's ability to filter based on column values directly matches the requirement to filter based on user preferences. Storing and joining the data in SQL is possible, but less performant when dealing with a TB of data and requiring low latency filtering. Option A and B propose using Spark MLlib and Cloud Dataproc. While powerful for batch processing and model training, they are not suitable for real-time label generation or filtering. Also, training and deploying multiple models adds unnecessary complexity to the solution. In essence, Cloud Video Intelligence API provides the labelling capabilities, while Bigtable provides scalability and the low latency query mechanism for the vast amount of data.

Authoritative links:

Cloud Video Intelligence API: <https://cloud.google.com/video-intelligence/docs>

Cloud Bigtable: <https://cloud.google.com/bigtable/docs>

Comparison of Cloud SQL and Cloud Bigtable: <https://cloud.google.com/learn/what-is-cloud-sql-vs-cloud-bigtable>

Question: 68

CertyIQ

You are selecting services to write and transform JSON messages from Cloud Pub/Sub to BigQuery for a data pipeline on Google Cloud. You want to minimize service costs. You also want to monitor and accommodate input data volume that will vary in size with minimal manual intervention. What should you do?

- A. Use Cloud Dataproc to run your transformations. Monitor CPU utilization for the cluster. Resize the number of worker nodes in your cluster via the command line.
- B. Use Cloud Dataproc to run your transformations. Use the diagnose command to generate an operational output archive. Locate the bottleneck and adjust cluster resources.
- C. Use Cloud Dataflow to run your transformations. Monitor the job system lag with Stackdriver. Use the default autoscaling setting for worker instances.
- D. Use Cloud Dataflow to run your transformations. Monitor the total execution time for a sampling of jobs. Configure the job to use non-default Compute Engine machine types when needed.

Answer: C

Explanation:

Option C, using Cloud Dataflow, is the most suitable solution for transforming JSON messages from Cloud Pub/Sub to BigQuery while minimizing costs and adapting to varying data volumes. Cloud Dataflow is a fully managed, serverless data processing service designed for both batch and stream data pipelines. Its autoscaling feature, a key aspect of the default setting mentioned, automatically adjusts the number of worker instances based on the workload, eliminating the need for manual intervention and optimizing resource utilization. This reduces operational overhead and minimizes costs during periods of low data volume. Monitoring "system lag" in Stackdriver provides critical insight into the pipeline's health, enabling you to identify and address bottlenecks. Options A and B, using Cloud Dataproc, are less suitable for this specific use case. Dataproc is primarily designed for running Hadoop and Spark clusters, requiring manual cluster

management and scaling, leading to increased operational burden and costs. While Dataproc offers cluster resizing, it involves manual intervention, contradicting the requirement for minimal manual effort. Option D while using Cloud Dataflow, focuses on execution time sampling which is not the most effective way to optimize a data pipeline. Autoscaling is Dataflow's strength and should be used. In summary, Dataflow's serverless nature, autoscaling capabilities, and integration with Stackdriver monitoring make it the ideal choice for a cost-effective and adaptable data pipeline, addressing the prompt's requirements effectively.

Authoritative Links:

Cloud Dataflow: <https://cloud.google.com/dataflow>

Dataflow Autoscaling: <https://cloud.google.com/dataflow/docs/guides/autoscaling>

Cloud Pub/Sub: <https://cloud.google.com/pubsub>

BigQuery: <https://cloud.google.com/bigquery>

Stackdriver (Google Cloud Observability): <https://cloud.google.com/stackdriver>

Cloud Dataproc: <https://cloud.google.com/dataproc>

Question: 69

CertyIQ

Your infrastructure includes a set of YouTube channels. You have been tasked with creating a process for sending the YouTube channel data to Google Cloud for analysis. You want to design a solution that allows your world-wide marketing teams to perform ANSI SQL and other types of analysis on up-to-date YouTube channels log data. How should you set up the log data transfer into Google Cloud?

- A. Use Storage Transfer Service to transfer the offsite backup files to a Cloud Storage Multi-Regional storage bucket as a final destination.
- B. Use Storage Transfer Service to transfer the offsite backup files to a Cloud Storage Regional bucket as a final destination.
- C. Use BigQuery Data Transfer Service to transfer the offsite backup files to a Cloud Storage Multi-Regional storage bucket as a final destination.
- D. Use BigQuery Data Transfer Service to transfer the offsite backup files to a Cloud Storage Regional storage bucket as a final destination.

Answer: C

Explanation:

In my opinion, BigQuery has the capability to retrieve "YouTube channel reports" directly from a YouTube streaming channel (<https://cloud.google.com/bigquery-transfer/docs/introduction>)

Upon transfer it can be exported to Cloud Storage using a simple file export or any service that google provides to have a federated datasource for BigQuery.

This data can again be accessed by BigQuery using the BigQuery Transfer Service and analyzed using ANSI-SQL

For the second part "worldwide" makes it "multi-regional"

Question: 70

CertyIQ

You are designing storage for very large text files for a data pipeline on Google Cloud. You want to support ANSI SQL queries. You also want to support compression and parallel load from the input locations using Google recommended practices. What should you do?

- A. Transform text files to compressed Avro using Cloud Dataflow. Use BigQuery for storage and query.

- B. Transform text files to compressed Avro using Cloud Dataflow. Use Cloud Storage and BigQuery permanent linked tables for query.
- C. Compress text files to gzip using the Grid Computing Tools. Use BigQuery for storage and query.
- D. Compress text files to gzip using the Grid Computing Tools. Use Cloud Storage, and then import into Cloud Bigtable for query.

Answer: B

Explanation:

Option B is the most suitable choice because it leverages Google Cloud's recommended practices for handling large text files while enabling ANSI SQL queries. Cloud Dataflow is a robust service designed for data transformation, making it ideal for converting the raw text files into compressed Avro format. Avro is a row-based data serialization format that supports schema evolution and efficient compression, optimizing both storage and query performance. Storing the compressed Avro files in Cloud Storage provides cost-effective and scalable storage. BigQuery permanent linked tables then allow querying the data directly from Cloud Storage using ANSI SQL without the need to import it. This approach eliminates redundant storage and simplifies the overall architecture. Using Cloud Dataflow and BigQuery together aligns with Google's modern data processing best practices. Options A and C involve directly loading or using Gzip files with BigQuery, which while feasible is less optimized for query performance compared to using Avro files. Option D is unsuitable as Bigtable is not designed for analytical SQL queries, and it does not work well with large text files directly.

Authoritative Links for further research:

Cloud Dataflow: <https://cloud.google.com/dataflow/docs>

Avro format: <https://avro.apache.org/>

Cloud Storage: <https://cloud.google.com/storage/docs>

BigQuery Permanent Linked Tables: <https://cloud.google.com/bigquery/docs/external-data-cloud-storage>

BigQuery Best Practices: <https://cloud.google.com/bigquery/docs/best-practices-performance>

Question: 71

CertyIQ

You are developing an application on Google Cloud that will automatically generate subject labels for users' blog posts. You are under competitive pressure to add this feature quickly, and you have no additional developer resources. No one on your team has experience with machine learning. What should you do?

- A. Call the Cloud Natural Language API from your application. Process the generated Entity Analysis as labels.
- B. Call the Cloud Natural Language API from your application. Process the generated Sentiment Analysis as labels.
- C. Build and train a text classification model using TensorFlow. Deploy the model using Cloud Machine Learning Engine. Call the model from your application and process the results as labels.
- D. Build and train a text classification model using TensorFlow. Deploy the model using a Kubernetes Engine cluster. Call the model from your application and process the results as labels.

Answer: A

Explanation:

The correct answer is **A. Call the Cloud Natural Language API from your application. Process the generated Entity Analysis as labels.**

Here's why:

The scenario emphasizes speed and limited resources, especially the lack of ML expertise. Option A leverages

the pre-trained Cloud Natural Language API, a managed service. This eliminates the need to build, train, and deploy a custom ML model, which requires considerable time, effort, and expertise (as suggested in Options C and D). Entity analysis, a core feature of the Natural Language API, identifies key concepts (entities) within text, which directly translates to appropriate subject labels for blog posts. Sentiment analysis (Option B) focuses on the emotional tone of text, not relevant to subject labeling. Options C and D are not feasible due to the lack of internal machine learning knowledge, and using Kubernetes for option D further increases complexity. Utilizing an existing API like Cloud Natural Language allows for rapid implementation and avoids the overhead of managing infrastructure or model training, meeting the pressing need to add the feature quickly. Therefore, Cloud Natural Language API's Entity Analysis offers the most pragmatic and efficient approach.

Further research on Cloud Natural Language API and its capabilities can be found here:

Google Cloud Natural Language API Documentation: <https://cloud.google.com/natural-language/docs>
Entity Analysis documentation: <https://cloud.google.com/natural-language/docs/analyzing-entities>

Question: 72

CertyIQ

You are designing storage for 20 TB of text files as part of deploying a data pipeline on Google Cloud. Your input data is in CSV format. You want to minimize the cost of querying aggregate values for multiple users who will query the data in Cloud Storage with multiple engines. Which storage service and schema design should you use?

- A. Use Cloud Bigtable for storage. Install the HBase shell on a Compute Engine instance to query the Cloud Bigtable data.
- B. Use Cloud Bigtable for storage. Link as permanent tables in BigQuery for query.
- C. Use Cloud Storage for storage. Link as permanent tables in BigQuery for query.
- D. Use Cloud Storage for storage. Link as temporary tables in BigQuery for query.

Answer: C

Explanation:

The correct answer is C, using Cloud Storage for storage and linking it as permanent tables in BigQuery for querying.

Here's a detailed justification:

Cost Optimization: Cloud Storage is significantly cheaper than Cloud Bigtable for storing large volumes of data, especially text files like CSVs. Cloud Storage is designed for cost-effective object storage, making it ideal for data at rest. <https://cloud.google.com/storage/pricing>

Querying Aggregate Values: BigQuery is a fully managed, serverless data warehouse optimized for analyzing large datasets and performing aggregate queries. It's designed to handle complex analytical workloads efficiently. Cloud Bigtable is designed for low-latency reads/writes of large amounts of data and is not ideal for analytical queries. <https://cloud.google.com/bigquery/docs>

Multiple Users and Query Engines: BigQuery supports concurrent queries from multiple users and integrates seamlessly with various query engines (e.g., SQL, data science tools via APIs). This makes it ideal for serving multiple users with different analytical needs.

CSV Format: BigQuery can directly ingest and query CSV files stored in Cloud Storage. This eliminates the need for complex data transformations or schema migrations.

Permanent vs. Temporary Tables: Creating permanent tables in BigQuery provides a persistent, queryable dataset. Using temporary tables would require reloading the data each time, increasing cost and effort.

Permanent tables also enable better access control and data governance.

Cloud Bigtable Inappropriateness: Cloud Bigtable is a NoSQL database suitable for applications requiring very low latency reads and writes, such as real-time personalization or IoT data. It's not optimized for batch analytical queries or the CSV format. Using Cloud Bigtable and then querying it through BigQuery would add unnecessary complexity and cost.

HBase Shell Inefficiency: While HBase is an option for querying Cloud Bigtable, it's not suitable for ad-hoc aggregate queries from multiple users due to its complex setup and the nature of performing such analytical tasks on a NoSQL environment.

In summary, Cloud Storage offers the most cost-effective storage for 20 TB of text files. Linking this data as permanent tables in BigQuery allows multiple users to perform efficient aggregate queries with various query engines, fulfilling all requirements in the question. Bigtable would be overkill and more expensive for this scenario.

Question: 73

CertyIQ

You are designing storage for two relational tables that are part of a 10-TB database on Google Cloud. You want to support transactions that scale horizontally.

You also want to optimize data for range queries on non-key columns. What should you do?

- A. Use Cloud SQL for storage. Add secondary indexes to support query patterns.
- B. Use Cloud SQL for storage. Use Cloud Dataflow to transform data to support query patterns.
- C. Use Cloud Spanner for storage. Add secondary indexes to support query patterns.
- D. Use Cloud Spanner for storage. Use Cloud Dataflow to transform data to support query patterns.

Answer: C

Explanation:

The correct answer is C: "Use Cloud Spanner for storage. Add secondary indexes to support query patterns."

Here's why:

Horizontal Scalability and Transactions: Cloud Spanner is specifically designed to provide horizontally scalable, strongly consistent transactions. This is critical for a 10-TB database needing to scale without compromising data integrity. Cloud SQL, while offering transactions, generally faces more limitations in horizontal scaling for very large datasets compared to Spanner.

Range Queries on Non-Key Columns: Spanner supports secondary indexes, which allow you to optimize query performance on columns that are not part of the primary key. By creating secondary indexes on the non-key columns used in your range queries, you can significantly speed up those queries without needing to transform the data.

Why other options are not ideal:

Option A (Cloud SQL with secondary indexes): Cloud SQL's scalability limitations for a 10-TB database make it less suitable than Spanner. While Cloud SQL offers indexes, it may not scale as efficiently for large datasets and transaction volumes.

Option B (Cloud SQL with Cloud Dataflow): Using Cloud Dataflow to transform data to optimize for query patterns is more complex and costly than using Spanner with secondary indexes. It introduces an additional data processing step and potential latency. Dataflow is more relevant for ETL or complex transformations, not for optimizing basic query patterns on relational data.

Option D (Cloud Spanner with Cloud Dataflow): While Cloud Dataflow can be used with Spanner, it is not the

optimal solution for range queries on non-key columns. Spanner's secondary indexes offer a more direct and efficient solution for these types of queries. Using Dataflow would involve unnecessarily complex data transformation.

In summary, Cloud Spanner offers the necessary horizontal scalability and transaction support for a 10-TB database, and secondary indexes provide the most straightforward way to optimize range queries on non-key columns without requiring complex data transformations.

Authoritative Links:

Cloud Spanner Overview: <https://cloud.google.com/spanner/docs/overview>

Cloud Spanner Secondary Indexes: <https://cloud.google.com/spanner/docs/secondary-indexes>

Cloud SQL Overview: <https://cloud.google.com/sql/docs/mysql/introduction>

Cloud Dataflow Overview: <https://cloud.google.com/dataflow/docs/concepts/what-is-dataflow>

Question: 74

CertyIQ

Your financial services company is moving to cloud technology and wants to store 50 TB of financial time-series data in the cloud. This data is updated frequently and new data will be streaming in all the time. Your company also wants to move their existing Apache Hadoop jobs to the cloud to get insights into this data.

Which product should they use to store the data?

- A. Cloud Bigtable
- B. Google BigQuery
- C. Google Cloud Storage
- D. Google Cloud Datastore

Answer: A

Explanation:

The most appropriate choice for storing 50 TB of financial time-series data with frequent updates and continuous streaming, while also needing compatibility with Apache Hadoop jobs, is **Cloud Bigtable (A)**. Cloud Bigtable is a fully managed, scalable NoSQL database service designed for low-latency read and write access to large datasets. Its columnar storage model excels in handling time-series data, allowing for efficient storage and retrieval of data points across time. Furthermore, it's specifically built for high-throughput, which aligns with the requirement for continuous streaming. Crucially, Bigtable integrates well with the Hadoop ecosystem through its HBase compatibility, making it easy to migrate existing Hadoop jobs for data analysis. While BigQuery (B) is excellent for analytics, it's less suitable for transactional workloads with constant updates and streaming data. Cloud Storage (C) provides object storage, which isn't ideal for data that requires frequent updates, and Datastore (D) is better suited for application data, not large-scale time series data. Therefore, Bigtable's combination of high-throughput writes, low-latency reads, and Hadoop compatibility makes it the optimal solution for the company's needs.

Further research:

Cloud Bigtable Overview: <https://cloud.google.com/bigtable/docs/overview>

Cloud Bigtable for Time Series Data: <https://cloud.google.com/bigtable/docs/time-series>

HBase Compatibility in Bigtable: <https://cloud.google.com/bigtable/docs/hbase/>

Question: 75

CertyIQ

An organization maintains a Google BigQuery dataset that contains tables with user-level data. They want to expose aggregates of this data to other Google

Cloud projects, while still controlling access to the user-level data. Additionally, they need to minimize their overall storage cost and ensure the analysis cost for other projects is assigned to those projects. What should they do?

- A. Create and share an authorized view that provides the aggregate results.
- B. Create and share a new dataset and view that provides the aggregate results.
- C. Create and share a new dataset and table that contains the aggregate results.
- D. Create dataViewer Identity and Access Management (IAM) roles on the dataset to enable sharing.

Answer: A

Explanation:

The correct answer is A. Creating and sharing an authorized view is the most efficient and cost-effective method to achieve the described requirements. Authorized views in BigQuery allow you to share query results (aggregates in this case) without directly exposing the underlying user-level data. They operate like virtual tables; queries against the view are executed using the permissions of the view itself, not the user querying it. This enables secure sharing across projects. This approach minimizes storage costs because no new tables with aggregated data are created, hence no data duplication is involved. Since authorized views inherit billing from the querying project, query costs are correctly attributed to the consuming project, as opposed to the project where original data lies. Options B and C involve duplicating the data into new datasets/tables which can significantly increase storage costs and data management overhead. Option D would expose the entire dataset, including user-level data, which contradicts the core requirement of securing raw data.

Here are some authoritative links for further research:

BigQuery Authorized Views: <https://cloud.google.com/bigquery/docs/share-access-views>

BigQuery Cost Control: <https://cloud.google.com/bigquery/docs/cost-control-guide>

IAM for BigQuery: <https://cloud.google.com/bigquery/docs/access-control>

Question: 76

CertyIQ

Government regulations in your industry mandate that you have to maintain an auditable record of access to certain types of data. Assuming that all expiring logs will be archived correctly, where should you store data that is subject to that mandate?

- A. Encrypted on Cloud Storage with user-supplied encryption keys. A separate decryption key will be given to each authorized user.
- B. In a BigQuery dataset that is viewable only by authorized personnel, with the Data Access log used to provide the auditability.
- C. In Cloud SQL, with separate database user names to each user. The Cloud SQL Admin activity logs will be used to provide the auditability.
- D. In a bucket on Cloud Storage that is accessible only by an AppEngine service that collects user information and logs the access before providing a link to the bucket.

Answer: B

Explanation:

The most suitable option for storing auditable data, as mandated by regulations, is **B. In a BigQuery dataset that is viewable only by authorized personnel, with the Data Access log used to provide the auditability.**

Here's why: BigQuery is designed for analytical workloads and provides robust access control and audit logging features crucial for compliance. By restricting access to the dataset to only authorized personnel, you ensure data confidentiality. Furthermore, BigQuery's Data Access logs track who accessed what data and when, fulfilling the auditable record requirement. These logs are detailed, readily available, and designed

specifically for compliance and security auditing. Options A, C, and D fall short. While A offers encryption, it lacks the built-in audit logging necessary for compliance. Option C relies on Cloud SQL logs, which are less granular and geared towards administrative activities rather than data access auditing. Option D introduces unnecessary complexity by requiring an intermediate App Engine service, and the logging might not be as comprehensive as BigQuery's Data Access logs. BigQuery offers granular permission management using Identity and Access Management (IAM), ensuring only intended users can access specific parts of the data, enhancing control and accountability. This approach aligns with best practices for data governance and compliance in regulated industries.

Authoritative Links for further research:

BigQuery Security Overview: <https://cloud.google.com/bigquery/docs/security-overview>

BigQuery Audit Logging: <https://cloud.google.com/bigquery/docs/audit-logs>

Cloud IAM: <https://cloud.google.com/iam/docs>

Data Access Logs: <https://cloud.google.com/logging/docs/audit/data-access>

Question: 77

CertyIQ

Your neural network model is taking days to train. You want to increase the training speed. What can you do?

- A. Subsample your test dataset.
- B. Subsample your training dataset.
- C. Increase the number of input features to your model.
- D. Increase the number of layers in your neural network.

Answer: B

Explanation:

The correct answer is **B. Subsample your training dataset**. Here's a detailed justification:

Training a neural network, especially a large one, is computationally intensive. A major factor affecting training time is the size of the training dataset. A larger dataset means more data points to process during each training iteration (epoch), leading to longer training times. By subsampling, or reducing the size of the training dataset, we reduce the amount of computation needed per epoch. This results in faster training, although it might affect the model's generalization performance.

Subsampling doesn't directly impact the complexity of the model itself (number of layers or features). Options C and D, increasing input features or layers, would increase model complexity and increase training time. A test dataset is used for evaluation, not for training, so A would not affect training speed. Subsampling should be done carefully as a drastic reduction can lead to underfitting. It is a common technique used when facing training speed constraints. One might have to fine-tune the size of the subsampled dataset for the best trade-off between training speed and model quality.

Cloud platforms like Google Cloud Platform (GCP) often provide tools for data preprocessing and sampling, along with resources for parallelizing training on GPUs or TPUs, which will further reduce training time. Utilizing a managed service like Vertex AI will also manage the scaling of resources for you.

For further research on data sampling and model training on GCP:

[Google Cloud Vertex AI: Data Sampling](#)

[Google Cloud Vertex AI: Model Training](#)

[Google Cloud AI Platform: Training Overview](#)

Question: 78

You are responsible for writing your company's ETL pipelines to run on an Apache Hadoop cluster. The pipeline will require some checkpointing and splitting pipelines. Which method should you use to write the pipelines?

- A. PigLatin using Pig
- B. HiveQL using Hive
- C. Java using MapReduce
- D. Python using MapReduce

Answer: A

Explanation:

The best choice for writing ETL pipelines requiring checkpointing and splitting on an Apache Hadoop cluster is **A. PigLatin using Pig**. Pig is a high-level data flow language specifically designed for parallel data processing on Hadoop. Its scripting nature simplifies complex transformations, while its built-in operators and UDF (User Defined Function) support allow for flexible checkpointing by saving intermediate data sets and splitting workflows through logical branching based on filtering results. These functionalities facilitate easier pipeline management, recovery, and more granular control over processing stages compared to the other options. While Hive (B) is suitable for SQL-like queries on structured data and MapReduce (C, D) provides low-level control, Pig's data flow approach and built-in operators better align with the specific needs of a pipeline requiring checkpointing and splitting. Additionally, Pig's relatively simple scripting compared to Java in MapReduce can reduce the development and debugging time.

Authoritative Resources:

Apache Pig Documentation: <https://pig.apache.org/docs/r0.17.0/> - This is the official documentation for Apache Pig and can provide deeper insights into its features, data processing capabilities, and functionalities.

Hadoop: The Definitive Guide by Tom White: This book is an authoritative source on Hadoop and includes a detailed discussion of Pig's functionalities and its applications in ETL pipelines.

Question: 79

Your company maintains a hybrid deployment with GCP, where analytics are performed on your anonymized customer data. The data are imported to Cloud Storage from your data center through parallel uploads to a data transfer server running on GCP. Management informs you that the daily transfers take too long and have asked you to fix the problem. You want to maximize transfer speeds. Which action should you take?

- A. Increase the CPU size on your server.
- B. Increase the size of the Google Persistent Disk on your server.
- C. Increase your network bandwidth from your datacenter to GCP.
- D. Increase your network bandwidth from Compute Engine to Cloud Storage.

Answer: C

Explanation:

The correct answer is **C. Increase your network bandwidth from your datacenter to GCP**. Here's why: The bottleneck in this scenario is the speed at which data moves from the on-premises data center to Google Cloud Storage (GCS). The problem statement explicitly states that transfers are taking too long, indicating a limitation in the data ingestion pipeline. Since the data is being uploaded in parallel through a transfer server on GCP, the server's resources are less likely to be the primary constraint. Options A and B focus on increasing server resources, which address processing or storage bottlenecks. The key issue is the network

connection between the on-premises data center and GCP. Increasing the CPU size (A) would improve processing power but does not impact data transfer speeds from the source. Similarly, increasing the disk size (B) only changes storage capacity on the server and does not improve network throughput. Option D, while related to networking, addresses the connection between the transfer server and GCS, which is not where the primary bottleneck is occurring. The initial data transfer is from the data center to the transfer server. Therefore, the critical factor for improving transfer speed is the available network bandwidth connecting the on-premises network to Google's network. Increasing this bandwidth will allow for more data to be transferred simultaneously, reducing the overall time taken. This aligns with the fundamental principles of network performance, where bandwidth is the key limiting factor for data transfer speeds.

Authoritative Links:

Google Cloud Networking Overview: <https://cloud.google.com/network-connectivity/docs/overview> - This documentation provides a broad overview of Google Cloud networking, highlighting the importance of bandwidth and network capacity.

Hybrid Connectivity: <https://cloud.google.com/hybrid-connectivity/docs> - This resource specifically discusses options for connecting on-premises environments to Google Cloud, emphasizing the role of bandwidth.

Optimizing Storage Transfers : <https://cloud.google.com/storage/docs/optimizing-transfers> - This doc highlights best practices for optimizing the transfer of data to GCS, which includes adequate network bandwidth.

Question: 80

CertyIQ

MJTelco Case Study -

Company Overview -

MJTelco is a startup that plans to build networks in rapidly growing, underserved markets around the world. The company has patents for innovative optical communications hardware. Based on these patents, they can create many reliable, high-speed backbone links with inexpensive hardware.

Company Background -

Founded by experienced telecom executives, MJTelco uses technologies originally developed to overcome communications challenges in space. Fundamental to their operation, they need to create a distributed data infrastructure that drives real-time analysis and incorporates machine learning to continuously optimize their topologies. Because their hardware is inexpensive, they plan to overdeploy the network allowing them to account for the impact of dynamic regional politics on location availability and cost.

Their management and operations teams are situated all around the globe creating many-to-many relationship between data consumers and provides in their system. After careful consideration, they decided public cloud is the perfect environment to support their needs.

Solution Concept -

MJTelco is running a successful proof-of-concept (PoC) project in its labs. They have two primary needs:

- Scale and harden their PoC to support significantly more data flows generated when they ramp to more than 50,000 installations.
- Refine their machine-learning cycles to verify and improve the dynamic models they use to control topology definition.

MJTelco will also use three separate operating environments " development/test, staging, and production " to meet the needs of running experiments, deploying new features, and serving production customers.

Business Requirements -

- Scale up their production environment with minimal cost, instantiating resources when and where needed in an unpredictable, distributed telecom user community.
- Ensure security of their proprietary data to protect their leading-edge machine learning and analysis.
- Provide reliable and timely access to data for analysis from distributed research workers
- Maintain isolated environments that support rapid iteration of their machine-learning models without affecting their customers.

Technical Requirements -

Ensure secure and efficient transport and storage of telemetry data

Rapidly scale instances to support between 10,000 and 100,000 data providers with multiple flows each.

Allow analysis and presentation against data tables tracking up to 2 years of data storing approximately 100m records/day

Support rapid iteration of monitoring infrastructure focused on awareness of data pipeline problems both in telemetry flows and in production learning cycles.

CEO Statement -

Our business model relies on our patents, analytics and dynamic machine learning. Our inexpensive hardware is organized to be highly reliable, which gives us cost advantages. We need to quickly stabilize our large distributed data pipelines to meet our reliability and capacity commitments.

CTO Statement -

Our public cloud services must operate as advertised. We need resources that scale and keep our data secure. We also need environments in which our data scientists can carefully study and quickly adapt our models. Because we rely on automation to process our data, we also need our development and test environments to work as we iterate.

CFO Statement -

The project is too large for us to maintain the hardware and software required for the data and analysis. Also, we cannot afford to staff an operations team to monitor so many data feeds, so we will rely on automation and infrastructure. Google Cloud's machine learning will allow our quantitative researchers to work on our high-value problems instead of problems with our data pipelines.

MJTelco is building a custom interface to share data. They have these requirements:

1. They need to do aggregations over their petabyte-scale datasets.
2. They need to scan specific time range rows with a very fast response time (milliseconds).

Which combination of Google Cloud Platform products should you recommend?

A. Cloud Datastore and Cloud Bigtable

B. Cloud Bigtable and Cloud SQL

C. BigQuery and Cloud Bigtable

D. BigQuery and Cloud Storage

Answer: C

Explanation:

The correct answer is **C. BigQuery and Cloud Bigtable**. Here's why:

MJTelco has two primary data access needs: large-scale aggregations and fast, time-series-based retrieval. BigQuery is Google Cloud's fully-managed, serverless data warehouse, perfectly suited for running complex analytical queries on petabyte-scale datasets. It excels at aggregations and offers the ability to perform analysis on historical data, making it ideal for requirement 1. On the other hand, Cloud Bigtable is a NoSQL wide-column database ideal for storing and quickly retrieving large amounts of time-series data with low latency. This satisfies requirement 2, providing the required milliseconds response time for scanning specific time ranges.

Cloud Datastore (option A) is suitable for transactional data and not for large-scale aggregations. Cloud SQL (option B) is a relational database, which isn't optimal for high-throughput time-series data and struggles to match Bigtable's scale and speed for time-based queries. Cloud Storage (option D) provides object storage, not suited for the fast querying of specific rows in the required milliseconds as specified for use case #2. BigQuery on the other hand excels at large-scale aggregations (use case #1) and can manage petabyte-scale data as described. Thus, the combination of BigQuery and Cloud Bigtable efficiently addresses MJTelco's distinct data requirements. The choice between Bigtable and alternatives like Datastore revolves around the required read speed which big table delivers far better.

Authoritative Links:

BigQuery: <https://cloud.google.com/bigquery/docs>

Cloud Bigtable: <https://cloud.google.com/bigtable/docs>

Cloud Datastore: <https://cloud.google.com/datastore/docs>

Cloud SQL: <https://cloud.google.com/sql/docs>

Question: 81

MJTelco Case Study -

Company Overview -

MJTelco is a startup that plans to build networks in rapidly growing, underserved markets around the world. The company has patents for innovative optical communications hardware. Based on these patents, they can create many reliable, high-speed backbone links with inexpensive hardware.

Company Background -

Founded by experienced telecom executives, MJTelco uses technologies originally developed to overcome communications challenges in space. Fundamental to their operation, they need to create a distributed data infrastructure that drives real-time analysis and incorporates machine learning to continuously optimize their topologies. Because their hardware is inexpensive, they plan to overdeploy the network allowing them to account for the impact of dynamic regional politics on location availability and cost.

Their management and operations teams are situated all around the globe creating many-to-many relationship between data consumers and provides in their system. After careful consideration, they decided public cloud is the perfect environment to support their needs.

Solution Concept -

MJTelco is running a successful proof-of-concept (PoC) project in its labs. They have two primary needs:

- Scale and harden their PoC to support significantly more data flows generated when they ramp to more than 50,000 installations.

- Refine their machine-learning cycles to verify and improve the dynamic models they use to control topology definition.

MJTelco will also use three separate operating environments " development/test, staging, and production " to meet the needs of running experiments, deploying new features, and serving production customers.

Business Requirements -

- Scale up their production environment with minimal cost, instantiating resources when and where needed in an unpredictable, distributed telecom user community.

- Ensure security of their proprietary data to protect their leading-edge machine learning and analysis.

- Provide reliable and timely access to data for analysis from distributed research workers

- Maintain isolated environments that support rapid iteration of their machine-learning models without affecting their customers.

Technical Requirements -

Ensure secure and efficient transport and storage of telemetry data

Rapidly scale instances to support between 10,000 and 100,000 data providers with multiple flows each.

Allow analysis and presentation against data tables tracking up to 2 years of data storing approximately 100m records/day

Support rapid iteration of monitoring infrastructure focused on awareness of data pipeline problems both in telemetry flows and in production learning cycles.

CEO Statement -

Our business model relies on our patents, analytics and dynamic machine learning. Our inexpensive hardware is organized to be highly reliable, which gives us cost advantages. We need to quickly stabilize our large distributed data pipelines to meet our reliability and capacity commitments.

CTO Statement -

Our public cloud services must operate as advertised. We need resources that scale and keep our data secure. We also need environments in which our data scientists can carefully study and quickly adapt our models. Because we rely on automation to process our data, we also need our development and test environments to work as we iterate.

CFO Statement -

The project is too large for us to maintain the hardware and software required for the data and analysis. Also, we cannot afford to staff an operations team to monitor so many data feeds, so we will rely on automation and infrastructure. Google Cloud's machine learning will allow our quantitative researchers to work on our high-value problems instead of problems with our data pipelines.

You need to compose visualization for operations teams with the following requirements:

- ⇒ Telemetry must include data from all 50,000 installations for the most recent 6 weeks (sampling once every minute)
 - ⇒ The report must not be more than 3 hours delayed from live data.
 - ⇒ The actionable report should only show suboptimal links.
 - ⇒ Most suboptimal links should be sorted to the top.
- Suboptimal links can be grouped and filtered by regional geography.

-
- ⇒ User response time to load the report must be <5 seconds.

You create a data source to store the last 6 weeks of data, and create visualizations that allow viewers to see multiple date ranges, distinct geographic regions, and unique installation types. You always show the latest data without any changes to your visualizations. You want to avoid creating and updating new visualizations each month. What should you do?

- A. Look through the current data and compose a series of charts and tables, one for each possible combination of criteria.
- B. Look through the current data and compose a small set of generalized charts and tables bound to criteria filters that allow value selection.
- C. Export the data to a spreadsheet, compose a series of charts and tables, one for each possible combination of criteria, and spread them across multiple tabs.
- D. Load the data into relational database tables, write a Google App Engine application that queries all rows, summarizes the data across each criteria, and then renders results using the Google Charts and visualization API.

Answer: B

Explanation:

B. Look through the current data and compose a small set of generalized charts and tables bound to criteria filters that allow value selection.

Generalized Charts and Tables: Creating a few core visualizations that are designed to work with various data combinations (date, region, installation type) is highly efficient and scalable. This avoids the need to build and maintain a new chart for every possible permutation. This directly addresses the goal of avoiding monthly updates.

Criteria Filters: Using filters or parameters bound to the charts allows users to dynamically select the date range, region, and installation type they want to see. This provides the flexibility and interactivity required without changing the underlying visualization structure. This also contributes to the goal of avoiding monthly updates.

Performance: A well-designed dashboard with filters is significantly faster and more performant than loading a massive number of pre-rendered charts. A small set of charts will load quickly, and filtering the data is typically an efficient operation, helping to meet the <5 second response time goal.

Why other options are incorrect:

A. Look through the current data and compose a series of charts and tables, one for each possible combination of criteria: This would create an unmanageable number of visualizations. With three criteria (date, region, installation type), each with multiple values, the number of combinations would be enormous, making the report slow to load and difficult to maintain. This violates the avoid creating and updating new visualizations each month and the <5 second response time goals.

C. Export the data to a spreadsheet, compose a series of charts and tables, one for each possible combination of criteria, and spread them across multiple tabs: Similar to option A, this is an unscalable approach. Spreadsheets are not designed for handling the complexity of multiple data dimensions and would become extremely cumbersome and slow. It would also require constant manual updates, violating the maintenance goal.

D. Load the data into relational database tables, write a Google App Engine application that queries all rows, summarizes the data across each criteria, and then renders results using the Google Charts and visualization

API: While this is a technically viable approach, it is an overly complex and costly solution for this problem. Building a custom application is far more work than using a modern business intelligence tool (like Google Data Studio, which is implied by the Google Charts API reference) that already has built-in filtering and visualization capabilities. This solution is overkill and not the most efficient way to achieve the stated goals.

Question: 82

CertyIQ

MJTelco Case Study -

Company Overview -

MJTelco is a startup that plans to build networks in rapidly growing, underserved markets around the world. The company has patents for innovative optical communications hardware. Based on these patents, they can create many reliable, high-speed backbone links with inexpensive hardware.

Company Background -

Founded by experienced telecom executives, MJTelco uses technologies originally developed to overcome communications challenges in space. Fundamental to their operation, they need to create a distributed data infrastructure that drives real-time analysis and incorporates machine learning to continuously optimize their topologies. Because their hardware is inexpensive, they plan to overdeploy the network allowing them to account for the impact of dynamic regional politics on location availability and cost.

Their management and operations teams are situated all around the globe creating many-to-many relationship between data consumers and provides in their system. After careful consideration, they decided public cloud is the perfect environment to support their needs.

Solution Concept -

MJTelco is running a successful proof-of-concept (PoC) project in its labs. They have two primary needs:

- Scale and harden their PoC to support significantly more data flows generated when they ramp to more than 50,000 installations.
- Refine their machine-learning cycles to verify and improve the dynamic models they use to control topology definition.

MJTelco will also use three separate operating environments "development/test, staging, and production" to meet the needs of running experiments, deploying new features, and serving production customers.

Business Requirements -

- Scale up their production environment with minimal cost, instantiating resources when and where needed in an unpredictable, distributed telecom user community.
- Ensure security of their proprietary data to protect their leading-edge machine learning and analysis.
- Provide reliable and timely access to data for analysis from distributed research workers
- Maintain isolated environments that support rapid iteration of their machine-learning models without affecting their customers.

Technical Requirements -

Ensure secure and efficient transport and storage of telemetry data

Rapidly scale instances to support between 10,000 and 100,000 data providers with multiple flows each.

Allow analysis and presentation against data tables tracking up to 2 years of data storing approximately 100m records/day

Support rapid iteration of monitoring infrastructure focused on awareness of data pipeline problems both in telemetry flows and in production learning cycles.

CEO Statement -

Our business model relies on our patents, analytics and dynamic machine learning. Our inexpensive hardware is organized to be highly reliable, which gives us cost advantages. We need to quickly stabilize our large distributed data pipelines to meet our reliability and capacity commitments.

CTO Statement -

Our public cloud services must operate as advertised. We need resources that scale and keep our data secure. We also need environments in which our data scientists can carefully study and quickly adapt our models. Because we rely on automation to process our data, we also need our development and test environments to work as we iterate.

CFO Statement -

The project is too large for us to maintain the hardware and software required for the data and analysis. Also, we

cannot afford to staff an operations team to monitor so many data feeds, so we will rely on automation and infrastructure. Google Cloud's machine learning will allow our quantitative researchers to work on our high-value problems instead of problems with our data pipelines. Given the record streams MJTelco is interested in ingesting per day, they are concerned about the cost of Google BigQuery increasing. MJTelco asks you to provide a design solution. They require a single large data table called `tracking_table`. Additionally, they want to minimize the cost of daily queries while performing fine-grained analysis of each day's events. They also want to use streaming ingestion. What should you do?

- A. Create a table called `tracking_table` and include a `DATE` column.
- B. Create a partitioned table called `tracking_table` and include a `TIMESTAMP` column.
- C. Create sharded tables for each day following the pattern `tracking_table_YYYYMMDD`.
- D. Create a table called `tracking_table` with a `TIMESTAMP` column to represent the day.

Answer: B

Explanation:

Here's a detailed justification for choosing option B:

Why Option B is the best solution:

Option B, creating a partitioned table called `tracking_table` with a `TIMESTAMP` column, is the most efficient and cost-effective approach for MJTelco's requirements. BigQuery partitioned tables excel at handling large datasets where queries target specific subsets of the data. By partitioning the table by a `TIMESTAMP` column, ideally representing the date of each event, MJTelco can achieve significant cost savings. This partitioning allows BigQuery to only scan the relevant partitions of the data when a query filters on date ranges, thus avoiding unnecessary full-table scans. This directly addresses MJTelco's concern about increasing BigQuery costs by minimizing data scanned in daily fine-grained analyses. Streaming ingestion of data into partitioned tables is also supported.

Why other options are less suitable:

Option A: Creating a table with a `DATE` column but without partitioning will result in full-table scans for queries targeting specific dates or ranges, incurring higher costs and slower query times. This approach does not provide cost advantages.

Option C: Sharding tables by date (e.g., `tracking_table_YYYYMMDD`) leads to management complexity. It requires managing multiple table schemas, querying multiple tables with `UNION ALL`, and significantly increases data complexity, which is not aligned with the prompt.

Option D: Creating a non-partitioned table with a `TIMESTAMP` column won't offer the same performance and cost benefits as a partitioned table when querying by date ranges.

Key Concepts & Cloud Computing Principles:

Partitioning: Partitioning is a technique for dividing a large table into smaller, more manageable segments based on a specific column. In BigQuery, it drastically improves query performance and reduces costs by limiting the data that needs to be scanned.

Data Ingestion: Streaming ingestion allows for continuous data ingestion into BigQuery without the need for batch processing.

Cost Optimization: Partitioning is a key strategy to reduce costs in BigQuery because it avoids full table scans.

Fine-Grained Analysis: Partitioning is an important aspect when performing analysis by date because it will scan only the relevant partition instead of the entire dataset.

Authoritative Links for Further Research:

BigQuery Partitioned Tables: <https://cloud.google.com/bigquery/docs/partitioned-tables>

BigQuery Streaming Ingestion: <https://cloud.google.com/bigquery/streaming-data-into-bigquery>

In summary, partitioning tracking_table by TIMESTAMP provides an efficient, scalable, and cost-effective solution that aligns with MJTelco's requirements for data ingestion, analysis, and cost optimization in BigQuery.

Question: 83

CertyIQ

Flowlogistic Case Study -

Company Overview -

Flowlogistic is a leading logistics and supply chain provider. They help businesses throughout the world manage their resources and transport them to their final destination. The company has grown rapidly, expanding their offerings to include rail, truck, aircraft, and oceanic shipping.

Company Background -

The company started as a regional trucking company, and then expanded into other logistics market. Because they have not updated their infrastructure, managing and tracking orders and shipments has become a bottleneck. To improve operations, Flowlogistic developed proprietary technology for tracking shipments in real time at the parcel level. However, they are unable to deploy it because their technology stack, based on Apache Kafka, cannot support the processing volume. In addition, Flowlogistic wants to further analyze their orders and shipments to determine how best to deploy their resources.

Solution Concept -

Flowlogistic wants to implement two concepts using the cloud:

- ⇒ Use their proprietary technology in a real-time inventory-tracking system that indicates the location of their loads
- ⇒ Perform analytics on all their orders and shipment logs, which contain both structured and unstructured data, to determine how best to deploy resources, which markets to expand into. They also want to use predictive analytics to learn earlier when a shipment will be delayed.

Existing Technical Environment -

Flowlogistic architecture resides in a single data center:

- ⇒ Databases
 - 8 physical servers in 2 clusters
 - SQL Server "" user data, inventory, static data
 - 3 physical servers
 - Cassandra "" metadata, tracking messages
- 10 Kafka servers "" tracking message aggregation and batch insert
- ⇒ Application servers "" customer front end, middleware for order/customs
 - 60 virtual machines across 20 physical servers
 - Tomcat "" Java services
 - Nginx "" static content
 - Batch servers
- ⇒ Storage appliances
 - iSCSI for virtual machine (VM) hosts
 - Fibre Channel storage area network (FC SAN) "" SQL server storage
- Network-attached storage (NAS) image storage, logs, backups
 - ⇒ 10 Apache Hadoop /Spark servers
 - Core Data Lake
 - Data analysis workloads
 - ⇒ 20 miscellaneous servers
 - Jenkins, monitoring, bastion hosts,

Business Requirements -

- ⇒ Build a reliable and reproducible environment with scaled parity of production.
- ⇒ Aggregate data in a centralized Data Lake for analysis
- ⇒ Use historical data to perform predictive analytics on future shipments
- ⇒ Accurately track every shipment worldwide using proprietary technology
- ⇒ Improve business agility and speed of innovation through rapid provisioning of new resources
- ⇒ Analyze and optimize architecture for performance in the cloud
- ⇒ Migrate fully to the cloud if all other requirements are met

Technical Requirements -

- Handle both streaming and batch data
- Migrate existing Hadoop workloads
- Ensure architecture is scalable and elastic to meet the changing demands of the company.
- Use managed services whenever possible
- Encrypt data flight and at rest

Connect a VPN between the production data center and cloud environment

SEO Statement -

We have grown so quickly that our inability to upgrade our infrastructure is really hampering further growth and efficiency. We are efficient at moving shipments around the world, but we are inefficient at moving data around. We need to organize our information so we can more easily understand where our customers are and what they are shipping.

CTO Statement -

IT has never been a priority for us, so as our data has grown, we have not invested enough in our technology. I have a good staff to manage IT, but they are so busy managing our infrastructure that I cannot get them to do the things that really matter, such as organizing our data, building the analytics, and figuring out how to implement the CFO's tracking technology.

CFO Statement -

Part of our competitive advantage is that we penalize ourselves for late shipments and deliveries. Knowing where our shipments are at all times has a direct correlation to our bottom line and profitability. Additionally, I don't want to commit capital to building out a server environment.

Flowlogistic's management has determined that the current Apache Kafka servers cannot handle the data volume for their real-time inventory tracking system.

You need to build a new system on Google Cloud Platform (GCP) that will feed the proprietary tracking software. The system must be able to ingest data from a variety of global sources, process and query in real-time, and store the data reliably. Which combination of GCP products should you choose?

- A. Cloud Pub/Sub, Cloud Dataflow, and Cloud Storage
- B. Cloud Pub/Sub, Cloud Dataflow, and Local SSD
- C. Cloud Pub/Sub, Cloud SQL, and Cloud Storage
- D. Cloud Load Balancing, Cloud Dataflow, and Cloud Storage
- E. Cloud Dataflow, Cloud SQL, and Cloud Storage

Answer: A

Explanation:

The correct answer is A: Cloud Pub/Sub, Cloud Dataflow, and Cloud Storage. Here's why:

Flowlogistic needs a real-time system to ingest, process, and store tracking data from various global sources.

Cloud Pub/Sub is the ideal choice for this, as it's a globally scalable messaging service that can reliably ingest streaming data from numerous sources. This addresses the need to handle data from a variety of locations. <https://cloud.google.com/pubsub/docs/overview>

Next, the data needs to be processed in real-time. **Cloud Dataflow** is a fully managed, serverless stream and batch processing service that's perfect for transforming, analyzing, and enriching the incoming tracking data from Pub/Sub. It provides the capabilities to process and query the data in real-time.

<https://cloud.google.com/dataflow/docs/concepts/dataflow-model>

Finally, the processed data must be reliably stored. **Cloud Storage** is a scalable, durable, and cost-effective object storage solution suitable for storing vast amounts of tracking data over time. It ensures the data is reliably stored for future use and analysis. <https://cloud.google.com/storage/docs/concepts/>

Options B, C, D, and E are not ideal because:

B: Local SSD is ephemeral and not suitable for reliable data storage.

C: Cloud SQL is a relational database and not optimized for real-time stream processing and large-scale

unstructured data.

D: Cloud Load Balancing is used for distributing traffic and not for stream ingestion or data processing.

E: Cloud SQL is not suitable for storing real time messages.

Therefore, Cloud Pub/Sub, Cloud Dataflow, and Cloud Storage are the best combination to address Flowlogistic's requirements for ingesting, processing, and storing their real-time shipment tracking data. This aligns with the need to build a scalable, reliable, and managed solution on GCP.

Question: 84

CertyIQ

After migrating ETL jobs to run on BigQuery, you need to verify that the output of the migrated jobs is the same as the output of the original. You've loaded a table containing the output of the original job and want to compare the contents with output from the migrated job to show that they are identical. The tables do not contain a primary key column that would enable you to join them together for comparison.

What should you do?

- A. Select random samples from the tables using the RAND() function and compare the samples.
- B. Select random samples from the tables using the HASH() function and compare the samples.
- C. Use a Dataproc cluster and the BigQuery Hadoop connector to read the data from each table and calculate a hash from non-timestamp columns of the table after sorting. Compare the hashes of each table.
- D. Create stratified random samples using the OVER() function and compare equivalent samples from each table.

Answer: C

Explanation:

The correct approach is **C: Use a Dataproc cluster and the BigQuery Hadoop connector to read the data from each table and calculate a hash from non-timestamp columns of the table after sorting. Compare the hashes of each table.** This method provides a robust way to verify data equivalence without relying on a primary key.

Here's the justification:

Handling Large Datasets: Migrated data, especially in ETL, often involves substantial volume. Dataproc clusters are designed for processing massive datasets using distributed computing frameworks like Hadoop and Spark, making it suitable for this scenario.

BigQuery Hadoop Connector: This connector allows Dataproc clusters to directly interact with BigQuery tables, enabling efficient data retrieval for processing. This is crucial for large table comparisons.

Hash-Based Comparison: Calculating a hash over the relevant columns (excluding timestamps to avoid comparison discrepancies) provides a concise and verifiable representation of the data. If the hashes match, it strongly suggests that the content of those columns across the two tables are identical. Sorting the rows before hashing ensures that the order of the records does not impact the hash result, allowing comparison irrespective of data insertion/ordering differences between the original and migrated tables.

Comprehensive Comparison: Option C compares the entire datasets, rather than just random samples as suggested in options A, B and D, giving high assurance that they are identical. Options A and B could potentially miss differences if samples happen to be the same while not reflecting the whole datasets. Stratified random samples as suggested in D could be complex to implement and still miss differences. Option C is more straightforward and ensures complete data comparison.

Why other options are not suitable:

A & B (Random Sampling): Comparing samples may reveal some discrepancies but cannot guarantee complete data equivalence. The samples could by chance be identical even if the whole datasets are different.

Additionally, hashing a sample (B) doesn't help more than comparing directly (A), and would not be a robust method to verify dataset equality.

D (Stratified Random Samples): Stratified sampling is a useful technique to ensure the representation of population across different strata, however, this option will only provide a limited view of the complete datasets and does not verify if the datasets are entirely the same. Stratification and comparison are complex to set up in this scenario.

Authoritative links for further research:

Dataproc: <https://cloud.google.com/dataproc>

BigQuery Hadoop Connector: <https://cloud.google.com/dataproc/docs/concepts/connectors/bigquery>

Hashing: https://en.wikipedia.org/wiki/Hash_function

Data Validation and Testing: <https://cloud.google.com/architecture/data-validation-testing>

In summary, option C offers a practical, scalable, and reliable method to compare two large tables without a primary key, making it the optimal solution for this data migration verification scenario.

Question: 85

CertyIQ

You are a head of BI at a large enterprise company with multiple business units that each have different priorities and budgets. You use on-demand pricing for

BigQuery with a quota of 2K concurrent on-demand slots per project. Users at your organization sometimes don't get slots to execute their query and you need to correct this. You'd like to avoid introducing new projects to your account.

What should you do?

- A. Convert your batch BQ queries into interactive BQ queries.
- B. Create an additional project to overcome the 2K on-demand per-project quota.
- C. Switch to flat-rate pricing and establish a hierarchical priority model for your projects.
- D. Increase the amount of concurrent slots per project at the Quotas page at the Cloud Console.

Answer: C

Explanation:

The correct answer is C, switching to flat-rate pricing and establishing a hierarchical priority model for projects. Here's why:

The problem highlights resource contention with BigQuery's on-demand pricing. When users can't get slots, it indicates the 2,000 on-demand slot limit is being reached. The goal is to provide consistent access without creating new projects.

Option A is incorrect because converting batch queries to interactive queries would exacerbate the slot contention problem, as interactive queries are typically prioritized and consume slots faster.

Option B is undesirable as it explicitly states to avoid new projects. The problem statement is that the business units are on different budgets and this is a large enterprise, implying that managing multiple projects and the cost allocation across those projects would be costly, require administrative overhead, and complex.

Option D is incorrect because it's not possible to increase the on-demand slot quota per project. The 2,000 slot limit is a hard limit enforced by Google Cloud.

Option C directly addresses the problem. Flat-rate pricing allows you to purchase a dedicated number of slots, eliminating the contention inherent in on-demand pricing. Establishing a hierarchical priority model

enables different business units to receive prioritized access to the purchased slots based on their business needs and budgets. This ensures critical queries from high-priority units are processed before less urgent ones from other units. This allows you to define which projects get priority, providing a mechanism to manage resources according to different business unit priorities.

Flat-rate pricing gives predictable costs. Hierarchical prioritization allows for optimized resource allocation. This approach avoids creating new projects and effectively manages BigQuery resource allocation across different business units with varying priorities.

Further Reading:

[BigQuery pricing](#)

[BigQuery Reservations](#)

Question: 86

CertyIQ

You have an Apache Kafka cluster on-prem with topics containing web application logs. You need to replicate the data to Google Cloud for analysis in BigQuery and Cloud Storage. The preferred replication method is mirroring to avoid deployment of Kafka Connect plugins.

What should you do?

- A. Deploy a Kafka cluster on GCE VM Instances. Configure your on-prem cluster to mirror your topics to the cluster running in GCE. Use a Dataproc cluster or Dataflow job to read from Kafka and write to GCS.
- B. Deploy a Kafka cluster on GCE VM Instances with the Pub/Sub Kafka connector configured as a Sink connector. Use a Dataproc cluster or Dataflow job to read from Kafka and write to GCS.
- C. Deploy the Pub/Sub Kafka connector to your on-prem Kafka cluster and configure Pub/Sub as a Source connector. Use a Dataflow job to read from Pub/Sub and write to GCS.
- D. Deploy the Pub/Sub Kafka connector to your on-prem Kafka cluster and configure Pub/Sub as a Sink connector. Use a Dataflow job to read from Pub/Sub and write to GCS.

Answer: A

Explanation:

The correct answer is A. Here's why:

Requirement: The problem states mirroring is the preferred replication method to avoid Kafka Connect plugins. Mirroring allows you to replicate topics between Kafka clusters.

On-prem to GCP: To mirror your on-prem Kafka data to GCP, you need a Kafka cluster in GCP. Deploying a Kafka cluster on GCE VM Instances satisfies this.

Mirroring Configuration: Mirroring is configured between the on-prem and the GCE-hosted Kafka cluster. The on-prem cluster will mirror topics to the GCE cluster.

Data Transfer to GCS/BigQuery: After mirroring, the logs are available in the GCE-hosted Kafka cluster. A Dataproc cluster or Dataflow job can then read data from the Kafka cluster in GCP and write it to Cloud Storage and/or BigQuery. Dataproc is optimized for batch processing while Dataflow is better for streaming.

The other options are less suitable:

B: This option incorrectly suggests using the Pub/Sub Kafka connector while also proposing Dataflow/Dataproc to read data from Kafka and write it to GCS. This adds unnecessary complexity.

C and D: These options involve deploying the Pub/Sub Kafka connector on-prem. While it's a viable method for moving data to GCP, they don't align with the stated preference for mirroring to avoid the deployment of Kafka Connect plugins initially.

In summary, setting up a Kafka cluster on GCE and using mirroring combined with Dataproc or Dataflow is the most straightforward and cost-effective way to replicate the data while adhering to the mirroring preference.

Relevant links:

Mirroring Kafka: <https://cwiki.apache.org/confluence/display/KAFKA/KIP-605%3A+Kafka+MirrorMaker2.0>

Dataprocc: <https://cloud.google.com/dataprocc>

Dataflow: <https://cloud.google.com/dataflow>

Question: 87

CertyIQ

You've migrated a Hadoop job from an on-prem cluster to dataprocc and GCS. Your Spark job is a complicated analytical workload that consists of many shuffling operations and initial data are parquet files (on average 200-400 MB size each). You see some degradation in performance after the migration to Dataprocc, so you'd like to optimize for it. You need to keep in mind that your organization is very cost-sensitive, so you'd like to continue using Dataprocc on preemptibles (with 2 non-preemptible workers only) for this workload.

What should you do?

- A. Increase the size of your parquet files to ensure them to be 1 GB minimum.
- B. Switch to TFRecords formats (appr. 200MB per file) instead of parquet files.
- C. Switch from HDDs to SSDs, copy initial data from GCS to HDFS, run the Spark job and copy results back to GCS.
- D. Switch from HDDs to SSDs, override the preemptible VMs configuration to increase the boot disk size.

Answer: D

Explanation:

The correct answer is **D. Switch from HDDs to SSDs, override the preemptible VMs configuration to increase the boot disk size.**

Here's why:

The problem describes a Spark job with heavy shuffling on Dataprocc using preemptible VMs and Parquet files. Performance degradation is observed after migration from an on-premise Hadoop cluster. The organization is cost-sensitive. The key bottlenecks likely involve I/O operations during shuffling and limited disk space. Preemptible VMs, while cost-effective, have a risk of termination, which can increase the shuffling overhead if shuffle data is lost.

SSDs vs. HDDs: Switching from HDDs to SSDs directly addresses I/O performance bottlenecks. Shuffling involves writing and reading intermediate data to disk. SSDs offer significantly faster read/write speeds, crucial for improving shuffle performance. <https://cloud.google.com/compute/docs/disks>

Boot Disk Size: Increasing the boot disk size, especially when using preemptible VMs, provides more local storage for shuffle data. Spark's shuffle operations benefit from having ample local disk space, reducing the need to constantly access slower external storage like GCS. Overriding the default boot disk configuration allows you to provision adequate space for the shuffle data. Without sufficient space, shuffle data spills to slower storage, greatly impacting performance.

Preemptible Considerations: While GCS is durable, frequent access during shuffling adds latency. Configuring larger SSD-backed boot disks enables a larger local cache for shuffle data, minimizing the impact of preemptible VM terminations.

Why other options are less suitable:

A: Increasing Parquet file size might help if the initial file count is extremely high, but the current file size (200-400MB) is reasonable. The primary issue is I/O performance during shuffle, not file I/O of the initial data.

B: Switching to TFRecords doesn't fundamentally solve the shuffle I/O bottleneck. While it's a different format, the core issue of slow disk access during shuffle remains.

C: Copying data to HDFS on the cluster's HDDs doesn't leverage the speed benefits that the SSDs provide. Also, managing data transfer between GCS and HDFS adds operational overhead. This option is overly complex, and expensive compared to simply increasing the size of the boot disk.

<https://cloud.google.com/dataproc/docs/concepts/configuring-clusters/local-nvme> While this article describes local NVMe disks, it also supports the general idea of leveraging local storage for better performance.

Question: 88

CertyIQ

Your team is responsible for developing and maintaining ETLs in your company. One of your Dataflow jobs is failing because of some errors in the input data, and you need to improve reliability of the pipeline (incl. being able to reprocess all failing data).

What should you do?

A. Add a filtering step to skip these types of errors in the future, extract erroneous rows from logs. that transforms the data, extract erroneous rows from logs. . that can be stored to PubSub later.

Answer: D

Explanation:

Your team is responsible for developing and maintaining ETLs in your company. One of your Dataflow jobs is failing because of some errors in the input data, and you need to improve reliability of the pipeline (incl. being able to reprocess all failing data).

What should you do?

1. **A. Add a filtering step to skip these types of errors in the future, extract erroneous rows from logs.**
2. **B. Add a try| catch block to your DoFn that transforms the data, extract erroneous rows from logs.**
3. **C. Add a try| catch block to your DoFn that transforms the data, write erroneous rows to Pub/Sub PubSub directly from the DoFn.**
4. **D. Add a try| catch block to your DoFn that transforms the data, use a sideOutput to create a PCollection that can be stored to Pub/Sub later.**

Option D, which entails implementing a dead-letter queue (DLQ) using Pub/Sub, is the most appropriate solution for improving the reliability of the Dataflow pipeline and enabling reprocessing of failed data. Here's why:

1. **Data Reliability:** Dataflow pipelines are designed for reliability, but errors can occur due to unforeseen issues in source data. A DLQ acts as a safety net, preventing faulty records from halting the entire pipeline.
2. **Error Isolation:** When a record fails processing, instead of causing the pipeline to fail, it's routed to the DLQ. This isolates erroneous records, allowing the pipeline to continue processing valid data.
3. **Reprocessing Capability:** The DLQ, usually backed by Pub/Sub, persists failed records. This allows developers to investigate the errors, fix any upstream issues or data inconsistencies, and later reprocess the data by re-injecting them into the pipeline.
4. **Non-Disruptive Improvement:** Implementing a DLQ does not require altering the core processing logic. It adds a handling mechanism for exceptions and improves the overall data flow.
5. **Pub/Sub as DLQ:** Pub/Sub is a durable and scalable messaging service. It is a great choice for a DLQ, ensuring that failed records are not lost and are readily available for reprocessing.
6. **Avoiding Data Loss:** A DLQ ensures that data is not lost when an error happens. This is critical because data can be valuable, and there might be a necessity to reprocess it after corrections or a

bugfix is implemented.

The other options are less effective:

1. **Option A:** Skipping errors can lead to data loss and would not allow the user to address and reprocess the faulty data. Additionally, logs are not reliable and can potentially be rotated/deleted.
2. **Option B:** Transforming data in place does not isolate errors or provide an ability to reprocess. Additionally, if the transform failed for a batch of records, they would be lost, not processed.
3. **Option C:** While logging is useful for debugging, it doesn't provide a reliable mechanism to identify and reprocess the faulty records.

In summary, the DLQ with Pub/Sub ensures the pipeline's robustness by isolating and storing erroneous records for further investigation and reprocessing. This aligns with best practices for building resilient data pipelines.

Authoritative Links:

1. **Google Cloud Dataflow Error Handling:** <https://cloud.google.com/dataflow/docs/concepts/error-handling>
2. **Pub/Sub Dead Letter Queues:** <https://cloud.google.com/pubsub/docs/dead-letter-queues>
3. **Dataflow Best Practices for Reliability:** <https://cloud.google.com/dataflow/docs/guides/best-practices#reliability>

Question: 89

CertyIQ

You're training a model to predict housing prices based on an available dataset with real estate properties. Your plan is to train a fully connected neural net, and you've discovered that the dataset contains latitude and longitude of the property. Real estate professionals have told you that the location of the property is highly influential on price, so you'd like to engineer a feature that incorporates this physical dependency. What should you do?

- A. Provide latitude and longitude as input vectors to your neural net.
- B. Create a numeric column from a feature cross of latitude and longitude.
- C. Create a feature cross of latitude and longitude, bucketize it at the minute level and use L1 regularization during optimization.
- D. Create a feature cross of latitude and longitude, bucketize it at the minute level and use L2 regularization during optimization.

Answer: C

Explanation:

The most appropriate approach is option C: create a feature cross of latitude and longitude, bucketize it at the minute level, and use L1 regularization during optimization.

Here's why:

Feature Cross: Combining latitude and longitude creates a single feature that represents a specific geographical location, capturing the interaction between these two variables, which is crucial as their combined effect on housing prices is likely non-linear. Simply feeding them in as individual vectors (Option A) might not capture this interaction.

Bucketization at the Minute Level: Bucketing, in this case, grouping locations with similar coordinates, allows the model to learn price trends within specific regions, creating clusters of similar housing prices rather than treating each individual coordinate as unique and separate. Bucketing at the minute level offers a reasonably granular representation of location that maintains some level of abstraction without being too specific, which

could lead to overfitting.

L1 Regularization: L1 regularization adds the sum of the absolute values of the weights to the loss function. This encourages sparsity in the model, effectively performing feature selection. In the context of a dense feature like a location-based feature cross, this could help the model prioritize the more important buckets, effectively regularizing and improving its generalization. L2 regularization (Option D), on the other hand, penalizes large weights, which might not be as helpful in identifying the most important buckets.

Numeric Column from Feature Cross (Option B): While using a numeric column may seem efficient, it does not capture the geographic importance of different location clusters. This loses much of the spatial information and introduces an artificial linear relationship.

Why not Option A: Directly feeding latitude and longitude as vectors ignores their inherent combined effect on property prices, not exploiting their spatial correlation.

In summary, option C offers a method that acknowledges the non-linear spatial relationship and uses a combination of feature engineering (feature crossing and bucketization) and model regularization techniques (L1), leading to better performance and avoiding overfitting.

Supporting Links:

Feature crosses in TensorFlow:

https://www.tensorflow.org/tutorials/structured_data/feature_columns#feature_crosses

Bucketing in TensorFlow:

https://www.tensorflow.org/api_docs/python/tf/feature_column/bucketized_column

L1 Regularization: <https://developers.google.com/machine-learning/crash-course/regularization-for-simplicity/l1-regularization>

Question: 90

CertyIQ

You are deploying MariaDB SQL databases on GCE VM Instances and need to configure monitoring and alerting. You want to collect metrics including network connections, disk IO and replication status from MariaDB with minimal development effort and use StackDriver for dashboards and alerts. What should you do?

- A. Install the OpenCensus Agent and create a custom metric collection application with a StackDriver exporter.
- B. Place the MariaDB instances in an Instance Group with a Health Check.
- C. Install the StackDriver Logging Agent and configure fluentd in_tail plugin to read MariaDB logs.
- D. Install the StackDriver Agent and configure the MySQL plugin.

Answer: D

Explanation:

The correct answer is D, installing the StackDriver Agent and configuring the MySQL plugin. Here's why:

Stackdriver (now part of Google Cloud Observability) offers a robust suite of monitoring and logging tools. The Stackdriver Agent is designed to collect metrics and logs from various sources, including databases. Specifically, it includes a dedicated MySQL plugin that can directly gather relevant MariaDB metrics without requiring custom development. These metrics encompass network connections, disk I/O, and replication status, all of which are critical for database performance and health monitoring. By configuring the MySQL plugin, you leverage existing, optimized tools for metric collection. Option A, using OpenCensus and custom metric collection, requires significant development effort and maintenance, contrary to the goal of minimal development. Option B, instance groups with health checks, are crucial for high availability, but do not directly provide the detailed metrics needed for monitoring. Option C, focusing on logs with the logging agent and fluentd, while valuable for debugging and auditing, doesn't directly provide performance metrics required for monitoring. The MySQL plugin in the Stackdriver Agent offers pre-built functionality to capture and export

these metrics to Stackdriver, making it the most efficient and least development-intensive solution. This direct integration allows you to utilize Stackdriver's dashboard and alerting capabilities immediately.

Authoritative Links:

Google Cloud Observability: <https://cloud.google.com/stackdriver/docs>

Stackdriver Agent Installation: <https://cloud.google.com/monitoring/agent/install-agent>

Stackdriver Agent Configurations for Third-Party Applications:

<https://cloud.google.com/monitoring/agent/plugins/third-party>

MySQL plugin:<https://cloud.google.com/monitoring/agent/plugins/mysql>

Question: 91

CertyIQ

You work for a bank. You have a labelled dataset that contains information on already granted loan application and whether these applications have been defaulted. You have been asked to train a model to predict default rates for credit applicants.

What should you do?

- A. Increase the size of the dataset by collecting additional data.
- B. Train a linear regression to predict a credit default risk score.
- C. Remove the bias from the data and collect applications that have been declined loans.
- D. Match loan applicants with their social profiles to enable feature engineering.

Answer: B

Explanation:

The optimal approach for predicting credit default risk using a labeled dataset of past loan applications is to train a classification model, and amongst the options, training a linear regression is a reasonable starting point which can provide a numeric default risk score. Linear regression, while typically used for regression tasks, can be adapted for binary classification by setting a threshold on its output to classify applications as either likely to default or not. Option B is more aligned with the prompt than the others.

Option A, while generally beneficial, is not the immediate next step. Option C suggests removing bias and collecting declined loan applications, which are good practices for the long term, but do not align with the training prompt; they address data issues not model choice. Option D proposes feature engineering with social profiles, which can be useful but requires additional data and effort that's not necessarily crucial as the first step. Therefore option B provides a suitable model choice for the scenario given.

Given the nature of the problem, where the outcome (default or not) is binary, a logistic regression could be a more suitable choice as it directly provides probabilities of default. However, given options, linear regression is the most relevant model in provided answer. Linear regression models for binary classification are commonly employed in financial settings as well (although less suited than logistics regression).

Furthermore, linear regression is computationally efficient and relatively straightforward to implement, making it a good initial choice for model development and baseline. After the initial model is developed, the model could then be improved by testing additional algorithms, such as logistic regression, decision trees, support vector machines or neural networks.

Authoritative Links:

Linear Regression: <https://developers.google.com/machine-learning/crash-course/descending-into-ml/linear-regression>

Binary Classification: <https://developers.google.com/machine-learning/crash-course/classification/video-lecture>

Question: 92

CertyIQ

You need to migrate a 2TB relational database to Google Cloud Platform. You do not have the resources to significantly refactor the application that uses this database and cost to operate is of primary concern. Which service do you select for storing and serving your data?

- A. Cloud Spanner
- B. Cloud Bigtable
- C. Cloud Firestore
- D. Cloud SQL

Answer: D

Explanation:

The correct answer is D, Cloud SQL. Here's why: Cloud SQL is a fully managed relational database service, making it a good fit for migrating an existing 2TB relational database with minimal application refactoring. This aligns directly with the requirement of not having resources for significant code changes. Cloud SQL offers various database engines (e.g., PostgreSQL, MySQL, SQL Server), likely allowing you to choose the same engine as your on-premises system, further simplifying the migration. Additionally, it's cost-effective for traditional relational database workloads, making it suitable when cost is a primary concern. Cloud Spanner and Bigtable are better suited for horizontally scalable and highly available scenarios and are more complex to manage or cost-prohibitive for a 2TB dataset with limited refactoring capabilities. Cloud Firestore is a NoSQL document database, which would require significant application refactoring, thus not meeting the stated needs. Cloud SQL provides a managed environment, reducing operational overhead, and simplifying the migration and ongoing management of a relational database.

For further research, refer to the official Google Cloud documentation:

Cloud SQL Overview: <https://cloud.google.com/sql/docs/>

Cloud Spanner Overview: <https://cloud.google.com/spanner/docs>

Cloud Bigtable Overview: <https://cloud.google.com/bigtable/docs>

Cloud Firestore Overview: <https://cloud.google.com/firestore/docs>

Question: 93

CertyIQ

You're using Bigtable for a real-time application, and you have a heavy load that is a mix of read and writes. You've recently identified an additional use case and need to perform hourly an analytical job to calculate certain statistics across the whole database. You need to ensure both the reliability of your production application as well as the analytical workload.

What should you do?

- A. Export Bigtable dump to GCS and run your analytical job on top of the exported files.
- B. Add a second cluster to an existing instance with a multi-cluster routing, use live-traffic app profile for your regular workload and batch-analytics profile for the analytics workload.
- C. Add a second cluster to an existing instance with a single-cluster routing, use live-traffic app profile for your regular workload and batch-analytics profile for the analytics workload.
- D. Increase the size of your existing cluster twice and execute your analytics workload on your new resized cluster.

Answer: C

Explanation:

Here's a detailed justification for why option C is the most appropriate solution, along with explanations of why the other options are less ideal:

Justification for Option C (Add a second cluster with single-cluster routing):

Option C proposes adding a second cluster to the existing Bigtable instance while maintaining single-cluster routing. This is advantageous for several reasons:

1. **Isolation of Analytical Workload:** Creating a separate cluster specifically for the hourly analytical job isolates it from the real-time application's read and write operations. This prevents the analytical workload from negatively impacting the performance and reliability of the production system. By routing all traffic for the analytical job to the second cluster, we ensure the production cluster remains dedicated to the critical application.
2. **Performance Tuning:** The second cluster can be sized and configured independently to optimally handle the analytical workload's resource requirements. This allows for tuning its node count and storage to match the analytical task, without affecting the primary cluster's setup for serving real-time requests.
3. **App Profiles for Workload Management:** Using separate app profiles ("live-traffic" for the production application and "batch-analytics" for the analytical job) provides granular control over routing and priority. The "live-traffic" app profile can be configured to prioritize low-latency reads and writes for the real-time application, while the "batch-analytics" profile can be tailored to handle the bulk data processing demands of the analytical workload. This can involve setting different priorities or even routing policies to further optimize performance and availability.
4. **Cost Efficiency:** By segregating the analytical job to its own cluster and only running it hourly, you can scale down that cluster significantly when not in use. This reduces operational costs when compared to constantly maintaining a larger cluster capable of handling both types of workloads simultaneously.

Why other options are less ideal:

Option A (Export to GCS): While exporting to GCS isolates the analytical workload, it introduces complexities with data freshness. The data in GCS is only as current as the last export, which means the analytical job would be working on potentially stale data. Furthermore, exporting a full Bigtable database hourly is a resource-intensive operation in itself and can incur significant costs and time.

Option B (Multi-cluster routing): Multi-cluster routing is typically employed for high availability and replication across geographically distributed clusters. Using it in this scenario introduces unnecessary complexity and doesn't guarantee the analytical workload won't still impact the real-time application, especially if the clusters are not in different regions. The primary advantage of multi-cluster routing is data redundancy and disaster recovery, not workload isolation within the same region.

Option D (Increase existing cluster size): Simply increasing the size of the existing cluster does not isolate the workloads. Both the real-time application and the analytical job would be competing for the same resources, which can lead to performance degradation for the real-time application during the hourly analytical job. It's also not the most cost-effective approach as you are paying for the resources to serve the peak load at all times.

Authoritative Links:

Bigtable Instances: <https://cloud.google.com/bigtable/docs/instances>

Bigtable App Profiles: <https://cloud.google.com/bigtable/docs/app-profiles>

Bigtable Performance Considerations: <https://cloud.google.com/bigtable/docs/performance>

In conclusion, creating a second cluster with single-cluster routing and using app profiles for workload management offers the best balance of performance, reliability, isolation, and cost-effectiveness for handling both real-time and analytical workloads in Bigtable.

Question: 94

CertyIQ

You are designing an Apache Beam pipeline to enrich data from Cloud Pub/Sub with static reference data from BigQuery. The reference data is small enough to fit in memory on a single worker. The pipeline should write enriched results to BigQuery for analysis. Which job type and transforms should this pipeline use?

- A. Batch job, PubSubIO, side-inputs
- B. Streaming job, PubSubIO, JdbcIO, side-outputs
- C. Streaming job, PubSubIO, BigQueryIO, side-inputs
- D. Streaming job, PubSubIO, BigQueryIO, side-outputs

Answer: C

Explanation:

The correct answer is **C. Streaming job, PubSubIO, BigQueryIO, side-inputs**. Here's a detailed justification:

1. **Streaming Job:** Since the data originates from Cloud Pub/Sub, which delivers messages in real-time, a streaming job is necessary to process the incoming data continuously. A batch job wouldn't be suitable for this real-time processing.
2. **PubSubIO:** This transform is essential for reading data from the Cloud Pub/Sub topic. It acts as the source for our streaming pipeline.
3. **BigQueryIO:** This transform is crucial for writing the enriched data to BigQuery, which is the designated destination for analysis.
4. **Side-Inputs:** The reference data from BigQuery is small enough to fit in memory. This makes side inputs the ideal choice. Side inputs allow you to provide static data to each worker without requiring per-element lookups. The pipeline reads the BigQuery data once and makes it accessible to the processing of each Pub/Sub message.
5. **Why not the other options?:**

Option A suggests a batch job which isn't appropriate for real-time Pub/Sub data.

Option B uses JdbcIO, which is not needed since the reference data comes from BigQuery. Side outputs are not the right mechanism for providing reference data to each worker.

Option D suggests side outputs which are designed for splitting data into different output branches based on processing results, and not for holding reference data.

In summary, the chosen solution utilizes the appropriate job type (streaming) and transforms (PubSubIO, BigQueryIO) along with side-inputs to efficiently process incoming Pub/Sub messages and enrich them with static data from BigQuery before storing the results in BigQuery for analysis.

Authoritative Links:

Apache Beam Streaming: <https://beam.apache.org/documentation/programming-guide/#streaming>

Apache Beam PubSubIO: <https://beam.apache.org/documentation/io/built-in/google-cloud-pubsub/>

Apache Beam BigQueryIO: <https://beam.apache.org/documentation/io/built-in/google-bigquery/>

Apache Beam Side Inputs: <https://beam.apache.org/documentation/programming-guide/#side-inputs>

Question: 95

You have a data pipeline that writes data to Cloud Bigtable using well-designed row keys. You want to monitor your pipeline to determine when to increase the size of your Cloud Bigtable cluster. Which two actions can you take to accomplish this? (Choose two.)

- A. Review Key Visualizer metrics. Increase the size of the Cloud Bigtable cluster when the Read pressure index is above 100.
- B. Review Key Visualizer metrics. Increase the size of the Cloud Bigtable cluster when the Write pressure index is above 100.
- C. Monitor the latency of write operations. Increase the size of the Cloud Bigtable cluster when there is a sustained increase in write latency.
- D. Monitor storage utilization. Increase the size of the Cloud Bigtable cluster when utilization increases above 70% of max capacity.
- E. Monitor latency of read operations. Increase the size of the Cloud Bigtable cluster of read operations take longer than 100 ms.

Answer: CD

Explanation:

Here's a justification for why options C and D are the correct actions for determining when to scale a Cloud Bigtable cluster, and why the other options are less suitable:

Correct Options (C & D):

C. Monitor the latency of write operations. Increase the size of the Cloud Bigtable cluster when there is a sustained increase in write latency. High write latency is a strong indicator that the cluster is struggling to keep up with the incoming data volume. As the cluster becomes overloaded, write operations will take longer to complete, impacting the performance of the data pipeline. Scaling up the cluster by adding more nodes will provide additional resources to handle the workload, reducing write latency.

<https://cloud.google.com/bigtable/docs/monitoring-performance#latency>

D. Monitor storage utilization. Increase the size of the Cloud Bigtable cluster when utilization increases above 70% of max capacity. Approaching the storage capacity limit puts stress on the cluster. When utilization exceeds a certain threshold (e.g., 70%), it can lead to performance degradation and potential data loss or unavailability as the cluster struggles to manage data growth. Scaling out the cluster provides more storage space and distributes the load across a larger pool of resources.

https://cloud.google.com/bigtable/docs/best-practices-performance#monitor_your_cluster_s_storage_utilization

Incorrect Options (A, B, & E):

A. Review Key Visualizer metrics. Increase the size of the Cloud Bigtable cluster when the Read pressure index is above 100. While Key Visualizer helps identify access patterns and hotspots, a high read pressure index doesn't necessarily mean the write load is causing a problem. It indicates issues with reading data efficiently, which might need optimizations in query design or row key strategy, not necessarily a larger cluster.

B. Review Key Visualizer metrics. Increase the size of the Cloud Bigtable cluster when the Write pressure index is above 100. Similarly to A, while Key Visualizer write pressure index is useful for identifying hotspots in writes, it is not the best metric to indicate overall cluster capacity is insufficient to handle data writes. It points out uneven or inefficient write access patterns. Addressing hotspots identified by key visualizer should be done by improving the row key strategy, not necessarily increasing cluster size.

E. Monitor latency of read operations. Increase the size of the Cloud Bigtable cluster of read operations take longer than 100 ms. While read latency is an important metric, it is not the direct indicator for scaling

decision when the data pipeline writes data to BigTable. High read latency is a separate concern that might require different tuning, like improving query design, optimizing filtering, or optimizing row key design to improve data locality, not always by increasing the cluster size.

In summary, the focus should be on monitoring metrics directly related to the cluster's capacity to handle write operations (latency and storage utilization). The Key Visualizer, while a helpful tool, is for analyzing performance issues and hotspots, not a direct indicator of when the cluster needs to be scaled due to write volume. Monitoring read latency is essential for optimizing the performance of data retrievals, but is not an indicator of insufficient write capacity.

Question: 96

CertyIQ

You want to analyze hundreds of thousands of social media posts daily at the lowest cost and with the fewest steps.

You have the following requirements:

- You will batch-load the posts once per day and run them through the Cloud Natural Language API.
- You will extract topics and sentiment from the posts.
- You must store the raw posts for archiving and reprocessing.
- You will create dashboards to be shared with people both inside and outside your organization.

You need to store both the data extracted from the API to perform analysis as well as the raw social media posts for historical archiving. What should you do?

- A. Store the social media posts and the data extracted from the API in BigQuery.
- B. Store the social media posts and the data extracted from the API in Cloud SQL.
- C. Store the raw social media posts in Cloud Storage, and write the data extracted from the API into BigQuery.
- D. Feed to social media posts into the API directly from the source, and write the extracted data from the API into BigQuery.

Answer: C

Explanation:

Okay, let's break down why option C is the best choice for this scenario.

The core requirements focus on cost-effectiveness, minimal steps, batch processing, and storing both raw data and analysis results. Option C, which suggests storing raw posts in Cloud Storage and extracted data in BigQuery, optimally addresses these needs.

Cloud Storage is designed for cost-effective and scalable storage of unstructured data, making it ideal for archiving the raw social media posts. Its pay-as-you-go model is significantly cheaper than options like databases for this use case. BigQuery, on the other hand, is a fully managed, serverless data warehouse optimized for large-scale analysis. It is perfect for storing and querying the structured data extracted by the Natural Language API (topics and sentiment). This separation of storage based on data type is a key best practice for cloud data solutions.

Option A, storing everything in BigQuery, would lead to unnecessary storage costs for the raw, unstructured posts. Option B, Cloud SQL, is not suited for storing huge amounts of unstructured data like raw social media posts, and it is significantly more expensive and complex than Cloud Storage for this use case. Option D, feeding directly to the API, would lose the requirement for archiving raw posts and could also complicate reprocessing later. By batch loading from Cloud Storage to process through the NLP API, we ensure that raw data is available and can be reprocessed should our requirements change. The dashboard requirement fits well into BigQuery since it also provides integration with data visualization tools. Therefore, using Cloud Storage for archiving and BigQuery for analysis gives the most scalable and cost-effective solution.

Here are some resources for further information:

Cloud Storage: <https://cloud.google.com/storage>

BigQuery: <https://cloud.google.com/bigquery>

Cloud Natural Language API: <https://cloud.google.com/natural-language>

Data Warehousing on Google Cloud: <https://cloud.google.com/solutions/data-analytics/data-warehousing>

Choosing between Cloud Storage and Cloud SQL: <https://cloud.google.com/blog/products/gcp/choosing-between-cloud-storage-and-cloud-sql-for-your-data>

Question: 97

CertyIQ

You store historic data in Cloud Storage. You need to perform analytics on the historic data. You want to use a solution to detect invalid data entries and perform data transformations that will not require programming or knowledge of SQL.

What should you do?

- A. Use Cloud Dataflow with Beam to detect errors and perform transformations.
- B. Use Cloud Dataprep with recipes to detect errors and perform transformations.
- C. Use Cloud Dataproc with a Hadoop job to detect errors and perform transformations.
- D. Use federated tables in BigQuery with queries to detect errors and perform transformations.

Answer: B

Explanation:

The question requires a solution for analyzing historic Cloud Storage data, detecting invalid entries, and performing transformations without requiring programming or SQL knowledge. Option B, utilizing Cloud Dataprep, is the most suitable choice. Dataprep is a serverless, intelligent data service designed for visually exploring, cleaning, and preparing data. It allows users to build "recipes" through a graphical interface, performing transformations and data quality checks without writing code. This aligns perfectly with the requirement of avoiding programming and SQL.

Option A, Cloud Dataflow with Beam, while powerful for data processing, necessitates programming with Beam SDK, directly contradicting the question's constraints. Option C, Cloud Dataproc with Hadoop jobs, also involves coding in languages like Java or Python. Option D, federated tables in BigQuery, enables querying data in Cloud Storage via SQL, which the question specifically seeks to avoid.

Dataprep's visual interface, intelligent suggestions for transformations, and focus on data preparation, make it ideal for users lacking deep coding experience. The other options, while capable, require programming or SQL expertise that the use case expressly prohibits. Dataprep handles data discovery, schema inference, and provides interactive tools to reshape data, fulfilling all of the criteria described in the prompt.

Further research can be conducted using these links:

Cloud Dataprep Documentation: <https://cloud.google.com/dataprep/docs>

Cloud Dataprep product page: <https://cloud.google.com/dataprep>

Introduction to Cloud Dataprep: <https://cloud.google.com/blog/products/data-analytics/announcing-cloud-dataprep-a-new-intelligent-data-service>

Question: 98

CertyIQ

Your company needs to upload their historic data to Cloud Storage. The security rules don't allow access from external IPs to their on-premises resources. After an initial upload, they will add new data from existing on-premises applications every day. What should they do?

- A. Execute gsutil rsync from the on-premises servers.

- B. Use Dataflow and write the data to Cloud Storage.
- C. Write a job template in Dataproc to perform the data transfer.
- D. Install an FTP server on a Compute Engine VM to receive the files and move them to Cloud Storage.

Answer: A

Explanation:

The correct answer is A, executing gsutil rsync from on-premises servers. Here's a detailed justification:

The problem states the company needs to upload data to Cloud Storage from on-premises servers, but direct external IP access from on-premises to cloud resources is prohibited. This limitation rules out solutions that inherently require publicly accessible services or intermediary VMs exposed to the external internet.

Why A is correct: gsutil rsync allows synchronization of data between on-premises and Cloud Storage. Importantly, gsutil can be configured to use a private Google Access configuration via Cloud VPN or Cloud Interconnect. This enables the on-premises servers to communicate securely with Google Cloud services without needing public IPs. Once the initial setup using Cloud VPN or Interconnect is in place, automated gsutil rsync jobs can be scheduled for the daily data updates. This is the most straightforward and efficient solution.

Why B is incorrect: Using Dataflow implies the need for a managed service (Dataflow) to access the data. While technically possible, it's overkill for a simple file transfer. Dataflow is primarily designed for complex data transformations and pipelines, adding unnecessary overhead and cost. Furthermore, the question states applications exist on-premises; rewriting these to utilize Dataflow is not efficient or feasible, especially considering existing security constraints.

Why C is incorrect: Dataproc is a managed Hadoop and Spark service, designed for big data processing and analytics. Similar to Dataflow, using Dataproc solely for data transfer is overly complex and expensive. It's not the right tool for the job when a simpler solution like gsutil rsync is available. It also necessitates more infrastructure to manage than gsutil.

Why D is incorrect: Installing an FTP server on a Compute Engine VM is problematic because it reintroduces the need for a publicly accessible endpoint. While the VM itself might be secured to some extent, it adds a new security risk and management overhead. The problem states external IP access is restricted, and this solution directly violates that constraint.

Therefore, gsutil rsync combined with a secure connection (Cloud VPN/Interconnect) provides the most practical, secure, and efficient solution for transferring data from on-premises to Cloud Storage under the given constraints. It avoids the complexity and cost of Dataflow or Dataproc, and it addresses the security restriction unlike the FTP server option.

Authoritative Links:

gsutil rsync: <https://cloud.google.com/storage/docs/gsutil/commands/rsync>

Private Google Access: <https://cloud.google.com/vpc/docs/private-google-access>

Cloud Interconnect: <https://cloud.google.com/interconnect>

Cloud VPN: <https://cloud.google.com/vpn>

Question: 99

CertyIQ

You have a query that filters a BigQuery table using a WHERE clause on timestamp and ID columns. By using bq query "--dry_run you learn that the query triggers a full scan of the table, even though the filter on timestamp and ID select a tiny fraction of the overall data. You want to reduce the amount of data scanned by BigQuery with minimal changes to existing SQL queries. What should you do?

- A. Create a separate table for each ID.
- B. Use the LIMIT keyword to reduce the number of rows returned.
- C. Recreate the table with a partitioning column and clustering column.
- D. Use the bq query --maximum_bytes_billed flag to restrict the number of bytes billed.

Answer: C

Explanation:

The correct answer is **C. Recreate the table with a partitioning column and clustering column**. Here's a detailed justification:

The problem stems from BigQuery performing a full table scan because the WHERE clause filters on timestamp and ID, but the table isn't optimized for these filters. A full scan means BigQuery has to read through all the data in the table regardless of the filter, leading to increased cost and query latency.

Option A (creating separate tables per ID) is impractical for a large number of IDs and would significantly complicate data management and querying. While it technically avoids a full scan within each smaller table, the overhead and inflexibility make it unsuitable.

Option B (using LIMIT) only reduces the number of rows returned to the user. It doesn't reduce the amount of data scanned by BigQuery; the full table scan still happens before the LIMIT is applied. It addresses the symptom (too many rows returned) not the root cause (full table scan).

Option D (using --maximum_bytes_billed) prevents runaway costs by stopping the query if it exceeds a specified data scan limit. While helpful for cost control, it doesn't reduce the amount of data scanned. The query will still attempt a full scan initially.

Option C, partitioning and clustering, is the most effective solution. Partitioning divides the table into smaller, more manageable segments based on a column, typically a timestamp or date column. Clustering further sorts the data within each partition based on one or more other columns, in this case likely the ID.

By partitioning on the timestamp (or date) and clustering on the ID, BigQuery can use these optimizations to drastically reduce the amount of data scanned. When a query includes a WHERE clause that filters on the partition column (timestamp) and clustering column (ID), BigQuery can efficiently skip irrelevant partitions and clusters, reading only the data that potentially satisfies the filter conditions. This significantly reduces the query execution time and cost.

For example, if the table is partitioned by date and clustered by ID, a query with WHERE timestamp BETWEEN '2023-10-26 00:00:00' AND '2023-10-26 23:59:59' AND ID = '123' will only scan the '2023-10-26' partition and, within that partition, efficiently locate data associated with ID '123' due to the clustering.

Therefore, partitioning and clustering are fundamental BigQuery optimization techniques to avoid full table scans and improve query performance. <https://cloud.google.com/bigquery/docs/partitioned-tables><https://cloud.google.com/bigquery/docs/clustered-tables>

Question: 100

CertyIQ

You have a requirement to insert minute-resolution data from 50,000 sensors into a BigQuery table. You expect significant growth in data volume and need the data to be available within 1 minute of ingestion for real-time analysis of aggregated trends. What should you do?

- A. Use bq load to load a batch of sensor data every 60 seconds.
- B. Use a Cloud Dataflow pipeline to stream data into the BigQuery table.

- C. Use the INSERT statement to insert a batch of data every 60 seconds.
- D. Use the MERGE statement to apply updates in batch every 60 seconds.

Answer: B

Explanation:

The optimal solution for ingesting high-volume, minute-resolution sensor data into BigQuery with near real-time availability is **(B) Use a Cloud Dataflow pipeline to stream data into the BigQuery table.**

Here's why:

Streaming Capabilities: Cloud Dataflow is a fully managed, scalable, and serverless stream processing service. It is specifically designed for handling continuous data streams, making it ideal for ingesting sensor data in real-time. (Source: [Google Cloud Dataflow Documentation](#))

Near Real-Time Ingestion: Dataflow processes data as it arrives, allowing for near-instantaneous ingestion into BigQuery, meeting the requirement of data availability within 1 minute.

Scalability: Dataflow automatically scales its processing resources to handle the expected growth in sensor data volume. This is crucial when dealing with 50,000 sensors and potential future expansion.

BigQuery Streaming API Integration: Dataflow seamlessly integrates with BigQuery's streaming API, which is designed for high-speed data ingestion. It avoids the limitations of batch loading and insert/merge operations.

Fault Tolerance: Dataflow provides robust fault tolerance and data consistency, ensuring that data is processed and ingested reliably, even during unexpected events.

Avoids Batch Limitations: Options A, C, and D involve batch processing (either loading or using SQL statements). These methods are not suitable for achieving near real-time ingestion at scale. They introduce latency because data must accumulate into batches before loading or updating the table.

Insert Statement Inefficiency: Using the INSERT statement (Option C) for a high volume of individual inserts is highly inefficient for BigQuery and could overwhelm the service and introduce performance issues.

Merge Statement Complexity: Option D, using MERGE statements, is suitable for updates, not the initial insertion of new streaming data. It's also a batch operation, making it unsuitable for this scenario.

In conclusion, Dataflow's streaming capabilities, scalability, and seamless integration with BigQuery's streaming API make it the most appropriate service for ingesting high-volume, minute-resolution data with near real-time availability for real-time analysis of aggregated trends.

Question: 101

CertyIQ

You need to copy millions of sensitive patient records from a relational database to BigQuery. The total size of the database is 10 TB. You need to design a solution that is secure and time-efficient. What should you do?

- A. Export the records from the database as an Avro file. Upload the file to GCS using gsutil, and then load the Avro file into BigQuery using the BigQuery web UI in the GCP Console.
- B. Export the records from the database as an Avro file. Copy the file onto a Transfer Appliance and send it to Google, and then load the Avro file into BigQuery using the BigQuery web UI in the GCP Console.
- C. Export the records from the database into a CSV file. Create a public URL for the CSV file, and then use Storage Transfer Service to move the file to Cloud Storage. Load the CSV file into BigQuery using the BigQuery web UI in the GCP Console.
- D. Export the records from the database as an Avro file. Create a public URL for the Avro file, and then use Storage Transfer Service to move the file to Cloud Storage. Load the Avro file into BigQuery using the BigQuery web UI in the GCP Console.

Answer: B

Explanation:

The correct answer is **B** because it addresses both the security and time-efficiency requirements effectively. Option A, while utilizing Avro for efficient data handling, involves uploading data via gsutil. This method can be slower for large datasets like 10TB and might expose sensitive data during transit if not properly secured through HTTPS. Option C is unsuitable because using a CSV with a public URL poses a significant security risk for sensitive patient data. Option D shares the security vulnerability of C, additionally Storage Transfer Service is not optimal for on-premises database backups. Using Transfer Appliance (as in B) physically moves the data, bypassing network bottlenecks and ensuring data encryption in transit, making it time-efficient and secure, especially for large and sensitive datasets. Once the Transfer Appliance data reaches Google, loading the Avro file to BigQuery using the BigQuery web UI is a standard and reliable procedure. This eliminates the risk of network-based transfers during the initial migration, making it ideal for sensitive patient data. Moreover, Avro format is a great choice for data transfer into BigQuery due to its schema evolution capabilities.

Relevant Links:

Transfer Appliance: <https://cloud.google.com/transfer-appliance>

BigQuery Loading Data: <https://cloud.google.com/bigquery/docs/loading-data>

Avro Format: <https://avro.apache.org/>

gsutil: <https://cloud.google.com/storage/docs/gsutil>

Storage Transfer Service: <https://cloud.google.com/storage-transfer-service>

Question: 102

CertyIQ

You need to create a near real-time inventory dashboard that reads the main inventory tables in your BigQuery data warehouse. Historical inventory data is stored as inventory balances by item and location. You have several thousand updates to inventory every hour. You want to maximize performance of the dashboard and ensure that the data is accurate. What should you do?

- A. Leverage BigQuery UPDATE statements to update the inventory balances as they are changing.
- B. Partition the inventory balance table by item to reduce the amount of data scanned with each inventory update.
- C. Use the BigQuery streaming the stream changes into a daily inventory movement table. Calculate balances in a view that joins it to the historical inventory balance table. Update the inventory balance table nightly.
- D. Use the BigQuery bulk loader to batch load inventory changes into a daily inventory movement table. Calculate balances in a view that joins it to the historical inventory balance table. Update the inventory balance table nightly.

Answer: C

Explanation:

The correct answer is **C**. Here's why:

Option C proposes using BigQuery streaming to ingest real-time inventory changes into a daily movement table. This leverages BigQuery's streaming capabilities, designed for high-volume, near real-time data ingestion, aligning perfectly with the problem's requirements of thousands of hourly updates. It also avoids direct updates to the historical balance table, which can be costly and inefficient in BigQuery, instead focusing on appending new records. Then, a view joins this movement table with the historical balance table to compute the current balances. This approach ensures data accuracy as all changes are captured, while also enabling efficient dashboard performance by minimizing the amount of data to be scanned for current inventory. The daily inventory balance table update ensures that the view has a snapshot of the data and avoids any performance issues from constantly recalculating balances.

Option A, using BigQuery UPDATE statements directly on the inventory balance table, is not ideal for high-frequency updates. BigQuery is designed for analytical workloads, not transactional updates, making

frequent UPDATE statements costly and impacting performance.

<https://cloud.google.com/bigquery/docs/best-practices-performance-updates> states "Avoid DML (data manipulation language) statements when possible...Use DML statements sparingly and only when other solutions are not feasible."

Option B, partitioning by item, while useful for query performance when filtering on items, doesn't directly improve the performance of updates to the inventory balance table. Partitioning is used for query and storage optimization, not update efficiency. <https://cloud.google.com/bigquery/docs/partitioned-tables>

Option D, using the BigQuery bulk loader, is more appropriate for batch loading large amounts of data periodically, not for near real-time data. It would cause a delay in displaying the updates to the dashboard and fail to meet the near real time updates requirement. <https://cloud.google.com/bigquery/docs/loading-data> Therefore, Option C delivers the best balance of performance, accuracy, and real-time updates.

Question: 103

CertyIQ

You have a data stored in BigQuery. The data in the BigQuery dataset must be highly available. You need to define a storage, backup, and recovery strategy of this data that minimizes cost. How should you configure the BigQuery table that have a recovery point objective (RPO) of 30 days?

- A. Set the BigQuery dataset to be regional. In the event of an emergency, use a point-in-time snapshot to recover the data.
- B. Set the BigQuery dataset to be regional. Create a scheduled query to make copies of the data to tables suffixed with the time of the backup. In the event of an emergency, use the backup copy of the table.
- C. Set the BigQuery dataset to be multi-regional. In the event of an emergency, use a point-in-time snapshot to recover the data.
- D. Set the BigQuery dataset to be multi-regional. Create a scheduled query to make copies of the data to tables suffixed with the time of the backup. In the event of an emergency, use the backup copy of the table.

Answer: C

Explanation:

Here's a detailed justification for why option C is the correct answer, focusing on cost minimization, high availability, and a 30-day recovery point objective (RPO) for BigQuery data:

Explanation:

The key to this question lies in understanding BigQuery's multi-regional and regional datasets, point-in-time snapshots, and how they relate to cost and recovery objectives. Multi-regional datasets inherently offer higher availability and resilience by replicating data across multiple geographic locations within a larger region, such as the US or EU. This built-in redundancy provides fault tolerance. Option C leverages this inherent high availability by choosing a multi-regional dataset and employing point-in-time snapshots. Point-in-time snapshots are a cost-effective and efficient way to maintain historical data states for recovery. They're managed by BigQuery, minimizing operational overhead. With a 30-day RPO, taking snapshots is a more cost-efficient approach than scheduled full table copies. Option A would work similarly with point in time snapshots for recovery but is not as highly available as multi-regional data.

Scheduled query copies (as suggested in B and D) incur additional storage costs for each full copy made, especially when repeated regularly to meet RPO needs. This process can become expensive as data volumes increase. While these copies can provide backup, the built-in resilience and snapshot mechanism are usually a more suitable, cost-effective solution for this use case. The multi-regional data inherently makes the data more highly available, whereas a regional data set, in the event of the regional failure would no longer be highly available.

Therefore, setting the BigQuery dataset to multi-regional and using point-in-time snapshots is the most cost-effective method for achieving high availability and meeting the 30-day RPO. Multi-regional datasets provide the high availability requirement and snapshots provide point in time recovery of the last 30 days.

Supporting Concepts:

Multi-Regional Datasets: Offer higher availability and resilience by replicating data across multiple availability zones within a larger geographical area.

Regional Datasets: Store data within a single geographical area, offering reduced data transfer costs but less fault tolerance.

Point-in-Time Snapshots: Cost-effective, efficient copies of the data at a specific point in time, enabling recovery to a previous state without needing to store redundant copies of the whole table, but the data is still in the same location as the source table.

Recovery Point Objective (RPO): Defines the maximum acceptable data loss in the event of an incident.

Authoritative Links:

BigQuery Datasets Locations: <https://cloud.google.com/bigquery/docs/locations>

BigQuery Point-in-Time Recovery: <https://cloud.google.com/bigquery/docs/time-travel>

BigQuery Backup and Disaster Recovery: <https://cloud.google.com/architecture/backup-and-disaster-recovery-bigquery>

Question: 104

CertyIQ

You used Dataprep to create a recipe on a sample of data in a BigQuery table. You want to reuse this recipe on a daily upload of data with the same schema, after the load job with variable execution time completes. What should you do?

- A. Create a cron schedule in Dataprep.
- B. Create an App Engine cron job to schedule the execution of the Dataprep job.
- C. Export the recipe as a Dataprep template, and create a job in Cloud Scheduler.
- D. Export the Dataprep job as a Dataflow template, and incorporate it into a Composer job.

Answer: D

Explanation:

Option D is the most appropriate solution for automating the daily execution of a Dataprep recipe on new data. Here's why:

Dataprep's limitations: Dataprep itself lacks built-in scheduling capabilities for production-level pipelines. Option A, suggesting a cron schedule within Dataprep, is incorrect as such functionality doesn't exist.

App Engine limitations: Option B, proposing App Engine cron, is also inappropriate. While App Engine cron can schedule tasks, it wouldn't directly integrate with the Dataprep workflow. You would need to implement custom logic to trigger the Dataprep job, adding complexity.

Templates and orchestration: Options C and D leverage templates for reusable processing logic. Templates encapsulate the steps of your Dataprep recipe.

Dataflow's role: Option D utilizes Dataflow templates, which are ideal for executing data processing pipelines at scale.

Cloud Composer integration: Option D proposes incorporating the Dataflow template into a Composer job. Composer is a fully managed workflow orchestration service built on Apache Airflow. This provides robust scheduling, monitoring, and error handling capabilities crucial for production environments.

Dataflow templates are the preferred approach: Exporting a Dataprep job as a Dataflow template allows you to harness Dataflow's scalable processing engine while also decoupling the logic from the Dataprep UI.

Orchestration with Composer is ideal: Composer enables you to schedule, monitor, and manage the entire data processing pipeline, including the execution of the Dataflow job triggered from the template.

Workflow Management: Composer allows dependencies, retries, and failure handling for a more robust workflow, ensuring the transformation happens after the BigQuery load, which has variable timing.

Scalability and Reliability: Dataflow and Composer together create a scalable and reliable solution that is well-suited for production environments.

Therefore, exporting the Dataprep job as a Dataflow template and incorporating it into a Composer job (Option D) is the most effective and recommended approach for automating the daily execution of the recipe. This solution leverages specialized tools for their designed purposes: Dataflow for data processing and Composer for workflow orchestration.

Authoritative Links for Further Research:

Cloud Dataprep: <https://cloud.google.com/dataprep>

Cloud Dataflow: <https://cloud.google.com/dataflow>

Cloud Composer: <https://cloud.google.com/composer>

Dataflow Templates: <https://cloud.google.com/dataflow/docs/concepts/templates>

Question: 105

CertyIQ

You want to automate execution of a multi-step data pipeline running on Google Cloud. The pipeline includes Dataproc and Dataflow jobs that have multiple dependencies on each other. You want to use managed services where possible, and the pipeline will run every day. Which tool should you use?

- A. cron
- B. Cloud Composer
- C. Cloud Scheduler
- D. Workflow Templates on Dataproc

Answer: B

Explanation:

The correct answer is **B. Cloud Composer**. Here's why:

Cloud Composer is a fully managed workflow orchestration service built on Apache Airflow. It's designed to handle complex, multi-step data pipelines with dependencies, making it ideal for this scenario. Composer allows you to define workflows as Directed Acyclic Graphs (DAGs), representing the relationships between Dataproc and Dataflow jobs, ensuring they execute in the correct order. This satisfies the requirement of managing dependencies between jobs.

Cron, while suitable for scheduling single tasks, is not designed for orchestrating complex workflows with interdependencies, making it less suitable for a multi-step pipeline like this.

[<https://cloud.google.com/scheduler/docs/comparison-cron-scheduler>]

Cloud Scheduler is useful for scheduling simple tasks, such as triggering a Cloud Function or sending an HTTP request, but lacks the advanced orchestration capabilities needed for managing dependent jobs in a pipeline. [<https://cloud.google.com/scheduler/docs>]

Workflow Templates on Dataproc are useful for templating and deploying Dataproc jobs but do not handle orchestration of jobs outside of Dataproc, like Dataflow jobs, nor the sophisticated dependency management across different services needed here. [<https://cloud.google.com/dataproc/docs/concepts/workflows/using-workflow-templates>]

Cloud Composer, with its built-in dependency management, retry mechanisms, and centralized monitoring features, provides the most comprehensive solution for automating and orchestrating a multi-step data pipeline involving Dataproc and Dataflow jobs, thus fulfilling all requirements. It also offers a managed service experience, reducing overhead. The pipeline's daily schedule can also be easily accommodated with Composer's scheduling capabilities. [<https://cloud.google.com/composer/docs>]

Question: 106

CertyIQ

You are managing a Cloud Dataproc cluster. You need to make a job run faster while minimizing costs, without losing work in progress on your clusters. What should you do?

- A. Increase the cluster size with more non-preemptible workers.
- B. Increase the cluster size with preemptible worker nodes, and configure them to forcefully decommission.
- C. Increase the cluster size with preemptible worker nodes, and use Cloud Stackdriver to trigger a script to preserve work.
- D. Increase the cluster size with preemptible worker nodes, and configure them to use graceful decommissioning.

Answer: D

Explanation:

The correct answer is **D: Increase the cluster size with preemptible worker nodes, and configure them to use graceful decommissioning**. Here's why:

Preemptible VMs offer significant cost savings compared to regular VMs, making them ideal for scaling up compute power without breaking the budget. Graceful decommissioning is crucial here. This setting allows preemptible nodes to finish their currently running tasks before being shut down, preventing data loss and ensuring work in progress is completed, which aligns directly with the requirement to not lose work. Option A, while scaling up, increases costs because it uses non-preemptible VMs, which directly contradicts the goal of minimizing costs. Option B, using forceful decommissioning, would result in data loss, failing to satisfy the requirement of preserving work in progress. Option C, while using preemptible VMs, relies on a script triggered by Stackdriver. This is less efficient and requires additional setup, compared to a built-in Dataproc feature like graceful decommissioning. Graceful decommissioning is a more streamlined and effective method for managing preemptible instances, letting work complete while minimizing cost. This makes option D the best choice for achieving the specific objective of faster job completion while minimizing costs and protecting in-progress work.

Relevant documentation links:

Preemptible VMs: <https://cloud.google.com/compute/docs/instances/preemptible>

Graceful decommissioning: https://cloud.google.com/dataproc/docs/concepts/configuring-clusters/preemptible-vms#graceful_decommissioning

Dataproc Cluster Configuration: <https://cloud.google.com/dataproc/docs/concepts/configuring-clusters>

Question: 107

CertyIQ

You work for a shipping company that uses handheld scanners to read shipping labels. Your company has strict data privacy standards that require scanners to only transmit tracking numbers when events are sent to Kafka topics. A recent software update caused the scanners to accidentally transmit recipients' personally identifiable information (PII) to analytics systems, which violates user privacy rules. You want to quickly build a scalable solution using cloud-native managed services to prevent exposure of PII to the analytics systems. What should you do?

- A. Create an authorized view in BigQuery to restrict access to tables with sensitive data.
- B. Install a third-party data validation tool on Compute Engine virtual machines to check the incoming data for sensitive information.
- C. Use Cloud Logging to analyze the data passed through the total pipeline to identify transactions that may contain sensitive information.
- D. Build a Cloud Function that reads the topics and makes a call to the Cloud Data Loss Prevention (Cloud DLP) API. Use the tagging and confidence levels to either pass or quarantine the data in a bucket for review.

Answer: D

Explanation:

The correct answer is **D: Build a Cloud Function that reads the topics and makes a call to the Cloud Data Loss Prevention (Cloud DLP) API. Use the tagging and confidence levels to either pass or quarantine the data in a bucket for review.**

Here's why:

This approach directly addresses the problem of PII exposure in real-time, fulfilling the requirement for a quick and scalable solution. Cloud Functions provide serverless compute, enabling automatic scaling to handle variable data volumes. The Cloud Data Loss Prevention (Cloud DLP) API is specifically designed to inspect data for sensitive information like PII using pattern matching, regular expressions, and machine learning. By calling the Cloud DLP API from the Cloud Function, data can be inspected before reaching downstream systems. DLP provides confidence levels for findings, allowing for informed decisions to pass safe data or quarantine suspicious data for review. Tagging capabilities allow for identification of sensitive fields within the data stream. This method ensures that data privacy rules are enforced, and the process is completely managed on Google Cloud Platform. BigQuery authorized views (option A) only restrict access to existing data within BigQuery, not data before it gets there. Third-party tools on Compute Engine (option B) introduce operational overhead and do not leverage serverless managed services. Cloud Logging (option C) is helpful for audit trails, but doesn't prevent PII from being transmitted. Option D offers the most efficient, effective, and scalable means to achieve the goal of preventing PII exposure.

Authoritative Links:

Cloud Functions: <https://cloud.google.com/functions/docs>

Cloud Data Loss Prevention (DLP): <https://cloud.google.com/dlp/docs>

Question: 108

CertyIQ

You have developed three data processing jobs. One executes a Cloud Dataflow pipeline that transforms data uploaded to Cloud Storage and writes results to BigQuery. The second ingests data from on-premises servers and uploads it to Cloud Storage. The third is a Cloud Dataflow pipeline that gets information from third-party data providers and uploads the information to Cloud Storage. You need to be able to schedule and monitor the execution of these three workflows and manually execute them when needed. What should you do?

- A. Create a Direct Acyclic Graph in Cloud Composer to schedule and monitor the jobs.
- B. Use Stackdriver Monitoring and set up an alert with a Webhook notification to trigger the jobs.
- C. Develop an App Engine application to schedule and request the status of the jobs using GCP API calls.
- D. Set up cron jobs in a Compute Engine instance to schedule and monitor the pipelines using GCP API calls.

Answer: A

Explanation:

The correct answer is **A. Create a Direct Acyclic Graph in Cloud Composer to schedule and monitor the jobs.**

Cloud Composer is a fully managed workflow orchestration service built on Apache Airflow. It excels at scheduling, managing, and monitoring complex workflows, especially those involving multiple steps and dependencies, like the scenario described. Cloud Composer allows defining workflows as Directed Acyclic Graphs (DAGs) using Python code, which enables specifying the sequence of tasks and dependencies. Each task within a DAG can represent the execution of a data processing job like a Dataflow pipeline, a data transfer operation to Cloud Storage, or interaction with external APIs. It includes built-in monitoring features to track the status of DAGs and tasks, offering clear insights into successful executions, failures, and any delays. Furthermore, Cloud Composer provides a user interface for easy manual triggering of DAGs when needed, aligning with the requirement to "manually execute them when needed." Therefore, it perfectly addresses the need for scheduling, monitoring, and manual execution of the three mentioned workflows.

Why other options are incorrect:

B. Use Stackdriver Monitoring and set up an alert with a Webhook notification to trigger the jobs:

Stackdriver (now Cloud Monitoring) excels at monitoring resources and triggering alerts based on metrics, but it is not a workflow orchestration tool. While Webhooks could potentially trigger jobs, this approach lacks the structure for managing dependencies or tracking the overall progress of a workflow.

C. Develop an App Engine application to schedule and request the status of the jobs using GCP API calls:

While App Engine can execute scheduled tasks and interact with GCP APIs, it involves more development effort than using a purpose-built workflow management tool like Composer. App Engine is less suited to visualize workflow dependencies and monitor jobs directly.

D. Set up cron jobs in a Compute Engine instance to schedule and monitor the pipelines using GCP API calls:

While cron jobs can schedule tasks, this approach lacks the robust workflow management capabilities of Cloud Composer and involves more manual configuration and management, making it less efficient for complex workflows.

Authoritative Links:

Cloud Composer Documentation: <https://cloud.google.com/composer/docs>

Apache Airflow Documentation: <https://airflow.apache.org/>

Cloud Dataflow Documentation: <https://cloud.google.com/dataflow/docs>

Question: 109

CertyIQ

You have Cloud Functions written in Node.js that pull messages from Cloud Pub/Sub and send the data to BigQuery. You observe that the message processing rate on the Pub/Sub topic is orders of magnitude higher than anticipated, but there is no error logged in Cloud Logging. What are the two most likely causes of this problem? (Choose two.)

- A. Publisher throughput quota is too small.
- B. Total outstanding messages exceed the 10-MB maximum.
- C. Error handling in the subscriber code is not handling run-time errors properly.
- D. The subscriber code cannot keep up with the messages.
- E. The subscriber code does not acknowledge the messages that it pulls.

Answer: CE

Explanation:

Here's a detailed justification for why options C and E are the most likely causes of the described problem:

Option C: Error handling in the subscriber code is not handling run-time errors properly.

Cloud Functions, like any code, can encounter runtime errors. If the Node.js Cloud Function processing Pub/Sub messages experiences an uncaught error during data processing (e.g., parsing, data transformation, or interaction with BigQuery), it might fail silently without logging the error. Because Cloud Functions relies on acknowledging the message, unhandled errors prevent the acknowledgment from being sent back to Pub/Sub. This failure causes the message to become stuck and retried, continuously attempting to process the same failing message. The high processing rate observed on the Pub/Sub topic is the result of the function repeatedly attempting to handle the same message. Proper error handling within the function, such as using try-catch blocks, is crucial for catching such errors and ensuring they are logged, along with the necessary logic to acknowledge or potentially discard failing messages gracefully.

Option E: The subscriber code does not acknowledge the messages that it pulls.

Cloud Pub/Sub utilizes an acknowledgment system to ensure reliable delivery of messages. When a subscriber (in this case, the Cloud Function) pulls a message, it must acknowledge it after successful processing. Failure to acknowledge the message within a specified timeframe leads Pub/Sub to believe that the message was not processed successfully. Consequently, Pub/Sub re-delivers the unacknowledged message. If the Cloud Function is pulling messages but not explicitly acknowledging them, Pub/Sub will continually resend the same messages, creating a loop and resulting in a significantly higher-than-expected message processing rate on the topic. This is independent of the function being able to successfully handle the message, it just needs to call `message.ack()`.

Why the other options are less likely:

Option A: Publisher throughput quota is too small. Publisher quotas relate to the rate at which messages are published to Pub/Sub, not the rate at which they are processed by subscribers. While a small publisher quota might limit throughput, it wouldn't cause a subscriber to process the same messages repeatedly.

Option B: Total outstanding messages exceed the 10-MB maximum. The 10-MB limit typically refers to the size of individual messages not a cumulative total. While large messages can cause performance issues, it would likely result in errors rather than repetitive processing of the same message. It is also not a direct cause of a high processing rate.

Option D: The subscriber code cannot keep up with the messages. If a function is overwhelmed it is very likely to lead to timeout or out of memory errors that are caught and logged. Also, the subscription would back up and not have excessive rate on the topic.

Authoritative Links:

Cloud Pub/Sub Message Acknowledgment: <https://cloud.google.com/pubsub/docs/subscriber#ack>

Cloud Functions Error Handling: <https://cloud.google.com/functions/docs/best-practices/error-handling>

Node.js Cloud Functions Pub/Sub Trigger: <https://cloud.google.com/functions/docs/calling/pubsub>

Question: 110**CertyIQ**

You are creating a new pipeline in Google Cloud to stream IoT data from Cloud Pub/Sub through Cloud Dataflow to BigQuery. While previewing the data, you notice that roughly 2% of the data appears to be corrupt. You need to modify the Cloud Dataflow pipeline to filter out this corrupt data. What should you do?

- A. Add a SideInput that returns a Boolean if the element is corrupt.
- B. Add a ParDo transform in Cloud Dataflow to discard corrupt elements.
- C. Add a Partition transform in Cloud Dataflow to separate valid data from corrupt data.
- D. Add a GroupByKey transform in Cloud Dataflow to group all of the valid data together and discard the rest.

Answer: B

Explanation:

The correct answer is **B. Add a ParDo transform in Cloud Dataflow to discard corrupt elements**. Here's a detailed justification:

Cloud Dataflow is a powerful service for stream and batch data processing. When dealing with data quality issues like corrupt records, the most straightforward approach is to introduce a transformation step that can identify and handle these problematic records. A ParDo transform is perfectly suited for this task because it allows you to apply a user-defined function to each element of the input PCollection independently. This function can implement the logic for determining if a record is corrupt based on defined rules.

In this scenario, a ParDo would be configured to process each IoT data record. Inside the function, the record would be examined for corruption. If deemed valid, the record would be output as part of the PCollection used for subsequent steps. If a record is corrupt, the ParDo could skip outputting it, effectively filtering it out of the pipeline, thus fulfilling the requirement to discard corrupt data. This approach avoids unnecessary complex logic and maintains pipeline readability. Option A is incorrect because SideInput is generally used for supplementary data, not filtering criteria. Options C and D are incorrect because partitioning or grouping would not effectively filter out the 2% corrupt data in a direct manner, adding unnecessary complexity. ParDo is the established method to implement a one-to-one or one-to-many transformation within Dataflow.

Relevant links:

Apache Beam ParDo: <https://beam.apache.org/documentation/programming-guide/#pardo>

Google Cloud Dataflow Transforms: <https://cloud.google.com/dataflow/docs/concepts/transforms/>

Dataflow SideInputs: <https://cloud.google.com/dataflow/docs/concepts/side-inputs>

Question: 111

CertyIQ

You have historical data covering the last three years in BigQuery and a data pipeline that delivers new data to BigQuery daily. You have noticed that when the Data Science team runs a query filtered on a date column and limited to 30~90 days of data, the query scans the entire table. You also noticed that your bill is increasing more quickly than you expected. You want to resolve the issue as cost-effectively as possible while maintaining the ability to conduct SQL queries. What should you do?

- A. Re-create the tables using DDL. Partition the tables by a column containing a TIMESTAMP or DATE Type.
- B. Recommend that the Data Science team export the table to a CSV file on Cloud Storage and use Cloud Datalab to explore the data by reading the files directly.
- C. Modify your pipeline to maintain the last 30~90 days of data in one table and the longer history in a different table to minimize full table scans over the entire history.
- D. Write an Apache Beam pipeline that creates a BigQuery table per day. Recommend that the Data Science team use wildcards on the table name suffixes to select the data they need.

Answer: A

Explanation:

The best solution is **A. Re-create the tables using DDL. Partition the tables by a column containing a TIMESTAMP or DATE Type**.

Here's why:

Cost Optimization: BigQuery charges based on the amount of data scanned. Partitioning significantly reduces the amount of data scanned for queries filtered by the partitioning column (in this case, date). By partitioning the tables by a TIMESTAMP or DATE column, BigQuery can efficiently read only the relevant partitions, dramatically lowering query costs.

Performance Improvement: Partitioning limits the data scanned, which leads to faster query execution times. This is especially noticeable when querying a subset of the data based on date ranges.

SQL Query Compatibility: Partitioning is transparent to the Data Science team. They can continue using their existing SQL queries with a WHERE clause based on the partitioning column without any modifications.

No Pipeline Changes: While recreating the tables is needed, the existing data pipeline that delivers daily data can remain mostly unchanged. The new data can be directly written to the appropriate partition.

Why other options are less ideal:

B (Export to CSV/Cloud Datalab): This is a workaround, not a solution. It requires the Data Science team to learn new tools and processes, and introduces data management overhead outside of BigQuery. It moves the workload to the client rather than optimizing server-side processing.

C (Separate Tables for Recent/Historical Data): This requires significant changes to the existing pipeline to maintain two tables. It can also complicate querying across the entire dataset.

D (Daily Tables with Apache Beam/Wildcards): While technically feasible, creating a table per day introduces a large number of small tables, which can negatively impact BigQuery's performance. Wildcard queries can be inefficient, and this approach is unnecessarily complex.

Partitioning is the most straightforward, cost-effective, and performance-enhancing solution that requires minimal changes to the existing data pipeline and query patterns. The data will continue to be ingested daily without any changes to its underlying structure besides the partitioning itself.

Relevant Links:

BigQuery Partitioned Tables: <https://cloud.google.com/bigquery/docs/partitioned-tables>

BigQuery Pricing: <https://cloud.google.com/bigquery/pricing>

Question: 112

CertyIQ

You operate a logistics company, and you want to improve event delivery reliability for vehicle-based sensors. You operate small data centers around the world to capture these events, but leased lines that provide connectivity from your event collection infrastructure to your event processing infrastructure are unreliable, with unpredictable latency. You want to address this issue in the most cost-effective way. What should you do?

- A. Deploy small Kafka clusters in your data centers to buffer events.
- B. Have the data acquisition devices publish data to Cloud Pub/Sub.
- C. Establish a Cloud Interconnect between all remote data centers and Google.
- D. Write a Cloud Dataflow pipeline that aggregates all data in session windows.

Answer: B

Explanation:

The most cost-effective solution for improving event delivery reliability with unreliable leased lines is **B. Have the data acquisition devices publish data to Cloud Pub/Sub.**

Here's why:

Pub/Sub as a Buffer: Cloud Pub/Sub acts as a globally distributed message queue. Data acquisition devices can publish events to Pub/Sub without directly relying on the continuous availability of the leased lines to the event processing infrastructure. Pub/Sub will buffer the messages until the processing infrastructure is available to consume them. This buffering decouples event producers from consumers, improving reliability.

Managed Service = Reduced Overhead: Pub/Sub is a fully managed service. This means Google handles the infrastructure, scaling, and reliability aspects of the message queue, reducing operational overhead

compared to managing your own Kafka clusters (Option A).

Cost-Effectiveness: Managed services often have a lower total cost of ownership compared to self-managed solutions. With Pub/Sub, you pay only for the data volume and operations you use.

No Need for Direct Interconnect: Option C, establishing Cloud Interconnect, is a costly solution. It requires dedicated physical connections, which negates the goal of cost-effectiveness and may not entirely address the intermittent nature of the leased line issues.

Dataflow is for Processing, not Buffering: Option D, using Dataflow for aggregation, is a processing step, not a reliable data ingestion and buffering mechanism. While Dataflow can handle windowing, it presupposes reliable data ingestion, which is the problem the scenario highlights. Dataflow also introduces complexity and cost that are unnecessary at the buffering stage.

In summary, Cloud Pub/Sub provides a cost-effective, managed, and reliable solution for decoupling event producers from consumers, mitigating the issues caused by unreliable leased lines and improving event delivery reliability.

Authoritative Links:

Cloud Pub/Sub Documentation: <https://cloud.google.com/pubsub/docs>

Pub/Sub Pricing: <https://cloud.google.com/pubsub/pricing>

Question: 113

CertyIQ

You are a retailer that wants to integrate your online sales capabilities with different in-home assistants, such as Google Home. You need to interpret customer voice commands and issue an order to the backend systems. Which solutions should you choose?

- A. Speech-to-Text API
- B. Cloud Natural Language API
- C. Dialogflow Enterprise Edition
- D. AutoML Natural Language

Answer: C

Explanation:

The correct answer is C, Dialogflow Enterprise Edition. This solution is ideal because it's specifically designed for building conversational interfaces, exactly what's needed to interpret voice commands from in-home assistants and trigger actions in backend systems. Dialogflow provides pre-built agents for common intents and entities, accelerating development. It also handles speech-to-text conversion implicitly, allowing you to focus on the conversational logic rather than low-level audio processing. Option A, Speech-to-Text API, is only responsible for transcribing audio into text, but does not interpret meaning or manage conversation flow. Cloud Natural Language API (B) focuses on text analysis such as sentiment and entity recognition, not conversational interaction. AutoML Natural Language (D) is used to build custom models for classifying and analyzing text but doesn't handle dialogue. Dialogflow leverages the power of Google's machine learning expertise to understand user intent from natural language. It also allows you to create context-aware conversations that maintain state across multiple turns. Dialogflow can be easily integrated with various platforms, including Google Home, as well as other backend systems, ensuring a seamless workflow for ordering via voice commands. Therefore, Dialogflow Enterprise Edition provides a comprehensive platform for building the desired interactive voice experience with strong integration capabilities.
<https://cloud.google.com/dialogflow>
<https://cloud.google.com/dialogflow/docs>
<https://cloud.google.com/speech-to-text>
<https://cloud.google.com/natural-language>
<https://cloud.google.com/automl/docs/natural-language>

Question: 114

CertyIQ

Your company has a hybrid cloud initiative. You have a complex data pipeline that moves data between cloud provider services and leverages services from each of the cloud providers. Which cloud-native service should you use to orchestrate the entire pipeline?

- A. Cloud Dataflow
- B. Cloud Composer
- C. Cloud Dataprep
- D. Cloud Dataproc

Answer: B

Explanation:

The correct answer is **B. Cloud Composer**. Here's why:

Cloud Composer is Google Cloud's fully managed workflow orchestration service built on Apache Airflow. It excels at managing complex, multi-step data pipelines that span different environments, including hybrid cloud setups. Unlike other options, Composer is designed specifically for orchestrating workflows, coordinating tasks, and scheduling jobs across various systems and cloud providers.

Cloud Dataflow (A) is a powerful data processing service for batch and stream data, but it's not primarily an orchestrator; it's more focused on the actual data transformations and computations. Cloud Dataprep (C) is a data preparation tool for cleaning and shaping data, not a workflow orchestrator. Cloud Dataproc (D) is a managed Hadoop and Spark service suitable for large-scale data processing but doesn't offer the same level of cross-cloud orchestration capabilities as Composer.

In a hybrid cloud environment, a data pipeline might involve extracting data from an on-premises database, transforming it using services on one cloud provider, and then loading it into services on another. Cloud Composer can seamlessly manage this complex flow by connecting to and coordinating with various services, including those outside of Google Cloud. It allows users to define workflows as Directed Acyclic Graphs (DAGs), schedule execution, monitor progress, and handle errors.

Therefore, Cloud Composer is the most suitable cloud-native service for orchestrating a complex data pipeline across different cloud providers, offering the necessary flexibility and connectivity needed in a hybrid cloud setup. Its focus on workflow management, scheduling, and interoperability makes it the ideal choice.

Authoritative Links:

Google Cloud Composer Documentation: <https://cloud.google.com/composer/docs>

Apache Airflow: <https://airflow.apache.org/>

Google Cloud Hybrid and Multi-Cloud: <https://cloud.google.com/hybrid>

Question: 115

CertyIQ

You use a dataset in BigQuery for analysis. You want to provide third-party companies with access to the same dataset. You need to keep the costs of data sharing low and ensure that the data is current. Which solution should you choose?

- A. Use Analytics Hub to control data access, and provide third party companies with access to the dataset.
- B. Use Cloud Scheduler to export the data on a regular basis to Cloud Storage, and provide third-party companies with access to the bucket.
- C. Create a separate dataset in BigQuery that contains the relevant data to share, and provide third-party

companies with access to the new dataset.

D. Create a Dataflow job that reads the data in frequent time intervals, and writes it to the relevant BigQuery dataset or Cloud Storage bucket for third-party companies to use.

Answer: A

Explanation:

The best solution is to use Analytics Hub (Option A) for sharing the BigQuery dataset with third-party companies. Analytics Hub is a fully managed service specifically designed for secure and efficient data sharing across organizations. It allows you to publish datasets and grant access to external entities without having to duplicate or physically move data. This significantly reduces costs associated with storage and data transfer. By utilizing Analytics Hub, you maintain a single source of truth for your data, ensuring third parties always have access to the most current information. Furthermore, Analytics Hub provides granular access controls, enabling you to manage who can view and query your data. This avoids the need to create new datasets (Option C) or manually move data using Cloud Scheduler (Option B) or Dataflow (Option D), all of which add unnecessary complexity and costs. Options B, C, and D all involve data duplication or movement, which increases storage and egress costs, goes against keeping costs low, and may cause delays in data updates. Therefore, Analytics Hub directly addresses the requirements of low cost and data freshness, making it the optimal solution.

Further research:

Google Cloud Analytics Hub Documentation: <https://cloud.google.com/analytics-hub/docs>

Sharing Data with Analytics Hub: <https://cloud.google.com/blog/products/data-analytics/share-bigquery-data-securely-with-analytics-hub>

Question: 116

CertyIQ

Your company is in the process of migrating its on-premises data warehousing solutions to BigQuery. The existing data warehouse uses trigger-based change data capture (CDC) to apply updates from multiple transactional database sources on a daily basis. With BigQuery, your company hopes to improve its handling of CDC so that changes to the source systems are available to query in BigQuery in near-real time using log-based CDC streams, while also optimizing for the performance of applying changes to the data warehouse. Which two steps should they take to ensure that changes are available in the BigQuery reporting table with minimal latency while reducing compute overhead? (Choose two.)

- A. Perform a DML INSERT, UPDATE, or DELETE to replicate each individual CDC record in real time directly on the reporting table.
- B. Insert each new CDC record and corresponding operation type to a staging table in real time.
- C. Periodically DELETE outdated records from the reporting table.
- D. Periodically use a DML MERGE to perform several DML INSERT, UPDATE, and DELETE operations at the same time on the reporting table.
- E. Insert each new CDC record and corresponding operation type in real time to the reporting table, and use a materialized view to expose only the newest version of each unique record.

Answer: BD

Explanation:

The correct answer is **B and D**. Here's why:

B. Insert each new CDC record and corresponding operation type to a staging table in real time. This is crucial for a near real-time CDC pipeline. Instead of directly manipulating the target reporting table, a staging table acts as a buffer. It allows for efficient ingestion of CDC records without impacting the reporting table's query performance. This approach decouples the ingestion and transformation stages. Think of it as a landing

zone for incoming changes.

D. Periodically use a DML MERGE to perform several DML INSERT, UPDATE, and DELETE operations at the same time on the reporting table. Once CDC data is in the staging table, batch processing using the MERGE statement becomes effective. The MERGE statement combines insertion, updating, and deletion logic, enabling optimized data manipulation within a single command. This reduces the overhead compared to performing individual DML operations for each record. It also ensures atomicity, meaning all the changes of a batch either occur or don't. This avoids partial updates in case of errors.

Let's examine why the other options are less suitable:

A. Perform a DML INSERT, UPDATE, or DELETE to replicate each individual CDC record in real time directly on the reporting table. Performing individual DML operations for each record is highly inefficient for large datasets. This leads to high compute overhead and potentially slow query times on the reporting table due to excessive writes.

C. Periodically DELETE outdated records from the reporting table. While deleting outdated records is part of CDC management, it's not the primary step to achieving real-time updates. Furthermore, this should be part of the MERGE statement rather than a separate operation.

E. Insert each new CDC record and corresponding operation type in real time to the reporting table, and use a materialized view to expose only the newest version of each unique record. Inserting directly into the reporting table with individual inserts will have the same performance issues as option A. While materialized views can help with query performance, they do not solve the problem of writing directly to the table inefficiently.

By using a staging table for real-time ingestion and then applying bulk updates using MERGE, the proposed solution ensures timely availability of data in the reporting table with significantly reduced overhead.

Authoritative Links for further research:

BigQuery MERGE statement: https://cloud.google.com/bigquery/docs/reference/standard-sql/dml-syntax#merge_statement

Change Data Capture (CDC) best practices with Google Cloud: <https://cloud.google.com/architecture/real-time-data-analysis-with-change-data-capture>

BigQuery DML Statements: <https://cloud.google.com/bigquery/docs/data-manipulation-language>

Question: 117

CertyIQ

You are designing a data processing pipeline. The pipeline must be able to scale automatically as load increases. Messages must be processed at least once and must be ordered within windows of 1 hour. How should you design the solution?

- A. Use Apache Kafka for message ingestion and use Cloud Dataproc for streaming analysis.
- B. Use Apache Kafka for message ingestion and use Cloud Dataflow for streaming analysis.
- C. Use Cloud Pub/Sub for message ingestion and Cloud Dataproc for streaming analysis.
- D. Use Cloud Pub/Sub for message ingestion and Cloud Dataflow for streaming analysis.

Answer: D

Explanation:

The correct answer is D. Here's a detailed justification:

Why D is the best option:

Cloud Pub/Sub: Pub/Sub is Google Cloud's fully managed, real-time messaging service. It's ideal for ingesting

data because it's highly scalable, durable, and supports at-least-once message delivery, fulfilling the requirement of "messages must be processed at least once."

Cloud Dataflow: Dataflow is Google Cloud's fully managed, unified stream and batch processing service. It excels at automatically scaling data processing pipelines. Dataflow also offers robust windowing capabilities, allowing you to group messages into time-based windows (in this case, 1-hour windows) to maintain order within those windows.

Why other options are less suitable:

A & B (Apache Kafka): While Kafka is a valid messaging system, choosing Cloud Pub/Sub offers the benefit of a fully managed service, reducing operational overhead and management complexity. This aligns well with a cloud-native approach.

A & C (Cloud Dataproc): Dataproc is suitable for batch processing and running Hadoop/Spark jobs, but Dataflow is better suited for real-time, streaming data processing with automatic scaling. Dataflow's inherent stream processing capabilities and windowing support make it a more natural fit for the specified requirements.

In summary:

Option D (Pub/Sub and Dataflow) provides a fully managed, scalable, and reliable solution that directly addresses the requirements of at-least-once message processing, automatic scaling, and ordered processing within 1-hour windows. Pub/Sub ensures message delivery, and Dataflow provides the processing power and windowing features needed for stream analytics.

Supporting documentation:

Cloud Pub/Sub: <https://cloud.google.com/pubsub/docs>

Cloud Dataflow: <https://cloud.google.com/dataflow/docs>

Question: 118

CertyIQ

You need to set access to BigQuery for different departments within your company. Your solution should comply with the following requirements:

- Each department should have access only to their data.
- Each department will have one or more leads who need to be able to create and update tables and provide them to their team.
- Each department has data analysts who need to be able to query but not modify data.

How should you set access to the data in BigQuery?

- A. Create a dataset for each department. Assign the department leads the role of OWNER, and assign the data analysts the role of WRITER on their dataset.
- B. Create a dataset for each department. Assign the department leads the role of WRITER, and assign the data analysts the role of READER on their dataset.
- C. Create a table for each department. Assign the department leads the role of Owner, and assign the data analysts the role of Editor on the project the table is in.
- D. Create a table for each department. Assign the department leads the role of Editor, and assign the data analysts the role of Viewer on the project the table is in.

Answer: B

Explanation:

Okay, let's break down why option B is the correct answer for managing BigQuery access across departments.

The core principle here is **least privilege**, granting only the necessary permissions to each user group. BigQuery uses datasets as a logical container for tables, which aligns well with the departmental data

segregation requirement.

Option B proposes creating a dedicated dataset for each department, enabling data isolation. Assigning the "WRITER" role to department leads allows them to create, modify, and manage tables within their respective datasets – fulfilling their need to structure and update data. Crucially, it does not grant ownership over the dataset itself, a crucial security distinction. The "READER" role for data analysts provides them with the ability to query the data but prevents accidental or malicious modifications, ensuring data integrity.

Option A is incorrect because the "OWNER" role grants excessive permissions (including managing the dataset and its access policies), which is unnecessary for departmental leads who primarily need to manage tables within the dataset. Furthermore, the "WRITER" role on a dataset does not grant the ability to modify dataset level security policies. Options C and D are flawed because they propose using tables for segregation, which is not scalable and lacks the inherent logical separation provided by datasets. Project-level roles are also too broad and would violate the principle of least privilege. Datasets are the logical unit to assign permission. Table level permission should be assigned when data level visibility is required across departments.

In short, option B provides the correct level of access for both departmental leads and data analysts by utilizing datasets for logical segregation and applying appropriate BigQuery roles.

Authoritative Links:

BigQuery Access Control: <https://cloud.google.com/bigquery/docs/access-control>

BigQuery IAM Roles: <https://cloud.google.com/bigquery/docs/access-control-iam>

Understanding BigQuery Datasets: <https://cloud.google.com/bigquery/docs/datasets>

Question: 119

CertyIQ

You operate a database that stores stock trades and an application that retrieves average stock price for a given company over an adjustable window of time. The data is stored in Cloud Bigtable where the datetime of the stock trade is the beginning of the row key. Your application has thousands of concurrent users, and you notice that performance is starting to degrade as more stocks are added. What should you do to improve the performance of your application?

- A. Change the row key syntax in your Cloud Bigtable table to begin with the stock symbol.
- B. Change the row key syntax in your Cloud Bigtable table to begin with a random number per second.
- C. Change the data pipeline to use BigQuery for storing stock trades, and update your application.
- D. Use Cloud Dataflow to write a summary of each day's stock trades to an Avro file on Cloud Storage. Update your application to read from Cloud Storage and Cloud Bigtable to compute the responses.

Answer: A

Explanation:

The problem describes a performance bottleneck in a Cloud Bigtable application due to increasing data volume and concurrent user load. The current row key structure, beginning with datetime, creates a "hotspotting" issue. Bigtable distributes data based on row keys, and sequential timestamps cause writes and reads to concentrate on a single tablet server, leading to performance degradation. Option A, changing the row key to start with the stock symbol, addresses this by distributing data across tablet servers based on stock rather than time. This better utilizes Bigtable's horizontal scaling capabilities and avoids concentrating requests on a small number of servers. Option B, using a random number, would distribute writes, but then reads for a specific stock would need to scatter across the entire table making reads inefficient. Option C, migrating to BigQuery, is an unnecessary overhaul of the system and introduces complexity by requiring an entirely different querying approach, impacting real-time calculations. Option D, utilizing Cloud Dataflow to create daily summaries, might improve performance for daily aggregations, but does not address the

underlying issue for users wanting custom time windows and therefore does not solve the direct problem. Since the application requires data for arbitrary time windows, the daily summaries from option D will not cover those use cases. Therefore, A is the best option by evenly distributing read/write operations across Bigtable using the stock symbol as the primary element of the row key, ensuring a balanced workload and optimal performance. This aligns with the best practices for designing Bigtable schemas.

Authoritative Links:

[Designing your schema](#)

[Key visualizer: Diagnose performance issues](#)

Question: 120

CertyIQ

You are operating a Cloud Dataflow streaming pipeline. The pipeline aggregates events from a Cloud Pub/Sub subscription source, within a window, and sinks the resulting aggregation to a Cloud Storage bucket. The source has consistent throughput. You want to monitor an alert on behavior of the pipeline with Cloud Stackdriver to ensure that it is processing data. Which Stackdriver alerts should you create?

- A. An alert based on a decrease of subscription/num_undelivered_messages for the source and a rate of change increase of instance/storage/used_bytes for the destination
- B. An alert based on an increase of subscription/num_undelivered_messages for the source and a rate of change decrease of instance/storage/used_bytes for the destination
- C. An alert based on a decrease of instance/storage/used_bytes for the source and a rate of change increase of subscription/num_undelivered_messages for the destination
- D. An alert based on an increase of instance/storage/used_bytes for the source and a rate of change decrease of subscription/num_undelivered_messages for the destination

Answer: B

Explanation:

The correct answer is **B**. Here's a detailed justification:

The scenario describes a streaming Dataflow pipeline consuming data from a Cloud Pub/Sub subscription and writing aggregated results to Cloud Storage. To monitor pipeline health and ensure data processing, we need to track both the input and output.

Option B suggests monitoring an increase in subscription/num_undelivered_messages. This is crucial because an increase in undelivered messages indicates that the Dataflow pipeline might not be processing messages as quickly as they arrive, potentially signaling a bottleneck or a pipeline issue. Simultaneously, monitoring a decrease in the rate of change of instance/storage/used_bytes for the destination (Cloud Storage) is important. A decrease in the rate of storage growth means the pipeline is writing data at a slower pace, likely correlated with the backlog at the input stage. This combination effectively detects a pipeline stall or processing issue.

Option A is incorrect because a decrease in subscription/num_undelivered_messages indicates the pipeline is catching up with the source and doesn't indicate any issues, it's a positive sign. Furthermore, an increase in storage usage rate is not a signal of a problem by itself. Option C is incorrect, as instance/storage/used_bytes is a property of Cloud Storage (destination) not of the Cloud Pub/Sub subscription (source). Furthermore, a decrease in storage usage is not the problem in our scenario. Option D incorrectly suggests an increase of storage usage in the source. Also, a decrease in undelivered messages is a positive sign and does not indicate a problem.

Monitoring these two metrics (backlog in Pub/Sub and slow output to Cloud Storage) together provides a comprehensive view of the pipeline's health. An increase in Pub/Sub backlog and a decrease in storage output growth strongly suggests the pipeline is unable to keep up with the incoming stream. Cloud Stackdriver is the

appropriate service for these types of alerts, providing timely notification of issues.

Authoritative Links:

Cloud Monitoring Metrics: <https://cloud.google.com/monitoring/api/metrics>

Pub/Sub Metrics: <https://cloud.google.com/pubsub/docs/monitoring-with-cloud-monitoring>

Dataflow Monitoring: <https://cloud.google.com/dataflow/docs/guides/using-monitoring-dashboard>

Stackdriver Alerting: <https://cloud.google.com/monitoring/alerts>

Question: 121

CertyIQ

You currently have a single on-premises Kafka cluster in a data center in the us-east region that is responsible for ingesting messages from IoT devices globally.

Because large parts of globe have poor internet connectivity, messages sometimes batch at the edge, come in all at once, and cause a spike in load on your Kafka cluster. This is becoming difficult to manage and prohibitively expensive. What is the Google-recommended cloud native architecture for this scenario?

- A. Edge TPUs as sensor devices for storing and transmitting the messages.
- B. Cloud Dataflow connected to the Kafka cluster to scale the processing of incoming messages.
- C. An IoT gateway connected to Cloud Pub/Sub, with Cloud Dataflow to read and process the messages from Cloud Pub/Sub.
- D. A Kafka cluster virtualized on Compute Engine in us-east with Cloud Load Balancing to connect to the devices around the world.

Answer: C

Explanation:

Option C is the recommended cloud-native architecture because it addresses the core issues of intermittent connectivity and load spikes outlined in the scenario. An IoT gateway acts as a buffer at the edge, aggregating messages from devices even during connectivity outages. This allows for resilient message collection. The gateway then forwards these messages to Cloud Pub/Sub, a globally scalable, fully managed messaging service. Pub/Sub decouples the ingestion of data from its processing, effectively absorbing the bursty load from edge devices. Finally, Cloud Dataflow reads from Pub/Sub and provides robust stream processing capabilities, allowing for flexible and scalable analysis of the incoming data. This pipeline isolates the Kafka cluster and prevents it from being overloaded, improving system stability and reducing costs.

Option A is unsuitable because Edge TPUs are specialized hardware for machine learning, not ideal for general message storage and transmission. Option B, while using Dataflow, fails to address the crucial problem of buffering messages at the edge and still relies on direct Kafka interaction leading to the same load issues. Option D, virtualizing Kafka in the cloud, does not remove the core challenges and can be more expensive without the benefits of serverless offerings like Pub/Sub. The selected approach offers a flexible, decoupled, and scalable solution, aligning with Google Cloud's best practices for IoT workloads.

Supporting Links:

Cloud IoT Core (Gateway concept): While IoT Core is now deprecated, it provided good documentation on the need for gateways. <https://cloud.google.com/iot-core/docs> (Though deprecated, concepts still apply.)

Cloud Pub/Sub: <https://cloud.google.com/pubsub/docs> (For more on message decoupling and scalability)

Cloud Dataflow: <https://cloud.google.com/dataflow/docs> (For stream processing capabilities)

Google Cloud IoT solutions: <https://cloud.google.com/solutions/iot>

Question: 122

You decided to use Cloud Datastore to ingest vehicle telemetry data in real time. You want to build a storage system that will account for the long-term data growth, while keeping the costs low. You also want to create snapshots of the data periodically, so that you can make a point-in-time (PIT) recovery, or clone a copy of the data for Cloud Datastore in a different environment. You want to archive these snapshots for a long time. Which two methods can accomplish this? (Choose two.)

- A. Use managed export, and store the data in a Cloud Storage bucket using Nearline or Coldline class.
- B. Use managed export, and then import to Cloud Datastore in a separate project under a unique namespace reserved for that export.
- C. Use managed export, and then import the data into a BigQuery table created just for that export, and delete temporary export files.
- D. Write an application that uses Cloud Datastore client libraries to read all the entities. Treat each entity as a BigQuery table row via BigQuery streaming insert. Assign an export timestamp for each export, and attach it as an extra column for each row. Make sure that the BigQuery table is partitioned using the export timestamp column.
- E. Write an application that uses Cloud Datastore client libraries to read all the entities. Format the exported data into a JSON file. Apply compression before storing the data in Cloud Source Repositories.

Answer: AB

Explanation:

The correct options are A and B. Let's examine why.

Option A, using managed export to Cloud Storage, is a suitable approach for long-term, cost-effective storage of Datastore snapshots. Cloud Storage Nearline and Coldline offer lower storage costs for infrequently accessed data, making them ideal for archives. Managed export simplifies the process, handling the transfer of data from Datastore to Cloud Storage efficiently. This addresses the requirements of long-term growth and cost minimization.

Option B, using managed export to a separate Datastore project with unique namespaces, allows for point-in-time recovery and cloning. By exporting snapshots to a different project and using a distinct namespace, you maintain an isolated and restorable copy of the data at the time of the export. This facilitates PIT recovery by restoring a specific namespace from the snapshot, and cloning the data by replicating the specific namespace on another environment.

Options C, D, and E are not ideal in this context: Option C, importing Datastore data to BigQuery just for export, does not provide a direct mechanism for point-in-time restoration back to Cloud Datastore, negating the requirement for PIT recovery. Option D, using client libraries to stream data to BigQuery, is not a managed solution and would require significant operational overhead and more complex application code. Option E, using client libraries, formatting JSON, and storing in Cloud Source Repositories, is not a practical approach for large datasets because Cloud Source Repositories are mainly used for code storage and versioning, not for large data archiving.

Therefore, using managed export to Cloud Storage (A) for archiving and managed export to a separate Datastore project (B) with distinct namespaces for PIT recovery satisfies the requirements in an efficient and managed manner.

Authoritative Links:

Cloud Datastore Managed Export/Import: <https://cloud.google.com/datastore/docs/export-import-entities>
Cloud Storage Storage Classes: <https://cloud.google.com/storage/docs/storage-classes>

Question: 123

You need to create a data pipeline that copies time-series transaction data so that it can be queried from within BigQuery by your data science team for analysis. Every hour, thousands of transactions are updated with a new status. The size of the initial dataset is 1.5 PB, and it will grow by 3 TB per day. The data is heavily structured, and your data science team will build machine learning models based on this data. You want to maximize performance and usability for your data science team. Which two strategies should you adopt? (Choose two.)

- A. Denormalize the data as much as possible.
- B. Preserve the structure of the data as much as possible.
- C. Use BigQuery UPDATE to further reduce the size of the dataset.
- D. Develop a data pipeline where status updates are appended to BigQuery instead of updated.
- E. Copy a daily snapshot of transaction data to Cloud Storage and store it as an Avro file. Use BigQuery's support for external data sources to query.

Answer: AD

Explanation:

The correct strategies are A and D. Denormalizing the data (A) is crucial for analytical workloads in BigQuery. Joins across multiple tables can be expensive and slow down queries. By denormalizing, you pre-combine the necessary data into fewer tables, making analysis faster and more efficient for the data science team. This aligns with BigQuery's columnar storage and its optimized performance for analytical queries. Secondly, appending updates (D) instead of using UPDATE operations in BigQuery is vital for performance and cost-effectiveness. UPDATE operations in BigQuery can be resource-intensive as they involve rewriting large amounts of data. Appending new data and maintaining history allows for efficient processing and avoids these expensive rewrites. It also provides an audit trail of changes. Option B, preserving structure, hinders analytical query performance. Option C, using UPDATE to reduce the dataset, is not an efficient approach and directly contradicts BigQuery best practices due to performance implications and associated costs. Option E, using external sources for querying, does not leverage BigQuery's capabilities effectively, especially with the size of the dataset which will be costly due to data transfers and slower query times.

Relevant Links:

BigQuery Best Practices - Denormalization: <https://cloud.google.com/bigquery/docs/best-practices-performance#denormalization>

BigQuery Best Practices - DML Performance: https://cloud.google.com/bigquery/docs/best-practices-dml#dml_performance

BigQuery data loading: <https://cloud.google.com/bigquery/docs/loading-data>

Question: 124

You are designing a cloud-native historical data processing system to meet the following conditions:

- The data being analyzed is in CSV, Avro, and PDF formats and will be accessed by multiple analysis tools including Dataproc, BigQuery, and Compute Engine.
- A batch pipeline moves daily data.
- Performance is not a factor in the solution.
- The solution design should maximize availability.

How should you design data storage for this solution?

- A. Create a Dataproc cluster with high availability. Store the data in HDFS, and perform analysis as needed.
- B. Store the data in BigQuery. Access the data using the BigQuery Connector on Dataproc and Compute Engine.
- C. Store the data in a regional Cloud Storage bucket. Access the bucket directly using Dataproc, BigQuery, and Compute Engine.

D. Store the data in a multi-regional Cloud Storage bucket. Access the data directly using Dataproc, BigQuery, and Compute Engine.

Answer: D

Explanation:

The correct answer is D: "Store the data in a multi-regional Cloud Storage bucket. Access the bucket directly using Dataproc, BigQuery, and Compute Engine." Here's why:

The primary requirement is to maximize availability. Multi-regional Cloud Storage inherently provides the highest level of availability by replicating data across multiple geographic regions. This ensures that even if one region experiences an outage, the data remains accessible from other regions. This directly addresses the availability requirement.

Cloud Storage is suitable for storing various file formats (CSV, Avro, PDF). Both Dataproc, BigQuery, and Compute Engine can directly access data in Cloud Storage using appropriate connectors or libraries. Dataproc can use the Cloud Storage connector for Hadoop. BigQuery can directly query data in Cloud Storage using external tables. Compute Engine instances can access Cloud Storage via the Google Cloud Storage client libraries.

Storing in BigQuery (option B) would require ingesting all data into BigQuery which might not be desirable, especially for PDF files or if certain tools need direct file access. It also adds cost. Storing data in HDFS (option A) introduces single points of failure, impacting availability. Regional Cloud Storage (option C) offers better availability than HDFS, but is less resilient compared to multi-regional Cloud Storage in case of regional outages.

The requirement mentions performance is not a factor. Therefore, multi-regional Cloud Storage's slightly higher latency (due to geographic distribution) isn't a concern. The simplicity of accessing the data from different services directly from Cloud Storage, combined with the highest availability provided by multi-regional storage, makes it the ideal solution.

Further reading:

Cloud Storage Availability: <https://cloud.google.com/storage/docs/locations>

BigQuery External Tables: <https://cloud.google.com/bigquery/docs/external-data-cloud-storage>

Cloud Storage Connector for Hadoop: <https://cloud.google.com/dataproc/docs/concepts/connectors/cloud-storage>

Question: 125

CertyIQ

You have a petabyte of analytics data and need to design a storage and processing platform for it. You must be able to perform data warehouse-style analytics on the data in Google Cloud and expose the dataset as files for batch analysis tools in other cloud providers. What should you do?

- A. Store and process the entire dataset in BigQuery.
- B. Store and process the entire dataset in Bigtable.
- C. Store the full dataset in BigQuery, and store a compressed copy of the data in a Cloud Storage bucket.
- D. Store the warm data as files in Cloud Storage, and store the active data in BigQuery. Keep this ratio as 80% warm and 20% active.

Answer: C

Explanation:

The correct answer is C because it balances the requirements of data warehousing style analytics within

Google Cloud and file-based access for external batch processing.

Here's a detailed justification:

BigQuery for Analytics: BigQuery is Google Cloud's fully managed, serverless data warehouse optimized for large-scale data analytics. It's perfectly suited for performing data warehouse-style analytics on petabyte-scale datasets due to its MPP (Massively Parallel Processing) architecture, SQL support, and efficient querying capabilities.<https://cloud.google.com/bigquery/docs>

Cloud Storage for File Access: Storing a compressed copy of the data in Cloud Storage provides a cost-effective and accessible way to make the dataset available as files for batch analysis tools in other cloud providers. Cloud Storage offers object storage with high durability, availability, and scalability. Compression further reduces storage costs and transfer times.<https://cloud.google.com/storage/docs>

Why not Bigtable? Bigtable is a NoSQL wide-column database designed for low-latency, high-throughput workloads. While it can handle large datasets, it's not ideal for data warehouse-style analytics, which typically involve complex queries and aggregations that are more efficiently handled by a data warehouse like BigQuery.

Why not solely BigQuery? While feasible, solely using BigQuery would make directly exposing data as files to external cloud providers more complex. It would require exporting the data from BigQuery into a file format, which adds an extra step and potential overhead. Having a direct copy in Cloud Storage is more efficient.

Why not a tiered approach with warm/active data? While a tiered approach can optimize costs, the problem statement doesn't indicate any access frequency differences significant enough to justify the added complexity of splitting the data between Cloud Storage and BigQuery. Keeping the full dataset in both systems is the most straightforward approach.

In summary, storing the full dataset in BigQuery caters to internal analytical needs, while the compressed copy in Cloud Storage fulfills the requirement of providing file-based access for external tools, making option C the most efficient and effective solution.

Question: 126

CertyIQ

You work for a manufacturing company that sources up to 750 different components, each from a different supplier. You've collected a labeled dataset that has on average 1000 examples for each unique component. Your team wants to implement an app to help warehouse workers recognize incoming components based on a photo of the component. You want to implement the first working version of this app (as Proof-Of-Concept) within a few working days. What should you do?

- A. Use Cloud Vision AutoML with the existing dataset.
- B. Use Cloud Vision AutoML, but reduce your dataset twice.
- C. Use Cloud Vision API by providing custom labels as recognition hints.
- D. Train your own image recognition model leveraging transfer learning techniques.

Answer: A

Explanation:

The best approach for quickly building a proof-of-concept component recognition app, given the limited timeframe and existing dataset, is to utilize Cloud Vision AutoML (Option A). Cloud Vision AutoML is designed for custom image classification tasks and allows you to train a model using your labeled dataset without requiring extensive machine learning expertise. With 1000 examples per component on average, the dataset is reasonably sized for AutoML, making it a good fit for a first version. This approach significantly reduces development time compared to building and training a custom model from scratch. Option B, reducing the

dataset twice, would likely degrade model performance unnecessarily and isn't advisable for a first version. Option C, using the base Cloud Vision API with hints, is not suitable for recognizing 750 different custom components as the API is designed for general-purpose object recognition and not specific product identification. Option D, building a custom model, is feasible but would take much longer than a few days, thus contradicting the proof of concept timeline. Cloud Vision AutoML streamlines the model development process, allowing the team to quickly deploy a functional app for initial testing and validation. This approach provides a faster path to demonstrating value and gathering initial user feedback. AutoML leverages Google's pre-existing infrastructure and expertise in machine learning, ensuring efficiency and accuracy within a short timeframe.

Supporting Links:

Cloud Vision AutoML Documentation: <https://cloud.google.com/vision/automl/docs>

Cloud Vision API Documentation: <https://cloud.google.com/vision/docs>

Question: 127

CertyIQ

You are working on a niche product in the image recognition domain. Your team has developed a model that is dominated by custom C++ TensorFlow ops your team has implemented. These ops are used inside your main training loop and are performing bulky matrix multiplications. It currently takes up to several days to train a model. You want to decrease this time significantly and keep the cost low by using an accelerator on Google Cloud. What should you do?

- A. Use Cloud TPUs without any additional adjustment to your code.
- B. Use Cloud TPUs after implementing GPU kernel support for your custom ops.
- C. Use Cloud GPUs after implementing GPU kernel support for your custom ops.
- D. Stay on CPUs, and increase the size of the cluster you're training your model on.

Answer: C

Explanation:

The correct answer is **C. Use Cloud GPUs after implementing GPU kernel support for your custom ops.** Here's the justification:

The core issue is the lengthy training time due to computationally intensive matrix multiplications within custom C++ TensorFlow ops. Cloud GPUs are well-suited for parallel processing of such operations, drastically accelerating training. However, the custom ops are designed for CPUs and don't automatically utilize GPUs. To leverage GPUs, these ops require specific GPU kernel implementations.

Option A is incorrect because Cloud TPUs, while highly powerful, have specific programming requirements, and custom ops need to be ported to their architecture, a more complex undertaking than GPU kernels. Option B is incorrect because it suggests using TPUs which are complex for custom ops. Option D, using only CPUs and scaling the cluster, would improve performance to a degree, but would be less effective and likely more expensive than accelerating operations with GPUs.

Therefore, implementing GPU kernel support for custom ops and running on Cloud GPUs provides the most efficient way to reduce training time and improve performance, as this addresses the computational bottleneck directly. GPUs excel at the type of matrix operations used here and are easier to use with TF ops compared to TPUs. This approach will likely be more cost-effective than scaling out a large CPU cluster.

Authoritative Links:

TensorFlow Custom Ops: <https://www.tensorflow.org/guide/extend/op>

Google Cloud GPUs: <https://cloud.google.com/gpu>

Question: 128**CertyIQ**

You work on a regression problem in a natural language processing domain, and you have 100M labeled examples in your dataset. You have randomly shuffled your data and split your dataset into train and test samples (in a 90/10 ratio). After you trained the neural network and evaluated your model on a test set, you discover that the root-mean-squared error (RMSE) of your model is twice as high on the train set as on the test set. How should you improve the performance of your model?

- A. Increase the share of the test sample in the train-test split.
- B. Try to collect more data and increase the size of your dataset.
- C. Try out regularization techniques (e.g., dropout or batch normalization) to avoid overfitting.
- D. Increase the complexity of your model by, e.g., introducing an additional layer or increase the size of vocabularies or n-grams used.

Answer: D**Explanation:**

The problem described indicates underfitting, not overfitting, which is counterintuitive given the typical concern of models performing better on training data than test data. The higher RMSE on the training set suggests the model is not learning the underlying patterns in the training data effectively. This indicates that the model is too simple to capture the complexity of the relationship between the features and the target variable. Options A and B are not the primary focus because data split ratio and size are not relevant here. Option C deals with regularization which mitigates overfitting, which is not the issue as defined in the problem statement. Option D addresses underfitting by increasing model complexity, such as adding layers or enlarging vocabularies, giving the model more ability to capture intricate relationships in the data. These adjustments make the model more capable of learning the training data, hence reducing training error and ultimately improving performance. Since the training error is much higher than testing error, indicating underfitting. Therefore, D is the most appropriate solution.

Further Research:

Underfitting vs Overfitting: <https://www.geeksforgeeks.org/underfitting-and-overfitting-in-machine-learning/>

Model Complexity: <https://machinelearningmastery.com/model-complexity-for-machine-learning-algorithms/>

Neural Network Architecture: <https://developers.google.com/machine-learning/crash-course/ml-intro/check-your-intuition>

Question: 129**CertyIQ**

You use BigQuery as your centralized analytics platform. New data is loaded every day, and an ETL pipeline modifies the original data and prepares it for the final users. This ETL pipeline is regularly modified and can generate errors, but sometimes the errors are detected only after 2 weeks. You need to provide a method to recover from these errors, and your backups should be optimized for storage costs. How should you organize your data in BigQuery and store your backups?

- A. Organize your data in a single table, export, and compress and store the BigQuery data in Cloud Storage.
- B. Organize your data in separate tables for each month, and export, compress, and store the data in Cloud Storage.
- C. Organize your data in separate tables for each month, and duplicate your data on a separate dataset in

BigQuery.

D. Organize your data in separate tables for each month, and use snapshot decorators to restore the table to a time prior to the corruption.

Answer: B

Explanation:

The optimal solution is to organize data into monthly tables and back them up to Cloud Storage (Option B). This approach is superior to option A as it offers more granular recovery. Having one large table makes recovery complex and costly, requiring potential restoration of the entire table even for localized errors. Monthly tables, on the other hand, enable restoration of only affected months, thus reducing time and costs. While option C suggests using a separate dataset, it incurs higher storage costs and might lead to difficulties in managing two large datasets. Additionally, using snapshots (option D) is not suitable for long term recovery due to their limitations regarding retention and potential performance impact when dealing with large datasets. Exporting to Cloud Storage provides a cost-effective, independent backup mechanism. Furthermore, compression when exporting minimizes storage consumption and transfer times. This allows for a consistent and cost-optimized backup strategy that can quickly restore data affected by pipeline errors. Cloud Storage is designed for scalable and cost-effective object storage, perfect for housing these compressed backups.

Authoritative Links:

BigQuery Storage Optimization: <https://cloud.google.com/bigquery/docs/best-practices-storage>

BigQuery Export Data: <https://cloud.google.com/bigquery/docs/exporting-data>

Cloud Storage Best Practices: <https://cloud.google.com/storage/docs/best-practices>

Question: 130

CertyIQ

The marketing team at your organization provides regular updates of a segment of your customer dataset. The marketing team has given you a CSV with 1 million records that must be updated in BigQuery. When you use the UPDATE statement in BigQuery, you receive a quotaExceeded error. What should you do?

- A. Reduce the number of records updated each day to stay within the BigQuery UPDATE DML statement limit.
- B. Increase the BigQuery UPDATE DML statement limit in the Quota management section of the Google Cloud Platform Console.
- C. Split the source CSV file into smaller CSV files in Cloud Storage to reduce the number of BigQuery UPDATE DML statements per BigQuery job.
- D. Import the new records from the CSV file into a new BigQuery table. Create a BigQuery job that merges the new records with the existing records and writes the results to a new BigQuery table.

Answer: D

Explanation:

The correct answer is **D** because it addresses the underlying issue of BigQuery's limitations with large-scale UPDATE operations and provides a more efficient solution. BigQuery's DML (Data Manipulation Language) statements, particularly UPDATE, are not designed for frequent large-scale modifications. They are optimized for smaller, targeted updates. Attempting to update 1 million records with a single UPDATE statement will likely exceed the quotas associated with these operations, resulting in a quotaExceeded error. Option A, while technically feasible, would slow down the data integration significantly and not be an ideal approach in a scalable system. Option B is incorrect because BigQuery quotas for DML statements like UPDATE are not generally adjustable by users in this manner. Option C would not solve the problem, as the issue is with the size of the operation itself rather than the CSV file size. Importing the data into a new table and then

performing a merge into a new table (Option D) using a MERGE statement or a similar operation is a more efficient and performant approach for large datasets. This leverages BigQuery's columnar storage and distributed processing capabilities to handle large data merges. It avoids large, problematic UPDATE statements that are resource intensive and prone to exceeding quotas. Option D aligns with BigQuery's best practices for large batch updates. It creates a new table with the merged data which is then available for use.**Relevant Links:**

[BigQuery DML Statements](#)

[BigQuery Quotas & Limits](#)

[BigQuery MERGE Statement](#)

Question: 131

CertyIQ

As your organization expands its usage of GCP, many teams have started to create their own projects. Projects are further multiplied to accommodate different stages of deployments and target audiences. Each project requires unique access control configurations. The central IT team needs to have access to all projects. Furthermore, data from Cloud Storage buckets and BigQuery datasets must be shared for use in other projects in an ad hoc way. You want to simplify access control management by minimizing the number of policies. Which two steps should you take? (Choose two.)

- A. Use Cloud Deployment Manager to automate access provision.
- B. Introduce resource hierarchy to leverage access control policy inheritance.
- C. Create distinct groups for various teams, and specify groups in Cloud IAM policies.
- D. Only use service accounts when sharing data for Cloud Storage buckets and BigQuery datasets.
- E. For each Cloud Storage bucket or BigQuery dataset, decide which projects need access. Find all the active members who have access to these projects, and create a Cloud IAM policy to grant access to all these users.

Answer: BC

Explanation:

Here's a detailed justification for why options B and C are the correct choices for simplifying access control management in the scenario described, while options A, D, and E are less suitable:

Option B: Introduce resource hierarchy to leverage access control policy inheritance. This is crucial for centralized management. GCP's resource hierarchy (Organization, Folders, Projects) enables you to set IAM policies at higher levels, which then inherit down to child resources. By placing all projects under a common organization or folder, the central IT team can be granted broad access at the higher level. This avoids having to manage individual access permissions on every project, significantly reducing complexity. The concept of policy inheritance is fundamental to efficient IAM management in GCP. <https://cloud.google.com/resource-manager/docs/cloud-platform-resource-hierarchy>

Option C: Create distinct groups for various teams, and specify groups in Cloud IAM policies. Managing access using Google Groups, instead of individual user accounts, promotes scalability and ease of maintenance. Adding or removing members from a group automatically updates access permissions across all resources where the group is granted access. The central IT team can have their own group, simplifying its access management across all projects, and making it easier to update permissions for individuals joining or leaving the team. This avoids dealing with a long list of individual users in IAM policies and reduces the risk of errors. <https://cloud.google.com/iam/docs/overview>

Why other options are incorrect:

Option A: Use Cloud Deployment Manager to automate access provision. While automation is good, Cloud Deployment Manager is primarily for infrastructure deployment and not the best tool for managing recurring, dynamic IAM access. It also does not address the core problem of decentralized policy management. It can be

helpful in consistently provisioning access, but it is not central to the solution, unlike policy inheritance and group membership.

Option D: Only use service accounts when sharing data... Service accounts are essential for automated access, but they do not solve the problem of managing access for the central IT team or simplify ad hoc data sharing for other projects, the core issues in the given scenario. They add extra complexity and do not simplify overall user access management. Service accounts should be used for applications and services, not directly for individual team members' access.

Option E: For each Cloud Storage bucket or BigQuery dataset, decide which projects need access... This creates a highly fragmented and complex management scheme. It goes against the principle of centralized control and promotes policy sprawl. It would also require constant updates every time a new project or team member requires access. This approach is not scalable and is very error-prone. In conclusion, combining the resource hierarchy for policy inheritance (B) with group-based access management (C) provides the most effective solution for simplifying IAM access in a large, multi-project GCP environment.

Question: 132

CertyIQ

Your United States-based company has created an application for assessing and responding to user actions. The primary table's data volume grows by 250,000 records per second. Many third parties use your application's APIs to build the functionality into their own frontend applications. Your application's APIs should comply with the following requirements:

- Single global endpoint
- ANSI SQL support
- Consistent access to the most up-to-date data

What should you do?

- A. Implement BigQuery with no region selected for storage or processing.
- B. Implement Cloud Spanner with the leader in North America and read-only replicas in Asia and Europe.
- C. Implement Cloud SQL for PostgreSQL with the master in North America and read replicas in Asia and Europe.
- D. Implement Bigtable with the primary cluster in North America and secondary clusters in Asia and Europe.

Answer: B

Explanation:

The correct answer is **B. Implement Cloud Spanner with the leader in North America and read-only replicas in Asia and Europe.**

Here's why:

The question emphasizes global access, ANSI SQL support, and consistent, up-to-date data for an application with extremely high write volumes. Cloud Spanner excels in these areas. Spanner is a globally distributed, scalable, strongly consistent database service offering ANSI SQL support. By designating a leader in North America, the primary region for writes, and setting up read-only replicas in Asia and Europe, the application provides low-latency reads to global users while maintaining strong consistency. Reads from the replicas will be globally consistent with the leader. Cloud Spanner also offers transactional consistency, making it suitable for high-throughput, data-intensive applications.

Option A, BigQuery, is primarily an analytical data warehouse and does not directly support low-latency access to most recent data updates. BigQuery is not designed for use as a transactional database with high write volume, despite its ability to ingest a large data volume.

Option C, Cloud SQL, is a regional service, unsuitable for a global application and has limitations on scalability compared to Spanner. While Cloud SQL offers read replicas, they are asynchronous, and thus there could be delays. Additionally, Cloud SQL's write capacity may not accommodate 250,000 records per second.

Option D, Bigtable, is a NoSQL database service that is not suitable as it does not offer ANSI SQL support. While Bigtable can handle high write throughput and is globally scalable, it requires custom logic to handle transactional data, and is not a good fit for the requirements of SQL support. Additionally, the question requires a "single global endpoint", which using multiple Bigtable clusters would not satisfy.

Authoritative Links:

Cloud Spanner Overview: <https://cloud.google.com/spanner/docs/overview>

Cloud Spanner Instance Configuration: <https://cloud.google.com/spanner/docs/instance-config>

BigQuery Overview: <https://cloud.google.com/bigquery/docs/introduction>

Cloud SQL Overview: <https://cloud.google.com/sql/docs/postgres/overview>

Bigtable Overview: <https://cloud.google.com/bigtable/docs/overview>

Question: 133

CertyIQ

A data scientist has created a BigQuery ML model and asks you to create an ML pipeline to serve predictions. You have a REST API application with the requirement to serve predictions for an individual user ID with latency under 100 milliseconds. You use the following query to generate predictions: `SELECT predicted_label, user_id FROM ML.PREDICT (MODEL 'dataset.model', table user_features)`. How should you create the ML pipeline?

- A. Add a WHERE clause to the query, and grant the BigQuery Data Viewer role to the application service account.
- B. Create an Authorized View with the provided query. Share the dataset that contains the view with the application service account.
- C. Create a Dataflow pipeline using BigQueryIO to read results from the query. Grant the Dataflow Worker role to the application service account.
- D. Create a Dataflow pipeline using BigQueryIO to read predictions for all users from the query. Write the results to Bigtable using BigtableIO. Grant the Bigtable Reader role to the application service account so that the application can read predictions for individual users from Bigtable.

Answer: D

Explanation:

Okay, here's a detailed justification for why option D is the correct approach for creating an ML pipeline to serve low-latency predictions, along with why the other options are less suitable:

Option D proposes a pipeline that pre-computes and stores predictions, which perfectly aligns with the latency requirement. Using Dataflow, you execute the BigQuery ML prediction query once for all users. Then, you write the `user_id` and `predicted_label` pairs to Bigtable. Bigtable, being a low-latency NoSQL database, is ideal for fast lookups. The application then reads individual user predictions using their ID as the key, which is extremely quick. The Bigtable Reader role ensures the application has the necessary access. This approach effectively decouples prediction generation from the request, eliminating latency introduced by executing the query for each request.

Option A suggests adding a WHERE clause and granting BigQuery Data Viewer. This means BigQuery would execute the prediction query every time a user prediction is requested. This will not satisfy the <100 millisecond latency requirement as BigQuery is not designed for real-time, high-frequency single row lookups. Option B proposes an Authorized View. While this adds a layer of access control, it still fundamentally involves BigQuery executing the query on each request and therefore would not satisfy the latency requirement. Option C suggests Dataflow reading results directly. This approach has the same latency problem as options A and B. The application would still need to query and process the BigQuery results in real-time, which would be slower.

Key Cloud Concepts:

Precomputation: Computing predictions beforehand avoids latency.

Low-Latency Database: Bigtable is optimized for fast key-value lookups.

Decoupling: Separating prediction generation from prediction serving enhances performance.

BigQuery ML: Allows for model creation within BigQuery.

Dataflow: A powerful service for data processing and pipeline creation.

BigQueryIO and BigtableIO: Connectors for reading and writing to BigQuery and Bigtable respectively.

Authoritative Links:

Bigtable: <https://cloud.google.com/bigtable>

BigQuery ML: <https://cloud.google.com/bigquery/docs/bigqueryml-intro>

Dataflow: <https://cloud.google.com/dataflow>

In summary, option D creates a pipeline optimized for the low-latency retrieval of pre-computed predictions, effectively addressing the requirements. Other solutions rely on real-time BigQuery queries that would not satisfy the performance constraints.

Question: 134

CertyIQ

You are building an application to share financial market data with consumers, who will receive data feeds. Data is collected from the markets in real time.

Consumers will receive the data in the following ways:

- ⇒ Real-time event stream
- ⇒ ANSI SQL access to real-time stream and historical data
- ⇒ Batch historical exports

Which solution should you use?

- A. Cloud Dataflow, Cloud SQL, Cloud Spanner
- B. Cloud Pub/Sub, Cloud Storage, BigQuery
- C. Cloud Dataproc, Cloud Dataflow, BigQuery
- D. Cloud Pub/Sub, Cloud Dataproc, Cloud SQL

Answer: B

Explanation:

The correct solution is **B. Cloud Pub/Sub, Cloud Storage, BigQuery**. Here's why:

Cloud Pub/Sub is ideal for the real-time event stream requirement. It's a fully managed, scalable message broker that enables asynchronous communication between applications, perfect for ingesting financial market data as it arrives. <https://cloud.google.com/pubsub>

Cloud Storage serves as a cost-effective and durable storage layer for historical batch data exports and can also act as a staging area for other services. It's designed for large amounts of unstructured data. <https://cloud.google.com/storage>

BigQuery is the right choice for enabling ANSI SQL access to both real-time and historical data. Its serverless data warehouse capabilities make it easy to analyze large datasets efficiently. BigQuery supports querying streamed data via its streaming inserts feature, and batch data can be loaded from Cloud Storage. <https://cloud.google.com/bigquery>

Option A, using Cloud Dataflow, Cloud SQL, and Cloud Spanner, is less suitable. While Dataflow handles real-time stream processing, Cloud SQL and Spanner are better suited for operational databases, not analytical workloads on large historical datasets like BigQuery.

Option C, involving Cloud Dataproc, is unnecessary. Dataproc is primarily used for large-scale batch processing with Hadoop/Spark. Here, a simpler approach with Cloud Pub/Sub and BigQuery for real-time and historical data respectively, is more appropriate. Dataflow might also be used here to perform transformations before writing to BigQuery, but is not as essential to the solution as Pub/Sub.

Option D is not suitable because while Pub/Sub is correct for real-time data ingestion, Cloud Dataproc is not the ideal choice for querying real-time stream and historical data, nor is Cloud SQL. Cloud SQL is for transaction databases, not analytical data warehouses.

Therefore, the combination of Cloud Pub/Sub for real-time streams, Cloud Storage for batch exports and data staging, and BigQuery for SQL access to both streams and historical data, provides the most fitting solution based on the problem requirements.

Question: 135

CertyIQ

You are building a new application that you need to collect data from in a scalable way. Data arrives continuously from the application throughout the day, and you expect to generate approximately 150 GB of JSON data per day by the end of the year. Your requirements are:

- Decoupling producer from consumer
- Space and cost-efficient storage of the raw ingested data, which is to be stored indefinitely
- Near real-time SQL query
- Maintain at least 2 years of historical data, which will be queried with SQL

Which pipeline should you use to meet these requirements?

- A. Create an application that provides an API. Write a tool to poll the API and write data to Cloud Storage as gzipped JSON files.
- B. Create an application that writes to a Cloud SQL database to store the data. Set up periodic exports of the database to write to Cloud Storage and load into BigQuery.
- C. Create an application that publishes events to Cloud Pub/Sub, and create Spark jobs on Cloud Dataproc to convert the JSON data to Avro format, stored on HDFS on Persistent Disk.
- D. Create an application that publishes events to Cloud Pub/Sub, and create a Cloud Dataflow pipeline that transforms the JSON event payloads to Avro, writing the data to Cloud Storage and BigQuery.

Answer: D

Explanation:

Option D is the most suitable solution because it efficiently addresses all the stated requirements.

Here's why:

Decoupling: Cloud Pub/Sub acts as a message queue, decoupling the application (producer) from the data processing pipeline (consumer). This allows the application to continue sending data regardless of the status of the downstream processing. <https://cloud.google.com/pubsub/docs/overview>

Scalable Ingestion: Pub/Sub is designed for high-volume, real-time data ingestion.

Space and Cost-Efficient Storage: Cloud Dataflow transforms the JSON data into Avro format. Avro is a compact, binary format that reduces storage space compared to JSON. Storing the Avro data in Cloud Storage provides cost-effective, durable, and scalable storage for the raw ingested data indefinitely. <https://cloud.google.com/dataflow/docs/concepts/choosing-storage-location>

Near Real-Time SQL Query & Historical Data: Cloud Dataflow also writes the Avro data to BigQuery. BigQuery is a fully managed, serverless data warehouse that allows you to run SQL queries on large datasets in near real-time. Maintaining at least two years of historical data is easily managed in BigQuery due to its scalable storage and query capabilities. <https://cloud.google.com/bigquery/docs/introduction>

Transformation: Dataflow's ability to transform from JSON to Avro is critical.

Let's examine why other options are less ideal:

Option A: Polling an API is less efficient and less scalable than a push-based system like Pub/Sub. Gzipped JSON is less storage efficient than Avro.

Option B: Cloud SQL is not designed for high-volume data ingestion at this scale. Exporting and loading the data into BigQuery would introduce significant latency and complexity. Cloud SQL is also significantly more expensive than Cloud Storage for storing the data.

Option C: Cloud Dataproc requires managing a cluster, which adds operational overhead. HDFS on Persistent Disk is more expensive and less scalable than Cloud Storage for long-term storage.

In conclusion, option D provides the best balance of scalability, cost-effectiveness, and real-time query capabilities while meeting all the specified requirements. Dataflow's ability to transform, load and write into different formats/services make it a great candidate for ingestion and storage.

Question: 136

CertyIQ

You are running a pipeline in Dataflow that receives messages from a Pub/Sub topic and writes the results to a BigQuery dataset in the EU. Currently, your pipeline is located in europe-west4 and has a maximum of 3 workers, instance type n1-standard-1. You notice that during peak periods, your pipeline is struggling to process records in a timely fashion, when all 3 workers are at maximum CPU utilization. Which two actions can you take to increase performance of your pipeline? (Choose two.)

- A. Increase the number of max workers
- B. Use a larger instance type for your Dataflow workers
- C. Change the zone of your Dataflow pipeline to run in us-central1
- D. Create a temporary table in Bigtable that will act as a buffer for new data. Create a new step in your pipeline to write to this table first, and then create a new pipeline to write from Bigtable to BigQuery
- E. Create a temporary table in Cloud Spanner that will act as a buffer for new data. Create a new step in your pipeline to write to this table first, and then create a new pipeline to write from Cloud Spanner to BigQuery

Answer: AB

Explanation:

The correct actions to improve Dataflow pipeline performance are A (Increase the number of max workers) and B (Use a larger instance type for your Dataflow workers).

Justification:

A. Increasing the number of max workers: Dataflow distributes processing across multiple workers. When all workers are at maximum CPU, increasing their number allows the pipeline to handle more parallel processing, thereby reducing processing time for incoming messages. This is horizontal scaling, where you add more compute resources of the same type. <https://cloud.google.com/dataflow/docs/concepts/execution-details#autoscaling> explains Dataflow's autoscaling capabilities.

B. Using a larger instance type: Larger instance types (e.g., n1-standard-2 or higher) have more CPU and memory resources per worker. This allows each worker to process more data concurrently, increasing overall throughput. This is vertical scaling, where you add more resources to existing compute resources. <https://cloud.google.com/compute/docs/machine-types> provides details about different Google Compute Engine machine types.

Why other options are incorrect:

C. Changing the zone: Changing the zone to us-central1 does not directly address resource constraints causing CPU bottlenecks. It may impact latency due to geographical distance between the pipeline and its

data sources, but it doesn't increase processing capacity.

D. Using Bigtable as a buffer: Introducing Bigtable adds complexity with no direct benefit. While Bigtable could be used as a temporary buffer, it doesn't address the core issue of insufficient Dataflow resources. Furthermore, this adds overhead from a new pipeline and introduces eventual consistency to the pipeline.

E. Using Cloud Spanner as a buffer: Similar to Bigtable, using Spanner as a buffer introduces complexity, and does not directly address the bottlenecks in the pipeline. It adds latency and complexity when simpler and more effective options are available.

In summary, by increasing worker count (A) and upgrading instance types (B), the pipeline can leverage both horizontal and vertical scaling to improve performance by handling more processing load concurrently and more efficiently.

Question: 137

CertyIQ

You have a data pipeline with a Cloud Dataflow job that aggregates and writes time series metrics to Cloud Bigtable. This data feeds a dashboard used by thousands of users across the organization. You need to support additional concurrent users and reduce the amount of time required to write the data. Which two actions should you take? (Choose two.)

- A. Configure your Cloud Dataflow pipeline to use local execution
- B. Increase the maximum number of Cloud Dataflow workers by setting `maxNumWorkers` in `PipelineOptions`
- C. Increase the number of nodes in the Cloud Bigtable cluster
- D. Modify your Cloud Dataflow pipeline to use the Flatten transform before writing to Cloud Bigtable
- E. Modify your Cloud Dataflow pipeline to use the `CoGroupByKey` transform before writing to Cloud Bigtable

Answer: BC

Explanation:

Justification for Choosing Options B and C:

The primary goals are to support more concurrent users accessing the dashboard and reduce write latency to Cloud Bigtable. Option B, increasing the `maxNumWorkers` in Dataflow, directly addresses write latency by scaling out the processing power. More workers mean the data pipeline can process data in parallel, reducing the time needed to aggregate and prepare metrics for writing to Bigtable. This speeds up the process and allows the pipeline to handle a higher volume of data. Cloud Dataflow's parallel processing capabilities are specifically designed for this kind of workload.

Option C, increasing the number of nodes in the Cloud Bigtable cluster, tackles the bottleneck on the storage side. Bigtable's performance is tied to the number of nodes in the cluster. More nodes mean more resources available to handle write requests, leading to faster write speeds and improved overall performance. This also enhances read performance, which is crucial for the increased number of dashboard users. Bigtable is designed for scalable, low-latency read and write access.

Options A, D, and E are not ideal for these specific requirements. Local execution (A) is useful for debugging and development but not for production pipelines. Flatten (D) is typically used to combine collections; it doesn't directly reduce write time to Bigtable. `CoGroupByKey` (E) is used for merging multiple data streams based on keys, and it is not ideal when data needs to be written to Bigtable.

In summary, adding Dataflow workers (B) boosts processing throughput, and adding Bigtable nodes (C) increases write and read capacity. These two changes work together to meet the given requirements.**Relevant Links for Further Research:**

Cloud Dataflow Autoscaling: <https://cloud.google.com/dataflow/docs/concepts/autoscaling>

Cloud Bigtable Scaling: <https://cloud.google.com/bigtable/docs/scaling>

Cloud Dataflow Pipeline Options: <https://beam.apache.org/documentation/programming-guide/#pipeline-options>

Cloud Bigtable Performance Optimization: <https://cloud.google.com/bigtable/docs/performance>

Question: 138

CertyIQ

You have several Spark jobs that run on a Cloud Dataproc cluster on a schedule. Some of the jobs run in sequence, and some of the jobs run concurrently. You need to automate this process. What should you do?

- A. Create a Cloud Dataproc Workflow Template
- B. Create an initialization action to execute the jobs
- C. Create a Directed Acyclic Graph in Cloud Composer
- D. Create a Bash script that uses the Cloud SDK to create a cluster, execute jobs, and then tear down the cluster

Answer: C

Explanation:

The correct answer is C: Create a Directed Acyclic Graph in Cloud Composer. Here's why:

Cloud Composer is a fully managed workflow orchestration service built on Apache Airflow. It allows you to define, schedule, and monitor complex workflows with dependencies. In this scenario, you have Spark jobs that need to run on a schedule, some sequentially and some concurrently. Composer's DAG (Directed Acyclic Graph) feature is perfectly suited for modeling these dependencies and scheduling the jobs. Each Spark job can be represented as a task in the DAG, and dependencies between tasks define the order of execution. Composer can then orchestrate the execution of these tasks on your Cloud Dataproc cluster. This allows for full automation and monitoring of the entire workflow.

Option A (Cloud Dataproc Workflow Template) is insufficient because while it can execute jobs, it doesn't easily handle complex dependency management or scheduling as required here. Workflow Templates in Dataproc are best suited for simpler workflows that don't need complex orchestration. They are not as robust or flexible as Composer.

Option B (Initialization action) is only suitable for setting up the Dataproc cluster when it is created. It doesn't provide the scheduling and dependency management needed for this use case. Initialization actions run once during cluster creation, not repeatedly on a schedule.

Option D (Bash script) is a possible but less desirable solution. While you could write a script to create a cluster, execute jobs using the Cloud SDK, and tear down the cluster, this approach is much less manageable, scalable, and observable than using Cloud Composer. Orchestration and dependency management would require custom coding, error handling, and monitoring. Composer offers these features out-of-the-box.

In conclusion, Cloud Composer offers a robust and scalable solution for managing complex workflows with dependencies, making it the best choice for automating the Spark jobs on Cloud Dataproc. Composer also supports features like retries, alerting, and detailed monitoring, which are important for production-level workloads.

Further Research:

Cloud Composer: <https://cloud.google.com/composer/docs>

Apache Airflow: <https://airflow.apache.org/>

Cloud Dataproc Workflow Templates: <https://cloud.google.com/dataproc/docs/concepts/workflows/using-workflow-templates>

Question: 139

CertyIQ

You are building a new data pipeline to share data between two different types of applications: jobs generators and job runners. Your solution must scale to accommodate increases in usage and must accommodate the addition of new applications without negatively affecting the performance of existing ones. What should you do?

- A. Create an API using App Engine to receive and send messages to the applications
- B. Use a Cloud Pub/Sub topic to publish jobs, and use subscriptions to execute them
- C. Create a table on Cloud SQL, and insert and delete rows with the job information
- D. Create a table on Cloud Spanner, and insert and delete rows with the job information

Answer: B

Explanation:

The correct answer is **B. Use a Cloud Pub/Sub topic to publish jobs, and use subscriptions to execute them.** Here's why:

Cloud Pub/Sub is a messaging service designed for asynchronous communication, making it ideal for decoupling applications. In this scenario, job generators (publishers) send job information to a Pub/Sub topic. Job runners (subscribers) then receive these messages through subscriptions. This architecture inherently scales because publishers and subscribers operate independently. Adding new job generators or runners doesn't disrupt existing workflows, as the messaging system handles the routing and distribution of messages. This contrasts with API-based solutions (option A) which can become bottlenecks under heavy load. Relational databases like Cloud SQL (option C) or Cloud Spanner (option D) are not suitable for this scenario. While they can store data, they are not designed for real-time message passing and would likely introduce performance limitations and concurrency issues. Cloud Pub/Sub's push and pull delivery mechanisms also offer flexibility in how subscribers consume messages. This allows for efficient resource utilization and adaptable processing rates based on subscriber capacity. The publish/subscribe model inherently promotes loose coupling between components, making the system resilient to individual component failures.

Authoritative Links for Further Research:

Cloud Pub/Sub Documentation: <https://cloud.google.com/pubsub/docs>

Pub/Sub concepts: <https://cloud.google.com/pubsub/docs/overview>

Benefits of loose coupling: <https://cloud.google.com/architecture/decoupling-microservices-using-pub-sub>

Question: 140

CertyIQ

You need to create a new transaction table in Cloud Spanner that stores product sales data. You are deciding what to use as a primary key. From a performance perspective, which strategy should you choose?

- A. The current epoch time
- B. A concatenation of the product name and the current epoch time
- C. A random universally unique identifier number (version 4 UUID)
- D. The original order identification number from the sales system, which is a monotonically increasing integer

Answer: C

Explanation:

The correct answer is **C. A random universally unique identifier number (version 4 UUID).**

Here's why:

Cloud Spanner excels at scaling reads and writes across many servers. To achieve this, it needs to distribute data evenly. The primary key plays a critical role in data distribution. Spanner shards tables based on the key range, assigning data to different nodes.

Option A (Epoch Time): Using epoch time alone as a primary key creates a "hotspot." Epoch time is monotonically increasing, meaning new data will always be written to the same node, overwhelming it while others remain idle. This destroys Spanner's ability to distribute load.

Option B (Product Name & Epoch Time): While prepending the product name helps a little, the epoch time is still monotonically increasing within each product. This reduces, but doesn't eliminate, the hot-spotting problem. Writes for a popular product would still concentrate on a single node for that product.

Option C (UUID): Version 4 UUIDs are pseudo-random. This randomness ensures a nearly uniform distribution of data across Spanner's nodes. While completely uniform isn't guaranteed (statistically unlikely collisions are possible), UUIDs are typically the best choice for maximizing write throughput and minimizing hot-spotting, especially in rapidly growing tables.

Option D (Monotonically Increasing Integer): Similar to epoch time, using a monotonically increasing order ID will create a hotspot, directing all new writes to a single node. This severely limits write scalability.

Because Spanner distributes data based on key ranges, predictable and increasing keys are detrimental to performance. A random UUID is optimal for spreading the write load across the entire database cluster, taking advantage of Spanner's distributed architecture. In scenarios where data locality based on product is needed, consider interleaving the transaction table with a product table. However, for maximized write throughput, UUIDs are generally preferable.

Supporting Documentation:

Schema design: <https://cloud.google.com/spanner/docs/schema-design>

Keys and indexes: <https://cloud.google.com/spanner/docs/keys>

Performance best practices: <https://cloud.google.com/spanner/docs/performance-tips>

Question: 141

CertyIQ

Data Analysts in your company have the Cloud IAM Owner role assigned to them in their projects to allow them to work with multiple GCP products in their projects. Your organization requires that all BigQuery data access logs be retained for 6 months. You need to ensure that only audit personnel in your company can access the data access logs for all projects. What should you do?

- A. Enable data access logs in each Data Analyst's project. Restrict access to Stackdriver Logging via Cloud IAM roles.
- B. Export the data access logs via a project-level export sink to a Cloud Storage bucket in the Data Analysts' projects. Restrict access to the Cloud Storage bucket.
- C. Export the data access logs via a project-level export sink to a Cloud Storage bucket in a newly created projects for audit logs. Restrict access to the project with the exported logs.
- D. Export the data access logs via an aggregated export sink to a Cloud Storage bucket in a newly created project for audit logs. Restrict access to the project that contains the exported logs.

Answer: D

Explanation:

The correct solution is **D**, exporting data access logs via an aggregated export sink to a dedicated audit logs project and restricting access to that project. Here's why:

Problem: Data Analysts have excessive permissions (Owner role) which grants them access to all resources, including audit logs. The goal is to isolate and secure BigQuery data access logs, making them accessible only to audit personnel.

Why Option D is correct:

Centralized Logging: Aggregated sinks export logs from multiple projects to a single destination. This is crucial for consistent and manageable audit logging across the entire organization (and not just individual analysts' projects as in A, B, or C). <https://cloud.google.com/logging/docs/export/aggregated-exports>

Dedicated Project: Storing the logs in a dedicated audit project helps with clear separation of concerns and minimizes the blast radius in case of any security issues. Option C is close but does not use an aggregated sink.

Access Control: Restricting access to the audit project and the contained Cloud Storage bucket using Cloud IAM ensures only authorized audit personnel can view the logs. This limits access and reduces risk, preventing data analysts who initially had broad permissions from accessing them, which B and C do not prevent.

Scalability: This approach is scalable. As new projects are added, their logs can be easily routed to the same centralized audit project.

Compliance: Centralized audit logs facilitate compliance requirements, such as data retention, and simplify auditing processes. <https://cloud.google.com/logging/docs/audit>

Single Source of Truth: Option D provides a single, reliable location for all audit logs, simplifying analysis and monitoring.

Why other options are incorrect:

Option A: Enabling logs in each analyst's project and limiting Stackdriver access doesn't truly restrict access as Owner role grants access to Cloud Logging regardless of IAM settings. Also, this means logs are not centralized.

Option B: Exporting logs to buckets within each analysts' projects means data analysts still have access and also does not address the need for centralized logging and audit.

Option C: Similar to B, putting logs in a separate project but using project level sinks lacks the centralized benefits of aggregated sinks and leaves a gap in audit logs for projects that were not configured for the sinks initially.

In summary, option D, utilizing aggregated sinks and a dedicated audit project with restricted access, provides the most secure, scalable, and manageable solution for centralizing and protecting BigQuery data access logs, ensuring only authorized personnel can access them and is the only choice which prevents the data analysts from seeing the logs.

Question: 142

CertyIQ

Each analytics team in your organization is running BigQuery jobs in their own projects. You want to enable each team to monitor slot usage within their projects.

What should you do?

- A. Create a Cloud Monitoring dashboard based on the BigQuery metric query/scanned_bytes
- B. Create a Cloud Monitoring dashboard based on the BigQuery metric slots/allocated_for_project
- C. Create a log export for each project, capture the BigQuery job execution logs, create a custom metric based on the totalSlotMs, and create a Cloud Monitoring dashboard based on the custom metric
- D. Create an aggregated log export at the organization level, capture the BigQuery job execution logs, create a custom metric based on the totalSlotMs, and create a Cloud Monitoring dashboard based on the custom metric

Answer: B

Explanation:

The correct answer is **B. Create a Cloud Monitoring dashboard based on the BigQuery metric slots/allocated_for_project**. This approach directly addresses the requirement to monitor slot usage per project. BigQuery slots represent the computational capacity available for running queries. The slots/allocated_for_project metric, readily available in Cloud Monitoring, specifically tracks the number of slots allocated to each project. This avoids the complexities and overhead associated with log exports and custom metrics, making it the most efficient and straightforward solution. Option A, focusing on query/scanned_bytes, measures data processing volume rather than slot consumption, which isn't the primary focus here. Options C and D, involving log exports and custom metrics based on totalSlotMs, add unnecessary complexity. While totalSlotMs can indicate resource usage, it's less direct and requires significant processing of logs. The slots/allocated_for_project metric provides an instant, native view of project-level slot usage. Cloud Monitoring dashboards are ideal for visualizing these metrics, offering real-time insights into slot allocation and utilization across different projects. This enables teams to effectively monitor their resource consumption and potentially identify areas for optimization. In summary, using pre-built metrics directly from Cloud Monitoring offers the fastest and easiest way to monitor BigQuery slot usage per project.

Authoritative Links:

Cloud Monitoring metrics for BigQuery: https://cloud.google.com/monitoring/api/metrics_gcp#gcp-bigquery
(Search for "bigquery.googleapis.com/slots/allocated_for_project" on this page)

Cloud Monitoring Dashboards: <https://cloud.google.com/monitoring/dashboards>

BigQuery Slots: <https://cloud.google.com/bigquery/docs/reservations-intro>

Question: 143

CertyIQ

You are operating a streaming Cloud Dataflow pipeline. Your engineers have a new version of the pipeline with a different windowing algorithm and triggering strategy. You want to update the running pipeline with the new version. You want to ensure that no data is lost during the update. What should you do?

- A. Update the Cloud Dataflow pipeline inflight by passing the --update option with the --jobName set to the existing job name
- B. Update the Cloud Dataflow pipeline inflight by passing the --update option with the --jobName set to a new unique job name
- C. Stop the Cloud Dataflow pipeline with the Cancel option. Create a new Cloud Dataflow job with the updated code
- D. Stop the Cloud Dataflow pipeline with the Drain option. Create a new Cloud Dataflow job with the updated code

Answer: D

Explanation:

The correct answer is **D: Stop the Cloud Dataflow pipeline with the Drain option. Create a new Cloud Dataflow job with the updated code.**

Here's why:

Dataflow pipelines, especially streaming ones, are designed for continuous data processing. Simply updating a live pipeline with a significantly different windowing algorithm and triggering strategy, as options A and B propose, risks data loss and incorrect results. The new algorithm might not be compatible with the existing state, leading to inconsistencies. Option B which creates a new job, doesn't address data loss in the existing job.

Option C, canceling the pipeline, abruptly stops processing and potentially loses any data in transit or buffered within the pipeline. This approach violates the requirement of ensuring no data loss.

Option D, using the Drain option, is the correct way to gracefully handle a pipeline update. The Drain command allows the existing pipeline to complete processing any data currently in-flight and write all buffered data before stopping. This ensures no data is lost. After the pipeline is successfully drained, a new pipeline job can be created with the updated code. This process ensures a smooth transition and data integrity.

Draining allows Dataflow to commit all pending writes to sinks before shutting down the pipeline. Once the pipeline has successfully drained, it is safe to launch a new pipeline with the desired changes. This approach enables seamless upgrades to streaming pipelines without losing any in-flight or buffered data.

Key Concepts:

Drain: Dataflow's graceful shutdown mechanism for completing processing without data loss.

Update (In-flight): This usually supports minor code changes, not major structural alterations like windowing and triggering changes.

Windowing and Triggering: Core elements in stream processing that define how data is grouped and when output is produced. Altering them impacts the logic of the pipeline.

Data Loss: Loss of data during processing is often unacceptable. Minimizing this is essential for data integrity.

Authoritative Links:

[Updating a pipeline - Dataflow Documentation](#): Specifically, the sections discussing the drain functionality highlight its suitability for scenarios with significant changes.

[Managing Dataflow jobs - Dataflow Documentation](#): Further clarifies the differences between cancelling, draining, and updating Dataflow jobs.

Question: 144

CertyIQ

You need to move 2 PB of historical data from an on-premises storage appliance to Cloud Storage within six months, and your outbound network capacity is constrained to 20 Mb/sec. How should you migrate this data to Cloud Storage?

- A. Use Transfer Appliance to copy the data to Cloud Storage
- B. Use `gsutil cp -J` to compress the content being uploaded to Cloud Storage
- C. Create a private URL for the historical data, and then use Storage Transfer Service to copy the data to Cloud Storage
- D. Use `trickle` or `ionice` along with `gsutil cp` to limit the amount of bandwidth `gsutil` utilizes to less than 20 Mb/sec so it does not interfere with the production traffic

Answer: A

Explanation:

The correct answer is A, using Transfer Appliance. Here's why:

The challenge lies in moving 2 PB of data within a 6-month timeframe with a severely limited 20 Mb/sec network bandwidth. Let's analyze each option:

Option A (Transfer Appliance): This is the ideal solution for large-scale data migration with limited bandwidth. Transfer Appliance is a physical, high-capacity storage device that Google ships to the on-premises location. You copy your data onto it, then ship it back to Google where they upload it to Cloud Storage. This bypasses the network constraint, making a 2 PB transfer feasible within the timeframe. This approach aligns perfectly with scenarios where massive datasets need to be moved without reliance on network speed.

<https://cloud.google.com/transfer-appliance>

Option B (`gsutil cp` with compression): While compression can reduce the data size, the initial network

bottleneck of 20 Mb/sec remains. Compressing 2 PB still takes a prohibitively long time to transfer over that connection. This approach is simply not viable for such a massive dataset and low bandwidth.

Option C (Private URL and Storage Transfer Service): This solution requires that the data be accessible via a publicly facing URL, which is not ideal and possibly problematic with on-premise data. This would still have to go through the 20 Mb/sec bottleneck. Additionally, STS is usually used for transferring data already in the cloud, not for a local on-premise data migration of this scale.

Option D (gsutil cp with trickle/ionice): Limiting bandwidth usage with trickle/ionice might be useful in some situations but will still be constrained by the 20Mb/sec limit and is simply far too slow for the sheer size of the 2PB data. This will still take months, if not years, to complete the migration.

Therefore, Transfer Appliance is the only viable solution that overcomes the network bandwidth constraint to move 2 PB of data within the specified timeframe. It's designed specifically for offline data migration of this magnitude. It bypasses the bottleneck, and the physical shipment allows for rapid transfer speeds to the cloud. Options B, C, and D, while potentially useful in other scenarios, cannot meet the challenge given the scale of the data and the network limitation.

Question: 145

CertyIQ

You receive data files in CSV format monthly from a third party. You need to cleanse this data, but every third month the schema of the files changes. Your requirements for implementing these transformations include:

- Executing the transformations on a schedule
- Enabling non-developer analysts to modify transformations
- Providing a graphical tool for designing transformations

What should you do?

- A. Use Dataprep by Trifacta to build and maintain the transformation recipes, and execute them on a scheduled basis
- B. Load each month's CSV data into BigQuery, and write a SQL query to transform the data to a standard schema. Merge the transformed tables together with a SQL query
- C. Help the analysts write a Dataflow pipeline in Python to perform the transformation. The Python code should be stored in a revision control system and modified as the incoming data's schema changes
- D. Use Apache Spark on Dataproc to infer the schema of the CSV file before creating a Dataframe. Then implement the transformations in Spark SQL before writing the data out to Cloud Storage and loading into BigQuery

Answer: A

Explanation:

Option A, using Dataprep by Trifacta, is the most suitable solution. Dataprep excels at data wrangling and transformation tasks, particularly when dealing with varying schemas. Its graphical interface empowers non-developer analysts to create, modify, and maintain transformation recipes without needing to code. Dataprep facilitates scheduled execution of these recipes, addressing the requirement for scheduled processing. The tool's intelligent schema detection can handle changes in input data schema.

<https://cloud.google.com/dataprep/docs>

Option B involves using SQL on BigQuery, which while powerful, doesn't address the need for non-developer access and visual design. SQL requires technical expertise to modify schema transformations, hindering analyst involvement.

Option C suggests a Dataflow pipeline in Python, which, although scalable and powerful, relies heavily on coding skills, directly contradicting the requirement of enabling non-developers to modify transformations. Managing code in a revision control system also creates a more complex workflow for the analysts.

Option D, leveraging Apache Spark on Dataproc, presents a similar issue with option C, in that it mandates coding and is not ideal for a non-developer audience. While Spark excels in data processing, it doesn't offer an intuitive graphical interface for transformation design. Also, the schema detection isn't the core strength of Spark as compared to tools dedicated to such requirements. Dataprep's built-in features, therefore, offer better alignment with the given requirements.

Question: 146

CertyIQ

You want to migrate an on-premises Hadoop system to Cloud Dataproc. Hive is the primary tool in use, and the data format is Optimized Row Columnar (ORC).

All ORC files have been successfully copied to a Cloud Storage bucket. You need to replicate some data to the cluster's local Hadoop Distributed File System

(HDFS) to maximize performance. What are two ways to start using Hive in Cloud Dataproc? (Choose two.)

- A. Run the gsutil utility to transfer all ORC files from the Cloud Storage bucket to HDFS. Mount the Hive tables locally.
- B. Run the gsutil utility to transfer all ORC files from the Cloud Storage bucket to any node of the Dataproc cluster. Mount the Hive tables locally.
- C. Run the gsutil utility to transfer all ORC files from the Cloud Storage bucket to the master node of the Dataproc cluster. Then run the Hadoop utility to copy them to HDFS. Mount the Hive tables from HDFS.
- D. Leverage Cloud Storage connector for Hadoop to mount the ORC files as external Hive tables. Replicate external Hive tables to the native ones.
- E. Load the ORC files into BigQuery. Leverage BigQuery connector for Hadoop to mount the BigQuery tables as external Hive tables. Replicate external Hive tables to the native ones.

Answer: BC

Explanation:

The correct answer options are **B and C**. Let's break down why:

Option B: This option is viable, although not optimal for maximizing performance. Using gsutil to copy ORC files to any node within the Dataproc cluster makes the data accessible from that specific node's local storage. While this provides faster access than directly reading from Cloud Storage, it won't be available for computations happening on other worker nodes in the cluster. This option also doesn't leverage the distributed nature of HDFS for fault tolerance. Mount the Hive tables locally means that the table definitions are within the Hive environment but point to the files' location, which could be on a local file system, rather than HDFS, which does not replicate data.

Option C: This is a better approach than option B. First, gsutil is used to copy all ORC files from Cloud Storage to the master node in Dataproc. Then, Hadoop utility is used to distribute copies of data to the nodes in HDFS which is recommended in the question. By transferring the data to HDFS via Hadoop utilities like `hdfs dfs -cp`, you're distributing the data across the cluster, ensuring data availability and performance. Mount the Hive tables locally, which references the location of files on HDFS. This is essential to leverage HDFS as the optimal location for data used by Spark, Hive, and Hadoop. This option achieves the goal of replicating data to HDFS for maximized performance by utilizing the correct procedure.

Why other options are incorrect:

Option A: This option suggests using gsutil to transfer data directly to HDFS, which is incorrect. gsutil interacts with Cloud Storage, not directly with HDFS. The Hadoop file system utility is needed to move files inside the HDFS.

Option D: While using the Cloud Storage connector is viable, it doesn't directly address the performance requirement of replicating the data to HDFS, which the question explicitly mentions. It focuses on accessing

external tables, which is not ideal when aiming for maximum performance. Replication would add extra, unneeded steps.

Option E: Loading data into BigQuery introduces an unnecessary intermediate step, increasing costs and complexity, and does not fulfill the objective of using HDFS on the Dataproc cluster.

Key Concepts:

Cloud Dataproc: Managed Hadoop and Spark service on Google Cloud. <https://cloud.google.com/dataproc>

Cloud Storage: Object storage service on Google Cloud. <https://cloud.google.com/storage>

HDFS (Hadoop Distributed File System): Distributed file system used in Hadoop clusters.

<https://hadoop.apache.org/docs/current/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>

gsutil: Command-line tool for working with Cloud Storage. <https://cloud.google.com/storage/docs/gsutil>

Hive: Data warehousing tool built on top of Hadoop. <https://hive.apache.org/>

Hadoop utilities: command line tools for handling files, etc on a hadoop filesystem.

In summary, options B and C offer viable and more optimal methods for using Hive on Dataproc by leveraging local filesystems and HDFS respectively, addressing the specific needs of the question. Option C provides better performance and adherence to the problem, therefore being more recommended.

Question: 147

CertyIQ

You are implementing several batch jobs that must be executed on a schedule. These jobs have many interdependent steps that must be executed in a specific order. Portions of the jobs involve executing shell scripts, running Hadoop jobs, and running queries in BigQuery. The jobs are expected to run for many minutes up to several hours. If the steps fail, they must be retried a fixed number of times. Which service should you use to manage the execution of these jobs?

- A. Cloud Scheduler
- B. Cloud Dataflow
- C. Cloud Functions
- D. Cloud Composer

Answer: D

Explanation:

The correct answer is **D. Cloud Composer**. Here's a detailed justification:

Cloud Composer is a managed workflow orchestration service based on Apache Airflow, perfectly suited for managing complex, multi-step batch jobs. These jobs often involve interdependent tasks needing specific execution order, a key feature of workflows. Composer allows defining workflows as Directed Acyclic Graphs (DAGs), clearly outlining task dependencies and sequencing. The question describes jobs involving shell scripts, Hadoop jobs, and BigQuery queries, which are all easily integrated into Composer DAGs using its operators. Furthermore, Composer handles retries automatically, making it resilient to transient failures. It's designed for long-running jobs (minutes to hours), unlike options better suited for shorter functions.

Cloud Scheduler (A) is primarily for triggering jobs on a schedule, but it doesn't manage workflow complexities or task dependencies like Composer. Cloud Dataflow (B) is ideal for parallel data processing and streaming, not orchestration of diverse batch jobs involving shell scripts. Cloud Functions (C) are best for event-driven, short-lived tasks, not for orchestrating long-running, multi-step processes. Therefore, Cloud Composer stands out as the best solution for managing scheduled batch jobs with complex dependencies and various task types.

Authoritative Links:

Cloud Composer Documentation: <https://cloud.google.com/composer/docs>

Apache Airflow Documentation: <https://airflow.apache.org/>

Comparison of Google Cloud Workflow Tools: <https://cloud.google.com/architecture/choosing-workflow-automation-service>

Question: 148

CertyIQ

You work for a shipping company that has distribution centers where packages move on delivery lines to route them properly. The company wants to add cameras to the delivery lines to detect and track any visual damage to the packages in transit. You need to create a way to automate the detection of damaged packages and flag them for human review in real time while the packages are in transit. Which solution should you choose?

- A. Use BigQuery machine learning to be able to train the model at scale, so you can analyze the packages in batches.
- B. Train an AutoML model on your corpus of images, and build an API around that model to integrate with the package tracking applications.
- C. Use the Cloud Vision API to detect for damage, and raise an alert through Cloud Functions. Integrate the package tracking applications with this function.
- D. Use TensorFlow to create a model that is trained on your corpus of images. Create a Python notebook in Cloud Datalab that uses this model so you can analyze for damaged packages.

Answer: B

Explanation:

Option B, using AutoML to train a custom model and creating an API, is the most suitable solution for real-time, automated damage detection on a high-throughput delivery line. AutoML simplifies the machine learning model development process, allowing users without extensive coding expertise to build high-quality models, which are trained using the company's specific image dataset. The resulting model, deployed as an API, provides a scalable and responsive endpoint for real-time inference as packages move past the cameras. The package tracking application can easily query this API with images, receiving immediate damage detection results, which facilitates prompt flagging for human review. This approach ensures the system can keep up with real-time requirements, and scales to accommodate different volumes of packages. Options A, C, and D are not as appropriate. Option A (BigQuery ML) is designed for batch processing and analysis and is not suitable for real-time needs. Option C (Cloud Vision API) does not provide a specific "damage" detection model, and might lack the fine-grained accuracy for damage specific to packages. Option D (TensorFlow and Cloud Datalab) would be a valid method for model creation, however it would require substantial manual effort for model development and deployment, and not a service designed to serve inference at scale compared to an API, making option B a superior choice.

Authoritative Links:

AutoML Vision: <https://cloud.google.com/vision/automl/docs>

AutoML API: <https://cloud.google.com/automl/docs/reference/rest>

Real-time Inference: <https://cloud.google.com/ai-platform/prediction/docs/online-prediction>

Question: 149

CertyIQ

You are migrating your data warehouse to BigQuery. You have migrated all of your data into tables in a dataset. Multiple users from your organization will be using the data. They should only see certain tables based on their team membership. How should you set user permissions?

- A. Assign the users/groups data viewer access at the table level for each table
- B. Create SQL views for each team in the same dataset in which the data resides, and assign the users/groups

data viewer access to the SQL views

C. Create authorized views for each team in the same dataset in which the data resides, and assign the users/groups data viewer access to the authorized views

D. Create authorized views for each team in datasets created for each team. Assign the authorized views data viewer access to the dataset in which the data resides. Assign the users/groups data viewer access to the datasets in which the authorized views reside

Answer: A

Explanation:

The correct answer is **A. Assign the users/groups data viewer access at the table level for each table.**

Here's why:

Option A is the most straightforward and granular approach to manage permissions based on team access requirements. By granting "data viewer" access directly at the table level, you ensure users only see the specific tables relevant to their team. This method leverages BigQuery's IAM (Identity and Access Management) capabilities, which allow for precise control over resources. This method requires explicit permission assignment for each table to every user or group that requires access.

Option B, while seeming simpler, creates a maintenance headache. Changing underlying table structures would also require modifying all affected views. Additionally, users would still need explicit dataset access to see the views. Option C involves using authorized views, which is more suited for scenarios where users need to access the data in a way that's different from the underlying data structure, while still having specific access per view and would be an over-complication of the use case. Option D is incorrect because it misuses authorized views and also overcomplicates the setup, adding an unnecessary layer of complexity through separate datasets for each team. Authorized views grant access to the tables within a dataset, however users would still require to have read permissions to the table where data reside.

Therefore, directly assigning data viewer access at the table level is the most efficient and appropriate method for managing team-based data access in BigQuery, fulfilling the requirements of the scenario while adhering to principles of least privilege and simplifying management.

Relevant links for further research:

BigQuery IAM: <https://cloud.google.com/bigquery/docs/access-control>

BigQuery Table-Level Permissions: <https://cloud.google.com/bigquery/docs/table-access-controls>

BigQuery Views: <https://cloud.google.com/bigquery/docs/views>

BigQuery Authorized Views: <https://cloud.google.com/bigquery/docs/authorized-views>

Question: 150

CertyIQ

You want to build a managed Hadoop system as your data lake. The data transformation process is composed of a series of Hadoop jobs executed in sequence.

To accomplish the design of separating storage from compute, you decided to use the Cloud Storage connector to store all input data, output data, and intermediary data. However, you noticed that one Hadoop job runs very slowly with Cloud Dataproc, when compared with the on-premises bare-metal Hadoop environment (8-core nodes with 100-GB RAM). Analysis shows that this particular Hadoop job is disk I/O intensive. You want to resolve the issue. What should you do?

A. Allocate sufficient memory to the Hadoop cluster, so that the intermediary data of that particular Hadoop job can be held in memory

B. Allocate sufficient persistent disk space to the Hadoop cluster, and store the intermediate data of that particular Hadoop job on native HDFS

C. Allocate more CPU cores of the virtual machine instances of the Hadoop cluster so that the networking bandwidth for each instance can scale up

D. Allocate additional network interface card (NIC), and configure link aggregation in the operating system to use the combined throughput when working with Cloud Storage

Answer: B

Explanation:

The correct answer is **B. Allocate sufficient persistent disk space to the Hadoop cluster, and store the intermediate data of that particular Hadoop job on native HDFS.**

Here's the justification:

The core issue is the slow performance of a disk I/O intensive Hadoop job in Cloud Dataproc compared to an on-premises setup. Cloud Storage, while excellent for durable and scalable storage, introduces latency compared to local disk access, particularly for intermediate data. Hadoop jobs often perform numerous read/write operations on intermediate data between map and reduce phases. Storing this intermediate data in Cloud Storage forces the cluster to repeatedly access data over the network, which significantly slows down the processing.

Option B addresses this directly by leveraging HDFS, the native distributed file system of Hadoop. By allocating sufficient persistent disk space to the Dataproc cluster, and configuring Hadoop to store intermediate data within HDFS, we significantly reduce the network latency. Data access is localized to the nodes where the processing takes place, dramatically improving disk I/O performance for intermediate data.

Option A, allocating more memory, might help with in-memory processing but doesn't address the core issue of network I/O bottlenecks for intermediate data. Option C, allocating more CPU cores, also doesn't solve the problem of latency due to network calls for intermediary storage. Option D, adding NICs and aggregating links, might slightly improve network bandwidth but it would be far less effective at reducing network latency compared to local disk storage on HDFS. HDFS is designed for high performance disk based storage for intermediate data.

By using HDFS for intermediate data and Cloud Storage for input and output data, we achieve the desired separation of compute and storage while optimizing performance for I/O-intensive jobs.

Further research:

Google Cloud Dataproc Documentation on Storage Options:

<https://cloud.google.com/dataproc/docs/concepts/storage>

Hadoop Distributed File System (HDFS) Overview: <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>

Best practices for optimizing Hadoop jobs: [Search for "hadoop optimization techniques" on your preferred search engine]

Question: 151

CertyIQ

You work for an advertising company, and you've developed a Spark ML model to predict click-through rates at advertisement blocks. You've been developing everything at your on-premises data center, and now your company is migrating to Google Cloud. Your data center will be closing soon, so a rapid lift-and-shift migration is necessary. However, the data you've been using will be migrated to BigQuery. You periodically retrain your Spark ML models, so you need to migrate existing training pipelines to Google Cloud. What should you do?

- A. Use Vertex AI for training existing Spark ML models
- B. Rewrite your models on TensorFlow, and start using Vertex AI
- C. Use Dataproc for training existing Spark ML models, but start reading data directly from BigQuery
- D. Spin up a Spark cluster on Compute Engine, and train Spark ML models on the data exported from BigQuery

Answer: C

Explanation:

The best solution is **C. Use Dataproc for training existing Spark ML models, but start reading data directly from BigQuery.**

Here's why:

Rapid Lift-and-Shift: The requirement for a rapid lift-and-shift migration means minimizing code changes. Option C allows you to keep your existing Spark ML models largely intact.

BigQuery Integration: The data is migrating to BigQuery, so you need a way for your Spark jobs to access it. Dataproc has native connectors to read data directly from BigQuery.

Dataproc for Spark: Dataproc is a managed Spark and Hadoop service on Google Cloud. It's designed to run Spark workloads efficiently and integrates well with other Google Cloud services.

Minimal Code Changes: Option C requires minimal changes to your existing Spark code. You'll primarily need to update the data source configuration to point to BigQuery.

Cost-Effectiveness: Dataproc offers autoscaling and customizable cluster configurations, which can help you optimize costs.

Why the other options are less suitable:

A. Use Vertex AI for training existing Spark ML models: While Vertex AI is a powerful ML platform, it doesn't natively support Spark ML models. Migrating to Vertex AI would require significant refactoring or a complete rewrite of your models.

B. Rewrite your models on TensorFlow, and start using Vertex AI: Rewriting your models on TensorFlow is time-consuming and contradicts the need for a rapid migration. It would also require significant retraining and validation.

D. Spin up a Spark cluster on Compute Engine, and train Spark ML models on the data exported from BigQuery: While possible, this approach is less managed than using Dataproc. You would be responsible for configuring, managing, and scaling the Spark cluster on Compute Engine. Also, exporting data out of BigQuery into another storage (such as GCS) for the cluster to access is less efficient compared to directly reading from BigQuery with the Dataproc's BigQuery connector.

In summary, Dataproc provides the most straightforward path to migrate your existing Spark ML pipelines to Google Cloud while leveraging BigQuery as your data source. It minimizes code changes, provides a managed Spark environment, and integrates well with other Google Cloud services.

Here are links to relevant documentation:

Dataproc: <https://cloud.google.com/dataproc>

BigQuery Connector for Spark: <https://cloud.google.com/dataproc/docs/concepts/connectors/spark-jdbc>

Question: 152

CertyIQ

You work for a global shipping company. You want to train a model on 40 TB of data to predict which ships in each geographic region are likely to cause delivery delays on any given day. The model will be based on multiple attributes collected from multiple sources. Telemetry data, including location in GeoJSON format, will be pulled from each ship and loaded every hour. You want to have a dashboard that shows how many and which ships are likely to cause delays within a region. You want to use a storage solution that has native functionality for prediction and geospatial processing. Which storage solution should you use?

- A. BigQuery
- B. Cloud Bigtable
- C. Cloud Datastore

Answer: A

Explanation:

The correct answer is **A. BigQuery**. Here's why:

BigQuery is Google Cloud's serverless, highly scalable data warehouse designed for large-scale analytics. It excels at handling massive datasets like the 40 TB mentioned in the scenario. Its columnar storage and query optimization capabilities make it ideal for analytical workloads involving complex queries on large volumes of data. Critically, BigQuery offers native support for geospatial data types and functions, including processing GeoJSON, which is essential for the ship location data. This enables efficient analysis based on geographical regions. Furthermore, BigQuery provides machine learning integration via BigQuery ML, allowing models to be trained and deployed directly within the data warehouse without requiring complex data transfer. This built-in functionality supports the prediction goal of identifying ships likely to cause delays. The ability to easily create visualizations and dashboards by connecting BigQuery to tools like Looker or Data Studio is also a key benefit, meeting the requirement for a dashboard showing at-risk ships. While Cloud Bigtable is suitable for large, operational data, it lacks built-in analytical capabilities. Cloud Datastore is a NoSQL document database, not designed for large scale analytical processing or geospatial analysis. Cloud SQL for PostgreSQL could handle geospatial data, but would require additional setup and would not scale as well as BigQuery for the 40 TB dataset. BigQuery is the only option which provides the required geospatial processing and machine learning capabilities on large datasets for analytical workloads without significant integration needs.

Authoritative Links:

BigQuery Overview: <https://cloud.google.com/bigquery/docs/introduction>

BigQuery GIS: <https://cloud.google.com/bigquery/docs/gis-data>

BigQuery ML: <https://cloud.google.com/bigquery/docs/bqml-introduction>

Question: 153

CertyIQ

You operate an IoT pipeline built around Apache Kafka that normally receives around 5000 messages per second. You want to use Google Cloud Platform to create an alert as soon as the moving average over 1 hour drops below 4000 messages per second. What should you do?

- A. Consume the stream of data in Dataflow using Kafka IO. Set a sliding time window of 1 hour every 5 minutes. Compute the average when the window closes, and send an alert if the average is less than 4000 messages.
- B. Consume the stream of data in Dataflow using Kafka IO. Set a fixed time window of 1 hour. Compute the average when the window closes, and send an alert if the average is less than 4000 messages.
- C. Use Kafka Connect to link your Kafka message queue to Pub/Sub. Use a Dataflow template to write your messages from Pub/Sub to Bigtable. Use Cloud Scheduler to run a script every hour that counts the number of rows created in Bigtable in the last hour. If that number falls below 4000, send an alert.
- D. Use Kafka Connect to link your Kafka message queue to Pub/Sub. Use a Dataflow template to write your messages from Pub/Sub to BigQuery. Use Cloud Scheduler to run a script every five minutes that counts the number of rows created in BigQuery in the last hour. If that number falls below 4000, send an alert.

Answer: A

Explanation:

The correct answer is **A**. Here's a detailed justification:

Option A leverages Dataflow's powerful windowing capabilities to accurately calculate the moving average. By using Kafka IO to read the stream, Dataflow can then apply a sliding time window of 1 hour, moving every 5

minutes. This means every 5 minutes, the average message rate for the previous hour is calculated. This ensures a near real-time moving average, fulfilling the requirement of detecting a drop below 4000 messages per second. Once the average is calculated for a window, the logic can easily trigger an alert if it's less than 4000 messages per second. This approach is efficient and designed for real-time stream processing.

Option B, using a fixed 1-hour window, would not provide a true moving average. It would only calculate the average over the previous 1-hour period, once every hour, not every 5 minutes. This lacks the responsiveness desired in the question.

Option C introduces unnecessary complexity with Bigtable. Furthermore, using Cloud Scheduler to query Bigtable only hourly does not match the requirement for detecting a drop in real-time. This approach lacks both accuracy and responsiveness for the given problem.

Option D is also overly complex, using BigQuery and Cloud Scheduler. Querying BigQuery every five minutes to check the last hour's worth of rows is possible but inefficient and introduces higher latency compared to Dataflow's real-time calculations. It's also prone to cost overruns due to repeated queries on large datasets.

Dataflow is specifically designed for real-time data processing and has windowing functionalities making Option A the most effective and cost-efficient approach. Using Dataflow with KafkaIO allows for a direct connection to the Kafka stream with the flexibility to define the desired time windows for the alert. This is also a serverless compute option, meaning less operational overhead.

Relevant Documentation:

Apache Beam (Dataflow's Programming Model): <https://beam.apache.org/documentation/programming-guide/>

Dataflow Windowing: <https://cloud.google.com/dataflow/docs/concepts/windowing>

Dataflow Kafka IO: <https://beam.apache.org/documentation/io/connectors/kafka/>

Question: 154

CertyIQ

You plan to deploy Cloud SQL using MySQL. You need to ensure high availability in the event of a zone failure. What should you do?

- A. Create a Cloud SQL instance in one zone, and create a failover replica in another zone within the same region.
- B. Create a Cloud SQL instance in one zone, and create a read replica in another zone within the same region.
- C. Create a Cloud SQL instance in one zone, and configure an external read replica in a zone in a different region.
- D. Create a Cloud SQL instance in a region, and configure automatic backup to a Cloud Storage bucket in the same region.

Answer: A

Explanation:

The correct answer is **A. Create a Cloud SQL instance in one zone, and create a failover replica in another zone within the same region.** This configuration ensures high availability (HA) for your Cloud SQL instance in the event of a zone failure.

Here's why:

Failover Replicas: Failover replicas are designed specifically for HA. They maintain a synchronous copy of the primary instance's data in a different zone within the same region.

Automatic Failover: In case of a primary zone failure, Cloud SQL will automatically promote the failover replica to become the new primary instance, minimizing downtime. This automatic promotion is crucial for

maintaining application availability.

Same Region Requirement: The failover replica must reside in the same region as the primary instance due to latency concerns and to ensure synchronous replication. Using a different region would introduce significant latency, impacting performance and making synchronous replication impossible.

Read Replicas (Option B): Read replicas are for scaling read operations and not designed for automatic failover. Promoting a read replica to be a writeable primary instance involves extra steps, and won't provide the automatic HA feature of a failover replica.

External Read Replicas (Option C): External read replicas in different regions are beneficial for disaster recovery (DR) and not for high availability within a region, as these would entail too much latency for synchronous replication. Also DR usually is handled with an RPO different from HA

Automatic Backup (Option D): While backups are essential for data recovery, they do not offer HA by themselves. Restoring from a backup takes time, resulting in significant downtime.

Therefore, a failover replica within the same region is the appropriate way to achieve high availability for Cloud SQL instances. It allows for automatic failover upon a zone failure, ensuring minimal downtime for the database.

Supporting Links:

Cloud SQL High Availability: <https://cloud.google.com/sql/docs/mysql/high-availability>

About Cloud SQL Replicas: <https://cloud.google.com/sql/docs/mysql/replication>

Question: 155

CertyIQ

Your company is selecting a system to centralize data ingestion and delivery. You are considering messaging and data integration systems to address the requirements. The key requirements are:

- ⇒ The ability to seek to a particular offset in a topic, possibly back to the start of all data ever captured
- ⇒ Support for publish/subscribe semantics on hundreds of topics

Retain per-key ordering -

Which system should you choose?

- A. Apache Kafka
- B. Cloud Storage
- C. Dataflow
- D. Firebase Cloud Messaging

Answer: A

Explanation:

It's the only technology that met the requirements

Question: 156

CertyIQ

You are planning to migrate your current on-premises Apache Hadoop deployment to the cloud. You need to ensure that the deployment is as fault-tolerant and cost-effective as possible for long-running batch jobs. You want to use a managed service. What should you do?

- A. Deploy a Dataproc cluster. Use a standard persistent disk and 50% preemptible workers. Store data in Cloud Storage, and change references in scripts from `hdfs://` to `gs://`
- B. Deploy a Dataproc cluster. Use an SSD persistent disk and 50% preemptible workers. Store data in Cloud Storage, and change references in scripts from `hdfs://` to `gs://`

C. Install Hadoop and Spark on a 10-node Compute Engine instance group with standard instances. Install the Cloud Storage connector, and store the data in Cloud Storage. Change references in scripts from `hdfs://` to `gs://`

D. Install Hadoop and Spark on a 10-node Compute Engine instance group with preemptible instances. Store data in HDFS. Change references in scripts from `hdfs://` to `gs://`

Answer: A

Explanation:

The correct answer is **A: Deploy a Dataproc cluster. Use a standard persistent disk and 50% preemptible workers. Store data in Cloud Storage, and change references in scripts from `hdfs://` to `gs://`**

Here's why:

Dataproc is a managed Hadoop and Spark service on Google Cloud, which addresses the need for a managed solution, simplifying deployment and management of Hadoop clusters. This contrasts with options C and D, which require manual installation and maintenance on Compute Engine, adding operational overhead. Using a managed service like Dataproc also allows for easier autoscaling.

For cost-effectiveness with long-running batch jobs, preemptible workers are ideal. They offer substantial cost savings compared to standard instances, especially for batch processing where interruptions can be tolerated. Option A and B employ them while Option C uses standard instances that have a higher cost and Option D uses preemptible workers but is not a managed service. Option A makes use of standard persistent disk as there's no mention of requiring high-performance disk.

Storing data in Cloud Storage (`gs://`) is crucial for decoupling compute and storage. This architecture enhances fault tolerance, as the data persists independently of the Dataproc cluster. Furthermore, Cloud Storage is cheaper and more durable than HDFS storage on the worker nodes. Options A, B and C use cloud storage while D uses HDFS storage.

Changing `hdfs://` references to `gs://` in scripts ensures that the Hadoop jobs can correctly access data from Cloud Storage. Dataproc is designed to work with Cloud Storage as a primary data source.

SSD persistent disks, as in option B, are generally overkill for batch processing and adds unnecessary costs where no higher IOPS is required. Standard persistent disks in option A are sufficient for the task. Option D incorrectly suggests storing data in HDFS, which is less resilient and not cost-effective for cloud environments.

In summary, option A combines the benefits of a managed service (Dataproc), cost-effective instances (preemptible), resilient and durable storage (Cloud Storage), and correct data access protocols, making it the optimal choice for the scenario.

Authoritative links:

Dataproc Overview: <https://cloud.google.com/dataproc/docs/concepts/overview>

Preemptible VMs: <https://cloud.google.com/compute/docs/instances/preemptible>

Cloud Storage: <https://cloud.google.com/storage/docs/>

Cloud Storage Connector: <https://cloud.google.com/hadoop/docs/connectors/>

Question: 157

CertyIQ

Your team is working on a binary classification problem. You have trained a support vector machine (SVM) classifier with default parameters, and received an area under the Curve (AUC) of 0.87 on the validation set. You want to increase the AUC of the model. What should you do?

A. Perform hyperparameter tuning

- B. Train a classifier with deep neural networks, because neural networks would always beat SVMs
- C. Deploy the model and measure the real-world AUC; it's always higher because of generalization
- D. Scale predictions you get out of the model (tune a scaling factor as a hyperparameter) in order to get the highest AUC

Answer: A

Explanation:

The correct answer is **A. Perform hyperparameter tuning**. Here's why:

An AUC of 0.87 indicates a reasonably good model, but there's room for improvement. SVMs, like many machine learning algorithms, have hyperparameters that significantly affect their performance. Default parameters often provide a good starting point but are rarely optimal for a specific dataset. Hyperparameter tuning involves systematically searching for the best combination of these parameters that maximizes performance on a validation set. Options like C, gamma, and kernel type can be adjusted to optimize for higher AUC.

Option B is incorrect because deep neural networks don't always outperform SVMs, especially on smaller datasets or when data isn't very complex. Neural networks also require significantly more training data and computational resources. Option C is misguided; real-world AUC isn't inherently higher, and model performance is not usually optimized at deployment. Option D, while scaling predictions could be a post-processing step, won't fundamentally improve the model's underlying ability to discriminate between classes, thereby impacting the AUC as much as tuning parameters. Instead of scaling a poor model, tuning is a more fundamental approach.

Therefore, hyperparameter tuning is the most direct and appropriate approach to improve the AUC of the SVM model in this scenario.

Supporting Links:

Hyperparameter Tuning:

[Google Cloud AI Platform: Hyperparameter Tuning](#)

[Scikit-learn: Tuning the hyper-parameters of an estimator](#)

Support Vector Machines:

[Scikit-learn: Support Vector Machines](#)

AUC (Area Under the Curve):

[Wikipedia: Receiver operating characteristic](#)

Question: 158

CertyIQ

You need to deploy additional dependencies to all nodes of a Cloud Dataproc cluster at startup using an existing initialization action. Company security policies require that Cloud Dataproc nodes do not have access to the Internet so public initialization actions cannot fetch resources. What should you do?

- A. Deploy the Cloud SQL Proxy on the Cloud Dataproc master
- B. Use an SSH tunnel to give the Cloud Dataproc cluster access to the Internet
- C. Copy all dependencies to a Cloud Storage bucket within your VPC security perimeter
- D. Use Resource Manager to add the service account used by the Cloud Dataproc cluster to the Network User role

Answer: C

Explanation:

The correct answer is **C. Copy all dependencies to a Cloud Storage bucket within your VPC security perimeter**. Here's why:

Cloud Dataproc clusters, as per the question's constraints, cannot access the internet directly. This restriction prevents using public initialization actions that fetch resources from external locations. Initialization actions are scripts run at cluster startup, often used to install dependencies or perform configurations. Option A, deploying the Cloud SQL Proxy, is irrelevant to the requirement of deploying arbitrary dependencies. Option B, using an SSH tunnel, contradicts the policy of no internet access and introduces unnecessary complexity. Option D, dealing with Network User roles, is completely unrelated to dependency deployment for Cloud Dataproc. Therefore, the appropriate approach is to make all required dependencies accessible within the secure network.

Cloud Storage buckets, residing within the user's VPC, fit this need perfectly. By copying dependencies into a designated bucket within the VPC, initialization scripts can securely access and install them without needing external internet access. The initialization action script can be modified to download the dependencies from this secure bucket. This ensures compliance with security policies while achieving the required outcome. This aligns with the best practice of restricting internet access for cloud resources for enhanced security. Cloud Storage provides scalability and reliable storage within the user's defined network perimeter.

Here are relevant authoritative links for further research:

Cloud Dataproc Initialization Actions: <https://cloud.google.com/dataproc/docs/concepts/init-actions>

Cloud Storage: <https://cloud.google.com/storage/docs>

VPC Service Controls: <https://cloud.google.com/vpc-service-controls/docs>

Question: 159

CertyIQ

You need to choose a database for a new project that has the following requirements:

- ⇒ Fully managed
- ⇒ Able to automatically scale up
- ⇒ Transactionally consistent
- ⇒ Able to scale up to 6 TB
- ⇒ Able to be queried using SQL

Which database do you choose?

- A. Cloud SQL
- B. Cloud Bigtable
- C. Cloud Spanner
- D. Cloud Datastore

Answer: A

Explanation:

The correct answer is A. Cloud SQL satisfies the requirements outlined. Firstly, Cloud SQL is a fully managed database service offered by Google Cloud Platform, eliminating the need for manual infrastructure management. It also supports automatic scaling up of compute resources to handle increased workloads. Transactional consistency is a core feature of relational databases like Cloud SQL, ensuring data integrity. Furthermore, Cloud SQL instances can be configured to store data up to 6 TB, fulfilling that requirement. Importantly, Cloud SQL is queried using standard SQL, allowing for familiar data manipulation. Cloud Bigtable (B) is a NoSQL database, not SQL, and while it scales massively, it does not provide a transactional consistency in the same way that SQL databases do. Cloud Spanner (C) meets scalability and SQL requirements but has more complexity and is more suitable when truly global scale is needed, it can also

exceed the 6TB limit. Finally, Cloud Datastore (D) is a NoSQL option, not SQL, and also is not the correct choice when transactional consistency is a priority. Cloud SQL is therefore the most suitable option, striking the right balance of features for the needs described.

For further information:

Cloud SQL Overview: <https://cloud.google.com/sql/docs/>

Cloud SQL Features: <https://cloud.google.com/sql/docs/features>

Cloud SQL Scaling: <https://cloud.google.com/sql/docs/mysql/manage-scaling>

Question: 160

CertyIQ

You work for a mid-sized enterprise that needs to move its operational system transaction data from an on-premises database to GCP. The database is about 20 TB in size. Which database should you choose?

- A. Cloud SQL
- B. Cloud Bigtable
- C. Cloud Spanner
- D. Cloud Datastore

Answer: A

Explanation:

Here's a detailed justification for choosing Cloud SQL in this scenario:

The primary requirement is migrating operational system transaction data, which implies a need for a relational database with ACID properties (Atomicity, Consistency, Isolation, Durability) to ensure data integrity. The database size (20TB) falls comfortably within the capabilities of Cloud SQL.

Cloud SQL, a fully managed relational database service, offers MySQL, PostgreSQL, and SQL Server options, all suitable for handling transaction data. It simplifies database administration tasks like patching, backups, and replication. Cloud SQL is well-suited for traditional applications requiring relational data modeling and transactional consistency.

Cloud Bigtable, on the other hand, is a NoSQL wide-column store designed for massive datasets and high-throughput, low-latency access, more appropriate for analytical workloads or applications requiring extreme scalability beyond 20TB. Cloud Spanner offers global scalability and strong consistency, but its complexity and cost are higher than Cloud SQL. Cloud Datastore, another NoSQL database, is suitable for document-oriented data and auto-scaling, but not ideal for complex transactions or relational data.

Therefore, Cloud SQL provides the best balance of functionality, cost-effectiveness, and ease of management for migrating a 20TB operational system transaction database to GCP. The other options are either overkill or lack the necessary relational database capabilities for this specific workload. The operational nature necessitates transactional support, making Cloud SQL the suitable choice.

Here are some authoritative links for further research:

Cloud SQL Overview: <https://cloud.google.com/sql/docs/introduction>

Cloud Bigtable Overview: <https://cloud.google.com/bigtable/docs/overview>

Cloud Spanner Overview: <https://cloud.google.com/spanner/docs/overview>

Cloud Datastore Overview (App Engine Datastore):

<https://cloud.google.com/appengine/docs/standard/python/datastore/>

Question: 161

CertyIQ

You need to choose a database to store time series CPU and memory usage for millions of computers. You need to store this data in one-second interval samples. Analysts will be performing real-time, ad hoc analytics against the database. You want to avoid being charged for every query executed and ensure that the schema design will allow for future growth of the dataset. Which database and data model should you choose?

- A. Create a table in BigQuery, and append the new samples for CPU and memory to the table
- B. Create a wide table in BigQuery, create a column for the sample value at each second, and update the row with the interval for each second
- C. Create a narrow table in Bigtable with a row key that combines the Computer Engine computer identifier with the sample time at each second
- D. Create a wide table in Bigtable with a row key that combines the computer identifier with the sample time at each minute, and combine the values for each second as column data.

Answer: C

Explanation:

The correct answer is **C. Create a narrow table in Bigtable with a row key that combines the Computer Engine computer identifier with the sample time at each second.**

Here's why:

Bigtable is a NoSQL, wide-column database specifically designed for time-series data and large-scale workloads. Its architecture excels at handling high write throughput and real-time reads, making it ideal for storing frequent, second-by-second metrics from millions of computers. Option A, using BigQuery, is not optimal for this use case. BigQuery is designed for analytical workloads and batch processing, not high-frequency data ingestion and real-time analysis. Appending new rows for each sample in BigQuery would be less efficient and costly compared to Bigtable. Option B, creating a wide table in BigQuery with columns for each second, is highly impractical due to the extreme width it would create for years of data, impacting performance. Moreover, BigQuery charges per query; therefore, it's not the ideal option for minimizing costs. Option D, using a wide table in Bigtable with minute granularity, would lose the second-level precision required. A narrow table in Bigtable, with the row key combining the computer ID and timestamp at each second (Option C), is the optimal choice. This structure allows for efficient retrieval of data based on computer and time ranges. It also facilitates scalability and minimizes costs associated with query execution, as Bigtable's pricing is primarily based on storage and throughput rather than query counts. Bigtable's schema design is highly flexible, allowing for future additions of more data points, making it the preferred option for future growth.

Authoritative Links:

Google Cloud Bigtable Overview: <https://cloud.google.com/bigtable/docs/overview>

Bigtable Schema Design: <https://cloud.google.com/bigtable/docs/schema-design>

BigQuery vs. Bigtable: <https://cloud.google.com/blog/products/data-analytics/choosing-the-right-google-cloud-database-for-time-series-data>

Question: 162

CertyIQ

You want to archive data in Cloud Storage. Because some data is very sensitive, you want to use the 'Trust No One' (TNO) approach to encrypt your data to prevent the cloud provider staff from decrypting your data. What should you do?

- A. Use gcloud kms keys create to create a symmetric key. Then use gcloud kms encrypt to encrypt each archival file with the key and unique additional authenticated data (AAD). Use gsutil cp to upload each encrypted file to the Cloud Storage bucket, and keep the AAD outside of Google Cloud.

B. Use `gcloud kms keys create` to create a symmetric key. Then use `gcloud kms encrypt` to encrypt each archival file with the key. Use `gsutil cp` to upload each encrypted file to the Cloud Storage bucket. Manually destroy the key previously used for encryption, and rotate the key once.

C. Specify customer-supplied encryption key (CSEK) in the `.boto` configuration file. Use `gsutil cp` to upload each archival file to the Cloud Storage bucket. Save the CSEK in Cloud Memorystore as permanent storage of the secret.

D. Specify customer-supplied encryption key (CSEK) in the `.boto` configuration file. Use `gsutil cp` to upload each archival file to the Cloud Storage bucket. Save the CSEK in a different project that only the security team can access.

Answer: A

Explanation:

The correct answer is **A**. Here's why:

Option A implements a "Trust No One" (TNO) approach by leveraging Cloud KMS for encryption while maintaining control over critical components of the encryption process, specifically, the Additional Authenticated Data (AAD). Here's the breakdown:

1. **Key Generation with Cloud KMS:** `gcloud kms keys create` creates a symmetric key managed within Google Cloud KMS. This ensures robust key management and access controls within the cloud environment. (Link: <https://cloud.google.com/kms/docs/creating-keys>)
2. **Encryption with Unique AAD:** Using `gcloud kms encrypt` to encrypt each archival file with the key and unique AAD is crucial. AAD adds context to the encryption process, meaning even if an attacker somehow accesses the encrypted data and the KMS key, they cannot decrypt the data without the specific AAD used. By generating unique AAD for each file, you effectively prevent a "copy and paste" attack, further strengthening the TNO stance.
3. **Storage and Upload:** The encrypted files are then uploaded to Cloud Storage using `gsutil cp`.
4. **Control Over AAD:** Keeping the AAD outside of Google Cloud ensures that the cloud provider cannot decrypt the data, fulfilling the TNO principle.

Option B fails because destroying the key after encryption renders the data unrecoverable without a mechanism for managing key rotation and subsequent access. Option C is problematic because storing CSEK in Memorystore can introduce security vulnerabilities. Furthermore, relying on CSEK alone is not as robust as encryption with a KMS-managed key and unique AAD. Option D, although better than option C, does not fully achieve TNO as it still relies on Google Cloud infrastructure to store the CSEK, albeit in a different project. The security team can still be compromised and leak the key. It does not protect against the service provider itself. TNO requires the user to maintain a complete separation between key and data. Option A is the best solution as the user can keep the AAD separated and not share it with google, the user is still using the managed key rotation and infrastructure within GCP.

In summary, Option A offers the most secure approach by combining the benefits of Google Cloud KMS's robust key management with user control of a vital part of the encryption process (the AAD), effectively implementing a TNO strategy. The user controls all information required for data decryption, meaning google would not be able to decrypt the data even if they wanted to.

Question: 163

CertyIQ

You have data pipelines running on BigQuery, Dataflow, and Dataproc. You need to perform health checks and monitor their behavior, and then notify the team managing the pipelines if they fail. You also need to be able to work across multiple projects. Your preference is to use managed products or features of the platform. What should you do?

- A. Export the information to Cloud Monitoring, and set up an Alerting policy
- B. Run a Virtual Machine in Compute Engine with Airflow, and export the information to Cloud Monitoring
- C. Export the logs to BigQuery, and set up App Engine to read that information and send emails if you find a failure in the logs
- D. Develop an App Engine application to consume logs using GCP API calls, and send emails if you find a failure in the logs

Answer: A

Explanation:

The most efficient and scalable approach to monitoring and alerting for data pipelines across multiple Google Cloud projects, utilizing managed services, is option A: exporting metrics and logs to Cloud Monitoring and setting up alerting policies. Cloud Monitoring is a fully managed service designed for this purpose, offering built-in integrations with BigQuery, Dataflow, and Dataproc. It provides a centralized platform to visualize, analyze, and set alerts based on various metrics and logs. Exporting data directly to Cloud Monitoring avoids the complexities of managing infrastructure like VMs (option B) or custom log analysis applications (options C and D). Utilizing Cloud Monitoring's alerting policies allows for proactive notifications based on predefined conditions, ensuring timely response to pipeline failures. This approach leverages the platform's managed services, reducing operational overhead. Alternatives like running Airflow on a VM require managing infrastructure, introducing unnecessary complexity. While analyzing logs in BigQuery or creating custom API consumption applications are possible, they demand significant development and maintenance effort, lacking the built-in alerting mechanisms of Cloud Monitoring. Thus, Cloud Monitoring's native integration, alerting capabilities, and managed nature makes it the most suitable solution.

Authoritative Links for further research:

Cloud Monitoring Overview: <https://cloud.google.com/monitoring/>

Creating Alerting Policies: <https://cloud.google.com/monitoring/alerts/>

Monitoring BigQuery: https://cloud.google.com/monitoring/api/metrics_gcp#gcp-bigquery

Monitoring Dataflow: https://cloud.google.com/monitoring/api/metrics_gcp#gcp-dataflow

Monitoring Dataproc: https://cloud.google.com/monitoring/api/metrics_gcp#gcp-dataproc

Question: 164

CertyIQ

You are working on a linear regression model on BigQuery ML to predict a customer's likelihood of purchasing your company's products. Your model uses a city name variable as a key predictive component. In order to train and serve the model, your data must be organized in columns. You want to prepare your data using the least amount of coding while maintaining the predictable variables. What should you do?

- A. Create a new view with BigQuery that does not include a column with city information.
- B. Use SQL in BigQuery to transform the state column using a one-hot encoding method, and make each city a column with binary values.
- C. Use TensorFlow to create a categorical variable with a vocabulary list. Create the vocabulary file and upload that as part of your model to BigQuery ML.
- D. Use Cloud Data Fusion to assign each city to a region that is labeled as 1, 2, 3, 4, or 5, and then use that number to represent the city in the model.

Answer: B

Explanation:

The correct answer is **B**. Here's why:

The core challenge lies in converting the city name, a categorical variable, into a numerical format suitable for

linear regression within BigQuery ML. Linear regression models require numerical inputs, and directly using string city names will not work. Options A, C, and D offer less optimal approaches. Option A eliminates the city variable entirely, losing valuable predictive information. Option C introduces TensorFlow and vocabulary files, adding complexity and moving away from the "least amount of coding" requirement, thus not leveraging BigQuery ML directly. Option D, using Data Fusion for region mapping, involves external tooling and adds an unnecessary intermediate step before using BigQuery ML.

Option B, utilizing SQL's one-hot encoding within BigQuery, directly addresses the problem with minimal code. One-hot encoding transforms the city column into multiple binary columns (one for each city), indicating the presence (1) or absence (0) of that city for each record. This is a standard method for representing categorical variables in machine learning and is efficiently performed using BigQuery SQL. Therefore, option B achieves the goal of preparing the data for a linear regression model in BigQuery ML using the least amount of code while maintaining city data as a predictor.

Further Research:

BigQuery ML documentation on creating models: <https://cloud.google.com/bigquery-ml/docs/create-model>

BigQuery SQL documentation on data transformation:

<https://cloud.google.com/bigquery/docs/reference/standard-sql/query-syntax>

One-hot encoding concept: <https://machinelearningmastery.com/one-hot-encoding-for-categorical-data/>

Question: 165

CertyIQ

You work for a large bank that operates in locations throughout North America. You are setting up a data storage system that will handle bank account transactions. You require ACID compliance and the ability to access data with SQL. Which solution is appropriate?

- A. Store transaction data in Cloud Spanner. Enable stale reads to reduce latency.
- B. Store transaction in Cloud Spanner. Use locking read-write transactions.
- C. Store transaction data in BigQuery. Disabled the query cache to ensure consistency.
- D. Store transaction data in Cloud SQL. Use a federated query BigQuery for analysis.

Answer: B

Explanation:

The correct answer is B: "Store transaction in Cloud Spanner. Use locking read-write transactions." This choice directly addresses the requirements of ACID compliance and SQL accessibility for handling bank account transactions. Cloud Spanner is a globally distributed, scalable database service that provides strong consistency, crucial for financial data. ACID properties (Atomicity, Consistency, Isolation, Durability) guarantee reliable transaction processing, ensuring data integrity. Specifically, the use of "locking read-write transactions" within Spanner enforces isolation, preventing concurrent modifications from corrupting data. Option A, while using Spanner, suggests stale reads, which compromise consistency and are unsuitable for financial transactions. Option C utilizes BigQuery, which is optimized for analytical workloads, not transactional data, and lacks the necessary ACID compliance. Option D uses Cloud SQL, which might provide ACID properties but misses the need for scaling and federation is complex. Cloud Spanner inherently supports SQL, making it easy to access and manipulate data. Therefore, option B is the ideal solution due to its provision of strong consistency, scalability, and SQL support, satisfying all required parameters of the scenario.

Authoritative links for further research:

Cloud Spanner Overview: <https://cloud.google.com/spanner/docs/overview>

Cloud Spanner Transactions: <https://cloud.google.com/spanner/docs/transactions>

Question: 166

A shipping company has live package-tracking data that is sent to an Apache Kafka stream in real time. This is then loaded into BigQuery. Analysts in your company want to query the tracking data in BigQuery to analyze geospatial trends in the lifecycle of a package. The table was originally created with ingest-date partitioning. Over time, the query processing time has increased. You need to implement a change that would improve query performance in BigQuery. What should you do?

- A. Implement clustering in BigQuery on the ingest date column.
- B. Implement clustering in BigQuery on the package-tracking ID column.
- C. Tier older data onto Cloud Storage files and create a BigQuery table using Cloud Storage as an external data source.
- D. Re-create the table using data partitioning on the package delivery date.

Answer: B**Explanation:**

The correct answer is **B. Implement clustering in BigQuery on the package-tracking ID column.**

Here's why:

The problem describes increased query processing time in a BigQuery table originally partitioned by `ingest_date`. While partitioning helps filter data based on the date it was ingested, it might not be optimal for analyzing geospatial trends related to specific packages. Analysts want to analyze trends in the lifecycle of a package, meaning they'll likely be querying for a specific package-tracking ID to understand its journey.

Clustering in BigQuery is designed to improve query performance by sorting data within each partition based on specified columns. When analysts query for a specific package-tracking ID, BigQuery can use the clustering to quickly locate the relevant blocks of data within the partitions, thus reducing the amount of data scanned. Clustering on the package-tracking ID allows BigQuery to efficiently retrieve all events related to that specific package, which is crucial for analyzing its lifecycle.

Option A is incorrect because clustering on the `ingest_date` is redundant since the table is already partitioned by it. Clustering on the same column as partitioning does not provide significant performance benefits and could even negatively impact it.

Option C involves tiering data to Cloud Storage and creating an external table. This is more suitable for archival purposes and might slow down query performance, as querying external tables is generally slower than querying native BigQuery tables. While cost-effective for infrequently accessed data, it doesn't directly address the need for improved performance when querying current package tracking data.

Option D, re-partitioning on the package delivery date, might be helpful for queries focused on deliveries within specific date ranges. However, it doesn't help queries that analyze the complete lifecycle of a package which involves querying across different delivery dates to trace a single package's journey. Furthermore, it requires rewriting the table, which is more disruptive than simply adding clustering.

Therefore, clustering on the package-tracking ID is the most effective solution for improving query performance when analyzing the lifecycle of individual packages in BigQuery.

Supporting Links:

BigQuery Clustering: <https://cloud.google.com/bigquery/docs/clustered-tables>

BigQuery Partitioning: <https://cloud.google.com/bigquery/docs/partitioned-tables>

Question: 167**CertyIQ**

Your company currently runs a large on-premises cluster using Spark, Hive, and HDFS in a colocation facility. The cluster is designed to accommodate peak usage on the system; however, many jobs are batch in nature, and usage of the cluster fluctuates quite dramatically. Your company is eager to move to the cloud to reduce the overhead associated with on-premises infrastructure and maintenance and to benefit from the cost savings. They are also hoping to modernize their existing infrastructure to use more serverless offerings in order to take advantage of the cloud. Because of the timing of their contract renewal with the colocation facility, they have only 2 months for their initial migration. How would you recommend they approach their upcoming migration strategy so they can maximize their cost savings in the cloud while still executing the migration in time?

- A. Migrate the workloads to Dataproc plus HDFS; modernize later.
- B. Migrate the workloads to Dataproc plus Cloud Storage; modernize later.
- C. Migrate the Spark workload to Dataproc plus HDFS, and modernize the Hive workload for BigQuery.
- D. Modernize the Spark workload for Dataflow and the Hive workload for BigQuery.

Answer: B**Explanation:**

The correct answer is **B. Migrate the workloads to Dataproc plus Cloud Storage; modernize later.** Here's why:

Given the tight two-month migration timeline and the desire for cost savings, a phased approach is crucial. Option B provides the most pragmatic path. Dataproc offers a managed Hadoop and Spark service, facilitating a relatively straightforward lift-and-shift of the existing Spark and Hive workloads. Replacing HDFS with Cloud Storage (Google Cloud Storage) is a critical improvement. Cloud Storage provides a scalable, durable, and cost-effective object storage solution, eliminating the need to manage HDFS infrastructure and its associated costs. This migration doesn't require substantial code changes, allowing for faster implementation.

Options A and C, while seemingly similar, retain HDFS, missing the opportunity to reduce operational overhead and cost, which is a key concern for the company. Option D advocates for a complete modernization to Dataflow and BigQuery within the short timeframe, which would involve significant code refactoring and potential delays, increasing risks in such a short migration window.

The "modernize later" aspect of Option B is vital, allowing the company to focus on a successful migration first. This approach aligns well with a phased migration strategy. After the initial migration, the company can then gradually modernize parts of their application to leverage serverless services like Dataflow and BigQuery at their own pace.

Therefore, Option B is the most suitable choice for a quick and cost-effective cloud migration, using Dataproc paired with Cloud Storage, focusing on the initial migration and modernization in later phases.

Relevant Links:

Google Cloud Dataproc: <https://cloud.google.com/dataproc>

Google Cloud Storage: <https://cloud.google.com/storage>

Phased Migration Strategy: <https://cloud.google.com/blog/products/gcp/six-strategies-for-migrating-to-the-cloud>

Question: 168**CertyIQ**

You work for a financial institution that lets customers register online. As new customers register, their user data is

sent to Pub/Sub before being ingested into BigQuery. For security reasons, you decide to redact your customers' Government issued Identification Number while allowing customer service representatives to view the original values when necessary. What should you do?

- A. Use BigQuery's built-in AEAD encryption to encrypt the SSN column. Save the keys to a new table that is only viewable by permissioned users.
- B. Use BigQuery column-level security. Set the table permissions so that only members of the Customer Service user group can see the SSN column.
- C. Before loading the data into BigQuery, use Cloud Data Loss Prevention (DLP) to replace input values with a cryptographic hash.
- D. Before loading the data into BigQuery, use Cloud Data Loss Prevention (DLP) to replace input values with a cryptographic format-preserving encryption token.

Answer: D

Explanation:

The correct answer is **D**, using Cloud Data Loss Prevention (DLP) to replace the government-issued ID with a cryptographic format-preserving encryption token before loading data into BigQuery. Here's why:

Option D provides the best solution for protecting sensitive data while allowing authorized users access to the original values. DLP's format-preserving encryption (FPE) ensures the redacted data maintains the same format as the original, which is crucial for downstream systems that may rely on specific patterns. This method also provides the capability to reverse the redaction (de-identify) with the proper key.

Option A, using BigQuery's AEAD encryption, is unsuitable because it's primarily for encrypting data at rest. While it provides security, it does not mask the column values. Moreover, it introduces complexity in key management which might not be manageable at scale.

Option B, using BigQuery's column-level security, is an access control mechanism that restricts user access to specific columns. However, the data is not masked, meaning users with access will see the original sensitive ID. Column-level security is complementary to data masking or tokenization, not a replacement for them.

Option C, using DLP to replace data with a cryptographic hash, is a one-way irreversible process that prevents authorized users from seeing the original values. While hashing provides strong protection, it does not address the requirement of allowing customer service representatives to view the original values when needed.

DLP's FPE, on the other hand, lets you de-identify the values in a manner that is reversible using the encryption key, which is managed outside the data storage. This approach adheres to the principle of least privilege by redacting the sensitive data for general usage while maintaining the ability to view the original data for authorized personnel. By applying this transformation before loading data into BigQuery, we ensure that sensitive values are never visible at rest unless specifically authorized. This reduces risk of exposure and complies with data privacy practices.

Authoritative Links:

Cloud DLP Overview: <https://cloud.google.com/dlp/docs/overview>

Cloud DLP Format-Preserving Encryption: <https://cloud.google.com/dlp/docs/transformations-format-preserving-encryption>

BigQuery Column-level Security: <https://cloud.google.com/bigquery/docs/column-level-security-intro>

Question: 169

CertyIQ

You are migrating a table to BigQuery and are deciding on the data model. Your table stores information related to purchases made across several store locations and includes information like the time of the transaction, items

purchased, the store ID, and the city and state in which the store is located. You frequently query this table to see how many of each item were sold over the past 30 days and to look at purchasing trends by state, city, and individual store. How would you model this table for the best query performance?

- A. Partition by transaction time; cluster by state first, then city, then store ID.
- B. Partition by transaction time; cluster by store ID first, then city, then state.
- C. Top-level cluster by state first, then city, then store ID.
- D. Top-level cluster by store ID first, then city, then state.

Answer: A

Explanation:

Here's a detailed justification for choosing option A, which is to partition by transaction time and cluster by state, then city, then store ID:

The primary goal is optimizing query performance, especially for filtering data within a 30-day window and analyzing trends by location. BigQuery's partitioning and clustering features are key for this. Partitioning divides a table into segments based on a column, typically a timestamp or date, reducing the amount of data scanned for queries that filter on the partition column. Since we frequently query sales over the past 30 days, partitioning by transaction time is the most effective way to minimize data scans, focusing on recent transactions, hence reducing query costs and time. Clustering further optimizes queries by sorting data within each partition based on specified columns, improving data retrieval for filtered queries. Given that queries analyze trends by state, city, and then store, clustering should follow this hierarchical order. Grouping by state initially will quickly reduce the scan to only relevant states, then filtering to particular cities within those states, and finally focusing on individual stores within those cities. This approach dramatically narrows down the amount of data being scanned at each stage of the query, enhancing query efficiency and reducing costs. Option B would have clustering by store ID first, which would make queries on states and cities very inefficient because the database would have to scan all the stores first. Options C and D don't utilize partitioning, which can cause inefficient scans for time based queries. The chosen strategy uses both partitioning and clustering effectively to minimize scanned data, ultimately leading to optimized query performance and reduced resource consumption. This approach is highly recommended for scenarios involving temporal data and hierarchical analysis, aligning with best practices for BigQuery.

Further research:

BigQuery Partitioned Tables: <https://cloud.google.com/bigquery/docs/partitioned-tables>

BigQuery Clustered Tables: <https://cloud.google.com/bigquery/docs/clustered-tables>

Optimizing query performance: <https://cloud.google.com/bigquery/docs/best-practices-performance>

Question: 170

CertyIQ

You are updating the code for a subscriber to a Pub/Sub feed. You are concerned that upon deployment the subscriber may erroneously acknowledge messages, leading to message loss. Your subscriber is not set up to retain acknowledged messages. What should you do to ensure that you can recover from errors after deployment?

- A. Set up the Pub/Sub emulator on your local machine. Validate the behavior of your new subscriber logic before deploying it to production.
- B. Create a Pub/Sub snapshot before deploying new subscriber code. Use a Seek operation to re-deliver messages that became available after the snapshot was created.
- C. Use Cloud Build for your deployment. If an error occurs after deployment, use a Seek operation to locate a timestamp logged by Cloud Build at the start of the deployment.
- D. Enable dead-lettering on the Pub/Sub topic to capture messages that aren't successfully acknowledged. If an error occurs after deployment, re-deliver any messages captured by the dead-letter queue.

Answer: B

Explanation:

Option B, creating a Pub/Sub snapshot before deploying new subscriber code, is the most effective approach for recovering from potential message loss due to erroneous acknowledgements during deployment. A Pub/Sub snapshot captures the current state of the subscription, allowing you to rewind the subscription to that point later. If errors occur and messages are incorrectly acknowledged after deployment, you can use the Seek operation to revert the subscription back to the snapshot's position. This effectively re-delivers the messages that were potentially lost. This solution directly addresses the concern of message loss caused by incorrect acknowledgements and provides a reliable mechanism for recovery.

Option A, using the emulator, is beneficial for testing but does not provide recovery capabilities after a production deployment. Option C, relying on Cloud Build logs and seek operation using timestamps, is indirect and less precise. It requires accurate timestamp correlation which might be problematic if the deployment process takes significant time. Option D, dead-lettering, is valuable for capturing messages that fail processing consistently, but it doesn't address the scenario of immediate, erroneous acknowledgements, as they're not failures of processing itself. Therefore, a snapshot with the Seek operation provides the most direct and reliable method to prevent the permanent loss of messages in this context. The key here is the ability to rewind to a previously known good point in the message stream, which snapshots and Seek provide.

Authoritative Links:

Pub/Sub Snapshots: <https://cloud.google.com/pubsub/docs/snapshots>

Pub/Sub Seek Operation: <https://cloud.google.com/pubsub/docs/replay-overview#seek>

Pub/Sub Dead-letter queues: <https://cloud.google.com/pubsub/docs/dead-letter-queues>

Question: 171

CertyIQ

You work for a large real estate firm and are preparing 6 TB of home sales data to be used for machine learning. You will use SQL to transform the data and use BigQuery ML to create a machine learning model. You plan to use the model for predictions against a raw dataset that has not been transformed. How should you set up your workflow in order to prevent skew at prediction time?

- A. When creating your model, use BigQuery's TRANSFORM clause to define preprocessing steps. At prediction time, use BigQuery's ML.EVALUATE clause without specifying any transformations on the raw input data.
- B. When creating your model, use BigQuery's TRANSFORM clause to define preprocessing steps. Before requesting predictions, use a saved query to transform your raw input data, and then use ML.EVALUATE.
- C. Use a BigQuery view to define your preprocessing logic. When creating your model, use the view as your model training data. At prediction time, use BigQuery's ML.EVALUATE clause without specifying any transformations on the raw input data.
- D. Preprocess all data using Dataflow. At prediction time, use BigQuery's ML.EVALUATE clause without specifying any further transformations on the input data.

Answer: A

Explanation:

The correct answer is A. Using BigQuery's TRANSFORM clause during model creation ensures that the same preprocessing steps are consistently applied to both training data and prediction data. By defining these steps within the model definition, you avoid the risk of inconsistencies or "skew" between training and prediction inputs. Option A correctly emphasizes the use of the TRANSFORM clause for model creation and specifies that the ML.PREDICT function (not ML.EVALUATE) should be called directly on raw data. BigQuery ML will then automatically apply transformations defined in the TRANSFORM clause during prediction. This ensures the raw data undergoes identical preprocessing as the training data, preventing the model from seeing differently formatted features at prediction time than what it learned from during training. Options B, C

and D do not consistently apply the same transformations to the training and evaluation phases. Option B requires a separate, explicit transformation of data which may not perfectly match the training transformations. Option C implies that evaluation would occur on untransformed data even if the training occurred on transformed data. Option D introduces additional complexity by using a different service (Dataflow) for processing that has to be manually maintained to match training transformations, when the transformations can be specified directly in the BigQuery ML model. Thus, A is the correct approach for achieving consistency and avoiding skew with BigQuery ML.

Authoritative Links:

[BigQuery ML TRANSFORM clause](#): Learn about defining transformations within the model creation.

[BigQuery ML Prediction](#): Understand how predictions work with models that have transformations.

[Avoiding Training Serving Skew in Machine Learning](#): Provides general guidance on avoiding skew when training ML models.

Question: 172

CertyIQ

You are analyzing the price of a company's stock. Every 5 seconds, you need to compute a moving average of the past 30 seconds' worth of data. You are reading data from Pub/Sub and using DataFlow to conduct the analysis. How should you set up your windowed pipeline?

- A. Use a fixed window with a duration of 5 seconds. Emit results by setting the following trigger: `AfterProcessingTime.pastFirstElementInPane().plusDelayOf (Duration.standardSeconds(30))`
- B. Use a fixed window with a duration of 30 seconds. Emit results by setting the following trigger: `AfterWatermark.pastEndOfWindow().plusDelayOf (Duration.standardSeconds(5))`
- C. Use a sliding window with a duration of 5 seconds. Emit results by setting the following trigger: `AfterProcessingTime.pastFirstElementInPane().plusDelayOf (Duration.standardSeconds(30))`
- D. Use a sliding window with a duration of 30 seconds and a period of 5 seconds. Emit results by setting the following trigger: `AfterWatermark.pastEndOfWindow ()`

Answer: D

Explanation:

Here's a detailed justification for why option D is the correct approach for calculating a 30-second moving average of stock prices every 5 seconds using Dataflow with Pub/Sub data:

The core requirement is to compute a "moving" average, which implies overlapping time windows. This immediately rules out fixed windows (options A and B), as they process data in non-overlapping chunks. Option A's 5-second fixed window would compute a simple average every 5 seconds, not a moving 30-second average. Option B's 30-second fixed window would compute the 30-second average only once every 30 seconds.

Option C uses a sliding window of 5 seconds, which is conceptually wrong for a 30-second average. Sliding windows, defined by duration and period, are needed here. The "duration" of 30 seconds captures the span of data for each average. The "period" of 5 seconds dictates how frequently we want to update the average by moving the window. Thus, D, with a 30-second duration and a 5-second period, is the proper way to achieve this behavior, generating a new average every five seconds based on the prior 30 seconds.

The `AfterWatermark.pastEndOfWindow()` trigger is the appropriate choice. This trigger ensures that Dataflow waits until all data expected within the window (given the processing watermark) has arrived before calculating the average. Using processing time triggers, as in A and C, is not recommended in streaming processing unless low latency is critical and some out-of-order arrival or late data is tolerable. This is not the optimal approach for most data analytics needs where completeness is prioritized.

In summary, Dataflow's sliding window with a 30-second duration and 5-second period, combined with an `AfterWatermark.pastEndOfWindow()` trigger, provides the necessary windowing logic to compute the 30-second moving average every 5 seconds from a Pub/Sub stream. The other options offer either incorrect windowing or inappropriate triggers for the desired analytics.

Authoritative Links:

Apache Beam Windowing: <https://beam.apache.org/documentation/programming-guide/#windowing>

Dataflow Windowing: <https://cloud.google.com/dataflow/docs/concepts/windowing>

Apache Beam Triggers: <https://beam.apache.org/documentation/programming-guide/#triggers>

Dataflow Triggers: <https://cloud.google.com/dataflow/docs/concepts/triggers>

Question: 173

CertyIQ

You are designing a pipeline that publishes application events to a Pub/Sub topic. Although message ordering is not important, you need to be able to aggregate events across disjoint hourly intervals before loading the results to BigQuery for analysis. What technology should you use to process and load this data to BigQuery while ensuring that it will scale with large volumes of events?

- A. Create a Cloud Function to perform the necessary data processing that executes using the Pub/Sub trigger every time a new message is published to the topic.
- B. Schedule a Cloud Function to run hourly, pulling all available messages from the Pub/Sub topic and performing the necessary aggregations.
- C. Schedule a batch Dataflow job to run hourly, pulling all available messages from the Pub/Sub topic and performing the necessary aggregations.
- D. Create a streaming Dataflow job that reads continually from the Pub/Sub topic and performs the necessary aggregations using tumbling windows.

Answer: D

Explanation:

Option D, using a streaming Dataflow job with tumbling windows, is the most suitable solution for this scenario. Here's why: The core requirement is to aggregate events across disjoint hourly intervals before loading them into BigQuery. Dataflow excels at both stream and batch processing, making it a versatile choice. Because the application events are published in real-time to Pub/Sub, a streaming job allows for continuous processing, which avoids the delay associated with batch processing options (like options B and C). Dataflow's tumbling windows are specifically designed for time-based aggregations, naturally segmenting the event stream into hourly chunks. This aligns perfectly with the requirement to aggregate events across hourly intervals. The streaming Dataflow job reads continuously from the Pub/Sub topic, eliminating the need for polling or scheduled runs required by other options, leading to a more efficient and responsive system. Dataflow is a fully managed service and built for scalability. It can handle large volumes of events with its autoscaling capabilities, ensuring consistent performance even with increased event traffic. This scalability makes it a better fit than a Cloud Function, which may have limitations in terms of resource allocation and execution time for high data volumes. After the aggregations are computed for each hourly window, the data can then be readily written to BigQuery using Dataflow's BigQueryIO connector. Therefore, a streaming Dataflow job with tumbling windows offers the most efficient, scalable, and reliable way to process and load the data according to the specified requirements.

Relevant links for further research:

Dataflow Windowing: <https://cloud.google.com/dataflow/docs/concepts/windowing>

Dataflow Streaming: <https://cloud.google.com/dataflow/docs/concepts/streaming-data>

Dataflow Pub/Sub IO: <https://cloud.google.com/dataflow/docs/io/pubsub>

Dataflow BigQuery IO: <https://cloud.google.com/dataflow/docs/io/bigquery>

Question: 174

CertyIQ

You work for a large financial institution that is planning to use Dialogflow to create a chatbot for the company's mobile app. You have reviewed old chat logs and tagged each conversation for intent based on each customer's stated intention for contacting customer service. About 70% of customer requests are simple requests that are solved within 10 intents. The remaining 30% of inquiries require much longer, more complicated requests. Which intents should you automate first?

- A. Automate the 10 intents that cover 70% of the requests so that live agents can handle more complicated requests.
- B. Automate the more complicated requests first because those require more of the agents' time.
- C. Automate a blend of the shortest and longest intents to be representative of all intents.
- D. Automate intents in places where common words such as 'payment' appear only once so the software isn't confused.

Answer: A

Explanation:

The optimal strategy is to prioritize automating the 10 intents covering 70% of customer requests (Option A). This approach leverages the Pareto principle, also known as the 80/20 rule, which suggests that roughly 80% of effects come from 20% of causes. In this case, automating the simpler, high-frequency intents will significantly reduce the overall workload on live agents, allowing them to dedicate their expertise to the more complex, time-consuming 30%. This reduces operational costs associated with manual intervention. Focusing on the simpler requests first also provides a faster return on investment and allows for quicker testing and refinement of the chatbot model. By automating the most common requests, the chatbot becomes more immediately useful and decreases the need for agent interactions, improving the overall customer experience. Starting with the simpler intents allows for a less complex implementation and reduces the risk associated with initial bot deployment. Automating the simpler requests first also allows you to learn and analyze the success of the automated flows and makes it easier to determine where to prioritize efforts to further improve the bot.

Options B, C, and D are less efficient initial strategies. Automating complex requests first (Option B) is more difficult and time-consuming and will likely not provide the same immediate impact as the first option. Automating a blend of intents (Option C) is not as effective as targeting the most frequent ones, because it doesn't address the biggest time-consuming factor. Focusing on word frequency within intents (Option D) has little relevance to efficiency in reducing workload.

Further research on best practices for chatbot development and the Pareto principle in business can be found at:

Dialogflow Documentation: <https://cloud.google.com/dialogflow/docs>

Pareto Principle: https://en.wikipedia.org/wiki/Pareto_principle

Chatbot Strategy Best Practices: Search for articles on "chatbot deployment strategy" or "AI customer service best practices".

Question: 175

CertyIQ

Your company is implementing a data warehouse using BigQuery, and you have been tasked with designing the data model. You move your on-premises sales data warehouse with a star data schema to BigQuery but notice performance issues when querying the data of the past 30 days. Based on Google's recommended practices, what should you do to speed up the query without increasing storage costs?

- A. Denormalize the data.

- B. Shard the data by customer ID.
- C. Materialize the dimensional data in views.
- D. Partition the data by transaction date.

Answer: D

Explanation:

The correct answer is D, partitioning the data by transaction date. This approach directly addresses the performance issue described by limiting the amount of data BigQuery needs to scan during queries focused on the last 30 days. By partitioning, BigQuery can prune data outside the specified date range, significantly reducing query time and cost by processing less data. Options A, B, and C, while potentially valid in some contexts, are less effective for this specific scenario. Denormalization (A) can improve read performance in some cases but will also increase storage costs which the question asks to avoid. Sharding by customer ID (B) would be suitable for queries based on customer-specific data but doesn't optimize for time-based queries. Materialized views (C) can speed up aggregations but aren't effective for this problem since the prompt is focused on a set timeframe. Partitioning by date, on the other hand, is Google's recommended best practice for time-series data in BigQuery, because it leverages the inherent time-based nature of the queries to improve performance by skipping partitions that do not match the query. This is a powerful feature of BigQuery specifically tailored for temporal data, ensuring efficient query execution without requiring additional storage costs.

For further information, consult Google's official documentation on BigQuery partitioning:

[Introduction to partitioned tables](#)

[Best practices for using partitioned tables](#)

Question: 176

CertyIQ

You have uploaded 5 years of log data to Cloud Storage. A user reported that some data points in the log data are outside of their expected ranges, which indicates errors. You need to address this issue and be able to run the process again in the future while keeping the original data for compliance reasons. What should you do?

- A. Import the data from Cloud Storage into BigQuery. Create a new BigQuery table, and skip the rows with errors.
- B. Create a Compute Engine instance and create a new copy of the data in Cloud Storage. Skip the rows with errors.
- C. Create a Dataflow workflow that reads the data from Cloud Storage, checks for values outside the expected range, sets the value to an appropriate default, and writes the updated records to a new dataset in Cloud Storage.
- D. Create a Dataflow workflow that reads the data from Cloud Storage, checks for values outside the expected range, sets the value to an appropriate default, and writes the updated records to the same dataset in Cloud Storage.

Answer: C

Explanation:

The correct answer is C because it addresses all requirements: identifying and correcting errors, preserving original data, and automating the process for future use.

Here's why other options are less suitable:

A: Importing data into BigQuery and skipping rows is not a good solution since you're losing data rather than correcting it. Additionally, the question asks for correcting data outside the accepted range not deleting it.

B: Using Compute Engine would require manual scripting and management, making it less scalable and maintainable than a managed service like Dataflow. Also, the cost is higher for managing a compute engine for the solution

D: Writing the updated records back to the same Cloud Storage dataset overwrites the original data, violating the compliance requirement of keeping the original data intact.

Option C utilizes Dataflow, a fully managed, serverless data processing service. Dataflow can read from Cloud Storage, perform data transformations (checking for out-of-range values and setting defaults), and write the corrected data to a new Cloud Storage location.

The benefits of using Dataflow are:

1. **Scalability:** It can handle large datasets efficiently.
2. **Automation:** The workflow can be scheduled and executed automatically in the future.
3. **Data Integrity:** Preserves the original data in Cloud Storage for compliance.
4. **Cost-Effectiveness:** Pay-as-you-go pricing model.
5. **Data Transformation:** Allows for data cleaning and standardization.

Dataflow pipelines are designed for both batch and stream processing, making them suitable for this type of data correction task. By reading the data, identifying and correcting invalid values, and writing the updated data to a new location, the solution ensures data quality while complying with data retention requirements.

Authoritative Links:

Google Cloud Dataflow: <https://cloud.google.com/dataflow>

Google Cloud Storage: <https://cloud.google.com/storage>

Question: 177

CertyIQ

You want to rebuild your batch pipeline for structured data on Google Cloud. You are using PySpark to conduct data transformations at scale, but your pipelines are taking over twelve hours to run. To expedite development and pipeline run time, you want to use a serverless tool and SQL syntax. You have already moved your raw data into Cloud Storage. How should you build the pipeline on Google Cloud while meeting speed and processing requirements?

- A. Convert your PySpark commands into SparkSQL queries to transform the data, and then run your pipeline on Dataproc to write the data into BigQuery.
- B. Ingest your data into Cloud SQL, convert your PySpark commands into SparkSQL queries to transform the data, and then use federated queries from BigQuery for machine learning.
- C. Ingest your data into BigQuery from Cloud Storage, convert your PySpark commands into BigQuery SQL queries to transform the data, and then write the transformations to a new table.
- D. Use Apache Beam Python SDK to build the transformation pipelines, and write the data into BigQuery.

Answer: C

Explanation:

The correct answer is **C**. Here's why:

The problem statement highlights the need for a serverless, SQL-based solution to expedite pipeline development and runtime for structured data transformations. Option C directly addresses these requirements. BigQuery is a serverless, fully managed data warehouse that uses SQL as its primary query language. Loading data from Cloud Storage into BigQuery is a straightforward process. Converting PySpark commands to BigQuery SQL allows for the transformations to run within the BigQuery environment, leveraging its massively parallel processing capabilities, significantly reducing execution time compared to a traditional Spark-based approach. Furthermore, writing the transformed data back into a new BigQuery table

fulfills the pipeline requirement and maintains structured data format. Option A introduces Dataproc, which is a managed Spark and Hadoop service. While powerful, it isn't fully serverless and requires cluster management, contradicting the requirement for a serverless approach. Options B and D are also unsuitable. Option B brings in Cloud SQL and federated queries, which are not needed and overly complicated for the stated requirements. Option D uses Apache Beam, which although serverless, does not use SQL as the primary language as specified in the problem statement. It also does not inherently solve the existing 12-hour runtime issue since the processing is still required to run.

Authoritative Links for Further Research:

BigQuery Overview: <https://cloud.google.com/bigquery/docs/introduction>

Loading Data into BigQuery: <https://cloud.google.com/bigquery/docs/loading-data-cloud-storage>

BigQuery SQL Reference: <https://cloud.google.com/bigquery/docs/reference/standard-sql>

Dataproc Overview: <https://cloud.google.com/dataproc/docs/concepts/overview>

Cloud SQL Overview: <https://cloud.google.com/sql/docs/introduction>

Apache Beam Overview: <https://beam.apache.org/documentation/>

Question: 178

CertyIQ

You are testing a Dataflow pipeline to ingest and transform text files. The files are compressed gzip, errors are written to a dead-letter queue, and you are using SideInputs to join data. You noticed that the pipeline is taking longer to complete than expected; what should you do to expedite the Dataflow job?

- A. Switch to compressed Avro files.
- B. Reduce the batch size.
- C. Retry records that throw an error.
- D. Use CoGroupByKey instead of the SideInput.

Answer: D

Explanation:

The correct answer is **D. Use CoGroupByKey instead of the SideInput**. Here's why:

Side inputs, while convenient, can introduce performance bottlenecks in Dataflow pipelines, especially with large datasets. They involve broadcasting the entire side input dataset to each worker. This can lead to increased memory usage and network traffic as workers repeatedly access the same data, causing slowdowns. CoGroupByKey, on the other hand, performs a distributed join operation. Data is grouped by keys across the main input and the "side input," which is now a regular PCollection. This grouping is handled by the Dataflow service, distributing the data processing across the worker pool, resulting in better resource utilization, reduced network traffic, and therefore a faster overall execution time. The process avoids broadcasting the side input, making it much more scalable. Using CoGroupByKey can be more performant than side inputs for large scale data operations.

While other options might have some impact, they don't directly address the described bottleneck:

A. Switch to compressed Avro files: Avro is a good format, but it's unlikely to address the core problem of side input inefficiency. File format affects I/O but not the join processing logic.

B. Reduce the batch size: Reducing batch size might improve responsiveness, but it could also increase overhead, and doesn't directly tackle the scaling problem introduced by side inputs.

C. Retry records that throw an error: Retrying records may be needed for data quality, but it won't directly affect overall pipeline performance concerning the usage of side inputs vs. CoGroupByKey.

Authoritative Links:

Apache Beam Programming Guide: Side Inputs: <https://beam.apache.org/documentation/programming-guide/#side-inputs> - This official Apache Beam documentation provides detailed information on the concept and implementation of side inputs and their limitations.

Apache Beam Programming Guide: Core Transforms: Grouping: <https://beam.apache.org/documentation/programming-guide/#grouping> - This page delves into CoGroupByKey and its benefits for distributed processing.

Question: 179

CertyIQ

You are building a real-time prediction engine that streams files, which may contain PII (personal identifiable information) data, into Cloud Storage and eventually into BigQuery. You want to ensure that the sensitive data is masked but still maintains referential integrity, because names and emails are often used as join keys. How should you use the Cloud Data Loss Prevention API (DLP API) to ensure that the PII data is not accessible by unauthorized individuals?

- A. Create a pseudonym by replacing the PII data with cryptogenic tokens, and store the non-tokenized data in a locked-down bucket.
- B. Redact all PII data, and store a version of the unredacted data in a locked-down bucket.
- C. Scan every table in BigQuery, and mask the data it finds that has PII.
- D. Create a pseudonym by replacing PII data with a cryptographic format-preserving token.

Answer: D

Explanation:

The correct answer is D. Here's a detailed justification:

The scenario requires masking PII data while maintaining referential integrity, which is crucial because the data (names, emails) are used as join keys. The DLP API offers various transformation methods, and format-preserving encryption (FPE) is ideal for this use case.

Option D leverages FPE. FPE is a type of encryption that preserves the format of the original data. This is crucial because it allows you to replace PII data (like email addresses) with tokens that have the same format as email addresses, so they can still be used as join keys without exposing the sensitive data. The format-preserving aspect ensures downstream applications and systems dependent on the original data format continue to function without modification. It maintains data integrity, usability, and analytical capabilities. Cryptographic methods ensure the tokenization is reversible with appropriate keys, but the tokenized data itself doesn't reveal the underlying PII.

Option A proposes using "cryptogenic tokens" which is a less precise term than FPE. It also mentions storing non-tokenized data in a locked-down bucket. While securing the original data is important, the problem focuses on masking during processing for referential integrity, which FPE directly addresses. The key advantage of FPE is that it avoids needing a separate store of the original values, reducing complexity and risk.

Option B, redacting all PII data, removes the data entirely, breaking referential integrity. The problem explicitly states that these PII values are used as join keys; therefore, redaction is not a viable solution. Storing unredacted data in a locked-down bucket simply moves the security problem without solving it.

Option C suggests scanning every table in BigQuery, which is a reactive approach. The problem requires masking before the data even lands in BigQuery. Scanning tables retroactively is resource-intensive and doesn't prevent initial exposure. Furthermore, it would require ongoing scanning as new data arrives. The streaming nature of the problem indicates that the data must be masked before it is written to BigQuery.

In summary, FPE ensures the masked data can still be used for joins, while the original data remains protected. It is a proactive approach that prevents PII exposure from the beginning of the data pipeline.

Relevant Links:

Cloud DLP Transformation Methods: <https://cloud.google.com/dlp/docs/transformations>

Format-Preserving Encryption: <https://cloud.google.com/dlp/docs/transformations-pseudonymization#format-preserving-encryption>

Question: 180

CertyIQ

You are migrating an application that tracks library books and information about each book, such as author or year published, from an on-premises data warehouse to BigQuery. In your current relational database, the author information is kept in a separate table and joined to the book information on a common key. Based on Google's recommended practice for schema design, how would you structure the data to ensure optimal speed of queries about the author of each book that has been borrowed?

- A. Keep the schema the same, maintain the different tables for the book and each of the attributes, and query as you are doing today.
- B. Create a table that is wide and includes a column for each attribute, including the author's first name, last name, date of birth, etc.
- C. Create a table that includes information about the books and authors, but nest the author fields inside the author column.
- D. Keep the schema the same, create a view that joins all of the tables, and always query the view.

Answer: C

Explanation:

The correct answer is C, which recommends nesting author information within the book record. This aligns with BigQuery's best practices for schema design, which prioritize denormalization for analytical workloads. Option A, maintaining separate tables, would necessitate joins, which can be costly and slower in BigQuery, especially when dealing with large datasets. Option B, creating a wide table with separate columns for each author attribute, could lead to redundancy and inefficient storage, as multiple books might have the same author information. Option D, using a view for joins, doesn't resolve the core performance issue of performing joins frequently. Nesting, as suggested in option C, allows BigQuery to read related data together within a single record, minimizing I/O operations and maximizing query speed. This approach is consistent with the common practice of "schema on read," where data is preprocessed for analytical access and queries aren't slowed by joins. By using nested and repeated fields, we effectively replicate a relational join within a single BigQuery table. This improves query performance because BigQuery excels in scanning massive amounts of data within a single table. The repeated fields also allow you to store multiple authors. This is optimized for analytical workloads where the focus is on reading and scanning large amounts of data, which is perfect for this scenario. Denormalization techniques are beneficial in read-heavy analytical databases like BigQuery where query speed is a priority.

For further research, see:

BigQuery Schema Design Best Practices: <https://cloud.google.com/bigquery/docs/best-practices-schema>

Nested and Repeated Fields in BigQuery: <https://cloud.google.com/bigquery/docs/nested-repeated>

Denormalization in Data Warehouses: <https://www.talend.com/resources/denormalization/>

Question: 181

CertyIQ

You need to give new website users a globally unique identifier (GUID) using a service that takes in data points and

returns a GUID. This data is sourced from both internal and external systems via HTTP calls that you will make via microservices within your pipeline. There will be tens of thousands of messages per second and that can be multi-threaded. and you worry about the backpressure on the system. How should you design your pipeline to minimize that backpressure?

- A. Call out to the service via HTTP.
- B. Create the pipeline statically in the class definition.
- C. Create a new object in the startBundle method of DoFn.
- D. Batch the job into ten-second increments.

Answer: D

Explanation:

The correct answer is **D. Batch the job into ten-second increments**. Here's why:

The core problem is managing backpressure from a high-volume, multi-threaded stream of incoming HTTP requests to a GUID generation service. Directly calling the service on each individual request (Option A) creates a bottleneck and can easily overwhelm the GUID service, leading to failures and data loss. Creating the pipeline statically (Option B) does not address backpressure. Creating objects in startBundle (Option C) is about state management within the pipeline but doesn't directly help with the overall throughput issue.

Batching the requests (Option D) significantly reduces the frequency of calls to the GUID service. Instead of tens of thousands of individual requests per second, we send fewer, larger requests of aggregated data every ten seconds. This reduces load and allows the GUID service to process a smaller number of requests at a more manageable rate. By accumulating data, and sending them together, it effectively introduces a natural pause that can help reduce the system backpressure.

Furthermore, batching aligns with best practices in stream processing. It allows for more efficient processing of data, as overhead associated with individual requests are reduced. It reduces API calls and makes the system more robust against fluctuations in incoming message rates. It also reduces the chance that the GUID service will be overwhelmed. Batches can be processed in parallel, helping to increase the overall system's throughput, within limits.

This method inherently addresses backpressure issues by smoothing out the data flow, preventing the GUID service from being constantly bombarded with individual requests, making it more resilient and scalable.

Authoritative Links:

Google Cloud Dataflow: Batch vs. Stream processing:

<https://cloud.google.com/dataflow/docs/concepts/streaming-vs-batch> (Highlights the trade-offs and scenarios for batch and stream processing).

Google Cloud Dataflow: Windowing: <https://cloud.google.com/dataflow/docs/concepts/windowing> (Explains how data is grouped and processed in windows, which is related to batching strategies).

Google Cloud Dataflow: Understanding backpressure:

<https://cloud.google.com/dataflow/docs/guides/troubleshooting-your-pipeline#backpressure> (Provides a detailed overview on backpressure in Dataflow and how to handle it).

Question: 182

CertyIQ

You are migrating your data warehouse to Google Cloud and decommissioning your on-premises data center. Because this is a priority for your company, you know that bandwidth will be made available for the initial data load to the cloud. The files being transferred are not large in number, but each file is 90 GB. Additionally, you want your transactional systems to continually update the warehouse on Google Cloud in real time. What tools should you use to migrate the data and ensure that it continues to write to your warehouse?

- A. Storage Transfer Service for the migration; Pub/Sub and Cloud Data Fusion for the real-time updates
- B. BigQuery Data Transfer Service for the migration; Pub/Sub and Dataproc for the real-time updates
- C. gsutil for the migration; Pub/Sub and Dataflow for the real-time updates
- D. gsutil for both the migration and the real-time updates

Answer: C

Explanation:

Here's a detailed justification for why option C is the most suitable solution, along with supporting explanations and links.

Justification for Option C: gsutil, Pub/Sub, and Dataflow

The initial data migration involves transferring a relatively small number of large (90GB) files. gsutil is a command-line tool that is efficient for transferring files to Cloud Storage, especially for large files because it provides options for parallel uploads and resumability, making it suitable for handling the 90GB files.

<https://cloud.google.com/storage/docs/gsutil>

For real-time updates to the data warehouse, a combination of Pub/Sub and Dataflow is the best approach. Pub/Sub acts as a message queue, enabling transactional systems to publish data events as they occur.

<https://cloud.google.com/pubsub/docs/overview>

Dataflow is a fully managed, serverless stream processing service that can consume these Pub/Sub messages and transform and load the data into the data warehouse (e.g., BigQuery). It's designed for continuous data pipelines and provides fault tolerance and scalability.

<https://cloud.google.com/dataflow/docs/concepts/what-is-dataflow>

Why other options are less suitable:

A (Storage Transfer Service & Cloud Data Fusion): Storage Transfer Service is better for transferring large quantities of files across various storage systems over a longer period and isn't optimized for a smaller number of very large files where maximizing bandwidth is crucial. Cloud Data Fusion, while capable, is more oriented towards batch and scheduled ETL jobs rather than real-time stream processing compared to Dataflow.

B (BigQuery Data Transfer Service & Dataproc): BigQuery Data Transfer Service is ideal for scheduled data transfers from specific SaaS applications to BigQuery, not for migrating arbitrary files. Dataproc is a managed Hadoop/Spark service, which is generally overkill for simply processing streams from Pub/Sub and loading them into a data warehouse. Dataflow offers a more streamlined and cost-effective solution for this specific real-time pipeline.

D (gsutil only): While gsutil is excellent for the initial data load, it does not inherently provide a mechanism for handling real-time data streams. It's a tool for file transfer, not a streaming data processing pipeline. You need services like Pub/Sub and Dataflow to handle the continuous, real-time updates effectively.

In summary, gsutil is appropriate for the initial one-time data migration of large files, while Pub/Sub and Dataflow together provide a robust and scalable solution for capturing and processing real-time updates to the data warehouse.

Question: 183

CertyIQ

You are using Bigtable to persist and serve stock market data for each of the major indices. To serve the trading application, you need to access only the most recent stock prices that are streaming in. How should you design your row key and tables to ensure that you can access the data with the simplest query?

- A. Create one unique table for all of the indices, and then use the index and timestamp as the row key design.
- B. Create one unique table for all of the indices, and then use a reverse timestamp as the row key design.
- C. For each index, have a separate table and use a timestamp as the row key design.
- D. For each index, have a separate table and use a reverse timestamp as the row key design.

Answer: B

Explanation:

The correct answer is **B: Create one unique table for all of the indices, and then use a reverse timestamp as the row key design.**

Here's why:

Bigtable is optimized for accessing data sorted by row key. Option B leverages this by using a reverse timestamp as the row key. This design has several advantages for accessing recent stock prices:

1. **Efficient Access to Recent Data:** Since reverse timestamps are used, the most recent data will be lexicographically first in the table. A simple scan operation (or a single-row read with timestamp-based filtering) will quickly retrieve the latest price for all indices without having to scan the entire table.
2. **Single Table Simplicity:** Using a single table is simpler to manage than creating separate tables for each index, particularly when the number of indices might change. It avoids the complexity of managing multiple table schemas and access patterns.
3. **Scalability:** Bigtable scales horizontally. A single table can accommodate a large volume of data from all indices, and Bigtable's automatic sharding will distribute load appropriately.
4. **Avoidance of Hotspotting:** Using the reverse timestamp avoids potential hotspotting issues when writing data. If forward timestamps were used, all new writes would be to the end of the table, potentially overloading the underlying servers managing that range. The reverse timestamp, especially when combined with the index as a secondary component of the key, helps distribute writes.
5. **Ease of Querying:** Retrieval of the most recent price for each index can be done by performing a scan operation that starts at the "newest" reverse timestamp and filters on the index ID. This approach simplifies queries compared to other configurations.

Options A, C, and D are less suitable:

Option A: While using timestamps as part of the row key works, it doesn't enable easy access to the most recent data without scanning the entire dataset and filtering for the maximum timestamp.

Options C & D: Using a separate table per index adds unnecessary management overhead and complicates cross-index analysis if needed. They introduce complexity in terms of managing multiple tables. Option D shares the advantage of using a reverse timestamp but is less manageable due to the multiple tables.

Authoritative links for further research:

Bigtable Schema Design: <https://cloud.google.com/bigtable/docs/schema-design>

Bigtable Row Keys: <https://cloud.google.com/bigtable/docs/row-keys>

Time-Series Data in Bigtable: <https://cloud.google.com/blog/products/gcp/time-series-data-in-cloud-bigtable>

Question: 184

CertyIQ

You are building a report-only data warehouse where the data is streamed into BigQuery via the streaming API. Following Google's best practices, you have both a staging and a production table for the data. How should you design your data loading to ensure that there is only one master dataset without affecting performance on either

the ingestion or reporting pieces?

- A. Have a staging table that is an append-only model, and then update the production table every three hours with the changes written to staging.
- B. Have a staging table that is an append-only model, and then update the production table every ninety minutes with the changes written to staging.
- C. Have a staging table that moves the staged data over to the production table and deletes the contents of the staging table every three hours.
- D. Have a staging table that moves the staged data over to the production table and deletes the contents of the staging table every thirty minutes.

Answer: C

Explanation:

Here's a detailed justification for why option C is the correct approach for managing streamed data in BigQuery with staging and production tables:

Option C, having a staging table that moves data to the production table and then clears the staging table every three hours, aligns with best practices for several reasons. Firstly, the staging table acts as a buffer for incoming streaming data, ensuring continuous ingestion without impacting reporting performance from the production table. This decouples the write operations from read operations. Secondly, moving data from staging to production in batches every three hours improves performance. This batch approach is generally more efficient than continuously updating the production table with individual streamed records as this could cause contention and lock issues in the production environment. Thirdly, removing data from the staging table after moving it to production maintains a clean and manageable pipeline. This avoids the staging table from accumulating redundant data over time, which could cause performance issues and increase storage costs. The data is migrated to production on a reasonably frequent basis, ensuring that the production data is fresh without jeopardizing the ingestion process. Options A and B, using an append-only staging table and updating production with changes, while seemingly plausible, would require more complex change management logic (like merge or update operations), which are not ideal for streamed data. Option D, although similar to C, proposes data transfer every 30 minutes which is too frequent for a streaming pipeline and can introduce unnecessary overhead.

In essence, option C offers a balance between data freshness, ingestion performance, and simplicity, making it the best fit for the described data warehousing scenario. The staging table serves its purpose as an interim area before the data becomes a master dataset in the production table, following a common extract-load-transform paradigm.

Authoritative Links:

Google Cloud Documentation - Streaming Data into BigQuery:

<https://cloud.google.com/bigquery/streaming-data-into-bigquery>

Google Cloud Documentation - BigQuery Data Loading: <https://cloud.google.com/bigquery/docs/loading-data>

Best Practices for Loading data: <https://cloud.google.com/bigquery/docs/best-practices-loading-data>

Question: 185

CertyIQ

You issue a new batch job to Dataflow. The job starts successfully, processes a few elements, and then suddenly fails and shuts down. You navigate to the Dataflow monitoring interface where you find errors related to a particular DoFn in your pipeline. What is the most likely cause of the errors?

- A. Job validation

- B. Exceptions in worker code
- C. Graph or pipeline construction
- D. Insufficient permissions

Answer: B

Explanation:

The most likely cause of a Dataflow job failing after initial success, exhibiting errors specifically within a DoFn, is **B. Exceptions in worker code**. Here's why:

Dataflow pipelines are executed by worker machines, and DoFns encapsulate the custom processing logic applied to each element. When a Dataflow job starts, it validates the overall pipeline structure (option A) during setup. If graph/pipeline construction (option C) or permission issues (option D) were the primary problems, the job would likely fail at initialization or be blocked from the start. Since the job processes some elements successfully before failing, the pipeline itself, permissions, and graph construction are likely correct. The sudden failure after initial success strongly suggests an issue within the DoFn's execution logic.

Specifically, exceptions occurring within the worker's code while processing a particular element or a specific condition can halt the processing of that element and often the entire worker thread. This leads to the Dataflow job failing with reported errors related to the specific DoFn. These exceptions might stem from unhandled input cases, type mismatches, null pointers, external service failures, or any unexpected condition the custom logic wasn't designed to handle. The logs and monitoring would pinpoint the DoFn responsible, confirming worker code exceptions as the root cause. Therefore, the focus for resolution lies within debugging and error handling of the custom DoFn code.

For further research, you can consult these resources:

1. **Dataflow Troubleshooting:** <https://cloud.google.com/dataflow/docs/guides/troubleshooting> This official guide provides information on diagnosing and resolving various Dataflow job failures.
2. **Dataflow Logging and Monitoring:** <https://cloud.google.com/dataflow/docs/guides/using-logging> Learn how to access and interpret logs to identify error sources.
3. **Dataflow Programming Model:** <https://beam.apache.org/documentation/programming-guide/> This document explains how to implement error handling and handle various failure modes when constructing Apache Beam pipelines.

Question: 186

CertyIQ

Your new customer has requested daily reports that show their net consumption of Google Cloud compute resources and who used the resources. You need to quickly and efficiently generate these daily reports. What should you do?

- A. Do daily exports of Cloud Logging data to BigQuery. Create views filtering by project, log type, resource, and user.
- B. Filter data in Cloud Logging by project, resource, and user; then export the data in CSV format.
- C. Filter data in Cloud Logging by project, log type, resource, and user, then import the data into BigQuery.
- D. Export Cloud Logging data to Cloud Storage in CSV format. Cleanse the data using Dataprep, filtering by project, resource, and user.

Answer: A

Explanation:

The correct answer is A because it leverages the strengths of both Cloud Logging and BigQuery for efficient

data analysis. Cloud Logging captures audit logs, which contain crucial information about resource consumption and user actions. Directly exporting these logs to BigQuery enables efficient querying and analysis at scale. By creating views within BigQuery, we can tailor the data to meet the specific reporting requirements, filtering by project, log type (like Compute Engine usage), resource, and user identity. This approach avoids the complexities of manual data processing and ensures consistently structured reports. Unlike option B, which suggests CSV export and manual handling, option A utilizes BigQuery's analytical capabilities. Option C involves redundant data transfer and extra import steps from Cloud Logging to BigQuery, making it less efficient than direct export. Option D introduces unnecessary complexity with intermediate storage and Dataprep processing, which is not required for the given requirements and adds latency to the process. Direct log export to BigQuery and subsequent view creation within BigQuery provides the most effective, scalable, and efficient method for creating the needed daily reports.

Here are some links for further research:

Cloud Logging Export to BigQuery: https://cloud.google.com/logging/docs/export/using_exported_logs

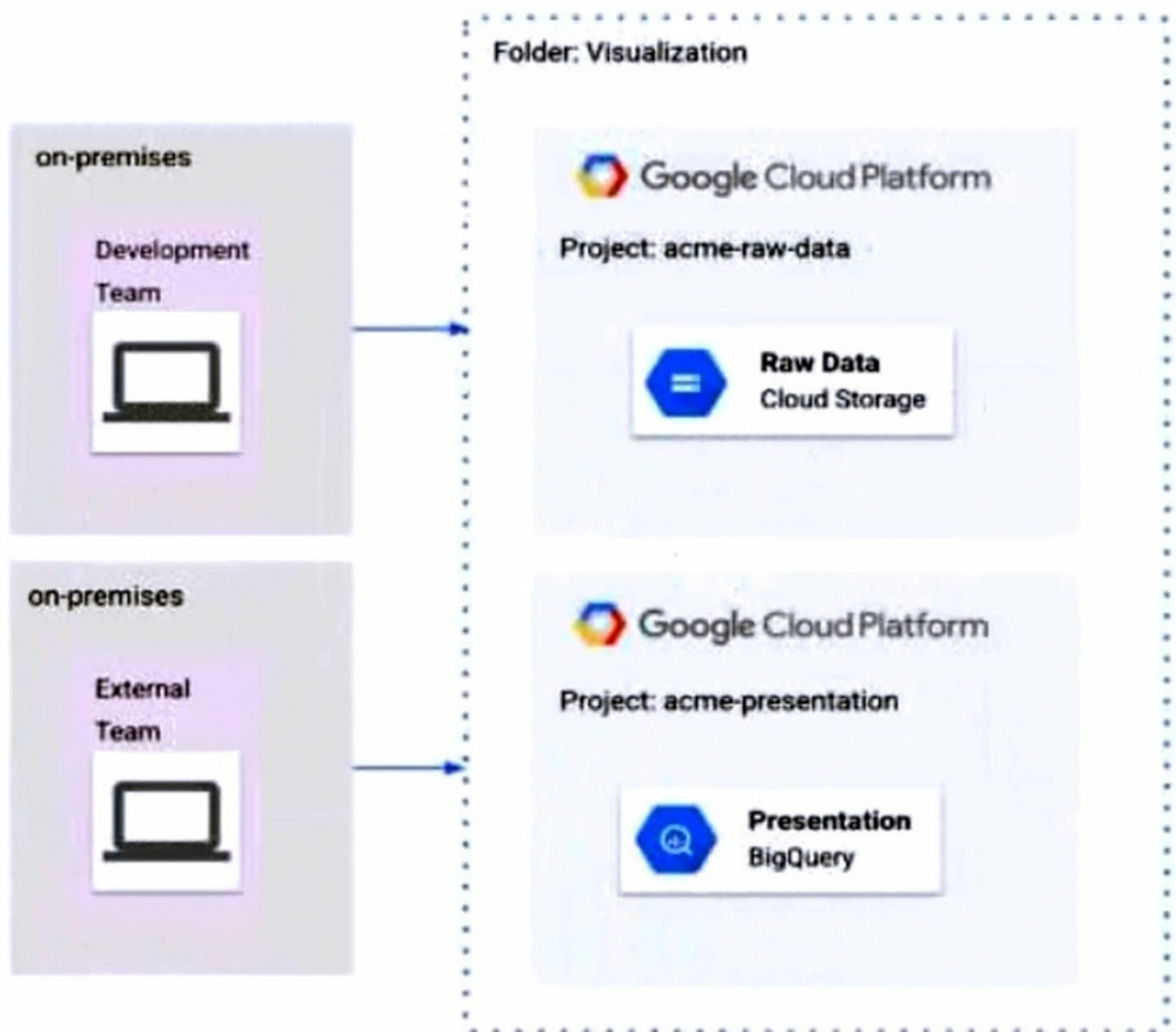
BigQuery Views: <https://cloud.google.com/bigquery/docs/views>

Audit Logs in Cloud Logging: <https://cloud.google.com/logging/docs/audit/>

Question: 187

CertyIQ

The Development and External teams have the project viewer Identity and Access Management (IAM) role in a folder named Visualization. You want the Development Team to be able to read data from both Cloud Storage and BigQuery, but the External Team should only be able to read data from BigQuery. What should you do?



- A. Remove Cloud Storage IAM permissions to the External Team on the acme-raw-data project.
- B. Create Virtual Private Cloud (VPC) firewall rules on the acme-raw-data project that deny all ingress traffic from the External Team CIDR range.
- C. Create a VPC Service Controls perimeter containing both projects and BigQuery as a restricted API. Add the External Team users to the perimeter's Access Level.
- D. Create a VPC Service Controls perimeter containing both projects and Cloud Storage as a restricted API. Add the Development Team users to the perimeter's Access Level.

Answer: D

Explanation:

Create a VPC Service Controls perimeter containing both projects and Cloud Storage as a restricted API. Add the Development Team users to the perimeter's Access Level.

Question: 188

CertyIQ

Your startup has a web application that currently serves customers out of a single region in Asia. You are targeting funding that will allow your startup to serve customers globally. Your current goal is to optimize for cost, and your post-funding goal is to optimize for global presence and performance. You must use a native JDBC driver. What should you do?

- A. Use Cloud Spanner to configure a single region instance initially, and then configure multi-region Cloud Spanner instances after securing funding.
- B. Use a Cloud SQL for PostgreSQL highly available instance first, and Bigtable with US, Europe, and Asia replication after securing funding.
- C. Use a Cloud SQL for PostgreSQL zonal instance first, and Bigtable with US, Europe, and Asia after securing funding.
- D. Use a Cloud SQL for PostgreSQL zonal instance first, and Cloud SQL for PostgreSQL with highly available configuration after securing funding.

Answer: A

Explanation:

The correct answer is A, utilizing Cloud Spanner. Initially, a single-region Cloud Spanner instance fulfills the requirement for cost optimization due to its lower resource footprint compared to a multi-region configuration. This aligns with the startup's current focus on managing expenses before securing funding. Cloud Spanner's architecture inherently supports horizontal scaling and strong consistency, making it suitable for global expansion. After securing funding, the startup can seamlessly transition to a multi-region Cloud Spanner instance, achieving global presence and performance optimization. This transition leverages Spanner's ability to replicate data across multiple regions, minimizing latency for users worldwide and improving fault tolerance. Options B, C, and D are less suitable. While Bigtable offers global capabilities, it is more appropriate for large analytical datasets, not the transactional use case implied by a web application using JDBC. Options B and C suggest initially using Cloud SQL for PostgreSQL, which is easier to set up but lacks the automatic horizontal scaling and global distribution required for future growth and the ability to directly use JDBC to connect to the Cloud SQL instance from within the client application. Option D's migration from zonal to high-availability Cloud SQL for PostgreSQL still doesn't achieve global reach, which is the desired outcome post-funding. Cloud Spanner is designed to handle the transactional workload of a web application. In contrast, Bigtable is suitable for large-scale analytic data rather than transactional data. Cloud SQL is useful for transactional use, but does not scale to global presence and has poor support for native JDBC. Therefore, option A is the only choice which aligns with both the initial need for cost optimization and the future need for global presence.

Further Research:

Cloud Spanner: <https://cloud.google.com/spanner>

Cloud SQL for PostgreSQL: <https://cloud.google.com/sql/docs/postgres>

Cloud Bigtable: <https://cloud.google.com/bigtable>

Question: 189

CertyIQ

You need to migrate 1 PB of data from an on-premises data center to Google Cloud. Data transfer time during the migration should take only a few hours. You want to follow Google-recommended practices to facilitate the large data transfer over a secure connection. What should you do?

- A. Establish a Cloud Interconnect connection between the on-premises data center and Google Cloud, and then use the Storage Transfer Service.
- B. Use a Transfer Appliance and have engineers manually encrypt, decrypt, and verify the data.
- C. Establish a Cloud VPN connection, start gcloud compute scp jobs in parallel, and run checksums to verify the data.
- D. Reduce the data into 3 TB batches, transfer the data using gsutil, and run checksums to verify the data.

Answer: A

Explanation:

Option A is the correct solution because it best addresses the requirements of transferring 1 PB of data in a few hours securely, aligning with Google's recommended practices. Cloud Interconnect provides a dedicated, high-bandwidth, low-latency connection between the on-premises data center and Google Cloud, bypassing the public internet and significantly speeding up data transfer. Storage Transfer Service is a fully managed service optimized for transferring large datasets into Google Cloud Storage, providing features like automated retries, data validation, and scheduling capabilities. This combination ensures a fast, reliable, and secure data transfer process, which is crucial for migrating such a large volume of data.

Option B, while involving physical appliances, requires manual encryption/decryption and verification, adding complexity and time, failing to meet the "few hours" requirement for transfer and increasing the risk of human error. Option C, using Cloud VPN and `gcloud compute scp`, relies on internet bandwidth and is not optimized for large-scale data transfers, making it a bottleneck. Parallel `scp` operations might help speed it up, but wouldn't be efficient and reliable like dedicated lines provided by Interconnect for such volume. Option D, breaking data into 3 TB batches and using `gsutil`, is also not suitable for petabyte-scale transfers; the internet connectivity will again be the bottleneck and the process can become unreliable for very large datasets. Furthermore, manually checksum verification for such a vast quantity of data would be time-consuming. Therefore, Option A's combination of a dedicated connection and a fully managed transfer service aligns with best practices for efficiently and securely moving large datasets like this one.

Further research links:

1. **Cloud Interconnect:** <https://cloud.google.com/interconnect/docs/>
2. **Storage Transfer Service:** <https://cloud.google.com/storage-transfer-service/docs>

Question: 190

CertyIQ

You are loading CSV files from Cloud Storage to BigQuery. The files have known data quality issues, including mismatched data types, such as STRINGS and INT64s in the same column, and inconsistent formatting of values such as phone numbers or addresses. You need to create the data pipeline to maintain data quality and perform the required cleansing and transformation. What should you do?

- A. Use Data Fusion to transform the data before loading it into BigQuery.
- B. Use Data Fusion to convert the CSV files to a self-describing data format, such as AVRO, before loading the data to BigQuery.
- C. Load the CSV files into a staging table with the desired schema, perform the transformations with SQL, and then write the results to the final destination table.
- D. Create a table with the desired schema, load the CSV files into the table, and perform the transformations in place using SQL.

Answer: A

Explanation:

Okay, let's break down why option A is the most suitable solution for handling CSV data quality issues during a BigQuery load process, and why the other options are less ideal.

Option A, using Data Fusion for pre-load transformation, aligns perfectly with the requirements. Data Fusion is a fully managed, visual data integration service that excels at building data pipelines, including data cleaning and transformation. It provides a user-friendly interface to design complex data flows using pre-built connectors and transformation components. This allows for visually defining steps to address data quality issues, such as type conversions, format standardization, and value cleansing before the data even reaches BigQuery. Crucially, this approach avoids ingesting dirty data into BigQuery, keeping the target tables clean and consistent.

Option B, converting to AVRO with Data Fusion before loading, while AVRO is self-describing and beneficial, doesn't directly address the data quality issues. AVRO defines schema alongside data, but still requires transformation logic before the conversion to clean up type mismatches and format inconsistencies. This option adds an unnecessary step compared to Option A, as the transformations are still required, and Data Fusion can directly address these with its transformation capabilities.

Option C, loading to a staging table for SQL-based transformation, is a viable approach but less efficient in the context of pre-load cleansing. While SQL can be used to transform data, moving the data to BigQuery initially creates an extra step and potentially stores poor quality data unnecessarily, incurring cost of storage of the stage table. Data Fusion can perform such transformations before data ever lands in bigquery. This approach also means data would be processed twice, once to load the stage table, then again using SQL. It's also more complex to manage and would make the pipeline harder to visualize.

Option D, directly loading and transforming in place, introduces many issues. It involves modifying the data directly in BigQuery, which can be complex to implement correctly, especially for schema-related problems and would create a very expensive and non efficient approach. Transforming in-place using SQL in BigQuery could become computationally expensive on big data sets. Data manipulation directly on BigQuery may cause operational issues, especially at very large scale and should be avoided when possible.

In summary, Data Fusion offers a robust visual environment, minimizing the need to write complex code and allowing for efficient and proactive data quality management before the data is ingested into BigQuery. It directly addresses the requirement of handling data quality issues.

Authoritative Links:

Google Cloud Data Fusion Overview: <https://cloud.google.com/data-fusion/docs/overview>

Data Fusion Transformations: <https://cloud.google.com/data-fusion/docs/concepts/transformation-reference>

BigQuery Loading Data: <https://cloud.google.com/bigquery/docs/loading-data>

Question: 191

CertyIQ

You are developing a new deep learning model that predicts a customer's likelihood to buy on your ecommerce site. After running an evaluation of the model against both the original training data and new test data, you find that your model is overfitting the data. You want to improve the accuracy of the model when predicting new data. What should you do?

- A. Increase the size of the training dataset, and increase the number of input features.
- B. Increase the size of the training dataset, and decrease the number of input features.
- C. Reduce the size of the training dataset, and increase the number of input features.
- D. Reduce the size of the training dataset, and decrease the number of input features.

Answer: B

Explanation:

The correct answer is B: Increase the size of the training dataset, and decrease the number of input features. Overfitting occurs when a model learns the training data too well, including its noise and random fluctuations. This results in poor generalization to new, unseen data. To combat overfitting, we need to help the model learn the underlying patterns instead of memorizing the training set. Increasing the size of the training dataset provides the model with more examples to learn from, making it less likely to overfit specific patterns or noise in the original training data. This allows the model to learn the general characteristics better, leading to better generalization. Decreasing the number of input features, also known as feature selection or dimensionality reduction, reduces the complexity of the model. A model with fewer features is less susceptible to overfitting, as it has fewer parameters to tune to noise. Reducing features often involves selecting the most relevant and informative ones, which also improves model accuracy and generalization. Options A, C, and D are incorrect.

Increasing the number of input features while trying to reduce overfitting will only make the problem worse, as the model will become more complex and likely capture more noise in the data. Reducing training data is counterintuitive, as it hinders the model's ability to learn generalized patterns, making it even more prone to overfitting.

Authoritative Links:

Google Cloud AI Platform: Machine Learning Crash Course - Overfitting:

<https://developers.google.com/machine-learning/crash-course/generalization/peril-of-overfitting>

Google Cloud AI Platform: Feature Selection: <https://cloud.google.com/ai-platform/docs/ml-solutions/feature-selection>

Machine Learning Mastery - Overfitting and Underfitting With Machine Learning Algorithms:

<https://machinelearningmastery.com/overfitting-and-underfitting-with-machine-learning-algorithms/>

Question: 192

CertyIQ

You are implementing a chatbot to help an online retailer streamline their customer service. The chatbot must be able to respond to both text and voice inquiries.

You are looking for a low-code or no-code option, and you want to be able to easily train the chatbot to provide answers to keywords. What should you do?

- A. Use the Cloud Speech-to-Text API to build a Python application in App Engine.
- B. Use the Cloud Speech-to-Text API to build a Python application in a Compute Engine instance.
- C. Use Dialogflow for simple queries and the Cloud Speech-to-Text API for complex queries.
- D. Use Dialogflow to implement the chatbot, defining the intents based on the most common queries collected.

Answer: D

Explanation:

The correct answer is **D. Use Dialogflow to implement the chatbot, defining the intents based on the most common queries collected.**

Here's why: Dialogflow is a Google Cloud service specifically designed for building conversational interfaces, including chatbots. It provides a low-code/no-code environment where you can easily define intents (user goals or actions) and corresponding responses, without extensive programming. This aligns with the requirement for a low-code/no-code solution and easy training based on keywords. Intents represent the user's intention, and Dialogflow uses machine learning to match user input (text or voice) to the appropriate intent. This handles the complexity of natural language understanding. The question also specifies responding to both text and voice, which is a primary feature of Dialogflow. You can integrate Dialogflow with various platforms, including websites and messaging apps. Speech-to-Text API is used for converting audio to text, but it doesn't provide a chatbot framework. While Dialogflow uses Google's speech-to-text under the hood, the service handles this integration, saving development time. Options A and B involve custom application development using Python and Cloud Speech-to-Text, which contradicts the "low-code/no-code" requirement and introduces more complexity than necessary. Option C suggests a hybrid approach, but Dialogflow is capable of handling both simple and relatively complex queries through its agent design.

Authoritative links:

Dialogflow Documentation: <https://cloud.google.com/dialogflow/docs>

Dialogflow Concepts: <https://cloud.google.com/dialogflow/docs/concepts>

What is Dialogflow? <https://cloud.google.com/dialogflow/what-is-dialogflow>

Question: 193

An aerospace company uses a proprietary data format to store its flight data. You need to connect this new data source to BigQuery and stream the data into BigQuery. You want to efficiently import the data into BigQuery while consuming as few resources as possible. What should you do?

- A. Write a shell script that triggers a Cloud Function that performs periodic ETL batch jobs on the new data source.
- B. Use a standard Dataflow pipeline to store the raw data in BigQuery, and then transform the format later when the data is used.
- C. Use Apache Hive to write a Dataproc job that streams the data into BigQuery in CSV format.
- D. Use an Apache Beam custom connector to write a Dataflow pipeline that streams the data into BigQuery in Avro format.

Answer: D

Explanation:

The correct answer is **D: Use an Apache Beam custom connector to write a Dataflow pipeline that streams the data into BigQuery in Avro format**. Here's why:

Option D leverages the power of Apache Beam and Dataflow for efficient and scalable data streaming. Apache Beam's flexible programming model allows creating custom connectors, perfect for handling the aerospace company's proprietary data format. By building a custom connector, you can directly read the proprietary data. Dataflow, a fully managed and serverless service, then executes the Beam pipeline, efficiently transforming the data and streaming it into BigQuery. Using Avro as the intermediate format is crucial. Avro is a binary, row-based data serialization format, renowned for its efficiency in data storage and processing, especially for streaming data. This minimizes the resources required compared to other formats such as CSV, aligning with the requirement of resource efficiency.

Option A, using shell scripts and Cloud Functions for periodic batch ETL, introduces latency and is less scalable than a streaming solution. Option B, using a standard Dataflow pipeline, would require an extra transformation step later for a non-standard format, adding complexity and resource usage. Option C, using Apache Hive and Dataproc, involves managing a cluster, which contradicts the requirement of minimal resource consumption, and is not designed for low latency streaming.

In summary, using a custom connector in an Apache Beam Dataflow pipeline coupled with Avro is the most efficient and appropriate solution to stream the proprietary data into BigQuery while minimizing resources and maximizing scalability.

Authoritative Links:

Apache Beam Custom Connectors: <https://beam.apache.org/documentation/programming-guide/#custom-io>

Google Cloud Dataflow: <https://cloud.google.com/dataflow/docs>

Avro Data Format: <https://avro.apache.org/>

BigQuery Streaming Inserts: <https://cloud.google.com/bigquery/streaming-data-into-bigquery>

Question: 194

An online brokerage company requires a high volume trade processing architecture. You need to create a secure queuing system that triggers jobs. The jobs will run in Google Cloud and call the company's Python API to execute trades. You need to efficiently implement a solution. What should you do?

- A. Use a Pub/Sub push subscription to trigger a Cloud Function to pass the data to the Python API.
- B. Write an application hosted on a Compute Engine instance that makes a push subscription to the Pub/Sub

topic.

C. Write an application that makes a queue in a NoSQL database.

D. Use Cloud Composer to subscribe to a Pub/Sub topic and call the Python API.

Answer: A

Explanation:

The correct answer is **A. Use a Pub/Sub push subscription to trigger a Cloud Function to pass the data to the Python API.**

Here's why:

Pub/Sub for Decoupling and Scalability: Pub/Sub is a highly scalable, fully managed messaging service ideal for high-volume event ingestion. It decouples the trade processing logic from the event producers (e.g., trade requests), improving system resilience and flexibility. <https://cloud.google.com/pubsub/docs/overview>

Push Subscriptions for Automatic Triggering: A push subscription automatically delivers messages to a specified endpoint, such as a Cloud Function. This eliminates the need for continuous polling by an application, saving resources and reducing latency.

Cloud Functions for Serverless Execution: Cloud Functions provide a serverless execution environment. They can be triggered by events, in this case, messages from the Pub/Sub subscription. This eliminates the overhead of managing server instances. <https://cloud.google.com/functions/docs/concepts/overview>

Python API Integration: Cloud Functions support multiple runtimes, including Python, making it easy to integrate with the company's existing Python API.

Efficiency: This approach is highly efficient because it leverages serverless components, auto-scaling, and push mechanisms. It only consumes resources when trades occur, making it cost-effective.

Let's analyze why the other options are not ideal:

B. Write an application hosted on a Compute Engine instance that makes a push subscription to the Pub/Sub topic: While Compute Engine can handle this, it requires managing instances, scaling them, and involves more operational overhead than Cloud Functions. The need to build, manage, and scale a custom application that's always on is less efficient than a serverless function.

C. Write an application that makes a queue in a NoSQL database: While NoSQL databases can act as queues, this introduces additional complexity in managing the queue and polling mechanism. Pub/Sub offers a purpose-built, fully managed solution specifically for messaging, making it more efficient for this use case. Using a database as a queue is an anti-pattern.

D. Use Cloud Composer to subscribe to a Pub/Sub topic and call the Python API: Cloud Composer is primarily an orchestration service for data pipelines. It's overkill for a simple trade execution workflow. Composer is better suited for scheduled, multi-step workflows, while this use case is event-driven and can be achieved by a serverless method more efficiently.

Question: 195

CertyIQ

Your company wants to be able to retrieve large result sets of medical information from your current system, which has over 10 TBs in the database, and store the data in new tables for further query. The database must have a low-maintenance architecture and be accessible via SQL. You need to implement a cost-effective solution that can support data analytics for large result sets. What should you do?

A. Use Cloud SQL, but first organize the data into tables. Use JOIN in queries to retrieve data.

B. Use BigQuery as a data warehouse. Set output destinations for caching large queries.

C. Use a MySQL cluster installed on a Compute Engine managed instance group for scalability.

D. Use Cloud Spanner to replicate the data across regions. Normalize the data in a series of tables.

Answer: B

Explanation:

The most appropriate solution is **B. Use BigQuery as a data warehouse. Set output destinations for caching large queries.** Here's why:

BigQuery is Google Cloud's fully managed, serverless data warehouse designed for analyzing large datasets. Its architecture excels at handling massive data volumes (like 10 TBs+) and executing complex SQL queries efficiently. This directly addresses the requirement for retrieving large result sets and storing them for further analysis. BigQuery's serverless nature minimizes maintenance overhead, aligning with the low-maintenance architecture requirement. The ability to cache large query results enhances performance and reduces costs for repeated analyses.

Cloud SQL (option A) is primarily for transactional workloads and struggles with very large datasets, requiring manual optimization for complex queries and joins. Using a MySQL cluster on Compute Engine (option C) introduces management complexities and scalability challenges. Cloud Spanner (option D), while suitable for globally distributed transactional data, is overkill for analytical workloads and is more expensive than BigQuery. BigQuery's cost model based on query processing and storage makes it a cost-effective solution for large-scale data analysis. Moreover, BigQuery natively supports SQL, meeting the accessibility requirement.

Authoritative Links:

BigQuery Documentation: <https://cloud.google.com/bigquery/docs>

Comparison of Data Warehouses: <https://cloud.google.com/architecture/data-warehouse>

BigQuery Pricing: <https://cloud.google.com/bigquery/pricing>

Question: 196

CertyIQ

You have 15 TB of data in your on-premises data center that you want to transfer to Google Cloud. Your data changes weekly and is stored in a POSIX-compliant source. The network operations team has granted you 500 Mbps bandwidth to the public internet. You want to follow Google-recommended practices to reliably transfer your data to Google Cloud on a weekly basis. What should you do?

- A. Use Cloud Scheduler to trigger the gsutil command. Use the -m parameter for optimal parallelism.
- B. Use Transfer Appliance to migrate your data into a Google Kubernetes Engine cluster, and then configure a weekly transfer job.
- C. Install Storage Transfer Service for on-premises data in your data center, and then configure a weekly transfer job.
- D. Install Storage Transfer Service for on-premises data on a Google Cloud virtual machine, and then configure a weekly transfer job.

Answer: C

Explanation:

The correct answer is **C. Install Storage Transfer Service for on-premises data in your data center, and then configure a weekly transfer job.**

Here's the justification:

Option C directly addresses the core requirements: a large on-premises dataset (15 TB), weekly updates, and a need for reliable transfer to Google Cloud. Storage Transfer Service (STS) for on-premises data is specifically designed for this scenario. STS leverages an agent installed in the source environment to securely transfer data to Google Cloud Storage. This approach avoids complex scripting and maximizes efficiency. It handles

data transfer scheduling, retries, and monitoring. Since the source is a POSIX-compliant system, STS can read directly from the location where the data is stored. This solution aligns with Google's recommendation for transferring large datasets. The service can intelligently handle changes, meaning only the new and modified files will be transferred during the weekly updates, saving time and bandwidth. The provided 500 Mbps bandwidth, while not very fast, can be handled by STS which will automatically make the most effective use of the available bandwidth.

Let's examine why the other options are less suitable:

A. Use Cloud Scheduler to trigger the gsutil command. While gsutil can transfer data, it's less robust for large, scheduled transfers, and does not scale as effectively as the STS. Also, gsutil doesn't have built-in change detection or resilience mechanisms, especially when dealing with a live file system.

B. Use Transfer Appliance to migrate your data into a Google Kubernetes Engine cluster. Transfer Appliance is best for initial large-scale transfers when network limitations are extreme or to avoid internet usage altogether, not recurring transfers. GKE is not a data storage solution itself, making this approach convoluted for weekly transfers. Moving data first to GKE then setting a transfer job is an inefficient and complicated workaround.

D. Install Storage Transfer Service for on-premises data on a Google Cloud virtual machine. This option adds unnecessary complexity. STS needs to be installed in the source data center to access the data directly, not on Google Cloud. Installing STS in a VM in Google Cloud won't have direct access to the on-premise data.

Authoritative links:

Storage Transfer Service Documentation: <https://cloud.google.com/storage-transfer-service/docs>

Storage Transfer Service for on-premises: <https://cloud.google.com/storage-transfer-service/docs/on-prem-overview>

Google Cloud Best Practices for Data Transfer: <https://cloud.google.com/architecture/data-transfer-methods>

Question: 197

CertyIQ

You are designing a system that requires an ACID-compliant database. You must ensure that the system requires minimal human intervention in case of a failure.

What should you do?

- A. Configure a Cloud SQL for MySQL instance with point-in-time recovery enabled.
- B. Configure a Cloud SQL for PostgreSQL instance with high availability enabled.
- C. Configure a Bigtable instance with more than one cluster.
- D. Configure a BigQuery table with a multi-region configuration.

Answer: B

Explanation:

The correct answer is **B. Configure a Cloud SQL for PostgreSQL instance with high availability enabled.**

Here's why:

ACID Compliance: PostgreSQL is known for its strong ACID (Atomicity, Consistency, Isolation, Durability) properties, making it suitable for applications requiring reliable transaction processing. Cloud SQL for PostgreSQL inherits these characteristics.

High Availability (HA): Enabling HA in Cloud SQL for PostgreSQL sets up a primary instance and a synchronous standby instance in a different zone within the same region. If the primary instance fails, the standby instance automatically takes over, minimizing downtime and requiring minimal manual intervention.

This failover is typically fast, often within seconds.

Minimal Human Intervention: The automatic failover in an HA configuration means that human operators don't need to manually intervene to restore database service after a failure. The system recovers automatically.

Why the other options are less suitable:

A. Cloud SQL for MySQL with point-in-time recovery: While point-in-time recovery is important for restoring data to a specific state, it doesn't address minimizing downtime during a failure. Recovery from a point-in-time backup still requires manual intervention and takes time, violating the minimal intervention requirement.

Although MySQL does offer HA options via Group Replication and InnoDB Cluster, these configurations are not specifically mentioned in the choice, whereas PostgreSQL explicitly offers HA and is fully ACID compliant.

C. Bigtable with multiple clusters: Bigtable is a NoSQL database, designed for scalability and speed, but it prioritizes availability and partition tolerance over strong consistency (CAP theorem). It doesn't provide the same level of ACID guarantees as a relational database like PostgreSQL. It is optimized for scale and is not strongly ACID compliant. Also, failover will still potentially require manual intervention to reroute traffic.

D. BigQuery with a multi-region configuration: BigQuery is a data warehouse designed for analytical workloads, not transactional applications that require ACID compliance. It is not suitable for handling real-time transactions with strict consistency requirements.

In summary, Cloud SQL for PostgreSQL with high availability provides the best combination of ACID compliance and automatic failover capabilities, ensuring minimal human intervention in case of a failure.

Relevant Links:

Cloud SQL for PostgreSQL: <https://cloud.google.com/sql/docs/postgres>

Cloud SQL High Availability: <https://cloud.google.com/sql/docs/postgres/configure-ha>

ACID Properties: <https://en.wikipedia.org/wiki/ACID>

Question: 198

CertyIQ

You are implementing workflow pipeline scheduling using open source-based tools and Google Kubernetes Engine (GKE). You want to use a Google managed service to simplify and automate the task. You also want to accommodate Shared VPC networking considerations. What should you do?

- A. Use Dataflow for your workflow pipelines. Use Cloud Run triggers for scheduling.
- B. Use Dataflow for your workflow pipelines. Use shell scripts to schedule workflows.
- C. Use Cloud Composer in a Shared VPC configuration. Place the Cloud Composer resources in the host project.
- D. Use Cloud Composer in a Shared VPC configuration. Place the Cloud Composer resources in the service project.

Answer: D

Explanation:

The correct answer is **D**. Cloud Composer is Google's managed Apache Airflow service, ideal for orchestrating workflow pipelines. It directly addresses the requirement for a managed service to simplify workflow scheduling, aligning with the problem statement's request. Shared VPC considerations are crucial in this context. To adhere to best practices with Shared VPC, Cloud Composer resources should reside in the service project. This approach maintains a logical separation between the network infrastructure (host project) and the application resources (service project), promoting security and management efficiency. Placing resources in the host project would violate the principle of least privilege and create unnecessary coupling between projects.

Dataflow, while suitable for data processing pipelines, doesn't directly handle scheduling in the same way as

a workflow orchestrator like Airflow. Using Cloud Run triggers or shell scripts for scheduling introduces additional management overhead that a managed workflow service like Cloud Composer aims to avoid. Therefore, option A is not ideal due to using Cloud Run for scheduling, and option B doesn't leverage managed services for scheduling. Option C, placing resources in the host project, incorrectly applies Shared VPC principles.

By choosing Cloud Composer in a Shared VPC configuration and deploying its resources within the service project, the design leverages a managed workflow tool with appropriate network management, aligning with the principles of the problem statement.

Authoritative Links:

Cloud Composer: <https://cloud.google.com/composer/docs>

Shared VPC: <https://cloud.google.com/vpc/docs/shared-vpc>

Cloud Composer with Shared VPC: <https://cloud.google.com/composer/docs/how-to/managing/shared-vpc>

Question: 199

CertyIQ

You are using BigQuery and Data Studio to design a customer-facing dashboard that displays large quantities of aggregated data. You expect a high volume of concurrent users. You need to optimize the dashboard to provide quick visualizations with minimal latency. What should you do?

- A. Use BigQuery BI Engine with materialized views.
- B. Use BigQuery BI Engine with logical views.
- C. Use BigQuery BI Engine with streaming data.
- D. Use BigQuery BI Engine with authorized views.

Answer: A

Explanation:

The optimal solution for low-latency visualization of large, aggregated datasets in a high-concurrency environment using BigQuery and Data Studio is to leverage BigQuery BI Engine with materialized views (Option A). BI Engine is a fast, in-memory analysis service that significantly accelerates query performance, especially for interactive dashboards. Materialized views, unlike logical views, precompute and store the results of a query, essentially creating a snapshot of the aggregated data. This precomputation means that Data Studio queries against these materialized views will retrieve data quickly from the cached results, bypassing the need to scan large tables each time. Logical views, in contrast, only define a query; they do not store any data, and therefore, the query must still run every time, leading to higher latency. Streaming data (Option C) is not relevant as the need is to optimize aggregated data. Authorized views (Option D) provide access control and do not impact query performance directly. Therefore, the combination of BigQuery BI Engine's in-memory processing and materialized views' precomputed results offers the best performance for high-volume, concurrent dashboard access, minimizing latency and delivering a smooth user experience.

Further research:

BigQuery BI Engine: <https://cloud.google.com/bigquery/docs/bi-engine-intro>

Materialized Views in BigQuery: <https://cloud.google.com/bigquery/docs/materialized-views-intro>

Logical Views in BigQuery: <https://cloud.google.com/bigquery/docs/views>

Question: 200

CertyIQ

Government regulations in the banking industry mandate the protection of clients' personally identifiable

information (PII). Your company requires PII to be access controlled, encrypted, and compliant with major data protection standards. In addition to using Cloud Data Loss Prevention (Cloud DLP), you want to follow Google-recommended practices and use service accounts to control access to PII. What should you do?

- A. Assign the required Identity and Access Management (IAM) roles to every employee, and create a single service account to access project resources.
- B. Use one service account to access a Cloud SQL database, and use separate service accounts for each human user.
- C. Use Cloud Storage to comply with major data protection standards. Use one service account shared by all users.
- D. Use Cloud Storage to comply with major data protection standards. Use multiple service accounts attached to IAM groups to grant the appropriate access to each group.

Answer: D

Explanation:

Here's a detailed justification for why option D is the correct approach, along with relevant cloud concepts and supporting links:

Option D, using Cloud Storage with multiple service accounts attached to IAM groups, aligns best with Google's recommended practices for secure PII handling. The core principle is to grant least privilege, meaning each user or application should only have the permissions they absolutely need. Sharing a single service account (options A & C) violates this, creating a security risk. If that single account is compromised, all PII is potentially exposed. Option B, while separating access for the database, falls short as it doesn't address PII access across other services, like Cloud Storage which is also mentioned in the question.

IAM groups streamline access management. By associating service accounts with groups, you can manage permissions at a group level instead of having to update individual service account permissions every time a user's role changes. This significantly simplifies security administration and reduces the likelihood of human error. Each service account represents a distinct identity for a specific group or application. This aligns perfectly with the principle of least privilege and allows for precise audit trails.

Cloud Storage is also a cornerstone of secure data management on GCP, offering encryption at rest and in transit as well as mechanisms to control access. Combining its security features with granular access control using IAM and service accounts provides the multi-layered security necessary for compliance with data protection regulations.

Furthermore, using service accounts allows for programmatic access to resources, essential for automated processes interacting with PII. These processes can have their access limited to only the resources needed, which further strengthens the security stance.

In essence, option D demonstrates best practices of using multiple service accounts to grant least privilege based on IAM groups to align with data protection regulations.

Relevant links:

Service Accounts: <https://cloud.google.com/iam/docs/service-accounts>

IAM Best Practices: <https://cloud.google.com/iam/docs/best-practices>

Cloud Storage Security: <https://cloud.google.com/storage/docs/security>

Principle of Least Privilege: <https://cloud.google.com/blog/products/identity-security/least-privilege-access-on-google-cloud>

Question: 201

CertyIQ

You need to migrate a Redis database from an on-premises data center to a Memorystore for Redis instance. You

want to follow Google-recommended practices and perform the migration for minimal cost, time and effort. What should you do?

- A. Make an RDB backup of the Redis database, use the gsutil utility to copy the RDB file into a Cloud Storage bucket, and then import the RDB file into the Memorystore for Redis instance.
- B. Make a secondary instance of the Redis database on a Compute Engine instance and then perform a live cutover.
- C. Create a Dataflow job to read the Redis database from the on-premises data center and write the data to a Memorystore for Redis instance.
- D. Write a shell script to migrate the Redis data and create a new Memorystore for Redis instance.

Answer: A

Explanation:

Option A is the most efficient and cost-effective approach for migrating an on-premises Redis database to Memorystore, adhering to Google's recommended practices. Creating an RDB backup of the existing Redis instance captures a snapshot of its data. Using gsutil to transfer this backup to a Cloud Storage bucket leverages a secure and scalable storage mechanism. Memorystore for Redis supports importing RDB files directly, simplifying the migration process without complex transformations. This method minimizes downtime by avoiding live synchronization and uses readily available tools and features, thus reducing both effort and time. It also avoids the cost overhead of maintaining additional compute instances or creating custom data pipelines. Options B, C, and D involve more complex, time-consuming, and potentially expensive approaches. A secondary Redis instance (B) requires resource provisioning and management. Dataflow (C) introduces unnecessary complexity for a straightforward data transfer. Scripting a custom solution (D) requires manual development and maintenance, increasing effort and risk.

Justification Breakdown:

RDB Backup: Redis's built-in RDB mechanism efficiently creates a point-in-time snapshot of data.

gsutil: A command-line utility that provides fast and reliable data transfer to and from Cloud Storage.

Cloud Storage: Scalable, durable, and cost-effective object storage for data staging.

Memorystore Import: Memorystore directly supports importing data from RDB files for a streamlined migration.

Cost Efficiency: Avoids complex architecture and resource overhead.

Minimal Effort: Utilizes readily available tools and features of Redis and GCP.

Reduced Time: Faster migration compared to live migration and custom code.

Authoritative Links:

Memorystore for Redis Documentation (Import data):

<https://cloud.google.com/memorystore/docs/redis/import-export-rdb>

gsutil documentation: <https://cloud.google.com/storage/docs/gsutil>

Redis RDB documentation: <https://redis.io/docs/data-types/persistence/#rdb-snapshots>

Question: 202

CertyIQ

Your platform on your on-premises environment generates 100 GB of data daily, composed of millions of structured JSON text files. Your on-premises environment cannot be accessed from the public internet. You want to use Google Cloud products to query and explore the platform data. What should you do?

- A. Use Cloud Scheduler to copy data daily from your on-premises environment to Cloud Storage. Use the BigQuery Data Transfer Service to import data into BigQuery.
- B. Use a Transfer Appliance to copy data from your on-premises environment to Cloud Storage. Use the BigQuery Data Transfer Service to import data into BigQuery.

C. Use Transfer Service for on-premises data to copy data from your on-premises environment to Cloud Storage. Use the BigQuery Data Transfer Service to import data into BigQuery.

D. Use the BigQuery Data Transfer Service dataset copy to transfer all data into BigQuery.

Answer: C

Explanation:

The correct answer is C because it provides the most efficient and suitable solution for transferring the described on-premises data to Google Cloud Platform (GCP) for analysis with BigQuery. Since the on-premises environment is not publicly accessible, options involving direct internet transfers or public data transfer services are inappropriate.

Option A is incorrect because Cloud Scheduler is a job scheduling service and does not have the functionality to directly copy data from an on-premises environment, especially one that is not publicly accessible.

Option B is incorrect. While a Transfer Appliance is suitable for large, initial data migrations, it is overkill for a daily transfer of 100 GB of data. Transfer Appliances are best when handling significant amounts of data to minimize time and bandwidth constraints for initial setup, and not designed to be used for daily transfers of this size.

Option C, the chosen answer, utilizes Transfer Service for on-premises data, a Google Cloud service specifically designed to securely and efficiently move data from on-premises storage to Cloud Storage without public internet access. This solution works by creating an agent that resides inside the on-premises environment, connects to the cloud, and then transfers the data to cloud storage. Once the data lands in Cloud Storage, the BigQuery Data Transfer Service can then easily import the JSON text files into BigQuery for querying. The structure of the data (JSON text files) is well-suited for BigQuery's schema detection capabilities, making this an effective choice.

Option D is incorrect as the BigQuery Data Transfer Service dataset copy is designed for moving data between BigQuery datasets, not for transferring data from an on-premises environment. The dataset copy functionality is not designed to ingest external data sources.

Therefore, option C provides the correct methodology, using the appropriate tools for the task of securely transferring the data to Google Cloud from the on-premise, non-publicly accessible location and then loading the data into BigQuery.

Supporting Links:

Transfer Service for on-premises data: <https://cloud.google.com/transfer/docs/on-premises-overview>

BigQuery Data Transfer Service: <https://cloud.google.com/bigquery/docs/transfer-service>

Cloud Scheduler: <https://cloud.google.com/scheduler>

Transfer Appliance: <https://cloud.google.com/transfer-appliance/docs/overview>

Question: 203

CertyIQ

A TensorFlow machine learning model on Compute Engine virtual machines (n2-standard-32) takes two days to complete training. The model has custom TensorFlow operations that must run partially on a CPU. You want to reduce the training time in a cost-effective manner. What should you do?

- A. Change the VM type to n2-highmem-32.
- B. Change the VM type to e2-standard-32.
- C. Train the model using a VM with a GPU hardware accelerator.
- D. Train the model using a VM with a TPU hardware accelerator.

Answer: C

Explanation:

The most cost-effective way to reduce the training time of the TensorFlow model, given the constraint of needing CPU processing for custom operations, is to leverage a GPU. Here's why:

Option A (n2-highmem-32) increases memory but doesn't address the processing bottleneck. While more memory might slightly improve performance, it's unlikely to provide a significant reduction in training time proportional to the increased cost.

Option B (e2-standard-32) might be cheaper than n2-standard-32, but it generally offers lower performance. Therefore, it would likely increase training time, not decrease it.

Option C (GPU VM) is the most suitable. GPUs are highly optimized for the tensor operations that form the core of most machine learning workloads. Even with some operations needing CPU intervention, the bulk of the computation can be offloaded to the GPU, leading to substantial speedups. The cost-effectiveness comes from the ability of a GPU to process massive amounts of data in parallel, reducing the overall wall-clock training time and potentially allowing for using a smaller, less expensive CPU configuration.

Option D (TPU VM) is designed for high-performance machine learning, however, TPUs have more limited support for custom operations compared to GPUs. Since the problem description explicitly mentions custom TensorFlow operations that run on the CPU, using TPUs might require significant refactoring or not be feasible at all. The cost to refactor and potential performance limitations outweigh the benefits in this case. Furthermore, TPUs require the model to be specifically coded to take advantage of the TPU architecture which would represent significant time investment.

In summary, a GPU-enabled VM offers the best balance between performance improvement and code compatibility, directly addressing the computationally intensive part of the TensorFlow training while allowing the existing CPU-based custom operations to function as intended.

Supporting links:

GPUs for Machine Learning on Google Cloud: <https://cloud.google.com/gpu>

TensorFlow performance with GPUs: <https://www.tensorflow.org/guide/gpu>

Question: 204

CertyIQ

You want to create a machine learning model using BigQuery ML and create an endpoint for hosting the model using Vertex AI. This will enable the processing of continuous streaming data in near-real time from multiple vendors. The data may contain invalid values. What should you do?

- A. Create a new BigQuery dataset and use streaming inserts to land the data from multiple vendors. Configure your BigQuery ML model to use the "ingestion" dataset as the framing data.
- B. Use BigQuery streaming inserts to land the data from multiple vendors where your BigQuery dataset ML model is deployed.
- C. Create a Pub/Sub topic and send all vendor data to it. Connect a Cloud Function to the topic to process the data and store it in BigQuery.
- D. Create a Pub/Sub topic and send all vendor data to it. Use Dataflow to process and sanitize the Pub/Sub data and stream it to BigQuery.

Answer: D

Explanation:

The most appropriate solution is **D**, leveraging Pub/Sub, Dataflow, and BigQuery. Here's why:

Pub/Sub for Streaming Ingestion: Pub/Sub is ideal for ingesting high-volume, real-time data streams from multiple sources. This decouples the data producers from the consumers, providing scalability and resilience. <https://cloud.google.com/pubsub/docs/overview>

Dataflow for Stream Processing and Sanitization: Dataflow is a fully managed stream processing service that excels at data transformation and cleaning. It can read from Pub/Sub, handle invalid data, and apply necessary sanitization rules before the data lands in BigQuery.

<https://cloud.google.com/dataflow/docs/concepts/dataflow-model>

BigQuery as a Data Sink: BigQuery is a highly scalable data warehouse, perfect for storing and querying large datasets, including those intended for machine learning. It can handle the continuous stream of sanitized data coming from Dataflow. <https://cloud.google.com/bigquery/docs/introduction>

End-to-End Solution: This approach creates an end-to-end streaming pipeline where data is ingested, cleaned, and then stored, ready for BigQuery ML modeling and subsequent hosting on Vertex AI.

Let's address why the other options are less suitable:

Option A: Using BigQuery streaming inserts directly with "ingestion" as framing data, without a prior processing step, is risky because it cannot handle invalid values effectively. Also, using BigQuery as the ingestion layer is generally not recommended for high-throughput streams and lacks the flexibility offered by Pub/Sub.

Option B: Directly using BigQuery streaming inserts without preprocessing will suffer from the same issue as Option A regarding invalid values and is not ideal for complex streaming data.

Option C: Although Cloud Functions can process data from Pub/Sub, they are better suited for event-driven, short-lived processes, and not for the heavy lifting involved in continuous, stream processing and sanitization of large volumes of data. Dataflow offers significantly better performance and scalability for this type of task.

In summary, the combination of Pub/Sub, Dataflow, and BigQuery in option **D** provides a scalable, fault-tolerant, and comprehensive solution for ingesting, cleaning, and storing streaming data, making it ideal for creating a machine learning model with BigQuery ML and deploying it through Vertex AI.

Question: 205

CertyIQ

You have a data processing application that runs on Google Kubernetes Engine (GKE). Containers need to be launched with their latest available configurations from a container registry. Your GKE nodes need to have GPUs, local SSDs, and 8 Gbps bandwidth. You want to efficiently provision the data processing infrastructure and manage the deployment process. What should you do?

- A. Use Compute Engine startup scripts to pull container images, and use gcloud commands to provision the infrastructure.
- B. Use Cloud Build to schedule a job using Terraform build to provision the infrastructure and launch with the most current container images.
- C. Use GKE to autoscale containers, and use gcloud commands to provision the infrastructure.
- D. Use Dataflow to provision the data pipeline, and use Cloud Scheduler to run the job.

Answer: B

Explanation:

Okay, let's break down why option B is the correct approach and why the others are less suitable for this scenario.

Justification for Option B (Cloud Build + Terraform):

Option B leverages the power of infrastructure-as-code (IaC) through Terraform, which is ideal for repeatable and consistent provisioning of complex infrastructure like GKE clusters. Terraform can define the GKE cluster with specific node configurations, including GPUs, local SSDs, and desired network bandwidth (8 Gbps), ensuring that the infrastructure meets the application's requirements. Cloud Build automates the entire process by scheduling a job that first utilizes Terraform to provision the GKE cluster with the required specifications. Following the provisioning, Cloud Build then automates the process of deploying the application container with its latest image. This addresses the requirements for using the most recent container configurations. This combination provides an end-to-end automated and reliable deployment pipeline. Using Cloud Build and Terraform ensures that infrastructure changes are version-controlled, reproducible, and auditable. This aligns with best practices for managing cloud resources.

Why Other Options Are Less Suitable:

Option A (Compute Engine Startup Scripts + gcloud): While startup scripts can pull container images, this method isn't as robust or manageable as using Terraform for infrastructure provisioning. Gcloud commands alone lack the versioning and lifecycle management capabilities offered by Terraform. Managing infrastructure via scripts becomes cumbersome and error-prone at scale. It doesn't guarantee the use of the latest container images, and it doesn't effectively automate the deployment of the application to GKE.

Option C (GKE Autoscaling + gcloud): GKE autoscaling addresses dynamic resource allocation, but not the initial provisioning of the required hardware (GPUs, local SSDs, bandwidth). Also, relying solely on gcloud for provisioning lacks infrastructure-as-code benefits. This method also does not enforce the launch of the latest container configurations. Autoscaling would kick in after the infrastructure was already created.

Option D (Dataflow + Cloud Scheduler): Dataflow is for data processing pipelines, not for general-purpose containerized applications. Cloud Scheduler is useful for triggering jobs but doesn't help with infrastructure creation or container deployment. This choice focuses on the wrong type of workload. It also doesn't directly address the specific needs of managing container images on a GKE cluster.

Authoritative Links:

Terraform: <https://www.terraform.io/> (For infrastructure as code)

Cloud Build: <https://cloud.google.com/build> (For automated CI/CD)

Google Kubernetes Engine (GKE): <https://cloud.google.com/kubernetes-engine> (For container orchestration)

In summary, option B offers the best solution because it combines infrastructure provisioning with the right tools (Terraform), container deployment, and versioning within a fully automated CI/CD pipeline (Cloud Build). This enables efficient, repeatable, and reliable deployment, fulfilling all requirements detailed in the question.

Question: 206

CertyIQ

You need ads data to serve AI models and historical data for analytics. Longtail and outlier data points need to be identified. You want to cleanse the data in near-real time before running it through AI models. What should you do?

- A. Use Cloud Storage as a data warehouse, shell scripts for processing, and BigQuery to create views for desired datasets.
- B. Use Dataflow to identify longtail and outlier data points programmatically, with BigQuery as a sink.
- C. Use BigQuery to ingest, prepare, and then analyze the data, and then run queries to create views.
- D. Use Cloud Composer to identify longtail and outlier data points, and then output a usable dataset to BigQuery.

Answer: B

Explanation:

The correct answer is B, using Dataflow to identify longtail and outlier data points with BigQuery as a sink. Here's why:

Near Real-time Processing: Dataflow is a fully managed, serverless stream and batch processing service. It's ideal for near real-time data cleansing because it can process data as it arrives, unlike batch-oriented approaches. This aligns with the requirement for pre-processing before AI models and avoids batch latency. (Ref: <https://cloud.google.com/dataflow/docs/concepts/dataflow-model>)

Programmatic Identification: Identifying longtail and outlier data requires custom logic. Dataflow allows developers to write code (in Python, Java, etc.) to implement these identification algorithms, providing the flexibility needed for complex data analysis. (Ref: <https://cloud.google.com/dataflow/docs/programming-model>)

BigQuery as a Sink: BigQuery is a scalable data warehouse designed for analytical workloads. It's perfect for storing the cleaned data and historical data after being processed by Dataflow. Storing the clean data in BigQuery enables both analytics and serving of AI models. (Ref: <https://cloud.google.com/bigquery/docs/introduction>)

Let's examine why the other options are less suitable:

Option A (Cloud Storage, Shell Scripts, BigQuery): Using Cloud Storage as a data warehouse and shell scripts for processing is not scalable or easily manageable, especially for real-time or near real-time requirements. Shell scripts are brittle and lack the fault tolerance and scalability of Dataflow.

Option C (BigQuery for all): While BigQuery can perform data transformations, it's primarily designed for analysis after data is ingested. It's less efficient for complex, real-time transformations such as outlier detection, and isn't its primary use case.

Option D (Cloud Composer, BigQuery): Cloud Composer is an orchestration service for workflows, not the best tool for data transformation. While it can schedule tasks, it doesn't handle the real-time, streaming processing required for cleansing data for AI models. Dataflow would be the processing engine for these tasks in a Composer workflow.

In conclusion, Dataflow offers the necessary capabilities for real-time, programmable data cleansing, making it the best option for identifying longtail and outlier data before loading it into BigQuery for AI model serving and historical analysis.

Question: 207

CertyIQ

You are collecting IoT sensor data from millions of devices across the world and storing the data in BigQuery. Your access pattern is based on recent data, filtered by location_id and device_version with the following query:

```
SELECT
    MAX(temperature)
FROM
    acme_iot_data.sensors
WHERE
    create_date > DATE_SUB(CURRENT_DATE(), INTERVAL 7 day)
    AND location_id = "SW1W9TQ"
    AND device_version = "202007r3"
```

You want to optimize your queries for cost and performance. How should you structure your data?

- A.Partition table data by create_date, location_id, and device_version.
- B.Partition table data by create_date, cluster table data by location_id, and device_version.
- C.Cluster table data by create_date, location_id, and device_version.
- D.Cluster table data by create_date, partition by location_id, and device_version.

Answer: B

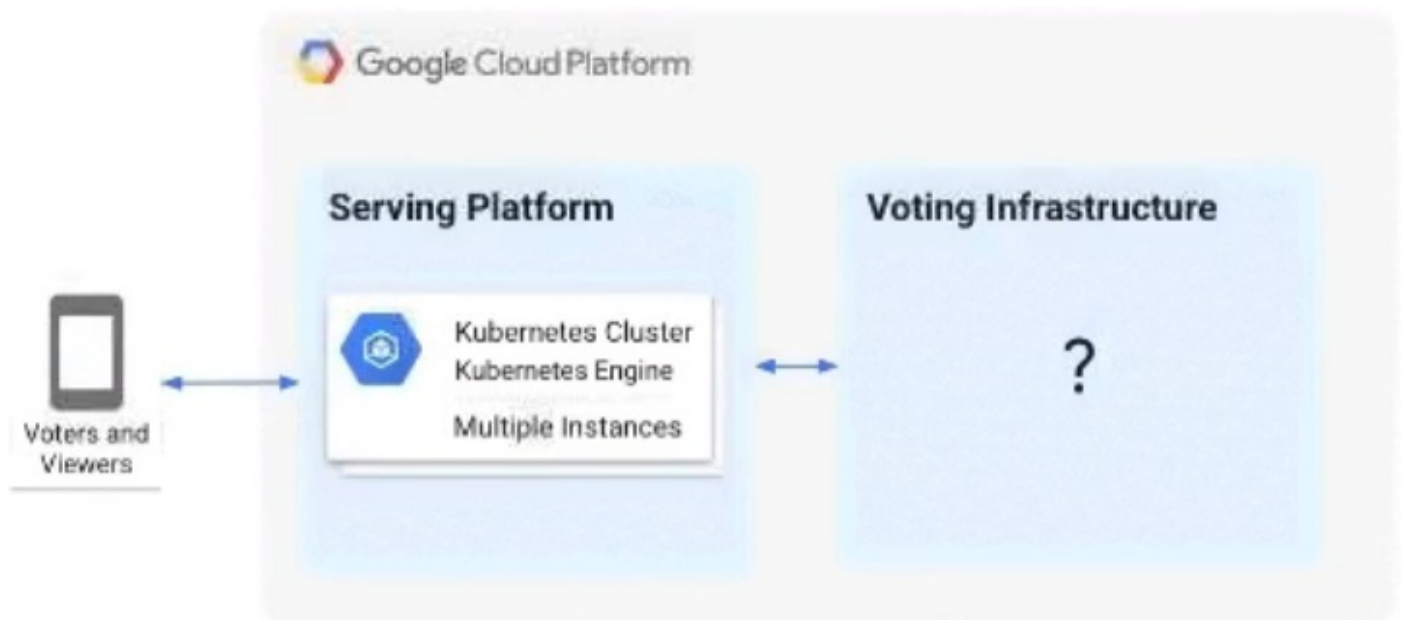
Explanation:

1. Answer is B:- Partitioning the table by create_date allows us to efficiently query data based on time, which is common in access patterns that prioritize recent data. - Clustering the table by location_id and device_version further organizes the data within each partition, making queries filtered by these columns more efficient and cost-effective.
2. The best answer is B. Partition table data by create_date, cluster table data by location_id, and device_version.Here's a breakdown of why this structure is optimal:Partitioning by create_date:Aligns with query pattern: Filters for recent data based on create_date, so partitioning by this column allows BigQuery to quickly narrow down the data to scan, reducing query costs and improving performance.Manages data growth: Partitioning effectively segments data by date, making it easier to manage large datasets and optimize storage costs.Clustering by location_id and device_version:Enhances filtering: Frequently filtering by location_id and device_version, clustering physically co-locates related data within partitions, further reducing scan time and improving performance.

Question: 208

CertyIQ

A live TV show asks viewers to cast votes using their mobile phones. The event generates a large volume of data during a 3-minute period. You are in charge of the "Voting infrastructure" and must ensure that the platform can handle the load and that all votes are processed. You must display partial results while voting is open. After voting closes, you need to count the votes exactly once while optimizing cost. What should you do?



- A.Create a Memystore instance with a high availability (HA) configuration.
- B.Create a Cloud SQL for PostgreSQL database with high availability (HA) configuration and multiple read replicas.
- C.Write votes to a Pub/Sub topic and have Cloud Functions subscribe to it and write votes to BigQuery.
- D.Write votes to a Pub/Sub topic and load into both Bigtable and BigQuery via a Dataflow pipeline. Query Bigtable for real-time results and BigQuery for later analysis. Shut down the Bigtable instance when voting concludes.

Answer: D

Explanation:

Answer is D:- Google Cloud Pub/Sub can manage the high-volume data ingestion. - Google Cloud Dataflow can efficiently process and route data to both Bigtable and BigQuery.- Bigtable is excellent for handling high-throughput writes and reads, making it suitable for real-time vote tallying. - BigQuery is ideal for exact vote counting and deeper analysis once voting concludes.

Question: 209

CertyIQ

A shipping company has live package-tracking data that is sent to an Apache Kafka stream in real time. This is then loaded into BigQuery. Analysts in your company want to query the tracking data in BigQuery to analyze geospatial trends in the lifecycle of a package. The table was originally created with ingest-date partitioning. Over time, the query processing time has increased. You need to copy all the data to a new clustered table. What should you do?

- A.Re-create the table using data partitioning on the package delivery date.
- B.Implement clustering in BigQuery on the package-tracking ID column.
- C.Implement clustering in BigQuery on the ingest date column.
- D.Tier older data onto Cloud Storage files and create a BigQuery table using Cloud Storage as an external data source.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct answer, and why other options are not ideal:

Option B: Implement clustering in BigQuery on the package-tracking ID column.

Why it's correct: Clustering in BigQuery improves query performance by co-locating data that is frequently accessed together. In this scenario, the analysts are interested in the lifecycle of a package, which implies they will be frequently filtering or joining on the package-tracking ID. Clustering by this column will physically organize the data so that all records related to a specific package are stored in the same block. This dramatically reduces the amount of data BigQuery needs to scan during a query, leading to significantly faster query execution times. This is more effective than relying on date partitioning alone for analyzing package lifecycles.

Relevance: The question highlights the need for better query performance for geospatial trends in the lifecycle of a package, directly pointing toward the package ID as a key field for optimization.

How it addresses the problem: It addresses the increasing query processing time by optimizing how data is stored and retrieved.

Why other options are not ideal:

Option A: Re-create the table using data partitioning on the package delivery date. While partitioning by delivery date could be helpful for time-based queries, the main use case is tracking the lifecycle of a package, which would require filtering based on tracking ID, not delivery date. This wouldn't help to optimize queries for package lifecycle analysis, and could even fragment package information across multiple partitions if a package moves over multiple days.

Option C: Implement clustering in BigQuery on the ingest date column. Clustering on the ingest date would not provide any significant benefit since the primary queries are about the package lifecycle. This would be similar to the existing partitioning scheme, and wouldn't optimize for queries filtered by package ID. Ingest date is not relevant to analyze package's lifecycle.

Option D: Tier older data onto Cloud Storage files and create a BigQuery table using Cloud Storage as an external data source. While cost-effective for archiving, this approach is not optimal for performance.

Querying external data sources in BigQuery is usually slower compared to querying native BigQuery tables as it involves additional overhead of reading data from a different storage system. This would degrade rather than improve query performance.

Authoritative Links:

BigQuery Clustering: <https://cloud.google.com/bigquery/docs/clustered-tables>

BigQuery Partitioning: <https://cloud.google.com/bigquery/docs/partitioned-tables>

BigQuery External Tables: <https://cloud.google.com/bigquery/docs/external-data-sources>

In conclusion, clustering on package-tracking ID (Option B) is the most effective solution to improve query performance for analysts querying package lifecycle data in BigQuery. It aligns directly with the analysis requirements and will reduce the amount of data needed to be scanned per query, which is far better than the other options provided.

Question: 210

CertyIQ

You are designing a data mesh on Google Cloud with multiple distinct data engineering teams building data products. The typical data curation design pattern consists of landing files in Cloud Storage, transforming raw data in Cloud Storage and BigQuery datasets, and storing the final curated data product in BigQuery datasets. You need to configure Dataplex to ensure that each team can access only the assets needed to build their data products. You also need to ensure that teams can easily share the curated data product. What should you do?

- A.1. Create a single Dataplex virtual lake and create a single zone to contain landing, raw, and curated data.
2. Provide each data engineering team access to the virtual lake.
- B.1. Create a single Dataplex virtual lake and create a single zone to contain landing, raw, and curated data.
2. Build separate assets for each data product within the zone.
3. Assign permissions to the data engineering teams at the zone level.
- C.1. Create a Dataplex virtual lake for each data product, and create a single zone to contain landing, raw, and curated data.
2. Provide the data engineering teams with full access to the virtual lake assigned to their data product.
- D.1. Create a Dataplex virtual lake for each data product, and create multiple zones for landing, raw, and curated data.
2. Provide the data engineering teams with full access to the virtual lake assigned to their data product.

Answer: D

Explanation:

The correct answer is **D**. Here's a detailed justification:

Option D promotes a data mesh architecture by creating separate Dataplex virtual lakes for each data product. This aligns with the principle of domain ownership within a data mesh, where each team manages its own data. Furthermore, within each lake, defining separate zones for landing, raw, and curated data enforces a clear data lifecycle management and separation of concerns. Giving teams full access to their assigned virtual lake empowers them with autonomy over their data assets. This isolation is crucial for enforcing access control and preventing accidental data manipulation by other teams. The separation of zones ensures that data is not accidentally modified outside of the intended transformation pipelines.

Options A, B, and C all violate the data mesh principles. Option A fails to provide access isolation and domain ownership by placing all data into a single lake. Option B also uses a single lake and assigns team permissions at the zone level. Though this isolates by data product, it creates permission challenges and is not as aligned to the domain ownership principle of a data mesh as D. Option C is better as it separates data by a virtual lake

but using a single zone for all data, it does not separate the various states in the data life cycle and adds additional complexity in managing access to data at different stages.

In summary, option D enables data domain ownership and separation within a Dataplex data mesh environment. This ensures clear separation of data products, strong access control, and facilitates easy sharing of curated data while aligning to data mesh principles.

Authoritative Links:

Google Cloud Dataplex Documentation: <https://cloud.google.com/dataplex/docs>

Data Mesh Principles: <https://martinfowler.com/articles/data-mesh-principles.html>

Google Cloud Architecture Center on Data Mesh: <https://cloud.google.com/architecture/data-mesh>

Question: 211

CertyIQ

You are using BigQuery with a multi-region dataset that includes a table with the daily sales volumes. This table is updated multiple times per day. You need to protect your sales table in case of regional failures with a recovery point objective (RPO) of less than 24 hours, while keeping costs to a minimum. What should you do?

- A. Schedule a daily export of the table to a Cloud Storage dual or multi-region bucket.
- B. Schedule a daily copy of the dataset to a backup region.
- C. Schedule a daily BigQuery snapshot of the table.
- D. Modify ETL job to load the data into both the current and another backup region.

Answer: A

Explanation:

The correct answer is **A. Schedule a daily export of the table to a Cloud Storage dual or multi-region bucket.**

Here's why:

Option A provides a cost-effective and straightforward way to achieve the desired RPO. By exporting the daily sales table to a Cloud Storage bucket that's either dual-region or multi-region, you are creating a redundant backup in a separate location. This mitigates the risk of regional failures affecting your data. Should the primary BigQuery dataset become unavailable, you can easily restore the table from the exported data in Cloud Storage. A daily export is sufficient for an RPO of less than 24 hours, as the latest version of the sales data would be available. Cloud Storage's durability ensures that your exported data is very reliably stored, ready for recovery purposes.

Let's examine why the other options are less suitable:

B. Schedule a daily copy of the dataset to a backup region: While this achieves disaster recovery, copying entire datasets is a more expensive and resource-intensive approach, particularly if only a single table needs protection. Also, dataset copies are usually best for large-scale disaster recovery, not for daily RPOs.

C. Schedule a daily BigQuery snapshot of the table: BigQuery snapshots are a great way to keep track of historical data and can allow point-in-time recovery within BigQuery, but they reside in the same region as the original table. Therefore, it won't offer protection from regional failures. They are best for accidental deletion.

D. Modify ETL job to load the data into both the current and another backup region: This option also achieves redundancy, but the complexity and overhead of modifying the ETL job and loading data into two regions every time is greater than exporting to Cloud Storage. Furthermore, this approach results in multiple BigQuery data storage fees instead of storing backups in the (typically) lower cost Cloud Storage option.

Key Concepts:

Recovery Point Objective (RPO): The maximum acceptable data loss in the event of an incident.

Dual/Multi-Region Cloud Storage: These buckets provide redundancy and availability across different geographical locations, thus protecting against regional failures.

BigQuery Exports: A cost-effective mechanism for moving data out of BigQuery for backup or archival purposes.

BigQuery Snapshots: A point-in-time copy of a table within BigQuery, suitable for restoring historical data within the same region.

Cost Optimization: Choosing solutions that minimize expense without sacrificing requirements.

Authoritative Links:

BigQuery Exports: <https://cloud.google.com/bigquery/docs/exporting-data>

Cloud Storage Locations: <https://cloud.google.com/storage/docs/locations>

Disaster Recovery Planning: <https://cloud.google.com/solutions/disaster-recovery>

Question: 212

CertyIQ

You are troubleshooting your Dataflow pipeline that processes data from Cloud Storage to BigQuery. You have discovered that the Dataflow worker nodes cannot communicate with one another. Your networking team relies on Google Cloud network tags to define firewall rules. You need to identify the issue while following Google-recommended networking security practices. What should you do?

- A. Determine whether your Dataflow pipeline has a custom network tag set.
- B. Determine whether there is a firewall rule set to allow traffic on TCP ports 12345 and 12346 for the Dataflow network tag.
- C. Determine whether there is a firewall rule set to allow traffic on TCP ports 12345 and 12346 on the subnet used by Dataflow workers.
- D. Determine whether your Dataflow pipeline is deployed with the external IP address option enabled.

Answer: B

Explanation:

The correct answer is **B. Determine whether there is a firewall rule set to allow traffic on TCP ports 12345 and 12346 for the Dataflow network tag.**

Here's the justification:

Dataflow worker nodes require communication amongst themselves to shuffle data during processing. This communication happens over specific TCP ports, primarily 12345 and 12346. Google recommends using network tags for firewall rule management, as it allows for granular control and avoids managing firewall rules based on IP ranges, which can be less flexible and prone to errors. If the firewall rules aren't configured correctly for Dataflow's network tag, the workers will be unable to communicate, leading to pipeline failure.

Option A is not the core issue. While knowing if a custom tag is being used is good information, it doesn't directly explain the communication failure. The problem is whether the required communication is allowed, not whether a custom tag exists. Option C, while related to networking, is less precise than using tags. Dataflow worker nodes may use specific subnetworks, but relying on subnet-based rules is less granular than tags, contradicting the best practice of using network tags. Option D is incorrect; external IP addresses are not required for worker-to-worker communication within the same VPC network; relying on internal IPs is more secure and cost-effective.

Therefore, the primary issue revolves around the firewall rules configured for the Dataflow network tag and whether they are permitting traffic over the required ports. The most direct and efficient approach is to verify the existence and correctness of firewall rules associated with the relevant network tag and ensure these

rules allow TCP traffic on ports 12345 and 12346.

Authoritative Links for Further Research:

Dataflow Networking: <https://cloud.google.com/dataflow/docs/concepts/security-and-networking>

Dataflow Firewall Rules: <https://cloud.google.com/dataflow/docs/guides/firewall-rules>

Google Cloud Network Tags: https://cloud.google.com/vpc/docs/firewalls#network_tags

Google Dataflow Known Issues: <https://cloud.google.com/dataflow/docs/guides/troubleshooting-your-pipeline#worker-communication>

Question: 213

CertyIQ

Your company's customer_order table in BigQuery stores the order history for 10 million customers, with a table size of 10 PB. You need to create a dashboard for the support team to view the order history. The dashboard has two filters, country_name and username. Both are string data types in the BigQuery table. When a filter is applied, the dashboard fetches the order history from the table and displays the query results. However, the dashboard is slow to show the results when applying the filters to the following query:

```
SELECT date, order, status FROM customer_order
WHERE country = '<country_name>' AND username = '<username>'
```

How should you redesign the BigQuery table to support faster access?

- A. Cluster the table by country and username fields.
- B. Cluster the table by country field, and partition by username field.
- C. Partition the table by country and username fields.
- D. Partition the table by _PARTITIONTIME.

Answer: A

Explanation:

Clustering organizes the data based on the specified columns (in this case, country_name and username). - When a query filters on these columns, BigQuery can efficiently scan only the relevant parts of the table

Question: 214

CertyIQ

You have a Standard Tier Memorystore for Redis instance deployed in a production environment. You need to simulate a Redis instance failover in the most accurate disaster recovery situation, and ensure that the failover has no impact on production data. What should you do?

- A. Create a Standard Tier Memorystore for Redis instance in the development environment. Initiate a manual failover by using the limited-data-loss data protection mode.
- B. Create a Standard Tier Memorystore for Redis instance in a development environment. Initiate a manual failover by using the force-data-loss data protection mode.
- C. Increase one replica to Redis instance in production environment. Initiate a manual failover by using the force-data-loss data protection mode.
- D. Initiate a manual failover by using the limited-data-loss data protection mode to the Memorystore for Redis instance in the production environment.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct answer:

The primary goal is to simulate a Redis failover in a production-like environment without impacting actual production data. Option B achieves this by creating a separate, identical Standard Tier Memorystore for Redis instance in a development environment. This isolation is crucial to avoid any accidental data loss or service disruption in production.

Using the "force-data-loss" mode for the manual failover on the development instance is appropriate in this simulation. While "limited-data-loss" is usually preferred for a real failover, here the focus is on observing failover behavior. The "force-data-loss" mode forces a more immediate failover, which is beneficial in testing scenarios where the duration of the failover process and the behavior of the application during a forced failover, is more important to observe than data loss prevention. It helps to ensure that failover mechanism is functioning as expected.

Option A is incorrect because initiating a manual failover using limited-data-loss mode on a development instance will not give you the most accurate simulation. Data loss prevention is not a concern in this simulation.

Option C is incorrect because increasing a replica on the production instance and then forcing a failover will still affect the production instance itself and could cause data loss or service disruption.

Option D is incorrect because initiating a failover on the production instance should be avoided in a simulation. This action may cause service interruption to the production environment and data loss.

Therefore, option B provides a safe and realistic environment to test the failover procedures without impacting production.

Key Concepts:

Disaster Recovery: A strategy to recover from a failure.

Failover: The process of switching to a backup system in the event of a primary system failure.

Memorystore for Redis: Google Cloud's fully managed in-memory data store service, compatible with Redis.

Standard Tier: Provides high availability through replication.

Data Protection Modes:

Limited-data-loss: Aims to minimize data loss during a failover.

Force-data-loss: Prioritizes a quick failover, potentially losing data in the process.

Development Environment: An isolated environment for testing and development.

Authoritative Links:

[Memorystore for Redis documentation](#): Explore detailed information about the service.

[Managing Failovers in Memorystore](#): In-depth documentation on initiating and understanding failovers

[Google Cloud Best Practices for Disaster Recovery](#): Learn more about disaster recovery best practices

Question: 215

CertyIQ

You are administering a BigQuery dataset that uses a customer-managed encryption key (CMEK). You need to share the dataset with a partner organization that does not have access to your CMEK. What should you do?

A. Provide the partner organization a copy of your CMEKs to decrypt the data.

B. Export the tables to parquet files to a Cloud Storage bucket and grant the storageinsights.viewer role on the bucket to the partner organization.

C. Copy the tables you need to share to a dataset without CMEKs. Create an Analytics Hub listing for this dataset.

D. Create an authorized view that contains the CMEK to decrypt the data when accessed.

Answer: C

Explanation:

The correct answer is **C. Copy the tables you need to share to a dataset without CMEKs. Create an Analytics Hub listing for this dataset.** Here's why:

Customer-Managed Encryption Keys (CMEKs) provide granular control over data encryption but also create dependencies. Sharing data encrypted with a CMEK requires the recipient to have access to the key. Option A, sharing your CMEK, is a severe security risk and violates the principle of least privilege. Option D, using an authorized view with the CMEK, would still require the partner to have some form of key access, defeating the purpose.

Option B, exporting to parquet files in Cloud Storage, would work in terms of sharing the data but does not provide the same level of data management and discovery offered by BigQuery and also implies you are making a copy of the data which is not ideal. Analytics Hub, as in option C, is designed for secure and governed data sharing.

Copying the specific tables to a new dataset without CMEKs breaks the direct dependency on your sensitive key. You retain control over the CMEK-protected data and its encryption. The new dataset can then be shared via Analytics Hub, which enables a catalogued and controlled discovery process for the partner. The partner does not need access to the CMEK, and they can only access the shared copy of your data.

This approach separates the concerns of secure encryption and controlled data sharing, maintaining security while enabling collaboration. It aligns with best practices for data governance and security within a cloud environment. Using Analytics Hub further offers features like data lineage and access control.

For further research:

Google Cloud Documentation on CMEK: <https://cloud.google.com/kms/docs/cmek>

Google Cloud Documentation on Analytics Hub: <https://cloud.google.com/analytics-hub/docs>

Google Cloud Documentation on Authorized Views: <https://cloud.google.com/bigquery/docs/share-access-views>

Google Cloud Documentation on Best practices for data sharing:
<https://cloud.google.com/architecture/data-sharing-best-practices>

Question: 216

CertyIQ

You are developing an Apache Beam pipeline to extract data from a Cloud SQL instance by using JdbcIO. You have two projects running in Google Cloud. The pipeline will be deployed and executed on Dataflow in Project A. The Cloud SQL instance is running in Project B and does not have a public IP address. After deploying the pipeline, you noticed that the pipeline failed to extract data from the Cloud SQL instance due to connection failure. You verified that VPC Service Controls and shared VPC are not in use in these projects. You want to resolve this error while ensuring that the data does not go through the public internet. What should you do?

- A. Set up VPC Network Peering between Project A and Project B. Add a firewall rule to allow the peered subnet range to access all instances on the network.
- B. Turn off the external IP addresses on the Dataflow worker. Enable Cloud NAT in Project A.
- C. Add the external IP addresses of the Dataflow worker as authorized networks in the Cloud SQL instance.
- D. Set up VPC Network Peering between Project A and Project B. Create a Compute Engine instance without external IP address in Project B on the peered subnet to serve as a proxy server to the Cloud SQL database.

Answer: D

Explanation:

The correct answer is **D**. Let's break down why:

Option D proposes using VPC Network Peering and a proxy server. VPC Network Peering allows private IP address communication between VPC networks in different projects. This eliminates the need for data to traverse the public internet. A Compute Engine instance, acting as a proxy, is placed within Project B on the peered subnet. The Dataflow workers in Project A, connected through the peer, send SQL connection requests to the proxy server. This proxy then relays the traffic to the Cloud SQL instance within Project B. This approach ensures secure, private communication.

Option A, while using VPC Peering, lacks the critical intermediary server. Firewalls alone wouldn't resolve the underlying issue of how the Dataflow workers in Project A connect to the private IP of Cloud SQL in Project B. Cloud SQL connections require an accessible IP address or proxy.

Option B suggests disabling external IPs and using Cloud NAT, which is not suitable here. Cloud NAT allows VMs without external IPs to access the internet, but it doesn't address connecting to resources in another project. Cloud NAT's outbound internet access is not applicable to resolving the lack of communication between two GCP private networks.

Option C is incorrect because Dataflow workers' external IPs are dynamic and ephemeral, making whitelisting them in Cloud SQL impractical and insecure. Moreover, exposing Cloud SQL to external IPs is a security risk that should be avoided, and the question specifically states the desire to avoid the public internet.

Therefore, option D provides the most appropriate solution by establishing private network connectivity via VPC peering and using a proxy server to manage the connection between Dataflow and the private Cloud SQL instance, while ensuring data remains within the private network and does not traverse the public internet.

Authoritative Links:

VPC Network Peering: <https://cloud.google.com/vpc/docs/vpc-peering>

Cloud SQL Private IP: <https://cloud.google.com/sql/docs/mysql/configure-private-ip>

Compute Engine Proxy: <https://cloud.google.com/compute/docs/tutorials/proxy>

Question: 217

CertyIQ

You have a BigQuery table that contains customer data, including sensitive information such as names and addresses. You need to share the customer data with your data analytics and consumer support teams securely. The data analytics team needs to access the data of all the customers, but must not be able to access the sensitive data. The consumer support team needs access to all data columns, but must not be able to access customers that no longer have active contracts. You enforced these requirements by using an authorized dataset and policy tags. After implementing these steps, the data analytics team reports that they still have access to the sensitive columns. You need to ensure that the data analytics team does not have access to restricted data. What should you do? (Choose two.)

- A. Create two separate authorized datasets; one for the data analytics team and another for the consumer support team.
- B. Ensure that the data analytics team members do not have the Data Catalog Fine-Grained Reader role for the policy tags.
- C. Replace the authorized dataset with an authorized view. Use row-level security and apply filter_expression to limit data access.
- D. Remove the bigquery.dataViewer role from the data analytics team on the authorized datasets.
- E. Enforce access control in the policy tag taxonomy.

Answer: BE

Explanation:

Okay, let's break down why options B and E are the correct choices for ensuring the data analytics team doesn't access sensitive data in this scenario.

The core problem is that policy tags, used for column-level access control, are not sufficient on their own. While they hide data from queries when the user lacks permissions on the tag, they don't prevent the user from seeing the tag or inferring the sensitive data by other means. The Data Catalog Fine-Grained Reader role allows a user to view the policy tags applied to the data. If they know the sensitive data is tagged, they can potentially infer the data's content even if they can't query it directly via BigQuery.

Why B is Correct: Removing the Data Catalog Fine-Grained Reader role for the analytics team prevents them from seeing which columns are protected by policy tags. This eliminates the ability to understand the sensitive data.

Why E is Correct: Enforcing access control in the policy tag taxonomy is crucial. This means ensuring that the data analytics team doesn't have permissions on the policy tags applied to the sensitive data. If the team only has access to the dataset and tables but not the policy tags, then column access is effectively blocked. Policy tags work by enforcing who has permission to read data with that tag – not just who has dataset permissions.

Why other options are incorrect:

A: Creating separate authorized datasets is unnecessary and inefficient since access control is already managed with policy tags. Datasets themselves don't inherently offer the type of fine-grained, column-level access control needed here.

C: Authorized views and row-level security would be relevant if the requirement was to filter rows for the consumer support team, not the data analytics team.

D: Removing the `bigquery.dataViewer` role would block access to the data, not just the sensitive parts. We want to allow access to some data for the data analytics team.

In summary: Option B prevents metadata knowledge of sensitive columns, and option E enforces access control to those sensitive column tags, effectively restricting the analytics team from seeing the sensitive data.

Authoritative Links:

Policy tags overview: <https://cloud.google.com/data-catalog/docs/concepts/policy-tags>

Fine-grained access control with policy tags: <https://cloud.google.com/data-catalog/docs/how-to/fine-grained-access-control>

Data Catalog roles: <https://cloud.google.com/data-catalog/docs/concepts/iam#roles>

Question: 218

CertyIQ

You have a Cloud SQL for PostgreSQL instance in Region1 with one read replica in Region2 and another read replica in Region3. An unexpected event in Region1 requires that you perform disaster recovery by promoting a read replica in Region2. You need to ensure that your application has the same database capacity available before you switch over the connections. What should you do?

- A. Enable zonal high availability on the primary instance. Create a new read replica in a new region.
- B. Create a cascading read replica from the existing read replica in Region3.
- C. Create two new read replicas from the new primary instance, one in Region3 and one in a new region.
- D. Create a new read replica in Region1, promote the new read replica to be the primary instance, and enable zonal high availability.

Answer: C

Explanation:

Okay, let's break down why option C is the correct approach for this disaster recovery scenario.

The core issue is that promoting the read replica in Region2 to primary status effectively changes the topology. Before, you had a primary in Region1 and replicas in Region2 and Region3. After the promotion, you have a new primary in Region2 and only one existing replica in Region3. This leaves you short of database capacity. Since the question specifies that the same database capacity should be available before switching the connections, we need to compensate for the lost replica.

Option A is incorrect as it focuses on high availability for the primary instance but doesn't address the need for replica count and is also not required prior to the failover. Option B is problematic because cascading read replicas can introduce latency and potential replication bottlenecks, and a cascade in this scenario doesn't solve the capacity requirement problem. Option D involves attempting to re-establish the original setup in the unavailable region, a problematic undertaking during a disaster recovery operation, and additionally includes enabling HA post promoting the replica. This leaves Option C: creating two new read replicas.

Option C directly addresses the capacity requirement. By creating one replica in Region3 (replacing the previous replica) and another in a new region, you immediately match the original database capacity before switching over the application. This is crucial for maintaining application performance during and immediately after the failover. This method ensures resilience and maintains the desired number of replicas without introducing complexity or delays.

Relevant concepts: disaster recovery, read replicas, database capacity, failover, Cloud SQL for PostgreSQL.

Authoritative Links:

Google Cloud SQL Read Replicas: <https://cloud.google.com/sql/docs/postgres/read-replicas>

Google Cloud SQL High Availability and Disaster Recovery:

<https://cloud.google.com/sql/docs/postgres/high-availability>

Google Cloud SQL Promotion of a Replica: <https://cloud.google.com/sql/docs/postgres/configure-ha#promote-failover>

Question: 219

CertyIQ

You orchestrate ETL pipelines by using Cloud Composer. One of the tasks in the Apache Airflow directed acyclic graph (DAG) relies on a third-party service. You want to be notified when the task does not succeed. What should you do?

- A. Assign a function with notification logic to the `on_retry_callback` parameter for the operator responsible for the task at risk.
- B. Configure a Cloud Monitoring alert on the `sla_missed` metric associated with the task at risk to trigger a notification.
- C. Assign a function with notification logic to the `on_failure_callback` parameter for the operator responsible for the task at risk.
- D. Assign a function with notification logic to the `sla_miss_callback` parameter for the operator responsible for the task at risk.

Answer: C

Explanation:

The correct answer is **C. Assign a function with notification logic to the `on_failure_callback` parameter for the operator responsible for the task at risk.**

Here's a detailed justification:

Apache Airflow provides several callback functions within operators to handle different task states. When a

task fails, the `on_failure_callback` is triggered. This parameter allows you to define a function that will be executed specifically upon a task's failure. Within this callback function, you can implement logic to send notifications via email, Slack, or other alerting mechanisms, ensuring you're immediately informed when a task using the third-party service encounters an issue.

Option A, `on_retry_callback`, is not suitable because it's invoked when a task retries, not when it ultimately fails after all retries are exhausted. Option B, using Cloud Monitoring on the `sla_missed` metric, focuses on Service Level Agreements (SLAs), which are different from immediate task failure detection. SLAs measure time constraints on tasks; a task might fail and still not miss an SLA. Option D, `sla_miss_callback`, is also related to SLA breaches, not general task failures.

Therefore, `on_failure_callback` provides the precise functionality needed to handle the scenario of wanting to be notified when a specific task in your Airflow DAG, particularly one interacting with a third-party service, does not succeed. It provides immediate feedback for actionable intervention.

Key Concepts:

Apache Airflow Operators: Units of work in an Airflow DAG, each having parameters like callback functions.

Callbacks: Functions triggered by specific events in an Airflow operator's lifecycle.

Task Failure: An execution result of an operator that is unsuccessful.

Authoritative Links:

Apache Airflow Documentation on Operators: <https://airflow.apache.org/docs/apache-airflow/stable/concepts/operators.html> (Specifically look for operator parameter descriptions)

Apache Airflow Documentation on Callbacks: <https://airflow.apache.org/docs/apache-airflow/stable/concepts/dags.html#callbacks>

Cloud Composer Documentation (Google Cloud): <https://cloud.google.com/composer/docs> (Search for related Airflow Concepts)

Question: 220

CertyIQ

You are migrating your on-premises data warehouse to BigQuery. One of the upstream data sources resides on a MySQL database that runs in your on-premises data center with no public IP addresses. You want to ensure that the data ingestion into BigQuery is done securely and does not go through the public internet. What should you do?

- A. Update your existing on-premises ETL tool to write to BigQuery by using the BigQuery Open Database Connectivity (ODBC) driver. Set up the proxy parameter in the `simba.googlebigqueryodbc.ini` file to point to your data center's NAT gateway.
- B. Use Datastream to replicate data from your on-premises MySQL database to BigQuery. Set up Cloud Interconnect between your on-premises data center and Google Cloud. Use Private connectivity as the connectivity method and allocate an IP address range within your VPC network to the Datastream connectivity configuration. Use Server-only as the encryption type when setting up the connection profile in Datastream.
- C. Use Datastream to replicate data from your on-premises MySQL database to BigQuery. Use Forward-SSH tunnel as the connectivity method to establish a secure tunnel between Datastream and your on-premises MySQL database through a tunnel server in your on-premises data center. Use None as the encryption type when setting up the connection profile in Datastream.
- D. Use Datastream to replicate data from your on-premises MySQL database to BigQuery. Gather Datastream public IP addresses of the Google Cloud region that will be used to set up the stream. Add those IP addresses to the firewall allowlist of your on-premises data center. Use IP Allowlisting as the connectivity method and Server-only as the encryption type when setting up the connection profile in Datastream.

Answer: B

Explanation:

The correct answer is B. Here's why:

Explanation:

The goal is to securely migrate data from an on-premises MySQL database to BigQuery without exposing data transfer to the public internet. Option B provides the most secure and reliable method for this scenario. Let's break down why:

1. **Datastream:** Datastream is Google Cloud's change data capture (CDC) service. It's designed to replicate data in real-time or near real-time from databases like MySQL to BigQuery. This ensures continuous data synchronization.
2. **Cloud Interconnect:** This establishes a private network connection between the on-premises data center and Google Cloud. By using Cloud Interconnect, data bypasses the public internet, ensuring data privacy and security. Cloud Interconnect offers more consistent and higher bandwidth than VPN.
3. **Private Connectivity:** Using private connectivity via Cloud Interconnect avoids public IP addresses and relies solely on the private network established between your on-premises environment and Google Cloud. This is the most secure method. You allocate an IP address range within your VPC network for Datastream to use.
4. **Server-only encryption:** Setting up a server-only encryption guarantees the data will be encrypted while at rest on the server, but it's not the only encryption method.

Why other options are incorrect:

Option A: While using the BigQuery ODBC driver might work, relying on a proxy parameter in the configuration file for security is not the most robust approach. Also, using ETL tools involves creating, managing, and maintaining ETL pipelines that is not suitable for a modern data architecture.

Option C: Forward-SSH tunneling can create a secure tunnel, but it is less scalable and potentially introduces performance bottlenecks compared to a direct private connection with Cloud Interconnect. It also mandates a tunnel server. Using "None" for encryption is also unacceptable when the goal is to ensure secure ingestion.

Option D: Using IP allowlisting is generally not recommended as a primary security mechanism. Public IP addresses are still involved, potentially exposing the data transfer to the internet. Furthermore, managing and maintaining a firewall allowlist can be cumbersome and error-prone.

Supporting Concepts:

Change Data Capture (CDC): A technique used to track and replicate changes made to a database in real-time or near real-time. Datastream utilizes CDC for efficient data replication.

Hybrid Cloud: A cloud computing model where an organization uses a mix of on-premises infrastructure and public cloud services.

Private Network Connectivity: Establishing secure and private connections between on-premises environments and cloud providers using services like Cloud Interconnect.

Authoritative Links:

Datastream Documentation: <https://cloud.google.com/datastream/docs/>

Cloud Interconnect Documentation: <https://cloud.google.com/interconnect/docs/>

You also have a variety of object stores across Microsoft Azure and Amazon Web Services (AWS), also in US regions. You want to query all your data in BigQuery daily with as little movement of data as possible. What should you do?

- A. Use BigQuery Data Transfer Service to load files from Azure and AWS into BigQuery.
- B. Create a Dataflow pipeline to ingest files from Azure and AWS to BigQuery.
- C. Load files from AWS and Azure to Cloud Storage with Cloud Shell `gsutil rsync` arguments.
- D. Use the BigQuery Omni functionality and BigLake tables to query files in Azure and AWS.

Answer: D

Explanation:

The optimal solution for querying data residing across Google Cloud's BigQuery, Microsoft Azure, and Amazon Web Services (AWS) with minimal data movement is **D. Use the BigQuery Omni functionality and BigLake tables to query files in Azure and AWS.**

Here's why:

BigQuery Omni is designed for precisely this scenario. It enables querying data stored in other cloud platforms (Azure and AWS) directly, without needing to physically move that data into Google Cloud Storage or BigQuery. This aligns perfectly with the requirement of minimizing data movement. BigLake tables, a feature of BigQuery, are used to manage and access data stored in these external cloud storage systems. They act as metadata layers on top of your external data, allowing you to perform standard BigQuery queries as if the data were directly within BigQuery. This eliminates the need for complex data loading and transformation pipelines.

Options A, B, and C involve moving data, which contradict the "as little movement of data as possible" requirement. Option A uses the Data Transfer Service to move data, Option B suggests building a data pipeline to move data, and Option C advocates copying data to GCS before querying. While those solutions are valid in other contexts, they're not appropriate when minimizing data movement is a priority. BigQuery Omni with BigLake tables provides direct querying capabilities, optimizing performance by avoiding unnecessary data transfers and reducing processing overhead, especially for daily recurring queries.

Authoritative Links:

BigQuery Omni: <https://cloud.google.com/bigquery/docs/omni-introduction>

BigLake Tables: <https://cloud.google.com/bigquery/docs/biglake-intro>

Question: 222

CertyIQ

You have a variety of files in Cloud Storage that your data science team wants to use in their models. Currently, users do not have a method to explore, cleanse, and validate the data in Cloud Storage. You are looking for a low code solution that can be used by your data science team to quickly cleanse and explore data within Cloud Storage. What should you do?

- A. Provide the data science team access to Dataflow to create a pipeline to prepare and validate the raw data and load data into BigQuery for data exploration.
- B. Create an external table in BigQuery and use SQL to transform the data as necessary. Provide the data science team access to the external tables to explore the raw data.
- C. Load the data into BigQuery and use SQL to transform the data as necessary. Provide the data science team access to staging tables to explore the raw data.
- D. Provide the data science team access to Dataprep to prepare, validate, and explore the data within Cloud Storage.

Answer: D

Explanation:

The correct answer is **D: Provide the data science team access to Dataprep to prepare, validate, and explore the data within Cloud Storage**. Here's why:

Dataprep by Trifacta is a serverless data preparation service specifically designed for users, including data scientists, who need a low-code/no-code environment to explore, cleanse, and validate data. It directly integrates with Cloud Storage, allowing users to connect to their files and visually transform data using a point-and-click interface. This aligns perfectly with the requirement for a low-code solution that enables data exploration and cleansing for the data science team.

Options A, B, and C are less suitable due to the following reasons. Option A, utilizing Dataflow, is more appropriate for complex, batch-oriented data transformation pipelines requiring programming expertise, not aligning with a low-code objective. Option B, while utilizing BigQuery external tables, still mandates SQL knowledge for transformation, and is less intuitive for exploration purposes compared to Dataprep. Option C, loading data into BigQuery and using SQL, also relies on SQL proficiency for exploration and transformation, not optimal for data scientists seeking a more visual, low-code experience. Moreover, it introduces an extra step of data loading. Dataprep directly works with data in Cloud Storage without needing such a load step.

Dataprep's visual interface with interactive suggestions and profiling features is ideal for quick data exploration and cleansing tasks. Data scientists can immediately understand data characteristics, identify outliers, and perform required operations, thereby significantly accelerating their model development process. Its low-code/no-code nature empowers data scientists, reducing the need for constant engineering support.

Authoritative Links:

Dataprep Documentation: <https://cloud.google.com/dataprep/docs>

Dataprep Overview: <https://cloud.google.com/dataprep>

Dataflow Documentation: <https://cloud.google.com/dataflow/docs>

BigQuery Documentation: <https://cloud.google.com/bigquery/docs>

Question: 223

CertyIQ

You are building an ELT solution in BigQuery by using Dataform. You need to perform uniqueness and null value checks on your final tables. What should you do to efficiently integrate these checks into your pipeline?

- A. Build BigQuery user-defined functions (UDFs).
- B. Create Dataplex data quality tasks.
- C. Build Dataform assertions into your code.
- D. Write a Spark-based stored procedure.

Answer: C

Explanation:

The most efficient approach to integrate uniqueness and null value checks into a Dataform-based ELT pipeline in BigQuery is to leverage Dataform's built-in assertion capabilities. Dataform assertions allow developers to define tests directly within their SQLX code, enabling these checks to be executed as part of the data transformation pipeline. This means that data quality checks are seamlessly integrated into the workflow, becoming a fundamental part of the build process, rather than a separate, external task. When an assertion fails, Dataform will immediately stop the pipeline and report the specific violations. This provides early detection of data quality issues, allowing for timely correction. Using Dataform assertions requires

writing simple SQL queries that return an empty result if the check passes. This simplicity makes them easy to implement and maintain within the Dataform project. Creating BigQuery UDFs or using Spark-based stored procedures would introduce unnecessary complexity and add overhead as these solutions need to be managed separately. Dataplex data quality tasks, while powerful, are more suitable for broader data quality management and governance across multiple datasets and are not as directly integrated into Dataform pipelines. Dataform assertions offer the best balance of direct integration, ease of use, and targeted data quality checks specifically needed for the transformations happening in Dataform.<https://cloud.google.com/dataform/docs/assertions><https://cloud.google.com/dataform/docs/assertions-examples>

Question: 224

CertyIQ

A web server sends click events to a Pub/Sub topic as messages. The web server includes an eventTimestamp attribute in the messages, which is the time when the click occurred. You have a Dataflow streaming job that reads from this Pub/Sub topic through a subscription, applies some transformations, and writes the result to another Pub/Sub topic for use by the advertising department. The advertising department needs to receive each message within 30 seconds of the corresponding click occurrence, but they report receiving the messages late. Your Dataflow job's system lag is about 5 seconds, and the data freshness is about 40 seconds. Inspecting a few messages show no more than 1 second lag between their eventTimestamp and publishTime. What is the problem and what should you do?

- A. The advertising department is causing delays when consuming the messages. Work with the advertising department to fix this.
- B. Messages in your Dataflow job are taking more than 30 seconds to process. Optimize your job or increase the number of workers to fix this.
- G. Messages in your Dataflow job are processed in less than 30 seconds, but your job cannot keep up with the backlog in the Pub/Sub subscription. Optimize your job or increase the number of workers to fix this.
- D. The web server is not pushing messages fast enough to Pub/Sub. Work with the web server team to fix this.

Answer: G

Explanation:

Here's a detailed justification for why the correct answer is **G**, and why the other options are incorrect:

Understanding the Problem:

The core issue is that messages are arriving at the advertising department significantly later than the 30-second SLA, despite the click event occurring close to the message's publish time. The data freshness of 40 seconds and a system lag of 5 seconds strongly indicate a backlog within the Dataflow job itself rather than at the source or destination. The fact that the messages have a minimal delay between their eventTimestamp and publishTime indicates that the issue is not with the web server producing the messages.

Why Option G is Correct:

Option G states: "Messages in your Dataflow job are processed in less than 30 seconds, but your job cannot keep up with the backlog in the Pub/Sub subscription. Optimize your job or increase the number of workers to fix this." This aligns perfectly with the given information.

Data Freshness: A 40-second data freshness means that on average, the oldest data being processed by the Dataflow pipeline is 40 seconds behind the current time. This implies the pipeline is lagging behind the incoming data stream.

System Lag: The 5-second system lag signifies that, while data is eventually processed, there's an average delay within the Dataflow pipeline's processing engine before data reaches the final output. This value does not account for the full backlog causing the 40-second freshness.

Subscription Backlog: The combination of high data freshness and low system lag suggests that messages are queuing up in the Dataflow job's Pub/Sub subscription faster than they're being processed. This indicates that Dataflow cannot keep pace with the volume or speed of incoming events. The solution is then to optimize the pipeline to process faster or add more workers to increase throughput.

Why Other Options are Incorrect:

Option A: "The advertising department is causing delays when consuming the messages. Work with the advertising department to fix this." This is unlikely. The problem is that the data is not being processed by dataflow within the required time frame. The freshness and system lag metrics do not point to the destination as the cause.

Option B: "Messages in your Dataflow job are taking more than 30 seconds to process. Optimize your job or increase the number of workers to fix this." While it's possible that individual message processing could be slow, the system lag is only 5 seconds, indicating that the job is not spending 30 seconds processing each message. The system is more likely backlogged instead of slowed.

Option D: "The web server is not pushing messages fast enough to Pub/Sub. Work with the web server team to fix this." The problem statement specifies that messages are pushed to the Pub/Sub topic with a very minimal lag of one second between the event time and publish time, so the problem is not with the data source.

Supporting Concepts & Authoritative Links:

Dataflow Streaming: Dataflow is a fully managed stream processing service. It consumes events from a streaming source such as Pub/Sub, performs transformations, and outputs the results. See <https://cloud.google.com/dataflow/docs/concepts/streaming-pipelines> for more information on streaming pipelines in Dataflow.

Pub/Sub Subscriptions: Dataflow reads from Pub/Sub through subscriptions. The subscription acts as a buffer between the published data and the consumption of that data. More information on subscription can be found at: <https://cloud.google.com/pubsub/docs/subscriber>

Data Freshness: The age of the oldest data being processed by a Dataflow pipeline. <https://cloud.google.com/dataflow/docs/guides/monitoring-your-pipeline#data-freshness>

System Lag: The time it takes for a record to be processed within the Dataflow pipeline. <https://cloud.google.com/dataflow/docs/guides/monitoring-your-pipeline#system-lag>

Backlogs: If a stream's data ingestion rate exceeds the processing rate, messages will queue up in a backlog which can be handled by scaling the stream processing job.

Conclusion:

The primary issue is a processing bottleneck within the Dataflow job. While the system isn't individually slow, the job can't process the incoming data stream fast enough, creating a backlog. Option G correctly identifies this and proposes the right actions: optimizing the job or scaling the job by increasing the worker count.

Question: 225

CertyIQ

Your organization stores customer data in an on-premises Apache Hadoop cluster in Apache Parquet format. Data is processed on a daily basis by Apache Spark jobs that run on the cluster. You are migrating the Spark jobs and Parquet data to Google Cloud. BigQuery will be used on future transformation pipelines so you need to ensure that your data is available in BigQuery. You want to use managed services, while minimizing ETL data processing changes and overhead costs. What should you do?

A.Migrate your data to Cloud Storage and migrate the metadata to Dataproc Metastore (DPMS). Refactor Spark pipelines to write and read data on Cloud Storage, and run them on Dataproc Serverless.

B.Migrate your data to Cloud Storage and register the bucket as a Dataplex asset. Refactor Spark pipelines to write and read data on Cloud Storage, and run them on Dataproc Serverless.

C.Migrate your data to BigQuery. Refactor Spark pipelines to write and read data on BigQuery, and run them on Dataproc Serverless.

D.Migrate your data to BigLake. Refactor Spark pipelines to write and read data on Cloud Storage, and run them on Dataproc on Compute Engine.

Answer: C

Explanation:

The correct answer is **C. Migrate your data to BigQuery. Refactor Spark pipelines to write and read data on BigQuery, and run them on Dataproc Serverless.** Here's why:

The core requirement is to make the existing Parquet data available in BigQuery for future transformation pipelines, while minimizing changes and leveraging managed services. Option C directly addresses this by migrating the data to BigQuery. BigQuery is a fully managed, serverless data warehouse ideal for analytical workloads. Migrating the data to BigQuery ensures direct access, eliminating extra steps. Refactoring Spark jobs to read and write from BigQuery is necessary but minimizes overhead. Dataproc Serverless allows running Spark jobs without the overhead of cluster management, adhering to the requirement for managed services.

Option A proposes using Dataproc Metastore, but DPMS is designed for metadata management related to metastores (like Hive), not data migration to BigQuery, which is the primary goal. Option B incorrectly suggests Dataplex for metadata, which while valuable, isn't as direct a path for getting data into BigQuery. Additionally, BigLake (option D) is used for accessing data stored in multiple sources from BigQuery and not a suitable destination. Option D also recommends running Dataproc on Compute Engine, which contradicts the requirement of using managed services by involving infrastructure management. Using Dataproc Serverless aligns well with the need to use managed services. The refactoring to use BigQuery as the data source and sink is the most direct and effective approach in this scenario.

Supporting links:

BigQuery: <https://cloud.google.com/bigquery>

Dataproc Serverless: <https://cloud.google.com/dataproc/docs/concepts/serverless/overview>

Dataproc Metastore: <https://cloud.google.com/dataproc/docs/concepts/metastore/overview>

Dataplex: <https://cloud.google.com/dataplex/docs/overview>

BigLake: <https://cloud.google.com/bigquery/docs/biglake-intro>

Question: 226

CertyIQ

Your organization has two Google Cloud projects, project A and project B. In project A, you have a Pub/Sub topic that receives data from confidential sources. Only the resources in project A should be able to access the data in that topic. You want to ensure that project B and any future project cannot access data in the project A topic. What should you do?

A.Add firewall rules in project A so only traffic from the VPC in project A is permitted.

B.Configure VPC Service Controls in the organization with a perimeter around project A.

C.Use Identity and Access Management conditions to ensure that only users and service accounts in project A. can access resources in project A.

D.Configure VPC Service Controls in the organization with a perimeter around the VPC of project A.

Answer: B

Explanation:

The correct answer is **B. Configure VPC Service Controls in the organization with a perimeter around project**

A.

Here's why:

VPC Service Controls (VPC SC) provide a security perimeter that limits the ability of resources within a VPC to access Google Cloud services outside of that perimeter. By creating a perimeter around project A, you effectively isolate it from project B and any future projects. This prevents unauthorized access to the sensitive Pub/Sub topic, even if a principal in another project possesses the correct IAM permissions. IAM controls access to Google Cloud resources but does not control access based on network location or boundaries. Firewall rules control network traffic but do not prevent a project with appropriate permissions from accessing a Pub/Sub topic. VPC SC operates at a higher level, enforcing access controls based on defined service perimeters. It blocks access at the API level, preventing data from crossing the perimeter, regardless of the assigned IAM permissions. Option D is incorrect as VPC SC perimeters are based on Google Cloud projects, not VPCs, and using VPC's for definition will allow external access.

Option A (Firewall Rules): Firewalls only control network traffic and cannot prevent authorized principals (like a service account) from outside the project accessing a Pub/Sub topic.

Option C (IAM Conditions): IAM conditions control access based on identity, but if a service account in project B is given the proper IAM roles/permissions to access the Pub/Sub topic in project A, it would still have access.

Option D (VPC SC around the VPC): VPC Service Controls create perimeters around Google Cloud projects, not VPCs, and the solution should ensure all access is blocked from outside the defined project.

Key takeaway: VPC Service Controls are specifically designed to prevent data exfiltration by establishing security perimeters around projects, protecting sensitive data even if the proper IAM roles are assigned to outside users/service accounts.

Authoritative Links:

VPC Service Controls Overview: <https://cloud.google.com/vpc-service-controls/docs/overview>

VPC Service Controls Perimeters: <https://cloud.google.com/vpc-service-controls/docs/perimeters>

Question: 227

CertyIQ

You stream order data by using a Dataflow pipeline, and write the aggregated result to Memorystore. You provisioned a Memorystore for Redis instance with Basic Tier, 4 GB capacity, which is used by 40 clients for read-only access. You are expecting the number of read-only clients to increase significantly to a few hundred and you need to be able to support the demand. You want to ensure that read and write access availability is not impacted, and any changes you make can be deployed quickly. What should you do?

- A. Create a new Memorystore for Redis instance with Standard Tier. Set capacity to 4 GB and read replica to No read replicas (high availability only). Delete the old instance.
- B. Create a new Memorystore for Redis instance with Standard Tier. Set capacity to 5 GB and create multiple read replicas. Delete the old instance.
- C. Create a new Memorystore for Memcached instance. Set a minimum of three nodes, and memory per node to 4 GB. Modify the Dataflow pipeline and all clients to use the Memcached instance. Delete the old instance.
- D. Create multiple new Memorystore for Redis instances with Basic Tier (4 GB capacity). Modify the Dataflow pipeline and new clients to use all instances.

Answer: B

Explanation:

The correct answer is **B. Create a new Memorystore for Redis instance with Standard Tier. Set capacity to 5 GB and create multiple read replicas. Delete the old instance.**

Here's the justification:

The existing setup uses a Basic Tier Redis instance, which is unsuitable for high read-load scenarios. The Basic Tier offers no read replicas and limited availability. To support a significant increase in read-only clients without impacting write performance, we need to move to a solution with read scaling capabilities.

Standard Tier Redis instances allow the creation of read replicas. These replicas handle read requests, offloading the primary instance and improving read performance and overall availability. Increasing the capacity to 5GB provides slightly more overhead, which is beneficial considering the workload changes. This approach enables quick deployment as it involves creating a new instance with desired properties and a simple switch-over of clients and pipeline to the new instance.

Option A is insufficient because it doesn't leverage read replicas and relies solely on the primary node for both read and write operations, which is not scalable for hundreds of clients. Option C is incorrect because Memcached is optimized for caching, and is not as well-suited for the complex, reliable persistence needed as a primary datastore like Redis, and migrating the system to Memcached may not be trivial. Option D introduces complexity by requiring application-level sharding and load balancing across multiple instances, increasing maintenance overhead and adding extra deployment work.

Therefore, upgrading to a Standard Tier Redis instance with read replicas is the most efficient, scalable, and maintainable solution for the given problem.

Authoritative Links:

Memorystore for Redis Tier Comparison: <https://cloud.google.com/memorystore/docs/redis/redis-tiers>

Memorystore for Redis Read Replicas: <https://cloud.google.com/memorystore/docs/redis/read-replicas>

Question: 228

CertyIQ

You have a streaming pipeline that ingests data from Pub/Sub in production. You need to update this streaming pipeline with improved business logic. You need to ensure that the updated pipeline reprocesses the previous two days of delivered Pub/Sub messages. What should you do? (Choose two.)

- A. Use the Pub/Sub subscription clear-retry-policy flag
- B. Use Pub/Sub Snapshot capture two days before the deployment.
- C. Create a new Pub/Sub subscription two days before the deployment.
- D. Use the Pub/Sub subscription retain-acked-messages flag.
- E. Use Pub/Sub Seek with a timestamp.

Answer: BE

Explanation:

Okay, let's break down why options B and E are the correct choices for reprocessing historical Pub/Sub messages in this scenario.

Why Option B (Use Pub/Sub Snapshot) is correct:

Pub/Sub Snapshots are designed to capture a point-in-time state of a subscription. By creating a snapshot before deploying the updated pipeline, specifically two days prior as required, you preserve a copy of the messages that were available on the subscription at that time. Once the new pipeline is deployed, you can instruct it to consume from this snapshot. This effectively rewinds the subscription to the desired point, allowing the new pipeline to process the historical data. This approach is ideal for reprocessing specific historical data without impacting ongoing message delivery to other subscribers. It's a crucial feature for handling pipeline updates and ensuring data integrity.

Why Option E (Use Pub/Sub Seek with a timestamp) is correct:

Pub/Sub's Seek functionality allows a subscription to be repositioned to a specific point in time or message ID. If your subscription has message retention enabled (which is essential for these kinds of historical reprocessing), using seek with a timestamp two days in the past before you deploy the updated pipeline will cause the new pipeline to begin consuming messages from that point. This method bypasses the need to create a snapshot and instead directly alters the position from which the subscription is reading, making it another viable method for historical reprocessing. This is very useful for ensuring that the new logic also applies to older data.

Why other options are incorrect:

Option A (clear-retry-policy): The clear-retry-policy flag relates to how Pub/Sub handles failed acknowledgements, not historical message reprocessing. It's not relevant to the requirements of the problem.

Option C (Create a new subscription two days before deployment): While this would technically work in some situations, it's not the most efficient or resource-friendly approach. It also introduces the complexity of managing two subscriptions for the same data stream. Snapshots and seek are intended for this scenario, and are better practices.

Option D (retain-acked-messages flag): While retain-acked-messages is needed for historical reprocessing with seek, it is not the solution by itself. This flag just ensures the messages are kept available after consumption but doesn't help in moving the cursor back. It is a prerequisite for both the correct options in this case.

In summary:

The correct combination (B & E) leverages Pub/Sub's snapshot and seek features to reprocess historical data effectively while minimizing disruption. Option B helps avoid impacts to current pipelines, and E provides a more immediate method for repositioning the subscription directly. Both approaches are fundamental to managing stream processing updates.

Authoritative Links:

Pub/Sub Snapshots: <https://cloud.google.com/pubsub/docs/snapshots>

Pub/Sub Seek: <https://cloud.google.com/pubsub/docs/replay-overview>

Pub/Sub Message Retention: https://cloud.google.com/pubsub/docs/subscriber#message_retention

Question: 229

CertyIQ

You currently use a SQL-based tool to visualize your data stored in BigQuery. The data visualizations require the use of outer joins and analytic functions. Visualizations must be based on data that is no less than 4 hours old. Business users are complaining that the visualizations are too slow to generate. You want to improve the performance of the visualization queries while minimizing the maintenance overhead of the data preparation pipeline. What should you do?

- A. Create materialized views with the `allow_non_incremental_definition` option set to true for the visualization queries. Specify the `max_staleness` parameter to 4 hours and the `enable_refresh` parameter to true. Reference the materialized views in the data visualization tool.
- B. Create views for the visualization queries. Reference the views in the data visualization tool.
- C. Create a Cloud Function instance to export the visualization query results as parquet files to a Cloud Storage bucket. Use Cloud Scheduler to trigger the Cloud Function every 4 hours. Reference the parquet files in the data visualization tool.
- D. Create materialized views for the visualization queries. Use the incremental updates capability of BigQuery materialized views to handle changed data automatically. Reference the materialized views in the data visualization tool.

Answer: A

Explanation:

The optimal solution is to use materialized views with the `allow_non_incremental_definition` option set to true, a `max_staleness` of 4 hours, and `enable_refresh` set to true (Option A). Materialized views precompute and store the results of complex queries, including those with outer joins and analytic functions, dramatically improving query performance for downstream visualizations. Specifying `allow_non_incremental_definition` enables the creation of materialized views with features like outer joins which are not suitable for incremental updates, ensuring the desired transformation. The `max_staleness` parameter allows the materialized view to be used as long as it's no more than 4 hours old, meeting the data freshness requirement. Setting `enable_refresh` to true allows for the view to be refreshed automatically. By referencing these materialized views in the visualization tool, the business users will experience significantly faster load times. This approach minimizes maintenance overhead as BigQuery automatically manages the materialization and refresh processes.

Option B (using regular views) would not improve performance because it doesn't precompute results. The queries would be executed every time, resulting in the same slowness. Option C (exporting to Cloud Storage with Cloud Function) introduces additional complexity and maintenance (Cloud Function, Cloud Scheduler, parquet management) for the same benefit. It also adds latency since data has to be exported before being read in the visualization tool. Option D (materialized views with incremental updates) is not appropriate in this case because the queries require outer joins which are not compatible with incremental updates and would require the `allow_non_incremental_definition` parameter.

Here are resources for further reading:

BigQuery Materialized Views: <https://cloud.google.com/bigquery/docs/materialized-views>

Materialized View Refresh: <https://cloud.google.com/bigquery/docs/materialized-views#refresh>

Allowing non-incremental definition: https://cloud.google.com/bigquery/docs/reference/standard-sql/create-materialized-view#allow_non_incremental_definition

Question: 230

CertyIQ

You need to modernize your existing on-premises data strategy. Your organization currently uses:

- Apache Hadoop clusters for processing multiple large data sets, including on-premises Hadoop Distributed File System (HDFS) for data replication.
- Apache Airflow to orchestrate hundreds of ETL pipelines with thousands of job steps.

You need to set up a new architecture in Google Cloud that can handle your Hadoop workloads and requires minimal changes to your existing orchestration processes. What should you do?

- A. Use Bigtable for your large workloads, with connections to Cloud Storage to handle any HDFS use cases. Orchestrate your pipelines with Cloud Composer.
- B. Use Dataproc to migrate Hadoop clusters to Google Cloud, and Cloud Storage to handle any HDFS use cases. Orchestrate your pipelines with Cloud Composer.
- C. Use Dataproc to migrate Hadoop clusters to Google Cloud, and Cloud Storage to handle any HDFS use cases. Convert your ETL pipelines to Dataflow.
- D. Use Dataproc to migrate your Hadoop clusters to Google Cloud, and Cloud Storage to handle any HDFS use cases. Use Cloud Data Fusion to visually design and deploy your ETL pipelines.

Answer: B

Explanation:

The most appropriate solution is **B**, which proposes using Dataproc and Cloud Composer. Here's why:

Dataproc for Hadoop Migration: Dataproc is a managed Hadoop and Spark service on Google Cloud. It allows

you to migrate your existing Hadoop clusters with minimal code changes, preserving your familiar Hadoop ecosystem (HDFS, YARN, etc.). This aligns with the requirement of handling existing Hadoop workloads.

Cloud Storage for HDFS: Cloud Storage is a scalable, durable object storage service that can seamlessly replace HDFS for data storage and replication. Dataproc can directly interact with Cloud Storage, enabling a smooth transition from on-premises HDFS.

Cloud Composer for Orchestration: Cloud Composer is a managed workflow orchestration service based on Apache Airflow. Since your organization already uses Airflow, adopting Cloud Composer minimizes the learning curve and allows you to reuse your existing Airflow-based ETL pipeline definitions with minimal changes. This directly addresses the need to maintain existing orchestration processes.

Why other options are less suitable:

Option A (Bigtable): While Bigtable is a great NoSQL database, it's not a direct replacement for HDFS and Hadoop processing. Using Bigtable would require a significant re-architecting of your workloads and is not ideal for handling Hadoop-based workloads that need to be migrated with minimal code changes.

Option C (Dataflow): Dataflow is a powerful stream and batch processing service, but rewriting existing Airflow-based ETL pipelines to Dataflow pipelines is a large and unnecessary undertaking when you already have a well-established orchestration system. Dataflow is also not suited for directly migrating existing Hadoop clusters and workloads.

Option D (Cloud Data Fusion): Cloud Data Fusion is a visual ETL service, however, it would necessitate rewriting existing Airflow pipelines and it does not assist with the migration of the Hadoop infrastructure.

In essence, **option B provides a lift-and-shift approach** that allows you to leverage the power of Google Cloud while minimizing disruptions to existing processes. It aligns with the requirement of maintaining the existing orchestration system and handling Hadoop workloads effectively.

Authoritative Links for further research:

Dataproc: <https://cloud.google.com/dataproc>

Cloud Storage: <https://cloud.google.com/storage>

Cloud Composer: <https://cloud.google.com/composer>

Bigtable: <https://cloud.google.com/bigtable>

Dataflow: <https://cloud.google.com/dataflow>

Cloud Data Fusion: <https://cloud.google.com/data-fusion>

Question: 231

CertyIQ

You recently deployed several data processing jobs into your Cloud Composer 2 environment. You notice that some tasks are failing in Apache Airflow. On the monitoring dashboard, you see an increase in the total workers memory usage, and there were worker pod evictions. You need to resolve these errors. What should you do? (Choose two.)

- A. Increase the directed acyclic graph (DAG) file parsing interval.
- B. Increase the Cloud Composer 2 environment size from medium to large.
- C. Increase the maximum number of workers and reduce worker concurrency.
- D. Increase the memory available to the Airflow workers.
- E. Increase the memory available to the Airflow triggerer.

Answer: C

Explanation:

Here's a detailed justification for why options C and D are the correct choices, and why the others are incorrect:

The Problem: The core issue is that Airflow workers are running out of memory, leading to task failures and

pod evictions. This indicates that the current worker resources are insufficient to handle the processing load.

Correct Options:

C. Increase the maximum number of workers and reduce worker concurrency. Increasing the maximum number of workers allows Airflow to parallelize tasks more effectively, spreading the load. Reducing worker concurrency (the number of tasks each worker can handle simultaneously) also alleviates memory pressure on individual workers. This combination allows for handling the processing load more efficiently by using more, less-burdened workers, rather than fewer overloaded ones. This approach addresses the core issue of memory exhaustion and pod eviction.

D. Increase the memory available to the Airflow workers. Directly increasing the memory available to the workers provides more space for processing data, reducing the chance of memory-related failures. This directly tackles the symptom of memory shortage, giving each worker more breathing room. This is a straightforward solution when workers are simply running out of available memory.

Incorrect Options:

A. Increase the directed acyclic graph (DAG) file parsing interval. DAG parsing frequency affects how often Airflow reads and processes DAG files. It does not directly impact memory usage of the tasks being executed by the workers, and therefore, it won't address the issue of memory shortage.

B. Increase the Cloud Composer 2 environment size from medium to large. While a larger environment might provide more resources, it's a more expensive and less focused solution than the specific needs indicate. Upgrading the entire environment is not targeted to address the specific issue of worker memory shortage, and would be a large, broad solution that would have additional implications like cost. Increasing worker resources or reducing concurrency is a more precise action.

E. Increase the memory available to the Airflow triggerer. The triggerer is responsible for scheduling and triggering tasks, but it is not involved in actual data processing. Increasing triggerer memory would not directly solve the worker memory issue.

In summary: The core problem is related to worker resource constraints, specifically memory. The recommended actions (C and D) directly address this by increasing the number of workers, lowering concurrency and increasing memory, enabling a better processing environment for Airflow workers. Increasing DAG parsing intervals or triggerer memory are not related to the issue, and increasing the whole environment size is too broad of a solution.

Authoritative Links:

Cloud Composer Scaling: <https://cloud.google.com/composer/docs/concepts/scaling>

Airflow Worker Configuration: <https://airflow.apache.org/docs/apache-airflow/stable/configurations-ref.html#worker>

Kubernetes Pod Eviction: <https://kubernetes.io/docs/concepts/scheduling-eviction/node-pressure-eviction/>

Question: 232

CertyIQ

You are on the data governance team and are implementing security requirements to deploy resources. You need to ensure that resources are limited to only the europe-west3 region. You want to follow Google-recommended practices.

What should you do?

- A. Set the constraints/gcp.resourceLocations organization policy constraint to in:europe-west3-locations.
- B. Deploy resources with Terraform and implement a variable validation rule to ensure that the region is set to the europe-west3 region for all resources.
- C. Set the constraints/gcp.resourceLocations organization policy constraint to in:eu-locations.
- D. Create a Cloud Function to monitor all resources created and automatically destroy the ones created outside

the europe-west3 region.

Answer: A

Explanation:

The correct answer is A because it directly leverages Google Cloud's Organization Policy service to enforce resource location constraints, which is the recommended approach for centrally managing and enforcing such requirements. Setting the constraints/gcp.resourceLocations policy with in:europe-west3-locations ensures that all new resources created within the organization or folders inheriting this policy are restricted to only the europe-west3 region. This method is declarative, meaning the policy is defined once and then automatically enforced by Google Cloud, reducing the chance for errors and misconfigurations compared to manual checks or custom scripting. Options B, C, and D are less ideal. Terraform validation in option B adds complexity and is not a centralized enforcement point like organization policies, potentially leading to inconsistencies across teams and projects. Option C is too broad, allowing resources in all EU locations rather than the specific europe-west3 region. Option D, using a Cloud Function to monitor and delete resources, is reactive, inefficient, and creates unnecessary resource churn. It also increases operational overhead. Utilizing organization policies for this scenario is aligned with Google Cloud's best practices for data governance and security.

Authoritative Links:

Organization Policy: <https://cloud.google.com/resource-manager/docs/organization-policy/overview>
constraints/gcp.resourceLocations policy: https://cloud.google.com/resource-manager/docs/organization-policy/defining-constraints#resource_location_constraint

Question: 233

CertyIQ

You are a BigQuery admin supporting a team of data consumers who run ad hoc queries and downstream reporting in tools such as Looker. All data and users are combined under a single organizational project. You recently noticed some slowness in query results and want to troubleshoot where the slowdowns are occurring. You think that there might be some job queuing or slot contention occurring as users run jobs, which slows down access to results. You need to investigate the query job information and determine where performance is being affected. What should you do?

- A. Use slot reservations for your project to ensure that you have enough query processing capacity and are able to allocate available slots to the slower queries.
- B. Use Cloud Monitoring to view BigQuery metrics and set up alerts that let you know when a certain percentage of slots were used.
- C. Use available administrative resource charts to determine how slots are being used and how jobs are performing over time. Run a query on the INFORMATION_SCHEMA to review query performance.
- D. Use Cloud Logging to determine if any users or downstream consumers are changing or deleting access grants on tagged resources.

Answer: C

Explanation:

The correct answer is C. Here's why:

Option C directly addresses the problem by suggesting the use of BigQuery administrative resource charts and the INFORMATION_SCHEMA. These tools provide insight into slot utilization, job performance, and historical trends. Specifically, the administrative resource charts in the BigQuery UI visually display how slots are being consumed over time, helping identify periods of high usage or potential bottlenecks. The INFORMATION_SCHEMA provides detailed query job metadata that can be queried directly, allowing you to analyze job durations, slot consumption per job, and other relevant performance metrics. These are ideal for

diagnosing slowness and contention.

Option A, using slot reservations, is a potential solution but a premature step. It doesn't first diagnose the root cause of the issue. Reservations should be implemented after identifying and confirming contention.

Option B, using Cloud Monitoring, offers valuable metrics and alerting capabilities but doesn't provide the job-specific details needed for troubleshooting individual query performance issues. Monitoring will show aggregate consumption, not necessarily specific problems per query.

Option D, using Cloud Logging, is relevant to access control but not to query performance issues. The scenario specifically mentions slot contention and query slowdowns, not changes in access control.

Therefore, C is the most appropriate initial action as it enables direct investigation of the suspected issue – slot contention and query performance slowdown – through BigQuery's native tools.

Relevant Concepts:

Slot Contention: When multiple BigQuery jobs compete for a limited number of processing slots, query performance can degrade.

BigQuery Administrative Resource Charts: Visual tools in the BigQuery UI that display slot utilization, job counts, and performance metrics over time.

INFORMATION_SCHEMA: A set of read-only tables in BigQuery that contains metadata about your BigQuery resources, including job information and performance details.

Authoritative Links:

BigQuery INFORMATION_SCHEMA: <https://cloud.google.com/bigquery/docs/information-schema-intro>

BigQuery Monitoring: <https://cloud.google.com/bigquery/docs/monitoring>

BigQuery Reservations: <https://cloud.google.com/bigquery/docs/reservations-intro>

Question: 234

CertyIQ

You migrated a data backend for an application that serves 10 PB of historical product data for analytics. Only the last known state for a product, which is about 10 GB of data, needs to be served through an API to the other applications. You need to choose a cost-effective persistent storage solution that can accommodate the analytics requirements and the API performance of up to 1000 queries per second (QPS) with less than 1 second latency. What should you do?

- A.1. Store the historical data in BigQuery for analytics.
- 2. Use a materialized view to precompute the last state of a product.
- 3. Serve the last state data directly from BigQuery to the API.
- B.1. Store the products as a collection in Firestore with each product having a set of historical changes.
- 2. Use simple and compound queries for analytics.
- 3. Serve the last state data directly from Firestore to the API.
- C.1. Store the historical data in Cloud SQL for analytics.
- 2. In a separate table, store the last state of the product after every product change.
- 3. Serve the last state data directly from Cloud SQL to the API.
- D.1. Store the historical data in BigQuery for analytics.
- 2. In a Cloud SQL table, store the last state of the product after every product change.
- 3. Serve the last state data directly from Cloud SQL to the API.

Answer: D

Explanation:

The correct answer is **D**. Here's why:

BigQuery for Analytics (D.1): BigQuery is a fully managed, serverless data warehouse optimized for large-scale analytical workloads. It is perfect for storing and processing the 10 PB of historical product data due to its scalability and cost-effectiveness for analytical queries. (Learn more: <https://cloud.google.com/bigquery>)

Cloud SQL for API Data (D.2 & D.3): Cloud SQL offers managed relational databases, ideal for low-latency, transactional operations. Storing the last known state of each product in a Cloud SQL table ensures that the API can serve the 10GB of required data with the needed 1000 QPS and sub-second latency. Relational databases are designed for quick reads and writes of specific data points, making them perfect for this API use case. (Learn more: <https://cloud.google.com/sql>)

Why other options are less suitable:

Option A: While using BigQuery for analytics is sound, relying on a materialized view to serve real-time API traffic is not efficient. Materialized views in BigQuery are typically not optimized for the low-latency, high-QPS requirements of an API. BigQuery is optimized for analytics, not transactional loads. Serving direct from BQ will cause performance issues and will be very expensive.

Option B: Firestore, while suitable for some document storage, isn't optimized for large-scale analytical workloads like the 10 PB historical data. Additionally, handling complex analytical queries directly on Firestore can become inefficient and costly.

Option C: Using Cloud SQL for 10 PB of historical data would be incredibly expensive and inefficient. Cloud SQL is not designed for handling massive analytical datasets.

In summary, solution D provides a cost-effective and performant architecture by separating the analytical (BigQuery) and transactional (Cloud SQL) workloads.

Question: 235

CertyIQ

You want to schedule a number of sequential load and transformation jobs. Data files will be added to a Cloud Storage bucket by an upstream process. There is no fixed schedule for when the new data arrives. Next, a Dataproc job is triggered to perform some transformations and write the data to BigQuery. You then need to run additional transformation jobs in BigQuery. The transformation jobs are different for every table. These jobs might take hours to complete. You need to determine the most efficient and maintainable workflow to process hundreds of tables and provide the freshest data to your end users. What should you do?

- A.1. Create an Apache Airflow directed acyclic graph (DAG) in Cloud Composer with sequential tasks by using the Cloud Storage, Dataproc, and BigQuery operators.
- 2. Use a single shared DAG for all tables that need to go through the pipeline.
- 3. Schedule the DAG to run hourly.
- B.1. Create an Apache Airflow directed acyclic graph (DAG) in Cloud Composer with sequential tasks by using the Cloud Storage, Dataproc, and BigQuery operators.
- 2. Create a separate DAG for each table that needs to go through the pipeline.
- 3. Schedule the DAGs to run hourly.
- C.1. Create an Apache Airflow directed acyclic graph (DAG) in Cloud Composer with sequential tasks by using the Dataproc and BigQuery operators.
- 2. Use a single shared DAG for all tables that need to go through the pipeline.
- 3. Use a Cloud Storage object trigger to launch a Cloud Function that triggers the DAG.
- D.1. Create an Apache Airflow directed acyclic graph (DAG) in Cloud Composer with sequential tasks by using the Dataproc and BigQuery operators.
- 2. Create a separate DAG for each table that needs to go through the pipeline.
- 3. Use a Cloud Storage object trigger to launch a Cloud Function that triggers the DAG.

Answer: D

Explanation:

The correct answer is **D**. Here's a detailed justification:

Option D provides the most efficient and maintainable solution for processing hundreds of tables with varying transformation needs based on new data arrival in Cloud Storage. Using Cloud Composer with Apache Airflow allows us to define workflows as DAGs (Directed Acyclic Graphs). Separating DAGs for each table (D.2) allows for independent scheduling, monitoring, and error handling, preventing a failure in one table's pipeline from blocking the others. This also ensures flexibility since each table has unique transformations. Triggering the DAGs using a Cloud Function that responds to Cloud Storage object creation events (D.3) ensures data is processed as soon as it arrives, providing the freshest data. This event-driven approach avoids the need for polling and reduces latency. This method also maximizes resource utilization since the processing happens when data is ready. The pipeline avoids unnecessary processing and doesn't run on a timer when no new data arrives. Option A is inefficient because it schedules the pipeline hourly regardless of new data availability. Option B unnecessarily runs all pipelines every hour, wasting resources, and can cause lag in data processing. Option C doesn't effectively use the trigger mechanism for all data and doesn't use individual DAGs for the flexibility it would offer. Therefore, separating workflows into multiple DAGs triggered by Cloud Storage events via Cloud Functions, as in option D, provides scalability, efficiency, maintainability, and real-time processing capabilities.

Authoritative Links for Further Research:

Cloud Composer: <https://cloud.google.com/composer/docs>

Apache Airflow: <https://airflow.apache.org/docs/>

Cloud Storage Triggers: <https://cloud.google.com/functions/docs/calling/storage>

Cloud Functions: <https://cloud.google.com/functions/docs>

Dataproc: <https://cloud.google.com/dataproc/docs>

BigQuery: <https://cloud.google.com/bigquery/docs>

Question: 236

CertyIQ

You are deploying a MySQL database workload onto Cloud SQL. The database must be able to scale up to support several readers from various geographic regions. The database must be highly available and meet low RTO and RPO requirements, even in the event of a regional outage. You need to ensure that interruptions to the readers are minimal during a database failover. What should you do?

- A. Create a highly available Cloud SQL instance in region A. Create a highly available read replica in region B. Scale up read workloads by creating cascading read replicas in multiple regions. Backup the Cloud SQL instances to a multi-regional Cloud Storage bucket. Restore the Cloud SQL backup to a new instance in another region when Region A is down.
- B. Create a highly available Cloud SQL instance in region A. Scale up read workloads by creating read replicas in multiple regions. Promote one of the read replicas when region A is down.
- C. Create a highly available Cloud SQL instance in region A. Create a highly available read replica in region B. Scale up read workloads by creating cascading read replicas in multiple regions. Promote the read replica in region B when region A is down.
- D. Create a highly available Cloud SQL instance in region A. Scale up read workloads by creating read replicas in the same region. Failover to the standby Cloud SQL instance when the primary instance fails.

Answer: C

Explanation:

The correct answer is **C**. Here's why:

Option C provides a robust solution for high availability, scalability, and disaster recovery for a MySQL database on Cloud SQL, effectively addressing the problem requirements. Creating a highly available primary instance in region A and a highly available read replica in region B ensures immediate failover capabilities, fulfilling the low RTO and RPO requirements. The read replica in region B acts as a hot standby, minimizing

downtime during a regional outage. Cascading read replicas in multiple regions, stemming from the read replica in region B, enable the scaling of read workloads across different geographic locations. This ensures low-latency access for readers irrespective of their location. Promoting the read replica in region B to become the new primary instance upon failure of region A guarantees uninterrupted database access, aligning with the need to minimize interruptions to the readers. Options A, B, and D lack the cross-regional redundancy, the availability of a hot standby, or the scalable read architecture, thereby making them unsuitable for meeting the problem requirements. For further reading, refer to [Cloud SQL High Availability](#) and [Cloud SQL Read Replicas](#) documentation on Google Cloud's website.

Question: 237

CertyIQ

You are planning to load some of your existing on-premises data into BigQuery on Google Cloud. You want to either stream or batch-load data, depending on your use case. Additionally, you want to mask some sensitive data before loading into BigQuery. You need to do this in a programmatic way while keeping costs to a minimum. What should you do?

- A. Use Cloud Data Fusion to design your pipeline, use the Cloud DLP plug-in to de-identify data within your pipeline, and then move the data into BigQuery.
- B. Use the BigQuery Data Transfer Service to schedule your migration. After the data is populated in BigQuery, use the connection to the Cloud Data Loss Prevention (Cloud DLP) API to de-identify the necessary data.
- C. Create your pipeline with Dataflow through the Apache Beam SDK for Python, customizing separate options within your code for streaming, batch processing, and Cloud DLP. Select BigQuery as your data sink.
- D. Set up Datastream to replicate your on-premise data on BigQuery.

Answer: C

Explanation:

The correct answer is C because it provides the most flexible and cost-effective solution for handling both streaming and batch data loading while integrating data masking programmatically. Dataflow, through the Apache Beam SDK, offers a unified programming model capable of processing both batch and stream data. This flexibility avoids the need for separate tools or pipelines for different loading methods, keeping costs down. Furthermore, the Apache Beam SDK allows you to integrate with Cloud Data Loss Prevention (DLP) directly within your pipeline code, enabling in-stream data masking before writing to BigQuery. This avoids extra steps and costs associated with processing data twice.

Option A, using Cloud Data Fusion, while it provides a graphical interface, doesn't provide as much granular control and might not be the most cost-effective for customized programmatic needs. Option B, utilizing BigQuery Data Transfer Service with subsequent Cloud DLP API calls, introduces unnecessary steps and costs. The transfer service is mainly for scheduled data movement and doesn't handle data transformation in the process. Calling Cloud DLP after loading in BigQuery also adds extra processing and storage costs. Lastly, Datastream in option D is designed for replicating databases, and it is not ideal for general data loading, nor does it provide built-in masking capabilities.

Therefore, Dataflow with Apache Beam offers the optimal solution due to its unified processing, programmatic control, in-stream DLP capabilities, and cost-efficiency.

Supporting Links:

Apache Beam: <https://beam.apache.org/>

Google Cloud Dataflow: <https://cloud.google.com/dataflow>

Google Cloud Data Loss Prevention (DLP): <https://cloud.google.com/dlp>

BigQuery Data Transfer Service: <https://cloud.google.com/bigquery/docs/data-transfer-service>

Google Cloud Datastream: <https://cloud.google.com/datastream>

Question: 238

CertyIQ

You want to encrypt the customer data stored in BigQuery. You need to implement per-user crypto-deletion on data stored in your tables. You want to adopt native features in Google Cloud to avoid custom solutions. What should you do?

- A. Implement Authenticated Encryption with Associated Data (AEAD) BigQuery functions while storing your data in BigQuery.
- B. Create a customer-managed encryption key (CMEK) in Cloud KMS. Associate the key to the table while creating the table.
- C. Create a customer-managed encryption key (CMEK) in Cloud KMS. Use the key to encrypt data before storing in BigQuery.
- D. Encrypt your data during ingestion by using a cryptographic library supported by your ETL pipeline.

Answer: A

Explanation:

The correct answer is **A. Implement Authenticated Encryption with Associated Data (AEAD) BigQuery functions while storing your data in BigQuery.**

Here's the justification:

The core requirement is per-user crypto-deletion, meaning the ability to effectively erase data for individual users without affecting others, using native Google Cloud features. AEAD functions in BigQuery are designed to achieve this. These functions encrypt data using a key derived from user-specific information, like a user ID. This allows for selective data access and, crucially, crypto-deletion by destroying the user-specific key, rendering that user's data unreadable without impacting other users' encrypted data.

Options B and C utilize Customer-Managed Encryption Keys (CMEK) which provide encryption at rest, but they don't offer granular, per-user crypto-deletion. CMEK applies to the entire table or dataset, not individual records associated with a user. Revoking access to the CMEK would impact all data, failing the requirement of selective removal. Option D suggests using a cryptographic library within the ETL pipeline, which moves the encryption logic outside of BigQuery. While valid for encryption, it creates complexities in managing keys and doesn't align with the directive to utilize native Google Cloud features for this specific requirement of per-user deletion within BigQuery. Therefore, AEAD functions provide the necessary native functionality for per-user crypto-deletion within BigQuery, making option A the best choice.

Further Reading:

BigQuery AEAD functions: <https://cloud.google.com/bigquery/docs/encryption-functions>

BigQuery Encryption Overview: <https://cloud.google.com/bigquery/docs/encryption>

Cloud KMS: <https://cloud.google.com/kms/docs>

Question: 239

CertyIQ

The data analyst team at your company uses BigQuery for ad-hoc queries and scheduled SQL pipelines in a Google Cloud project with a slot reservation of 2000 slots. However, with the recent introduction of hundreds of new non time-sensitive SQL pipelines, the team is encountering frequent quota errors. You examine the logs and notice that approximately 1500 queries are being triggered concurrently during peak time. You need to resolve the concurrency issue. What should you do?

- A. Increase the slot capacity of the project with baseline as 0 and maximum reservation size as 3000.
- B. Update SQL pipelines to run as a batch query, and run ad-hoc queries as interactive query jobs.

- C. Increase the slot capacity of the project with baseline as 2000 and maximum reservation size as 3000.
- D. Update SQL pipelines and ad-hoc queries to run as interactive query jobs.

Answer: B

Explanation:

The core issue is that the data analyst team's 2000-slot BigQuery reservation is insufficient to handle the concurrent execution of 1500 queries, primarily due to the introduction of new, non-time-sensitive pipelines. Option B, updating SQL pipelines to run as batch queries and ad-hoc queries as interactive jobs, is the most effective solution. BigQuery distinguishes between batch and interactive query priorities. Batch queries run in the background with lower priority, using available slots when interactive queries aren't consuming them. This approach doesn't require immediate resource increases, thereby reducing costs and resource contention. Interactive queries are given higher priority and resources, thus catering to time-sensitive tasks. This ensures that ad-hoc queries used for analysis remain fast and responsive. Option A, while increasing slot capacity, is not the most cost-effective and does not address the underlying issue of uncontrolled pipeline execution. Option C is also not an ideal option as it does not differentiate between query types and the maximum slot allocation can potentially lead to unnecessary expenses. Option D assigns lower priority, non time-sensitive pipeline queries to higher priority interactive jobs, which is inefficient and can lead to resource contention. By utilizing the appropriate query types, option B optimizes resource usage within the existing allocation, providing an elegant solution without increased expenditure.

Further Research:

BigQuery query types: https://cloud.google.com/bigquery/docs/running-queries#types_of_queries

BigQuery reservations: <https://cloud.google.com/bigquery/docs/reservations-intro>

Question: 240

CertyIQ

You are designing a data mesh on Google Cloud by using Dataplex to manage data in BigQuery and Cloud Storage. You want to simplify data asset permissions. You are creating a customer virtual lake with two user groups:

- Data engineers, which require full data lake access
- Analytic users, which require access to curated data

You need to assign access rights to these two groups. What should you do?

- A.1. Grant the dataplex.dataOwner role to the data engineer group on the customer data lake.
2. Grant the dataplex.dataReader role to the analytic user group on the customer curated zone.
- B.1. Grant the dataplex.dataReader role to the data engineer group on the customer data lake.
2. Grant the dataplex.dataOwner to the analytic user group on the customer curated zone.
- C.1. Grant the bigquery.dataOwner role on BigQuery datasets and the storage.objectCreator role on Cloud Storage buckets to data engineers.
2. Grant the bigquery.dataViewer role on BigQuery datasets and the storage.objectViewer role on Cloud Storage buckets to analytic users.
- D.1. Grant the bigquery.dataViewer role on BigQuery datasets and the storage.objectViewer role on Cloud Storage buckets to data engineers.
2. Grant the bigquery.dataOwner role on BigQuery datasets and the storage.objectEditor role on Cloud Storage buckets to analytic users.

Answer: A

Explanation:

The correct answer is A because it leverages Dataplex's role-based access control to efficiently manage permissions for different user groups within the data mesh. Option A grants the dataplex.dataOwner role to

data engineers at the data lake level, giving them full control over all data assets within the lake. This aligns with the requirement that data engineers need full data lake access. Then, it grants the `dataplex.dataReader` role to analytic users at the curated zone level. This provides read-only access to the curated data, satisfying their need to consume but not modify the data. Dataplex roles simplify access management compared to directly granting BigQuery and Storage roles, making it a better fit for a data mesh.

Options B, C, and D are incorrect for various reasons. Option B reverses the roles, granting read access to data engineers and write access to analysts, which is contrary to the requirements. Options C and D bypass Dataplex, granting roles directly on BigQuery datasets and Cloud Storage buckets. While this works, it doesn't fully utilize the benefits of Dataplex for centralized access management and scalability within the data mesh. Directly managing fine-grained BigQuery and Storage permissions is more complex than using Dataplex roles.

By using Dataplex roles, option A offers a streamlined and centralized approach to access control, ensuring that each user group has the appropriate permissions for its intended purpose within the data mesh. The roles also reflect the intention of the question, where Dataplex simplifies data access.

For further research on Dataplex

roles:<https://cloud.google.com/dataplex/docs/iam/roles><https://cloud.google.com/dataplex/docs/access-control>

Question: 241

CertyIQ

You are designing the architecture of your application to store data in Cloud Storage. Your application consists of pipelines that read data from a Cloud Storage bucket that contains raw data, and write the data to a second bucket after processing. You want to design an architecture with Cloud Storage resources that are capable of being resilient if a Google Cloud regional failure occurs. You want to minimize the recovery point objective (RPO) if a failure occurs, with no impact on applications that use the stored data. What should you do?

- A. Adopt multi-regional Cloud Storage buckets in your architecture.
- B. Adopt two regional Cloud Storage buckets, and update your application to write the output on both buckets.
- C. Adopt a dual-region Cloud Storage bucket, and enable turbo replication in your architecture.
- D. Adopt two regional Cloud Storage buckets, and create a daily task to copy from one bucket to the other.

Answer: C

Explanation:

The correct answer is C, adopting a dual-region Cloud Storage bucket with turbo replication. This approach best addresses the requirements of high resilience with a minimized RPO (Recovery Point Objective) and no impact on application access, especially in the context of regional failures.

Here's why:

Dual-region buckets: Unlike single-region buckets, dual-region buckets store data in two separate geographical regions. This inherent redundancy ensures that data remains available even if one region experiences a failure. This is fundamental for disaster recovery and business continuity.

Turbo Replication: Turbo replication, a feature of dual-region Cloud Storage, provides synchronous data replication between the two regions, meaning data is written to both locations concurrently. This significantly reduces the RPO, almost to zero in most cases, by avoiding the delay associated with asynchronous replication.

Minimized RPO: Options A and B don't give RPO guarantees. Option D, with a daily task, has a high RPO (up to 24 hours). The goal is minimal RPO. Turbo replication in C ensures that in case of regional failures, the failover

to the secondary region happens very quickly.

No impact on application: Because the dual-region setup is managed by Cloud Storage, applications interact with the bucket as a single entity. They do not need to be aware of the underlying replication, minimizing disruption in the event of a failover.

Option A (Multi-regional): Multi-regional buckets are a better fit for data access from multiple geographically diverse locations, which isn't the primary need here. They also generally have higher latency than dual-region buckets.

Option B (Two Regional Buckets + Application Updates): This option introduces complexity in the application to handle writes to both buckets, leading to higher maintenance effort. It also does not guarantee data consistency.

Option D (Two Regional Buckets + Daily Copy Task): This is a cost-effective solution but it has high RPO and could result in significant data loss in case of a failure, violating the requirement of minimized RPO.

In summary, the dual-region bucket with turbo replication combines the benefits of redundancy and near-zero RPO with minimal application impact making it the most suitable choice for the given scenario.

Authoritative Links:

Cloud Storage Locations: <https://cloud.google.com/storage/docs/locations>

Dual-region Storage: <https://cloud.google.com/storage/docs/dual-region>

Turbo Replication: <https://cloud.google.com/storage/docs/dual-region#turbo-replication>

Question: 242

CertyIQ

You have designed an Apache Beam processing pipeline that reads from a Pub/Sub topic. The topic has a message retention duration of one day, and writes to a Cloud Storage bucket. You need to select a bucket location and processing strategy to prevent data loss in case of a regional outage with an RPO of 15 minutes. What should you do?

- A.1. Use a dual-region Cloud Storage bucket.
- 2. Monitor Dataflow metrics with Cloud Monitoring to determine when an outage occurs.
- 3. Seek the subscription back in time by 15 minutes to recover the acknowledged messages.
- 4. Start the Dataflow job in a secondary region.
- B.1. Use a multi-regional Cloud Storage bucket.
- 2. Monitor Dataflow metrics with Cloud Monitoring to determine when an outage occurs.
- 3. Seek the subscription back in time by 60 minutes to recover the acknowledged messages.
- 4. Start the Dataflow job in a secondary region.
- C.1. Use a regional Cloud Storage bucket.
- 2. Monitor Dataflow metrics with Cloud Monitoring to determine when an outage occurs.
- 3. Seek the subscription back in time by one day to recover the acknowledged messages.
- 4. Start the Dataflow job in a secondary region and write in a bucket in the same region.
- D.1. Use a dual-region Cloud Storage bucket with turbo replication enabled.
- 2. Monitor Dataflow metrics with Cloud Monitoring to determine when an outage occurs.
- 3. Seek the subscription back in time by 60 minutes to recover the acknowledged messages.
- 4. Start the Dataflow job in a secondary region.

Answer: D

Explanation:

Okay, here's a detailed justification for why option D is the correct solution, along with supporting concepts and links:

Justification for Option D:

Option D is the most robust solution for achieving a 15-minute Recovery Point Objective (RPO) in the face of a regional outage for a Dataflow pipeline processing Pub/Sub messages and writing to Cloud Storage. Here's why each component of the answer is crucial:

1. **Dual-region Cloud Storage with Turbo Replication:** A dual-region bucket inherently provides higher availability than a single-region bucket by synchronously replicating data between two geographically separated regions. Turbo replication, specifically, prioritizes low latency and minimizes the time it takes to replicate data, ensuring that recent data is quickly made available in both regions. This is critical for achieving a tight RPO like 15 minutes (although strictly speaking, this can be slower than that). <https://cloud.google.com/storage/docs/locations>, <https://cloud.google.com/storage/docs/dual-region>
2. **Cloud Monitoring for Outage Detection:** Monitoring Dataflow metrics is essential to quickly identify an outage. Cloud Monitoring allows you to set up alerts that trigger when key metrics, like job failures or processing delays, exceed predefined thresholds, initiating the failover process. This is crucial for a timely response. <https://cloud.google.com/monitoring/>
3. **Seeking the Subscription Back 60 Minutes:** Pub/Sub retains messages for a specific duration. By seeking the subscription back by 60 minutes, we're accounting for the 15-minute RPO and some buffer, and ensure messages that were being processed just prior to the outage can be reprocessed by the secondary Dataflow job. This avoids data loss due to any acknowledge issues during the outage and is safe to be used even if the acknowledge succeeded. This accounts for any latency issues and time for the switchover between the primary and secondary jobs. The original topic has messages retained for one day, ensuring data availability for back-seeking. <https://cloud.google.com/pubsub/docs/subscriber>
4. **Starting the Dataflow Job in a Secondary Region:** Launching a Dataflow job in a secondary region is critical for ensuring continuous processing during a regional outage. By having a job ready to process, data can be ingested in a timely manner in the even of a disaster. With the secondary job processing data from Pub/Sub, with the subscription seek time applied, data written to cloud storage will be reprocessed and no data will be lost.

Why other options are incorrect:

Option A: Seeking the subscription back only 15 minutes is insufficient, as it doesn't account for potential latency, or issues with message acknowledgment prior to the outage, and might miss messages that were in-flight prior to the regional outage.

Option B: A multi-regional bucket offers higher redundancy, but does not offer the same low-latency replication as a dual-region bucket with turbo replication making it less suitable to meet the tight RPO. Seeking back 60 minutes also works here, but again dual-region is better here.

Option C: A regional Cloud Storage bucket lacks the data replication required to protect against a regional outage. Seeking the subscription back a day will reprocess significant amounts of data, not just the data that was lost during an outage. It also makes no sense to start the secondary job in the same region since this region is suffering an outage.

In summary, option D leverages the most appropriate features of Google Cloud Platform to create a highly resilient system that minimizes data loss and ensures business continuity even in the event of a regional outage.

Question: 243

CertyIQ

You are preparing data that your machine learning team will use to train a model using BigQueryML. They want to predict the price per square foot of real estate. The training data has a column for the price and a column for the number of square feet. Another feature column called 'feature1' contains null values due to missing data. You want

to replace the nulls with zeros to keep more data points. Which query should you use?

A.

```
SELECT * EXCEPT(feature1),  
       IFNULL(feature1, 0) AS feature1_cleaned  
FROM training_data;
```

B.

```
SELECT * EXCEPT(price, square_feet),  
       price/square_feet AS price_per_sqft  
FROM training_data  
WHERE feature1 IS NOT NULL;
```

C.

```
SELECT * EXCEPT(price, square_feet, feature1),  
       price/square_feet AS price_per_sqft,  
       IFNULL(feature1, 0) AS feature1_cleaned  
FROM training_data;
```

D.

```
SELECT *  
FROM training_data  
WHERE feature1 IS NOT NULL;
```

Answer: A

Explanation:

```
SELECT * EXCEPT(feature1),  
       IFNULL(feature1, 0) AS feature1_cleaned  
FROM training_data;
```

Question: 244

CertyIQ

Different teams in your organization store customer and performance data in BigQuery. Each team needs to keep full control of their collected data, be able to query data within their projects, and be able to exchange their data with other teams. You need to implement an organization-wide solution, while minimizing operational tasks and costs. What should you do?

- A. Ask each team to create authorized views of their data. Grant the `biquery.jobUser` role to each team.
- B. Create a BigQuery scheduled query to replicate all customer data into team projects.
- C. Ask each team to publish their data in Analytics Hub. Direct the other teams to subscribe to them.
- D. Enable each team to create materialized views of the data they need to access in their projects.

Answer: C

Explanation:

The best solution is C: Ask each team to publish their data in Analytics Hub. Direct the other teams to subscribe to them.

Here's why:

Data Sovereignty and Control: Analytics Hub respects the data ownership of each team. They retain full control over their BigQuery datasets and only share data they explicitly choose to.

Decentralized Management: Each team manages their own data publishing in Analytics Hub, minimizing the need for central IT intervention and reducing operational overhead.

Secure Data Sharing: Analytics Hub provides secure data sharing capabilities between teams within the organization, allowing controlled access without directly granting access to underlying datasets.

Cost Optimization: Data is not replicated. Instead, teams subscribe to datasets in Analytics Hub, eliminating storage costs and reducing query processing costs associated with replicated data.

Real-time/Near Real-time Access: Analytics Hub provides real-time and near-real-time access to shared datasets, so teams can use current information for insights and decision-making.

Auditing and Governance: Analytics Hub offers auditing capabilities, enabling administrators to track data sharing activities and enforce data governance policies.

Analytics Hub: Allows sharing data assets across organizations.

Let's analyze why the other options are less suitable:

A. Ask each team to create authorized views of their data. Grant the `biquery.jobUser` role to each team: This approach requires each team to manage views and grant access, potentially becoming complex and cumbersome. It also doesn't scale well as the number of teams and views grows. The `bigquery.jobUser` role is insufficient for querying the data within views, as it only allows running jobs on the project. It doesn't grant access to the underlying data.

B. Create a BigQuery scheduled query to replicate all customer data into team projects: This approach is inefficient and costly because it duplicates large amounts of data. Storage costs will increase significantly. Replication also introduces data consistency issues.

D. Enable each team to create materialized views of the data they need to access in their projects: While materialized views can improve query performance, they still involve data duplication and increase storage costs. It also creates additional management overhead for each team to maintain their materialized views. It is not appropriate for data sharing.

Here are some useful links to learn more about BigQuery and Analytics Hub:

BigQuery: <https://cloud.google.com/bigquery>

Analytics Hub: <https://cloud.google.com/analytics-hub>

Question: 245

CertyIQ

You are developing a model to identify the factors that lead to sales conversions for your customers. You have completed processing your data. You want to continue through the model development lifecycle. What should you do next?

- A. Use your model to run predictions on fresh customer input data.
- B. Monitor your model performance, and make any adjustments needed.
- C. Delineate what data will be used for testing and what will be used for training the model.
- D. Test and evaluate your model on your curated data to determine how well the model performs.

Answer: C

Explanation:

The correct answer is **C. Delineate what data will be used for testing and what will be used for training the model.**

Here's the justification:

Before a model can be trained and evaluated, it is crucial to establish a clear separation of data into training and testing sets. This is a fundamental step in any machine learning project.

Training Data: This dataset is used to teach the model the underlying patterns and relationships in the data, enabling it to learn how to make accurate predictions.

Testing Data: This separate dataset is used to assess the model's generalization performance on unseen data, preventing overfitting (where the model performs well on the training data but poorly on new data).

Option A (Running predictions) is premature without evaluating model performance. Option B (Monitoring performance) is necessary after the model is deployed. Option D (Testing and evaluation) is the step following data delineation. The split into training and testing datasets precedes both testing and evaluation, and therefore comes before either of them. It ensures that the model is validated on data it hasn't seen during training.

This process of delineating data promotes robust model development by:

1. **Preventing Overfitting:** by testing with data not used for training.
2. **Ensuring Generalization:** ensuring the model can perform on new data it hasn't encountered.
3. **Providing an Unbiased Performance Evaluation:** the testing dataset provides a true gauge of model performance.

This step is fundamental to the iterative process of model development and aligns with established machine learning best practices.

Authoritative Links:

Google Cloud AI Platform Documentation: <https://cloud.google.com/ai-platform/docs> - Review sections on data preparation and model training.

Machine Learning Crash Course by Google: <https://developers.google.com/machine-learning/crash-course/> - Look into sections on data splitting and model validation.

Stanford Machine Learning Course: <https://www.coursera.org/learn/machine-learning> - Review lessons on model selection and evaluation.

Question: 246

CertyIQ

You have one BigQuery dataset which includes customers' street addresses. You want to retrieve all occurrences of street addresses from the dataset. What should you do?

- A. Write a SQL query in BigQuery by using REGEXP_CONTAINS on all tables in your dataset to find rows where the word "street" appears.
- B. Create a deep inspection job on each table in your dataset with Cloud Data Loss Prevention and create an

inspection template that includes the STREET_ADDRESS infoType.

C.Create a discovery scan configuration on your organization with Cloud Data Loss Prevention and create an inspection template that includes the STREET_ADDRESS infoType.

D.Create a de-identification job in Cloud Data Loss Prevention and use the masking transformation.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct approach:

Option B, using Cloud Data Loss Prevention (DLP) with an inspection job and the STREET_ADDRESS infoType, is the most accurate and appropriate method for identifying street addresses within a BigQuery dataset. DLP is specifically designed to detect and protect sensitive data, offering pre-built infoTypes like STREET_ADDRESS that employ advanced pattern matching and context analysis to identify addresses accurately, beyond simple keyword searches. Options A, using REGEXP_CONTAINS to search for the word "street", would be insufficient and prone to false positives, as the word could exist in many contexts unrelated to actual street addresses. Option C, creating a discovery scan, is intended for discovering data across an entire organization and not for extracting specific values from a specific dataset. Option D, a de-identification job, is used for masking or transforming data, not for identification.

DLP allows for configurable inspection templates, allowing us to focus specifically on street addresses. By running an inspection job against each table in your BigQuery dataset, you can reliably locate rows containing street addresses using the power of the infoType. This method is far more robust than relying on a simple REGEXP_CONTAINS query that only identifies instances of the word "street," ignoring street names and address components in the result. DLP understands address formats across various regions and is not just doing string matching. DLP leverages the power of Google's natural language processing. This approach allows a more secure and accurate way to find street addresses without needing to write regular expressions manually.

In summary, Option B utilizes the right tool for the job. DLP is made to find sensitive data using specific info types.<https://cloud.google.com/dlp/docs><https://cloud.google.com/dlp/docs/infotypes-reference><https://cloud.google.com/dlp/docs/inspecting-storage>

Question: 247

CertyIQ

Your company operates in three domains: airlines, hotels, and ride-hailing services. Each domain has two teams: analytics and data science, which create data assets in BigQuery with the help of a central data platform team. However, as each domain is evolving rapidly, the central data platform team is becoming a bottleneck. This is causing delays in deriving insights from data, and resulting in stale data when pipelines are not kept up to date. You need to design a data mesh architecture by using Dataplex to eliminate the bottleneck. What should you do?

- A.1. Create one lake for each team. Inside each lake, create one zone for each domain.
2. Attach each of the BigQuery datasets created by the individual teams as assets to the respective zone.
3. Have the central data platform team manage all zones' data assets.
- B.1. Create one lake for each team. Inside each lake, create one zone for each domain.
2. Attach each of the BigQuery datasets created by the individual teams as assets to the respective zone.
3. Direct each domain to manage their own zone's data assets.
- C.1. Create one lake for each domain. Inside each lake, create one zone for each team.
2. Attach each of the BigQuery datasets created by the individual teams as assets to the respective zone.
3. Direct each domain to manage their own lake's data assets.
- D.1. Create one lake for each domain. Inside each lake, create one zone for each team.
2. Attach each of the BigQuery datasets created by the individual teams as assets to the respective zone.
3. Have the central data platform team manage all lakes' data assets.

Answer: C

Explanation:

The correct answer is **C**. Here's a detailed justification:

Option C implements a data mesh architecture effectively by aligning data ownership and management with the business domains. Creating one lake per domain (airlines, hotels, ride-hailing) directly addresses the problem of the central data platform team becoming a bottleneck. Each domain is then responsible for its own data within its lake, including its zones which are structured by team (analytics, data science). This fosters decentralization and agility, as each domain can manage its data assets independently and respond to its specific needs without waiting on a central team. This ownership shift enables faster iteration and reduces the risk of data becoming stale. Dataplex's role here is to provide the platform for this distributed management, enabling each domain to manage its BigQuery datasets as assets within their own zone. This aligns with the core principles of a data mesh, emphasizing domain ownership, data as a product, self-serve data infrastructure, and federated computational governance. Option A and B incorrectly structure the lakes by teams, which defeats the purpose of a data mesh which is aligning data management with the business domains. Option D returns the responsibility of managing data assets to the central platform team, which introduces the same bottleneck it is trying to avoid.

Further research:

Data Mesh Principles: <https://martinfowler.com/articles/data-mesh-principles.html> This article offers the original conceptualization of the data mesh architecture.

Dataplex Documentation: <https://cloud.google.com/dataplex/docs> This official Google Cloud documentation details how Dataplex works.

Dataplex Data Mesh Implementation: <https://cloud.google.com/blog/products/data-analytics/implement-data-mesh-on-google-cloud-with-dataplex> This Google Cloud blog provides insights into how Dataplex can be used to implement a data mesh.

Question: 248

CertyIQ

dataset.inventory_vm sample records:

Row	id	name	components.name	components.qty
1	vm02781	d-jp-kfk-02-02	vcpu	2
			memory	8
			boot_disk	10
			disk_1	50
2	vm11490	i-jp-kfk-02-07	vcpu	16
			memory	64
			boot_disk	10
			disk_1	200
3	vm18130	i-jp-kfk-02-08	vcpu	8
			memory	8
			boot_disk	10

You have an inventory of VM data stored in the BigQuery table. You want to prepare the data for regular reporting in the most cost-effective way. You need to exclude VM rows with fewer than 8 vCPU in your report. What should you do?

- A. Create a view with a filter to drop rows with fewer than 8 vCPU, and use the UNNEST operator.
- B. Create a materialized view with a filter to drop rows with fewer than 8 vCPU, and use the WITH common table expression.
- C. Create a view with a filter to drop rows with fewer than 8 vCPU, and use the WITH common table expression.
- D. Use Dataflow to batch process and write the result to another BigQuery table.

Answer: A

Explanation:

A. Create a view with a filter to drop rows with fewer than 8 vCPU, and use the UNNEST operator.

Question: 249

CertyIQ

Your team is building a data lake platform on Google Cloud. As a part of the data foundation design, you are planning to store all the raw data in Cloud Storage. You are expecting to ingest approximately 25 GB of data a day and your billing department is worried about the increasing cost of storing old data. The current business requirements are:

- The old data can be deleted anytime.
- There is no predefined access pattern of the old data.
- The old data should be available instantly when accessed.
- There should not be any charges for data retrieval.

What should you do to optimize for cost?

- A. Create the bucket with the Autoclass storage class feature.
- B. Create an Object Lifecycle Management policy to modify the storage class for data older than 30 days to nearline, 90 days to coldline, and 365 days to archive storage class. Delete old data as needed.
- C. Create an Object Lifecycle Management policy to modify the storage class for data older than 30 days to coldline, 90 days to nearline, and 365 days to archive storage class. Delete old data as needed.
- D. Create an Object Lifecycle Management policy to modify the storage class for data older than 30 days to nearline, 45 days to coldline, and 60 days to archive storage class. Delete old data as needed.

Answer: A

Explanation:

The correct answer is **A. Create the bucket with the Autoclass storage class feature.**

Here's why:

The core requirement is cost optimization for infrequently accessed old data while maintaining instant availability and avoiding retrieval charges. Autoclass elegantly addresses this need. Autoclass automatically transitions objects to more cost-effective storage classes based on access patterns. It intelligently analyzes how often data is accessed and moves it between Standard, Nearline, Coldline, and Archive tiers without manual intervention. Specifically, it will initially store data as Standard, which enables instant access with the highest cost, then eventually move it to less expensive options once it's deemed inactive, all while ensuring data is instantly available when accessed. It removes the need for manual lifecycle management and complex rules.

Options B, C, and D require manual creation of lifecycle policies based on static age thresholds. While

effective for predictable access patterns, they lack the adaptability of Autoclass. These options require careful planning based on presumed access patterns and will still have a delayed transition to cost-optimized storage if accessed. They also introduce complexity of defining and managing these policies. Furthermore, they do not meet the business requirement of no data retrieval charges, as they still apply for data stored in nearline, coldline or archive. Since we can delete old data anytime and don't have predefined access patterns, Autoclass is the most suitable solution here.

Therefore, Autoclass offers the most streamlined, hands-off, and cost-effective solution by aligning to the requirements of low cost while still offering instant access, which is only offered by the Standard tier. By dynamically moving data to appropriate tiers, it guarantees optimized storage costs without compromising availability.

Authoritative Links:

Google Cloud Storage Autoclass: <https://cloud.google.com/storage/docs/autoclass>

Google Cloud Storage Storage Classes: <https://cloud.google.com/storage/docs/storage-classes>

Google Cloud Storage Object Lifecycle Management: <https://cloud.google.com/storage/docs/lifecycle>

Question: 250

CertyIQ

Your company's data platform ingests CSV file dumps of booking and user profile data from upstream sources into Cloud Storage. The data analyst team wants to join these datasets on the email field available in both the datasets to perform analysis. However, personally identifiable information (PII) should not be accessible to the analysts. You need to de-identify the email field in both the datasets before loading them into BigQuery for analysts. What should you do?

- A.1. Create a pipeline to de-identify the email field by using recordTransformations in Cloud Data Loss Prevention (Cloud DLP) with masking as the de-identification transformations type.
- 2. Load the booking and user profile data into a BigQuery table.
- B.1. Create a pipeline to de-identify the email field by using recordTransformations in Cloud DLP with format-preserving encryption with FFX as the de-identification transformation type.
- 2. Load the booking and user profile data into a BigQuery table.
- C.1. Load the CSV files from Cloud Storage into a BigQuery table, and enable dynamic data masking.
- 2. Create a policy tag with the email mask as the data masking rule.
- 3. Assign the policy to the email field in both tables. A
- 4. Assign the Identity and Access Management bigquerydatapolicy.maskedReader role for the BigQuery tables to the analysts.
- D.1. Load the CSV files from Cloud Storage into a BigQuery table, and enable dynamic data masking.
- 2. Create a policy tag with the default masking value as the data masking rule.
- 3. Assign the policy to the email field in both tables.
- 4. Assign the Identity and Access Management bigquerydatapolicy.maskedReader role for the BigQuery tables to the analysts

Answer: B

Explanation:

The correct answer is B because it utilizes format-preserving encryption, which is crucial for maintaining data utility while de-identifying sensitive information. Here's a detailed justification:

Option B leverages Cloud Data Loss Prevention (DLP) with format-preserving encryption using FFX (Format-preserving, Feistel-based encryption). This method is suitable because it replaces the original email addresses with encrypted values that retain the same format (e.g., length and character types). This ensures the analysts can still perform joins on the de-identified email fields without exposing the actual PII. The data remains consistent and functional for analytical purposes after de-identification.

Option A, using simple masking, would replace email addresses with a generic string or a fixed pattern. While

effective for de-identification, this method prevents joins on the email field because the masked values become meaningless.

Option C and D propose using BigQuery's dynamic data masking. While data masking is a valid de-identification technique, loading the data into BigQuery before de-identification poses a security risk. Data analysts could potentially access the raw PII during the loading process or before the masking policy is fully applied, violating compliance and security best practices. Moreover, dynamic data masking typically applies a standard replacement (e.g., NULL or a fixed string), which would prevent joining the two tables based on the de-identified email. Dynamic data masking is generally less performant than transforming the data once as in Option B.

Therefore, de-identifying the data before loading it into BigQuery using format-preserving encryption (option B) is the most secure and functional solution. It minimizes exposure of PII, preserves data relationships for analysis, and addresses the core requirements outlined in the prompt.
<https://cloud.google.com/dlp/docs/transformations-concept>
<https://cloud.google.com/bigquery/docs/masked-data>

Question: 251

CertyIQ

You have important legal hold documents in a Cloud Storage bucket. You need to ensure that these documents are not deleted or modified. What should you do?

- A. Set a retention policy. Lock the retention policy.
- B. Set a retention policy. Set the default storage class to Archive for long-term digital preservation.
- C. Enable the Object Versioning feature. Add a lifecycle rule.
- D. Enable the Object Versioning feature. Create a copy in a bucket in a different region.

Answer: A

Explanation:

Justification for Answer A:

The correct answer is **A. Set a retention policy. Lock the retention policy.** This approach directly addresses the requirement of preventing deletion or modification of legal hold documents within a Cloud Storage bucket.

Retention policies in Cloud Storage, as documented [here](#), are designed to enforce data retention rules. By setting a retention policy, you specify a duration during which objects in the bucket cannot be deleted or modified. Once a retention policy is locked, it becomes immutable; meaning it cannot be shortened or removed. This effectively provides a "write once, read many" (WORM) mechanism, crucial for legal hold scenarios, ensuring compliance and integrity of the data.

Option B, while including a retention policy, incorrectly suggests archiving the files. Archiving might reduce storage costs but does not directly contribute to the primary requirement of preventing modifications. Furthermore, moving files to Archive storage class is reversible.

Options C and D suggest using Object Versioning. While useful for recovering accidentally deleted or overwritten objects, as explained [here](#), it doesn't inherently prevent deletion once the number of versions exceeds the retention limit or a version is explicitly deleted. Object versioning allows multiple versions to co-exist, but it does not preclude modification of newer versions or eventual deletion of all versions. Creating a copy in a different region (option D) offers redundancy and disaster recovery capabilities, but it doesn't prevent alteration of the legal hold documents in either location.

Therefore, a locked retention policy directly and completely fulfills the requirement of safeguarding legal hold documents against deletion and modification, making option A the only correct answer.

Question: 252

CertyIQ

You are designing a data warehouse in BigQuery to analyze sales data for a telecommunication service provider. You need to create a data model for customers, products, and subscriptions. All customers, products, and subscriptions can be updated monthly, but you must maintain a historical record of all data. You plan to use the visualization layer for current and historical reporting. You need to ensure that the data model is simple, easy-to-use, and cost-effective. What should you do?

- A. Create a normalized model with tables for each entity. Use snapshots before updates to track historical data.
- B. Create a normalized model with tables for each entity. Keep all input files in a Cloud Storage bucket to track historical data.
- C. Create a denormalized model with nested and repeated fields. Update the table and use snapshots to track historical data.
- D. Create a denormalized, append-only model with nested and repeated fields. Use the ingestion timestamp to track historical data.

Answer: D

Explanation:

The optimal solution is **D. Create a denormalized, append-only model with nested and repeated fields. Use the ingestion timestamp to track historical data.**

Here's why:

Denormalization for Simplicity & Cost: Denormalizing by using nested and repeated fields within a single table reduces the need for complex joins, simplifying queries and improving performance, especially with BigQuery's columnar storage. This also minimizes costs associated with data scanning and processing.

Append-Only Model for Historical Tracking: An append-only model means that instead of updating existing rows, new rows are added with each monthly update. This inherently maintains a historical record of all changes, simplifying historical analysis. This is particularly cost-effective and efficient in BigQuery compared to frequent updates or snapshot creation.

Ingestion Timestamp for Versioning: Using the ingestion timestamp as a proxy for the update date allows for easy filtering and versioning of data. BigQuery automatically adds this timestamp, simplifying the tracking process and eliminating the need for maintaining separate timestamp columns or versioning mechanisms. It's a simple and effective approach for creating point-in-time views of data.

Suitability for Reporting: This denormalized structure is well-suited for visualization and reporting, allowing users to easily access current and historical data without the need for complex SQL queries. The model remains user-friendly, especially when reporting is primarily the focus.

Efficiency in BigQuery: BigQuery's optimized columnar storage and query engine work particularly well with denormalized data, leading to faster queries and lower costs. It minimizes data movement and improves query processing efficiency.

Options A and B, utilizing normalized models, require joins and either snapshots or Cloud Storage data to manage history. These approaches introduce complexity and increase operational overhead, potentially impacting performance and cost. Option C is not fully append-only, and introduces a more complex system for history management.

Authoritative Links:

BigQuery Data Modeling: <https://cloud.google.com/bigquery/docs/data-modeling>

BigQuery Schema Design: <https://cloud.google.com/bigquery/docs/schema-design>

BigQuery Ingestion Time: https://cloud.google.com/bigquery/docs/reference/standard-sql/functions-and-operators#_ingestion_time

Denormalization in BigQuery: <https://cloud.google.com/blog/topics/developers-practitioners/optimizing-bigquery-performance-data-denormalization>

Question: 253

CertyIQ

You are deploying a batch pipeline in Dataflow. This pipeline reads data from Cloud Storage, transforms the data, and then writes the data into BigQuery. The security team has enabled an organizational constraint in Google Cloud, requiring all Compute Engine instances to use only internal IP addresses and no external IP addresses. What should you do?

- A. Ensure that your workers have network tags to access Cloud Storage and BigQuery. Use Dataflow with only internal IP addresses.
- B. Ensure that the firewall rules allow access to Cloud Storage and BigQuery. Use Dataflow with only internal IPs.
- C. Create a VPC Service Controls perimeter that contains the VPC network and add Dataflow, Cloud Storage, and BigQuery as allowed services in the perimeter. Use Dataflow with only internal IP addresses.
- D. Ensure that Private Google Access is enabled in the subnetwork. Use Dataflow with only internal IP addresses.

Answer: D

Explanation:

The correct answer is **D. Ensure that Private Google Access is enabled in the subnetwork. Use Dataflow with only internal IP addresses.**

Here's the justification:

The core issue is enabling Dataflow workers, which only have internal IPs, to access Google Cloud services like Cloud Storage and BigQuery, without violating the organizational policy against external IPs.

Private Google Access allows Compute Engine instances, with only internal IP addresses, to reach Google APIs and services (like Cloud Storage and BigQuery) using Google's private network. This means traffic doesn't go through the public internet, adhering to the security constraint and avoiding external IP addresses on the workers.

Option A is insufficient. While network tags control firewall rules and can help with network segmentation, they don't inherently enable internal-IP-only instances to access Google services.

Option B is also incorrect. Firewall rules primarily govern how external traffic flows in and out of the VPC, it doesn't facilitate internal instances reaching Google services.

Option C introduces VPC Service Controls, which focuses on data exfiltration and broader access control based on resource perimeters. While important for security, it's not the correct solution for the immediate problem of internal-IP-only workers needing to reach Google APIs. VPC Service Controls don't replace the need for enabling Private Google Access in this specific scenario.

Private Google Access is the fundamental requirement to bridge the gap between workers with only internal IPs and the necessary Google APIs and services. It's designed precisely for this scenario. Other settings help to further refine the setup.

Authoritative Links:

Private Google Access: <https://cloud.google.com/vpc/docs/private-access-options#private-google-access>

Dataflow networking: <https://cloud.google.com/dataflow/docs/guides/network-options>

In essence, Private Google Access is the essential mechanism for internal-IP-only Dataflow workers to interact with other Google Cloud services.

Question: 254

CertyIQ

You are running a Dataflow streaming pipeline, with Streaming Engine and Horizontal Autoscaling enabled. You have set the maximum number of workers to 1000. The input of your pipeline is Pub/Sub messages with notifications from Cloud Storage. One of the pipeline transforms reads CSV files and emits an element for every CSV line. The job performance is low, the pipeline is using only 10 workers, and you notice that the autoscaler is not spinning up additional workers. What should you do to improve performance?

- A.Enable Vertical Autoscaling to let the pipeline use larger workers.
- B.Change the pipeline code, and introduce a Reshuffle step to prevent fusion.
- C.Update the job to increase the maximum number of workers.
- D.Use Dataflow Prime, and enable Right Fitting to increase the worker resources.

Answer: B

Explanation:

The correct answer is **B. Change the pipeline code, and introduce a Reshuffle step to prevent fusion.**

Here's the justification:

The issue stems from Dataflow's optimization techniques, specifically fusion. Dataflow automatically merges (fuses) operations into single worker threads to optimize processing. While this is generally beneficial, in scenarios with skewed data or resource-intensive transforms, it can become a bottleneck. In this case, reading CSV files is likely a computationally heavy step, and the pipeline is only using 10 workers. Fusion is probably grouping the Pub/Sub ingestion and CSV parsing into single steps.

The autoscaler needs sufficient workload backlog to justify adding more workers. If operations are fused into a single worker, they might be slow but not significantly backlogged. The system thus may not see a need for more workers. Introducing a Reshuffle operation after reading the CSV files effectively breaks fusion. A Reshuffle operation is a forced re-partitioning of the data, which adds network i/o, but creates an explicit boundary. This operation forces the data to go through a shuffle phase, which allows Dataflow to distribute the work for subsequent operations across more workers, thus increasing parallelism and worker utilization. By breaking fusion, Dataflow can see the downstream workload bottleneck and spin up additional workers, based on the actual processing demand. The autoscaler, seeing increased utilization, will begin creating workers, leading to improved throughput. Options A, C, and D do not directly address the core issue of fused operations that limit parallelism. Vertical autoscaling (A) might help if the existing workers are resource-constrained, but that is not the primary problem here. Option C, while seeming logical, is not the bottleneck because the autoscaler is not adding workers based on current utilization, not the upper limit. Similarly, Dataflow Prime (D) might be beneficial, but it addresses resource allocation, not the root cause of fusion limitation.

Authoritative Links:

[Dataflow Execution Details: Fusion](#)

[Dataflow Reshuffle](#)

[Dataflow Autoscaling](#)

Question: 255

CertyIQ

You have an Oracle database deployed in a VM as part of a Virtual Private Cloud (VPC) network. You want to replicate and continuously synchronize 50 tables to BigQuery. You want to minimize the need to manage infrastructure. What should you do?

- A. Deploy Apache Kafka in the same VPC network, use Kafka Connect Oracle Change Data Capture (CDC), and Dataflow to stream the Kafka topic to BigQuery.
- B. Create a Pub/Sub subscription to write to BigQuery directly. Deploy the Debezium Oracle connector to capture changes in the Oracle database, and sink to the Pub/Sub topic.
- C. Deploy Apache Kafka in the same VPC network, use Kafka Connect Oracle change data capture (CDC), and the Kafka Connect Google BigQuery Sink Connector.
- D. Create a Datastream service from Oracle to BigQuery, use a private connectivity configuration to the same VPC network, and a connection profile to BigQuery.

Answer: D

Explanation:

Option D is the most suitable solution because it leverages Google Cloud's fully managed services, minimizing the operational overhead of infrastructure management. Datastream is a serverless change data capture (CDC) and replication service explicitly designed for databases like Oracle, making it a natural fit. It allows continuous synchronization of data from heterogeneous sources to BigQuery with minimal code. The private connectivity configuration ensures secure and direct data transfer within the VPC, avoiding the public internet. A connection profile simplifies the connection to BigQuery. In contrast, Option A, while functional, introduces the need to manage Apache Kafka and Dataflow, increasing complexity. Option B requires managing Debezium, a Kafka-based solution, and Pub/Sub doesn't have direct CDC capability from Oracle, needing extra layers of engineering. Option C requires managing Kafka and its connectors. Datastream is specifically built for this use case providing a turnkey managed solution.

Authoritative Links:

Datastream Overview: <https://cloud.google.com/datastream/docs/overview>

Datastream for Oracle: <https://cloud.google.com/datastream/docs/sources/oracle>

Datastream for BigQuery: <https://cloud.google.com/datastream/docs/targets/bigquery>

Private Connectivity with Datastream:

<https://cloud.google.com/datastream/docs/concepts/connectivity/private-connectivity>

Question: 256

CertyIQ

You are deploying an Apache Airflow directed acyclic graph (DAG) in a Cloud Composer 2 instance. You have incoming files in a Cloud Storage bucket that the DAG processes, one file at a time. The Cloud Composer instance is deployed in a subnetwork with no Internet access. Instead of running the DAG based on a schedule, you want to run the DAG in a reactive way every time a new file is received. What should you do?

- A.1. Enable Private Google Access in the subnetwork, and set up Cloud Storage notifications to a Pub/Sub topic.
2. Create a push subscription that points to the web server URL.
- B.1. Enable the Cloud Composer API, and set up Cloud Storage notifications to trigger a Cloud Function.
2. Write a Cloud Function instance to call the DAG by using the Cloud Composer API and the web server URL.
3. Use VPC Serverless Access to reach the web server URL.
- C.1. Enable the Airflow REST API, and set up Cloud Storage notifications to trigger a Cloud Function instance.
2. Create a Private Service Connect (PSC) endpoint.
3. Write a Cloud Function that connects to the Cloud Composer cluster through the PSC endpoint.
- D.1. Enable the Airflow REST API, and set up Cloud Storage notifications to trigger a Cloud Function instance.
2. Write a Cloud Function instance to call the DAG by using the Airflow REST API and the web server URL.
3. Use VPC Serverless Access to reach the web server URL.

Answer: C

Explanation:

Okay, let's break down why option C is the correct approach for triggering an Airflow DAG in Cloud Composer reactively based on Cloud Storage file arrival, within a private network.

Here's the detailed justification:

Why Option C is correct:

1. **Reactive Triggering:** The core requirement is to trigger the DAG only when a new file appears in Cloud Storage. Cloud Storage notifications to Pub/Sub are the standard method for this kind of event-driven architecture. A Cloud Function, listening to Pub/Sub, then acts as the intermediary to invoke the DAG.
2. **Airflow REST API:** Option C correctly identifies that direct access to the Airflow web server is problematic in a private network. The Airflow REST API is designed for programmatic interactions with Airflow, including triggering DAG runs. Enabling this API is thus essential for this setup.
3. **Private Service Connect (PSC):** Since the Composer environment lacks internet access, directly connecting to the Airflow REST API using the web server URL is not feasible. Private Service Connect (PSC) allows a secure and private connection from the Cloud Function to the Airflow API within the network, without traversing the public internet. This maintains the desired private network security.
4. **Cloud Function Logic:** The Cloud Function then uses the PSC connection to authenticate with the Airflow REST API and trigger the desired DAG based on the received Cloud Storage notification.

Why other options are incorrect:

Option A: Using a push subscription is unsuitable for triggering DAGs because it involves exposing the Airflow web server URL publicly. Direct access to the web server is incompatible with the described private network and security constraints.

Option B: While enabling the Cloud Composer API is valid for general cluster management, it is not the correct method to trigger a DAG using HTTP. Additionally, it incorrectly suggests using the web server URL, which will not be accessible in this scenario. Furthermore, using Serverless VPC Access here is not as direct and secure as using PSC.

Option D: While this also mentions the Airflow REST API, it fails to address the critical security issue regarding the restricted network. It uses VPC Serverless Access, similar to Option B, which would not be as secure or direct as a PSC endpoint in this situation, and it still attempts to access the Airflow web server URL.

Key Concepts:

Cloud Storage Notifications: <https://cloud.google.com/storage/docs/pubsub-notifications> - The foundation of the reactive approach, allowing a Pub/Sub topic to be notified upon file changes.

Cloud Functions: <https://cloud.google.com/functions/docs> - Serverless compute platform to act upon events, in this case, triggering the DAG.

Airflow REST API: <https://airflow.apache.org/docs/apache-airflow/stable/stable-rest-api-ref.html> - A way to programmatically interact with Airflow.

Private Service Connect (PSC): <https://cloud.google.com/vpc/docs/private-service-connect> - A way to securely and privately access services in a private network.

In summary, Option C provides the most secure and effective solution because it leverages Cloud Storage notifications, the Airflow REST API for programmatic access, and Private Service Connect (PSC) to ensure secure communication within the isolated network, all without making the webserver publicly accessible.

Question: 257

You are planning to use Cloud Storage as part of your data lake solution. The Cloud Storage bucket will contain

objects ingested from external systems. Each object will be ingested once, and the access patterns of individual objects will be random. You want to minimize the cost of storing and retrieving these objects. You want to ensure that any cost optimization efforts are transparent to the users and applications. What should you do?

- A. Create a Cloud Storage bucket with Autoclass enabled.
- B. Create a Cloud Storage bucket with an Object Lifecycle Management policy to transition objects from Standard to Coldline storage class if an object age reaches 30 days.
- C. Create a Cloud Storage bucket with an Object Lifecycle Management policy to transition objects from Standard to Coldline storage class if an object is not live.
- D. Create two Cloud Storage buckets. Use the Standard storage class for the first bucket, and use the Coldline storage class for the second bucket. Migrate objects from the first bucket to the second bucket after 30 days.

Answer: A

Explanation:

The optimal solution is **A. Create a Cloud Storage bucket with Autoclass enabled.**

Autoclass is designed precisely for scenarios where access patterns are unpredictable, and cost optimization needs to be transparent. It automatically manages storage classes (Standard, Nearline, Coldline, Archive) based on object access frequency, moving objects to less expensive tiers when they're accessed less often, and back to more performant tiers when needed. This aligns perfectly with the requirements stated: random object access, cost minimization, and transparency to users and applications. Unlike manual lifecycle policies (options B and C), Autoclass is dynamic and doesn't rely on fixed rules like object age or liveliness. Option D requires manual management and is not transparent, hindering the desired ease of use. Autoclass handles tiered storage automatically, ensuring that cost optimization happens continuously and effortlessly, and eliminates the need for developers to make conscious storage tier decisions. By analyzing access patterns dynamically and adjusting storage tiers without manual intervention, Autoclass reduces management overhead and delivers the most cost-effective storage solution for the given scenario, all while optimizing the user experience by ensuring fast access when required. It is crucial for achieving the desired outcome efficiently.

Authoritative links:

Cloud Storage Autoclass: <https://cloud.google.com/storage/docs/autoclass>

Cloud Storage Storage Classes: <https://cloud.google.com/storage/docs/storage-classes>

Object Lifecycle Management: <https://cloud.google.com/storage/docs/lifecycle>

Question: 258

CertyIQ

You have several different file type data sources, such as Apache Parquet and CSV. You want to store the data in Cloud Storage. You need to set up an object sink for your data that allows you to use your own encryption keys. You want to use a GUI-based solution. What should you do?

- A. Use Storage Transfer Service to move files into Cloud Storage.
- B. Use Cloud Data Fusion to move files into Cloud Storage.
- C. Use Dataflow to move files into Cloud Storage.
- D. Use BigQuery Data Transfer Service to move files into BigQuery.

Answer: B

Explanation:

Here's a detailed justification for why the answer is B (Cloud Data Fusion) and why the other options are less suitable:

Justification for Cloud Data Fusion (Option B):

Cloud Data Fusion is an integration platform as a service (iPaaS) that's built upon Apache Beam, designed for building and managing data pipelines through a GUI. It's an ideal solution for scenarios where you need to ingest data from various sources, transform it, and load it into a destination like Cloud Storage, especially when dealing with different file types like Parquet and CSV. Data Fusion allows you to visually create pipelines without extensive coding using its pre-built connectors and transformations. Specifically, Data Fusion supports using Customer-Managed Encryption Keys (CMEK) for data at rest in Cloud Storage, satisfying the requirement of using your own encryption keys. Its GUI-based interface aligns with the requirement of using a GUI-based solution. It also provides built-in support for many file formats like CSV and Parquet.

Why other options are incorrect:

Option A (Storage Transfer Service): Storage Transfer Service is mainly for moving large amounts of data between online storage systems, like from AWS S3 to Cloud Storage or between Cloud Storage buckets. While it can move files to Cloud Storage, it primarily focuses on data transfer and does not provide the extensive data transformation and pipeline management capabilities needed when dealing with diverse file formats and the necessity to manage encryption keys through a graphical interface. It's less suitable for complex ingestion processes.

Option C (Dataflow): Dataflow can be used to ingest data into Cloud Storage from diverse sources and apply transformations. However, Dataflow primarily relies on coding (e.g., using Python or Java with the Apache Beam SDK) and isn't primarily a GUI-based solution. While you can use Dataflow's UI for monitoring jobs, the pipeline creation process is code-centric, thus not fulfilling the GUI requirement.

Option D (BigQuery Data Transfer Service): BigQuery Data Transfer Service is specifically designed for transferring data into BigQuery. It's not for moving data into Cloud Storage. Thus, it's not applicable to the specified problem.

In summary: Cloud Data Fusion is the best fit because it provides a GUI-based environment for creating data pipelines, supports various file formats (Parquet, CSV), and allows the use of Customer-Managed Encryption Keys (CMEK) for data in Cloud Storage.

Authoritative Links:

Cloud Data Fusion Documentation: <https://cloud.google.com/data-fusion/docs>

Cloud Storage Encryption Keys (CMEK): <https://cloud.google.com/storage/docs/encryption/customer-managed-keys>

Storage Transfer Service Documentation: <https://cloud.google.com/storage-transfer-service/docs>

Dataflow Documentation: <https://cloud.google.com/dataflow/docs>

BigQuery Data Transfer Service Documentation: <https://cloud.google.com/bigquery/docs/transfer-service-introduction>

Question: 259

CertyIQ

Your business users need a way to clean and prepare data before using the data for analysis. Your business users are less technically savvy and prefer to work with graphical user interfaces to define their transformations. After the data has been transformed, the business users want to perform their analysis directly in a spreadsheet. You need to recommend a solution that they can use. What should you do?

- A. Use Dataprep to clean the data, and write the results to BigQuery. Analyze the data by using Connected Sheets.
- B. Use Dataprep to clean the data, and write the results to BigQuery. Analyze the data by using Looker Studio.
- C. Use Dataflow to clean the data, and write the results to BigQuery. Analyze the data by using Connected

Sheets.

D. Use Dataflow to clean the data, and write the results to BigQuery. Analyze the data by using Looker Studio.

Answer: A

Explanation:

The correct answer is A because it best addresses the user's needs for a graphical interface for data transformation and subsequent spreadsheet analysis. Dataprep, a serverless data preparation tool from Google Cloud, offers a visual interface that allows less technical users to clean and transform data without writing code. This fulfills the requirement for ease of use. Writing the transformed data to BigQuery provides a scalable and cost-effective storage solution. Connected Sheets enables users to directly access and analyze BigQuery data within Google Sheets, meeting the need for spreadsheet-based analysis. Options C and D use Dataflow, which while powerful, requires more technical expertise, which contradicts the requirement for a user-friendly solution. Also, Looker Studio, offered in options B and D, while being a data visualization tool, does not fit the requested spreadsheet analysis method. Therefore, option A, utilizing Dataprep and Connected Sheets, aligns perfectly with the business user's requirements of visual data preparation and analysis via spreadsheets.

Further Reading:

Dataprep: <https://cloud.google.com/dataprep>

Connected Sheets: <https://support.google.com/docs/answer/9233340>

BigQuery: <https://cloud.google.com/bigquery>

Question: 260

CertyIQ

You have two projects where you run BigQuery jobs:

- One project runs production jobs that have strict completion time SLAs. These are high priority jobs that must have the required compute resources available when needed. These jobs generally never go below a 300 slot utilization, but occasionally spike up an additional 500 slots.
- The other project is for users to run ad-hoc analytical queries. This project generally never uses more than 200 slots at a time. You want these ad-hoc queries to be billed based on how much data users scan rather than by slot capacity.

You need to ensure that both projects have the appropriate compute resources available. What should you do?

A. Create a single Enterprise Edition reservation for both projects. Set a baseline of 300 slots. Enable autoscaling up to 700 slots.

B. Create two reservations, one for each of the projects. For the SLA project, use an Enterprise Edition with a baseline of 300 slots and enable autoscaling up to 500 slots. For the ad-hoc project, configure on-demand billing.

C. Create two Enterprise Edition reservations, one for each of the projects. For the SLA project, set a baseline of 300 slots and enable autoscaling up to 500 slots. For the ad-hoc project, set a reservation baseline of 0 slots and set the ignore idle slots flag to False.

D. Create two Enterprise Edition reservations, one for each of the projects. For the SLA project, set a baseline of 800 slots. For the ad-hoc project, enable autoscaling up to 200 slots.

Answer: B

Explanation:

The correct answer is **B**. Here's why:

Understanding BigQuery Reservations: BigQuery reservations allow you to purchase guaranteed compute capacity (slots) for your projects. This is crucial for workloads with strict SLAs, as it prevents resource contention and ensures predictable performance. You can also configure on-demand billing for projects that

don't require reserved capacity.

Option B Justification: Option B addresses the specific needs of both projects accurately. It proposes two separate reservations:

SLA Project: An Enterprise Edition reservation with a 300-slot baseline guarantees the necessary minimum compute power for the production jobs. Enabling autoscaling up to 500 slots allows the project to handle the occasional spikes in resource demands without manual intervention. This ensures the production jobs meet their SLAs.

Ad-hoc Project: Configuring on-demand billing is ideal for analytical queries that have unpredictable and variable resource usage patterns. On-demand billing bases costs on the volume of data processed, instead of paying for unused reserved capacity. This satisfies the requirement to bill users for data scanned, rather than slot capacity.

Why other options are incorrect:

Option A: Using a single reservation would not allow for on-demand billing for the ad-hoc project, forcing users to be billed for a share of the total reserved slots, even if they are not using them.

Option C: Setting the ad-hoc project's reservation to a baseline of 0 with ignore idle slots as false will result in charges for any capacity, even when not actively using resources. This option won't align with the goal of on-demand billing.

Option D: Setting the SLA project reservation to a baseline of 800 is excessive. It's generally best practice to only provision the capacity needed, and allow for autoscaling, which option B achieves. For the ad-hoc project, it would force the project to be billed for at least 200 slots even though they might use less, which is contradictory to the goal of having an on-demand pricing model.

In summary, option B effectively provides predictable performance for the production jobs while ensuring the ad-hoc project is billed according to usage, making it the most suitable solution.

Authoritative Links:

BigQuery Reservations: <https://cloud.google.com/bigquery/docs/reservations>

BigQuery Pricing: <https://cloud.google.com/bigquery/pricing>

Autoscaling in BigQuery Reservations: <https://cloud.google.com/bigquery/docs/reservations-autoscaling>

Question: 261

CertyIQ

You want to migrate your existing Teradata data warehouse to BigQuery. You want to move the historical data to BigQuery by using the most efficient method that requires the least amount of programming, but local storage space on your existing data warehouse is limited. What should you do?

- A. Use BigQuery Data Transfer Service by using the Java Database Connectivity (JDBC) driver with FastExport connection.
- B. Create a Teradata Parallel Transporter (TPT) export script to export the historical data, and import to BigQuery by using the bq command-line tool.
- C. Use BigQuery Data Transfer Service with the Teradata Parallel Transporter (TPT) tbuild utility.
- D. Create a script to export the historical data, and upload in batches to Cloud Storage. Set up a BigQuery Data Transfer Service instance from Cloud Storage to BigQuery.

Answer: A

Explanation:

The correct answer is A: "Use BigQuery Data Transfer Service by using the Java Database Connectivity (JDBC) driver with FastExport connection." Here's why:

BigQuery Data Transfer Service (DTS): This service automates and manages data loading from various sources into BigQuery. It's designed for efficiency and minimal coding effort.

JDBC Driver: Using a JDBC driver allows BigQuery DTS to connect directly to the Teradata data warehouse. JDBC is a standard Java API that facilitates database connectivity.

FastExport: Teradata's FastExport is a high-performance data extraction utility designed for efficient data unloading. Integrating it with the JDBC connection streamlines the data movement.

Efficiency and Minimal Programming: This combination offers a balanced approach, leveraging DTS's automation and FastExport's speed. It avoids the need for extensive scripting or manual data handling.

Limited Local Storage: Using DTS with direct connectivity (JDBC) minimizes the requirement for intermediate local storage. The data is transferred directly from Teradata to BigQuery, reducing the impact of local storage limitations. Options B and D both require temporary storage, which violates the requirements.

The other options are less suitable:

Option B (TPT Export Script + bq command-line tool): TPT is a valid method, but creating and managing TPT scripts requires more programming effort. It also might require local storage before uploading to BigQuery.

Option C (BigQuery DTS + Teradata Parallel Transporter (TPT) tbuild utility): While DTS can integrate with TPT, setting up the TPT tbuild utility can be more complex and might still involve local storage needs.

Option D (Export Script + Cloud Storage + BigQuery DTS from Cloud Storage): This involves writing a script to export the data, requiring more programming. It also introduces Cloud Storage as an intermediary, consuming storage space and adding an extra step.

Supporting Links:

BigQuery Data Transfer Service: <https://cloud.google.com/bigquery/docs/transfer-service>

Teradata FastExport: (search on teradata.com as their site structure is frequently changed)

JDBC: https://en.wikipedia.org/wiki/Java_Database_Connectivity

Question: 262

CertyIQ

You are on the data governance team and are implementing security requirements. You need to encrypt all your data in BigQuery by using an encryption key managed by your team. You must implement a mechanism to generate and store encryption material only on your on-premises hardware security module (HSM). You want to rely on Google managed solutions. What should you do?

- A. Create the encryption key in the on-premises HSM, and import it into a Cloud Key Management Service (Cloud KMS) key. Associate the created Cloud KMS key while creating the BigQuery resources.
- B. Create the encryption key in the on-premises HSM and link it to a Cloud External Key Manager (Cloud EKM) key. Associate the created Cloud KMS key while creating the BigQuery resources.
- C. Create the encryption key in the on-premises HSM, and import it into Cloud Key Management Service (Cloud HSM) key. Associate the created Cloud HSM key while creating the BigQuery resources.
- D. Create the encryption key in the on-premises HSM. Create BigQuery resources and encrypt data while ingesting them into BigQuery.

Answer: B

Explanation:

The correct answer is **B**. Here's a detailed justification:

The core requirement is to use an encryption key generated and stored exclusively within an on-premises Hardware Security Module (HSM), while leveraging Google's managed services. This necessitates using a mechanism that bridges the gap between the on-premises HSM and Google Cloud.

Option B correctly utilizes Cloud External Key Manager (Cloud EKM). Cloud EKM is specifically designed for scenarios where encryption keys are managed outside of Google's infrastructure, in this case, an on-premises

HSM. With Cloud EKM, the encryption key remains within the HSM, and only a reference to the key exists in Google Cloud. This allows BigQuery to leverage the key for encryption and decryption without ever having direct access to the key material. This fulfills the requirement of keeping the key material only in the on-premises HSM while enabling use with BigQuery via Google managed solutions.

Option A is incorrect because importing the key into Cloud KMS means the key material would exist within Google's infrastructure, which violates the requirement of keeping key material only within the on-premises HSM.

Option C is incorrect because Cloud HSM, although using HSMs, are Google-managed HSMs and do not satisfy the requirement to use an on-premises HSM. The keys would not remain only within the on-premises HSM.

Option D is incorrect because it does not leverage any Google managed solution for key management. While you could implement a system for on-prem key management and direct encryption, the question explicitly asks for Google managed solutions and this option lacks that.

In summary, Cloud EKM provides the necessary secure bridge to use keys held in an on-premises HSM with Google Cloud services like BigQuery, adhering to the stipulated security requirements and utilizing a Google managed solution.

Supporting Links:

Cloud EKM Documentation: <https://cloud.google.com/kms/docs/ekm>

BigQuery Encryption Options: <https://cloud.google.com/bigquery/docs/encryption-at-rest>

Question: 263

CertyIQ

You maintain ETL pipelines. You notice that a streaming pipeline running on Dataflow is taking a long time to process incoming data, which causes output delays. You also noticed that the pipeline graph was automatically optimized by Dataflow and merged into one step. You want to identify where the potential bottleneck is occurring. What should you do?

- A. Insert a Reshuffle operation after each processing step, and monitor the execution details in the Dataflow console.
- B. Insert output sinks after each key processing step, and observe the writing throughput of each block.
- C. Log debug information in each ParDo function, and analyze the logs at execution time.
- D. Verify that the Dataflow service accounts have appropriate permissions to write the processed data to the output sinks.

Answer: A

Explanation:

The correct answer is **A. Insert a Reshuffle operation after each processing step, and monitor the execution details in the Dataflow console.**

Here's why:

When Dataflow automatically optimizes a pipeline by merging steps, it can obscure the performance bottlenecks. Reshuffle operations force a grouping and repartitioning of data. By strategically inserting Reshuffles between logical steps, we prevent Dataflow from merging those steps into one. This allows us to monitor each step individually within the Dataflow console. The console provides detailed metrics on CPU utilization, memory usage, and element processing times for each step. These metrics help pinpoint the specific step causing the slowdown. Options B and C would not be ideal as Option B changes the pipeline structure by adding write sinks and Option C will create a lot of logs to analyze when the main issue is

optimization by the service. Option D is about permissions that should have been correctly set, it does not address the pipeline merging problem.

Reshuffle is a critical tool for performance debugging by preventing fusion. Dataflow's automatic optimization prioritizes efficiency but can hinder diagnosis of bottlenecks. By inserting a Reshuffle, we gain visibility into individual step performance, enabling us to identify the part of the pipeline where the slowness originates. This granular monitoring helps us identify where resources are insufficient or processing is inefficient. Once we've located the bottleneck, we can then focus on optimizing the specific step causing the problem.

Authoritative links for further research:

Reshuffle Operation: <https://beam.apache.org/documentation/programming-guide/#reshuffle> (Apache Beam documentation, which Dataflow is built upon)

Dataflow Monitoring Interface: <https://cloud.google.com/dataflow/docs/guides/using-monitoring-intf>

Dataflow Pipeline Optimization: <https://cloud.google.com/dataflow/docs/guides/optimizing-your-pipeline>

Question: 264

CertyIQ

You are running your BigQuery project in the on-demand billing model and are executing a change data capture (CDC) process that ingests data. The CDC process loads 1 GB of data every 10 minutes into a temporary table, and then performs a merge into a 10 TB target table. This process is very scan intensive and you want to explore options to enable a predictable cost model. You need to create a BigQuery reservation based on utilization information gathered from BigQuery Monitoring and apply the reservation to the CDC process. What should you do?

- A. Create a BigQuery reservation for the dataset.
- B. Create a BigQuery reservation for the job.
- C. Create a BigQuery reservation for the service account running the job.
- D. Create a BigQuery reservation for the project.

Answer: D

Explanation:

Here's a justification for choosing option D, creating a BigQuery reservation for the project:

BigQuery reservations provide a predictable cost model by allowing you to commit to a specific capacity (slots) rather than being charged based on on-demand query usage. The goal is to apply this reservation to the CDC process, ensuring its scan-intensive nature doesn't incur unpredictable costs.

Option D, creating a project-level reservation, is the correct approach. Reservations are primarily scoped at the project level. This means that any BigQuery jobs initiated within that project, regardless of the dataset or user account, can utilize the reserved slots if available. This aligns with the requirement of applying the reservation to the CDC process, which is a workload within the project.

Options A, B, and C are incorrect. Reservations do not directly apply to individual datasets or jobs. While a job's location (which dictates which reservation to use), and potentially a user account, might indirectly impact which reservation is ultimately utilized, the fundamental scope of a reservation is the project. Option C, a service account, is incorrect as you would need to modify the project and not a service account in order for the reservation to be picked up.

By creating a reservation at the project level, you effectively pool resources for any workloads in the project. This ensures that the CDC process, a key part of the project's functionality, is covered by the reservation and is no longer subject to the on-demand pricing model.

The usage of BigQuery Monitoring to determine how many slots to reserve is a vital step. This information

enables you to reserve enough slots to meet the CDC process's demands.

In summary, the project-level reservation model aligns with the structure of BigQuery reservations and is the most suitable method to apply predictable cost to all queries within a project.

Supporting Documentation:

BigQuery Reservations: <https://cloud.google.com/bigquery/docs/reservations-intro>

Working with Reservations: <https://cloud.google.com/bigquery/docs/reservations-workload>

BigQuery Monitoring: <https://cloud.google.com/bigquery/docs/monitoring>

Question: 265

CertyIQ

You are designing a fault-tolerant architecture to store data in a regional BigQuery dataset. You need to ensure that your application is able to recover from a corruption event in your tables that occurred within the past seven days. You want to adopt managed services with the lowest RPO and most cost-effective solution. What should you do?

- A. Access historical data by using time travel in BigQuery.
- B. Export the data from BigQuery into a new table that excludes the corrupted data
- C. Create a BigQuery table snapshot on a daily basis.
- D. Migrate your data to multi-region BigQuery buckets.

Answer: A

Explanation:

Option A, utilizing BigQuery's time travel feature, is the most appropriate solution. Time travel allows access to historical data within a defined window, which in BigQuery is seven days. This aligns directly with the requirement to recover from corruption within the past seven days. It offers the lowest Recovery Point Objective (RPO) as it allows access to data as it existed at any point within the time window, avoiding data loss associated with other recovery methods. Time travel is a managed service, thus eliminating the overhead of managing backups. Furthermore, it's more cost-effective than alternative solutions like daily snapshots (Option C) which consume additional storage, or multi-region datasets (Option D) which introduce replication costs. Exporting the data into a new table (Option B) requires identifying and filtering the corrupt data, adding manual complexity and potentially missing data if not done accurately. Option A is the simplest, most efficient, and cost-effective approach for this scenario.

Authoritative Links:

BigQuery time travel: Google's official documentation on time travel in BigQuery.

BigQuery disaster recovery: Google's guidance on disaster recovery strategies for BigQuery, where time travel is mentioned.

BigQuery pricing: Useful to understand the cost implications of different BigQuery features.

Question: 266

CertyIQ

You are building a streaming Dataflow pipeline that ingests noise level data from hundreds of sensors placed near construction sites across a city. The sensors measure noise level every ten seconds, and send that data to the pipeline when levels reach above 70 dBA. You need to detect the average noise level from a sensor when data is received for a duration of more than 30 minutes, but the window ends when no data has been received for 15 minutes. What should you do?

- A. Use session windows with a 15-minute gap duration.

- B. Use session windows with a 30-minute gap duration.
- C. Use hopping windows with a 15-minute window, and a thirty-minute period.
- D. Use tumbling windows with a 15-minute window and a fifteen-minute .withAllowedLateness operator.

Answer: A

Explanation:

The correct answer is **A. Use session windows with a 15-minute gap duration.**

Here's why:

Session windows are ideal for grouping data based on periods of activity separated by inactivity gaps. In this scenario, the active period is the duration when a sensor continuously sends data (at or above 70 dBA) and inactivity starts after no data is received for a specified time. A key requirement is detecting average noise level when sensor data arrives for more than 30 minutes, but the processing window must close when the data stream is silent for 15 minutes. This 15-minute silence determines the gap between sessions.

Option A directly addresses the problem with a 15-minute gap. The window starts when the first sensor reading arrives. Subsequent readings within 15 minutes extend the window. If no data comes within 15 mins of the most recent data point, then the window closes and the average calculation is performed. If new data arrives after a 15-minute gap, it will start a new session. This accurately captures the requirement of inactivity ending a processing window.

Option B incorrectly uses a 30-minute gap. The problem states a 15-minute gap should end the session, so this is too long of a gap duration.

Option C's hopping windows with fixed size cannot adapt to periods of inactivity and may cut off ongoing sessions, making it unsuitable for this fluctuating data stream.

Option D's tumbling window is unsuitable for periods of inactivity. Tumbling windows divide the stream based on fixed time intervals, which is not appropriate because the length of time the sensor emits data is not consistent. A 15-minute window would arbitrarily cut off data needed for the 30-minute duration requirement.

In summary, session windows are the correct choice because they are designed for data streams characterized by activity bursts and inactivity gaps, using the inactivity gap as the criteria for closing a session window.

Authoritative Links:

Apache Beam Windowing: <https://beam.apache.org/documentation/programming-guide/#windowing>

Dataflow Windowing Concepts: <https://cloud.google.com/dataflow/docs/concepts/windowing>

Question: 267

CertyIQ

You are creating a data model in BigQuery that will hold retail transaction data. Your two largest tables, sales_transaction_header and sales_transaction_line, have a tightly coupled immutable relationship. These tables are rarely modified after load and are frequently joined when queried. You need to model the sales_transaction_header and sales_transaction_line tables to improve the performance of data analytics queries. What should you do?

- A. Create a sales_transaction table that holds the sales_transaction_header information as rows and the sales_transaction_line rows as nested and repeated fields.
- B. Create a sales_transaction table that holds the sales_transaction_header and sales_transaction_line information as rows, duplicating the sales_transaction_header data for each line.
- C. Create a sales_transaction table that stores the sales_transaction_header and sales_transaction_line data as a JSON data type.

D.Create separate sales_transaction_header and sales_transaction_line tables and, when querying, specify the sales_transaction_line first in the WHERE clause.

Answer: A

Explanation:

The most efficient approach is to model the data into a single sales_transaction table with nested and repeated fields for the line items (option A). This strategy directly addresses the tightly coupled, immutable nature of the sales_transaction_header and sales_transaction_line data. When using BigQuery, joins across separate tables can be expensive, especially for frequently accessed, unchanging data. By nesting the line items within the header record, we eliminate the need for these joins, significantly improving query performance and reducing costs. BigQuery excels at handling nested and repeated fields, which are structured data within columns that reduce data redundancy. The repeated field structure accurately reflects the one-to-many relationship between the header and its associated lines. While option B duplicates data, which is inefficient and goes against best practices, Option C stores the data as JSON, which complicates data analysis and querying because native BigQuery functions are more suited to interacting with table structures. Option D retains the original structure, perpetuating the performance challenges associated with joins. Modeling with nested and repeated fields aligns with columnar data storage of BigQuery, allowing for efficient filtering on header attributes and individual fields within the line items. This is supported by [BigQuery documentation on nested and repeated fields](#), highlighting the effectiveness in managing relational data within BigQuery. The nested structure enables efficient compression and minimizes data scans during query execution. Therefore, option A provides optimal query performance, reduces costs, and fits the nature of BigQuery data organization.

Question: 268

CertyIQ

You created a new version of a Dataflow streaming data ingestion pipeline that reads from Pub/Sub and writes to BigQuery. The previous version of the pipeline that runs in production uses a 5-minute window for processing. You need to deploy the new version of the pipeline without losing any data, creating inconsistencies, or increasing the processing latency by more than 10 minutes. What should you do?

- A.Update the old pipeline with the new pipeline code.
- B.Snapshot the old pipeline, stop the old pipeline, and then start the new pipeline from the snapshot.
- C.Drain the old pipeline, then start the new pipeline.
- D.Cancel the old pipeline, then start the new pipeline.

Answer: C

Explanation:

Here's a detailed justification for why draining the old pipeline and then starting the new one (Option C) is the correct approach:

The key challenge here is migrating from one version of a streaming Dataflow pipeline to another without data loss or inconsistencies. We're dealing with a pipeline using a 5-minute window, and we need to avoid exceeding a 10-minute latency. Options A, B, and D fail to meet all these requirements. Updating the old pipeline (Option A) is risky as it can lead to pipeline failure due to code changes and data loss during the update process. Taking a snapshot (Option B) will only capture the current state of the pipeline, not the data in transit. Cancellation (Option D) would abruptly stop the pipeline, causing data loss, specifically the data that was being processed but was still in the window, which had not been processed yet.

Draining, on the other hand, gracefully stops the old pipeline. When a pipeline is drained, Dataflow allows in-flight data to finish processing within its existing window, ensuring no data is dropped. Once drained, no more

new data will be accepted by this pipeline, guaranteeing that all the data has been processed and written to BigQuery. Afterwards, a new pipeline can be started that can begin processing from the incoming message in Pub/Sub and write to BigQuery. This process avoids data loss and maintains data consistency. By draining the original pipeline with 5-minute windows, the maximum latency is at most 5 minutes after initiating the drain. Then the new pipeline can begin.

Therefore, option C ensures data consistency by completing the processing of in-flight data, avoids data loss, and minimizes latency during the transition, meeting all requirements of the scenario.

Authoritative Links:

Dataflow Pipeline Updates: <https://cloud.google.com/dataflow/docs/guides/updating-a-pipeline> - This document explains the methods for updating Dataflow pipelines, including the draining process.

Dataflow concepts: <https://cloud.google.com/dataflow/docs/concepts/dataflow-model> - This document explains the Dataflow Model including concepts of pipeline execution and data processing.

Question: 269

CertyIQ

Your organization's data assets are stored in BigQuery, Pub/Sub, and a PostgreSQL instance running on Compute Engine. Because there are multiple domains and diverse teams using the data, teams in your organization are unable to discover existing data assets. You need to design a solution to improve data discoverability while keeping development and configuration efforts to a minimum. What should you do?

- A. Use Data Catalog to automatically catalog BigQuery datasets. Use Data Catalog APIs to manually catalog Pub/Sub topics and PostgreSQL tables.
- B. Use Data Catalog to automatically catalog BigQuery datasets and Pub/Sub topics. Use Data Catalog APIs to manually catalog PostgreSQL tables.
- C. Use Data Catalog to automatically catalog BigQuery datasets and Pub/Sub topics. Use custom connectors to manually catalog PostgreSQL tables.
- D. Use custom connectors to manually catalog BigQuery datasets, Pub/Sub topics, and PostgreSQL tables.

Answer: B

Explanation:

The optimal solution leverages Data Catalog's built-in capabilities to minimize manual effort. BigQuery and Pub/Sub enjoy native integration with Data Catalog, allowing for automatic metadata discovery and cataloging upon creation or modification of datasets and topics respectively. This significantly reduces the overhead of manual cataloging processes. PostgreSQL, hosted on Compute Engine, does not have this native integration. Therefore, the Data Catalog API must be used to manually register PostgreSQL tables and their associated metadata, providing the necessary context for discovery. Custom connectors, option C, introduce unnecessary complexity, as manual API interactions are sufficient for PostgreSQL. Option D incorrectly suggests custom connectors for all resources, ignoring the automatic capabilities for BigQuery and Pub/Sub, resulting in excessive and inefficient development work. Option A is incorrect as Pub/Sub topics can be automatically cataloged and do not need to be done using the API. Using Data Catalog to automatically catalog BigQuery and Pub/Sub, and then using its APIs to manually catalog PostgreSQL tables, balances automation with the necessary intervention for a self-managed database, making it the most efficient approach.

Authoritative Links:

Data Catalog Overview: <https://cloud.google.com/data-catalog/docs/overview>

Data Catalog Automatic Metadata Discovery: <https://cloud.google.com/data-catalog/docs/how-to/auto-discovery>

Question: 270

You need to create a SQL pipeline. The pipeline runs an aggregate SQL transformation on a BigQuery table every two hours and appends the result to another existing BigQuery table. You need to configure the pipeline to retry if errors occur. You want the pipeline to send an email notification after three consecutive failures. What should you do?

- A. Use the `BigQueryUpsertTableOperator` in Cloud Composer, set the retry parameter to three, and set the `email_on_failure` parameter to true.
- B. Use the `BigQueryInsertJobOperator` in Cloud Composer, set the retry parameter to three, and set the `email_on_failure` parameter to true.
- C. Create a BigQuery scheduled query to run the SQL transformation with schedule options that repeats every two hours, and enable email notifications.
- D. Create a BigQuery scheduled query to run the SQL transformation with schedule options that repeats every two hours, and enable notification to Pub/Sub topic. Use Pub/Sub and Cloud Functions to send an email after three failed executions.

Answer: B**Explanation:**

The correct answer is **B. Use the `BigQueryInsertJobOperator` in Cloud Composer, set the retry parameter to three, and set the `email_on_failure` parameter to true.**

Here's a detailed justification:

Cloud Composer is a managed Apache Airflow service that excels at orchestrating data pipelines, making it the ideal choice for this scenario. The `BigQueryInsertJobOperator` is specifically designed to execute BigQuery SQL queries, which directly addresses the requirement for a SQL transformation pipeline.

Option B leverages the retry parameter within the `BigQueryInsertJobOperator`. Setting it to three ensures the task will attempt execution up to three times before marking it as failed. This fulfills the retry requirement in the prompt. Additionally, the `email_on_failure` parameter, when set to true, triggers an email notification upon task failure. Coupled with the retry configuration, this ensures that an email is sent after three consecutive failures, precisely as required.

Option A, using `BigQueryUpsertTableOperator`, is not suitable because it primarily deals with table updates and inserts rather than running arbitrary SQL queries. Option C, using BigQuery scheduled queries with email notifications, doesn't allow specifying retry attempts before triggering the email. The built-in notifications in BigQuery are triggered for every failure, not after a series of consecutive failures. Option D, while offering more granular control with Pub/Sub and Cloud Functions, is unnecessarily complex for the prompt's requirements, given the more straightforward solution offered by Cloud Composer. Moreover, the notification setup is less integrated and requires additional resources.

Cloud Composer provides a robust and managed environment for orchestrating these types of workflows, making Option B the most appropriate and efficient solution.

Authoritative Links:

Apache Airflow Documentation: <https://airflow.apache.org/docs/> (For understanding Airflow and its concepts)

Cloud Composer Documentation: <https://cloud.google.com/composer/docs> (For specifics of Google Cloud's managed Airflow)

BigQueryInsertJobOperator Documentation: <https://airflow.apache.org/docs/apache-airflow-providers->

Question: 271

CertyIQ

You are monitoring your organization's data lake hosted on BigQuery. The ingestion pipelines read data from Pub/Sub and write the data into tables on BigQuery. After a new version of the ingestion pipelines is deployed, the daily stored data increased by 50%. The volumes of data in Pub/Sub remained the same and only some tables had their daily partition data size doubled. You need to investigate and fix the cause of the data increase. What should you do?

- A.1. Check for duplicate rows in the BigQuery tables that have the daily partition data size doubled.
- 2. Schedule daily SQL jobs to deduplicate the affected tables.
- 3. Share the deduplication script with the other operational teams to reuse if this occurs to other tables.
- B.1. Check for code errors in the deployed pipelines.
- 2. Check for multiple writing to pipeline BigQuery sink.
- 3. Check for errors in Cloud Logging during the day of the release of the new pipelines.
- 4. If no errors, restore the BigQuery tables to their content before the last release by using time travel.
- C.1. Check for duplicate rows in the BigQuery tables that have the daily partition data size doubled.
- 2. Check the BigQuery Audit logs to find job IDs.
- 3. Use Cloud Monitoring to determine when the identified Dataflow jobs started and the pipeline code version.
- 4. When more than one pipeline ingests data into a table, stop all versions except the latest one.
- D.1. Roll back the last deployment.
- 2. Restore the BigQuery tables to their content before the last release by using time travel.
- 3. Restart the Dataflow jobs and replay the messages by seeking the subscription to the timestamp of the release.

Answer: C

Explanation:

Option C is the most appropriate course of action because it provides a structured approach to identifying and rectifying the data increase issue. First, checking for duplicate rows in the affected BigQuery tables directly addresses the symptom of the problem. This is crucial to understand if the size increase is directly from redundant data. Second, by examining BigQuery Audit logs, you can pinpoint specific jobs that wrote to the tables, allowing for a more focused investigation. Third, correlating the Dataflow jobs execution time (from Cloud Monitoring) with the pipeline code versions helps determine if the new version is responsible. If different pipeline versions write to the same table, it directly explains the increase. Finally, stopping older versions ensures that only the current and (hopefully) corrected version ingest data preventing further increases. Option A proposes deduplication, but it only addresses the symptom without finding the root cause which is inefficient. Option B suggests checking for code errors and multiple writing, which is correct but doesn't use Google Cloud specific tools to do this (Audit logs and Monitoring). Option B also suggests time travel for recovery which isn't ideal for investigation and would disrupt operations. Option D's immediate rollback might be necessary as a last resort but isn't ideal for identifying the cause or a more granular solution. The use of Cloud Monitoring, BigQuery audit logs, and Dataflow versioning are best practices for monitoring and debugging data pipelines on GCP.

<https://cloud.google.com/bigquery/docs/audit-logs><https://cloud.google.com/monitoring><https://cloud.google.com/dataflow/docs/concepts/versioning>

Question: 272

CertyIQ

You have a BigQuery dataset named "customers". All tables will be tagged by using a Data Catalog tag template named "gdpr". The template contains one mandatory field, "has_sensitive_data", with a boolean value. All

employees must be able to do a simple search and find tables in the dataset that have either true or false in the "has_sensitive_data" field. However, only the Human Resources (HR) group should be able to see the data inside the tables for which "has_sensitive_data" is true. You give the all employees group the bigquery.metadataViewer and bigquery.connectionUser roles on the dataset. You want to minimize configuration overhead. What should you do next?

A. Create the "gdpr" tag template with private visibility. Assign the bigquery.dataViewer role to the HR group on the tables that contain sensitive data.

B. Create the "gdpr" tag template with private visibility. Assign the datacatalog.tagTemplateViewer role on this tag to the all employees group, and assign the bigquery.dataViewer role to the HR group on the tables that contain sensitive data.

C. Create the "gdpr" tag template with public visibility. Assign the bigquery.dataViewer role to the HR group on the tables that contain sensitive data.

D. Create the "gdpr" tag template with public visibility. Assign the datacatalog.tagTemplateViewer role on this tag to the all employees group, and assign the bigquery.dataViewer role to the HR group on the tables that contain sensitive data.

Answer: C

Explanation:

Here's a detailed justification for why option C is the correct solution, along with supporting information:

The core requirement is to allow all employees to search for tables based on the "has_sensitive_data" tag while restricting data access to sensitive tables only to the HR group.

Option C correctly addresses this by creating a **publicly visible** "gdpr" tag template. This allows all employees to search and discover tables based on the tag, as required by the problem description, because Data Catalog tag templates with public visibility allow users with datacatalog.tagTemplateViewer to discover the tag and the resources with the tag. This does not grant permission to the data itself, only the metadata around the data.

Crucially, to restrict access to sensitive data, option C then grants the **bigquery.dataViewer role to the HR group only on the tables tagged as containing sensitive data**. This ensures that only the HR group can query the actual data within those specific tables.

Options A, B and D are incorrect because they try to leverage the data catalog datacatalog.tagTemplateViewer role on the template, when the metadata visibility is already granted by bigquery.metadataViewer on the dataset. In option A the employees would not have the visibility to search on the tags. In options B and D the role is unnecessary.

Using a public tag template reduces configuration overhead by avoiding the need to grant datacatalog.tagTemplateViewer to a large employee group. By leveraging the bigquery.dataViewer role directly on the specific sensitive tables, option C fulfills the requirement for restricted data access.

Here's why the other options are not ideal:

Option A: Incorrect, as this template visibility prevents the "all employees" group from searching by tag.

Option B: Incorrect because, granting datacatalog.tagTemplateViewer on the template is an unnecessary action since bigquery.metadataViewer already grants view access to the tag itself, also the template visibility needs to be public for the use case.

Option D: Incorrect because, granting datacatalog.tagTemplateViewer on the template is an unnecessary action since bigquery.metadataViewer already grants view access to the tag itself.

Authoritative Links:

Data Catalog Tag Templates: <https://cloud.google.com/data-catalog/docs/how-to/tag-template-apis>

BigQuery IAM Roles: <https://cloud.google.com/bigquery/docs/access-control>

Data Catalog IAM Roles: <https://cloud.google.com/data-catalog/docs/access-control>

Question: 273

You are creating the CI/CD cycle for the code of the directed acyclic graphs (DAGs) running in Cloud Composer. Your team has two Cloud Composer instances: one instance for development and another instance for production. Your team is using a Git repository to maintain and develop the code of the DAGs. You want to deploy the DAGs automatically to Cloud Composer when a certain tag is pushed to the Git repository. What should you do?

- A.1. Use Cloud Build to copy the code of the DAG to the Cloud Storage bucket of the development instance for DAG testing.
2. If the tests pass, use Cloud Build to copy the code to the bucket of the production instance.
- B.1. Use Cloud Build to build a container with the code of the DAG and the KubernetesPodOperator to deploy the code to the Google Kubernetes Engine (GKE) cluster of the development instance for testing.
2. If the tests pass, use the KubernetesPodOperator to deploy the container to the GKE cluster of the production instance.
- C.1. Use Cloud Build to build a container and the KubernetesPodOperator to deploy the code of the DAG to the Google Kubernetes Engine (GKE) cluster of the development instance for testing.
2. If the tests pass, copy the code to the Cloud Storage bucket of the production instance.
- D.1. Use Cloud Build to copy the code of the DAG to the Cloud Storage bucket of the development instance for DAG testing.
2. If the tests pass, use Cloud Build to build a container with the code of the DAG and the KubernetesPodOperator to deploy the container to the Google Kubernetes Engine (GKE) cluster of the production instance.

Answer: A

Explanation:

The correct answer is A because it directly addresses the requirement of deploying DAGs to Cloud Composer using a CI/CD pipeline triggered by Git tags, without introducing unnecessary complexity. Cloud Composer stores DAGs in a Cloud Storage bucket associated with the environment. Option A leverages Cloud Build to copy the DAG code from the Git repository to the respective Cloud Storage buckets of the development and production Cloud Composer instances. This approach aligns with how Cloud Composer operates: DAG files placed in its designated bucket are automatically picked up and executed.

Option B introduces Kubernetes and containerization unnecessarily, deviating from the question's focus on deploying DAGs to Cloud Composer. Composer already manages the execution of DAGs, rendering Kubernetes deployment redundant in this context. Option C also incorrectly mixes Composer DAG deployment with GKE and containers, which is not the desired method for this scenario. Similarly, option D introduces containerization for production deployment, which is not required for deploying DAGs to Composer.

Option A offers a simpler, more direct, and more efficient solution by using Cloud Build for the core task: copying code to the relevant Cloud Storage buckets. This eliminates the overhead of building containers, managing Kubernetes deployments, and keeps the deployment process aligned with the established Cloud Composer workflow. This directly solves the requirement by triggering the deployment upon tag push and pushing the code to the right buckets. Cloud Composer monitors the buckets and updates DAGs automatically.

Therefore, copying the DAG code directly to Cloud Storage buckets is the appropriate, cost-effective, and streamlined approach for automatically deploying DAGs in Cloud Composer based on Git tag events.

Authoritative Links:

Cloud Composer DAGs: <https://cloud.google.com/composer/docs/concepts/dags>

Cloud Build: <https://cloud.google.com/build/docs>

Cloud Storage: <https://cloud.google.com/storage/docs>

Question: 274

You have a BigQuery table that ingests data directly from a Pub/Sub subscription. The ingested data is encrypted with a Google-managed encryption key. You need to meet a new organization policy that requires you to use keys from a centralized Cloud Key Management Service (Cloud KMS) project to encrypt data at rest. What should you do?

- A. Use Cloud KMS encryption key with Dataflow to ingest the existing Pub/Sub subscription to the existing BigQuery table.
- B. Create a new BigQuery table by using customer-managed encryption keys (CMEK), and migrate the data from the old BigQuery table.
- C. Create a new Pub/Sub topic with CMEK and use the existing BigQuery table by using Google-managed encryption key.
- D. Create a new BigQuery table and Pub/Sub topic by using customer-managed encryption keys (CMEK), and migrate the data from the old BigQuery table.

Answer: B

Explanation:

The correct answer is **B**. Here's why:

The core requirement is to switch from Google-managed encryption keys to customer-managed encryption keys (CMEK) for the BigQuery table's data at rest. Option B directly addresses this by creating a new BigQuery table with CMEK enabled and then migrating the existing data. This approach aligns with best practices for changing encryption keys on data at rest.

Option A is incorrect because Dataflow doesn't handle BigQuery's encryption at rest. It would be useful for data transformations but not for directly changing encryption settings of a table. Option C is incorrect because while you could add encryption to the Pub/Sub topic, this doesn't change the existing BigQuery table's encryption. Option D, though it might appear feasible, is less efficient than B. You don't need to change the Pub/Sub topic encryption since the problem only focuses on the BigQuery table's encryption. Option B solves the issue with the least number of steps.

BigQuery encryption settings are applied at the table level when the table is created. Once a table is created, its encryption type cannot be changed. Therefore, migrating the data to a new table with the required encryption using Cloud KMS is the recommended solution. CMEK provides greater control and aligns with organizational policies for centralized key management, a core feature of Cloud KMS. The migration of data ensures that no data is lost when changing from the default Google-managed key to customer-managed keys.

Key takeaways are that BigQuery encryption happens at table creation time, and Cloud KMS is used for customer-managed encryption keys (CMEK). Dataflow does not control the encryption-at-rest configuration of BigQuery tables.

Authoritative Links:

BigQuery Encryption: <https://cloud.google.com/bigquery/docs/encryption-at-rest>

Cloud KMS: <https://cloud.google.com/kms/docs>

Customer-Managed Encryption Keys (CMEK): <https://cloud.google.com/kms/docs/cmek>

Question: 275

You created an analytics environment on Google Cloud so that your data scientist team can explore data without impacting the on-premises Apache Hadoop solution. The data in the on-premises Hadoop Distributed File System

(HDFS) cluster is in Optimized Row Columnar (ORC) formatted files with multiple columns of Hive partitioning. The data scientist team needs to be able to explore the data in a similar way as they used the on-premises HDFS cluster with SQL on the Hive query engine. You need to choose the most cost-effective storage and processing solution. What should you do?

- A.Import the ORC files to Bigtable tables for the data scientist team.
- B.Import the ORC files to BigQuery tables for the data scientist team.
- C.Copy the ORC files on Cloud Storage, then deploy a Dataproc cluster for the data scientist team.
- D.Copy the ORC files on Cloud Storage, then create external BigQuery tables for the data scientist team.

Answer: D

Explanation:

The correct answer is **D: Copy the ORC files on Cloud Storage, then create external BigQuery tables for the data scientist team.** Here's why:

Cost-Effectiveness: BigQuery excels at cost-effective analysis of large datasets. Option D leverages this, using cost-effective Cloud Storage for data persistence and BigQuery for query processing.

No Need for Managed Compute: Dataproc (option C) involves managing a cluster which incurs ongoing costs even when not actively querying the data. BigQuery is a serverless solution, eliminating these costs.

ORC Format and Hive Partitioning: BigQuery can directly read ORC files stored in Cloud Storage. It also handles Hive-style partitioning, making it a good fit for the existing data structure. [BigQuery Documentation on External Tables](#)

Data Exploration Needs: Data scientists are looking for an environment similar to their on-premises Hive setup, meaning they want to query data with SQL. BigQuery provides familiar SQL capabilities.

Bigtable is not suitable: Bigtable (option A) is a NoSQL database optimized for low-latency reads and writes of large, semi-structured data; it is not optimized for the analytical queries data scientists require and it is not cost-effective for this use case.

BigQuery Table Import not suitable: While BigQuery can import data (option B), importing copies the entire dataset into BigQuery storage, incurring more costs. Utilizing external tables references the data directly in Cloud Storage, reducing storage costs and time to setup.

Therefore, storing the ORC files on Cloud Storage and creating external tables in BigQuery provides the desired SQL query capabilities and preserves the data structure, while optimizing for costs and eliminating the operational overhead of a managed compute environment like Dataproc. This is the best fit for data exploration needs with cost-effectiveness.

Question: 276

CertyIQ

You are designing a Dataflow pipeline for a batch processing job. You want to mitigate multiple zonal failures at job submission time. What should you do?

- A.Submit duplicate pipelines in two different zones by using the --zone flag.
- B.Set the pipeline staging location as a regional Cloud Storage bucket.
- C.Specify a worker region by using the --region flag.
- D.Create an Eventarc trigger to resubmit the job in case of zonal failure when submitting the job.

Answer: C

Explanation:

The correct answer is **C. Specify a worker region by using the --region flag.**

Specifying a worker region using the --region flag is the most effective way to mitigate zonal failures during Dataflow job submission. Dataflow, by default, attempts to distribute workers across available zones within a region. When you use the --region flag, you're instructing Dataflow to create workers in any available zone within that specified region, not just a single zone. This offers inherent resilience to zonal outages. If one zone within the specified region goes down, Dataflow can continue operating using resources in the other available zones within that region. This is a key difference from the other options. Option A, submitting duplicate pipelines in two different zones using the --zone flag, doesn't prevent job failure at the submission time. It's a reactive solution that still might result in a failed submission if the target zone is unavailable during that critical phase. Option B, setting a regional Cloud Storage bucket for pipeline staging, is a good practice for data availability, but does not mitigate zonal failures during job submission. Option D, using Eventarc to resubmit the job, only provides mitigation after the initial job fails, making it reactive rather than proactive in avoiding submission failures. Submitting jobs within a region makes Dataflow services less impacted by a zonal failure, increasing the overall availability of the data processing jobs.

For further research, refer to the official Google Cloud documentation:

[Dataflow Command-Line Interface Reference](#)

[Dataflow Locations](#)

[Choosing a Dataflow Region](#)

Question: 277

CertyIQ

You are designing a real-time system for a ride hailing app that identifies areas with high demand for rides to effectively reroute available drivers to meet the demand. The system ingests data from multiple sources to Pub/Sub, processes the data, and stores the results for visualization and analysis in real-time dashboards. The data sources include driver location updates every 5 seconds and app-based booking events from riders. The data processing involves real-time aggregation of supply and demand data for the last 30 seconds, every 2 seconds, and storing the results in a low-latency system for visualization. What should you do?

- A. Group the data by using a tumbling window in a Dataflow pipeline, and write the aggregated data to Memorystore.
- B. Group the data by using a hopping window in a Dataflow pipeline, and write the aggregated data to Memorystore.
- C. Group the data by using a session window in a Dataflow pipeline, and write the aggregated data to BigQuery.
- D. Group the data by using a hopping window in a Dataflow pipeline, and write the aggregated data to BigQuery.

Answer: B

Explanation:

The correct answer is **B. Group the data by using a hopping window in a Dataflow pipeline, and write the aggregated data to Memorystore.**

Here's why:

The problem requires real-time aggregation of data over a 30-second window, calculated every 2 seconds. This means there will be overlapping windows. A **hopping window** is perfectly suited for this scenario. It slides forward by a fixed step size (2 seconds) while maintaining a fixed window length (30 seconds). This allows for continuous, overlapping aggregations, fulfilling the requirement for data every 2 seconds.

Option A, using a **tumbling window**, would not be appropriate because it creates non-overlapping windows. This would not achieve the required aggregation results every 2 seconds, rather they would produce results

every 30 seconds.

Options C and D propose storing the aggregated data in **BigQuery**. While BigQuery is excellent for analytical workloads, it isn't ideal for the low-latency requirements of real-time dashboards. **Memorystore**, an in-memory data store, offers the low-latency performance needed to update dashboards in near real-time.

Dataflow is the appropriate service for real-time stream processing. The pipeline can read data from Pub/Sub, perform the windowing and aggregation operations, and then write the results to Memorystore. This combination of Dataflow and Memorystore is commonly used for real-time analytics and dashboarding scenarios.

In summary, the hopping window in Dataflow provides the required continuous aggregation, and Memorystore provides the necessary low latency for the use case, making option B the most suitable choice.

Supporting Links:

Apache Beam Windowing: <https://beam.apache.org/documentation/programming-guide/#windowing>
(Dataflow uses Apache Beam)

Google Cloud Memorystore: <https://cloud.google.com/memorystore>

Google Cloud Dataflow: <https://cloud.google.com/dataflow>

Question: 278

CertyIQ

Your car factory is pushing machine measurements as messages into a Pub/Sub topic in your Google Cloud project. A Dataflow streaming job, that you wrote with the Apache Beam SDK, reads these messages, sends acknowledgment to Pub/Sub, applies some custom business logic in a DoFn instance, and writes the result to BigQuery. You want to ensure that if your business logic fails on a message, the message will be sent to a Pub/Sub topic that you want to monitor for alerting purposes. What should you do?

- A. Enable retaining of acknowledged messages in your Pub/Sub pull subscription. Use Cloud Monitoring to monitor the subscription/num_retained_acked_messages metric on this subscription.
- B. Use an exception handling block in your Dataflow's DoFn code to push the messages that failed to be transformed through a side output and to a new Pub/Sub topic. Use Cloud Monitoring to monitor the topic/num_unacked_messages_by_region metric on this new topic.
- C. Enable dead lettering in your Pub/Sub pull subscription, and specify a new Pub/Sub topic as the dead letter topic. Use Cloud Monitoring to monitor the subscription/dead_letter_message_count metric on your pull subscription.
- D. Create a snapshot of your Pub/Sub pull subscription. Use Cloud Monitoring to monitor the snapshot/num_messages metric on this snapshot.

Answer: B

Explanation:

The correct answer is **B**. Here's why:

Rationale:

The core requirement is to handle failures within the Dataflow's custom business logic (DoFn) and route failed messages to a separate monitoring channel. Option B directly addresses this by using exception handling within the DoFn. When an exception occurs during message processing, the DoFn can push the failed message to a side output. This side output then streams these failed messages to a dedicated Pub/Sub topic. This is a standard pattern for error handling in stream processing.

Cloud Monitoring can then track the topic/num_unacked_messages_by_region metric for this dedicated Pub/Sub topic, effectively alerting on the presence of failed messages. This approach allows for granular error handling at the application level, which is better suited for custom business logic errors compared to

options that are more about system level failures.

Why other options are incorrect:

A: Retaining acknowledged messages is useful for debugging and replaying scenarios, but it doesn't isolate and direct failed messages to a dedicated topic for monitoring. The subscription/num_retained_acked_messages metric tracks retained acknowledgements, not errors.

C: Pub/Sub dead-lettering is primarily for handling messages that fail to be acknowledged (e.g., due to delivery issues or the subscriber's failure), not errors during message transformation within the Dataflow pipeline. Dead-lettering is a system-level safety net, not a robust application-level error handling mechanism. The subscription/dead_letter_message_count metric monitors failed deliveries, not transformation errors.

D: Snapshots are used for preserving a point-in-time view of a subscription for offline analysis. They don't actively isolate errors and are not designed for a live monitoring scenario. The snapshot/num_messages metric tracks total messages in the snapshot, not specifically failed ones.

Key Concepts:

Apache Beam SDK: A unified programming model for batch and streaming data processing.

Dataflow: Google Cloud's fully managed stream and batch data processing service that runs Apache Beam pipelines.

DoFn: A function in the Beam programming model that specifies how individual data elements are transformed.

Side Outputs: A feature of Beam that allows a DoFn to emit elements to multiple outputs.

Pub/Sub: A fully managed messaging service that provides asynchronous messaging for reliable, scalable data ingestion.

Cloud Monitoring: Google Cloud's observability service for monitoring cloud resources.

Authoritative Links:

Apache Beam Side Outputs: <https://beam.apache.org/documentation/programming-guide/#side-inputs-and-side-outputs>

Dataflow Error Handling: <https://cloud.google.com/dataflow/docs/guides/handling-errors>

Pub/Sub Dead Letter Queues: <https://cloud.google.com/pubsub/docs/dead-letter-queues>

Cloud Monitoring metrics for Pub/Sub: https://cloud.google.com/monitoring/api/metrics_gcp#gcp-pubsub

In summary, option B implements the most appropriate error-handling mechanism within the Dataflow pipeline, allowing for both granular error capture and direct routing of failed messages to a monitoring topic.

Question: 279

CertyIQ

You want to store your team's shared tables in a single dataset to make data easily accessible to various analysts. You want to make this data readable but unmodifiable by analysts. At the same time, you want to provide the analysts with individual workspaces in the same project, where they can create and store tables for their own use, without the tables being accessible by other analysts. What should you do?

A. Give analysts the BigQuery Data Viewer role at the project level. Create one other dataset, and give the analysts the BigQuery Data Editor role on that dataset.

B. Give analysts the BigQuery Data Viewer role at the project level. Create a dataset for each analyst, and give each analyst the BigQuery Data Editor role at the project level.

C. Give analysts the BigQuery Data Viewer role on the shared dataset. Create a dataset for each analyst, and give each analyst the BigQuery Data Editor role at the dataset level for their assigned dataset.

D. Give analysts the BigQuery Data Viewer role on the shared dataset. Create one other dataset and give the analysts the BigQuery Data Editor role on that dataset.

Answer: C

Explanation:

The correct answer is **C** because it aligns with the principle of least privilege and effectively segregates access based on intended use. Giving analysts the BigQuery Data Viewer role on the shared dataset grants them read-only access, fulfilling the requirement that they cannot modify shared tables. By creating individual datasets for each analyst and assigning the BigQuery Data Editor role at the dataset level, analysts gain full control over their own workspaces, allowing them to create and modify tables without affecting others' data or the shared dataset. Option A grants project-wide read access which isn't optimal, as we want to give specific access on a dataset and not the whole project. Option B is incorrect as it gives each analyst the BigQuery Data Editor role at the project level, which would not satisfy the requirement that the tables are not accessible by other analysts. Option D is incorrect as it only creates a single dataset for all the analysts' workspaces which means that the analysts could view each others' tables when the question asks for them not to be able to do so. Option C is the only option that meets the stated conditions. This approach also promotes organized data management within the same project by isolating individual workspaces. This aligns with Google Cloud's recommendation to manage access on a need-to-use basis to reduce potential data breaches.

[Google Cloud IAM documentation](#)[BigQuery IAM documentation](#)

Question: 280

CertyIQ

You are running a streaming pipeline with Dataflow and are using hopping windows to group the data as the data arrives. You noticed that some data is arriving late but is not being marked as late data, which is resulting in inaccurate aggregations downstream. You need to find a solution that allows you to capture the late data in the appropriate window. What should you do?

- A. Use watermarks to define the expected data arrival window. Allow late data as it arrives.
- B. Change your windowing function to tumbling windows to avoid overlapping window periods.
- C. Change your windowing function to session windows to define your windows based on certain activity.
- D. Expand your hopping window so that the late data has more time to arrive within the grouping.

Answer: A

Explanation:

The correct answer is **A. Use watermarks to define the expected data arrival window. Allow late data as it arrives.** Here's why:

Dataflow uses watermarks to track the progress of data processing. A watermark represents a point in time before which all data is expected to have arrived. When data arrives after the watermark, it's considered late. By default, Dataflow might drop late data, leading to inaccurate aggregations. Hopping windows, as used in this scenario, inherently handle overlapping windows to produce periodic aggregations. Changing to tumbling or session windows won't address the issue of late data within the hopping window context.

The correct solution is to configure Dataflow to allow late data. This is done by specifying an allowed lateness duration with `.withAllowedLateness()`. This duration extends the window's processing time beyond the watermark, providing an opportunity for late-arriving data to be included in the appropriate aggregation. Option A accurately describes this approach. Expanding the hopping window, as in option D, would only delay processing and would not correctly address late data arriving outside the expanded window bounds.

Authoritative links:

[Dataflow documentation on late data](#) These resources offer a deep dive into watermarks, windowing strategies, and specifically, the handling of late data in Dataflow pipelines. They showcase that allowing late data via the `withAllowedLateness()` configuration, as described in answer A, is the correct method to capture delayed arrivals and achieve accurate aggregations within hopping windows.

Question: 281

You work for a large ecommerce company. You store your customer's order data in Bigtable. You have a garbage collection policy set to delete the data after 30 days and the number of versions is set to 1. When the data analysts run a query to report total customer spending, the analysts sometimes see customer data that is older than 30 days. You need to ensure that the analysts do not see customer data older than 30 days while minimizing cost and overhead. What should you do?

- A. Set the expiring values of the column families to 29 days and keep the number of versions to 1.
- B. Use a timestamp range filter in the query to fetch the customer's data for a specific range.
- C. Schedule a job daily to scan the data in the table and delete data older than 30 days.
- D. Set the expiring values of the column families to 30 days and set the number of versions to 2.

Answer: B

Explanation:

The correct answer is **B: Use a timestamp range filter in the query to fetch the customer's data for a specific range.**

Here's why:

Bigtable's garbage collection policies operate in the background and aren't guaranteed to be instantaneous. There might be a slight delay before data exceeding the configured age is physically removed. This explains why analysts sometimes see older data, even with the 30-day policy. Option A, while seemingly similar, doesn't address the inherent delay in garbage collection and could lead to premature data removal. Option C introduces unnecessary overhead with daily data scanning and deletion, which Bigtable already handles automatically. Option D is also problematic as keeping two versions doesn't solve the issue of seeing outdated records, it just stores extra versions.

Using a timestamp range filter directly within the query provides a real-time constraint. By filtering the data based on a timestamp range that only includes records within the last 30 days, the analysts will never retrieve outdated information, regardless of any delays in the garbage collection process. This method is more accurate and efficient than relying solely on garbage collection. Bigtable is designed for efficient read operations, so using a filter will not incur significant performance costs. It also aligns with the principle of minimizing operational overhead by leveraging Bigtable's built-in querying features.

Therefore, applying a timestamp range filter in the query is the most effective approach to ensuring data analysts see only data younger than 30 days while minimizing costs and overhead.

Further research:

Bigtable Garbage Collection: <https://cloud.google.com/bigtable/docs/garbage-collection>

Bigtable Filtering Data: <https://cloud.google.com/bigtable/docs/filtering>

Bigtable Timestamps: <https://cloud.google.com/bigtable/docs/overview#timestamps>

Question: 282**CertyIQ**

You are using a Dataflow streaming job to read messages from a message bus that does not support exactly-once delivery. Your job then applies some transformations, and loads the result into BigQuery. You want to ensure that your data is being streamed into BigQuery with exactly-once delivery semantics. You expect your ingestion throughput into BigQuery to be about 1.5 GB per second. What should you do?

- A. Use the BigQuery Storage Write API and ensure that your target BigQuery table is regional.
- B. Use the BigQuery Storage Write API and ensure that your target BigQuery table is multiregional.
- C. Use the BigQuery Streaming API and ensure that your target BigQuery table is regional.
- D. Use the BigQuery Streaming API and ensure that your target BigQuery table is multiregional.

Answer: A**Explanation:**

The correct answer is A: Use the BigQuery Storage Write API and ensure that your target BigQuery table is regional.

Here's why:

The primary goal is to achieve exactly-once delivery of data into BigQuery, despite the source message bus not guaranteeing this. The BigQuery Storage Write API offers robust support for exactly-once writes through its "append-based" writes mechanism. This API, unlike the legacy streaming API, doesn't rely on insert-based writes, which can lead to duplicates in case of retries. The Storage Write API allows for idempotent writes through its commit and stream concepts, meaning the same data sent multiple times will only be written once to BigQuery, achieving the exactly-once guarantee needed. For ingestion throughput of 1.5 GB per second, the BigQuery Storage Write API is well-suited; it can handle high throughput scenarios. Regional tables, by placing data within a single region, optimize write and read performance compared to multi-regional tables. While both regional and multi-regional tables can be used with the Storage Write API, regional locations provide lower latency and greater overall throughput if you do not need a multi-region setup. Choosing a regional table will not affect the exactly-once guarantee. The BigQuery Streaming API (options C & D), while a suitable alternative, does not guarantee exactly-once writes and may lead to duplicates when retries occur on the Dataflow job. Also, it's not ideal for very high throughput scenarios. Therefore, focusing on the Storage Write API with a regional table is the correct approach.

Authoritative Links:

[BigQuery Storage Write API Overview](#)[BigQuery Locations](#)[Dataflow Exactly-Once Semantics](#)**Question: 283****CertyIQ**

You have created an external table for Apache Hive partitioned data that resides in a Cloud Storage bucket, which contains a large number of files. You notice that queries against this table are slow. You want to improve the performance of these queries. What should you do?

- A. Change the storage class of the Hive partitioned data objects from Coldline to Standard.
- B. Create an individual external table for each Hive partition by using a common table name prefix. Use wildcard table queries to reference the partitioned data.
- C. Upgrade the external table to a BigLake table. Enable metadata caching for the table.
- D. Migrate the Hive partitioned data objects to a multi-region Cloud Storage bucket.

Answer: C**Explanation:**

The correct answer is C: Upgrade the external table to a BigLake table and enable metadata caching. Here's why:

Performance Bottlenecks with External Hive Tables:

Directly querying a large number of files in Cloud Storage through an external Hive table can be slow. This is because each query often requires reading metadata (file locations, schema, partitions) from Cloud Storage, resulting in significant latency, particularly with numerous files.

BigLake Tables and Metadata Caching:

BigLake tables are designed to improve performance when querying external data on Google Cloud. By converting the external table to a BigLake table, we gain the benefit of metadata caching. This significantly reduces the overhead of accessing the data's location, schema, and partitions during query execution. Instead of re-reading metadata from Cloud Storage each time, it's stored in BigQuery's high-performance cache. This speeds up query planning and execution, leading to substantial performance gains.

Why Other Options are Incorrect:

A. Changing storage class: While changing the storage class from Coldline to Standard reduces access latency, it doesn't directly address the metadata retrieval bottleneck, which is the primary cause of slow queries in this scenario. It's a cost vs. performance tradeoff, but doesn't tackle the core issue here. [Cloud Storage storage classes](#)

B. Creating multiple tables using wildcards: While wildcard table queries can be useful in certain situations, creating an individual external table for each partition creates management overhead and doesn't address metadata caching. This approach can even be slower than a single table with proper partitioning when dealing with numerous partitions as it involves scanning a large number of table definitions. This strategy also increases the risk of exceeding table limits in BigQuery. [BigQuery wildcard tables](#)

D. Migrating to a multi-region bucket: Multi-region buckets improve data availability and reduce latency for users in different geographic locations; however, it does not fix the issue of metadata retrieval during queries. While helpful for globally distributed applications, it wouldn't inherently address the identified performance issue. [Cloud Storage locations](#)

Conclusion:

Upgrading to a BigLake table and enabling metadata caching directly tackles the performance problem by caching the external data's metadata, leading to much faster query planning and execution against partitioned data in Cloud Storage. This approach optimizes query speed and avoids potential issues with alternative solutions.

Authoritative links: [BigLake tables overview](#) [Metadata caching in BigLake](#)

Question: 284

CertyIQ

You have a network of 1000 sensors. The sensors generate time series data: one metric per sensor per second, along with a timestamp. You already have 1 TB of data, and expect the data to grow by 1 GB every day. You need to access this data in two ways. The first access pattern requires retrieving the metric from one specific sensor stored at a specific timestamp, with a median single-digit millisecond latency. The second access pattern requires running complex analytic queries on the data, including joins, once a day. How should you store this data?

A. Store your data in BigQuery. Concatenate the sensor ID and timestamp, and use it as the primary key.

B. Store your data in Bigtable. Concatenate the sensor ID and timestamp and use it as the row key. Perform an export to BigQuery every day.

C. Store your data in Bigtable. Concatenate the sensor ID and metric, and use it as the row key. Perform an export to BigQuery every day.

D.Store your data in BigQuery. Use the metric as a primary key.

Answer: B

Explanation:

The correct answer is **B. Store your data in Bigtable. Concatenate the sensor ID and timestamp and use it as the row key. Perform an export to BigQuery every day.**

Here's a detailed justification:

Real-time Retrieval Requirements: The primary requirement is retrieving data from a specific sensor at a specific timestamp with single-digit millisecond latency. Bigtable excels in this area. Bigtable is a NoSQL, wide-column store designed for low-latency access to large datasets. Its ability to handle high read and write throughput makes it ideal for time-series data. The concatenation of sensor ID and timestamp as the row key allows for efficient retrieval of specific sensor readings at specific times because Bigtable efficiently serves queries based on row keys.

Analytical Query Needs: The second requirement involves running complex analytic queries daily. BigQuery is a fully-managed, serverless data warehouse optimized for large-scale data analytics. BigQuery's SQL interface and columnar storage enables efficient processing for complex queries, including joins, which are difficult to accomplish with Bigtable.

Data Transfer Strategy: Exporting data from Bigtable to BigQuery fulfills the need for analytical querying. This process can be automated on a daily basis, providing a cost-effective and performant solution for both real-time access and large-scale analytics. Dataflow can be employed to perform ETL from Bigtable to BigQuery.

Why other options are unsuitable:

Option A: Storing everything in BigQuery might not meet the real-time latency requirements. While BigQuery is fast for analytics, single-digit millisecond latency for individual record retrieval is not its strength, and the frequent queries could be expensive. Using a concatenated sensor ID and timestamp as a "primary key" in BigQuery isn't the same as in a traditional relational database. BigQuery doesn't enforce primary key constraints.

Option C: Concatenating sensor ID and metric as the row key in Bigtable is incorrect. Since the time series data includes the timestamp and the metrics vary for each sensor, you'll lose the ability to efficiently retrieve historical readings from one sensor with this storage schema. You also will not be able to efficiently perform analytics on different metrics over time.

Option D: Using the metric as a "primary key" in BigQuery is not ideal as each metric is a continuous value, not a distinct identifier. This will not allow you to efficiently query data for specific sensors.

Scalability and Cost: Bigtable is highly scalable to accommodate the expected data growth. The combination of Bigtable for real-time access and BigQuery for analytics is a common pattern for time-series data, optimizing both performance and cost.

Authoritative Links:

Bigtable: <https://cloud.google.com/bigtable/docs/overview>

BigQuery: <https://cloud.google.com/bigquery/docs/introduction>

Dataflow: <https://cloud.google.com/dataflow/docs/>

Question: 285

CertyIQ

You have 100 GB of data stored in a BigQuery table. This data is outdated and will only be accessed one or two

times a year for analytics with SQL. For backup purposes, you want to store this data to be immutable for 3 years. You want to minimize storage costs. What should you do?

- A.1. Create a BigQuery table clone.
2. Query the clone when you need to perform analytics.
- B.1. Create a BigQuery table snapshot.
2. Restore the snapshot when you need to perform analytics.
- C.1. Perform a BigQuery export to a Cloud Storage bucket with archive storage class.
2. Enable versioning on the bucket.
3. Create a BigQuery external table on the exported files.
- D.1. Perform a BigQuery export to a Cloud Storage bucket with archive storage class.
2. Set a locked retention policy on the bucket.
3. Create a BigQuery external table on the exported files.

Answer: D

Explanation:

Option D is the correct solution because it addresses all the requirements: minimizing storage costs, ensuring immutability for 3 years, and infrequent access for analytics. Exporting the BigQuery data to Cloud Storage with the "archive" storage class is the most cost-effective option for long-term, rarely accessed data. Archive storage is significantly cheaper than BigQuery storage or even other Cloud Storage classes. Setting a locked retention policy on the Cloud Storage bucket guarantees the immutability of the data for the specified 3-year period, preventing accidental deletion or modification. Creating a BigQuery external table on the exported files allows you to query the data using SQL when needed without having to import it back into BigQuery, saving on compute and storage costs. Option A is incorrect as cloning does not offer cost savings. Option B is incorrect as snapshots do not offer long-term immutability. Option C is incorrect, because enabling versioning will increase cost of storage and does not achieve immutability.

Here are some authoritative links for further research:

BigQuery Export Data: <https://cloud.google.com/bigquery/docs/exporting-data>

Cloud Storage Storage Classes: <https://cloud.google.com/storage/docs/storage-classes>

Cloud Storage Retention Policies: <https://cloud.google.com/storage/docs/retention-policy>

BigQuery External Tables: <https://cloud.google.com/bigquery/docs/external-data-sources>

Question: 286

CertyIQ

You have thousands of Apache Spark jobs running in your on-premises Apache Hadoop cluster. You want to migrate the jobs to Google Cloud. You want to use managed services to run your jobs instead of maintaining a long-lived Hadoop cluster yourself. You have a tight timeline and want to keep code changes to a minimum. What should you do?

- A. Move your data to BigQuery. Convert your Spark scripts to a SQL-based processing approach.
- B. Rewrite your jobs in Apache Beam. Run your jobs in Dataflow.
- C. Copy your data to Compute Engine disks. Manage and run your jobs directly on those instances.
- D. Move your data to Cloud Storage. Run your jobs on Dataproc.

Answer: D

Explanation:

The correct answer is **D. Move your data to Cloud Storage. Run your jobs on Dataproc.**

Here's why:

Dataproc as a Managed Hadoop Service: Dataproc is Google Cloud's managed service for Apache Hadoop and Spark. It allows users to run Hadoop and Spark jobs without managing the underlying infrastructure. This aligns perfectly with the requirement of using managed services and avoiding long-lived cluster maintenance.

[Google Dataproc Documentation](#)

Minimal Code Changes: Dataproc is designed to be compatible with existing Hadoop and Spark jobs. This means that, generally, the Spark jobs can be migrated with minimal code modifications, adhering to the tight timeline constraint.

Data Migration to Cloud Storage: Cloud Storage is Google Cloud's scalable and durable object storage service. It is ideal for storing the data that will be processed by Dataproc jobs. Using Cloud Storage as a data source allows Dataproc to efficiently access the required data. [Google Cloud Storage Documentation](#)

Why other options are less suitable:

A (BigQuery and SQL): While BigQuery is a powerful data warehouse, it requires converting the Spark jobs to SQL, entailing significant code changes, which contradicts the requirement for minimal code changes.

B (Apache Beam and Dataflow): Apache Beam is a great framework for data processing, but it also requires rewriting the jobs, violating the minimal code change constraint. Dataflow is a managed service for Apache Beam.

C (Compute Engine disks): Managing jobs directly on Compute Engine instances would require manually handling the Hadoop cluster setup and maintenance, defeating the purpose of using managed services.

Therefore, moving data to Cloud Storage and running jobs on Dataproc offers the most efficient path to migration while minimizing code modifications and utilizing managed services, aligning with all stated requirements.

Question: 287

CertyIQ

You are administering shared BigQuery datasets that contain views used by multiple teams in your organization. The marketing team is concerned about the variability of their monthly BigQuery analytics spend using the on-demand billing model. You need to help the marketing team establish a consistent BigQuery analytics spend each month. What should you do?

- A. Create a BigQuery Enterprise reservation with a baseline of 250 slots and autoscaling set to 500 for the marketing team, and bill them back accordingly.
- B. Establish a BigQuery quota for the marketing team, and limit the maximum number of bytes scanned each day.
- C. Create a BigQuery reservation with a baseline of 500 slots with no autoscaling for the marketing team, and bill them back accordingly.
- D. Create a BigQuery Standard pay-as-you-go reservation with a baseline of 0 slots and autoscaling set to 500 for the marketing team, and bill them back accordingly.

Answer: C

Explanation:

The correct answer is C because it directly addresses the marketing team's need for consistent BigQuery spending. Creating a fixed-slot reservation with no autoscaling ensures predictable costs. By assigning 500 slots, the marketing team has a dedicated compute capacity, and their monthly spending will be primarily determined by the reservation's cost, not the variability of queries. This allows them to budget effectively. Option A introduces autoscaling, which can lead to unpredictable costs depending on query loads. Option B, establishing a quota, limits the team's resource usage but does not guarantee cost predictability. It might result in failed queries if the limit is reached. Option D, a pay-as-you-go reservation with autoscaling, is still subject to cost variability based on usage. Therefore, a fixed-slot reservation (option C) offers the most control over cost for the marketing team. By allocating a baseline of 500 slots, they have a consistent computational resource for their analytics needs. This aligns with best practices for managing cloud costs by

opting for committed-use discounts over on-demand variable pricing. This strategy allows for cost predictability and avoids the uncertainty of on-demand pricing.

Further research:

BigQuery Reservations: <https://cloud.google.com/bigquery/docs/reservations-intro>

Understanding BigQuery Pricing: <https://cloud.google.com/bigquery/pricing>

BigQuery Capacity Management: <https://cloud.google.com/bigquery/docs/capacity-management>

Question: 288

CertyIQ

You are part of a healthcare organization where data is organized and managed by respective data owners in various storage services. As a result of this decentralized ecosystem, discovering and managing data has become difficult. You need to quickly identify and implement a cost-optimized solution to assist your organization with the following:

- Data management and discovery
- Data lineage tracking
- Data quality validation

How should you build the solution?

- A. Use BigLake to convert the current solution into a data lake architecture.
- B. Build a new data discovery tool on Google Kubernetes Engine that helps with new source onboarding and data lineage tracking.
- C. Use BigQuery to track data lineage, and use Dataprep to manage data and perform data quality validation.
- D. Use Dataplex to manage data, track data lineage, and perform data quality validation.

Answer: D

Explanation:

The correct answer is **D. Use Dataplex to manage data, track data lineage, and perform data quality validation.**

Dataplex is a Google Cloud service specifically designed to address the challenges outlined in the scenario: data management, discovery, lineage tracking, and quality validation across a decentralized data landscape. It provides a unified platform for managing data regardless of where it's stored. Dataplex's data discovery features enable users to easily find relevant data assets. Its lineage tracking capabilities automatically trace data transformations, offering visibility into data origins and flow. Moreover, Dataplex incorporates data quality tools to monitor and validate data integrity.

Option A, using BigLake, primarily focuses on creating a data lake architecture, not on data management across existing heterogeneous storage. Option B, building a custom tool on GKE, requires significant development effort and is less cost-optimized than using a pre-built service like Dataplex. Option C, while using BigQuery for lineage and Dataprep for data quality, doesn't offer the integrated, unified management experience that Dataplex provides across various data storage systems.

Dataplex is the most suitable solution because it natively provides the required features in a managed, scalable way. It helps to overcome the difficulties associated with decentralized data governance.

Here are some authoritative links for further research:

Google Cloud Dataplex documentation: <https://cloud.google.com/dataplex/docs>

Dataplex Overview: <https://cloud.google.com/dataplex/docs/concepts/overview>

Dataplex for Data Governance: <https://cloud.google.com/blog/products/data-analytics/new-google-cloud->

Question: 289

CertyIQ

You have data located in BigQuery that is used to generate reports for your company. You have noticed some weekly executive report fields do not correspond to format according to company standards. For example, report errors include different telephone formats and different country code identifiers. This is a frequent issue, so you need to create a recurring job to normalize the data. You want a quick solution that requires no coding. What should you do?

- A. Use Cloud Data Fusion and Wrangler to normalize the data, and set up a recurring job.
- B. Use Dataflow SQL to create a job that normalizes the data, and that after the first run of the job, schedule the pipeline to execute recurrently.
- C. Create a Spark job and submit it to Dataproc Serverless.
- D. Use BigQuery and GoogleSQL to normalize the data, and schedule recurring queries in BigQuery.

Answer: A**Explanation:**

The correct answer is A: Use Cloud Data Fusion and Wrangler to normalize the data, and set up a recurring job. Here's why:

Cloud Data Fusion is a fully managed, visual data integration service that excels at building data pipelines without extensive coding. Wrangler, a feature within Data Fusion, provides an interactive and intuitive interface for data preparation and transformation. This aligns perfectly with the requirement of a "quick solution that requires no coding." Wrangler allows users to visually manipulate data, applying transformations like formatting phone numbers and standardizing country codes through its drag-and-drop interface. Once the transformation steps are defined, a recurring pipeline can be scheduled in Data Fusion, satisfying the need for a recurring job to normalize the data.

Option B, using Dataflow SQL, requires writing SQL code, which contradicts the "no coding" constraint. While Dataflow is powerful for data processing, it adds a level of complexity that the question seeks to avoid. Option C, using Spark on Dataproc Serverless, also necessitates writing code (Spark code), making it unsuitable for the "no coding" requirement. Option D, while BigQuery SQL can achieve data normalization, it lacks the visual interactive transformation experience of Wrangler. Further, it does not inherently provide a straightforward way to manage iterative/recurring transformations and scheduling. Therefore, for a rapid, no-code solution focusing on interactive data transformation and recurrent execution, Cloud Data Fusion with Wrangler offers the most suitable approach.

[Cloud Data Fusion Documentation](#)[Cloud Data Fusion Wrangler Documentation](#)

Question: 290

CertyIQ

You are designing a messaging system by using Pub/Sub to process clickstream data with an event-driven consumer app that relies on a push subscription. You need to configure the messaging system that is reliable enough to handle temporary downtime of the consumer app. You also need the messaging system to store the input messages that cannot be consumed by the subscriber. The system needs to retry failed messages gradually, avoiding overloading the consumer app, and store the failed messages after a maximum of 10 retries in a topic. How should you configure the Pub/Sub subscription?

- A. Increase the acknowledgement deadline to 10 minutes.
- B. Use immediate redelivery as the subscription retry policy, and configure dead lettering to a different topic with maximum delivery attempts set to 10.

C. Use exponential backoff as the subscription retry policy, and configure dead lettering to the same source topic with maximum delivery attempts set to 10.

D. Use exponential backoff as the subscription retry policy, and configure dead lettering to a different topic with maximum delivery attempts set to 10.

Answer: D

Explanation:

The correct configuration for handling temporary consumer downtime, reliable message processing with retries, and storing failed messages in a separate topic involves using exponential backoff and dead lettering. Option D aligns with these requirements. Exponential backoff ensures that retries are not performed immediately, preventing overwhelming the consumer application during recovery from downtime. This gradual increase in retry intervals allows the consumer to recover without being bombarded by messages. Configuring dead lettering to a different topic ensures that messages failing after 10 retries are moved to a separate location for analysis and possible manual reprocessing, maintaining the primary subscription clean. Increasing the acknowledgement deadline (Option A) only delays message processing and does not handle message retries or storage of failed messages. Immediate redelivery (Option B) can overwhelm a recovering consumer. Dead lettering to the same source topic (Option C) would create an infinite loop of retries, not a solution. Google Cloud's Pub/Sub documentation strongly advises using exponential backoff for retries and dead-letter topics for handling unprocessable messages. These mechanisms help to ensure message durability and application resilience. In summary, Option D provides the optimal combination for reliable message handling, retry mechanisms, and failed message storage.

Authoritative Links:

[Pub/Sub Retry Policies](#)

[Pub/Sub Dead Letter Queues](#)

Question: 291

CertyIQ

You designed a data warehouse in BigQuery to analyze sales data. You want a self-serving, low-maintenance, and cost-effective solution to share the sales dataset to other business units in your organization. What should you do?

A. Create an Analytics Hub private exchange, and publish the sales dataset.

B. Enable the other business units' projects to access the authorized views of the sales dataset.

C. Create and share views with the users in the other business units.

D. Use the BigQuery Data Transfer Service to create a schedule that copies the sales dataset to the other business units' projects.

Answer: A

Explanation:

The most appropriate solution is to create an Analytics Hub private exchange and publish the sales dataset (Option A). Analytics Hub is designed for secure and efficient data sharing within an organization, providing a self-service model. By creating a private exchange, you control who can access the dataset, ensuring appropriate governance. Publishing the dataset allows other business units to discover and subscribe to it as needed, removing the need for manual data management. This approach minimizes maintenance overhead as the data is centrally managed, and costs are optimized through efficient data sharing instead of data duplication. Authorized views (Option B) and shared views (Option C) involve project-specific configurations and ongoing maintenance, making them less self-serving. Using the BigQuery Data Transfer Service (Option D) would lead to data duplication, increasing storage costs and maintenance overhead. In conclusion, Analytics Hub is the ideal solution to meet the stated requirements: self-serving, low-maintenance, and cost-

effective data sharing.

Authoritative Links:

Google Cloud Analytics Hub Documentation: <https://cloud.google.com/analytics-hub/docs>

BigQuery Authorized Views: <https://cloud.google.com/bigquery/docs/share-access-views>

BigQuery Data Transfer Service: <https://cloud.google.com/bigquery/docs/transfer-service>

Question: 292

CertyIQ

You have terabytes of customer behavioral data streaming from Google Analytics into BigQuery daily. Your customers' information, such as their preferences, is hosted on a Cloud SQL for MySQL database. Your CRM database is hosted on a Cloud SQL for PostgreSQL instance. The marketing team wants to use your customers' information from the two databases and the customer behavioral data to create marketing campaigns for yearly active customers. You need to ensure that the marketing team can run the campaigns over 100 times a day on typical days and up to 300 during sales. At the same time, you want to keep the load on the Cloud SQL databases to a minimum. What should you do?

- A. Create BigQuery connections to both Cloud SQL databases. Use BigQuery federated queries on the two databases and the Google Analytics data on BigQuery to run these queries.
- B. Create a job on Apache Spark with Dataproc Serverless to query both Cloud SQL databases and the Google Analytics data on BigQuery for these queries.
- C. Create streams in Datastream to replicate the required tables from both Cloud SQL databases to BigQuery for these queries.
- D. Create a Dataproc cluster with Trino to establish connections to both Cloud SQL databases and BigQuery, to execute the queries.

Answer: C

Explanation:

The optimal solution is **C: Create streams in Datastream to replicate the required tables from both Cloud SQL databases to BigQuery for these queries.**

Here's why:

Minimizing Cloud SQL Load: Directly querying Cloud SQL databases for hundreds of daily campaign runs would put significant and unnecessary strain on their resources. Datastream replicates data changes to BigQuery, allowing queries to be run against BigQuery instead, thus protecting the operational performance of the Cloud SQL databases.

Scalability: BigQuery is designed for large-scale analytical workloads and can handle the high query frequency (100-300 times daily) without performance issues. Replicating the data to BigQuery enables you to leverage its distributed processing power.

Data Consistency: Datastream ensures near real-time replication with change data capture (CDC), maintaining consistency between the source Cloud SQL databases and BigQuery.

Query Performance: BigQuery's columnar storage and optimized query engine will provide faster query times for analytical workloads, which is essential for running campaigns quickly and frequently.

Option A (BigQuery Federated Queries): While convenient, federated queries against Cloud SQL can still cause performance issues on the Cloud SQL instances, especially with the given query frequency. They also may be slower compared to querying a native BigQuery table.

Option B (Dataproc Serverless with Spark): This approach is more complex, as it requires coding in Spark and is less efficient than direct data replication. Spark introduces another layer of processing and is not the most

straightforward approach to this scenario.

Option D (Dataproc Cluster with Trino): While Trino can federate data sources, maintaining a Dataproc cluster for this purpose introduces unnecessary overhead and complexity compared to using Datastream for replication. This is also more expensive and less efficient than the chosen solution.

By replicating the required tables into BigQuery using Datastream, you achieve a performant, scalable, and cost-effective solution that minimizes the load on the operational Cloud SQL databases.

Supporting Links:

Google Cloud Datastream: <https://cloud.google.com/datastream/docs>

Google Cloud BigQuery: <https://cloud.google.com/bigquery/docs>

Change Data Capture (CDC): https://en.wikipedia.org/wiki/Change_data_capture

Question: 293

CertyIQ

Your organization is modernizing their IT services and migrating to Google Cloud. You need to organize the data that will be stored in Cloud Storage and BigQuery. You need to enable a data mesh approach to share the data between sales, product design, and marketing departments. What should you do?

- A.1. Create a project for storage of the data for each of your departments.
- 2. Enable each department to create Cloud Storage buckets and BigQuery datasets.
- 3. Create user groups for authorized readers for each bucket and dataset.
- 4. Enable the IT team to administer the user groups to add or remove users as the departments' request.
- B.1. Create multiple projects for storage of the data for each of your departments' applications.
- 2. Enable each department to create Cloud Storage buckets and BigQuery datasets.
- 3. Publish the data that each department shared in Analytics Hub.
- 4. Enable all departments to discover and subscribe to the data they need in Analytics Hub.
- C.1. Create a project for storage of the data for your organization.
- 2. Create a central Cloud Storage bucket with three folders to store the files for each department.
- 3. Create a central BigQuery dataset with tables prefixed with the department name.
- 4. Give viewer rights for the storage project for the users of your departments.
- D.1. Create multiple projects for storage of the data for each of your departments' applications.
- 2. Enable each department to create Cloud Storage buckets and BigQuery datasets.
- 3. In Dataplex, map each department to a data lake and the Cloud Storage buckets, and map the BigQuery datasets to zones.
- 4. Enable each department to own and share the data of their data lakes.

Answer: D

Explanation:

The correct answer is **D**, and here's why:

Option D aligns perfectly with the principles of a data mesh architecture. It promotes domain-driven ownership by creating separate projects for each department's applications, allowing them to manage their own data storage within their respective buckets and datasets. This decentralization of data ownership and management is a core tenet of a data mesh. Dataplex facilitates the organization and governance of these decentralized data assets by mapping departments to data lakes and zones, ensuring discoverability and enabling self-service access. This allows departments to own and manage their data, and share data through the platform.

Option A fails to promote autonomy, making the IT team a bottleneck for user access management. This contradicts the self-service principle of a data mesh. Option B introduces Analytics Hub, which isn't the primary tool for building a data mesh and adds unnecessary complexity. Option C centralizes everything into a single project, negating the idea of domain ownership and thus failing to establish a data mesh approach. Data

mesh aims for federated data governance and domain ownership. Option D achieves this by giving the different teams ownership of their data. Dataplex is the correct tool to enable the cataloging of this data.

Here are some links for further research:

Data Mesh: <https://martinfowler.com/articles/data-mesh-principles.html>

Google Cloud Dataplex: <https://cloud.google.com/dataplex/docs>

Data Mesh on Google Cloud: <https://cloud.google.com/blog/topics/data-analytics/building-a-data-mesh-on-google-cloud>

Question: 294

CertyIQ

You work for a large ecommerce company. You are using Pub/Sub to ingest the clickstream data to Google Cloud for analytics. You observe that when a new subscriber connects to an existing topic to analyze data, they are unable to subscribe to older data. For an upcoming yearly sale event in two months, you need a solution that, once implemented, will enable any new subscriber to read the last 30 days of data. What should you do?

- A. Create a new topic, and publish the last 30 days of data each time a new subscriber connects to an existing topic.
- B. Set the topic retention policy to 30 days.
- C. Set the subscriber retention policy to 30 days.
- D. Ask the source system to re-push the data to Pub/Sub, and subscribe to it.

Answer: B

Explanation:

The correct answer is **B. Set the topic retention policy to 30 days.**

Here's the justification:

Pub/Sub, by default, doesn't retain published messages indefinitely. Once a message has been successfully acknowledged by all subscribers, it's typically deleted from the topic. However, for use cases requiring historical data access, Pub/Sub offers the feature of topic retention. Setting a retention policy on a topic enables messages to be stored for a specified duration, regardless of subscriber acknowledgement.

In this scenario, the requirement is to allow new subscribers to access the last 30 days of clickstream data. This means the data needs to be preserved on the topic itself. By configuring the topic's retention policy to 30 days, Pub/Sub will retain messages for that period, making them available to new subscribers joining the topic within those 30 days. This addresses the need for accessing historical data.

Option A is inefficient and impractical, as it would involve manually re-publishing data each time a new subscriber joins. Option C is incorrect; subscriber retention is not a standard Pub/Sub feature, and it is the topic where the data is stored. Option D requires the source to have access to the old data and the capability to re-push it, which may be impossible or very complex.

Therefore, setting the topic retention policy to 30 days is the most straightforward and efficient solution to enable new subscribers to access the required historical data. This aligns with the Pub/Sub best practices for data retention.

Further research:

Pub/Sub Topic Retention: https://cloud.google.com/pubsub/docs/publisher#message_retention

Pub/Sub Concepts: <https://cloud.google.com/pubsub/docs/overview>

Question: 295

You are designing the architecture to process your data from Cloud Storage to BigQuery by using Dataflow. The network team provided you with the Shared VPC network and subnetwork to be used by your pipelines. You need to enable the deployment of the pipeline on the Shared VPC network. What should you do?

- A. Assign the `compute.networkUser` role to the Dataflow service agent.
- B. Assign the `compute.networkUser` role to the service account that executes the Dataflow pipeline.
- C. Assign the `dataflow.admin` role to the Dataflow service agent.
- D. Assign the `dataflow.admin` role to the service account that executes the Dataflow pipeline.

Answer: A

Explanation:

The correct answer is **A. Assign the `compute.networkUser` role to the Dataflow service agent.**

Here's a detailed justification:

When deploying Dataflow pipelines within a Shared VPC network, the Dataflow service agent, which manages the underlying resources on Google Cloud, requires specific permissions to interact with that network. The `compute.networkUser` role grants the necessary rights to use and connect to the specified network and subnet. This role allows the Dataflow service agent to create and manage virtual machine instances (workers) within the Shared VPC, effectively enabling data processing within the shared network environment. It doesn't grant any permissions to create or modify network resources but simply allows the Dataflow agent to use an already existing network.

The Dataflow service agent, rather than the user-provided service account running the pipeline, is responsible for establishing the connection to the Shared VPC network. Therefore, granting the `compute.networkUser` role to the service account used for the pipeline (option B) will not enable the pipeline to be deployed on the Shared VPC network. The `dataflow.admin` role (options C and D) deals with Dataflow itself, granting permission to administer Dataflow resources, and has nothing to do with VPC network permissions, hence are incorrect. The core requirement is about having permissions to utilize a specific network which the service agent requires and is provided by `compute.networkUser` role.

In summary, the Dataflow service agent needs the `compute.networkUser` role on the Shared VPC to successfully provision resources, while the service account for the pipeline does not require this permission.

Authoritative Links:

1. **Granting access to Shared VPC:** https://cloud.google.com/vpc/docs/shared-vpc#granting_access
2. **Dataflow security and permissions:** <https://cloud.google.com/dataflow/docs/concepts/security-and-permissions#permissions-for-using-shared-vpc>
3. **`compute.networkUser` role:** <https://cloud.google.com/iam/docs/understanding-roles#compute.networkUser>

Question: 296

Your infrastructure team has set up an interconnect link between Google Cloud and the on-premises network. You are designing a high-throughput streaming pipeline to ingest data in streaming from an Apache Kafka cluster hosted on-premises. You want to store the data in BigQuery, with as minimal latency as possible. What should you do?

- A. Setup a Kafka Connect bridge between Kafka and Pub/Sub. Use a Google-provided Dataflow template to read the data from Pub/Sub, and write the data to BigQuery.

B. Use a proxy host in the VPC in Google Cloud connecting to Kafka. Write a Dataflow pipeline, read data from the proxy host, and write the data to BigQuery.

C. Use Dataflow, write a pipeline that reads the data from Kafka, and writes the data to BigQuery.

D. Setup a Kafka Connect bridge between Kafka and Pub/Sub. Write a Dataflow pipeline, read the data from Pub/Sub, and write the data to BigQuery.

Answer: C

Explanation:

Option C is the most suitable solution for minimizing latency in this high-throughput streaming pipeline. Dataflow's capability to directly connect to Kafka allows for a streamlined data ingestion process. Instead of introducing an intermediary like Pub/Sub, which adds latency through an additional hop, Dataflow directly reads from Kafka over the interconnect link, leveraging its high-throughput capabilities. This minimizes the number of data transfer steps and potential bottlenecks, leading to lower latency and improved performance. Dataflow is also optimized for parallel processing, enabling it to handle the high volume of data from Kafka efficiently. It also has built-in support for writing directly to BigQuery. Options A and D introduce Pub/Sub as a middleman, adding an unnecessary extra step and thus latency. Option B introduces complexity by requiring a proxy host, which can be prone to performance issues and adds operational overhead. Therefore, Dataflow's direct connection to Kafka simplifies the architecture and provides the lowest latency solution.

[Apache Kafka Connect](#)[Google Dataflow](#)[Google BigQuery](#)[Google Cloud Interconnect](#)

Question: 297

CertyIQ

You migrated your on-premises Apache Hadoop Distributed File System (HDFS) data lake to Cloud Storage. The data scientist team needs to process the data by using Apache Spark and SQL. Security policies need to be enforced at the column level. You need a cost-effective solution that can scale into a data mesh. What should you do?

A.1. Deploy a long-living Dataproc cluster with Apache Hive and Ranger enabled.
2. Configure Ranger for column level security.
3. Process with Dataproc Spark or Hive SQL.

B.1. Define a BigLake table.
2. Create a taxonomy of policy tags in Data Catalog.
3. Add policy tags to columns.
4. Process with the Spark-BigQuery connector or BigQuery SQL.

C.1. Load the data to BigQuery tables.
2. Create a taxonomy of policy tags in Data Catalog.
3. Add policy tags to columns.
4. Process with the Spark-BigQuery connector or BigQuery SQL.

D.1. Apply an Identity and Access Management (IAM) policy at the file level in Cloud Storage.
2. Define a BigQuery external table for SQL processing.
3. Use Dataproc Spark to process the Cloud Storage files.

Answer: B

Explanation:

The optimal solution leverages BigQuery and Data Catalog for cost-effective, scalable, and secure data processing in a data mesh architecture. Option B uses BigLake tables, policy tags in Data Catalog, and Spark-BigQuery connector or BigQuery SQL to achieve this. BigLake offers a unified query interface for both structured and unstructured data across storage formats, while Data Catalog provides centralized metadata management and data governance, including column-level security through policy tags. This addresses the security requirements at the column level. Using BigQuery and Cloud Storage directly is more cost-effective

in the long run than a long-living Dataproc cluster (Option A). IAM policies at the file level (Option D) are cumbersome and don't provide fine-grained, column-level control, and don't easily scale in a data mesh. Loading the data to BigQuery first as in Option C, while viable, loses the advantage of direct querying using BigLake and adds an unnecessary ETL step, also BigLake tables still allow data to reside in Cloud Storage, optimizing for storage costs and access patterns. The Spark-BigQuery connector enables seamless integration between Spark and BigQuery for powerful data processing capabilities. This strategy allows the data scientist team to use both Spark and SQL while enforcing the required security policies.

Authoritative Links:

BigLake: <https://cloud.google.com/bigquery/docs/introduction-biglake>

Data Catalog Policy Tags: <https://cloud.google.com/data-catalog/docs/how-to/access-control/policy-tags>

Spark-BigQuery Connector: <https://cloud.google.com/dataproc/docs/concepts/connectors/spark-bigquery>

Question: 298

CertyIQ

One of your encryption keys stored in Cloud Key Management Service (Cloud KMS) was exposed. You need to re-encrypt all of your CMEK-protected Cloud Storage data that used that key, and then delete the compromised key. You also want to reduce the risk of objects getting written without customer-managed encryption key (CMEK) protection in the future. What should you do?

- A. Rotate the Cloud KMS key version. Continue to use the same Cloud Storage bucket.
- B. Create a new Cloud KMS key. Set the default CMEK key on the existing Cloud Storage bucket to the new one.
- C. Create a new Cloud KMS key. Create a new Cloud Storage bucket. Copy all objects from the old bucket to the new one bucket while specifying the new Cloud KMS key in the copy command.
- D. Create a new Cloud KMS key. Create a new Cloud Storage bucket configured to use the new key as the default CMEK key. Copy all objects from the old bucket to the new bucket without specifying a key.

Answer: D

Explanation:

The correct answer is **D**. Here's a breakdown of why and how it addresses the prompt's requirements:

Compromised Key Remediation: The first critical step is to stop using the compromised key. Creating a new Cloud KMS key immediately isolates the risk. Rotating the key version (option A) only mitigates the immediate vulnerability of the exposed version but doesn't address potential past compromises or vulnerabilities associated with the key itself.

Re-encryption: Re-encrypting data with a new key is essential to protect against unauthorized access using the exposed key. Creating a new bucket (as suggested in Options C and D) provides a clean environment for re-encrypting data during the transfer process.

Default CMEK: Setting a default CMEK key on the new Cloud Storage bucket (as suggested in Option D) enforces encryption on all newly written objects. By setting it at the bucket level, no individual object uploaded without an explicitly stated key will be saved without encryption, addressing the requirement to reduce the risk of objects being written without CMEK protection. Option B correctly establishes a new KMS key and default CMEK, but the new key is set on the existing bucket. Re-encrypting the old data would still need to be performed on the original bucket, increasing complexity.

Copy Process: Copying objects without specifying the key (as in Option D) to a bucket with a default CMEK uses the default key for encryption. This avoids the risk of accidentally re-encrypting objects with the compromised key. Option C specifies the new KMS key in the copy command. While this works, it isn't required given a default KMS key set on the destination bucket and adds an extra step.

Security Best Practice: Completely isolating the compromised key and utilizing a new Cloud Storage bucket ensures a clean separation and minimizes potential lingering security risks associated with the original infrastructure.

Therefore, creating a new Cloud KMS key, creating a new bucket with the new key as the default CMEK key, and copying objects to the new bucket is the safest and most compliant approach.

Authoritative Links:

Customer-managed encryption keys (CMEK): <https://cloud.google.com/storage/docs/encryption/customer-managed-keys>

Cloud KMS: <https://cloud.google.com/kms/docs>

Setting Default Bucket Encryption: https://cloud.google.com/storage/docs/encryption/using-customer-managed-keys#setting_the_default_cmek_key_on_a_bucket

Question: 299

CertyIQ

You have an upstream process that writes data to Cloud Storage. This data is then read by an Apache Spark job that runs on Dataproc. These jobs are run in the us-central1 region, but the data could be stored anywhere in the United States. You need to have a recovery process in place in case of a catastrophic single region failure. You need an approach with a maximum of 15 minutes of data loss (RPO=15 mins). You want to ensure that there is minimal latency when reading the data. What should you do?

- A.1. Create two regional Cloud Storage buckets, one in the us-central1 region and one in the us-south1 region.
- 2. Have the upstream process write data to the us-central1 bucket. Use the Storage Transfer Service to copy data hourly from the us-central1 bucket to the us-south1 bucket.
- 3. Run the Dataproc cluster in a zone in the us-central1 region, reading from the bucket in that region.
- 4. In case of regional failure, redeploy your Dataproc clusters to the us-south1 region and read from the bucket in that region instead.
- B.1. Create a Cloud Storage bucket in the US multi-region.
- 2. Run the Dataproc cluster in a zone in the us-central1 region, reading data from the US multi-region bucket.
- 3. In case of a regional failure, redeploy the Dataproc cluster to the us-central2 region and continue reading from the same bucket.
- C.1. Create a dual-region Cloud Storage bucket in the us-central1 and us-south1 regions.
- 2. Enable turbo replication.
- 3. Run the Dataproc cluster in a zone in the us-central1 region, reading from the bucket in the us-south1 region.
- 4. In case of a regional failure, redeploy your Dataproc cluster to the us-south1 region and continue reading from the same bucket.
- D.1. Create a dual-region Cloud Storage bucket in the us-central1 and us-south1 regions.
- 2. Enable turbo replication.
- 3. Run the Dataproc cluster in a zone in the us-central1 region, reading from the bucket in the same region.
- 4. In case of a regional failure, redeploy the Dataproc clusters to the us-south1 region and read from the same bucket.

Answer: D

Explanation:

The correct answer is **D**, and here's a detailed justification:

Option D provides the most suitable solution for achieving both low latency and an RPO of 15 minutes in the context of region failures. Creating a dual-region Cloud Storage bucket (us-central1 and us-south1) with turbo replication guarantees data redundancy and fast replication between the two regions. This addresses the RPO requirement by ensuring near real-time copying of the data to the secondary region.

Initially running the Dataproc cluster in us-central1 and reading from the same region ensures low latency for most operations because the data is close to the compute. If a regional failure occurs, the Dataproc cluster can be swiftly redeployed to us-south1 without requiring changes in the bucket location, and will begin

processing data from the replicated data which was synchronously copied. The turbo replication setting ensures minimal lag between data writes to both regions of the bucket and therefore helps meet the 15 minute RPO requirement.

The other options present some issues:

Option A uses the Storage Transfer Service for hourly copies, which violates the RPO of 15 minutes. This is also suboptimal because a full copy is required when changes are already stored.

Option B utilizes a US multi-region bucket, which is great for availability but would not provide low latency if the Dataproc clusters are in us-central1. Also there is no synchronous replication guarantee for a multi-region bucket, which could result in delayed writes in another region and may not meet the 15 min RPO.

Option C initially reads from the secondary region (us-south1), resulting in unnecessary latency when the Dataproc cluster is in us-central1.

Authoritative Links for Further Research:

Cloud Storage dual-region buckets: <https://cloud.google.com/storage/docs/locations#dual-regions>

Cloud Storage Turbo Replication: <https://cloud.google.com/storage/docs/turbo-replication>

Dataproc: <https://cloud.google.com/dataproc/>

Understanding RPO: <https://cloud.google.com/blog/products/gcp/understanding-recovery-time-objective-rto-and-recovery-point-objective-rpo>

Question: 300

CertyIQ

You currently have transactional data stored on-premises in a PostgreSQL database. To modernize your data environment, you want to run transactional workloads and support analytics needs with a single database. You need to move to Google Cloud without changing database management systems, and minimize cost and complexity. What should you do?

- A. Migrate and modernize your database with Cloud Spanner.
- B. Migrate your workloads to AlloyDB for PostgreSQL.
- C. Migrate to BigQuery to optimize analytics.
- D. Migrate your PostgreSQL database to Cloud SQL for PostgreSQL.

Answer: D

Explanation:

The correct answer is D, migrating the on-premises PostgreSQL database to Cloud SQL for PostgreSQL. This solution directly addresses the requirement of maintaining the same database management system (PostgreSQL) while transitioning to Google Cloud. Cloud SQL for PostgreSQL is a fully managed service, significantly reducing operational overhead and complexity compared to self-managing a database instance on Google Compute Engine. Option A, using Cloud Spanner, introduces a different database system, which violates the requirement of not changing the database management system. Option B, AlloyDB for PostgreSQL, is a newer, advanced option that may provide enhanced performance, but it might not be the simplest and most cost-effective solution for basic migration needs as stated in the prompt which mentions minimizing cost and complexity. It's also not a direct one-to-one migration as Cloud SQL would be. Option C, migrating to BigQuery, is not appropriate for transactional workloads; BigQuery is optimized for analytics, not for handling real-time transactions. Cloud SQL for PostgreSQL, in contrast, is designed for both transactional and operational purposes, fulfilling the prompt's requirements. This approach offers a lift-and-shift strategy, allowing a more seamless and less complex migration process with fewer changes for the existing application.

[Cloud SQL for PostgreSQL](#) [Cloud SQL vs self-managed Database Migration Strategies](#)

Question: 301

CertyIQ

You are architecting a data transformation solution for BigQuery. Your developers are proficient with SQL and want to use the ELT development technique. In addition, your developers need an intuitive coding environment and the ability to manage SQL as code. You need to identify a solution for your developers to build these pipelines. What should you do?

- A. Use Dataform to build, manage, and schedule SQL pipelines.
- B. Use Dataflow jobs to read data from Pub/Sub, transform the data, and load the data to BigQuery.
- C. Use Data Fusion to build and execute ETL pipelines.
- D. Use Cloud Composer to load data and run SQL pipelines by using the BigQuery job operators.

Answer: A

Explanation:

The correct answer is A, using Dataform. Here's why:

Dataform is explicitly designed for SQL-based data transformations within BigQuery. It provides a version-controlled environment where developers can write, test, and manage their SQL code as code, fulfilling the requirement of managing SQL as code. Dataform's workflow is centered around the ELT (Extract, Load, Transform) paradigm, which aligns perfectly with the stated preference for ELT. The intuitive coding environment of Dataform, integrated with the BigQuery context, makes it easy for developers proficient in SQL to quickly create and deploy pipelines. Furthermore, Dataform includes features for managing dependencies, scheduling transformations, and testing SQL code, making it a complete solution for the needs described.

Dataflow (option B) is more suitable for stream and batch processing using a flexible, code-based environment (typically Java or Python), which doesn't align with the preference for SQL-based transformations. Data Fusion (option C) is a UI-driven ETL tool, not explicitly for SQL-based ELT. Cloud Composer (option D) is an orchestration service, not a specific tool for writing and managing SQL-based transformations like Dataform; while it can trigger BigQuery jobs, it doesn't provide the development environment and SQL as code management features of Dataform.

In summary, Dataform's focus on SQL, ELT workflow, version control, and intuitive coding environment makes it the optimal choice for developers who want to create data transformation pipelines in BigQuery using SQL as code.

Further Research:

Dataform Overview: <https://cloud.google.com/dataform/docs/overview>

ELT with Dataform: <https://cloud.google.com/dataform/docs/elt-dataform>

Dataform Concepts: <https://cloud.google.com/dataform/docs/concepts>

Question: 302

CertyIQ

You work for a farming company. You have one BigQuery table named sensors, which is about 500 MB and contains the list of your 5000 sensors, with columns for id, name, and location. This table is updated every hour. Each sensor generates one metric every 30 seconds along with a timestamp, which you want to store in BigQuery. You want to run an analytical query on the data once a week for monitoring purposes. You also want to minimize costs. What data model should you use?

- A.1. Create a metrics column in the sensors table.
2. Set RECORD type and REPEATED mode for the metrics column.

3. Use an UPDATE statement every 30 seconds to add new metrics.

B.1. Create a metrics column in the sensors table.

2. Set RECORD type and REPEATED mode for the metrics column.

3. Use an INSERT statement every 30 seconds to add new metrics.

C.1. Create a metrics table partitioned by timestamp.

2. Create a sensorid column in the metrics table, that points to the id column in the sensors table.

3. Use an INSERT statement every 30 seconds to append new metrics to the metrics table.

4. Join the two tables, if needed, when running the analytical query.

D.1. Create a metrics table partitioned by timestamp.

2. Create a sensorid column in the metrics table, which points to the id column in the sensors table.

3. Use an UPDATE statement every 30 seconds to append new metrics to the metrics table.

4. Join the two tables, if needed, when running the analytical query.

Answer: C

Explanation:

Option C is the most appropriate data modeling strategy for this scenario due to its scalability, cost-effectiveness, and suitability for analytical workloads. Creating a separate metrics table, partitioned by timestamp, allows for efficient querying and reduces costs by only scanning relevant data when needed. The sensorid column in the metrics table establishes a relationship with the sensors table using a foreign key, enabling joins for comprehensive analysis. Using INSERT statements every 30 seconds appends new metrics without modifying existing data, which aligns with best practices for handling time-series data. This approach avoids the performance and cost overhead of frequent updates to a single table, as suggested in options A, B, and D. Option D is incorrect due to the use of UPDATE, which is less efficient than INSERT for appending new data, particularly in BigQuery. Options A and B propose storing metrics as a repeated record within the sensors table, which is inefficient and not recommended because it would require frequent updates and makes querying and analysing time-series data difficult, and cause BigQuery table schema limitations. The separate, partitioned metrics table approach allows for better performance when running the weekly analytical query, while minimizing costs associated with scanning and updating.

Here are some authoritative links for further research:

BigQuery Partitioned Tables: <https://cloud.google.com/bigquery/docs/partitioned-tables>

BigQuery Best Practices for Performance and Cost Optimization:

<https://cloud.google.com/bigquery/docs/best-practices-performance>

BigQuery Data Modeling Best Practices: <https://cloud.google.com/bigquery/docs/data-modeling>

Time Series Data in BigQuery: <https://cloud.google.com/blog/products/data-analytics/time-series-data-analysis-in-bigquery>

Question: 303

CertyIQ

You are managing a Dataplex environment with raw and curated zones. A data engineering team is uploading JSON and CSV files to a bucket asset in the curated zone but the files are not being automatically discovered by Dataplex. What should you do to ensure that the files are discovered by Dataplex?

A. Move the JSON and CSV files to the raw zone.

B. Enable auto-discovery of files for the curated zone.

C. Use the bg command-line tool to load the JSON and CSV files into BigQuery tables.

D. Grant object level access to the CSV and JSON files in Cloud Storage.

Answer: A

Explanation:

The correct answer is A. Here's why:

Dataplex utilizes a concept of zones, and each zone has a specific intended purpose within the data lifecycle. Raw zones typically serve as the initial landing point for data, while curated zones hold transformed, refined, and quality-assured data. Dataplex's auto-discovery feature is often configured to scan raw zones for new data to be cataloged. Directly placing data into the curated zone bypasses this initial discovery process.

The core issue is that the data is not being discovered in the curated zone, indicating that the auto-discovery mechanism is likely targeting the raw zone instead. Option B, enabling auto-discovery for the curated zone, may seem intuitive; however, it isn't typically the intended workflow. Raw zones are expected to be the source for auto-discovery as a best practice, and the movement to the curated zone is triggered by processing. Using the raw zone will ensure data is detected and cataloged. Options C and D are not pertinent to resolving the issue of automatic data discovery by Dataplex. Loading into BigQuery (C) does not address the Dataplex discovery problem. Object-level access (D) is a separate concern, primarily related to permissions and wouldn't directly impact the discovery.

Therefore, moving the JSON and CSV files to the raw zone will allow Dataplex's configured auto-discovery process to recognize the new data, catalog it appropriately, and enable downstream actions. This also follows the intended pattern of data movement through the Dataplex environment: raw zone first, processing second, curated zone third.

Authoritative Links:

Dataplex documentation on Zones: <https://cloud.google.com/dataplex/docs/concepts/zones>

Dataplex documentation on data discovery: <https://cloud.google.com/dataplex/docs/auto-data-discovery>

Question: 304

CertyIQ

You have a table that contains millions of rows of sales data, partitioned by date. Various applications and users query this data many times a minute. The query requires aggregating values by using AVG, MAX, and SUM, and does not require joining to other tables. The required aggregations are only computed over the past year of data, though you need to retain full historical data in the base tables. You want to ensure that the query results always include the latest data from the tables, while also reducing computation cost, maintenance overhead, and duration. What should you do?

- A. Create a materialized view to aggregate the base table data. Include a filter clause to specify the last one year of partitions.
- B. Create a materialized view to aggregate the base table data. Configure a partition expiration on the base table to retain only the last one year of partitions.
- C. Create a view to aggregate the base table data. Include a filter clause to specify the last year of partitions.
- D. Create a new table that aggregates the base table data. Include a filter clause to specify the last year of partitions. Set up a scheduled query to recreate the new table every hour.

Answer: A

Explanation:

The correct answer is **A. Create a materialized view to aggregate the base table data. Include a filter clause to specify the last one year of partitions.**

Here's why:

Materialized views (MVs) in BigQuery are pre-computed results of a query, offering significant performance improvements for frequently executed aggregation queries. They automatically update when the base tables change, ensuring the data is always current. This addresses the requirement for latest data and reduces computation time. Option A's filter clause on the last year of partitions further optimizes the MV by focusing

on the relevant data, reducing storage and processing costs.

Options B, C, and D are less suitable:

B: While partition expiration on the base table reduces storage cost, it loses historical data, violating the requirement to retain full history. Also, a materialized view will handle data update, the expiration is not a good approach.

C: A standard view is a virtual table based on a query, which is executed each time you query it. Therefore, it doesn't reduce query duration, which is a key requirement.

D: Creating and regularly recreating a new table introduces complexity in maintenance with its associated scheduled queries, cost and doesn't provide the same automatic update as MV.

In essence, materialized views are designed for the exact use case described in the question - frequently querying aggregated data that needs to be updated continuously with the source data. Their ability to incrementally update efficiently makes them ideal.

Authoritative Links for Further Research:

BigQuery Materialized Views: <https://cloud.google.com/bigquery/docs/materialized-views>

BigQuery Partitioned Tables: <https://cloud.google.com/bigquery/docs/partitioned-tables>

BigQuery Views: <https://cloud.google.com/bigquery/docs/views>

Question: 305

CertyIQ

Your organization uses a multi-cloud data storage strategy, storing data in Cloud Storage, and data in Amazon Web Services' (AWS) S3 storage buckets. All data resides in US regions. You want to query up-to-date data by using BigQuery, regardless of which cloud the data is stored in. You need to allow users to query the tables from BigQuery without giving direct access to the data in the storage buckets. What should you do?

- A. Setup a BigQuery Omni connection to the AWS S3 bucket data. Create BigLake tables over the Cloud Storage and S3 data and query the data using BigQuery directly.
- B. Set up a BigQuery Omni connection to the AWS S3 bucket data. Create external tables over the Cloud Storage and S3 data and query the data using BigQuery directly.
- C. Use the Storage Transfer Service to copy data from the AWS S3 buckets to Cloud Storage buckets. Create BigLake tables over the Cloud Storage data and query the data using BigQuery directly.
- D. Use the Storage Transfer Service to copy data from the AWS S3 buckets to Cloud Storage buckets. Create external tables over the Cloud Storage data and query the data using BigQuery directly.

Answer: A

Explanation:

Here's a detailed justification for why option A is the correct answer:

Option A is the most suitable approach because it directly addresses the requirement of querying data across both Cloud Storage and AWS S3 buckets within BigQuery without granting direct storage access. It leverages BigQuery Omni for accessing S3 data and BigLake tables for unifying access. BigQuery Omni allows BigQuery to query data residing in other cloud providers, like AWS, without needing to copy or move it. This fulfills the multi-cloud query requirement. By establishing a BigQuery Omni connection to the S3 bucket, you enable BigQuery to directly access and query the data. Simultaneously, you would create BigLake tables that act as an abstraction layer over data in both Cloud Storage and S3. These BigLake tables allow BigQuery to treat both data sources as unified tables for querying. This way, users can query data seamlessly through BigQuery using SQL, and their access is controlled at the BigLake table level, not at the storage level, thus meeting the security requirement. Options B, C, and D are incorrect. Option B uses external tables, which do not offer the same level of security and abstraction as BigLake tables, and external tables are not designed to be cross-cloud. Options C and D involve copying data from AWS S3 to Cloud Storage using Storage Transfer Service,

which is not ideal since it requires data movement and management overhead, and it will make the data not up-to-date as the question required.

Here are authoritative links for further research:

BigQuery Omni: <https://cloud.google.com/bigquery/docs/omni-introduction>

BigLake Tables: <https://cloud.google.com/bigquery/docs/biglake-introduction>

External Tables in BigQuery: <https://cloud.google.com/bigquery/docs/external-data-sources>

Storage Transfer Service: <https://cloud.google.com/storage-transfer-service/docs>

Question: 306

CertyIQ

You are preparing an organization-wide dataset. You need to preprocess customer data stored in a restricted bucket in Cloud Storage. The data will be used to create consumer analyses. You need to comply with data privacy requirements.

What should you do?

- A. Use Dataflow and the Cloud Data Loss Prevention API to mask sensitive data. Write the processed data in BigQuery.
- B. Use customer-managed encryption keys (CMEK) to directly encrypt the data in Cloud Storage. Use federated queries from BigQuery. Share the encryption key by following the principle of least privilege.
- C. Use the Cloud Data Loss Prevention API and Dataflow to detect and remove sensitive fields from the data in Cloud Storage. Write the filtered data in BigQuery.
- D. Use Dataflow and Cloud KMS to encrypt sensitive fields and write the encrypted data in BigQuery. Share the encryption key by following the principle of least privilege.

Answer: A

Explanation:

The correct answer is **A. Use Dataflow and the Cloud Data Loss Prevention API to mask sensitive data. Write the processed data in BigQuery.** This approach effectively addresses both data preprocessing and privacy requirements.

Dataflow, a fully managed stream and batch data processing service, is ideal for transforming large datasets. The Cloud Data Loss Prevention (DLP) API provides powerful tools to identify and mask sensitive information like Personally Identifiable Information (PII) within the data. This masking ensures that sensitive information is not directly accessible, complying with privacy requirements.

By processing the data with Dataflow and the DLP API, the customer data can be safely transformed before being written to BigQuery, a fully managed, scalable data warehouse service for analysis. BigQuery is suitable for further analysis and will not contain personally identifiable information.

Option B is problematic because encrypting data in storage does not inherently mask sensitive data. Federated queries also do not resolve the sensitivity issue, and sharing encryption keys raises security concerns if not done correctly.

Option C, while using DLP, proposes removing sensitive fields. This can be problematic as context might be lost by removing these fields. Masking, as in option A, retains the data for analytical purposes but hides the sensitive content.

Option D suggests encrypting sensitive fields with Cloud KMS, but this requires key management complexities on a granular level and is not the standard approach. Masking with DLP is more appropriate for anonymizing data. Moreover, the focus is on data privacy, not encryption per se.

Data privacy requirements are best addressed by masking sensitive data, which is what Dataflow and DLP API

accomplish effectively in option A. **Authoritative Links:**

Dataflow: <https://cloud.google.com/dataflow/docs>

Cloud Data Loss Prevention (DLP) API: <https://cloud.google.com/dlp/docs>

BigQuery: <https://cloud.google.com/bigquery/docs>

Data Privacy on Google Cloud: <https://cloud.google.com/security/privacy>

Question: 307

CertyIQ

You need to connect multiple applications with dynamic public IP addresses to a Cloud SQL instance. You configured users with strong passwords and enforced the SSL connection to your Cloud SQL instance. You want to use Cloud SQL public IP and ensure that you have secured connections. What should you do?

- A. Add CIDR 0.0.0.0/0 network to Authorized Network. Use Identity and Access Management (IAM) to add users.
- B. Add all application networks to Authorized Network and regularly update them.
- C. Leave the Authorized Network empty. Use Cloud SQL Auth proxy on all applications.
- D. Add CIDR 0.0.0.0/0 network to Authorized Network. Use Cloud SQL Auth proxy on all applications.

Answer: C

Explanation:

The correct answer is **C. Leave the Authorized Network empty. Use Cloud SQL Auth proxy on all applications.** Here's why:

Cloud SQL's Authorized Networks feature controls which IP addresses can directly connect to the database instance. Using 0.0.0.0/0 (option A and D) would expose the database to the entire internet, creating a severe security risk, despite having SSL and strong passwords. Option B is impractical and error-prone; manually updating authorized networks for dynamic public IPs is complex and doesn't scale. The most secure and manageable approach is to avoid adding any public IP addresses to the authorized networks and utilize the Cloud SQL Auth proxy.

The Cloud SQL Auth proxy creates a secure, encrypted tunnel between the application and the database instance. It handles authentication and authorization transparently, ensuring that only connections using the proxy can reach the Cloud SQL instance. This removes the need to manage public IP addresses for multiple, potentially changing, applications. The proxy relies on Google Cloud IAM for authentication and authorization, further enhancing security. The proxy also manages the SSL connection, ensuring encryption and secure transit. Leaving the Authorized Networks empty prevents direct access, limiting the attack surface and ensuring that all connections are established via the secured proxy. This allows secure access even when using Cloud SQL public IP without the risk of direct public connections.

Authoritative Links:

Cloud SQL Auth Proxy: <https://cloud.google.com/sql/docs/mysql/connect-auth-proxy>

Cloud SQL Authorized Networks: <https://cloud.google.com/sql/docs/mysql/configure-ip>

Understanding Network Security in GCP: <https://cloud.google.com/vpc/docs/firewalls>

Question: 308

CertyIQ

You are migrating a large number of files from a public HTTPS endpoint to Cloud Storage. The files are protected from unauthorized access using signed URLs. You created a TSV file that contains the list of object URLs and started a transfer job by using Storage Transfer Service. You notice that the job has run for a long time and eventually failed. Checking the logs of the transfer job reveals that the job was running fine until one point, and then it failed due to HTTP 403 errors on the remaining files. You verified that there were no changes to the source

system. You need to fix the problem to resume the migration process. What should you do?

- A. Set up Cloud Storage FUSE, and mount the Cloud Storage bucket on a Compute Engine instance. Remove the completed files from the TSV file. Use a shell script to iterate through the TSV file and download the remaining URLs to the FUSE mount point.
- B. Renew the TLS certificate of the HTTPS endpoint. Remove the completed files from the TSV file and rerun the Storage Transfer Service job.
- C. Create a new TSV file for the remaining files by generating signed URLs with a longer validity period. Split the TSV file into multiple smaller files and submit them as separate Storage Transfer Service jobs in parallel.
- D. Update the file checksums in the TSV file from using MD5 to SHA256. Remove the completed files from the TSV file and rerun the Storage Transfer Service job.

Answer: C

Explanation:

The core issue is the 403 errors, indicating authorization problems with the signed URLs. Signed URLs have a limited validity period. Option C correctly identifies this as the root cause. By creating new signed URLs with a longer validity period, we address the authorization failure. Splitting the large transfer job into smaller parallel jobs reduces the chance that any single job runs longer than the URL's validity and also makes the process more efficient by utilizing cloud infrastructure for parallel execution. This approach ensures files are transferred before signed URLs expire and offers better job manageability. Options A and D do not solve the 403 errors related to expired signed URLs. Option B focuses on TLS certificate problems, which are unrelated to the problem of expired signed URLs. Option A introduces complexity using FUSE and a script for a task that STS is designed to do. Option D addresses file checksums, which do not cause 403 errors related to signed URLs.

Relevant concepts:

Signed URLs: Provide temporary access to cloud storage resources.

<https://cloud.google.com/storage/docs/access-control/signed-urls>

Storage Transfer Service: Managed service for migrating data to and from Cloud Storage.

<https://cloud.google.com/storage-transfer-service/docs>

HTTP 403 Error: Indicates unauthorized access to a resource. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status/403>

Job splitting/Parallel execution: Optimizes performance and reduces the likelihood of failures due to time-outs.

Question: 309

CertyIQ

You work for an airline and you need to store weather data in a BigQuery table. Weather data will be used as input to a machine learning model. The model only uses the last 30 days of weather data. You want to avoid storing unnecessary data and minimize costs. What should you do?

- A. Create a BigQuery table where each record has an ingestion timestamp. Run a scheduled query to delete all the rows with an ingestion timestamp older than 30 days.
- B. Create a BigQuery table partitioned by datetime value of the weather date. Set up partition expiration to 30 days.
- C. Create a BigQuery table partitioned by ingestion time. Set up partition expiration to 30 days.
- D. Create a BigQuery table with a datetime column for the day the weather data refers to. Run a scheduled query to delete rows with a datetime value older than 30 days.

Answer: B

Explanation:

The correct answer is **B. Create a BigQuery table partitioned by the datetime value of the weather date. Set up partition expiration to 30 days.**

Here's why:

BigQuery partitioning is crucial for cost optimization and query performance. Partitioning a table based on a relevant datetime column, like the weather data's date, allows BigQuery to efficiently scan only the necessary data during queries. When the model only requires the last 30 days of data, we only need to query that range.

Setting a 30-day partition expiration automatically deletes partitions older than 30 days. This achieves the goal of only storing needed data and further minimizes storage costs and improves query performance.

Option A and D involve using a scheduled query to manually delete older data. This approach, while functional, is less efficient and can be more prone to errors than automated partition expiration. It also involves overhead of the scheduled job. Furthermore, it can incur charges for the deletion query processing.

Option C utilizes ingestion time, but the problem statement requires filtering by the weather data's date, not the time when the data was loaded. Therefore, it wouldn't achieve the desired outcome.

Therefore, partitioning by the weather date column with partition expiration provides the most cost-effective and efficient solution, automatically managing data retention and ensuring only the relevant data is stored and readily available for machine learning.

Authoritative Links:

[BigQuery Partitioned Tables](#): Official documentation on creating and managing partitioned tables.

[BigQuery Partition Expiration](#): Details how to set up automatic expiration for partitions.

[BigQuery Pricing](#): Understanding BigQuery storage and query costs highlights the importance of optimization strategies like partitioning and data expiration.

Question: 310

CertyIQ

You need to look at BigQuery data from a specific table multiple times a day. The underlying table you are querying is several petabytes in size, but you want to filter your data and provide simple aggregations to downstream users. You want to run queries faster and get up-to-date insights quicker. What should you do?

- A. Run a scheduled query to pull the necessary data at specific intervals daily.
- B. Use a cached query to accelerate time to results.
- C. Limit the query columns being pulled in the final result.
- D. Create a materialized view based off of the query being run.

Answer: D

Explanation:

Here's a detailed justification for why creating a materialized view (Option D) is the best approach for this scenario:

The core challenge is repeatedly querying a massive, multi-petabyte BigQuery table while needing filtered and aggregated results for downstream users with speed and timeliness. Materialized views directly address this by pre-computing and storing the results of a query. Unlike a standard view, a materialized view physically stores the data, significantly boosting performance for recurring queries. When the base table changes, BigQuery intelligently updates the materialized view, ensuring data freshness without requiring full table scans each time a user queries. This approach avoids the overhead of processing the entire petabyte table repeatedly and is specifically designed for aggregations and filters, perfectly fitting the described requirements. Cached queries (Option B) help with speed only if the exact same query is run. Option A,

scheduled queries, can provide regular data, but it doesn't optimize query performance and could result in stale data between runs. Option C, limiting columns, is good practice in general but doesn't address the core issue of repeatedly scanning a large table. Materialized views achieve the best balance of speed, freshness, and cost for recurring queries involving aggregation and filtering over large datasets. The user gets faster, pre-aggregated data which optimizes performance.

Further research:

[BigQuery materialized views documentation](#)

[BigQuery performance optimization guide](#)

Question: 311

CertyIQ

Your chemical company needs to manually check documentation for customer order. You use a pull subscription in Pub/Sub so that sales agents get details from the order. You must ensure that you do not process orders twice with different sales agents and that you do not add more complexity to this workflow. What should you do?

- A. Use a Deduplicate PTransform in Dataflow before sending the messages to the sales agents.
- B. Create a transactional database that monitors the pending messages.
- C. Use Pub/Sub exactly-once delivery in your pull subscription.
- D. Create a new Pub/Sub push subscription to monitor the orders processed in the agent's system.

Answer: C

Explanation:

The correct answer is C: Use Pub/Sub exactly-once delivery in your pull subscription. This solution directly addresses the requirement of preventing duplicate order processing by different sales agents without adding unnecessary complexity. Pub/Sub's exactly-once delivery mechanism, when enabled on a pull subscription, guarantees that each message will be delivered to a subscriber only once, even in the face of failures or retries. This eliminates the risk of a message being consumed by multiple agents and therefore avoids duplicate order processing. Option A, using a Deduplicate PTransform in Dataflow, introduces a separate data processing pipeline, which is overkill for this requirement and adds significant complexity. Option B, creating a transactional database, also introduces additional infrastructure and complexity for what Pub/Sub can handle natively with exactly-once delivery. Option D, using a push subscription to monitor processed orders, doesn't directly prevent duplicate processing of orders; it only helps track what has already been consumed and still adds complexity. Exactly-once delivery in Pub/Sub is designed for this exact scenario: ensuring data consistency while maintaining simplicity, thereby meeting the business need while adhering to the constraint of not adding complexity.

Relevant Concepts:

Pub/Sub exactly-once delivery: This feature ensures that each message is delivered to its subscribers only once, preventing duplicate processing. This is crucial for data consistency in many applications.

Pull Subscription: In this subscription type, the subscriber initiates the request to receive messages.

Dataflow: A fully managed, serverless data processing service for batch and streaming data pipelines. While powerful, it adds unnecessary complexity for a simple deduplication requirement.

Transactional Database: A database that ensures ACID (Atomicity, Consistency, Isolation, Durability) properties, which is not needed for this particular Pub/Sub use case.

Push Subscription: In this subscription type, Pub/Sub pushes messages to a subscriber-specified endpoint.

Authoritative Links:

Pub/Sub Exactly-Once Delivery: <https://cloud.google.com/pubsub/docs/exactly-once-delivery>

Question: 312

CertyIQ

You are migrating your on-premises data warehouse to BigQuery. As part of the migration, you want to facilitate cross-team collaboration to get the most value out of the organization's data. You need to design an architecture that would allow teams within the organization to securely publish, discover, and subscribe to read-only data in a self-service manner. You need to minimize costs while also maximizing data freshness. What should you do?

- A. Use Analytics Hub to facilitate data sharing.
- B. Create authorized datasets to publish shared data in the subscribing team's project.
- C. Create a new dataset for sharing in each individual team's project. Grant the subscribing team the `bigquery.dataViewer` role on the dataset.
- D. Use BigQuery Data Transfer Service to copy datasets to a centralized BigQuery project for sharing.

Answer: A

Explanation:

The correct answer is **A. Use Analytics Hub to facilitate data sharing.**

Analytics Hub is a fully managed service in Google Cloud designed specifically for securely sharing data within and outside an organization. It enables a self-service approach to publishing, discovering, and subscribing to datasets, catering perfectly to the scenario's requirements for cross-team collaboration. This contrasts with solutions like authorized datasets (B) which require direct access management between projects, hindering the self-service aspect. Creating individual datasets per team (C) also increases complexity and does not promote discoverability. While BigQuery Data Transfer Service (D) is useful for data ingestion, it's not intended for managed data sharing; it introduces unnecessary data duplication and is less efficient for sharing with multiple teams. Analytics Hub's publisher/subscriber model is optimized for data sharing scenarios, maximizing data freshness since subscribers access the source data directly. Additionally, its fine-grained access control ensures secure and controlled data sharing while minimizing costs by avoiding unnecessary data copies. The managed nature of Analytics Hub reduces operational overhead, making it the most efficient and secure choice for the given scenario.

For further reading on Analytics Hub:

Google Cloud Documentation: <https://cloud.google.com/analytics-hub/docs>

Google Cloud Blog: <https://cloud.google.com/blog/products/data-analytics/introducing-analytics-hub-a-new-way-to-share-data>

Question: 313

CertyIQ

You want to migrate an Apache Spark 3 batch job from on-premises to Google Cloud. You need to minimally change the job so that the job reads from Cloud Storage and writes the result to BigQuery. Your job is optimized for Spark, where each executor has 8 vCPU and 16 GB memory, and you want to be able to choose similar settings. You want to minimize installation and management effort to run your job. What should you do?

- A. Execute the job as part of a deployment in a new Google Kubernetes Engine cluster.
- B. Execute the job from a new Compute Engine VM.
- C. Execute the job in a new Dataproc cluster.
- D. Execute as a Dataproc Serverless job.

Answer: D

Explanation:

The best solution is **D. Execute as a Dataproc Serverless job**. Here's why:

Dataproc Serverless offers a managed, serverless environment for running Spark jobs, eliminating the need for cluster management, which aligns with the requirement to minimize installation and management effort. It allows you to specify executor resources like vCPUs and memory, as the question asks (8 vCPUs and 16 GB memory), ensuring optimal performance. This fits your need to maintain similar configurations to your on-premises setup. Dataproc Serverless integrates seamlessly with Google Cloud Storage (GCS) for reading data and BigQuery for writing results, simplifying the migration process. It also removes the overhead of creating and managing a persistent cluster, providing a more cost-effective solution for batch processing.

Option A, Google Kubernetes Engine (GKE), requires significant setup and management of the Kubernetes cluster and Spark deployment, failing the requirement to minimize management effort. Option B, a Compute Engine VM, involves manual installation of Spark and infrastructure management, again not minimizing effort. Option C, a Dataproc cluster, still requires cluster creation and management, which is more effort than serverless.

Therefore, Dataproc Serverless is the ideal solution because it offers a fully managed environment with simplified Spark execution, allowing you to focus on your job logic rather than infrastructure.

Authoritative Links for further research:

Dataproc Serverless: <https://cloud.google.com/dataproc/docs/concepts/serverless/overview>

Dataproc Serverless Spark: <https://cloud.google.com/dataproc/docs/concepts/serverless/spark/serverless-spark>

Google Cloud Storage: <https://cloud.google.com/storage>

BigQuery: <https://cloud.google.com/bigquery>

Question: 314**CertyIQ**

You are configuring networking for a Dataflow job. The data pipeline uses custom container images with the libraries that are required for the transformation logic preinstalled. The data pipeline reads the data from Cloud Storage and writes the data to BigQuery. You need to ensure cost-effective and secure communication between the pipeline and Google APIs and services. What should you do?

- A. Disable external IP addresses from worker VMs and enable Private Google Access.
- B. Leave external IP addresses assigned to worker VMs while enforcing firewall rules.
- C. Disable external IP addresses and establish a Private Service Connect endpoint IP address.
- D. Enable Cloud NAT to provide outbound internet connectivity while enforcing firewall rules.

Answer: A**Explanation:**

The optimal solution is to disable external IP addresses for the Dataflow worker VMs and enable Private Google Access (PGA). This approach enhances security and reduces costs. Disabling external IPs removes the attack surface associated with public internet access. PGA allows these VMs, without external IPs, to communicate with Google APIs and services (like Cloud Storage and BigQuery) using Google's internal network. This internal communication is more secure and typically incurs lower data transfer costs than public internet traffic. Custom container images, as described, negate the need for internet access to download dependencies at runtime, making disabling external IPs feasible. Private Service Connect (option C) is overkill and more complex for this simple need. Cloud NAT (option D) provides outbound internet access which is not desired or needed here and costs extra. Firewall rules (option B & D) alone don't prevent data exfiltration and external exposure; it's better to remove public IPs altogether.

Supporting concepts:

Private Google Access: Enables VMs without external IPs to communicate with Google services over Google's private network, improving security and reducing costs. <https://cloud.google.com/vpc/docs/private-google-access>

Security Best Practices: Reducing the attack surface by removing external IPs is a fundamental security practice.

Cost Optimization: Avoiding public internet traffic reduces data transfer costs.

By implementing option A, the data pipeline achieves secure, cost-effective communication with Google APIs while maintaining necessary isolation for its worker VMs.

Question: 315

CertyIQ

You are using Workflows to call an API that returns a 1KB JSON response, apply some complex business logic on this response, wait for the logic to complete, and then perform a load from a Cloud Storage file to BigQuery. The Workflows standard library does not have sufficient capabilities to perform your complex logic, and you want to use Python's standard library instead. You want to optimize your workflow for simplicity and speed of execution. What should you do?

- A. Create a Cloud Composer environment and run the logic in Cloud Composer.
- B. Create a Dataproc cluster, and use PySpark to apply the logic on your JSON file.
- C. Invoke a Cloud Function instance that uses Python to apply the logic on your JSON file.
- D. Invoke a subworkflow in Workflows to apply the logic on your JSON file.

Answer: C

Explanation:

The best option is to use a Cloud Function (C) because it provides a serverless, event-driven compute environment perfectly suited for running short-lived, custom logic, which aligns with the scenario's requirements. Cloud Functions can readily execute Python code, leveraging its standard library as needed for the complex business logic on the JSON response. This approach minimizes infrastructure management compared to options A and B. Passing the 1KB JSON response to the function is efficient, and the function will process it before returning the result. Once the function completes, Workflows can proceed to the next step, loading data from Cloud Storage into BigQuery. Using Cloud Composer (A) would introduce more complexity and overhead than needed for this single operation, as it is intended for orchestrating data pipelines. Dataproc (B) is designed for large-scale data processing, overkill for a 1KB file. A subworkflow in Workflows (D) would still be subject to the limitations of the standard library. By delegating the Python processing to a Cloud Function, we keep the Workflows code clean and focused on orchestration, achieving simplicity and speed.

Further research:

Cloud Functions: <https://cloud.google.com/functions/docs>

Workflows: <https://cloud.google.com/workflows/docs>

Cloud Composer: <https://cloud.google.com/composer/docs>

Dataproc: <https://cloud.google.com/dataproc/docs>

Question: 316

CertyIQ

You are administering a BigQuery on-demand environment. Your business intelligence tool is submitting hundreds of queries each day that aggregate a large (50 TB) sales history fact table at the day and month levels. These queries have a slow response time and are exceeding cost expectations. You need to decrease response time,

lower query costs, and minimize maintenance. What should you do?

- A. Build authorized views on top of the sales table to aggregate data at the day and month level.
- B. Enable BI Engine and add your sales table as a preferred table.
- C. Build materialized views on top of the sales table to aggregate data at the day and month level.
- D. Create a scheduled query to build sales day and sales month aggregate tables on an hourly basis.

Answer: C

Explanation:

The correct answer is **C. Build materialized views on top of the sales table to aggregate data at the day and month level.** Here's why:

Materialized views in BigQuery are precomputed results of a query stored as a table. By creating materialized views that aggregate data at the day and month levels, you directly address the performance and cost issues identified. When the BI tool queries for these aggregations, BigQuery can retrieve the results from the materialized view instead of scanning the entire 50 TB table, leading to significant performance improvements (faster response times) and reduced costs (less data processed).

Option A, using authorized views, simply restricts access to the data, not processing. It doesn't provide performance benefits for aggregate queries. Option B, enabling BI Engine, can improve performance, but it's best suited for interactive analysis. It wouldn't be the primary solution for pre-aggregated summaries. Option D, using scheduled queries to create aggregate tables, is similar to materialized views but requires more management and doesn't automatically reflect changes to the base table. Materialized views automatically refresh when the base data changes.

Materialized views are specifically designed for this use case: pre-calculating aggregations for faster and cheaper reads, with minimal maintenance overhead. They are more efficient than creating separate aggregated tables via scheduled queries because BigQuery automatically and incrementally maintains them. They are ideal for frequently accessed aggregate results.

Authoritative Links:

BigQuery materialized views overview: <https://cloud.google.com/bigquery/docs/materialized-views-intro>

Best practices for materialized views: <https://cloud.google.com/bigquery/docs/best-practices-materialized-views>

BigQuery authorized views: <https://cloud.google.com/bigquery/docs/share-access-views>

BigQuery BI Engine: <https://cloud.google.com/bi-engine/docs/bi-engine-overview>

Question: 317

CertyIQ

You have several different unstructured data sources, within your on-premises data center as well as in the cloud. The data is in various formats, such as Apache Parquet and CSV. You want to centralize this data in Cloud Storage. You need to set up an object sink for your data that allows you to use your own encryption keys. You want to use a GUI-based solution. What should you do?

- A. Use BigQuery Data Transfer Service to move files into BigQuery.
- B. Use Storage Transfer Service to move files into Cloud Storage
- C. Use Dataflow to move files into Cloud Storage
- D. Use Cloud Data Fusion to move files into Cloud Storage.

Answer: B

Explanation:

The correct answer is **B. Use Storage Transfer Service to move files into Cloud Storage**. Here's why:

Storage Transfer Service is specifically designed for moving large amounts of data into or between Cloud Storage buckets. It supports various data sources, including on-premises file systems, other cloud storage providers, and even other Cloud Storage buckets. This aligns directly with the scenario's need to centralize data from diverse locations into Cloud Storage. Crucially, Storage Transfer Service allows users to manage encryption keys, addressing the requirement to use custom encryption keys. Furthermore, it offers a GUI for configuration and management, fulfilling the need for a GUI-based solution.

BigQuery Data Transfer Service (option A) focuses on scheduling and automating data loading into BigQuery, not direct file transfers to Cloud Storage. While BigQuery can load data from Cloud Storage, this option bypasses the initial central storage requirement. Dataflow (option C) is a powerful data processing service, but it's more complex than necessary for simple data transfers. While Dataflow could be used, it's overkill for this task and lacks a native GUI management interface for file transfers. Finally, Cloud Data Fusion (option D) is primarily for building data pipelines and ETL processes, not simple file movement, and it also has a more complex setup than needed for a one-time transfer. Thus, Storage Transfer Service provides the most direct, GUI-based, and encryption-supported method to centralize the unstructured data into Cloud Storage.

For further research, refer to the official Google Cloud documentation:

Storage Transfer Service Overview: <https://cloud.google.com/storage-transfer-service/docs/overview>

Storage Transfer Service Features: <https://cloud.google.com/storage-transfer-service/docs/features>

Managing encryption with Storage Transfer Service: <https://cloud.google.com/storage-transfer-service/docs/encryption-keys>

Question: 318

CertyIQ

You are using BigQuery with a regional dataset that includes a table with the daily sales volumes. This table is updated multiple times per day. You need to protect your sales table in case of regional failures with a recovery point objective (RPO) of less than 24 hours, while keeping costs to a minimum. What should you do?

- A. Schedule a daily export of the table to a Cloud Storage dual or multi-region bucket.
- B. Schedule a daily copy of the dataset to a backup region.
- C. Schedule a daily BigQuery snapshot of the table.
- D. Modify ETL job to load the data into both the current and another backup region.

Answer: A

Explanation:

Option A, scheduling a daily export of the table to a Cloud Storage dual or multi-region bucket, is the most suitable solution. This approach satisfies the RPO requirement of less than 24 hours because the data is backed up daily. Cloud Storage offers dual or multi-region storage options, guaranteeing data redundancy and resilience against regional failures. Exporting to Cloud Storage is a cost-effective method compared to options that involve replicating entire datasets or modifying existing ETL jobs. This method avoids active replication and doesn't incur the extra cost associated with managing snapshots or BigQuery dataset copies. Data within Cloud Storage is readily accessible for restoration should a regional disaster occur in BigQuery. By storing data in Cloud Storage, the recovery process involves re-importing to BigQuery, which is relatively straightforward. In summary, option A offers a balance between cost-effectiveness, ease of implementation, and the desired RPO by leveraging the inherent high-availability features of Cloud Storage and a simple export/import

process. <https://cloud.google.com/storage/docs/locations> <https://cloud.google.com/bigquery/docs/exporting-data> <https://cloud.google.com/storage/docs/storage-classes>

Question: 319

You are preparing an organization-wide dataset. You need to preprocess customer data stored in a restricted bucket in Cloud Storage. The data will be used to create consumer analyses. You need to follow data privacy requirements, including protecting certain sensitive data elements, while also retaining all of the data for potential future use cases. What should you do?

- A. Use the Cloud Data Loss Prevention API and Dataflow to detect and remove sensitive fields from the data in Cloud Storage. Write the filtered data in BigQuery.
- B. Use customer-managed encryption keys (CMEK) to directly encrypt the data in Cloud Storage. Use federated queries from BigQuery. Share the encryption key by following the principle of least privilege.
- C. Use Dataflow and the Cloud Data Loss Prevention API to mask sensitive data. Write the processed data in BigQuery.
- D. Use Dataflow and Cloud KMS to encrypt sensitive fields and write the encrypted data in BigQuery. Share the encryption key by following the principle of least privilege.

Answer: C

Explanation:

The correct answer is **C. Use Dataflow and the Cloud Data Loss Prevention API to mask sensitive data. Write the processed data in BigQuery.**

Here's a detailed justification:

Data Privacy Requirements: The prompt explicitly states the need to follow data privacy requirements and protect sensitive data elements.

Retaining All Data: The prompt emphasizes the importance of retaining all data for potential future use cases. This means removing sensitive data (option A) is undesirable as it leads to data loss. While encryption (options B and D) preserves the data, it requires key management and introduces complexity in querying.

Masking for Protection and Retention: Masking sensitive data using the Cloud Data Loss Prevention (DLP) API replaces sensitive information with non-sensitive, but still meaningful data. For example, a phone number could be masked as XXX-XXX-1234. This approach satisfies both the data privacy requirements and the need to retain all data.

Dataflow for Scalable Processing: Dataflow is a fully managed, serverless stream and batch data processing service. It's suitable for large-scale data transformations like masking. It enables scalable and reliable processing of the customer data.

BigQuery for Analysis: BigQuery is a fully-managed, serverless data warehouse that enables scalable analysis over large datasets. It's a suitable destination for storing the masked data for consumer analyses.

Why other options are not optimal:

A: Removes sensitive data entirely, violating the requirement to retain all data.

B & D: While encryption is a valid security measure, masking using DLP is more appropriate for the prompt's requirements. Encryption requires key management and can complicate querying and analysis. The goal is not just to secure but also to create analyses that likely involve unmasking at some point. Furthermore, Cloud KMS is only used for encryption and decryption, not specifically for masking. Dataflow isn't automatically integrated to work with the Cloud KMS.

In summary, using Dataflow with the DLP API to mask sensitive data before loading it into BigQuery is the best approach because it satisfies all the requirements: protects sensitive data, retains all data, allows for analysis, and avoids unnecessary complexity in key management.

Relevant links:

Cloud Data Loss Prevention (DLP) API: <https://cloud.google.com/dlp/docs>

Question: 320

Your company uses Looker Studio connected to BigQuery for reporting. Users are experiencing slow dashboard load times due to complex queries on a large table. The queries involve aggregations and filtering on several columns. You need to optimize query performance to decrease the dashboard load times. What should you do?

- A. Configure Looker Studio to use a shorter data refresh interval to ensure fresh data is always displayed.
- B. Create a materialized view in BigQuery that pre-calculates the aggregations and filters used in the Looker Studio dashboards.
- C. Implement row-level security in BigQuery to restrict data access and reduce the amount of data processed by the queries.
- D. Use BigQuery BI Engine to accelerate query performance by caching frequently accessed data.

Answer: BD

Explanation:

Why option B is best

A materialized view in BigQuery stores the result of a complex aggregation/query once, then serves subsequent reads from that pre-computed table.

This eliminates the need to recompute the same heavy GROUPBY / FILTER logic on every dashboard refresh, reducing CPU and I/O dramatically.

The view can be refreshed on a schedule that matches the reporting cadence, giving near-real-time performance without impacting the operational transactional tables.

Why option D is best

BigQuery BI Engine is an in-memory columnar store that caches the most-accessed query results and accelerates interactive analytics.

When Looker Studio repeatedly queries the same aggregated data, the engine can serve results from its cache or from the optimized columnar layout, cutting latency to sub-second levels.

It works transparently with existing BigQuery tables, requiring no schema changes or additional pipelines.

Why option A is less suitable

Shortening the data-refresh interval only changes how often Looker Studio pulls fresh data; it does not address the underlying expensive query that must still be executed each refresh.

It may even increase load by forcing more frequent runs of the same heavy query.

Why option C is less suitable

Row-level security restricts which rows a user can see but does not reduce the amount of data the query engine must scan or aggregate.

The performance bottleneck remains the complex aggregation, so security alone cannot improve dashboard load times.

Conclusion

Implementing a **materialized view (B)** to pre-compute the needed aggregations and using **BigQuery BI Engine (D)** to cache and accelerate repeated queries together directly target the root cause of slow dashboard loads, delivering the most efficient and scalable solution.

Question: 321

Your company has data assets across multiple Cloud Storage buckets and BigQuery datasets containing raw and processed data. The requirement is to establish a unified data governance framework that allows for centralized metadata discovery, data quality monitoring, and consistent security policy application across these various data stores without physically moving or duplicating the data. You need to implement a solution to achieve this federated governance. What should you do?

- A. 1. Export metadata out of Dataplex Universal Catalog by running a metadata export job.
2. Implement Dataproc Metastore to manage table schemas and Apache Hive metastore for metadata discovery.
3. Manage security using a combination of BigQuery row-level security and Cloud Storage policies.
- B. 1. Use Dataplex to organize the BigQuery datasets and Cloud Storage buckets into lakes and zones.
2. Use Dataplex for automated metadata discovery, centralized security policy management, data profiling, and data quality tasks.
- C. 1. Deploy a centralized Cloud SQL database to store metadata extracted from BigQuery and Cloud Storage using custom scripts.
2. Integrate the database with Looker Studio for data discovery and visualization.
3. Implement a custom policy engine using Cloud Run functions triggered by changes in IAM policies to enforce consistent security across projects.
- D. 1. Create a Looker Studio dashboard on BigQuery INFORMATION_SCHEMA views to visualize and monitor data quality.
2. Manage security using IAM policies at the project level, supplemented by BigQuery authorized views for granular access control.

Answer: B

Explanation:

Why option B is the optimal solution

Unified metadata management –Dataplex can ingest and index metadata from both BigQuery datasets and Cloud Storage buckets, presenting them as a single logical view (lake/zone) without any data movement. This satisfies the requirement for centralized metadata discovery across disparate stores.

Automated data profiling & quality monitoring –Dataplex’s built-in data profiling and data-quality capabilities continuously scan the underlying assets, surface schema drift, null-rate anomalies, and schema-on-read vs. schema-on-write distinctions, enabling proactive quality checks.

Consistent security enforcement –Dataplex integrates with Cloud IAM and can enforce organization-wide security policies (e.g., uniform bucket-level or dataset-level access) across all registered locations, ensuring consistent security policy application without duplicating data.

Federated governance model –By registering existing buckets and BigQuery locations as Dataplex zones/lakes, the solution maintains the original data residency and avoids any physical copy, directly addressing the “federated governance” constraint.

Why the other options are less suitable

Option A relies on exporting metadata from Dataplex Universal Catalog and deploying a separate Dataproc Metastore/Hive metastore. This introduces unnecessary data movement and operational overhead, and it does not provide native data-quality or automated security enforcement across Cloud Storage.

Option C proposes building a custom metadata store in Cloud SQL and a bespoke policy engine in Cloud Run. While technically possible, it requires significant custom development, does not leverage native Google-managed services for profiling or security, and adds complexity that conflicts with the goal of a streamlined, managed governance framework.

Option D limits governance to a Looker Studio dashboard on INFORMATION_SCHEMA and project-level IAM. This approach lacks centralized metadata discovery, automated data-quality monitoring, and a unified security policy layer; it only offers visualization and coarse-grained access control, falling short of the

federated governance requirements.

Conclusion – OptionB leverages Google-native, fully managed capabilities (Dataplex zones/lakes, automated profiling, quality checks, and IAM-driven security) to deliver a cohesive, scalable, and low-maintenance governance layer across BigQuery and Cloud Storage without moving data. This aligns precisely with the exam's focus on designing efficient, Google-centric data-governance architectures.

Question: 322

CertyIQ

Your company is planning to migrate a large on-premises data warehouse to BigQuery. The data is currently stored in a proprietary, vendor-specific format. You need to perform a batch migration of this data to BigQuery. What should you do?

- A. Use the bq command-line tool to load the data directly from the on-premises data warehouse.
- B. Export the data to CSV files, upload the files to Cloud Storage, then load the files into BigQuery.
- C. Use the BigQuery Data Transfer Service.
- D. Use Datastream to replicate the data in real time.

Answer: B

Explanation:

Why optionB is the correct choice

Batch-oriented migration – The scenario describes moving a large on-premises data warehouse that is stored in a proprietary format. The most reliable way to handle such a bulk transfer is to **export the data to a neutral, widely supported format (CSV)**, copy the files to **Google Cloud Storage (GCS)**, and then use BigQuery's native **CSV loader** to ingest them. This approach works regardless of the source's proprietary technology and avoids any dependency on real-time replication or specialized connectors.

Simplicity and control – Exporting to CSV gives you full visibility into the data schema, allows you to cleanse or transform data before it lands in BigQuery, and lets you stage the files in GCS for parallel loading. BigQuery can ingest massive CSV files efficiently using the bq load command or the LOAD DATA API, making the process scalable and repeatable.

Cost-effective – Using GCS as an intermediate landing zone incurs only storage costs for the temporary files and the standard BigQuery loading fees, which are far cheaper than maintaining a continuous replication pipeline (e.g., Datastream) or provisioning dedicated network bandwidth for real-time streaming.

Why the other options are less suitable

A – Direct bq command-line load from on-premises

The bq CLI can only read from Cloud Storage, local file system paths that are mounted on a Google-managed environment, or from BigQuery itself. It cannot directly connect to an on-premises proprietary data warehouse, so this option is not technically feasible without first moving the data to a supported location.

C – BigQuery Data Transfer Service

This service is designed for scheduled transfers from supported sources such as Cloud Storage, Cloud SQL, or specific SaaS platforms. It does **not** provide a generic connector for arbitrary on-premises proprietary databases, making it inappropriate for a one-time batch migration of a large, vendor-specific dataset.

D – Datastream for real-time replication

Datastream excels at continuous change-data-capture (CDC) and real-time replication. The requirement here is a **batch** migration of a large historical dataset, not an ongoing stream of changes. Using Datastream would

add unnecessary complexity and cost for a scenario that does not need real-time latency guarantees.

Bottom line: Exporting the proprietary data to CSV, staging it in Cloud Storage, and then loading it into BigQuery (OptionB) provides a straightforward, scalable, and cost-efficient pathway for a large-scale batch migration.

Question: 323

CertyIQ

Your company uses separate Google Cloud projects for development, staging, and production. Developers need edit access in the development project, read-only access in the staging project, and no access in the production project. You need an effective and manageable solution to assign and enforce these permissions according to Google-recommended practices. What should you do?

- A. Create separate VPC Service Controls perimeters for each project and use access levels.
- B. Grant individual developers specific IAM roles directly on each project.
- C. Configure firewall rules within each project's VPC to block traffic from unauthorized developer machines.
- D. Create Google Groups for access levels, assign developers to groups, and grant the groups the appropriate IAM roles on each project.

Answer: D

Explanation:

Why option D is the correct, Google-recommended approach

Centralized, reusable policy model – By creating Google Groups that represent each access level (e.g., dev-editors, staging-viewers, prod-none), you can assign IAM roles to the groups once and reuse them across all three environments. This eliminates the need to manage individual developer permissions repeatedly.

Consistent enforcement across projects – Granting the group-level IAM role on each project ensures that the same permission set (edit, read-only, none) is applied uniformly, which aligns with the principle of least privilege and simplifies audit trails.

Scalable management – Adding or removing developers is a single operation (membership change in the group), rather than editing IAM policies per user or per project. This reduces operational overhead and the risk of permission drift.

Best-practice alignment – Google Cloud IAM documentation explicitly recommends using **Google Groups** to manage access at scale, especially when the same set of permissions must be applied to multiple resources or projects.

Why the other options are less suitable

Option A – VPC Service Controls perimeters – Service Controls protect network traffic to specific Google services, not IAM permissions. They do not provide a mechanism for assigning edit, read-only, or no access to developers, so they cannot satisfy the requirement.

Option B – Grant individual IAM roles directly – While IAM roles can enforce the needed permissions, assigning them to each developer individually is not scalable or manageable across multiple environments. It violates the “manageable” requirement and makes future changes cumbersome.

Option C – Firewall rules to block traffic – Firewall rules control network connectivity, not IAM-level permissions. They cannot differentiate between developers who need edit access in dev versus read-only in staging, and they do not address the core need for IAM role assignment.

Conclusion

Creating Google Groups for each required access level and assigning the appropriate IAM roles to those groups on the respective projects provides a scalable, auditable, and Google-recommended solution that

Question: 324

CertyIQ

Your financial services company has a critical daily reconciliation process that involves several distinct steps: fetching data from an external SFTP server, decrypting the files, loading them into Cloud Storage, and finally running a series of BigQuery SQL transformations. Each step has strict dependencies, and the entire process should notify you if not completed by 7:00 AM. Manual intervention for failures is costly and delays compliance reporting. You need a highly observable and robust solution that supports easy re-runs of individual steps if errors occur. What should you do?

- A. Develop a Cloud Composer DAG that includes a single PythonOperator to execute a Python script that runs each step sequentially, incorporating error handling and retries. Upload the scripts to a Cloud Composer environments DAGs folder, and configure it to run daily.
- B. Implement a Cloud Composer DAG, with each step defined as a separate task using appropriate Airflow operators, and schedule the DAG for daily execution.
- C. Define a Cloud Composer DAG to orchestrate the SFTP fetch and decryption steps, and then use Cloud Scheduler to trigger a separate Dataflow job that handles the Cloud Storage load and BigQuery transformations and schedule to run daily.
- D. Create a Cloud Composer DAG that includes a single BashOperator to execute a top-level shell script, which in turn calls individual scripts for each pipeline step. Upload the scripts to a Cloud Composer environment's DAGs folder, and configure it to run daily.

Answer: B

Explanation:

Why option B is the optimal choice

Granular task isolation – Each pipeline step (SFTP fetch, decryption, CloudStorage load, BigQuery transformation) is implemented as an individual Airflow task. This enables independent retries, logging, and monitoring for any single step without re-executing the entire workflow.

Native Airflow operators – Using the appropriate operators (e.g., SshOperator/SFTPGetFileOperator, PythonOperator for decryption, GcsUploadFileOperator, BigQueryInsertJobOperator) provides built-in error handling, retries, and seamless integration with GCP services, reducing custom code and operational risk.

Observability – Airflow's UI offers real-time task-level status, SLA metrics, and alerting, satisfying the "highly observable" requirement. Failures can be visualized per step, making root-cause analysis rapid.

Scheduled daily execution – The DAG can be scheduled with a cron expression that triggers at the required time, ensuring the process runs automatically each day and meets the 7:00AM deadline.

Scalability & maintainability – Adding, modifying, or removing a step only requires updating the DAG definition; the underlying infrastructure (Composer environment) remains unchanged, supporting future pipeline evolution.

Why the other options are less suitable

Option A – Consolidates all steps into a single PythonOperator. This defeats the purpose of granular observability and makes retries or isolated re-runs impossible; any failure forces a full re-execution, increasing latency and cost.

Option C – Introduces an external Dataflow job triggered by Cloud Scheduler, breaking the end-to-end orchestration within a single DAG. It adds unnecessary complexity, increases latency between steps, and reduces unified monitoring; error handling across service boundaries becomes harder.

Option D – Uses a single BashOperator to invoke a top-level shell script that calls other scripts. While it separates logic into files, it still treats the whole workflow as a monolithic script, limiting fine-grained task control, retry policies, and native Airflow integrations; debugging and logging are less detailed than with dedicated operators.

Therefore, optionB delivers the required isolation, observability, retry capability, and maintainability while leveraging Airflow's native GCP integrations, making it the most appropriate solution for the described financial-reconciliation pipeline.

Question: 325

CertyIQ

Your large e-commerce company uses BigQuery for its data warehousing needs. They have two primary types of workloads:

- Analyst dashboards: Interactive queries run by business analysts throughout the day, requiring low latency for real-time insights.
- ETL jobs: Large, complex batch queries that load and transform data overnight. These jobs are cost-sensitive and can tolerate longer execution times, and they do not have the same performance requirements as the dashboards.

Analysts are reporting inconsistent performance of their dashboards, particularly during peak business hours. You need to ensure reliable performance for both workload types while optimizing costs. What should you do?

- A. Use BigQuery editions to create an autoscaling reservation with dedicated slots for analyst dashboards, while running ETL jobs with a separate reservation or using the on-demand billing model.
- B. Implement a custom autoscaling solution using Cloud Run functions to dynamically adjust available BigQuery slots based on time of day.
- C. Migrate all ETL jobs to Cloud SQL to reduce BigQuery's workload.
- D. Run all analyst dashboard queries as interactive queries and configure ETL jobs to run as batch queries.

Answer: A

Explanation:

Why optionA is the optimal solution

Separate resource management – By creating a dedicated reservation (or set of reservations) with **autoscaling slots** for the interactive analyst dashboards, you guarantee a predictable, low-latency pool of query capacity during peak hours.

Cost-effective batch processing – The nightly ETL workload can run on a **different reservation** (or on-demand pricing) that does not compete for the dashboard slots, allowing the ETL jobs to consume as many slots as needed without impacting dashboard performance.

Native BigQuery features – BigQuery reservations and editions are built-in mechanisms for slot isolation and scaling; they require no custom code, additional services, or architectural changes that add operational overhead.

Scalability & reliability – Autoscaling ensures that the dashboard reservation expands automatically when query volume spikes, eliminating the “inconsistent performance” reported by analysts.

Why the other options are unsuitable

OptionB proposes a custom autoscaling engine built on Cloud Run. This introduces unnecessary complexity, adds latency, and does not integrate with BigQuery's native slot management, making it unreliable for guaranteeing low-latency interactive queries.

OptionC suggests moving all ETL jobs to Cloud SQL. This would force large batch transformations onto a transactional OLTP database, degrading performance, increasing cost, and losing the massive parallelism and petabyte-scale capabilities of BigQuery.

OptionD merely re-labels the existing workloads (“run all dashboard queries as interactive queries and configure ETL jobs as batch queries”) without addressing the underlying resource contention; it does not provide any mechanism for slot isolation or cost optimization.

Therefore, **OptionA** leverages BigQuery's native reservation and autoscaling capabilities to isolate and scale the two workload types independently, delivering reliable dashboard performance while keeping ETL costs

under control.

Question: 326

CertyIQ

Your team runs a complex analytical query daily that processes terabytes of data. Recently, after running for 20 minutes, the query fails with a “Resources exceeded” error. You need to resolve this issue. What should you do?

- A. Move from BigQuery on-demand to slot reservations.
- B. Analyze the SQL syntax for errors.
- C. Increase the maximum table size limit.
- D. Increase your project’s BigQuery API request quota.

Answer: A

Explanation:

Why option A is the correct solution

The query fails with “**Resources exceeded**”, which indicates that the allocated compute resources (slots or capacity) are insufficient for the workload.

BigQuery slot reservations let you pre-allocate a fixed amount of processing power, guaranteeing that a query can consume the required resources without hitting the default on-demand slot limits.

By moving from the on-demand model to a **slot-based reservation**, you can scale the reservation size (e.g., increase from 500 to 1000 slots) so that the daily terabyte-scale query completes within the allotted time and avoids the “Resources exceeded” error.

Slot reservations also provide predictable performance and cost control, which is essential for mission-critical, recurring analytical workloads.

Why the other options are not appropriate

B – Analyze the SQL syntax for errors

The error message is not a syntax-related issue; it is a resource-capacity problem. Fixing SQL syntax would not resolve the underlying compute shortage.

C – Increase the maximum table size limit

BigQuery does not enforce a hard “maximum table size” limit that would cause a “Resources exceeded” error. Table size growth is managed automatically, and the error is unrelated to storage limits.

D – Increase your project’s BigQuery API request quota

The “Resources exceeded” error occurs during query execution, not during API request throttling. Raising the API quota would not provide additional processing capacity for the query itself.

Conclusion

To eliminate the “Resources exceeded” failure for a large, daily analytical query, you must **allocate more dedicated query slots via a slot reservation**. This directly addresses the compute-capacity constraint that is causing the failure, while the other options either target unrelated aspects or do not affect query execution resources.

Question: 327

CertyIQ

Your company wants to implement a Retrieval-Augmented Generation (RAG) system to allow employees to query an extensive knowledge base of internal documents, such as policy manuals and project reports. You need to

prepare this unstructured text for embedding to be used in the RAG system. What should you do to ensure the system can retrieve the most relevant information?

- A. Use Cloud Data Loss Prevention (Cloud DLP) to scan and redact sensitive information within the documents before processing.
- B. Store the documents as compressed files in a traditional relational database to enable more efficient storage and retrieval.
- C. Convert the unstructured documents into high-dimensional numerical vectors that capture the semantic meaning and relationships of the text.
- D. Index each word from the documents into a search engine to enable keyword-based search.

Answer: C

Explanation:

Why option C is the correct choice

Semantic-aware representation – Converting the unstructured text into high-dimensional numerical vectors (e.g., using sentence-transformer embeddings) captures the meaning, context, and relationships between concepts. This enables the RAG retriever to match a user query to documents that are semantically similar, not just sharing exact keywords.

Scalable relevance ranking – Vector embeddings can be indexed in a vector database (e.g., Pinecone, Milvus, or Vertex AI Matching Engine) and queried with Approximate Nearest Neighbor (ANN) algorithms, delivering fast, relevance-ordered results even over millions of documents.

Preserves nuance – Unlike keyword matching, embeddings understand synonyms, paraphrasing, and domain-specific jargon, which is essential for internal policy manuals and project reports where terminology may vary.

Why the other options are less suitable

A – Cloud DLP redaction – While important for privacy, redaction does not improve retrieval relevance; it merely removes sensitive data. It does not address the core requirement of finding the most pertinent information.

B – Storing compressed files in a relational DB – Relational databases excel at structured queries but are inefficient for large-scale semantic search. Compression adds no value to relevance ranking and limits the ability to perform vector-based similarity searches.

D – Keyword-based indexing – Keyword search can retrieve documents containing exact terms, but it fails to capture conceptual relevance, synonyms, or context, leading to many false negatives/positives in a RAG pipeline.

Thus, **converting documents to semantic vectors (Option C)** directly fulfills the goal of retrieving the most relevant information for a RAG system.

Question: 328

CertyIQ

You need to load a dataset with multiple terabytes of clickstream data into BigQuery. The data arrives each day as compressed JSON files in a Cloud Storage bucket. You need a low-cost, programmatic, and scalable solution to load the data into BigQuery. What should you do?

- A. Create an external table in BigQuery pointing to the Cloud Storage bucket and run the INSERT INTO ... SELECT * FROM external_table command.
- B. Use the BigQuery Data Transfer Service from Cloud Storage.
- C. Create a Cloud Run function to run a Python script to read and parse each JSON file, and use the BigQuery streaming insert API.

D. Use Cloud Data Fusion to create a pipeline to load the JSON files into BigQuery.

Answer: A

Explanation:

Why option A is the best choice

Native BigQuery integration – An external table can read directly from CloudStorage, automatically handling compression (e.g., gzip/JSON-L) and schema inference, so no custom code or additional services are required.

Scalable & low-cost – The external table reads data on-demand; you pay only for the bytes scanned when you issue the `INSERT INTO ... SELECT * FROM external_table` query, which is cheaper than provisioning separate compute resources.

Programmatic & fully managed – No need to manage clusters, containers, or custom pipelines; the load operation can be scheduled (e.g., Cloud Scheduler) or triggered by a Cloud Function, keeping the solution fully serverless.

Performance-optimized – BigQuery can parallelize reads across the distributed storage layer, delivering fast ingestion even for multi-terabyte datasets.

Why the other options are less suitable

B – BigQuery Data Transfer Service – This service is designed for copying data from supported sources (e.g., Cloud SQL, other BigQuery datasets) into BigQuery, but it does **not** ingest raw files from CloudStorage; it expects a predefined schema and does not handle arbitrary JSON streams efficiently.

C – Cloud Run + streaming insert API – Streaming inserts are intended for low-volume, real-time event ingestion. Loading terabytes of daily JSON files would require millions of API calls, incurring high cost, throttling risks, and complex error-handling logic.

D – Cloud Data Fusion – While capable of orchestrating JSON parsing, Data Fusion adds operational overhead (cluster provisioning, pipeline maintenance) and licensing costs, making it unnecessarily expensive for a simple, low-cost ingestion pattern that can be achieved with native external tables.

Thus, creating an external table and inserting from it provides the most cost-effective, scalable, and fully managed solution for loading large, daily-arriving JSON clickstream files into BigQuery.

Question: 329

CertyIQ

Your Cloud Storage data lake has raw, processed, and historical data in different buckets. Data older than two years is rarely accessed, and all data must be retained for no longer than seven years. You are concerned about rising storage costs. How should you control costs for the historical data bucket?

- A. Write a script on a Compute Engine instance, triggered daily by Cloud Scheduler, to scan all objects and delete any older than seven years.
- B. Configure an Object Lifecycle Management rule to transition objects older than two years to the Archive storage class and eventually delete them after seven years.
- C. Enable the Autoclass feature on your Cloud Storage buckets and select Opt-in to object transitions to Coldline and Archive storage classes.
- D. Replicate the buckets to a different region with lower storage costs and configure an Object Lifecycle Management rule to delete objects after seven years.

Answer: B

Explanation:

Why option B is the correct choice

Lifecycle-aware cost control – Object Lifecycle Management (OLM) can automatically transition objects based on age or storage class, eliminating the need for custom scripts or external compute resources.

Targeted retention policy – By setting a rule that moves objects older than 2 years to **Archive** and then expires them after 7 years, you keep only the data that must be retained, directly addressing the cost-driven requirement.

Native integration & reliability – OLM is a built-in Google Cloud Storage feature, fully managed, and guarantees that transitions and deletions occur exactly as defined, avoiding operational overhead or failure points introduced by external schedulers.

Cost-effective tiering – Archive storage offers the lowest per-GB price for rarely accessed data, while Coldline (if used) is more expensive; OLM lets you place data in the cheapest appropriate tier without manual intervention.

Why the other options are less suitable

Option A – Requires a continuously running Compute Engine VM, a Cloud Scheduler job, and custom scripting. This adds operational complexity, incurs extra VM and scheduler costs, and introduces a single point of failure that is unnecessary when a native GCS feature exists.

Option C – Autoclass automatically moves objects between storage classes, but it does not enforce a fixed expiration after a specific number of years. It also may transition objects to Coldline rather than Archive, which is more costly for data that is accessed only once every two years, and it lacks explicit control over the final deletion date.

Option D – Replicating buckets to another region merely shifts storage costs; it does not reduce the total amount of stored data and may even increase expenses due to inter-region replication charges. Additionally, the lifecycle rule would still need to be configured, making the approach redundant and less economical.

Conclusion – Option B leverages the most efficient, native mechanism (Object Lifecycle Management) to tier historical data to Archive and enforce a seven-year retention limit, delivering the required cost savings with minimal operational overhead.

Question: 330

CertyIQ

Your data science team needs to perform interactive SQL queries on large datasets stored in Apache Parquet format within a Cloud Storage bucket. The team is familiar with Apache Hive and wants to leverage existing HiveQL queries. You need to provide an environment for the team to run their interactive HiveQL queries directly against the data in Cloud Storage. You want to keep operational overhead to a minimum. What should you do?

- A. Install and configure an Apache Hadoop and Hive cluster manually on a group of Compute Engine instances.
- B. Load the Parquet data into a BigQuery native table and use the BigQuery Connector for Hive to run the queries.
- C. Configure BigQuery with an external table definition pointing to the Parquet files.
- D. Deploy a Dataproc cluster with Hive services enabled.

Answer: D

Explanation:

Correct option: D – Deploy a Dataproc cluster with Hive services enabled.

Native Hive support on managed infrastructure – Dataproc provides a fully-managed Hadoop ecosystem where Hive is pre-installed and configured. This lets the team run existing HiveQL queries without any custom installation or configuration work.

Direct query on Parquet in Cloud Storage – Hive on Dataproc can read Parquet files directly from Cloud Storage using the Hive-Metastore-backed external tables or the built-in parquet serde, so no data movement is required.

Minimal operational overhead – Because Dataproc handles cluster provisioning, scaling, and patching, the team only needs to submit Hive queries (via hive, Beeline, or notebooks). No separate Hadoop or Hive administration is required.

Scalable compute – The cluster can be sized to match the data volume and query complexity, and can be automatically scaled down after use, keeping costs low.

Why the other options are less suitable

A – Manual Hadoop & Hive on Compute Engine – Requires extensive manual setup, ongoing maintenance, and expertise to manage Hadoop, Hive, and storage integrations, increasing operational overhead.

B – Load into BigQuery and use Hive connector – Involves a costly ETL step to load data into BigQuery, and the Hive connector adds complexity; it does not provide the simplest path for running existing HiveQL directly on the source Parquet files.

C – External table in BigQuery – While BigQuery can query external Parquet data, it uses its own SQL dialect, not HiveQL, and does not meet the team's requirement to reuse existing Hive queries unchanged.

Thus, deploying a Dataproc cluster with Hive services offers the most straightforward, low-maintenance environment for executing the team's existing HiveQL queries on Parquet data stored in Cloud Storage.

Question: 331

CertyIQ

You analyze user clickstream data to personalize content recommendations. The data arrives continuously and needs to be processed with low latency, including transformations such as sessionization (grouping clicks by user within a time window) and aggregation of user activity. You need to identify a scalable solution to handle millions of events each second and be resilient to late-arriving data. What should you do?

- A. Use Firebase Realtime Database for ingestion and storage, and Cloud Run functions for processing and analytics.
- B. Use Cloud Data Fusion for ingestion and transformation, and Cloud SQL for storage and analytics.
- C. Use Pub/Sub for ingestion, Dataflow with Apache Beam for processing, and BigQuery for storage and analytics.
- D. Use Cloud Storage for ingestion, Dataproc with Apache Spark for batch processing, and BigQuery for storage and analytics.

Answer: C

Explanation:

Why option C is the optimal choice

Pub/Sub provides globally distributed, horizontally-scalable ingestion of streaming click-stream events, automatically handling high throughput (millions of messages/second) and buffering late-arriving data until it can be processed.

Dataflow (Apache Beam) runs a fully managed, stateful pipeline that can perform low-latency sessionization and aggregations on the fly, scaling elastically with event volume and supporting exactly-once semantics for reliable processing.

BigQuery stores the resulting aggregated results and enables fast, ANSI-SQL analytics on petabyte-scale data, offering serverless query performance without operational overhead and seamless integration with downstream BI tools.

Why the other options are less suitable

A – Firebase Realtime Database + Cloud Run – Firebase is optimized for client-side, low-latency sync rather than massive, high-throughput ingestion; Cloud Run is better suited for request-driven micro-services, not for

continuous, stateful stream processing such as sessionization.

B – Cloud Data Fusion + Cloud SQL – Data Fusion is primarily a batch-oriented ETL tool with limited streaming capabilities; Cloud SQL is a traditional relational database that does not scale to the required write throughput or provide the analytical performance needed for millions of events per second.

D – Cloud Storage + Dataproc (Spark) for batch – Storing raw events in Cloud Storage and processing them later with Dataproc creates a batch-centric workflow with high latency, making it unsuitable for real-time personalization and for handling late-arriving data in a timely manner.

Thus, the combination of **Pub/Sub → Dataflow → BigQuery** delivers the required scalability, low-latency processing, and resilience to late events while keeping operational complexity minimal.

Question: 332

CertyIQ

You are designing BigQuery tables for large volumes of clickstream event data. Your data analyst team will most frequently query by specific event date ranges and filter by the user ID UUID. You want to optimize table structure for query cost and performance. What should you do?

- A. Partition the table by the user ID column and cluster the table by the event date column.
- B. Create an ingestion-time partitioned table and cluster it by the user ID column.
- C. Cluster the table by both the event date and the user ID columns.
- D. Partition the table by the event date column and cluster the table by user ID column.

Answer: D

Explanation:

Why option D is the optimal design

Partitioning by event date aligns with the most common query pattern – filtering on a specific date range. Partition pruning eliminates whole partitions that do not match the range, dramatically reducing the amount of data scanned and therefore query cost.

Clustering by user-ID groups rows that share the same UUID together within each partition. When analysts filter on `user_id`, BigQuery can prune not only partitions but also the clustered sub-segments, further limiting the scanned bytes.

The combination **partition-by-date+cluster-by-user_id** gives the best of both worlds: date-range pruning for cost efficiency and user-ID clustering for fast point-lookup filtering, while keeping the table definition simple and maintainable.

Why the other options are less suitable

Option A partitions on `user_id` and clusters on `event_date`. Partitioning by a high-cardinality column like UUID creates a huge number of tiny partitions, causing partition-management overhead and poor pruning because most queries will still need to scan many partitions.

Option B uses ingestion-time partitioning, which does not guarantee that future queries will align with the logical date range used by analysts; it also clusters by `user_id`, but without date-based partitioning the date-range filter cannot prune partitions effectively.

Option C clusters on both `event_date` and `user_id` but does **not** partition at all. Without partitioning, every query must scan the entire table (or at least the whole clustering key space), leading to higher cost and slower performance for date-range filters.

Thus, **partition the table by the event date column and cluster the table by the user-ID column (Option D)** provides the most efficient query execution and cost-effective storage for the described workload.

Question: 333

You are building a Dataflow pipeline to ingest customer feedback. Before loading to your data warehouse, you must validate email addresses and enrich unstructured comment strings with a generative AI sentiment classification. Invalid records need to be routed for manual review. How should you implement this pipeline?

- A. Configure the data source system to pre-validate data before sending it to Dataflow and use the BigQuery ML.GENERATE_TEXT command to assign a sentiment score.
- B. Use Dataflow to load all data into BigQuery and execute a SQL MERGE statement to flag invalid records and BigQuery ML to assign sentiment scores.
- C. After the Dataflow load completes, execute a Cloud Run function to scan for invalid entries and call Vertex AI to assign sentiment scores.
- D. Apply a ParDo transform in Dataflow to validate each element, use a RunInference transform to assign sentiment scores, and use side outputs to route valid/invalid records.

Answer: D

Explanation:

Why option D is the optimal design

End-to-end processing inside Dataflow – The ParDo transform can execute custom validation logic (e.g., regex or schema checks) on each incoming record, guaranteeing that only syntactically correct email addresses proceed.

Native integration with VertexAI – The RunInference transform consumes a model deployed to VertexAI and returns a sentiment label for each comment in the same pipeline, avoiding external network hops and preserving Dataflow's low-latency, stateful processing model.

Deterministic routing of bad records – By emitting records to a side-output stream, invalid entries are automatically separated from valid ones without needing a separate job or post-processing step, which simplifies error handling and downstream manual review.

Scalable and serverless – All components run on the Dataflow service, automatically scaling with data volume and handling back-pressure, so the pipeline remains cost-effective and operationally simple.

Why the other options are less suitable

Option A – Relies on the source system to pre-validate data, which is not under your control; also, ML.GENERATE_TEXT is a BigQuery ML function that cannot be called directly from Dataflow without an extra export step, breaking the tight coupling of validation and enrichment.

Option B – Involves a full load into BigQuery followed by a separate SQLMERGE and ML call. This adds unnecessary latency, incurs extra BigQuery storage and compute costs, and requires a second job to handle validation and sentiment, contradicting the goal of a single, streamlined Dataflow pipeline.

Option C – Uses a Cloud Run function triggered after the Dataflow load, introducing an asynchronous, external processing step. This breaks the atomicity of the transformation, complicates error routing, and adds operational overhead (function deployment, networking, and state management) that Dataflow can natively handle.

Therefore, option **D** provides a cohesive, scalable, and fully managed solution that validates, enriches, and routes records all within Dataflow, meeting the requirements for correctness, performance, and maintainability.

Question: 334

You need to orchestrate a pipeline with several Google Cloud services: a batch Dataflow job, then a BigQuery query

job followed by a Vertex AI batch prediction. The logic is sequential. You want a lightweight, serverless orchestration solution with minimal operational overhead. What service should you use?

- A. Select Cloud Composer.
- B. Select Compute Engine with cron.
- C. Select Dataproc with Apache Oozie.
- D. Select Cloud Workflows.

Answer: D

Explanation:

Why Cloud Workflows (OptionD) is the optimal choice

Fully serverless, managed orchestration – Cloud Workflows executes workflows as a native service; no clusters, VMs, or external schedulers need to be provisioned or maintained.

Native integration with Dataflow, BigQuery, and VertexAI – Each step can be invoked via built-in connectors or simple HTTP calls, allowing a pure sequential DAG without custom glue code.

Minimal operational overhead – Workflow definitions are declarative YAML/JSON; scaling, retries, and error handling are handled automatically, eliminating cron-job management or cluster lifecycle concerns.

Lightweight and cost-effective – You pay only for the executed steps and the underlying service; there is no need to over-provision Compute Engine or Dataproc resources for infrequent batch jobs.

Why the other options are less suitable

A. Cloud Composer – Although it can orchestrate the same services, Composer runs on managed Airflow clusters, requiring cluster provisioning, monitoring, and version upgrades. This adds operational overhead compared to the pure serverless model of Cloud Workflows.

B. Compute Engine with cron – Deploying VMs and configuring cron jobs introduces infrastructure management (VM lifecycle, patching, scaling) and manual error handling, contradicting the “minimal operational overhead” requirement.

C. Dataproc with Apache Oozie – Dataproc clusters must be provisioned, scaled, and maintained; Oozie adds another layer of complexity. This approach is heavier and less serverless than Cloud Workflows, making it a poor fit for lightweight orchestration.

Therefore, Cloud Workflows provides the most concise, serverless, and low-maintenance solution for sequencing a Dataflow batch job, a BigQuery query, and a VertexAI batch prediction.

Question: 335

CertyIQ

You have a data analyst team member who needs to analyze data by using BigQuery. The data analyst wants to create a data pipeline that would load 200 CSV files with an average size of 15MB from a Cloud Storage bucket into BigQuery daily. The data needs to be ingested and transformed before being accessed in BigQuery for analysis. You need to recommend a fully managed, no-code solution for the data analyst. What should you do?

- A. Create a Cloud Run function and schedule it to run daily using Cloud Scheduler to load the data into BigQuery.
- B. Use the BigQuery Data Transfer Service to load files from Cloud Storage to BigQuery, create a BigQuery job which transforms the data using BigQuery SQL and schedule it to run daily.
- C. Build a custom Apache Beam pipeline and run it on Dataflow to load the file from Cloud Storage to BigQuery and schedule it to run daily using Cloud Composer.
- D. Create a pipeline by using BigQuery pipelines and schedule it to load the data into BigQuery daily.

Answer: B

Explanation:

Why option B is the best choice

Fully managed, no-code ingestion – BigQuery Data Transfer Service (DTS) can automatically pull CSV files from Cloud Storage into BigQuery on a daily schedule without writing custom code.

Built-in transformation – DTS lets you attach a BigQuery SQL script that runs immediately after the load, so the data is transformed in-place before it becomes query-able.

Serverless scheduling – DTS jobs can be scheduled natively (daily, hourly, etc.) via the service's UI or API, eliminating the need for external schedulers or orchestration tools.

Scalable & cost-effective – The service scales to any file size or volume, and you only pay for the data transferred and the query processing used, which is ideal for 200×15MB files per day.

Certification-friendly – DTS is a Google-recommended, native solution for “load-and-transform” pipelines that aligns with the “no-code/low-code” requirement of the exam question.

Why the other options are less suitable

A – Cloud Run + Cloud Scheduler – Requires writing and deploying a custom function, which introduces code development, testing, and maintenance overhead; it is not a “no-code” solution.

C – Custom Apache Beam pipeline on Dataflow – Although powerful, building a Beam pipeline involves programming in Java/Python, setting up Dataflow workers, and managing the pipeline lifecycle – again, not a no-code approach.

D – BigQuery Pipelines – BigQuery Pipelines are designed for orchestrating a series of BigQuery SQL statements, not for ingesting external files from Cloud Storage; they lack native connectors for CSV import and would still need an external mechanism to trigger the load.

Thus, option **B** delivers a truly managed, code-free daily pipeline that loads, transforms, and makes the data ready for analysis in BigQuery.

Question: 336

CertyIQ

You are designing a stateful data processing pipeline that reads data from a Cloud Storage bucket and writes transformed data to a BigQuery table. The pipeline must be highly available and resilient to zonal failures within the us-central1 region. You need to configure a Dataflow pipeline ensuring minimal disruption during a zonal outage. What should you do?

- A. Launch the Dataflow job with the `--region=us-central1` parameter.
- B. Deploy the Dataflow job to a single zone within us-central1 and configure it to use a regional persistent disk to store its state.
- C. Deploy the Dataflow job to a single zone within us-central1 and use a multi-regional Cloud Storage bucket to store its state.
- D. Launch the Dataflow job with the `--zone=us-central1a` parameter.

Answer: A

Explanation:

Correct option: A – Launch the Dataflow job with the `--region=us-central1` parameter.

Regional deployment provides automatic zone-level redundancy. When a job is launched to a region (e.g., us-central1), Dataflow automatically runs the pipeline across multiple zones within that region. If one zone experiences an outage, the work is transparently rescheduled onto the remaining healthy zones, delivering built-in high availability without manual intervention.

State is stored in CloudDataflow's managed service, not on local disks. By using the regional flag, Dataflow

leverages its internal fault-tolerant execution model (checkpointing, autoscaling, and state replication) that survives zonal failures. No extra configuration of persistent disks or CloudStorage is required.

Minimal disruption and operational simplicity. The regional flag is a single parameter; the pipeline remains fully managed, and you avoid the overhead of manually managing disks or custom storage back-ends, which can become points of failure themselves.

Why the other options are less suitable

B – Single-zone deployment with a regional persistent disk

A single zone still represents a single point of failure; if that zone goes down, the entire job fails. Although a regional persistent disk spans zones, it is not designed for Dataflow state and adds complexity; the pipeline would still be vulnerable to zonal outages.

C – Single-zone deployment using a multi-regional CloudStorage bucket for state

Storing state in CloudStorage does not protect the compute resources themselves. The job remains confined to one zone, so a zonal outage still brings the pipeline down. Moreover, Dataflow's native state handling is more efficient and reliable than external storage for intermediate data.

D – Launch with a specific `--zone=us-central1a` flag

Explicitly fixing the job to a single zone (us-central1a) makes the pipeline fully dependent on that zone's availability. Any failure in that zone will cause an outage, directly contradicting the requirement for resilience to zonal failures.

Conclusion

Launching the Dataflow job with the `--region=us-central1` flag ensures the pipeline runs across multiple zones within the same region, providing automatic failover, built-in state durability, and the highest level of availability with the simplest configuration. This aligns precisely with the requirement for minimal disruption during a zonal outage.

Question: 337

CertyIQ

Your team has created several BigQuery curated datasets containing anonymized industry benchmark data. You want to make these datasets easily discoverable and accessible for querying by external partner companies within their own Google Cloud projects. You need a secure and scalable solution. What should you do?

- A. Publish the datasets as listings within BigQuery sharing (Analytics Hub).
- B. Export the datasets to partner-specific Cloud Storage buckets.
- C. Create authorized views for each dataset and grant access to each partner.
- D. Grant the roles/bigquery.dataViewer IAM role to the partner group email addresses on the datasets.

Answer: A

Explanation:

Correct option: A – Publish the datasets as listings within BigQuery sharing (Analytics Hub).

Secure, governed sharing – Analytics Hub provides a centralized catalog where datasets can be listed as “shared” resources. Access is controlled through Google Cloud IAM and can be granted at the project or dataset level, ensuring that only approved partner identities can discover and query the data.

Scalable multi-project consumption – Partners can add the shared listing to their own BigQuery environment with a single “Add to project” action; no need to copy data or manage separate storage buckets, which

simplifies scaling to many external users.

Built-in data-lineage & governance – The sharing mechanism preserves metadata, tags, and access policies, making it easy to audit who is querying the data and to revoke access centrally.

No data duplication – Since the underlying tables remain in the original project, there is no risk of stale copies and no extra storage cost.

Why the other options are less suitable

B – Export to partner-specific Cloud Storage buckets

Requires manual or automated export pipelines, leading to data duplication, eventual consistency, and additional operational overhead.

Access control must be re-implemented per bucket, and partners would need to load the data into BigQuery before querying, adding latency and complexity.

C – Create authorized views for each dataset and grant access to each partner

Authorized views are limited to the same project and cannot be shared across separate Google Cloud projects.

Managing a distinct view per partner and per dataset becomes unwieldy at scale, and revoking access would still require per-view changes.

D – Grant the roles/bigquery.dataViewer IAM role to the partner group email addresses on the datasets

IAM permissions are project-scoped; granting them directly would expose the entire project to external partners, which is often too permissive for curated, anonymized benchmark data.

It does not provide a catalog-level discovery experience and lacks the governance controls (e.g., usage metrics, data-lineage) that Analytics Hub offers.

Conclusion

Publishing the curated datasets via BigQuery sharing in Analytics Hub delivers a secure, centrally managed, and scalable way for external partners to discover and query the data across their own projects, while preserving governance and minimizing operational overhead.

Question: 338

CertyIQ

Your organization stores highly personal data in BigQuery and needs to comply with strict data privacy regulations. You need to ensure that sensitive data values are rendered unreadable whenever an employee leaves the organization. What should you do?

- A. Use column-level access controls with policy tags and revoke viewer permissions when employees leave the organization.
- B. Use dynamic data masking and revoke viewer permissions when employees leave the organization.
- C. Use customer-managed encryption keys (CMEK) and delete keys when employees leave the organization.
- D. Use AEAD functions and delete keys when employees leave the organization.

Answer: C

Explanation:

Correct option: C – Use customer-managed encryption keys (CMEK) and delete keys when employees leave the organization.

Why it works:

CMEK stores the data-encryption keys (DEKs) that protect BigQuery columns in a Cloud KMS key that you control.

When an employee's access is revoked, you can **disable or destroy the associated Cloud KMS key**; all data encrypted with that key becomes unrecoverable, rendering the stored values unreadable instantly.

This approach directly ties data confidentiality to key lifecycle management, satisfying strict privacy regulations that require "unreadable on departure."

Why the other options are less suitable:

A – Column-level access controls with policy tags: Revoking viewer permissions stops queries, but existing encrypted data remains accessible to any user who still holds the underlying DEKs; it does not make the data itself unreadable.

B – Dynamic data masking: Masking only obscures results for unauthorized viewers; it does not alter the underlying ciphertext, so a departing employee who still possesses the DEK could still decrypt the raw values.

D – AEAD functions and delete keys: While AEAD provides authenticated encryption, Google Cloud's AEAD APIs rely on underlying keys that are managed by the service; deleting those keys does not guarantee that all encrypted columns are protected, and it is not the recommended mechanism for enforcing data-at-rest confidentiality in BigQuery.

Thus, leveraging **customer-managed encryption keys and securely deleting them upon employee off-boarding** is the only option that guarantees the data becomes permanently unreadable in accordance with regulatory requirements.

Question: 339

CertyIQ

You monitor and optimize the BigQuery instance for your team. You notice that a particular daily report that uses a large JOIN operation is consistently slow. You want to examine the query's execution plan to identify potential performance bottlenecks within the JOIN as quickly as possible. What should you do?

- A. Use the `bg query --dry_run` command to review the estimated number of bytes read and review query syntax.
- B. Run a query on the `INFORMATION_SCHEMA.JOBS_BY_PROJECT` view filtering by the `job_ID` and analyze `total_bytes_processed`.
- C. Review the BigQuery audit logs in Cloud Logging.
- D. Leverage BigQuery's Query History view and analyze the execution graph.

Answer: D

Explanation:

Step-by-step justification

Goal: Quickly pinpoint performance bottlenecks inside a large JOIN of a daily report.

Desired action: Obtain an execution-graph/visual plan that shows how BigQuery actually processes the join (e.g., which side is built, hash/spill, broadcast, shuffle, etc.) and that displays metrics such as bytes processed per step.

Why option D is optimal:

The **Query History** feature renders an interactive execution graph that is created automatically when a query finishes.

The graph highlights each phase of the JOIN (e.g., "Broadcast Join", "Hash Join", "Merge Join"), the amount of data spilled, and the cost in processing units and bytes processed **per node**, allowing the analyst to spot expensive bottlenecks within seconds.

It also surfaces timing information, cache hits, and any operator-level errors that can be directly acted upon.

Why the other options are less suitable:

A. `bg query --dry_run` – Only checks syntax and returns an estimated bytes-read amount. It does **not** execute the query or reveal the real join strategy, so no detailed bottleneck information is available.

B. Querying `INFORMATION_SCHEMA.JOBS_BY_PROJECT` – Provides aggregated job metadata (total bytes processed, status) but does **not** expose the per-operation breakdown of the JOIN itself; you cannot see the execution plan without re-running the query.

C. Reviewing Cloud Audit logs – Audits capture access and security-related events (who ran what and when), not performance execution details. They do not show plan operators, hash-join decisions, or data skew.

Conclusion: To **examine the JOIN's execution plan and locate bottlenecks instantly**, you must look at the **Query History/Journey view**, which renders the live execution graph. Hence option **D** is the correct action.

References

BigQuery Query History and Execution Graph: <https://cloud.google.com/bigquery/docs/reference/ui/query-history>

Understanding Query Plans and Performance Recommendations:

<https://cloud.google.com/bigquery/docs/analyzing-queries-using-query-plans>

These links describe how the UI shows the execution graph and how to interpret the per-operator metrics to diagnose join-related performance issues.

Question: 340

CertyIQ

Your company stores vital operational sales data in a BigQuery dataset in us-central1. Your company requires a disaster recovery plan to restore this data to us-east1 with a recovery point objective (RPO) of 24 hours and a recovery time objective (RTO) of 4 hours if us-central1 experiences an outage. You need to implement the disaster recovery plan while keeping costs and complexity to a minimum. What should you do?

- A. Set up continuous queries and Pub/Sub to stream data changes from BigQuery tables in us-central1 to us-east1.
- B. Manually export data to a CSV file in a multi-regional Cloud Storage bucket daily and use `bq load` to restore to us-east1.
- C. Configure BigQuery cross-region dataset replication from us-central1 to us-east1.
- D. Take daily BigQuery table snapshots in us-central1.

Answer: B

Explanation:

Why option B is the best choice

It satisfies the **24-hour RPO** by exporting the data once per day, which is the most frequent cadence among the listed options without introducing extra services.

The export can be automated (e.g., Cloud Scheduler+Cloud Functions) so that the **RTO of 4hours** is achievable: the CSV file is staged in a multi-regional Cloud Storage bucket and then loaded into the replica dataset in **us-east1** within minutes.

Cost-wise it only uses **BigQuery export, Cloud Storage, and `bq load`**, all of which are covered by the standard free tier and have predictable per-GB pricing, keeping overall expense minimal.

It avoids the overhead of maintaining a separate streaming pipeline (Pub/Sub+continuous queries) or paying for cross-region dataset replication, which is not a native feature for BigQuery.

Why the other options are less suitable

OptionA – Continuous queries + Pub/Sub: Introduces additional managed services, custom code, and operational complexity. It also incurs higher egress and Pub/Sub costs, and the pipeline would need to guarantee exactly-once delivery to meet the 4-hour RTO, making it unnecessarily heavyweight for a daily RPO.

OptionC – BigQuery cross-region dataset replication: Replication is **not generally available** for BigQuery datasets across regions; it is limited to specific use-cases (e.g., shared-mode tables) and would require a paid replication service. Moreover, it does not meet the “minimum cost and complexity” requirement.

OptionD – Daily snapshots: Snapshots are stored in the same region and cannot be used to restore a dataset in another region. To achieve a DR copy in **us-east1**, you would still need to copy the snapshot, effectively replicating the steps of optionB but with added steps and no built-in cross-region support, leading to higher complexity.

Implementation sketch for optionB

1. Use a scheduled Cloud Function or Cloud Scheduler job to run bq extract from each production table in **us-central1** to a **multi-regional Cloud Storage** bucket (e.g., gs://dr-bucket/central/).
2. The export creates a daily CSV (or Avro) file; enable **versioning** on the bucket to retain the latest file.
3. In **us-east1**, create a replica BigQuery dataset. When an outage is declared, trigger a Cloud Function that runs bq load from the latest CSV in the bucket into the target tables, which typically completes within minutes, well inside the 4-hour RTO.

References

Exporting BigQuery data to Cloud Storage: <https://cloud.google.com/bigquery/docs/exporting-data>

Loading data into BigQuery from Cloud Storage: <https://cloud.google.com/bigquery/docs/loading-data-into-bigquery>

Question: 341

CertyIQ

You are integrating your legacy on-premises MySQL database into a new Google Cloud data warehouse. The database contains historical customer loyalty data which needs to be extracted daily and loaded into BigQuery. You want a managed integration solution that is secure and supports incremental loads. What should you do?

- A. Implement a data pipeline using Pub/Sub for streaming the data changes from MySQL.
- B. Configure a scheduled transfer in BigQuery Data Transfer Service to ingest data from the MySQL database with a scheduled transfer configuration.
- C. Leverage Database Migration Service for continuous replication of the entire database to Cloud SQL.
- D. Use Cloud SQL federated queries to directly query the on-premises MySQL database from BigQuery.

Answer: B

Explanation:

Justification

The scenario requires a **managed, secure, and incremental** ingestion from an on-premises MySQL instance into BigQuery on a **daily** basis.

Correct choice – B – Configuring a **scheduled transfer in BigQuery Data Transfer Service (DTS)** is the simplest, fully-managed way to move data from MySQL into BigQuery. DTS supports built-in MySQL connectors, allows you to define a daily recurrence, and can automatically capture incremental changes based on the source’s transaction log or timestamp columns, satisfying both the security (IAM-controlled

service account, VPC-native connectivity) and incremental-load requirements without custom code.

Why A is less suitable – Using Pub/Sub to stream changes from MySQL would necessitate a custom connector or CDC tool, adding operational complexity and lacking native incremental-load semantics within BigQuery. It is not the recommended managed integration path for this use case.

Why C is less suitable – Database Migration Service (DMS) is designed for **continuous replication of an entire source database to another Cloud instance** (e.g., Cloud SQL). It does not target BigQuery directly and does not provide built-in incremental loading into a warehouse; it also exceeds the scope of a simple daily extract.

Why D is less suitable – Leveraging Cloud SQL federated queries to query MySQL directly from BigQuery incurs higher latency, can degrade source performance, and does not provide an efficient, secure, and repeatable ingestion pipeline for daily bulk loads.

References

BigQuery Data Transfer Service – MySQL connector: <https://cloud.google.com/bigquery/docs/transfer-service-mysql>

Scheduled transfers in BigQuery: <https://cloud.google.com/bigquery/docs/scheduled-transfers>

Question: 342

CertyIQ

You are developing a fraud detection model using BigQuery ML. You have a raw transaction dataset and need to create new features such as the `average_transaction_amount_last_24_hours` and `time_since_last_transaction`. These features require aggregation and time-window calculations on the existing data. The goal is to ensure that these features are consistently applied during both model training and prediction without manual intervention. You need to prepare these features efficiently for your model. What should you do?

- A. Implement a Cloud Run function that triggers on new transactions, calculates the features, and inserts them into a feature store before model serving.
- B. Export the BigQuery data to Cloud Storage, perform feature engineering using a custom Python script in a Dataflow job, and then re-import the engineered features into BigQuery.
- C. Use the TRANSFORM clause within the CREATE MODEL statement, leveraging SQL functions for aggregations and time-based calculations.
- D. Create a separate BigQuery table containing pre-computed features using complex SQL queries and join this table with the raw data during model training and serving.

Answer: D

Explanation:

Analysis

OptionD directly satisfies the requirement of producing engineered features that are automatically available for both training and inference. By pre-computing the aggregations in a separate BigQuery table, the feature values are materialized and can be joined consistently when the model is trained or scored, eliminating the need for runtime calculations or manual feature pipelines. This approach leverages BigQuery's scalability and ensures that the same version of each feature is used throughout the model lifecycle.

In contrast, optionsA andC rely on real-time processing or in-query transformations that can drift between training and serving, potentially causing feature inconsistency. OptionB introduces unnecessary data movement and extra processing stages (export → Dataflow → re-import), adding complexity and latency while still requiring custom code to maintain feature consistency. Therefore, materializing the features in a dedicated BigQuery table (OptionD) provides the most reliable, cost-effective, and maintainable solution.

References

BigQuery ML documentation on using external feature tables: <https://cloud.google.com/bigquery-ml/docs/feature-tables>

Best practices for feature engineering in BigQuery: <https://cloud.google.com/bigquery/docs/feature-engineering-best-practices>

Question: 343

CertyIQ

Your organization has thousands of images stored in Cloud Storage. The organization's data analysts need to categorize these images upon analyzing the image content and store the results in a structured format for integration with other BigQuery datasets. You need a solution that allows the analysts to perform this task with the least amount of coding. What should you do?

- A. Use the Gemini 2.5 Pro model in Vertex AI to analyze the images and store the results in BigQuery.
- B. Use the ML.UNDERSTAND_TEXT function along with object tables to analyze the images, and store the JSON output in a Cloud Storage bucket.
- C. Use Vertex AI Vision API to analyze the images and store the results in BigQuery.
- D. Use the ML.GENERATE_TEXT function along with object tables to analyze the images and store the JSON output in a BigQuery table.

Answer: C

Explanation:

Justification

OptionC – VertexAI Vision API

Provides a purpose-built, serverless service for image classification and label detection that requires **no model training or custom code**.

The API returns a ready-to-use JSON payload containing detected categories, confidence scores, and other metadata.

Results can be streamed **directly into BigQuery** via Cloud Dataflow or the built-in “Export to BigQuery” feature, eliminating the need for intermediate storage or additional transformation steps.

This end-to-end flow satisfies the requirement for **minimal coding** while delivering accurate image-categorization and a structured output for downstream analytics.

OptionA – Gemini2.5Pro in VertexAI

Is a large multimodal language model intended for **generative, reasoning, and text-heavy** tasks.

While it can interpret images, it is **not optimized for efficient, large-scale image labeling** and would need custom prompt engineering, request throttling, and post-processing logic, increasing development effort.

OptionB – ML.UNDERSTAND_TEXT with object tables

The ML.UNDERSTAND_TEXT function extracts **structured information from unstructured text**, not from images.

Using object tables adds unnecessary complexity, and storing the output in Cloud Storage means an extra step to load it into BigQuery, further breaking the “least coding” requirement.

OptionD – ML.GENERATE_TEXT with object tables

Generates text from prompts or data, but is **not a dedicated image-analysis tool**.

Leveraging it for image categorization would require crafting prompts that describe images, leading to inconsistent results and additional processing to clean and load the generated JSON into BigQuery.

Conclusion – VertexAI Vision API is the most streamlined, purpose-built solution that directly converts image analysis into BigQuery-ready JSON with minimal engineering effort, making it the optimal choice for the described scenario.

References

Vertex AI Vision API documentation: <https://cloud.google.com/vertex-ai/docs/vision/overview>
Exporting Vision API results to BigQuery: <https://cloud.google.com/vertex-ai/docs/vision/guides/export-to-bigquery>

Question: 344

CertyIQ

You are building a system to process financial transactions. The system must handle a high throughput of concurrent user operations and each operation requires low-latency reads and writes to individual records. You need to identify a storage solution that guarantees ACID compliance for the processed transactions. You want a Google Cloud managed service. What should you do?

- A. Select Bigtable.
- B. Select BigQuery.
- C. Select Cloud Storage.
- D. Select Spanner.

Answer: D

Explanation:

Justification

Spanner is a globally distributed, strongly consistent relational database that offers ACID-compliant transactions across multiple regions, meeting the strict consistency and durability requirements of financial transaction processing.

It supports high query-per-second throughput with low-latency reads and writes to individual records, making it suitable for concurrent user operations.

As a fully managed Google Cloud service, Spanner eliminates operational overhead while scaling automatically to meet demand.

Bigtable is designed for massive analytical workloads with eventual consistency and does not provide ACID transactions.

BigQuery is an analytics engine optimized for large-scale data warehousing, not for real-time transactional processing.

Cloud Storage is an object store intended for bulk data archiving, not for low-latency transactional access.

References

[Cloud Spanner Documentation](#)
[Spanner Features and Pricing](#)

Question: 345

CertyIQ

You are preparing data to serve a sales demand prediction model. The training data undergoes several pre-processing steps, including scaling numerical features and one-hot encoding categorical features. The model is deployed on Vertex AI Endpoints. You need to prevent training-serving skew and ensure accurate predictions in production. You want a solution that is easy to implement.

What should you do?

- A. Implement a custom handler within the Vertex AI Endpoint to automatically perform data transformations before the model makes a prediction.
- B. Replicate the exact same pre-processing logic in the inference pipeline that was used during model training.
- C. Store the raw, unprocessed data in a separate Cloud Storage bucket exclusively for serving.
- D. Ensure the serving data is a smaller, random sample of the training data.

Answer: B

Explanation:

Technical justification

Option B – Replicate the exact same pre-processing logic in the inference pipeline is the most reliable way to eliminate training-serving skew.

Scaling and one-hot encoding are deterministic transformations; reproducing them verbatim on incoming data guarantees that the feature space seen by the model in production matches the space on which the model was trained.

This approach works regardless of model type (native or custom) because it operates at the data-ingestion layer before prediction.

Why the other options fail

A. Custom handler in a Vertex AI Endpoint introduces unnecessary complexity and maintenance overhead; the handler must mirror the training pipeline flawlessly, and any deviation can reintroduce skew.

C. Storing raw, unprocessed data in a separate bucket does not address the core issue of applying the correct transformations; serving directly from raw data would likely produce incorrect feature values.

D. Using a smaller random sample of the training data for serving does not preserve the statistical distribution of features; sampling can change feature correlations, leading to biased predictions.

Ease of implementation

The simplest production-ready solution is to embed the same preprocessing steps (e.g., using a `tf.keras.layers.Normalization` layer or scikit-learn pipelines) within the inference service, ensuring they are executed on every request before the model is invoked.

References

Vertex AI Documentation – “Preprocessing data for model predictions”

<https://cloud.google.com/vertex-ai/docs/predictions/preprocessing>

TensorFlow – “Feature Engineering and Pre-processing for TensorFlow Models”

https://www.tensorflow.org/tutorials/keras/feature_columns#preprocessing_layer

Question: 346

CertyIQ

Your retail company is concerned about their BigQuery analytics spend. The company runs several queries that require the use of the same aggregation for the store ID and real-time sales volume. You need to implement the optimal solution that minimizes analytics spend and returns faster results. What should you do?

- A. Create a new table from a CSV file with the repeated aggregation for the other queries to reference for faster processing.
- B. Create a materialized view to minimize repetitive computations.
- C. Use join acceleration with primary and foreign keys to increase query joining to live data.
- D. Leverage partitioning to minimize the number of bytes read.

Answer: B

Explanation:

Correct answer: B – Create a materialized view

A materialized view stores the result of a query physically and maintains its freshness on a schedule defined by you.

By persisting the aggregation that repeatedly joins on store_id and computes real-time sales volume, every subsequent query reads a pre-computed result instead of scanning the underlying large base tables. This eliminates the cost of re-scanning and re-aggregating the same rows each time, dramatically reducing the number of bytes processed and the query latency.

Materialized views are managed directly by BigQuery, require no extra storage-engine setup, and can be refreshed incrementally or on a fixed schedule that matches your real-time requirements.

Why the other options are less suitable

A. Create a new table from a CSV – Exporting to a CSV and loading it back adds data-load overhead, requires manual maintenance, and cannot be kept in sync automatically; it also introduces additional storage costs and does not guarantee the freshness needed for real-time analytics.

C. Use join acceleration with primary and foreign keys – Join acceleration works only for specific join patterns that meet strict column-order and indexing conditions; it does not avoid the repeated aggregation work across many queries and is less flexible when the aggregation logic itself needs to be reused.

D. Leverage partitioning – Partitioning limits the amount of data scanned when you can filter on the partition key, but it does not avoid the repeated computation of the same aggregation across many queries. If the aggregation must always scan the entire dataset, partitioning does not reduce the cost of the recurring work.

Implementation tip

Define the materialized view on the query that performs the store_id aggregation and includes the real-time sales volume column. Refresh it frequently enough (e.g., every few minutes) to meet the real-time requirement, and query the view directly for fast, low-cost results.

References

Materialized Views Overview: <https://cloud.google.com/bigquery/docs/materialized-views-intro>

Best Practices for Reducing Query Costs: https://cloud.google.com/bigquery/docs/optimizing-performance-costs#materialized_view

Question: 347

CertyIQ

You are defining the data governance strategy for a new BigQuery table with medical and financial data. You want a scalable solution that ensures the clinical researchers can access patient medical data without financial information, while allowing the accounting team to access only financial data with minimal patient identifiers. What should you do?

- A. Implement column-level security policies in BigQuery tables with IAM permissions.
- B. Create separate tables for personally identifiable information (PII), financial data, and anonymized medical data. Use IAM permissions to control access to each table.
- C. Implement row-level security policies in BigQuery tables with IAM permissions.
- D. Create separate datasets with authorized views exposing only approved data.

Answer: D

Explanation:

Correct solution – Option D

By creating separate datasets and exposing only the required columns through **authorized BigQuery views**, you can grant the clinical research team access to a view that contains the anonymized medical data while blocking any financial columns.

The accounting team receives its own dataset (or a different view) that contains only financial rows/columns, and any residual patient identifiers are eliminated or further obfuscated.

This approach enforces **dataset-level security boundaries**, minimizes cross-dataset exposure, and aligns with the **principle of least privilege** for multi-disciplinary data governance.

It also simplifies audit tracking and policy enforcement because each dataset can be independently managed and monitored in Cloud Console or through IAM policies.

Why the other options are less suitable

Option A (column-level security with IAM) – BigQuery does not support column-level IAM; permissions are applied at the **dataset or table** level. Using column-level security would require exotic workarounds and would not scale cleanly across multiple teams.

Option B (separate tables for PII, financial, anonymized data with IAM) – Although tables can be segregated, managing fine-grained access to specific columns within a table remains cumbersome. Researchers would still need to query across tables, increasing complexity and the risk of accidental cross-access.

Option C (row-level security with IAM) – Row-level security is not a native feature of BigQuery. Implementing it would require custom scripting or external tools, which defeats the purpose of a native, scalable design.

References

[BigQuery Authorized Views](#)

[Google Cloud IAM Documentation for BigQuery](#)

Question: 348

CertyIQ

You are designing a data lake on Google Cloud to store vast amounts of customer interaction data from various sources, such as websites, mobile apps, and social media. You need to ensure that this data, which arrives in different formats, is consistently cataloged and easy for data analysts to discover and use. You also want to perform basic data quality checks and transformations before the data is consumed by downstream applications. You need an automated and managed data governance solution. What should you do?

- A. Use Cloud Storage as the central repository. Use Vertex AI to classify and process the data and perform data quality checks.
- B. Stream all the data directly into BigQuery, where it is automatically cataloged and governed.
- C. Use Cloud Storage and BigQuery as repositories. Use Dataplex Universal Catalog for metadata discovery, data quality checks, and transformations.
- D. Use Cloud Storage as the central repository. Use a Cloud Run function to catalog, transform the data, and perform data quality checks.

Answer: C

Explanation:

Why option C is the optimal choice

Unified storage & query – Cloud Storage provides a scalable, cost-effective object lake, while BigQuery offers a serverless analytics engine that can directly query data stored in Cloud Storage (federated queries).

Managed metadata & governance – Dataplex Universal Catalog automatically discovers, catalogs, and enforces policy-based metadata across both Cloud Storage and BigQuery, delivering a single, searchable

data-asset view for analysts.

Integrated data-quality & transformation – Dataplex rules can trigger Dataflow or Cloud Functions for validation, cleansing, and enrichment before the data is consumed downstream, and the same catalog can expose the transformed datasets to downstream users.

Automation & managed service – All components (storage, compute, governance) are fully managed on GCP, requiring only minimal operational overhead and scaling automatically with data volume.

Why the other options are less suitable

A – Vertex AI is primarily a model-training platform; it lacks native data-cataloging and governance capabilities, and its classification functions are geared toward ML use-cases rather than enterprise-wide metadata management.

B – BigQuery does not automatically ingest all raw data formats; streaming directly into BigQuery imposes schema requirements and does not provide a comprehensive metadata layer or data-quality checks without additional orchestration.

D – Using only Cloud Run for cataloging adds custom development overhead, does not provide a persistent, searchable catalog across storage layers, and lacks built-in support for metadata governance and policy enforcement at scale.

References

Dataplex Universal Catalog documentation: <https://cloud.google.com/dataplex/docs/concepts/universal-catalog>

Integrating Cloud Storage with BigQuery federated queries: <https://cloud.google.com/bigquery/docs/storage-reads>

Question: 349

CertyIQ

You are using Dataflow to build a streaming data pipeline to analyze user website click activity from Pub/Sub. You need to calculate the number of clicks for each user site visit. A site visit is defined as a period of activity followed by 30 minutes of inactivity for a specific user. What should you do?

- A. Use tumbling windows with a 30-minute window.
- B. Use hopping windows with a 30-minute window, and a 1-minute period.
- C. Use hopping windows with a 30-minute window, and a 30-minute period.
- D. Use session windows with a 30-minute gap duration.

Answer: D

Explanation:

Why option D is the only correct choice

The requirement is to count clicks **per site-visit**, where a site-visit ends after a **30-minute gap** of inactivity for a given user.

Session windows in Apache Beam (the engine underlying Dataflow) are designed specifically for this use-case: they emit results when a fixed **gap duration** is observed, thereby grouping events that belong to the same logical session.

By configuring a session window with a **30-minute gap**, Beam automatically groups all clicks from a single user that occur before the gap and produces a single aggregation (e.g., click count) that represents that visit.

Why the other options do not work

A – Tumbling windows of 30minutes

Tumbling windows are fixed-size, non-overlapping, and do **not** collapse events across a pause of inactivity. A user could have multiple disjoint 30-minute windows in a single logical visit, causing over-counting.

B – Hopping windows of 30minutes with a 1-minute period

Hopping windows slide continuously and can produce **partial overlaps**.

They still do not treat a 30-minute idle period as a session boundary; they would keep counting clicks spread over many overlapping windows, leading to duplicated or fragmented counts.

C – Hopping windows of 30minutes with a 30-minute period

This creates a window that slides every 30minutes, effectively producing a new window **every 30minutes regardless of activity**.

It suffers from the same problem as (B) – the window logic is driven by time, not by idle gaps, so it cannot correctly delineate a session based on a 30-minute gap.

D – Session windows with a 30-minute gap

Exactly matches the definition of a site-visit: a burst of activity followed by ≤ 30 minutes of inactivity.

The framework automatically splits sessions when the gap exceeds 30minutes, ensuring each aggregation corresponds to one logical site-visit.

References

Session windows: <https://beam.apache.org/documentation/xyz/session-windows>

Windowing overview (including gap concept): <https://cloud.google.com/dataflow/docs/guides/streaming-data-transformation#windowing>

Answer: Use **session windows with a 30-minute gap duration** (option D).

Question: 350

CertyIQ

Suppose you have a table that includes a nested column called "city" inside a column called "person", but when you try to submit the following query in BigQuery, it gives you an error.

```
SELECT person FROM `project1.example.table1` WHERE city = "London"
```

How would you correct the error?

- A. Add ", UNNEST(person)" before the WHERE clause.
- B. Change "person" to "person.city".
- C. Change "person" to "city.person".
- D. Add ", UNNEST(city)" before the WHERE clause.

Answer: B

Explanation:

The error arises because the query attempts to filter a nested field ("city") directly within the "person" column without specifying the nested structure. BigQuery requires explicit pathing to access nested fields within a record. Option B, changing "person" to "person.city", correctly addresses this. It instructs BigQuery to look within the "person" record and then filter based on the "city" field.

Option A is incorrect because UNNEST is used for working with arrays, not nested records. UNNEST(person) would attempt to treat the "person" record as an array, which it isn't, leading to a different error. Option C is also incorrect; it reverses the nesting hierarchy, which is not aligned with the column structure described in the question. Option D is incorrect for similar reasons as A, using UNNEST(city) is inappropriate for this

scenario since city is a nested field within person, not an array itself.

By correctly referencing person.city, the query now properly navigates the nested data structure and filters the table based on the "city" field within each "person" record. This is a fundamental aspect of querying nested and repeated data in BigQuery.

Authoritative Links:

BigQuery documentation on querying nested and repeated data:

<https://cloud.google.com/bigquery/docs/nested-repeated>

BigQuery documentation on querying with dot notation:

<https://cloud.google.com/bigquery/docs/reference/standard-sql/data-types#struct-type> (See explanation about struct fields access)

Question: 351

CertyIQ

What are two of the benefits of using denormalized data structures in BigQuery?

- A. Reduces the amount of data processed, reduces the amount of storage required
- B. Increases query speed, makes queries simpler
- C. Reduces the amount of storage required, increases query speed
- D. Reduces the amount of data processed, increases query speed

Answer: B

Explanation:

The correct answer, B, highlights key advantages of denormalization in BigQuery. Denormalized data structures, which combine data from multiple related tables into a single table, often lead to increased query speed. This speed improvement stems from avoiding expensive JOIN operations which are necessary to combine data from multiple normalized tables. By having all the necessary information readily available in one table, BigQuery can directly access it, reducing processing time. Additionally, queries on denormalized data become simpler as they don't need complex JOIN clauses to retrieve the needed information. This streamlined query writing process results in less complex SQL, making maintenance and development easier.

Option A is incorrect because denormalization typically increases the amount of data stored, not reduces it, due to redundancy. Option C is incorrect as it implies denormalization reduces storage which is generally false; storage usually increases due to duplication of data. Option D, while partially correct about increased query speed, incorrectly states that the amount of data processed is reduced. Although the amount of data read might be reduced because there is less I/O on joins, denormalization results in more data being processed overall because duplicate data is repeated across rows.

In summary, denormalization in BigQuery prioritizes query performance by trading off storage space. It simplifies queries, making them faster and easier to write, especially when complex relationships exist between entities.

Further Research:

Google Cloud Documentation on BigQuery Schema Design:

<https://cloud.google.com/bigquery/docs/schema-design>

Understanding Denormalization: <https://www.vertica.com/blog/denormalizing-data-for-analytical-performance-advantages/>

Database Normalization vs. Denormalization: <https://www.guru99.com/database-normalization.html>

Question: 352

Which of these statements about exporting data from BigQuery is false?

- A. To export more than 1 GB of data, you need to put a wildcard in the destination filename.
- B. The only supported export destination is Google Cloud Storage.
- C. Data can only be exported in JSON or Avro format.
- D. The only compression option available is GZIP.

Answer: C

Explanation:

The correct answer is C. While BigQuery does support exporting data in JSON and Avro formats, these are not the only options. BigQuery allows exporting data in several formats, including CSV (Comma Separated Values), Parquet, and ORC (Optimized Row Columnar), among others. Therefore, statement C, which restricts export formats to only JSON or Avro, is factually incorrect. Statement A is true: for exports exceeding 1GB, BigQuery necessitates using a wildcard in the destination file name to facilitate sharding across multiple files. Statement B is also correct; Google Cloud Storage (GCS) is the exclusive destination for exporting data directly from BigQuery. Statement D is accurate as GZIP is indeed the primary compression method available for exports from BigQuery. This contrasts with the flexibility in import, where various compression formats can be employed. The limitation on export formats is imposed due to BigQuery's internal mechanisms and optimization strategies for data handling. In summary, statement C is the false assertion because it incorrectly limits the available export formats from BigQuery.

For more information, consult the official Google Cloud documentation:

Exporting Data from BigQuery: <https://cloud.google.com/bigquery/docs/exporting-data>

BigQuery Supported Export Formats: <https://cloud.google.com/bigquery/docs/export-data-formats>

Question: 353

What are all of the BigQuery operations that Google charges for?

- A. Storage, queries, and streaming inserts
- B. Storage, queries, and loading data from a file
- C. Storage, queries, and exporting data
- D. Queries and streaming inserts

Answer: A

Explanation:

The correct answer is A: Storage, queries, and streaming inserts. Google BigQuery's pricing model primarily revolves around these three consumption areas. Storage charges are incurred for the amount of data stored in BigQuery, whether it's actively queried or not. Query costs are based on the volume of data scanned during query execution, regardless of the query's complexity or outcome. Finally, streaming inserts, which involve writing data to BigQuery in near real-time, are also charged per operation and data volume.

While loading data from files and exporting data are integral BigQuery functions, they aren't directly charged. Loading data, whether through batch loads or data transfer services, is typically free of charge. Exporting data, while requiring processing power, doesn't involve the same kind of resource consumption as executing complex SQL queries and is not a billed operation. The costs for extracting and exporting data are based on where the data ends up such as the cloud storage bucket or third-party system used.

Therefore, focusing on the core operations of storing data, analyzing data with queries, and ingesting real-time data reveals the primary components of BigQuery's billing. Option B incorrectly includes data loading, and option C incorrectly includes data exporting. Option D is missing storage as a key charge area.

For more information, refer to the official Google Cloud documentation on BigQuery pricing:

<https://cloud.google.com/bigquery/pricing> and <https://cloud.google.com/bigquery/docs/introduction>. These resources detail the specific costs associated with storage, query processing, and streaming ingestion within BigQuery.

Question: 354

CertyIQ

Which of the following is not possible using primitive roles?

- A. Give a user viewer access to BigQuery and owner access to Google Compute Engine instances.
- B. Give UserA owner access and UserB editor access for all datasets in a project.
- C. Give a user access to view all datasets in a project, but not run queries on them.
- D. Give GroupA owner access and GroupB editor access for all datasets in a project.

Answer: C

Explanation:

The correct answer is C. Primitive roles in Google Cloud Platform (GCP) – Owner, Editor, and Viewer – provide broad permissions at the project level. They do not allow for granular control within specific services like BigQuery. Options A, B, and D are achievable using primitive roles. Option A, for instance, could involve granting the user the "Viewer" role on the project (providing BigQuery viewing permissions) and the "Owner" role on Compute Engine. Options B and D are direct applications of primitive roles at the project level, granting access to all project resources, including datasets.

However, option C requires a more refined level of control than primitive roles offer. While primitive roles can grant full access to all BigQuery datasets (via "Owner", "Editor", or "Viewer" project roles), they cannot differentiate between the ability to merely view datasets and the ability to run queries on them. To grant view permissions without query execution, you would need to employ Identity and Access Management (IAM) roles specific to BigQuery. These pre-defined roles, like `bigquery.dataViewer` (to view data) and `bigquery.user` (to run queries), or custom roles, allow for fine-grained access control that goes beyond the scope of primitive roles. Therefore, primitive roles alone cannot provide the specific access level described in option C. This limitation highlights the importance of utilizing service-specific IAM roles to achieve a principle of least privilege.

For more information on primitive roles, see: https://cloud.google.com/iam/docs/understanding-roles#primitive_roles For more information on BigQuery IAM roles, see: <https://cloud.google.com/bigquery/docs/access-control>

Question: 355

CertyIQ

Which of these statements about BigQuery caching is true?

- A. By default, a query's results are not cached.
- B. BigQuery caches query results for 48 hours.
- C. Query results are cached even if you specify a destination table.
- D. There is no charge for a query that retrieves its results from cache.

Answer: D

Explanation:

The correct answer is D, "There is no charge for a query that retrieves its results from cache." This statement accurately reflects BigQuery's pricing model regarding cached query results. BigQuery automatically caches query results to improve performance and reduce cost. When a query is executed, BigQuery first checks if the result is already cached. If a cached result is available and considered valid (no changes to the underlying data, identical query), BigQuery retrieves it from the cache instead of re-processing the query. This operation, referred to as cache retrieval, is completely free, as it consumes no computational resources. Option A is incorrect because BigQuery does cache query results by default. Option B is incorrect, as BigQuery's cache duration is approximately 24 hours, not 48 hours. While caching is automatic, option C is incorrect because if a destination table is specified, BigQuery will not use a cached result but write the new results to the destination table and also cache the result. Therefore, option D is the only statement correctly describing BigQuery's caching policy and its cost implication. This approach minimizes costs by avoiding redundant processing, thereby making data analysis more efficient.

Further Reading:

[BigQuery Caching Overview](#)

[BigQuery Pricing](#)

Question: 356

CertyIQ

Which of these sources can you not load data into BigQuery from?

- A. File upload
- B. Google Drive
- C. Google Cloud Storage
- D. Google Cloud SQL

Answer: D

Explanation:

The correct answer is **D. Google Cloud SQL**.

Here's why:

BigQuery is Google Cloud's fully managed, serverless data warehouse that enables scalable analysis over petabytes of data. Loading data into BigQuery involves transferring data from various sources to BigQuery's managed storage.

File Upload: BigQuery allows direct file uploads of small datasets using the BigQuery console or the command-line tool bq load. However, this method is generally not recommended for large-scale data ingestion due to size limitations and performance considerations.

Google Drive: While not a direct loading option within the BigQuery UI, you can load data from Google Drive. Typically, this is done by first copying the data to Google Cloud Storage. This approach is often used for ad-hoc analysis or for sharing data with external users who might not have access to other Google Cloud resources.

Google Cloud Storage (GCS): GCS is the primary and most efficient source for loading data into BigQuery. BigQuery can directly read data in various formats (CSV, JSON, Avro, Parquet, ORC) stored in GCS buckets, supporting efficient data transfer and transformation using BigQuery's capabilities. GCS offers scalability, durability, and cost-effectiveness, making it ideal for staging data for BigQuery.

Google Cloud SQL: Cloud SQL is a fully managed database service for relational databases like MySQL,

PostgreSQL, and SQL Server. While BigQuery can query Cloud SQL databases using federated queries (allowing you to query data residing in Cloud SQL), BigQuery does not directly load data from Cloud SQL in the same way as it does from GCS. To move data from Cloud SQL to BigQuery, you typically need to export the data from Cloud SQL to GCS (e.g., using a CSV or Avro format) and then load it into BigQuery from GCS. Alternatively, you could use Dataflow for more complex ETL processes.

Therefore, BigQuery directly supports data loading from file uploads and Google Cloud Storage, but not from Google Cloud SQL. Google Drive requires an intermediate step of copying data to GCS first. While federated queries allow querying Cloud SQL data, it's not the same as a bulk data load.

Supporting Documentation:

BigQuery Data Loading Overview: <https://cloud.google.com/bigquery/docs/loading-data>

Loading Data from Cloud Storage: <https://cloud.google.com/bigquery/docs/loading-data-cloud-storage>

External Data Sources: <https://cloud.google.com/bigquery/docs/external-data-sources>

Federated Queries to Cloud SQL: <https://cloud.google.com/bigquery/docs/cloud-sql-federated-queries>

Question: 357

CertyIQ

Which of the following statements about Legacy SQL and Standard SQL is not true?

- A. Standard SQL is the preferred query language for BigQuery.
- B. If you write a query in Legacy SQL, it might generate an error if you try to run it with Standard SQL.
- C. One difference between the two query languages is how you specify fully-qualified table names (i.e. table names that include their associated project name).
- D. You need to set a query language for each dataset and the default is Standard SQL.

Answer: D

Explanation:

Option D is incorrect because the query language setting in BigQuery is not at the dataset level; rather, it is configured at the query level. Datasets don't inherently possess a language setting; instead, BigQuery determines the language based on how you execute the query. Specifically, when running a query, you can specify whether to use Legacy SQL or Standard SQL. If you don't specify it, BigQuery typically defaults to Standard SQL in most environments, but this default applies at the context of the query, not the dataset. Options A, B, and C are true. Google recommends using Standard SQL because it is ANSI compliant and offers more robust features and functionality. Legacy SQL was the original query language, and its syntax and semantics differ significantly from Standard SQL. Therefore, attempting to execute a Legacy SQL query under Standard SQL can indeed result in errors. Moreover, the way fully-qualified table names are structured varies slightly between the two dialects. Legacy SQL uses a project:dataset.table format, while Standard SQL typically uses project.dataset.table or \project.dataset.table`.

Supporting Links:

BigQuery SQL Dialects: <https://cloud.google.com/bigquery/docs/reference/standard-sql/migrating-from-legacy-sql>

Specify Query Settings: <https://cloud.google.com/bigquery/docs/query-settings>

Standard SQL vs. Legacy SQL: https://cloud.google.com/bigquery/docs/reference/standard-sql/lexical#table_names

Question: 358

CertyIQ

How would you query specific partitions in a BigQuery table?

- A. Use the DAY column in the WHERE clause
- B. Use the EXTRACT(DAY) clause
- C. Use the __PARTITIONTIME pseudo-column in the WHERE clause
- D. Use DATE BETWEEN in the WHERE clause

Answer: C

Explanation:

The correct method to query specific partitions in a BigQuery table is by utilizing the __PARTITIONTIME pseudo-column in the WHERE clause. BigQuery employs partitioning to divide large tables into smaller segments based on time or ingestion, enhancing query performance and reducing costs. __PARTITIONTIME is an automatically generated column that represents the partition's time, which enables targeted queries. Options A, B, and D are incorrect because they attempt to filter on specific date values within user-defined columns, not directly on the partitioning mechanism. Option A uses a hypothetical DAY column, B extracts the day from a date column, and D uses a standard DATE BETWEEN clause. While these methods can filter rows, they don't specifically target partitions, therefore missing the key benefit of partitioned tables.

__PARTITIONTIME, on the other hand, allows BigQuery to directly access the specific partition without having to scan through unrelated data, resulting in faster processing times and lower cost. For instance, a query like `SELECT * FROM table WHERE __PARTITIONTIME BETWEEN TIMESTAMP('2023-10-26') AND TIMESTAMP('2023-10-27')` would only scan the partitions corresponding to those dates. This approach leverages BigQuery's partitioning optimization, a core concept in data warehousing on cloud platforms. Therefore, choosing option C aligns with best practices for efficient data retrieval from partitioned tables.

Here are some authoritative links for further research:

BigQuery Partitioned Tables: <https://cloud.google.com/bigquery/docs/partitioned-tables>

Pseudo-columns in BigQuery: <https://cloud.google.com/bigquery/docs/reference/standard-sql/pseudo-columns>

Querying partitioned tables: <https://cloud.google.com/bigquery/docs/querying-partitioned-tables>

Question: 359

CertyIQ

Which SQL keyword can be used to reduce the number of columns processed by BigQuery?

- A. BETWEEN
- B. WHERE
- C. SELECT
- D. LIMIT

Answer: C

Explanation:

The correct SQL keyword to reduce the number of columns processed by BigQuery is **C. SELECT**.

The SELECT clause is fundamental in SQL for specifying which columns from a table or view should be included in the result set. By listing only the necessary columns in the SELECT statement (e.g., `SELECT column_a, column_b FROM my_table`), you explicitly instruct BigQuery to retrieve and process only those designated columns. This is crucial for optimizing query performance and minimizing processing costs.

In contrast, WHERE (B) filters rows based on specified conditions, reducing the number of rows processed, not columns. BETWEEN (A) is a conditional operator often used within a WHERE clause, thus also affecting row filtering, not column selection. LIMIT (D) restricts the number of rows returned but still processes all columns mentioned in the SELECT clause for the specified number of rows.

Selecting fewer columns reduces data scan size, I/O operations, and memory consumption in BigQuery. This results in faster query execution and lower billing as you pay for the data processed by the query. Therefore, meticulously crafting the SELECT clause to include only necessary columns is a key optimization technique when working with large datasets in BigQuery. Limiting the columns directly impacts the query's projection operation and resource consumption. BigQuery's column-oriented storage inherently benefits from this selection; it can efficiently skip irrelevant column data blocks during data scanning.

For further information on optimizing SQL queries in BigQuery, consider these resources:

Google Cloud Documentation on Optimizing Queries: <https://cloud.google.com/bigquery/docs/best-practices-performance-optimization>

BigQuery SQL Reference - SELECT: https://cloud.google.com/bigquery/docs/reference/standard-sql/query-syntax#select_clause

Question: 360

CertyIQ

To give a user read permission for only the first three columns of a table, which access control method would you use?

- A. Primitive role
- B. Predefined role
- C. Authorized view
- D. It's not possible to give access to only the first three columns of a table.

Answer: C

Explanation:

The correct answer is **C. Authorized view**. Here's why:

Primitive roles (A) like Viewer, Editor, and Owner grant broad permissions at the resource level (project, dataset) and do not offer granular control over individual columns. Predefined roles (B), while more specific than primitive roles, also lack the ability to target specific columns within a table. Therefore, neither primitive nor predefined roles are suitable for limiting access to only the first three columns of a table. Option D, stating it's impossible, is incorrect.

Authorized views (C) are specifically designed to enable fine-grained access control in BigQuery. They act as a virtual table, defined by a SQL query, and can be used to filter data based on rows or columns, without needing to create a separate physical table. In this scenario, an authorized view would be constructed to select only the desired first three columns from the base table. The view is then shared with the user, granting them access only to the specific columns defined in its SQL query. This achieves the requirement of read permission for the first three columns only, while the user remains unaware of the underlying table schema. This concept enables data security by exposing only the necessary data to specific users and is crucial for compliance and data governance.

For further research and authoritative information, please refer to the official Google Cloud documentation:

BigQuery authorized views: <https://cloud.google.com/bigquery/docs/authorized-views>

BigQuery access control: <https://cloud.google.com/bigquery/docs/access-control>

Question: 361

CertyIQ

What are two methods that can be used to denormalize tables in BigQuery?

- A. 1) Split table into multiple tables; 2) Use a partitioned table
- B. 1) Join tables into one table; 2) Use nested repeated fields
- C. 1) Use a partitioned table; 2) Join tables into one table
- D. 1) Use nested repeated fields; 2) Use a partitioned table

Answer: B

Explanation:

Denormalization in BigQuery aims to improve query performance by reducing the need for joins. Option B accurately reflects two common denormalization techniques. Firstly, joining multiple tables into one table eliminates the overhead of performing joins during query execution. This pre-computation of relationships can significantly speed up analytical queries, where data from multiple related tables is often required. For instance, product information and sales data can be combined into a single denormalized table. Secondly, using nested repeated fields allows you to represent one-to-many relationships within a single record. For example, order details (multiple items) can be nested under an order record, avoiding the need for separate order items table. This nested structure simplifies queries by accessing related data directly within the same row.

Option A's split table concept moves away from denormalization, suggesting normalization instead. Option C includes partitioned tables, which improve query efficiency by limiting the scan size, but this isn't denormalization. Option D includes partitioned tables again and doesn't highlight that a primary denormalization strategy is to join tables. By removing the need for joins, the data retrieval process becomes far more efficient, improving query speed. The use of nested and repeated fields further reduces the need for additional joins by packaging related entities inside of a record. These approaches are commonly used to optimize query performance in analytical workloads, where data redundancy is acceptable in exchange for speed.

Further Reading:

BigQuery Denormalization Strategies: <https://cloud.google.com/blog/topics/developers-practitioners/denormalizing-bigquery-data-for-better-performance>

BigQuery Data Modeling: <https://cloud.google.com/bigquery/docs/data-modeling>

BigQuery Nested and Repeated Fields: <https://cloud.google.com/bigquery/docs/reference/standard-sql/data-types#nested-and-repeated-fields>

Question: 362

CertyIQ

Which of these is not a supported method of putting data into a partitioned table?

- A. If you have existing data in a separate file for each day, then create a partitioned table and upload each file into the appropriate partition.
- B. Run a query to get the records for a specific day from an existing table and for the destination table, specify a partitioned table ending with the day in the format "\$YYYYMMDD".
- C. Create a partitioned table and stream new records to it every day.
- D. Use ORDER BY to put a table's rows into chronological order and then change the table's type to "Partitioned".

Answer: D

Explanation:

The correct answer is D because it describes an invalid approach to creating partitioned tables in BigQuery. Partitioning fundamentally relies on organizing data based on a partitioning column (e.g., ingestion time, timestamp, integer range) as it's being loaded or created. Changing the table type after data is already loaded and sorted doesn't retrospectively partition that existing data. BigQuery's partitioning mechanism is inherently tied to data loading and ingest processes. Options A, B, and C correctly illustrate valid methods. Option A demonstrates direct loading into specific partitions based on file organization, aligning with data organization pre-load. Option B exemplifies using a query with destination table modification to dynamically add data into a partition based on the date present within the query. Finally, option C highlights the use of streaming data into partitions using the ingestion time. In contrast, option D falsely implies that sorting and then converting the table's type will enforce partitioning on previously unpartitioned data; this is not how BigQuery's partitioning works. Partitioning must be declared before or at the time of loading or creating a new table. BigQuery's partitioning documentation clearly outlines methods that involve specifying the partition key/type at the time of table creation or data insertion. In summary, BigQuery partitioning is an intrinsic property of data loading and creation processes, not a post-processing step applied to an existing table.

Relevant documentation:

Introduction to partitioned tables: This official documentation from Google provides comprehensive information on partitioned tables in BigQuery, their structure, and how they are created.

Creating and using partitioned tables: This page describes different ways to create partitioned tables within BigQuery.

Question: 363

CertyIQ

Which of these operations can you perform from the BigQuery Web UI?

- A. Upload a file in SQL format.
- B. Load data with nested and repeated fields.
- C. Upload a 20 MB file.
- D. Upload multiple files using a wildcard.

Answer: B

Explanation:

Let's analyze why option B is the correct answer and why the others are incorrect regarding BigQuery Web UI capabilities.

Option B, "Load data with nested and repeated fields," accurately describes a core function of the BigQuery Web UI. The UI provides a user-friendly interface to specify schema definitions, including the handling of nested (hierarchical) and repeated (array-like) fields during data loading from various sources like Cloud Storage, local files, or Google Drive. Defining such complex data structures via the UI makes it accessible to non-developers or those unfamiliar with command-line tools. The UI generates the correct JSON metadata based on user inputs.

Option A, "Upload a file in SQL format," is incorrect. While the BigQuery Web UI allows you to write and execute SQL queries, it doesn't directly "upload" a SQL file itself. SQL scripts can be saved and managed within BigQuery, but they aren't handled as data loading resources in the Web UI. The purpose of the UI is data management, querying, and results analysis.

Option C, "Upload a 20 MB file," while possible through other means, is not the primary method via the BigQuery Web UI. The UI focuses on smaller data uploads or linking to external storage locations like Google Cloud Storage. Directly uploading large local files through the browser is neither efficient nor the intended way to load large datasets in BigQuery. The Web UI is for quick loads, while bulk loads should happen through APIs or other Cloud Storage based options.

Option D, "Upload multiple files using a wildcard," is also incorrect within the BigQuery Web UI. While loading from Google Cloud Storage can utilize wildcards for batch loading operations, the BigQuery UI does not directly offer this functionality for local file uploads. When using GCS, this happens with schema inference and requires a linked service configuration for Cloud Storage, not direct UI manipulation of local file wildcards.

In conclusion, the BigQuery Web UI is designed for user-friendly interaction with data, particularly for tasks like data loading with complex schema definitions, making option B the accurate choice. While the UI can be used for small files, it's not meant for bulk uploads or uploading executable SQL files.

Further information can be found at:

[BigQuery Data Loading Overview](#): Explains the various methods for loading data into BigQuery.

[BigQuery Web UI Documentation](#): Provides details on using the BigQuery Web UI.

[BigQuery Schemas](#): Details on nested and repeated fields.

Question: 364

CertyIQ

Which methods can be used to reduce the number of rows processed by BigQuery?

- A. Splitting tables into multiple tables; putting data in partitions
- B. Splitting tables into multiple tables; putting data in partitions; using the LIMIT clause
- C. Putting data in partitions; using the LIMIT clause
- D. Splitting tables into multiple tables; using the LIMIT clause

Answer: A

Explanation:

The correct answer is **A. Splitting tables into multiple tables; putting data in partitions.**

Here's why:

Splitting tables into multiple tables, often referred to as denormalization or sharding, can reduce the number of rows scanned during queries. By distributing data across different tables based on specific criteria (e.g., by year or customer segment), you can target queries to only the necessary tables, thus avoiding full table scans. This reduces the processing scope and associated costs.

Partitioning, similarly, helps in reducing the data scanned. Partitions segment a table into logical parts based on a column value, such as date or region. When a query includes a filter on the partitioning column, BigQuery can efficiently scan only the relevant partitions, skipping the rest, leading to substantial performance improvements and cost reduction. For example, if you have a table partitioned by date and you query data from a single month, BigQuery will only process the data within that month's partition.

Option B is incorrect because the LIMIT clause, while reducing the results returned, does not reduce the rows processed. The entire dataset satisfying the WHERE clause still has to be scanned for evaluation, the LIMIT clause simply cuts off the final output.

Options C and D are incorrect as they omit the valuable table splitting technique, which when correctly

employed can also drastically reduce scanned data. Splitting tables can allow queries to target smaller, more relevant table structures, in addition to partitioning.

Therefore, splitting tables combined with partitioning provides the most comprehensive approach to reduce the number of rows processed by BigQuery queries.

Authoritative Links:

BigQuery Partitioned Tables: <https://cloud.google.com/bigquery/docs/partitioned-tables>

BigQuery Best Practices for Cost Control: <https://cloud.google.com/bigquery/docs/best-practices-costs>
(Search for "Partitioning" and "Clustering")

Optimizing Queries in BigQuery: <https://cloud.google.com/bigquery/docs/optimizing-queries>

Question: 365

CertyIQ

Why do you need to split a machine learning dataset into training data and test data?

- A. So you can try two different sets of features
- B. To make sure your model is generalized for more than just the training data
- C. To allow you to create unit tests in your code
- D. So you can use one dataset for a wide model and one for a deep model

Answer: B

Explanation:

Splitting a machine learning dataset into training and test sets is crucial for evaluating a model's ability to generalize, which is the primary reason option B is correct. The training dataset is used to teach the model patterns and relationships within the data, allowing it to learn the underlying trends. Crucially, if the model were evaluated only on the training data, it could simply memorize the specific instances, leading to excellent performance on that set, but dismal performance on new, unseen data. This phenomenon is known as overfitting, a major pitfall in machine learning. The test dataset, on the other hand, simulates real-world data the model hasn't seen before. By assessing the model's performance on this independent dataset, we gain a more realistic understanding of its effectiveness and predictive capabilities. A good model should perform well on both the training and test sets. If the model performs significantly better on the training set than the test set, it's likely overfit and will not generalize well to new data. This process of model evaluation on unseen data is foundational for developing robust and reliable machine learning solutions. Option A is incorrect because feature engineering happens before splitting, and options C and D are irrelevant to the primary purpose of the split.

Further research can be found here:

Google Cloud AI Platform Training Overview: <https://cloud.google.com/ai-platform/training/docs/overview> -

This documentation outlines the training process on Google Cloud and highlights the necessity of proper validation strategies.

Data Splitting Strategies: <https://developers.google.com/machine-learning/data-prep/divide-data> - This article provides a good explanation of why and how to split data into training, validation, and test sets.

Overfitting: https://www.tensorflow.org/tutorials/keras/overfit_and_underfit - This TensorFlow tutorial explores overfitting in detail, explaining its negative impact on model generalization.

Question: 366

CertyIQ

Which of these numbers are adjusted by a neural network as it learns from a training dataset (select 2 answers)?

- A. Weights
- B. Biases
- C. Continuous features
- D. Input values

Answer: AB

Explanation:

Neural networks learn by adjusting their internal parameters to minimize the difference between their predictions and the actual values in the training data. These adjustable parameters are primarily weights and biases. Weights represent the strength of connections between neurons in different layers, determining the influence of one neuron's output on another. Biases, on the other hand, are added to the weighted sum of inputs to introduce flexibility, allowing the network to learn even when inputs are all zero. The neural network iteratively adjusts these weights and biases during the training process using optimization algorithms like gradient descent. Continuous features, which are the input data themselves, are not modified by the neural network; they remain constant throughout training. Input values also remain unchanged during training as they form the basis of the model's learning. They drive the calculations but are not altered. The changes happen within the connections (weights) and the thresholds (biases).

Here are some links to authoritative resources:

Google Cloud Documentation on AI/ML: <https://cloud.google.com/learn/what-is-machine-learning> (This provides a general overview of machine learning concepts)

TensorFlow Documentation on Neural Network Basics:

https://www.tensorflow.org/guide/keras/train_and_evaluate (This offers in-depth information on how neural networks train and how parameters get adjusted)

Deep Learning Book (MIT Press): <https://www.deeplearningbook.org/> (This is a comprehensive resource for deep learning fundamentals, including weights and biases.)

Question: 367

CertyIQ

The CUSTOM tier for Cloud Machine Learning Engine allows you to specify the number of which types of cluster nodes?

- A. Workers
- B. Masters, workers, and parameter servers
- C. Workers and parameter servers
- D. Parameter servers

Answer: C

Explanation:

The correct answer is **C. Workers and parameter servers**. Google Cloud Machine Learning Engine (now Vertex AI) offers pre-defined and custom tiers for training jobs. When using the CUSTOM tier, you gain fine-grained control over the compute resources. Specifically, you can configure the number of worker nodes, which perform the core training computations, and parameter server nodes, which manage the model parameters during distributed training. Master nodes, on the other hand, are managed by the system and are not configurable by the user in the CUSTOM tier. Therefore, option B is incorrect. The absence of direct control over master nodes distinguishes the CUSTOM tier. Option A is partially correct, but it misses the inclusion of parameter servers, which are also configurable in the CUSTOM tier. Option D only refers to parameter servers, thus incomplete. The parameter server architecture facilitates faster training for larger

datasets and complex models through distributed optimization. Customization enables cost optimization and resource alignment to meet the specific needs of your ML workload. In summary, the CUSTOM tier provides flexibility by allowing users to tailor the scale of worker and parameter server nodes.

Authoritative Links:

[Vertex AI Training documentation: Custom training Overview of distributed training](#)
[Vertex AI Pricing](#)

Question: 368

CertyIQ

Which software libraries are supported by Cloud Machine Learning Engine?

- A. Theano and TensorFlow
- B. Theano and Torch
- C. TensorFlow
- D. TensorFlow and Torch

Answer: C

Explanation:

Cloud Machine Learning Engine (now known as Vertex AI) primarily supports TensorFlow for model training and prediction. This support is deeply integrated, allowing seamless use of TensorFlow's APIs for defining, training, and deploying machine learning models. The service optimizes its infrastructure to handle the computational demands of TensorFlow operations, such as those involving GPUs and TPUs. While other frameworks like PyTorch and Scikit-learn are popular for machine learning, Vertex AI directly supports these frameworks via custom containers and other methods, but they are not the primary native integrations. Although some services allow containerized deployments of models trained in different frameworks, the core engine and pre-built estimator capabilities focus on TensorFlow. The answer C aligns with this core design and functionality. The use of TensorFlow within Vertex AI streamlines the workflow for data scientists familiar with Google's own ML framework, making it a natural choice when using this particular cloud service.

[Further Research]

Vertex AI Overview: <https://cloud.google.com/vertex-ai/docs/general/overview> - This documentation outlines the capabilities of Vertex AI, highlighting TensorFlow's prominent role.

Vertex AI Training Overview: <https://cloud.google.com/vertex-ai/docs/training/overview> - This section details the training process and the integration with TensorFlow in Vertex AI.

Training with custom containers: <https://cloud.google.com/vertex-ai/docs/training/create-custom-container> - This demonstrates how you can use other frameworks by containerizing them.

Question: 369

CertyIQ

Which TensorFlow function can you use to configure a categorical column if you don't know all of the possible values for that column?

- A. categorical_column_with_vocabulary_list
- B. categorical_column_with_hash_bucket
- C. categorical_column_with_unknown_values
- D. sparse_column_with_keys

Answer: B

Explanation:

The correct answer is **B. categorical_column_with_hash_bucket**. Here's why:

When dealing with categorical features in TensorFlow, you often need to convert textual or non-numerical data into numerical representations for machine learning models. `categorical_column_with_vocabulary_list` (option A) requires you to explicitly provide a list of all possible values the column can take. This isn't feasible when you don't know them beforehand, which is the scenario described in the question. Option C, `categorical_column_with_unknown_values`, does not exist in TensorFlow. `sparse_column_with_keys` (option D) is used for handling sparse columns where most values are zeros, and isn't directly relevant to unknown categorical values.

`categorical_column_with_hash_bucket`, on the other hand, uses a hash function to map categorical values to a specific number of buckets. This allows TensorFlow to handle categories without knowing all possible values beforehand. It's a powerful technique because it allows you to learn representations of categories even if new, unseen values are introduced during training or inference. By using a hashing mechanism, it handles potential collisions or similar values by grouping them into a specified number of buckets. This is particularly useful with datasets that are constantly evolving or when dealing with large datasets where enumerating all possible categories would be impractical.

Therefore, `categorical_column_with_hash_bucket` is the appropriate choice when you lack prior knowledge about the possible values of a categorical feature and you need to handle potentially unknown values efficiently.

Authoritative Links for Further Research:

TensorFlow documentation on Feature Columns:

https://www.tensorflow.org/api_docs/python/tf/feature_column (Specifically look at `tf.feature_column.categorical_column_with_hash_bucket`)

Guide on Feature Columns in TensorFlow:

https://www.tensorflow.org/tutorials/structured_data/feature_columns

Question: 370

CertyIQ

Which of the following statements about the Wide & Deep Learning model are true? (Select 2 answers.)

- A. The wide model is used for memorization, while the deep model is used for generalization.
- B. A good use for the wide and deep model is a recommender system.
- C. The wide model is used for generalization, while the deep model is used for memorization.
- D. A good use for the wide and deep model is a small-scale linear regression problem.

Answer: AB

Explanation:

The correct answer is **A and B**. Let's delve into why.

Option A is correct: Wide and Deep models are designed with two distinct components. The "wide" part focuses on capturing feature interactions and directly associating them with the target variable, achieving memorization of specific patterns in the training data. This component is often a linear model with sparse features. Conversely, the "deep" part, typically a neural network with multiple layers, learns complex, non-linear relationships and representations, thus promoting generalization to unseen data. This allows the model

to infer beyond the exact training examples.

Option B is correct: Recommender systems are a prime application for Wide & Deep models. They benefit from both the memorization capabilities of the wide component (e.g., users who previously liked item X also tend to like item Y) and the generalization power of the deep component (e.g., discovering implicit preferences based on user and item attributes). By leveraging both aspects, the model can provide more personalized and relevant recommendations.

Option C is incorrect: This statement reverses the roles of the wide and deep components. The wide model handles memorization, not generalization. And, the deep model is designed for generalization through the learning of complex patterns, not memorization.

Option D is incorrect: Small-scale linear regression problems typically don't require the complexity of a Wide & Deep model. Simple linear regression or other traditional linear models often suffice for such scenarios. Wide & Deep architectures are better suited for large-scale datasets with complex feature relationships.

In essence, the Wide & Deep model is a hybrid approach designed to combine the strengths of both linear models and deep neural networks to address both memorization and generalization requirements effectively, particularly in recommendation systems and other large-scale classification tasks.

Authoritative Links for further research:

Google Research Blog on Wide & Deep Learning: <https://ai.googleblog.com/2016/06/wide-deep-learning-better-together-with.html>

TensorFlow Tutorial on Wide & Deep Learning:
https://www.tensorflow.org/tutorials/structured_data/wide_deep

Original paper: "Wide & Deep Learning for Recommender Systems" <https://arxiv.org/abs/1606.07792>

Question: 371

CertyIQ

To run a TensorFlow training job on your own computer using Cloud Machine Learning Engine, what would your command start with?

- A. `gcloud ml-engine local train`
- B. `gcloud ml-engine jobs submit training`
- C. `gcloud ml-engine jobs submit training local`
- D. You can't run a TensorFlow program on your own computer using Cloud ML Engine .

Answer: A

Explanation:

The correct command to execute a TensorFlow training job locally using Cloud Machine Learning Engine (CMLE) starts with `gcloud ml-engine local train`. This command allows developers to test and debug their TensorFlow models on their local machines before deploying them to the cloud for distributed training. CMLE provides a consistent interface for both local and cloud execution, making it easier to develop and deploy machine learning models.

Option A directly reflects this functionality, signaling to CMLE that the training process should be executed on the local machine. This is crucial for rapid iteration and debugging, enabling developers to identify issues early in the development cycle without incurring cloud compute costs.

Option B, `gcloud ml-engine jobs submit training`, is used to submit a training job to the cloud for execution on CMLE's infrastructure. This command is appropriate for large-scale, distributed training runs but not for local execution.

Option C, gcloud ml-engine jobs submit training local, is not a valid CMLE command. The submit subcommand is inherently designed for cloud execution, and the addition of "local" creates a contradiction.

Option D is incorrect because CMLE explicitly supports local training through the gcloud ml-engine local train command. This feature is designed to facilitate local development and testing of ML models. Using this approach, you can verify your model's functionality and debug any potential issues on your local environment before moving to larger-scale cloud-based training.

Therefore, the gcloud ml-engine local train command provides a mechanism to test and debug locally while leveraging CMLE's framework, making it the right answer. This approach significantly reduces development time and costs associated with using cloud resources prematurely.

Further Research:

[Cloud ML Engine Documentation](#)
[gcloud ml-engine local train](#)

Question: 372

CertyIQ

If you want to create a machine learning model that predicts the price of a particular stock based on its recent price history, what type of estimator should you use?

- A. Unsupervised learning
- B. Regressor
- C. Classifier
- D. Clustering estimator

Answer: B

Explanation:

The correct answer is **B. Regressor**. Here's why:

The core task is to predict a continuous numerical value, which is the stock price. Regression models are specifically designed for this type of prediction. They establish a relationship between input variables (in this case, past stock prices) and a continuous output variable (the future stock price). Unlike classifiers that predict discrete categories (like "buy" or "sell"), a regressor outputs a specific numerical value within a range. Unsupervised learning (option A) is not suitable here as it deals with identifying patterns in unlabeled data, not prediction. Clustering estimators (option D), are a type of unsupervised technique, used for grouping data points based on similarity, not for predicting a continuous value. While time-series data is often used in stock prediction, the choice of the model is dictated by the nature of the output which is a continuous value thus making a regressor model suitable. Examples of regressors would include linear regression, polynomial regression, or Support Vector Regression. These models are trained using historical data, allowing them to learn the underlying relationships between past and future stock prices, and thus, making predictions.<https://developers.google.com/machine-learning/crash-course/ml-intro><https://cloud.google.com/ml-engine/docs/ml-solutions/regression>https://scikit-learn.org/stable/supervised_learning.html#supervised-learning

Question: 373

CertyIQ

Suppose you have a dataset of images that are each labeled as to whether or not they contain a human face. To create a neural network that recognizes human faces in images using this labeled dataset, what approach would likely be the most effective?

- A. Use K-means Clustering to detect faces in the pixels.
- B. Use feature engineering to add features for eyes, noses, and mouths to the input data.
- C. Use deep learning by creating a neural network with multiple hidden layers to automatically detect features of faces.
- D. Build a neural network with an input layer of pixels, a hidden layer, and an output layer with two categories.

Answer: C

Explanation:

The most effective approach is **C. Use deep learning by creating a neural network with multiple hidden layers to automatically detect features of faces.** Here's why:

Deep learning excels at image recognition tasks because it can automatically learn hierarchical features from raw pixel data. Convolutional Neural Networks (CNNs), a type of deep learning model, are particularly well-suited for this. Unlike manual feature engineering (option B), CNNs can learn complex and abstract features like edges, shapes, and textures in their initial layers, and combine them into face components (eyes, nose, mouth) in deeper layers without explicit programming. This "feature learning" eliminates the need for domain expertise to manually define features. A simple neural network with a single hidden layer (option D) may not possess sufficient capacity to capture these complex facial patterns. K-means clustering (option A) is an unsupervised learning algorithm designed for partitioning data into clusters based on similarity, not for learning to discriminate between labeled classes, making it ineffective for this supervised classification problem. Deep learning models, specifically CNNs, can leverage the large amount of data present in image datasets and have proven to achieve state-of-the-art performance in image recognition tasks such as object detection and classification. Deep learning frameworks like TensorFlow or PyTorch provide tools to easily construct and train these complex models on cloud infrastructure like Google Cloud Platform's AI Platform. This allows for scalable training of models using GPUs or TPUs, which is crucial when dealing with large image datasets.

Authoritative Links:

Google Cloud AI Platform: <https://cloud.google.com/ai-platform> - Learn about Google's managed platform for training and deploying ML models.

Convolutional Neural Networks: <https://developers.google.com/machine-learning/practica/image-classification/convolutional-neural-networks> - Google's explanation of CNNs for image recognition.

Deep Learning: <https://www.tensorflow.org/> or <https://pytorch.org/> - Explore popular deep learning frameworks.

Question: 374

CertyIQ

What are two of the characteristics of using online prediction rather than batch prediction?

- A. It is optimized to handle a high volume of data instances in a job and to run more complex models.
- B. Predictions are returned in the response message.
- C. Predictions are written to output files in a Cloud Storage location that you specify.
- D. It is optimized to minimize the latency of serving predictions.

Answer: BD

Explanation:

Let's analyze the characteristics of online prediction versus batch prediction in the context of Google Cloud Platform's AI Platform (now Vertex AI). Online prediction, also known as real-time prediction, is designed for situations where low latency is critical. It returns predictions directly within the API response, making it

suitable for applications needing immediate results, such as user-facing web applications or mobile apps. Therefore, option B ("Predictions are returned in the response message.") is correct. The key performance metric for online prediction is minimizing latency, meaning how quickly a prediction is received after the request. Option D ("It is optimized to minimize the latency of serving predictions.") accurately reflects this focus. Batch prediction, conversely, is optimized for throughput and handling large datasets where immediate responses aren't needed. Batch predictions are usually written to Cloud Storage for later analysis or use. This directly contradicts options A and C. Option A, describing high volume jobs and complex models, is a characteristic of batch prediction, not online. Similarly, option C, stating results are written to Cloud Storage, also describes batch. The critical difference lies in the delivery method – responses versus stored files – and latency versus throughput, making B and D the correct characteristics of online prediction.

Further research can be conducted using these resources:

Vertex AI documentation on online predictions: <https://cloud.google.com/vertex-ai/docs/predictions/online-predictions>

Vertex AI documentation on batch predictions: <https://cloud.google.com/vertex-ai/docs/predictions/batch-predictions>

Comparison of prediction types: <https://cloud.google.com/vertex-ai/docs/predictions/get-predictions>

Question: 375

CertyIQ

Which of these are examples of a value in a sparse vector? (Select 2 answers.)

- A. [0, 5, 0, 0, 0, 0]
- B. [0, 0, 0, 1, 0, 0, 1]
- C. [0, 1]
- D. [1, 0, 0, 0, 0, 0, 0]

Answer: CD

Explanation:

Let's analyze the characteristics of sparse vectors to understand why options C and D are the correct choices. Sparse vectors are vectors where most of the elements are zero. Instead of storing all these zeros, they are often represented by only storing the non-zero values and their corresponding indices. This approach is crucial for efficient storage and processing, especially with high-dimensional data.

Options A and B, while containing a number of zeros, are not presented in the compact, index-based form typical of sparse vector representations. The key distinction is that the provided choices represent dense vectors with explicit zeros, not the sparse vector representation.

Option C, "[0, 1]", can be interpreted as representing a sparse vector where the first element at index 0 is zero, and a second element at an index 1 has a value of 1, which demonstrates the representation of a sparse vector with one value at a specific index. Option D, "[1, 0, 0, 0, 0, 0, 0]", represents a sparse vector where the first index has a value of 1, and the rest are implicit zeros. It also is a potential valid representation of a sparse vector.

In summary, options C and D showcase a sparse vector representation by only explicitly defining the values that aren't zeros, though without explicit index notations. Options A and B, while dense, do not illustrate the core characteristic of sparse vector storage.

For further research on sparse vector representations and their applications in data engineering, consider exploring resources on machine learning and data storage optimization. Specifically, look into concepts like sparse matrices and coordinate lists.

Authoritative Links:

Sparse matrix: https://en.wikipedia.org/wiki/Sparse_matrix

Coordinate list (COO format):

https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.coo_matrix.html (While COO relates to sparse matrices, it applies conceptually to sparse vector representations.)

Question: 376

CertyIQ

How can you get a neural network to learn about relationships between categories in a categorical feature?

- A. Create a multi-hot column
- B. Create a one-hot column
- C. Create a hash bucket
- D. Create an embedding column

Answer: D

Explanation:

The correct answer is **D. Create an embedding column**. Here's why:

Neural networks, unlike traditional models, don't inherently understand the relationships between categorical values represented as arbitrary labels (e.g., "red," "blue," "green"). To capture these relationships, we need to convert categories into numerical representations where semantic similarities are reflected by closeness in the numerical space.

One-hot encoding (B) creates a separate binary feature for each category. While this is useful for inputting categorical data to a neural network, it treats each category as entirely independent, thus failing to capture any relationships. Multi-hot encoding (A) is generally used for multi-label data, not relationships between categories within a single feature. Hash buckets (C) reduce dimensionality but can lead to collisions and don't explicitly learn category relationships.

Embedding columns, on the other hand, represent each category as a dense, low-dimensional vector. The neural network learns these vector representations during training. Categories with similar meanings or that appear together frequently in the data will tend to have closer embeddings in the vector space. This enables the network to discern and utilize relationships between categories for improved prediction accuracy. For instance, a model might learn that the embeddings for "cat" and "dog" are closer than those for "cat" and "car."

Using embedding columns allows the neural network to automatically learn a meaningful representation of categorical features based on the context within the dataset, allowing for generalization beyond simple one-hot encoding. This is a fundamental technique in deep learning when working with categorical data.

Further research:

TensorFlow Embeddings:

https://www.tensorflow.org/tutorials/structured_data/feature_columns#embedding_columns

Google Cloud Blog - Better ML models with categorical embeddings:

<https://cloud.google.com/blog/products/ai-machine-learning/better-ml-models-with-categorical-embeddings>

PyTorch Embedding Layers: <https://pytorch.org/docs/stable/generated/torch.nn.Embedding.html>

Question: 377

CertyIQ

If a dataset contains rows with individual people and columns for year of birth, country, and income, how many of the columns are continuous and how many are categorical?

- A. 1 continuous and 2 categorical
- B. 3 categorical
- C. 3 continuous
- D. 2 continuous and 1 categorical

Answer: A

Explanation:

Here's a detailed justification for why the correct answer is A: 1 continuous and 2 categorical.

The dataset consists of columns representing 'year of birth', 'country', and 'income'. Let's analyze each column's data type:

Year of birth: Although expressed as a whole number, it represents a point on a continuous timeline. Birth years can be meaningfully ordered, and differences between them have practical interpretations (e.g., age). This makes 'year of birth' a **continuous** variable. Note that even though the data is discrete in the sense that we do not have partial years, the nature of the data is such that it could be any value on the continuous timeline so it should still be treated as continuous.

Country: This column represents distinct, non-overlapping categories (e.g., USA, UK, France). There's no inherent order or mathematical relationship between different countries. Therefore, 'country' is a **categorical** variable.

Income: Income is a numerical value that represents the amount of money someone earns. This is an example of a **continuous** variable because values have meaning along an ordered range. Income could have decimal values which make the data type continuous.

Therefore we have one column representing **Year of birth** as continuous, and two columns representing **country** and **income** as categorical. So the answer is 1 continuous and 2 categorical.

Categorical data refers to variables with a limited set of possible values that represent distinct groups or categories. Continuous data, on the other hand, represents values that fall along a numerical scale with potentially infinite values between two given points. Understanding data types is fundamental for data analysis and machine learning tasks in Google Cloud Platform (GCP), where selecting the appropriate algorithms and techniques depends on the variable types. This can be further researched via the following links:

Google Cloud documentation on data types: <https://cloud.google.com/dataproc/docs/concepts/data-types>

General resource on continuous vs categorical data: <https://statisticsbyjim.com/basics/continuous-vs-discrete-data/>

Overview of data types in Google Cloud's BigQuery

<https://cloud.google.com/bigquery/docs/reference/standard-sql/data-types>

Understanding these concepts enables you to make better choices when using GCP services such as BigQuery, Dataflow, and Cloud ML Engine.

Question: 378

CertyIQ

Which of the following are examples of hyperparameters? (Select 2 answers.)

- A. Number of hidden layers

- B. Number of nodes in each hidden layer
- C. Biases
- D. Weights

Answer: AB

Explanation:

Hyperparameters are parameters that are set before the learning process begins, controlling the training process itself. They are not learned by the model during training. The number of hidden layers (A) and the number of nodes in each hidden layer (B) directly impact the architecture and complexity of a neural network and are defined before training commences. These values affect how the model learns representations of the data. Biases (C) and weights (D) are model parameters that the model learns during training through optimization algorithms. They are adjusted iteratively to minimize the loss function. Hyperparameter selection is critical for model performance and often requires experimentation and validation using techniques like grid search or random search. It defines the overall structure and learning configuration, unlike model parameters that are optimized based on the training data. Therefore, A and B are hyperparameters, whereas C and D are parameters learned through training.

Further Research:

1. **Google Cloud Documentation on Hyperparameter Tuning:** <https://cloud.google.com/vertex-ai/docs/training/hyperparameter-tuning> - Provides Google's perspective on hyperparameter tuning in machine learning.
2. **Machine Learning Mastery - Hyperparameters:** <https://machinelearningmastery.com/hyperparameters-for-deep-learning/> - Detailed blog post explaining common hyperparameters in deep learning.

Question: 379

CertyIQ

Which of the following are feature engineering techniques? (Select 2 answers)

- A. Hidden feature layers
- B. Feature prioritization
- C. Crossed feature columns
- D. Bucketization of a continuous feature

Answer: CD

Explanation:

The correct answer is C and D because they represent common feature engineering techniques used to improve model performance.

C. Crossed feature columns (also known as Feature Crossing): Feature crossing involves creating new features by combining two or more existing features. The aim is to capture interaction effects between the original features that a linear model, for instance, would otherwise miss. For example, combining "city" and "product" creates a new feature "city_product," which can capture variations in demand for specific products in specific cities. This interaction can significantly improve the model's ability to predict outcomes. In the context of Google Cloud, you can leverage TensorFlow Feature Columns for easy feature crossing.

D. Bucketization of a continuous feature (also known as Binning): This technique involves dividing a continuous numerical feature into discrete buckets or bins. This transformation can be beneficial for several

reasons. First, it can handle non-linear relationships between the feature and the target variable by allowing the model to treat different ranges of the feature differently. Second, it can reduce the impact of outliers by grouping them into the same bucket. Finally, it can make the feature more interpretable. Cloud Dataflow is a service suitable for implementing bucketization pipelines at scale.

A. Hidden feature layers are related to the architecture of deep learning models. Hidden layers are not a feature engineering technique but a component within neural networks. They are part of the model itself, not a preprocessing step.

B. Feature prioritization, while important in model development, refers to ranking or selecting features based on their importance or impact on the model. It's not a technique to create new features from existing ones; rather, it's about choosing which features to use. This can be achieved through techniques like permutation importance or feature selection algorithms integrated into tools like Vertex AI.

In summary, feature crossing and bucketization are methods that generate new, potentially more informative features from the original dataset, directly contributing to enhanced model accuracy.

Supporting links:

TensorFlow Feature Columns: https://www.tensorflow.org/api_docs/python/tf/feature_column

Cloud Dataflow: <https://cloud.google.com/dataflow/docs>

Vertex AI Feature Selection: <https://cloud.google.com/vertex-ai/docs/featurestore/latest/create-select-features>

Question: 380

CertyIQ

You want to use a BigQuery table as a data sink. In which writing mode(s) can you use BigQuery as a sink?

- A. Both batch and streaming
- B. BigQuery cannot be used as a sink
- C. Only batch
- D. Only streaming

Answer: A

Explanation:

The correct answer is A, indicating that BigQuery can be used as a data sink in both batch and streaming write modes. This capability stems from BigQuery's design as a scalable, serverless data warehouse suitable for both large-scale data loading (batch) and continuous data ingestion (streaming). In batch mode, large datasets are typically loaded into BigQuery using methods like Cloud Storage transfers or through BigQuery's load jobs, which efficiently handle the bulk import of data. In contrast, streaming data ingestion uses BigQuery's Storage Write API, which enables near real-time data insertion into tables, making it suitable for use cases such as IoT sensor data or web analytics streams. This functionality makes BigQuery highly versatile, allowing for both historical analysis and real-time insights. Therefore, choosing option A accurately reflects BigQuery's ability to serve as a data sink for both types of data processing paradigms.

Authoritative links:

BigQuery Batch Loading: <https://cloud.google.com/bigquery/docs/loading-data>

BigQuery Streaming Ingestion: <https://cloud.google.com/bigquery/streaming-data-into-bigquery>

BigQuery Storage Write API: <https://cloud.google.com/bigquery/docs/write-api>

Question: 381

You have a job that you want to cancel. It is a streaming pipeline, and you want to ensure that any data that is in-flight is processed and written to the output.

Which of the following commands can you use on the Dataflow monitoring console to stop the pipeline job?

- A. Cancel
- B. Drain
- C. Stop
- D. Finish

Answer: B

Explanation:

The correct answer is B, Drain. Here's why:

The goal is to cancel a Dataflow streaming pipeline while ensuring that all data currently in-flight is processed and written to the output. This requires a controlled shutdown, not an abrupt halt.

Drain (B): This is the correct choice. A drain operation in Dataflow gracefully shuts down the pipeline. It prevents new data from entering the pipeline while allowing existing data that's already in-flight to complete processing. Crucially, Dataflow will wait for all windows to close (in the case of windowed aggregations), all triggers to fire, and all pending writes to the output sinks to complete before the pipeline shuts down completely. This ensures no data is lost and that the job completes its current work.

Cancel (A): A cancel operation immediately terminates the pipeline. This means that any data currently being processed or waiting in buffers might be lost. It's an immediate halt, suitable when data loss is acceptable or the pipeline is fundamentally broken.

Stop (C): A stop operation does not exist as a native command within the Dataflow monitoring console.

Finish (D): This is also not a standard command in Dataflow's user interface or API for controlling pipeline execution.

Therefore, the drain operation is the only method that guarantees the complete processing of in-flight data before stopping the pipeline. Choosing cancel would risk data loss. Drain prioritizes data integrity and is the correct method for stopping a streaming pipeline when processing all data is a requirement.

Here are some authoritative links for further research:

Dataflow Pipeline Lifecycle: <https://cloud.google.com/dataflow/docs/guides/stopping-a-pipeline>

Stopping a Pipeline: <https://cloud.google.com/dataflow/docs/guides/stopping-a-pipeline>

Dataflow Documentation Overview: <https://cloud.google.com/dataflow/docs>

Question: 382

When running a pipeline that has a BigQuery source, on your local machine, you continue to get permission denied errors. What could be the reason for that?

- A. Your gcloud does not have access to the BigQuery resources
- B. BigQuery cannot be accessed from local machines
- C. You are missing gcloud on your machine
- D. Pipelines cannot be run locally

Answer: A

Explanation:

The correct answer is A, "Your gcloud does not have access to the BigQuery resources." Here's why: When running a pipeline locally that uses BigQuery as a source, your application needs the necessary credentials to authenticate and authorize access to your BigQuery datasets and tables. The gcloud command-line tool manages these credentials through configurations associated with your Google Cloud project and user account. If gcloud is not configured with appropriate permissions (e.g., the user account doesn't have the required BigQuery roles like roles/bigquery.dataViewer), it will result in permission denied errors. BigQuery can be accessed from local machines, as long as the necessary authentication and authorization mechanisms are in place (option B is incorrect). While a missing gcloud installation would prevent the use of the tool for initial configuration, it wouldn't directly cause permission denied errors during the pipeline execution; rather, the pipeline may not start at all (option C is incorrect). Pipelines can indeed be run locally (option D is incorrect). The primary issue is the lack of correct credentials for the client to interact with BigQuery on the GCP cloud. For example, setting the GOOGLE_APPLICATION_CREDENTIALS environment variable, which references a service account JSON key file with the appropriate permissions, would be one way to resolve the permissions issue. The error typically indicates that the application, when running locally, is not authenticating correctly as a user or service account with required access to the target BigQuery resources. Authoritative links for further research include: [Google Cloud Authentication Overview](#), [BigQuery Access Control](#), and [Using Application Default Credentials](#). These resources provide comprehensive information on managing GCP permissions and credentials.

Question: 383

CertyIQ

What Dataflow concept determines when a Window's contents should be output based on certain criteria being met?

- A. Sessions
- B. OutputCriteria
- C. Windows
- D. Triggers

Answer: D

Explanation:

The correct answer is **D. Triggers**. In Google Cloud Dataflow, triggers play a pivotal role in defining when aggregated data within a window should be emitted. Windows themselves establish a grouping of data based on time or count, but they don't dictate when the results are output. Sessions, on the other hand, represent a type of windowing based on inactivity gaps within a data stream. OutputCriteria is not an actual Dataflow concept. Triggers augment windowing by specifying the conditions that must be met before results are produced. These conditions can include elements like the end of the window, the arrival of a specific element, or a certain time period passing after a window's activation. Different trigger types, such as 'default', 'afterWatermark', and 'afterProcessingTime', provide flexibility in managing data emission. For example, using afterWatermark triggers an output when Dataflow determines no more late data will arrive for that window. By using triggers, Dataflow can handle out-of-order data, data lateness, and manage data emission efficiency. This combination of windowing and triggers is vital for building robust real-time and batch data processing pipelines. Understanding triggers is crucial for optimizing the performance and correctness of Dataflow jobs.

Further research can be conducted at the following authoritative links:

Apache Beam documentation on Triggers: <https://beam.apache.org/documentation/programming-guide/#triggers>

Google Cloud Dataflow documentation on Windowing and Triggers:
<https://cloud.google.com/dataflow/docs/concepts/windowing>

Question: 384

CertyIQ

Which of the following is NOT one of the three main types of triggers that Dataflow supports?

- A. Trigger based on element size in bytes
- B. Trigger that is a combination of other triggers
- C. Trigger based on element count
- D. Trigger based on time

Answer: A

Explanation:

The correct answer is A, "Trigger based on element size in bytes." Dataflow primarily utilizes triggers based on element count, time, and combinations of triggers. These mechanisms are crucial for controlling when data within a window is processed. Triggers based on element count fire after a specific number of elements has been accumulated within a window, allowing for batch-like processing of data. Time-based triggers, conversely, fire at designated time intervals, enabling time-sensitive analyses. Dataflow also permits combining these triggers using logical operators like or and and, crafting more complex firing conditions. While element size can indirectly influence processing time and memory consumption, Dataflow itself does not directly provide a trigger based on the cumulative size of elements within a window. Instead, it relies on the number of elements, timestamps, and combinations thereof. A size-based trigger would be difficult to implement efficiently due to the variable nature of element sizes and the potential for unpredictable performance impacts. Therefore, direct element size-based triggers aren't an efficient mechanism for Dataflow's distributed processing model. This focus on count, time, and compound triggers makes Dataflow more robust and scalable.

For additional research and detailed understanding of Dataflow triggers, refer to the official Google Cloud documentation: <https://cloud.google.com/dataflow/docs/concepts/windowing-triggers> and the Apache Beam documentation: <https://beam.apache.org/documentation/programming-guide/#triggers>

Question: 385

CertyIQ

Which Java SDK class can you use to run your Dataflow programs locally?

- A. LocalRunner
- B. DirectPipelineRunner
- C. MachineRunner
- D. LocalPipelineRunner

Answer: B

Explanation:

The correct answer is **B. DirectPipelineRunner**. Here's why:

Dataflow, Google Cloud's managed data processing service, allows you to execute batch and stream data pipelines. However, during development and testing, running your pipeline directly on the managed service can be inefficient. That's where local execution comes in.

The Apache Beam SDK, which Dataflow is based on, provides different runners to execute your pipeline. The DirectPipelineRunner (or DirectRunner in some SDK versions) is specifically designed for local execution. This

runner executes your Dataflow pipeline directly on your local machine, within the Java Virtual Machine (JVM). It simulates the Dataflow environment without deploying to the cloud. This allows for quick iteration, debugging, and unit testing of your pipeline logic before deploying to the full Dataflow service. Using the `DirectPipelineRunner` lets developers verify the transformation logic, and data flow in the code and catch potential issues early, without incurring the costs and delays associated with cloud deployments during early stages of development.

The other options are incorrect: `LocalRunner` and `LocalPipelineRunner` are not standard classes within the Apache Beam SDK. `MachineRunner` is also not a valid runner option in the context of Apache Beam SDK or Dataflow.

In summary, `DirectPipelineRunner` is the standard and appropriate class for running Dataflow pipelines locally, providing a convenient and cost-effective way for development and testing.

Authoritative Links:

Apache Beam Documentation on Runners: <https://beam.apache.org/documentation/runners/> - This link provides general documentation on the different runners available within Apache Beam.

Apache Beam Direct Runner Documentation: <https://beam.apache.org/documentation/runners/direct/> - This link provides specific documentation on the `DirectRunner` which serves the same purpose as the `DirectPipelineRunner` in older versions of the SDK. Note the class name `DirectRunner` is typically used in latest Apache Beam SDK version.

Google Cloud Dataflow Documentation: <https://cloud.google.com/dataflow/docs> - This is Google's official documentation on Dataflow and provides context on its functionality and integration with Apache Beam. While it does not delve into the nuances of each runner, it explains how developers can leverage the flexibility of different run options.

Question: 386

CertyIQ

The Dataflow SDKs have been recently transitioned into which Apache service?

- A. Apache Spark
- B. Apache Hadoop
- C. Apache Kafka
- D. Apache Beam

Answer: D

Explanation:

The correct answer is D, Apache Beam. The Dataflow SDKs, previously proprietary to Google Cloud Dataflow, were indeed transitioned to become part of the Apache Beam project. Apache Beam is an open-source, unified programming model for defining both batch and streaming data-parallel processing pipelines. This transition allowed for greater flexibility and portability for users, as Beam pipelines can be executed on various processing backends, including Google Cloud Dataflow, Apache Spark, Apache Flink, and others. This move towards an open-source model aligned with the industry trend of embracing open standards and promoting interoperability. Instead of being locked into a single vendor's ecosystem, users can now develop data processing logic using Beam's API and choose the most suitable execution engine based on their needs. The core idea behind Apache Beam is to decouple the definition of data processing from the underlying execution engine, making it possible to build once and run anywhere. The transition of the Dataflow SDKs into Apache Beam greatly enhanced its ecosystem and cemented Beam's role as a central framework for data processing. It also encourages collaboration and contributions from the wider open source community.

Here are authoritative links for further research:

Apache Beam Official Website: <https://beam.apache.org/>

Google Cloud Dataflow Documentation: <https://cloud.google.com/dataflow/docs> (Look for mentions of Apache Beam)

Blog post or announcement related to the transition (If available): Please search on the Apache Beam or Google Cloud blogs for a concrete announcement. You may find information on the motivations behind this transition

Question: 387

CertyIQ

The _____ for Cloud Bigtable makes it possible to use Cloud Bigtable in a Cloud Dataflow pipeline.

- A. Cloud Dataflow connector
- B. DataFlow SDK
- C. BigQuery API
- D. BigQuery Data Transfer Service

Answer: A

Explanation:

The correct answer is **A. Cloud Dataflow connector**. Here's the justification:

Cloud Dataflow is a fully managed, serverless data processing service used for batch and stream data processing. To interact with external data storage systems like Cloud Bigtable, Dataflow relies on specialized connectors. These connectors act as bridges, allowing Dataflow pipelines to read data from and write data to specific services.

The "Cloud Bigtable connector" is explicitly designed for this purpose. It provides the necessary APIs and functionalities for Dataflow to communicate with Bigtable. It manages the complexities of interacting with Bigtable's data model and APIs, enabling developers to focus on data processing logic rather than low-level data access details.

Option B, the Dataflow SDK, is a broader toolkit that provides the programming model and libraries used to define Dataflow pipelines. While necessary, it doesn't specifically handle the integration with Bigtable; that's the connector's role. Option C, the BigQuery API, is for interacting with Google BigQuery, a data warehousing service, not Bigtable. Option D, the BigQuery Data Transfer Service, is for transferring data into BigQuery, not for enabling Dataflow to connect to Bigtable.

In summary, to use Cloud Bigtable within a Dataflow pipeline, you need the specific Cloud Bigtable connector. This connector provides the necessary functionality to read from and write to Bigtable within the Dataflow processing environment.

Authoritative Links:

Cloud Dataflow Connectors: <https://cloud.google.com/dataflow/docs/concepts/connectors>

Cloud Bigtable IO in Apache Beam (Dataflow uses Beam): <https://beam.apache.org/documentation/io/built-in/google-cloud-bigtable/>

Using Cloud BigtableIO with Dataflow: <https://cloud.google.com/dataflow/docs/guides/using-bigtable-io>

Question: 388

CertyIQ

Does Dataflow process batch data pipelines or streaming data pipelines?

- A. Only Batch Data Pipelines
- B. Both Batch and Streaming Data Pipelines
- C. Only Streaming Data Pipelines
- D. None of the above

Answer: B

Explanation:

The correct answer is B, indicating that Dataflow processes both batch and streaming data pipelines. Google Cloud Dataflow is a fully managed, serverless data processing service designed for both batch and stream processing scenarios. It leverages a unified programming model based on the Apache Beam SDK, allowing developers to define pipelines that can execute in either batch or streaming mode without significant code changes. Batch processing involves processing a finite dataset at once, while streaming involves processing a continuous, unbounded flow of data. Dataflow's capabilities extend to handling large datasets in batch jobs, making it suitable for transformations, aggregations, and analysis. Simultaneously, Dataflow effectively manages real-time data ingestion and processing from sources like Pub/Sub for streaming pipelines. The framework dynamically autoscales the underlying infrastructure based on the workload, ensuring efficient processing whether it's a batch or a streaming job. This flexibility stems from its core design principles that treat both batch and stream data as unbounded data sets with different processing approaches. Therefore, Google Cloud Dataflow is capable of handling diverse data processing needs, showcasing its versatility and scalability for both batch and streaming workloads.

Authoritative Links:

Google Cloud Dataflow Documentation: <https://cloud.google.com/dataflow/docs> - This official documentation provides a comprehensive overview of Dataflow, including its support for both batch and streaming pipelines.

Apache Beam Documentation: <https://beam.apache.org/documentation/> - Dataflow is based on Apache Beam, learning about Beam further explains Dataflow's unified processing model.

Blog Post on Dataflow's Unified Approach: <https://cloud.google.com/blog/products/data-analytics/dataflow-unified-approach-to-batch-and-stream> This Google blog post explains the design philosophy behind Dataflow's unified approach.

Question: 389

CertyIQ

You are planning to use Google's Dataflow SDK to analyze customer data such as displayed below. Your project requirement is to extract only the customer name from the data source and then write to an output PCollection.

Tom,555 X street -

Tim,553 Y street -

Sam, 111 Z street -

Which operation is best suited for the above data processing requirement?

- A. ParDo
- B. Sink API
- C. Source API
- D. Data extraction

Answer: A

Explanation:

The best operation for extracting customer names from the given comma-separated data in Google Cloud Dataflow is **A. ParDo**.

ParDo, short for "Parallel Do", is a core transformation in Apache Beam (the underlying technology for Dataflow) designed for element-wise processing. It allows you to apply a user-defined function to each element in an input PCollection, potentially transforming it and emitting zero or more output elements to a new PCollection.

In this scenario, each line of the customer data represents an element in the input PCollection. The user-defined function within the ParDo operation would:

1. **Receive** each line as input (e.g., "Tom,555 X street").
2. **Split** the string by the delimiter comma (",").
3. **Extract** the first part of the resulting array, which corresponds to the customer name (e.g., "Tom").
4. **Output** the extracted customer name into a new PCollection.

The other options are not suited for this specific transformation:

B. Sink API: Sinks are used for writing data to an output location, not for data transformation.

C. Source API: Sources are used for reading data from an input location, not for data processing.

D. Data extraction This is a broad term, not a specific operation. Data extraction is a general process of pulling data from various sources.

ParDo facilitates the transformation in a parallel and scalable manner, leveraging Dataflow's distributed processing capabilities to handle large datasets efficiently. Its flexibility in applying custom logic to individual elements makes it ideal for parsing and manipulating data at this fine-grained level.

In essence, ParDo enables the necessary logic to process, transform, and ultimately extract the customer names, meeting the stated requirement.

Authoritative Links:

Apache Beam Programming Guide: ParDo: <https://beam.apache.org/documentation/programming-guide/#pardo>

Google Cloud Dataflow Documentation: ParDo:
<https://cloud.google.com/dataflow/docs/concepts/transforms/pardo>

Question: 390

CertyIQ

Which Cloud Dataflow / Beam feature should you use to aggregate data in an unbounded data source every hour based on the time when the data entered the pipeline?

- A. An hourly watermark
- B. An event time trigger
- C. The with Allowed Lateness method
- D. A processing time trigger

Answer: D

Explanation:

The correct answer is **D. A processing time trigger**. Here's a detailed justification:

Cloud Dataflow, powered by Apache Beam, processes both bounded and unbounded data. When dealing with unbounded data streams, you need mechanisms to group and aggregate data within specific time windows. The question specifies aggregating data "every hour based on the time when the data entered the pipeline,"

which refers to the system's clock or when Dataflow processes the event. This is distinct from the event's intrinsic timestamp (event time).

A processing time trigger dictates that the window will be finalized based on when the data is observed and processed within the pipeline, regardless of its actual timestamp. When the processing time hits the end of the hour for the current window, the aggregation is triggered. This works because Dataflow assigns a processing time to the event upon entering the pipeline.

Option A, "an hourly watermark," is related to event time and aims to track how far along in event time Dataflow has processed; it doesn't enforce hourly aggregations based on processing time. Option B, "an event time trigger," is directly relevant to aggregating data based on the timestamp within the data record itself, not when the data was processed. Option C, the "withAllowedLateness method," handles out-of-order events within event-time windowing. It is used in conjunction with an event-time trigger, not as an aggregation mechanism itself, and is not relevant to processing-time based aggregations.

Therefore, for hourly aggregations based on the time the data enters the Dataflow pipeline, a processing time trigger is the correct method. It ensures that data processed within a one-hour interval is collected and aggregated accordingly. The pipeline uses its internal processing clock to determine the end of each window.

For further research, refer to the official Google Cloud documentation on windowing in Dataflow:

Apache Beam Windowing: <https://beam.apache.org/documentation/programming-guide/#windowing>

Google Cloud Dataflow Windowing: <https://cloud.google.com/dataflow/docs/concepts/windowing>

Processing Time: <https://beam.apache.org/documentation/programming-guide/#processing-time>

Question: 391

CertyIQ

Which of the following is NOT true about Dataflow pipelines?

- A. Dataflow pipelines are tied to Dataflow, and cannot be run on any other runner
- B. Dataflow pipelines can consume data from other Google Cloud services
- C. Dataflow pipelines can be programmed in Java
- D. Dataflow pipelines use a unified programming model, so can work both with streaming and batch data sources

Answer: A

Explanation:

Option A is the statement that is NOT true about Dataflow pipelines. Dataflow is a fully managed, serverless data processing service on Google Cloud Platform, but its pipelines are not exclusively tied to the Dataflow runner. While Dataflow is the primary and often preferred runner, these pipelines, built using the Apache Beam programming model, can be executed on other Beam-compatible runners like Apache Spark and Apache Flink. This flexibility allows users to choose the execution environment best suited to their needs and resources. Options B, C, and D are all accurate. Dataflow pipelines are designed to consume data from various Google Cloud services such as BigQuery, Pub/Sub, and Cloud Storage (Option B). Furthermore, they can be programmed using different languages, including Java (Option C), Python, and Go. Crucially, Dataflow pipelines are built using the Apache Beam unified programming model, enabling them to handle both batch and streaming data processing scenarios (Option D) with a consistent approach. This model provides abstraction over the underlying execution engine, making the same pipeline code capable of processing data from bounded (batch) or unbounded (streaming) sources. Therefore, only option A incorrectly asserts a rigid restriction to the Dataflow runner.

Further reading:

Apache Beam Runners: <https://beam.apache.org/documentation/runners/>

Google Cloud Dataflow Overview: <https://cloud.google.com/dataflow/docs/concepts/dataflow-model>

Unified Batch and Streaming: <https://beam.apache.org/documentation/programming-guide/#unified-batch-and-streaming>

Question: 392

CertyIQ

You are developing a software application using Google's Dataflow SDK, and want to use conditional, for loops and other complex programming structures to create a branching pipeline. Which component will be used for the data processing operation?

- A. PCollection
- B. Transform
- C. Pipeline
- D. Sink API

Answer: B

Explanation:

Transforms in Apache Beam (used by Dataflow) are the core units for data processing logic. They allow developers to manipulate data within PCollections, which are the distributed data sets. Transforms can encapsulate any form of processing, including conditional logic, looping, and branching pipelines. A PCollection (option A) is simply a distributed dataset; it holds the data but doesn't perform operations. A Pipeline (option C) defines the overall execution flow, orchestrating the movement of data through various transforms, but it doesn't directly process individual data elements. Sink APIs (option D) are used for writing data out of the pipeline, not for in-pipeline transformations. Thus, Transforms are the ideal component for implementing complex processing logic that involves branching based on conditions or iterative operations. Using a Transform allows for creating a chain of operations that processes and transforms the PCollections based on specific criteria. The transform encapsulates the actual business logic required for your application. Therefore, the correct answer is **B. Transform**.

Further Reading:

Apache Beam Programming Guide: Transforms: <https://beam.apache.org/documentation/programming-guide/#transforms>

Google Cloud Dataflow Documentation: Transform:
<https://cloud.google.com/dataflow/docs/concepts/transforms>

Question: 393

CertyIQ

Which of the following IAM roles does your Compute Engine account require to be able to run pipeline jobs?

- A. dataflow.worker
- B. dataflow.compute
- C. dataflow.developer
- D. dataflow.viewer

Answer: A

Explanation:

The correct answer is **A. dataflow.worker**. Here's a detailed justification:

Dataflow pipelines execute on a managed service, and Compute Engine instances are often used as workers to perform the processing tasks. For these workers to function correctly, they need specific permissions. The `dataflow.worker` role grants the necessary permissions for a Compute Engine instance to act as a Dataflow worker, enabling it to fetch work from the Dataflow service, process data, and report status updates. It includes permissions related to interacting with the Dataflow service, such as reading and writing data within the pipeline, as well as managing temporary resources.

The other options are not directly related to what a Compute Engine instance needs as a worker for Dataflow jobs. The `dataflow.compute` role does not exist. The `dataflow.developer` role grants broader permissions for developing and managing Dataflow jobs, such as creating, updating, and deleting them. This role is meant for users who need to build and manage Dataflow pipelines, not for worker instances. The `dataflow.viewer` role provides read-only access to Dataflow job metadata, which is not sufficient for worker instances needing to perform actions related to data processing.

Therefore, `dataflow.worker` is the precise role required for a Compute Engine account functioning as a Dataflow pipeline worker. This role adheres to the principle of least privilege, granting only the specific permissions needed to fulfill its purpose, thus enhancing security. The compute engine account must have this role to process the actual pipeline data.

For more details, refer to the official Google Cloud documentation on Dataflow IAM roles: <https://cloud.google.com/dataflow/docs/concepts/security-and-permissions#dataflow-roles> <https://cloud.google.com/iam/docs/understanding-roles>

Question: 394

CertyIQ

Which of the following is not true about Dataflow pipelines?

- A. Pipelines are a set of operations
- B. Pipelines represent a data processing job
- C. Pipelines represent a directed graph of steps
- D. Pipelines can share data between instances

Answer: D

Explanation:

The correct answer is **D. Pipelines can share data between instances**. Here's why:

Dataflow pipelines are indeed sets of operations (A), representing a data processing job (B). They also form a directed graph of steps (C) where data flows through transformations. However, option D is incorrect. Dataflow pipelines, by their distributed nature, primarily process data in a parallel and independent manner across multiple workers or instances. While data can be shared between steps within the same pipeline, sharing data directly between different running instances or different executions of the same pipeline is not the core architectural principle and requires explicitly using mechanisms for such communication like persistent storage (Cloud Storage, BigQuery). Dataflow focuses on fault tolerance and scalability, so sharing mutable state between instances would introduce complexities and negatively impact performance and reliability. Instances are generally ephemeral and are created or destroyed as the pipeline scales, therefore sharing data directly between different instances within a running Dataflow pipeline isn't practical. Persistent storage allows different executions or pipelines to access the same data as required. Dataflow itself doesn't manage the complex synchronization and distributed access needed for direct instance-to-instance data sharing.

Authoritative Links for Further Research:

Google Cloud Dataflow Documentation: <https://cloud.google.com/dataflow/docs>

Dataflow Programming Model: <https://beam.apache.org/documentation/programming-guide/> (Dataflow uses Apache Beam as its SDK)

Key Concepts of Dataflow: <https://cloud.google.com/dataflow/docs/concepts/>

Question: 395

CertyIQ

By default, which of the following windowing behavior does Dataflow apply to unbounded data sets?

- A. Windows at every 100 MB of data
- B. Single, Global Window
- C. Windows at every 1 minute
- D. Windows at every 10 minutes

Answer: B

Explanation:

The correct answer is **B. Single, Global Window**. Dataflow, when processing unbounded datasets without explicit windowing, applies a default behavior of placing all incoming data into a single, global window. This means that all data, regardless of arrival time, is considered part of the same processing batch. This approach contrasts with time-based windowing where data is grouped into time-defined buckets (e.g., 1-minute or 10-minute windows). Dataflow's global window is not based on data size (like option A with 100MB), nor is it based on time by default, as mentioned in options C and D. It's crucial to understand that this default global windowing is primarily for simplified initial setup, and typically, data engineers implement specific, meaningful windowing for real-world streaming analysis. This enables the processing of data in temporal or other relevant segments, rather than treating the entire dataset as one large batch. Using specific windowing strategies (like fixed, sliding, or session windows) is vital for meaningful real-time insights. Therefore, without explicit windowing configuration, all incoming data becomes part of the global window. This makes the global window the default, and therefore the correct answer.

For further reading, see:

Google Cloud Dataflow Windowing Concepts: <https://cloud.google.com/dataflow/docs/concepts/windowing>
Apache Beam Windowing: <https://beam.apache.org/documentation/programming-guide/#windowing> (Note: Dataflow is built on top of Apache Beam).

Question: 396

CertyIQ

Which of the following job types are supported by Cloud Dataproc (select 3 answers)?

- A. Hive
- B. Pig
- C. YARN
- D. Spark

Answer: ABD

Explanation:

Cloud Dataproc is a managed Hadoop and Spark service on Google Cloud Platform, designed to process large datasets efficiently. It supports several popular data processing frameworks as job types. Hive is a data

warehouse system built on top of Hadoop, providing SQL-like queries for data analysis. Dataproc directly supports Hive jobs, allowing users to process structured data using familiar SQL syntax. Pig, another Apache project, is a high-level data flow language, also compatible with Dataproc. Users can utilize Pig scripts to transform and analyze data. Spark, a powerful in-memory data processing engine, is one of the core workloads for Dataproc. Spark jobs can be submitted to Dataproc clusters for batch and stream processing. YARN, while crucial for resource management within Hadoop, is not a job type itself; it is the underlying resource manager responsible for distributing compute resources to the different applications running on the cluster. Therefore, only Hive, Pig, and Spark are recognized as specific job types that users directly submit to a Cloud Dataproc cluster. YARN is an architectural component facilitating the execution of these jobs, rather than a job type itself.

Further reading on supported job types:

[Cloud Dataproc Documentation on Job Submission](#)[Cloud Dataproc Documentation on Supported Hadoop Versions](#)

Question: 397

CertyIQ

What are the minimum permissions needed for a service account used with Google Dataproc?

- A. Execute to Google Cloud Storage; write to Google Cloud Logging
- B. Write to Google Cloud Storage; read to Google Cloud Logging
- C. Execute to Google Cloud Storage; execute to Google Cloud Logging
- D. Read and write to Google Cloud Storage; write to Google Cloud Logging

Answer: D

Explanation:

The correct answer is D, "Read and write to Google Cloud Storage; write to Google Cloud Logging." This is because Dataproc clusters rely heavily on Cloud Storage for various operations. Dataproc needs to read data from Cloud Storage to process it, and it often writes the output of its computations back to Cloud Storage. Therefore, the service account used by Dataproc clusters needs both read and write permissions on Cloud Storage buckets. Additionally, Dataproc requires the ability to log its activities. The service account therefore needs the permission to write logs to Cloud Logging to ensure proper monitoring and troubleshooting of Dataproc jobs. Options A, B, and C are insufficient because they do not offer a comprehensive set of permissions. Option A lacks read access to Cloud Storage, Option B lacks write access to Cloud Storage, and Option C lacks necessary read access to Cloud Storage. Dataproc workers need both read and write capabilities for reading input files and storing results. Hence, Option D which grants both these permissions in addition to Cloud Logging writing permissions is the correct choice. For further information, please refer to the official Google Cloud documentation:

1. [Dataproc service account permissions](#): Explains the service account required for Dataproc and its associated permissions.
2. [Cloud Storage IAM permissions](#): Details the specific permissions for Google Cloud Storage.
3. [Cloud Logging IAM permissions](#): Details the specific permissions for Google Cloud Logging.

Question: 398

CertyIQ

Which role must be assigned to a service account used by the virtual machines in a Dataproc cluster so they can execute jobs?

- A. Dataproc Worker
- B. Dataproc Viewer
- C. Dataproc Runner
- D. Dataproc Editor

Answer: A

Explanation:

The correct answer is A, **Dataproc Worker**. Dataproc clusters utilize service accounts to authorize operations performed by virtual machines within the cluster. These virtual machines, acting as workers, need specific permissions to execute jobs and access necessary resources. The Dataproc Worker role provides the minimum necessary permissions for these virtual machines to interact with the cluster and run jobs effectively. This role includes permissions to read and write data in associated storage locations, launch jobs on the cluster, and communicate with cluster master nodes.

The Dataproc Viewer role (B) only grants read-only access, unsuitable for job execution. The Dataproc Runner role (C) is not a standard predefined role in Google Cloud's IAM. Finally, the Dataproc Editor role (D) offers more permissions than strictly necessary, including the ability to create and delete clusters, which is excessive for worker nodes focused solely on executing jobs. The principle of least privilege dictates that only the required permissions should be granted, making Dataproc Worker the optimal choice for worker nodes. The Dataproc Worker role enables efficient and secure job processing without granting unnecessary access to the underlying cluster infrastructure. This ensures a balance between functionality and security within the Dataproc environment.

[Google Cloud IAM RolesDataproc Service Account](#)

Question: 399

CertyIQ

When creating a new Cloud Dataproc cluster with the `projects.regions.clusters.create` operation, these four values are required: project, region, name, and ____.

- A. zone
- B. node
- C. label
- D. type

Answer: A

Explanation:

The correct answer is **A. zone**. When creating a Dataproc cluster using the `projects.regions.clusters.create` API, the required parameters are indeed project ID, region, cluster name, and **zone**. A project ID identifies the Google Cloud project where the cluster will be created. The region specifies the geographical location, like `us-central1`, within which the cluster resources will reside. The cluster name is a user-defined identifier for the cluster. Critically, the zone defines the specific availability zone within the selected region where the cluster's compute resources will be located. Zones provide isolation from failures in other zones within the same region, offering high availability for resources. Choosing the right zone is essential for performance and resilience within a given region. Options B, C, and D, node, label, and type are incorrect. While node configuration (e.g., master/worker nodes), labels for organization, and machine type are important cluster configurations, they are not strictly required by the `projects.regions.clusters.create` API; they are provided within the `clusterConfig` body. The zone parameter is mandatory for directing where the compute resources of

a Dataproc cluster will be physically allocated within a chosen region, making it an essential requirement of the creation operation. Further research can be conducted at the official Google Cloud documentation: <https://cloud.google.com/dataproc/docs/reference/rest/v1/projects.regions.clusters/create>

Question: 400

CertyIQ

Which Google Cloud Platform service is an alternative to Hadoop with Hive?

- A. Cloud Dataflow
- B. Cloud Bigtable
- C. BigQuery
- D. Cloud Datastore

Answer: C

Explanation:

The correct answer is **C. BigQuery**. Here's why:

Hadoop with Hive is a system for processing large datasets using a distributed file system (HDFS) and a SQL-like query language (HiveQL). The question is looking for a managed service on Google Cloud Platform (GCP) that provides a similar experience. BigQuery is GCP's fully managed, serverless data warehouse that handles large-scale data analysis. It offers a robust SQL interface, similar to HiveQL, allowing users to query petabytes of data efficiently. It abstracts away the complexities of managing infrastructure like Hadoop clusters, providing a seamless analytical experience. While Cloud Dataflow (A) excels at stream and batch data processing using pipelines, it's not primarily a SQL-based data warehouse. Cloud Bigtable (B) is a NoSQL database designed for low-latency access to large datasets, not for analytical queries with SQL. Cloud Datastore (D), now known as Cloud Firestore in Datastore mode, is a document database, unsuitable for large-scale analytical SQL queries similar to Hive. BigQuery is therefore the closest alternative to Hadoop with Hive because it's specifically designed for large-scale analytical processing with a SQL interface, handling massive data volumes while requiring minimal infrastructure management.

Authoritative Links:

BigQuery Documentation: <https://cloud.google.com/bigquery/docs>

Cloud Dataflow Documentation: <https://cloud.google.com/dataflow/docs>

Cloud Bigtable Documentation: <https://cloud.google.com/bigtable/docs>

Cloud Firestore (Datastore Mode) Documentation: <https://cloud.google.com/firestore/docs>

Question: 401

CertyIQ

Which of these rules apply when you add preemptible workers to a Dataproc cluster (select 2 answers)?

- A. Preemptible workers cannot use persistent disk.
- B. Preemptible workers cannot store data.
- C. If a preemptible worker is reclaimed, then a replacement worker must be added manually.
- D. A Dataproc cluster cannot have only preemptible workers.

Answer: BD

Explanation:

Here's a justification for why options B and D are the correct answers:

Option B: Preemptible workers cannot store data. This is correct because preemptible VMs (which are used for preemptible workers in Dataproc) are designed for fault-tolerant workloads. Their ephemeral nature means that they can be reclaimed by Google at any time if resources are needed elsewhere. Any data stored on the local disk of a preemptible instance is not guaranteed to be available and is likely to be lost upon preemption. Therefore, these workers should not be used for critical data storage. Instead, persistent storage solutions like Google Cloud Storage or HDFS on the cluster's primary nodes should be used for data that needs to be preserved.

Option D: A Dataproc cluster cannot have only preemptible workers. This is also accurate. A Dataproc cluster requires at least one primary worker node (also known as the master node) which is a standard non-preemptible VM. The master node runs the core cluster management services and is essential for the cluster's overall stability and operation. Preemptible workers are designed to augment the processing capacity of the cluster and to be used alongside the primary worker, not to replace it. Dataproc needs a stable master node to coordinate the entire cluster, a task that a preemptible VM is unsuitable for because it can be terminated at any time.

Why options A and C are incorrect:

Option A: Preemptible workers cannot use persistent disk. This is incorrect. Preemptible VMs can use persistent disk for storage, but data stored there is not guaranteed to persist if the instance is preempted. Using persistent disk for data in these VMs can be dangerous due to the possibility of data loss. The key is that it can be used, even though it's risky in terms of persistence.

Option C: If a preemptible worker is reclaimed, then a replacement worker must be added manually. This is incorrect. Dataproc, by default, automatically attempts to replace preemptible workers that are reclaimed. The cluster attempts to maintain the requested number of workers, automatically trying to spin up new workers after preemption events. This automated behavior is key for using preemptible workers effectively.

Authoritative Links:

Google Cloud Documentation on Preemptible VM Instances:

<https://cloud.google.com/compute/docs/instances/preemptible>

Google Cloud Documentation on Dataproc Preemptible Workers:

<https://cloud.google.com/dataproc/docs/concepts/preemptible-vm>

Google Cloud Documentation on Dataproc Cluster Architecture:

<https://cloud.google.com/dataproc/docs/concepts/cluster-architecture>

In summary, preemptible workers should not be used for critical data storage and a Dataproc cluster always needs a primary non-preemptible node to function correctly.

Question: 402

CertyIQ

When using Cloud Dataproc clusters, you can access the YARN web interface by configuring a browser to connect through a ____ proxy.

- A. HTTPS
- B. VPN
- C. SOCKS
- D. HTTP

Answer: C

Explanation:

The correct answer is **C. SOCKS**. Accessing the YARN web interface on a Cloud Dataproc cluster typically involves using a SOCKS proxy due to the network configuration. Dataproc clusters, often deployed within private networks or VPCs, are not directly exposed to the public internet for security reasons. Therefore, a direct HTTP connection from your local browser is not feasible. A SOCKS proxy acts as an intermediary, forwarding TCP connections from your browser to the cluster's internal network. This allows you to securely access web interfaces hosted on the cluster nodes, like the YARN resource manager UI, by tunneling your connection. HTTPS (A) is a security protocol for encrypting communication, but it doesn't provide the tunneling capability needed. A VPN (B) establishes a secure network connection, but it's more complex than a SOCKS proxy for this specific use case. HTTP (D) is the protocol used by web browsers, but it is the protocol which is proxied in this case, not the proxy itself. The SOCKS proxy establishes a secure and convenient tunnel through which you can access those web interfaces, without needing to setup a full VPN, or expose the cluster directly. This method balances security with ease of use in managing and monitoring Dataproc clusters. The SOCKS proxy allows for dynamic port forwarding and is generally easier to setup in Google cloud contexts for connecting to a specific service on a cluster.

Authoritative Links:

Google Cloud Dataproc Documentation on Web Interfaces:

<https://cloud.google.com/dataproc/docs/concepts/accessing/web-interfaces>

Google Cloud Documentation on SOCKS Proxies: <https://cloud.google.com/vpc/docs/configure-proxy-server>

Question: 403

CertyIQ

Cloud Dataproc is a managed Apache Hadoop and Apache _____ service.

- A. Blaze
- B. Spark
- C. Fire
- D. Ignite

Answer: B

Explanation:

The correct answer is **B. Spark**. Cloud Dataproc is indeed a managed service provided by Google Cloud Platform, designed to run Apache Hadoop and Apache Spark clusters. These open-source frameworks are foundational for big data processing. Hadoop excels at distributed storage and batch processing, while Spark is known for its speed and in-memory computation capabilities, making it suitable for various data analytics tasks, including machine learning. Dataproc simplifies the process of creating, managing, and scaling these clusters, eliminating the need for manual infrastructure setup. The service provides automated cluster configuration, job scheduling, and monitoring. Choices A, C, and D - Blaze, Fire, and Ignite - are not directly related to the core functionality of Cloud Dataproc and do not function as the core technology it manages alongside Hadoop. Spark's distributed processing model allows Dataproc users to efficiently process large datasets on a scalable cluster. The tight integration between Dataproc and Spark ensures compatibility and performance optimization.

For further information, refer to the official Google Cloud Dataproc documentation:

<https://cloud.google.com/dataproc> and the Apache Spark website: <https://spark.apache.org/>

Question: 404

CertyIQ

Which action can a Cloud Dataproc Viewer perform?

- A. Submit a job.
- B. Create a cluster.
- C. Delete a cluster.
- D. List the jobs.

Answer: D

Explanation:

Here's a justification for why option D, "List the jobs," is the correct answer for what a Cloud Dataproc Viewer can perform:

Cloud IAM (Identity and Access Management) roles in Google Cloud Platform provide granular control over resources. The Dataproc Viewer role grants read-only permissions for Dataproc resources. This means a user with this role can observe the state of the service and its components but cannot make modifications. Listing the jobs associated with a Dataproc cluster falls under this read-only category, allowing the viewer to monitor active and completed jobs without altering them. This permission adheres to the principle of least privilege, ensuring users only have the necessary access for their tasks.

Options A, B, and C are actions that involve creating, modifying, or deleting resources, thus requiring higher levels of permissions granted by roles such as Dataproc Editor or Dataproc Admin. Specifically, submitting a job (A), creating a cluster (B), and deleting a cluster (C) all constitute write or administrative actions. The Dataproc Viewer role lacks these permissions, focusing instead on providing visibility into existing Dataproc environments. The read-only nature of the Viewer role is critical for maintaining data security and service stability by restricting potential unauthorized or accidental modifications. It's designed for users needing to monitor and audit cluster activities, not manage them directly.

For further research, consult Google's official documentation:

Dataproc IAM Roles: <https://cloud.google.com/dataproc/docs/concepts/iam/roles>

Understanding Roles: <https://cloud.google.com/iam/docs/understanding-roles>

IAM Concepts: <https://cloud.google.com/iam/docs/overview>

Question: 405

CertyIQ

Dataproc clusters contain many configuration files. To update these files, you will need to use the `--properties` option. The format for the option is: `file_prefix:property=_____`.

- A. details
- B. value
- C. null
- D. id

Answer: B

Explanation:

The correct answer is **B. value**. The `--properties` option in Dataproc's cluster creation and update commands is designed to configure various settings. These settings are typically defined as key-value pairs, where the key specifies the configuration property, and the value specifies the desired setting for that property. The format `file_prefix:property=value` explicitly indicates this key-value structure. The `file_prefix` identifies the configuration file targeted, `property` refers to the specific parameter within the file, and `value` is the actual configuration setting to be applied. The system expects a concrete setting, hence, a value. Options like `details`, `null`, or `id` do not fit this structure. "Details" might provide information, but it's not what's placed after the equals sign.

"Null" would imply no setting, which is not the case here, and an "id" would be unsuitable as configuration properties typically have varied types like strings, numbers, or boolean. This key-value structure is a common pattern for configuration management in many systems, including Dataproc. This helps in parameterizing a multitude of configurations for a flexible and adaptable solution. Specifying an actual value ensures that the relevant component within the Dataproc cluster behaves as intended.

[Official Google Cloud Documentation on Dataproc Properties](#)[Google Cloud CLI Reference for Dataproc clusters create command](#)

Question: 406

CertyIQ

Scaling a Cloud Dataproc cluster typically involves ____.

- A. increasing or decreasing the number of worker nodes
- B. increasing or decreasing the number of master nodes
- C. moving memory to run more applications on a single node
- D. deleting applications from unused nodes periodically

Answer: A

Explanation:

The correct answer is **A. increasing or decreasing the number of worker nodes.**

Scaling a Cloud Dataproc cluster primarily involves adjusting the computational resources available for processing data. Worker nodes are the workhorses of a Dataproc cluster, responsible for executing the distributed processing tasks. By adding more worker nodes (scaling out), you increase the cluster's parallel processing capacity, allowing it to handle larger datasets and more complex workloads. Conversely, removing worker nodes (scaling in) reduces the cluster's capacity and overall cost, suitable for lighter workloads.

Master nodes, on the other hand, are responsible for managing the cluster, orchestrating tasks, and maintaining cluster health; they aren't meant to be scaled to increase the processing power. While their performance can become a bottleneck, scaling them in the same way as worker nodes isn't the standard practice. Directly moving memory or deleting applications from nodes don't represent the general scaling strategy. Scaling is about changing available resources not just utilizing what's already there more efficiently.

Therefore, horizontal scaling, specifically adjusting the number of worker nodes, is the fundamental mechanism for adapting Cloud Dataproc clusters to changing workload demands.

Authoritative links:

Google Cloud Dataproc Scaling: <https://cloud.google.com/dataproc/docs/concepts/scaling>

Cloud Dataproc Overview: <https://cloud.google.com/dataproc/docs/concepts/>

Question: 407

CertyIQ

Cloud Dataproc charges you only for what you really use with _____ billing.

- A. month-by-month
- B. minute-by-minute
- C. week-by-week
- D. hour-by-hour

Answer: B

Explanation:

Cloud Dataproc employs a minute-by-minute billing model, meaning charges are accrued based on the exact duration of resource utilization. This granular approach contrasts with fixed period billing (monthly, weekly, hourly) where costs are assessed for a full period even if resources aren't fully utilized. Minute-by-minute billing promotes cost optimization by only charging users for the compute power, storage, and network resources they actually consume, down to the minute. This is particularly beneficial for transient workloads like data processing and analytics where clusters may only be active for short durations. Other GCP services like Compute Engine also feature similar billing options, allowing users to pay only for what is provisioned and used. Therefore, users don't incur unnecessary costs associated with idle time, making this billing method highly efficient. The model minimizes wastage and encourages the optimization of data processing workflows. Choosing minute-by-minute billing allows customers to launch clusters as needed for short-term tasks, and terminate them immediately without incurring additional costs. This method enables efficient allocation of resources, allowing for significant cost savings for data engineering workloads with variable demands. It directly reflects the pay-as-you-go principle of cloud computing. The other options, month-by-month, week-by-week, and hour-by-hour, do not align with the actual charging model for Cloud Dataproc, where usage is calculated on a minute basis. For further understanding, refer to Google Cloud's official documentation on Dataproc pricing, which clearly specifies minute-by-minute billing.

Authoritative links:

[Google Cloud Dataproc Pricing](#)

[Google Cloud Pricing Overview](#)

Question: 408

CertyIQ

The YARN ResourceManager and the HDFS NameNode interfaces are available on a Cloud Dataproc cluster ____.

- A. application node
- B. conditional node
- C. master node
- D. worker node

Answer: C

Explanation:

The correct answer is C, master node. In a Cloud Dataproc cluster, the master node plays a crucial role in cluster management and coordination. It's the central point for accessing key services like the YARN ResourceManager and HDFS NameNode. The YARN ResourceManager is responsible for managing and allocating cluster resources, including CPU and memory, to various applications. The HDFS NameNode, on the other hand, manages the file system namespace, storing metadata about where files and directories reside in the distributed HDFS storage. These critical components reside on the master node because they require a centralized point of control for managing cluster operations and data integrity. Worker nodes, by contrast, are designed for executing tasks and storing data. Application nodes and conditional nodes are not standard components in a Dataproc cluster architecture. Accessing the ResourceManager and NameNode directly requires interacting with the master node, which exposes the necessary interfaces. This architecture ensures efficient resource utilization and centralized control of the HDFS file system within the Dataproc cluster. Choosing the master node ensures consistent and controlled interaction with these vital cluster management tools. For more information, refer to the official Google Cloud documentation on Dataproc architecture: <https://cloud.google.com/dataproc/docs/concepts/architecture> and on Resource Manager:

<https://hadoop.apache.org/docs/r3.3.4/hadoop-yarn/hadoop-yarn-site/ResourceManager.html>. Further information on HDFS NameNode can be found here: <https://hadoop.apache.org/docs/r3.3.4/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>

Question: 409

CertyIQ

Which of these is NOT a way to customize the software on Dataproc cluster instances?

- A. Set initialization actions
- B. Modify configuration files using cluster properties
- C. Configure the cluster using Cloud Deployment Manager
- D. Log into the master node and make changes from there

Answer: C

Explanation:

The correct answer is C, Configure the cluster using Cloud Deployment Manager. Let's examine why. Dataproc clusters offer several avenues for software customization during creation. Initialization actions (A) are shell scripts that execute on all nodes of the cluster as it's being provisioned. They're commonly used to install software, adjust system settings, or download custom libraries. Cluster properties (B) allow the user to modify core configuration files like `core-site.xml`, `hdfs-site.xml`, and `yarn-site.xml`, enabling fine-grained control over Hadoop, Spark, and other cluster components. Directly logging into the master node (D) provides interactive access where users can manually install software, modify configuration files, and troubleshoot specific issues, although this approach is generally discouraged for production due to maintenance challenges. Cloud Deployment Manager, however, primarily focuses on infrastructure deployment and management, defining resources like virtual machines, networks, and storage. While it can be used to create Dataproc clusters, it doesn't directly customize software on individual cluster instances. It provisions the cluster based on a configuration template, and that configuration might include initialization actions or cluster properties; the customization of software on the instance level is not a core function of Cloud Deployment Manager. Essentially, Deployment Manager sets up where the software is run, while initialization actions and properties customize what software is running and how. Therefore, Cloud Deployment Manager, itself, is not a method to directly modify the software on a Dataproc instance, making option C the correct answer.

For further research, consult these resources:

Google Cloud Dataproc Documentation - Initialization Actions:

<https://cloud.google.com/dataproc/docs/concepts/init-actions>

Google Cloud Dataproc Documentation - Cluster Properties:

<https://cloud.google.com/dataproc/docs/concepts/configuring/cluster-properties>

Google Cloud Deployment Manager Documentation: <https://cloud.google.com/deployment-manager/docs/>

Question: 410

CertyIQ

In order to securely transfer web traffic data from your computer's web browser to the Cloud Dataproc cluster you should use a(n) ____.

- A. VPN connection
- B. Special browser
- C. SSH tunnel
- D. FTP connection

Answer: C

Explanation:

The correct answer is **C. SSH tunnel**. An SSH tunnel, also known as port forwarding, establishes a secure, encrypted connection between your local machine and a remote server, in this case, a node within the Cloud Dataproc cluster. This method is essential for securely transferring sensitive data, such as web traffic, because it encrypts the data during transit, preventing eavesdropping or tampering. VPNs (A) are typically used to secure network-level access and are less suitable for direct data transfer to specific applications. A special browser (B) doesn't inherently provide secure data transfer mechanisms. FTP connections (D) are unencrypted and highly susceptible to security breaches, making them unsuitable for transmitting sensitive web traffic data. SSH tunnels leverage the cryptographic capabilities of SSH to provide a robust and secure channel for forwarding traffic to a specific port on the cluster, which can be configured to receive your data. This makes it a preferred method for securely pushing data to processing environments like Cloud Dataproc. This security and flexibility is a core tenant of secure cloud data handling. It allows users to securely interact with their applications and resources running in a remote environment.

Authoritative Links:

SSH Tunneling: <https://www.ssh.com/ssh/tunneling/> (Provides a general overview of SSH tunneling)

Google Cloud Dataproc Security: <https://cloud.google.com/dataproc/docs/concepts/security/overview>
(Describes security measures within Dataproc)

Port Forwarding with SSH: <https://www.digitalocean.com/community/tutorials/how-to-set-up-ssh-tunneling-port-forwarding> (Details how to set up port forwarding using SSH)

Question: 411

CertyIQ

All Google Cloud Bigtable client requests go through a front-end server _____ they are sent to a Cloud Bigtable node.

- A. before
- B. after
- C. only if
- D. once

Answer: A

Explanation:

The correct answer is A, "before". All client requests to Google Cloud Bigtable are first routed through a front-end server before reaching a specific Cloud Bigtable node. This front-end layer serves as a crucial intermediary, handling several essential functions. It acts as a load balancer, distributing incoming requests across the available cluster nodes to prevent overloading any single node. This ensures optimal resource utilization and high availability. Furthermore, the front-end server performs authentication and authorization checks, verifying the client's credentials and access permissions before allowing data interactions. It also handles connection management, maintaining persistent connections with clients for efficient data transfers. The front-end abstracts the complexities of the underlying Cloud Bigtable cluster from the client, simplifying the communication process. Clients interact solely with this front-end, unaware of the specific nodes processing their requests. This architecture enhances the scalability and manageability of the service. Without this front-end intermediary, clients would need to be aware of the individual nodes and would bear the responsibility for load balancing and connection management, severely complicating client applications and reducing service reliability.

For further research, refer to the official Google Cloud Bigtable documentation on architecture and request

handling:

1. **Google Cloud Bigtable Architecture:** <https://cloud.google.com/bigtable/docs/architecture>
2. **Google Cloud Bigtable Client Libraries:** <https://cloud.google.com/bigtable/docs/reference/libraries>

Question: 412

CertyIQ

What is the general recommendation when designing your row keys for a Cloud Bigtable schema?

- A. Include multiple time series values within the row key
- B. Keep the row key as an 8 bit integer
- C. Keep your row key reasonably short
- D. Keep your row key as long as the field permits

Answer: C

Explanation:

The correct answer is C, keeping row keys reasonably short, because it's a best practice for optimal performance in Cloud Bigtable. Bigtable stores data lexicographically by row key. Long row keys consume more memory and storage space, impacting efficiency, particularly at scale. Shorter keys, however, facilitate faster lookups and scans, reducing latency. Furthermore, compact keys minimize data transfer overhead. Option A is incorrect because mixing time series values directly within a row key can hinder efficient time-based queries; you'd typically store that in a column family. Option B, limiting to an 8-bit integer, is far too restrictive for most real-world use cases, while option D, using the maximum allowable length, is counterproductive, leading to resource waste. Practical row keys should be concise, unique, and reflect the query patterns of the data. Efficient data retrieval is paramount in Bigtable, and short row keys play a crucial role in achieving this goal. Therefore, designing row keys with a focus on brevity is a fundamental aspect of Bigtable schema optimization. You can find more information about this topic in the official Google Cloud documentation:

Cloud Bigtable Schema Design: <https://cloud.google.com/bigtable/docs/schema-design>

Optimizing Performance: <https://cloud.google.com/bigtable/docs/performance-optimizations>

Question: 413

CertyIQ

Which of the following statements is NOT true regarding Bigtable access roles?

- A. Using IAM roles, you cannot give a user access to only one table in a project, rather than all tables in a project.
- B. To give a user access to only one table in a project, grant the user the Bigtable Editor role for that table.
- C. You can configure access control only at the project level.
- D. To give a user access to only one table in a project, you must configure access through your application.

Answer: B

Explanation:

Option B is incorrect. Bigtable access control using IAM allows for granular permission management, down to the table level. You can grant users specific roles for individual tables, not just at the project level. While the Bigtable Editor role provides broad access, you can use more restricted roles, like the Bigtable Data Reader or Bigtable Data Writer, directly on a specific table to control access. Option A is correct; IAM does allow control

at table level. Option C is incorrect because IAM configuration is not limited to the project level. Option D is a correct technique since you can control access through the application and its code using, for instance, service accounts. Granting the Bigtable Editor role at the table level provides edit access not just read access. Therefore, the statement in option B is incorrect and violates the principle of least privilege by granting unnecessary broad privileges. The correct approach would be to use the appropriate data-level roles for specific tables and not the editor role for only read access.

Authoritative links:

IAM roles for Bigtable: <https://cloud.google.com/bigtable/docs/access-control#roles>

Bigtable IAM Granular Control: <https://cloud.google.com/bigtable/docs/access-control#table-level-permissions>

IAM overview: <https://cloud.google.com/iam/docs/overview>

Question: 414

CertyIQ

For the best possible performance, what is the recommended zone for your Compute Engine instance and Cloud Bigtable instance?

- A. Have the Compute Engine instance in the furthest zone from the Cloud Bigtable instance.
- B. Have both the Compute Engine instance and the Cloud Bigtable instance to be in different zones.
- C. Have both the Compute Engine instance and the Cloud Bigtable instance to be in the same zone.
- D. Have the Cloud Bigtable instance to be in the same zone as all of the consumers of your data.

Answer: C

Explanation:

The recommended practice for optimal performance when using Compute Engine instances and Cloud Bigtable is to have both resources located within the same zone (Option C). This co-location minimizes network latency, the time it takes for data to travel between the two services. Network latency is a significant factor impacting application performance, and keeping resources within the same zone ensures that data transfer occurs with minimal delays. When instances are in different zones, even within the same region, data must traverse a greater network distance, incurring additional latency. This can negatively affect read and write speeds for applications relying on Cloud Bigtable. Additionally, inter-zone network transfers can sometimes be subject to higher costs compared to intra-zone communication. Positioning both Compute Engine and Cloud Bigtable in the same zone promotes efficient data exchange, resulting in faster query execution and improved overall application responsiveness. This is particularly critical for high-performance applications where even small delays can significantly degrade the user experience. Option A is incorrect because distancing the instances increases latency. Option B, while potentially offering some level of availability, introduces latency compared to co-location. Option D, while considering consumer location for Cloud Bigtable, doesn't necessarily mean the Compute Engine instance should be in a different zone; they still should ideally be co-located. Therefore, for the best balance of performance and efficiency, choosing the same zone is the recommended practice.

Authoritative Links:

Google Cloud Documentation - Choosing a Bigtable Location:

<https://cloud.google.com/bigtable/docs/locations> (While not explicitly stating "same zone," the emphasis on regional locations and performance implies co-location within the region and typically within the same zone for optimal performance)

Google Cloud Documentation - Regions and Zones: <https://cloud.google.com/compute/docs/regions-zones> (Provides information on regions and zones and their impact on latency and availability)

Google Cloud Documentation - Network Performance: <https://cloud.google.com/network->

Question: 415

CertyIQ

Which row keys are likely to cause a disproportionate number of reads and/or writes on a particular node in a Bigtable cluster (select 2 answers)?

- A. A sequential numeric ID
- B. A timestamp followed by a stock symbol
- C. A non-sequential numeric ID
- D. A stock symbol followed by a timestamp

Answer: AB

Explanation:

Let's analyze why options A and B are the correct answers. Bigtable, Google's NoSQL wide-column store, distributes data across nodes based on row key ranges. Row keys are lexicographically ordered. Option A, a sequential numeric ID, will result in all writes and reads concentrating on a single node initially and shifting to the next node as the numeric IDs increment. This creates a "hotspot," causing uneven load distribution and performance bottlenecks. Similarly, Option B, a timestamp followed by a stock symbol, will also lead to hotspotting. Given that timestamps usually follow a temporal sequence, all new data pertaining to a specific stock symbol will be written to the same node initially until the timestamp range changes sufficiently. This scenario mirrors the problem with sequential IDs. In contrast, Option C, a non-sequential numeric ID, is designed to evenly distribute writes and reads since the non-sequential nature prevents clustering. Option D, while seemingly problematic, will be more spread out than B. Because different stock symbols will interleave, they'll create better load balancing across nodes than having all data for a given symbol go to the same node in a small time window. The key is to have diversity and unpredictability to distribute the keys better over the nodes. You should avoid sequential keys or keys based on time series since they result in hotspots in Bigtable.

[Bigtable Overview](#)[Designing Your Schema](#)[Key Visualizer](#)

Question: 416

CertyIQ

When a Cloud Bigtable node fails, ____ is lost.

- A. all data
- B. no data
- C. the last transaction
- D. the time dimension

Answer: B

Explanation:

Justification:

The correct answer is **B. no data**. Cloud Bigtable is designed with high availability and fault tolerance in mind. When a node fails, the data it holds is not lost because it's replicated across multiple nodes within a cluster. This replication strategy ensures that data remains accessible even if individual nodes become unavailable. Bigtable achieves this through its distributed storage architecture, where data is partitioned and replicated

across tablets. Upon node failure, the workload is automatically shifted to the replicas, providing seamless data access. Bigtable relies on a distributed consensus algorithm (Paxos) to maintain consistency across replicas. This ensures that, even during failovers, the data remains consistent and no data is lost from the user's perspective. The failure of a single node does not affect the overall operation or integrity of the Bigtable service.

Supporting Links:

1. **Cloud Bigtable Overview:** <https://cloud.google.com/bigtable/docs/overview> - This page provides a general overview of the Bigtable architecture and its key features.
2. **Bigtable Architecture:** <https://cloud.google.com/bigtable/docs/architecture> - This page specifically dives into the distributed nature of Bigtable, highlighting the replication mechanisms used.
3. **High Availability for Bigtable:** <https://cloud.google.com/bigtable/docs/best-practices-high-availability> - This article discusses strategies for achieving high availability with Cloud Bigtable, including the impact of node failures.
4. **Google Cloud Platform Reliability:** <https://cloud.google.com/reliability> - This page offers general information about Google's overall approach to building reliable cloud services, which includes replication and fault tolerance.

Question: 417

CertyIQ

Which is not a valid reason for poor Cloud Bigtable performance?

- A. The workload isn't appropriate for Cloud Bigtable.
- B. The table's schema is not designed correctly.
- C. The Cloud Bigtable cluster has too many nodes.
- D. There are issues with the network connection.

Answer: C

Explanation:

Option C, "The Cloud Bigtable cluster has too many nodes," is not a valid reason for poor Bigtable performance. Bigtable's performance generally scales linearly with the number of nodes up to a certain point. Adding more nodes usually leads to increased throughput and reduced latency, as the workload is distributed across more resources. The other options, A, B, and D, are all common causes of poor Bigtable performance. An inappropriate workload (Option A) might involve patterns that are not well-suited for Bigtable's key-value model, such as frequent range scans or complex joins. An incorrectly designed schema (Option B), particularly around row key selection, can lead to hotspots and uneven load distribution, severely impacting performance. Network connection issues (Option D), such as high latency or packet loss, will naturally hamper communication between clients and the Bigtable cluster, resulting in slow data access. While excessive node counts could potentially lead to increased costs without proportional gains, it wouldn't directly cause poor performance. In contrast, not having enough nodes is much more likely to lead to performance problems due to overload. In conclusion, option C doesn't inherently lead to bad performance; rather, it typically indicates over-provisioning, which is primarily a cost concern rather than a performance bottleneck.

Further research can be done using these resources:

1. **Google Cloud Bigtable Documentation:** <https://cloud.google.com/bigtable/docs> (Specifically look into performance optimization and schema design.)
2. **Google Cloud Bigtable Performance Considerations:** <https://cloud.google.com/bigtable/docs/performance-considerations>
3. **Optimizing Bigtable Performance:** <https://cloud.google.com/blog/products/databases/optimizing->

Question: 418

Which is the preferred method to use to avoid hotspotting in time series data in Bigtable?

- A. Field promotion
- B. Randomization
- C. Salting
- D. Hashing

Answer: A

Explanation:

The preferred method to avoid hotspotting in time series data in Bigtable is **Salting**.

Hotspotting in Bigtable occurs when a disproportionate amount of read or write requests are directed to a single node or a small range of nodes in the cluster. This usually happens when the row keys are sequential or have a common prefix, which is typical in time series data where data is often written chronologically. This leads to imbalanced load distribution and can degrade performance.

1. **Salting** involves adding a random prefix to the row key. This distributes the writes across multiple nodes in the Bigtable cluster because the prefixes ensure that rows with similar timestamps get scattered. The random prefix will break up the sequential nature of the timestamps and allow for more balanced distribution across the nodes. When reading data, you'll need to scan across all possible salt values.
2. **Field promotion** is a technique used in schema design and data modeling but doesn't directly address the hotspotting problem in Bigtable.
3. **Randomization** (if implying completely random row keys) can distribute writes but makes efficient querying of time ranges virtually impossible because you can't predict where the data for a specific time range will reside.
4. **Hashing** can be useful for distributing keys, but its effectiveness depends on the hash function and the distribution of the input keys. In the context of time series data, if the hash function is not carefully chosen, it can still lead to some hotspotting. Salting is simpler and easier to implement, therefore the preferred method. It's a common and well-understood technique for mitigating hotspotting in Bigtable. The prefix ensures the data is spread across the nodes, even with sequential input, without sacrificing too much in the ability to query.

Therefore, salting is the most appropriate method to avoid hotspotting in time-series data in Bigtable because it effectively distributes writes across multiple nodes while still allowing for reasonable querying using range scans.

Further reading:

1. [Designing your schema - Google Cloud Bigtable](#)
2. [Key Visualizer - Google Cloud Bigtable](#)

Question: 419

When you design a Google Cloud Bigtable schema it is recommended that you _____.

- A. Avoid schema designs that are based on NoSQL concepts
- B. Create schema designs that are based on a relational database design
- C. Avoid schema designs that require atomicity across rows
- D. Create schema designs that require atomicity across rows

Answer: C

Explanation:

The correct answer is C, avoiding schema designs requiring atomicity across rows, because Bigtable is a wide-column, NoSQL database designed for massive scalability and performance, not for traditional relational database operations. Relational database concepts (B) are a poor fit because Bigtable lacks features like joins and foreign keys. Option A is incorrect because Bigtable is fundamentally a NoSQL database, making NoSQL concepts applicable. Atomicity across rows (D) is difficult and often inefficient to achieve in Bigtable's distributed architecture. Operations typically target single rows, making cross-row atomicity unsuitable for its design. Instead, data is modeled to avoid such dependencies. Transactions in Bigtable are limited to single-row operations, emphasizing the need to design schemas that don't rely on ACID properties across multiple rows. Designing for eventual consistency and denormalization is preferred. This simplifies queries and enables higher throughput. Key design principles include choosing row keys that facilitate efficient data retrieval and ensuring data is partitioned effectively across the cluster. Row keys should facilitate optimal data locality and filtering. This approach aligns with best practices for distributed NoSQL databases.

Further reading:

Google Cloud Bigtable Schema Design: <https://cloud.google.com/bigtable/docs/schema-design>

Cloud Bigtable Overview: <https://cloud.google.com/bigtable/docs/overview>

NoSQL Database Concepts: <https://en.wikipedia.org/wiki/NoSQL>

Question: 420

CertyIQ

Which of the following is NOT a valid use case to select HDD (hard disk drives) as the storage for Google Cloud Bigtable?

- A. You expect to store at least 10 TB of data.
- B. You will mostly run batch workloads with scans and writes, rather than frequently executing random reads of a small number of rows.
- C. You need to integrate with Google BigQuery.
- D. You will not use the data to back a user-facing or latency-sensitive application.

Answer: C

Explanation:

The correct answer is C. While Bigtable integration with BigQuery is a powerful capability, it doesn't influence the choice between HDD and SSD storage. Options A, B, and D all point towards scenarios where HDDs might be a suitable choice for Bigtable. HDDs are cost-effective for large datasets (A) and excel at sequential I/O operations, making them good for batch workloads (B). They're also a better fit when low latency is not a primary concern (D). This means that applications that are not user-facing or latency-sensitive, such as offline analytics processing, would be an ideal use case for HDDs. However, integration with BigQuery is a feature that works with both SSD and HDD configurations, as both types of storage can be used as a data source for BigQuery. The critical factor in choosing between SSD and HDD is typically performance versus cost, where SSDs are generally faster, and HDDs are more affordable for large capacity. Therefore, integration with BigQuery is unrelated to the storage medium choice itself.

Question: 421**CertyIQ**

Cloud Bigtable is Google's _____ Big Data database service.

- A. Relational
- B. mySQL
- C. NoSQL
- D. SQL Server

Answer: C**Explanation:**

The correct answer is C, NoSQL. Cloud Bigtable is a fully managed, scalable, and wide-column NoSQL database service offered by Google Cloud Platform. Unlike relational databases (A) that organize data into tables with rows and columns with defined schemas, Bigtable is designed to handle massive amounts of unstructured or semi-structured data. It doesn't adhere to a rigid schema like MySQL (B) or SQL Server (D), which are examples of relational database systems. Bigtable excels at storing and retrieving data with high throughput and low latency, making it ideal for applications such as time-series data, IoT device data, and user activity tracking. It uses a sparse, distributed, and persistent multi-dimensional sorted map, allowing for efficient storage and retrieval of data based on row keys and column families. The inherent flexibility and scalability of Bigtable make it a cornerstone for handling large-scale, real-time analytics and applications needing high availability and performance. It does not offer SQL query support; rather, data is accessed through its native API. The focus on schema flexibility and horizontal scalability is a hallmark of NoSQL databases, distinguishing them from traditional relational databases. Therefore, 'NoSQL' is the most accurate descriptor for Cloud Bigtable.

Authoritative Links:

1. **Google Cloud Bigtable Documentation:** <https://cloud.google.com/bigtable/docs> - This is the official documentation for Cloud Bigtable and provides comprehensive information about its features, architecture, and use cases.
2. **Google Cloud Bigtable Overview:** <https://cloud.google.com/bigtable> - This page offers a high-level overview of Bigtable, outlining its capabilities and benefits.
3. **NoSQL Databases:** <https://en.wikipedia.org/wiki/NoSQL> - This Wikipedia article provides a general understanding of NoSQL databases and their characteristics.

Question: 422**CertyIQ**

When you store data in Cloud Bigtable, what is the recommended minimum amount of stored data?

- A. 500 TB
- B. 1 GB
- C. 1 TB
- D. 500 GB

Answer: C**Explanation:**

The recommended minimum amount of stored data in Cloud Bigtable is 1 TB (option C). This recommendation stems from Bigtable's architecture being optimized for massive datasets and high throughput. Smaller datasets might not fully leverage the platform's capabilities and could incur unnecessary costs, as resource allocation is primarily geared toward larger-scale operations. Bigtable's distributed nature and its cost-effectiveness shine when dealing with terabytes or petabytes of data, making it a powerful tool for applications requiring high scalability and low latency on a grand scale. While you can store less than 1TB, performance and cost-efficiency benefits are best realized when reaching this threshold. Moreover, the overhead associated with managing a Bigtable instance, which includes managing nodes and handling replication, becomes more justified with larger datasets. The resources allocated for a Bigtable instance are designed for substantial data volume. Hence, for smaller workloads, alternative database services might be more suitable and cost-effective. Official Google documentation and best practices highlight this 1 TB threshold as the point where Bigtable's benefits become significant.

For further research, consult the official Google Cloud documentation on Cloud Bigtable:
<https://cloud.google.com/bigtable/docs>
<https://cloud.google.com/bigtable/docs/choosing-storage>
<https://cloud.google.com/bigtable/pricing>

Question: 423

CertyIQ

If you're running a performance test that depends upon Cloud Bigtable, all the choices except one below are recommended steps. Which is NOT a recommended step to follow?

- A. Do not use a production instance.
- B. Run your test for at least 10 minutes.
- C. Before you test, run a heavy pre-test for several minutes.
- D. Use at least 300 GB of data.

Answer: A

Explanation:

The answer A, "Do not use a production instance," is the incorrect choice regarding recommended steps for performance testing Cloud Bigtable.

Here's why:

The primary principle of performance testing is to simulate realistic workload scenarios. Performance testing a production instance is not recommended due to potential disruptions to live user traffic. Instead, the creation of a test environment that mirrors the production setup (size, configuration) is essential.

Options B, C, and D are highly recommended:

B. Run your test for at least 10 minutes: Bigtable, like many distributed systems, can exhibit transient performance fluctuations at the start. A 10-minute run allows the system to reach a steady state, providing more reliable and representative performance data. This accounts for initial caching, load balancing, and other warm-up activities.

C. Before you test, run a heavy pre-test for several minutes: Similar to B, a pre-test warms up the system. It ensures that the caches are populated, data is distributed, and the Bigtable cluster has time to adapt to a high workload. This avoids skewed results from cold starts.

D. Use at least 300 GB of data: Bigtable is designed to perform optimally with large datasets. Using sufficient data ensures that the performance test effectively stresses the system's storage and query capabilities. Smaller datasets might fit entirely in memory or caches, masking potential performance bottlenecks related

to disk I/O and distributed data processing. A large dataset helps expose these issues.

Running tests on a non-production instance helps isolate the test from affecting live user experience and prevents data corruption in the production environment. So, in the context of performance tests that are dependant on Cloud Bigtable, using a production instance is absolutely not recommended.

<https://cloud.google.com/bigtable/docs/performance-testing>

Question: 424

CertyIQ

Cloud Bigtable is a recommended option for storing very large amounts of _____?

- A. multi-keyed data with very high latency
- B. multi-keyed data with very low latency
- C. single-keyed data with very low latency
- D. single-keyed data with very high latency

Answer: C

Explanation:

Cloud Bigtable is a fully managed, scalable NoSQL database service best suited for handling massive datasets. It's optimized for low-latency read and write operations. The key characteristic of Cloud Bigtable lies in its underlying data model, which is a sparse, sorted, multi-dimensional map indexed by a single row key. Unlike relational databases that use multiple columns for indexing, Bigtable relies primarily on a single row key, which determines data ordering and facilitates fast lookups. This single-key design enables Bigtable to achieve consistent low latencies even with very large-scale datasets. Options A and B incorrectly suggest multi-keyed data, which is not the fundamental indexing mechanism of Bigtable. Option D incorrectly implies high latency, which contradicts Bigtable's core strength. The correct answer, C, accurately reflects the service's focus on single-keyed data access with minimal latency, making it ideal for applications like IoT, time-series data, and web analytics requiring rapid data retrieval and update.

For further research, refer to the official Google Cloud documentation:

Cloud Bigtable Overview: <https://cloud.google.com/bigtable/docs/overview>

Key concepts: <https://cloud.google.com/bigtable/docs/key-concepts>

Question: 425

CertyIQ

Google Cloud Bigtable indexes a single value in each row. This value is called the _____.

- A. primary key
- B. unique key
- C. row key
- D. master key

Answer: C

Explanation:

The correct answer is **C. row key**. In Google Cloud Bigtable, data is stored in a sparse, multi-dimensional sorted map. This map is indexed by a single primary key that uniquely identifies each row. This primary key is

specifically known as the **row key**. Unlike relational databases that might have composite primary keys, Bigtable utilizes a single string value for its row key. The row key is crucial for efficient data retrieval and is used as the primary mechanism for distributing and sorting data within the table. When a read request is issued, Bigtable uses the row key to quickly locate the specific data, contributing to its low-latency performance. Other options like "primary key," "unique key," or "master key" are not specific to Bigtable's terminology for the single identifier within a row.

The choice of an effective row key is paramount for optimizing performance. Row keys should be designed to avoid hotspotting, which occurs when read and write requests are concentrated on a small number of nodes due to poorly distributed row keys. It's vital to consider the nature of queries and usage patterns when designing row keys to ensure efficient and scalable access to the data. The row key structure directly impacts how data is physically laid out in the table. Hence, its choice significantly affects overall system performance.

Authoritative Links:

1. **Google Cloud Bigtable Documentation:** <https://cloud.google.com/bigtable/docs/overview>
2. **Designing a Bigtable Schema:** <https://cloud.google.com/bigtable/docs/schema-design>
3. **Understanding Row Keys:** <https://cloud.google.com/bigtable/docs/row-keys>

Question: 426

CertyIQ

What is the HBase Shell for Cloud Bigtable?

- A. The HBase shell is a GUI based interface that performs administrative tasks, such as creating and deleting tables.
- B. The HBase shell is a command-line tool that performs administrative tasks, such as creating and deleting tables.
- C. The HBase shell is a hypervisor based shell that performs administrative tasks, such as creating and deleting new virtualized instances.
- D. The HBase shell is a command-line tool that performs only user account management functions to grant access to Cloud Bigtable instances.

Answer: B

Explanation:

The correct answer is B because the HBase shell is indeed a command-line interface (CLI) used to interact with Cloud Bigtable. Specifically, it allows administrators to manage tables, including creation, deletion, and modification of table schemas. Options A, C, and D are incorrect for distinct reasons. Option A incorrectly claims the HBase shell is a GUI, when it is fundamentally a command-line tool. Option C refers to a hypervisor, a concept unrelated to the HBase shell or Cloud Bigtable management. Hypervisors are for creating and managing virtual machines, not database administration. Option D is misleading as while user access is essential, the HBase shell manages much more than just user accounts; it handles table structure and data interaction. The HBase shell provides a direct line of interaction with the database, allowing for granular control over administrative tasks. Command-line interfaces are standard tools for managing databases because they enable precise commands and scripting capabilities. The HBase shell's functionality directly aligns with the needs of database administrators who work with Cloud Bigtable, empowering them to implement database structure changes and maintain the database effectively.

Authoritative links:

Cloud Bigtable Documentation on the HBase Shell: <https://cloud.google.com/bigtable/docs/hbase-shell>
General information on HBase Shell: <https://hbase.apache.org/book.html#shell>
Google Cloud Bigtable Overview: <https://cloud.google.com/bigtable/docs/overview>

Question: 427

CertyIQ

What is the recommended action to do in order to switch between SSD and HDD storage for your Google Cloud Bigtable instance?

- A. create a third instance and sync the data from the two storage types via batch jobs
- B. export the data from the existing instance and import the data into a new instance
- C. run parallel instances where one is HDD and the other is SDD
- D. the selection is final and you must resume using the same storage type

Answer: B

Explanation:

The correct answer is **B. export the data from the existing instance and import the data into a new instance**. Google Cloud Bigtable does not offer an in-place storage type change (from SSD to HDD or vice versa). Unlike some database systems that allow storage tier adjustments on the fly, Bigtable necessitates the creation of a new instance with the desired storage type. This is due to the fundamental architectural differences in how the data is physically stored and accessed with SSD and HDD. The recommended process is to first export the existing data using a mechanism like a Dataflow job to a Google Cloud Storage (GCS) bucket. Then, create a new Bigtable instance with the target storage type (either SSD or HDD). Finally, import the data from the GCS bucket into the new Bigtable instance. This ensures data integrity and a clean transition, minimizing downtime if planned properly. Option A, while theoretically possible, introduces unnecessary complexity and maintenance burden of managing a third instance for synchronization. Option C is wasteful in resources as it runs parallel, independent instances. Option D is incorrect, as switching between storage types is possible, though requiring data migration.

Authoritative Links:

1. **Bigtable Storage Options:** <https://cloud.google.com/bigtable/docs/storage-options>
2. **Import and Export Bigtable data:** <https://cloud.google.com/bigtable/docs/import-export>
3. **Choosing between HDD and SSD storage:** <https://cloud.google.com/bigtable/docs/choosing-storage>

Question: 428

CertyIQ

Your company is planning to migrate a large on-premises data warehouse to BigQuery. The data is currently stored in a proprietary, vendor specific format. You need to perform a batch migration of this data to BigQuery. What should you do?

- A. Export the data to CSV files, upload the files to Cloud Storage, then load the files into BigQuery.
- B. Use Datastream to replicate the data in real time.
- C. Use the BigQuery Data Transfer Service.
- D. Use the bq command-line tool to load the data directly from the on-premises data warehouse.

Answer: A

Explanation:

Correct Answer

Option A: Export the data to CSV files, upload the files to Cloud Storage, then load the files into BigQuery.

Explanation of the Correct Answer

Option A is the correct answer because it outlines a standard and reliable method for migrating data in bulk to BigQuery. The process involves converting the proprietary data into a widely supported format like CSV, storing it in Cloud Storage for scalability and cost-effectiveness, and then loading it into BigQuery. This approach is suitable for handling large datasets and ensures compatibility during the migration process.

Analysis of Each Option

Option A: This option is correct. It describes a practical approach for batch data migration, using CSV as a common format and Cloud Storage as a staging area.

Option B: This option is incorrect. Datastream is designed for real-time data replication, which is not suitable for a one-time batch migration.

Option C: This option is incorrect. The BigQuery Data Transfer Service supports specific sources and is not designed for arbitrary proprietary data warehouses.

Option D: This option is incorrect. The bq command-line tool cannot directly connect to and extract data from a proprietary on-premises data warehouse.

Key Takeaways

When migrating data from a proprietary format, converting it to a standard format like CSV is essential.

Cloud Storage is often used as a staging area for large-scale data loading into BigQuery.

Understanding the purpose of different Google Cloud services (like Datastream and BigQuery Data Transfer Service) is crucial for choosing the right migration strategy.

Avoid trying to directly load unsupported data formats into BigQuery.

Question: 429

CertyIQ

Your company wants to implement a Retrieval-Augmented Generation (RAG) system to allow employees to query an extensive knowledge base of internal documents, such as policy manuals and project reports. You need to prepare this unstructured text for embedding to be used in the RAG system. What should you do to ensure the system can retrieve the most relevant information?

- A. Store the documents as compressed files in a traditional relational database to enable more efficient storage and retrieval.
- B. Convert the unstructured documents into high-dimensional numerical vectors that capture the semantic meaning and relationships of the text.
- C. Use Cloud Data Loss Prevention (Cloud DLP) to scan and redact sensitive information within the documents before processing.
- D. Index each word from the documents into a search engine to enable keyword-based search.

Answer: B

Explanation:

Correct Answer

Option B: Convert the unstructured documents into high-dimensional numerical vectors that capture the semantic meaning and relationships of the text.

Explanation of the Correct Answer

The correct answer is B because Retrieval-Augmented Generation (RAG) systems rely on understanding the

meaning of both the query and the documents to retrieve relevant information. Converting documents into high-dimensional numerical vectors (embeddings) allows the system to capture the semantic meaning of words and phrases. Similar concepts will have vectors that are close to each other in the high-dimensional space. When a user asks a question, the question is also converted into an embedding vector. The system then finds the document embeddings that are closest to the query embedding, thus retrieving the most relevant information based on meaning.

RAG systems use semantic search to find relevant information.

The process can be represented as:

Document Text → Embedding Model → Document Vector (D_v)

Query Text → Embedding Model → Query Vector (Q_v)

Relevance Score = Similarity Function(D_v , Q_v) (e.g., cosine similarity)

Analysis of Each Option

Option A: This option is incorrect because relational databases are good for structured data, but they don't inherently understand the meaning of unstructured text. Compression helps with storage but doesn't aid in semantic retrieval.

Option B: This option is correct because converting text into high-dimensional numerical vectors (embeddings) allows the system to capture the semantic meaning of words and phrases.

Option C: This option is incorrect because Cloud DLP is important for data security and compliance, but it doesn't directly contribute to the retrieval of relevant information. It's a pre-processing step that should be done regardless of the RAG implementation.

Option D: This option is incorrect because keyword-based search is a basic form of information retrieval, but it's not as effective as semantic search for RAG. It relies on exact keyword matches and doesn't understand the context or meaning of the query, which can lead to irrelevant results.

Key Takeaways

RAG systems rely on semantic search to find relevant information, which means understanding the meaning of the query and the documents, not just matching keywords.

Keyword-based search is a basic form of information retrieval, but it's not as effective as semantic search for RAG.

Cloud DLP is important for data security and compliance, but it doesn't directly contribute to the retrieval of relevant information.

A common mistake is to focus on keyword matching instead of semantic meaning when preparing data for a RAG system.

Question: 430

CertyIQ

You are building a Dataflow pipeline to ingest customer feedback. Before loading to your data warehouse, you must validate email addresses and enrich unstructured comment strings with a generative AI sentiment classification. Invalid records need to be routed for manual review. How should you implement this pipeline?

- A. Use Dataflow to load all data into BigQuery and execute a SQL MERGE statement to flag invalid records and BigQuery ML to assign sentiment scores.
- B. Configure the data source system to pre-validate data before sending it to Dataflow and use the BigQuery

ML. GENERATE_TEXT command to assign a sentiment score

C. After the Dataflow load completes, execute a Cloud Run function to scan for invalid entries and call Vertex AI to assign sentiment scores.

D. Apply a Pardo transform in Dataflow to validate each element, use a RunInference transform to assign sentiment scores, and use side outputs to route valid/invalid records.

Answer: D

Explanation:

This option correctly describes how to implement the required steps within an efficient, scalable **Google Cloud Dataflow** pipeline:

Validate email addresses and route invalid records: The ParDo transform is the fundamental way to apply custom, element-wise logic in Dataflow. It can be used to check the email address format. By using **side outputs** (also known as a TupleTag), the pipeline can simultaneously produce two distinct collections: one for **valid records** (the primary output) and one for **invalid records** (the side output), which can then be routed to the manual review queue.

Enrich with generative AI sentiment classification: The **RunInference** transform is the dedicated Beam/Dataflow component for calling external machine learning models for predictions. It allows you to efficiently use a **Vertex AI** endpoint (which is the standard for generative AI on Google Cloud) to classify the comment string's sentiment as an integral part of the stream processing.

The other options are less appropriate:

A is incorrect because moving all the data to BigQuery first to run validation and ML is inefficient, violates the requirement to route **invalid records** before loading, and using a MERGE statement is designed for updating tables, not routing invalid data.

B shifts the validation burden to the source system, which is not guaranteed to be possible, and using **BigQuery ML GENERATE_TEXT** is typically for batch processing once data is in BigQuery, not for in-line stream enrichment.

C introduces significant latency and cost by processing the data twice: first to load it, and then using **Cloud Run** to scan after the load, which is a pattern for post-processing, not in-line validation and enrichment.

Question: 431

CertyIQ

You work for a multi-national pharmaceutical company that manages international data within a large single BigQuery table. Your company has data science teams worldwide who rely on this data for modeling within various tools. You need to ensure that each data science team is only able to access the data for their respective country in order to remain compliant with local regulations. You want a solution that involves the least amount of effort to implement. What should you do?

A. Create a permissions table and leveraging an inner join create an authorized view in BigQuery that only shows the data of the country that the data science team member has access to.

B. Create separate country-specific BigQuery tables containing customer information and assign each data science team member to their respective country's table.

C. Create a Looker dashboard with the BigQuery data. Create permissions by country and share the dashboard with the data science teams.

D. Assign each data science team a BigQuery row-level access grant by country.

Answer: D

Explanation:

The correct solution that involves the least amount of effort to implement for this scenario is **D**.

D. Assign each data science team a BigQuery row-level access grant by country.

Explanation

Row-Level Security (RLS) in BigQuery is designed specifically for this exact use case: controlling access to specific subsets of rows (like those belonging to a specific country) within a **single table**, based on the user's identity or group membership.

Why Option D is Best (Least Effort)

Centralized Management: You don't need to duplicate data, create complex views, or manage permissions on multiple objects. The data stays in one large table.

Simple Policy: You define a single ROW ACCESS POLICY on the table using a boolean expression (e.g., country = 'US' OR country = 'UK'), and then grant access to this policy to the respective data science teams (or their Google Groups).

Seamless for Users: When a data science team queries the main table, BigQuery automatically filters the result set to only show the rows they are authorized to see, ensuring compliance without any change to their existing query logic or tools.

Why Other Options are Less Suitable

A. Create a permissions table and leveraging an inner join create an authorized view...

This is a traditional but more complex approach. It requires creating and maintaining a separate permissions table, a VIEW (which adds a layer of abstraction), and configuring the VIEW as an Authorized View to reference the user's country from the permissions table. This is more effort than native RLS.

B. Create separate country-specific BigQuery tables...

This involves the **most effort**. You must manage data ingestion, maintenance, and schema updates across multiple tables, which is prone to error and increases complexity significantly.

C. Create a Looker dashboard...

This is incorrect because it only secures access for visualization tools (Looker). The data science teams need access to the raw data for **modeling within various tools**, not just a dashboard, meaning the underlying BigQuery data must be secured first.

Question: 432

CertyIQ

Your company has data assets across multiple Cloud Storage buckets and BigQuery datasets containing raw and processed data. The requirement is to establish a unified data governance framework that allows for centralized metadata discovery, data quality monitoring, and consistent security policy application across these various data stores without physically moving or duplicating the data. You need to implement a solution to achieve this federated governance. What should you do?

A. 1. Deploy a centralized Cloud SQL database to store metadata extracted from BigQuery and Cloud Storage using custom scripts.2. Integrate the database with Looker Studio for data discovery and visualization.3. Implement a custom policy engine using Cloud Run functions triggered by changes in IAM policies to enforce consistent security across projects.

B. 1. Export metadata out of Dataplex Universal Catalog by running a metadata export job.2. Implement Dataproc Metastore to manage table schemas and Apache Hive metastore for metadata discovery.3. Manage security using a combination of BigQuery row-level security and Cloud Storage policies.

C. 1. Create a Looker Studio dashboard on BigQuery INFORMATION SCHEMA views to visualize and monitor

data quality.2. Manage security using IAM policies at the project level, supplemented by BigQuery authorized views for granular access control.

D. 1. Use Dataplex to organize the BigQuery datasets and Cloud Storage buckets into lakes and zones.2. Use Dataplex for automated metadata discovery, centralized security policy management, data profiling, and data quality tasks.

Answer: D

Explanation:

Explanation of the Correct Answer

The problem requires establishing a unified, federated data governance framework across Cloud Storage buckets and BigQuery datasets. This framework must support centralized metadata discovery, data quality monitoring, and consistent security policy application without physically moving or duplicating data. Dataplex is Google Cloud's intelligent data fabric service specifically designed to address these requirements.

Dataplex allows users to organize diverse data assets (like BigQuery datasets and Cloud Storage buckets) into logical data lakes and zones. It provides automated metadata discovery by scanning and cataloging metadata from registered data sources. Furthermore, Dataplex enables centralized security policy management, integrating with IAM to apply consistent policies across various data assets. It also offers capabilities for data profiling and defining/monitoring data quality rules, directly fulfilling all the stated needs for federated governance without data movement.

Analysis of Each Option

Option A: This option proposes a custom solution involving a Cloud SQL database for metadata, Looker Studio for visualization, and custom Cloud Run functions for security. While technically possible, it requires significant custom development, integration, and ongoing maintenance. It does not leverage a unified, managed service for federated governance, making it complex, costly, and prone to errors compared to a purpose-built solution.

Option B: This option suggests exporting metadata from Dataplex (implying Dataplex is already in use or being considered, but the problem is about establishing governance), using Dataproc Metastore for schemas, and managing security with service-specific policies. Dataproc Metastore is primarily for Hive-compatible metadata and not a comprehensive solution for federated governance across BigQuery and Cloud Storage. The security mechanisms mentioned are service-specific and do not provide a unified or consistent security policy application across various data stores as required.

Option C: This option focuses on using Looker Studio for data quality visualization on BigQuery INFORMATION_SCHEMA views and managing security with project-level IAM and BigQuery authorized views. This approach is too limited in scope, primarily addressing BigQuery and offering only visualization for data quality, not active monitoring or enforcement. It does not cover Cloud Storage, centralized metadata discovery, or unified security across the entire data estate.

Option D: This option correctly identifies Dataplex as the solution. Dataplex is designed to organize data assets into lakes and zones, provide automated metadata discovery, enable centralized security policy management, and support data profiling and data quality tasks. This directly addresses all aspects of the problem statement: unified, federated governance, metadata discovery, data quality monitoring, and consistent security policy application across BigQuery and Cloud Storage without data movement or duplication.

Key Takeaways

For establishing a unified, federated data governance framework across diverse Google Cloud data assets (like BigQuery and Cloud Storage) that includes metadata discovery, data quality monitoring, and consistent security policy application without data movement, Google Cloud Dataplex is the recommended and most effective solution. It acts as an intelligent data fabric, simplifying management and governance across data

lakes, data warehouses, and data marts.

D. 1. Use Dataplex to organize the BigQuery datasets and Cloud Storage buckets into lakes and zones.

2. Use Dataplex for automated metadata discovery, centralized security policy management, data profiling, and data quality tasks.

Question: 433

CertyIQ

You are designing a data lake on Google Cloud to store vast amounts of customer interaction data from various sources, such as websites, mobile apps, and social media. You need to ensure that this data, which arrives in different formats, is consistently cataloged and easy for data analysts to discover and use. You also want to perform basic data quality checks and transformations before the data is consumed by downstream applications. You need an automated and managed data governance solution. What should you do?

- A. Stream all the data directly into BigQuery, where it is automatically cataloged and governed.
- B. Use Cloud Storage as the central repository. Use Vertex AI to classify and process the data and perform data quality checks.
- C. Use Cloud Storage as the central repository. Use a Cloud Run function to catalog, transform the data, and perform data quality checks.
- D. Use Cloud Storage and BigQuery as repositories. Use Dataplex Universal Catalog for metadata discovery, data quality checks, and transformations

Answer: D

Explanation:

Correct Answer

Option D: Use Cloud Storage and BigQuery as repositories. Use Dataplex Universal Catalog for metadata discovery, data quality checks, and transformations

Explanation of the Correct Answer

Option D is correct because it utilizes Cloud Storage for the data lake's primary storage and BigQuery for analytical workloads. Dataplex provides a unified platform for data governance, including metadata discovery (Universal Catalog), data quality checks, and data transformations, which aligns with the requirements for an automated and managed data governance solution.

Dataplex is specifically designed to address the requirements of a data lake with automated data governance on Google Cloud.

Analysis of Each Option

Option A: Incorrect. While BigQuery offers automatic cataloging and governance features, it is primarily a data warehouse optimized for structured data and analytics. Streaming all data directly into BigQuery might not be suitable for raw, diverse data formats typical of a data lake. It also lacks the flexibility for extensive data transformations before loading.

Option B: Incorrect. Vertex AI is more focused on machine learning tasks. While it can be adapted for data quality, it's not its primary strength. This approach would require significant custom development and integration for cataloging and governance.

Option C: Incorrect. This approach requires you to build and manage the entire data governance pipeline, including cataloging, data quality checks, and transformations, within the Cloud Run function. This involves significant development and operational overhead and lacks a managed data governance solution.

Option D: Correct. This is the most suitable option as it leverages a managed data governance service (Dataplex) to automate cataloging, data quality, and transformations.

Key Takeaways

Dataplex is a key service for data governance in Google Cloud, providing a unified platform for metadata discovery, data quality checks, and data transformations.

When designing a data lake, consider using Cloud Storage for storing raw data and BigQuery for analytical workloads.

Avoid manual implementation of data governance pipelines using Cloud Run functions, as it involves significant development and operational overhead.

Question: 434

CertyIQ

You are investigating a significant increase in the total_slot_ms usage for an active BigQuery job. You need to identify which user and query is causing the increase as quickly as possible. What should you do?

- A. Review Error Reporting in Cloud Logging to locate potentially errored queries.
- B. Analyze query performance by using the INFORMATION_SCHEMA.JOBS view.
- C. Review each running query by using the EXPLAIN statement in BigQuery.
- D. Filter the BigQuery audit logs by job ID in Cloud Logging.

Answer: B

Explanation:

Correct Answer

Option B: Analyze query performance by using the INFORMATION_SCHEMA.JOBS view.

Explanation of the Correct Answer

The INFORMATION_SCHEMA.JOBS view provides detailed information about BigQuery jobs, including the user who ran the query, the query text, and resource consumption metrics like total_slot_ms. This allows direct identification of the query and user responsible for the increased usage. The goal is to quickly identify the user and query causing increased total_slot_ms usage, and this view directly provides that information.

Analysis of Each Option

Option A: Error Reporting focuses on identifying errors, not necessarily resource consumption. A query can consume many resources without producing an error, making this an indirect approach.

Option B: The INFORMATION_SCHEMA.JOBS view provides detailed information about BigQuery jobs, including the user, query text, and resource consumption metrics like total_slot_ms, directly addressing the problem.

Option C: EXPLAIN provides a query plan, useful for understanding how BigQuery will execute a query, but it doesn't directly show the actual resource consumption (total_slot_ms) of running or completed queries. It's more useful for predicting query costs before they run.

Option D: While audit logs provide information about who ran what and when, extracting the total_slot_ms and aggregating it to find the biggest consumers would be more complex and slower than using INFORMATION_SCHEMA.

Key Takeaways

The INFORMATION_SCHEMA.JOBS view is specifically designed for querying job metadata and statistics, making it the most efficient way to identify resource-intensive queries.

Avoid using error logs for resource consumption analysis, as errors don't always correlate with high resource usage.

EXPLAIN is useful for query optimization before execution, not for identifying the source of existing high resource usage.

Question: 435

CertyIQ

You are designing a real-time data enrichment pipeline that processes IoT events written to Pub/Sub. The events need to be enriched with data from an infrequently updated BigQuery table before being written to Cloud Storage. You need to implement a highly scalable fully managed solution while keeping operational overhead to a minimum. What should you do?

- A. Use Dataflow to execute an Apache Beam pipeline with a side input read from BigQuery to enrich the events.
- B. Use Dataproc with a Spark Streaming pipeline to process the events and join with the BigQuery table.
- C. Use a Pub/Sub BigQuery subscription and schedule a BigQuery pipeline to periodically enrich and export the data.
- D. Use Cloud Run functions to process and enrich events with data from the BigQuery table.

Answer: A

Explanation:

Correct Answer

Option A: Use Dataflow to execute an Apache Beam pipeline with a side input read from BigQuery to enrich the events.

Explanation of the Correct Answer

Option A is correct because Dataflow is a fully managed and scalable data processing service ideal for real-time streaming. Using Apache Beam with a side input from BigQuery allows efficient enrichment of data, especially when the BigQuery table is infrequently updated. Side inputs are loaded into memory, minimizing repeated queries to BigQuery.

This solution leverages the strengths of Dataflow for scalable data processing and Apache Beam's side inputs for efficient data enrichment with infrequently updated data.

Analysis of Each Option

Option A: Correct. Dataflow with Apache Beam and side inputs provides a scalable, fully managed, and efficient solution for real-time data enrichment with infrequently updated data.

Option B: Incorrect. Dataproc involves more operational overhead due to cluster management, and joining with BigQuery in Spark Streaming can be less efficient than using side inputs in Dataflow.

Option C: Incorrect. This approach introduces latency and does not meet the real-time requirement of the problem.

Option D: Incorrect. Cloud Run functions may face scalability issues and inefficient BigQuery access for high-volume, real-time data enrichment.

Key Takeaways

Dataflow is a great choice for scalable, real-time data processing.

Apache Beam's side inputs are useful for efficiently enriching data with infrequently updated data sources.

Consider the operational overhead of different services when designing a solution.

Avoid repeatedly querying BigQuery for each event in high-volume scenarios.

Question: 436

CertyIQ

You are using BigQuery's ML.GENERATE_TEXT function to write marketing materials for a new product launch. The problem is, the AI is generating random text that does not always relate to the product. You want the simplest way to improve the output to generate a marketing copy. What should you do?

- A. Fine-tune a more specialized language model on a dataset of product features.
- B. Seed the prompt with examples of product descriptions for similar products.
- C. Experiment with different values for the temperature setting in ML.GENERATE_TEXT.
- D. Experiment with different values for the top_p setting in ML.GENERATE_TEXT.

Answer: B

Explanation:

The correct answer, representing the simplest way to improve the output to relate to the product, is **B**.

B. Seed the prompt with examples of product descriptions for similar products.

Explanation

The problem states that the AI (ML.GENERATE_TEXT function) is generating random text that **does not always relate to the product**. The goal is to find the **simplest way** to improve the output to make it relevant marketing copy.

B. Seed the prompt with examples... This technique is called **in-context learning** or **prompt engineering**. By providing the model with relevant examples (product descriptions for similar products) or key product features directly within the prompt, you give the Large Language Model (LLM) the necessary context and style guide immediately before it generates the new marketing material. This is the **simplest and fastest** method as it requires no training or infrastructure changes.

Why the other options are not the simplest:

A. Fine-tune a more specialized language model... Fine-tuning requires curating a substantial dataset, running a training job (which can be time-consuming and costly), and managing a new specialized model endpoint. This is a powerful but **complex and time-consuming** approach, not the "simplest way."

C. Experiment with different values for the temperature setting... The temperature setting controls the **randomness** or creativity of the output. While changing it might make the output less "random," it **will not guarantee relevance** to the new product. High temperature leads to creative, diverse text; low temperature leads to deterministic, repetitive text. It doesn't inherently inject new relevant information.

D. Experiment with different values for the top_p setting... The top_p setting (nucleus sampling) also controls the **diversity** and coherence of the output by selecting tokens from a smaller, more probable subset. Like temperature, it affects how the text is generated, not the underlying knowledge or context used, so it won't necessarily fix the relevance issue.

Question: 437

Your large e-commerce company uses BigQuery for its data warehousing needs. They have two primary types of workloads:

- **Analyst dashboards:** Interactive queries run by business analysts throughout the day, requiring low latency for real-time insights.
- **ETL jobs:** Large, complex batch queries that load and transform data overnight. These jobs are cost-sensitive and can tolerate longer execution times, and they do not have the same performance requirements as the dashboards. Analysts are reporting inconsistent performance of their dashboards, particularly during peak business hours. You need to ensure reliable performance for both workload types while optimizing costs. What should you do?

- A. Run all analyst dashboard queries as interactive queries and configure ETL jobs to run as batch queries.
- B. Implement a custom autoscaling solution using Cloud Run functions to dynamically adjust available BigQuery slots based on time of day.
- C. Migrate all ETL jobs to Cloud SQL to reduce BigQuery's workload
- D. Use BigQuery editions to create an autoscaling reservation with dedicated slots for analyst dashboards, while running ETL jobs with a separate reservation or using the on-demand billing model

Answer: D

Explanation:

D. Use BigQuery editions to create an autoscaling reservation with dedicated slots for analyst dashboards, while running ETL jobs with a separate reservation or using the on-demand billing model

Explanation of the Correct Answer

The problem describes a scenario where an e-commerce company uses BigQuery for data warehousing, with two distinct workload types: analyst dashboards requiring low latency and interactive performance, and ETL jobs that are large, complex, run overnight, are cost-sensitive, and can tolerate longer execution times. The core issue is inconsistent performance for analyst dashboards during peak hours, and the need to ensure reliable performance for both workloads while optimizing costs.

Option D directly addresses all aspects of the problem by leveraging BigQuery's native capabilities for workload management. By using BigQuery editions to create an autoscaling reservation specifically for analyst dashboards, it ensures a dedicated pool of slots, preventing contention with ETL jobs and guaranteeing consistent performance, especially during peak hours. The autoscaling nature means that capacity adjusts to demand, optimizing costs. For ETL jobs, running them with a separate reservation or using the on-demand billing model allows for flexibility and cost optimization, aligning with their tolerance for longer execution times and cost sensitivity. This approach effectively isolates resources for the two workload types, preventing ETL jobs from impacting dashboard performance, and utilizes BigQuery's intended and most effective way to manage workloads and resources.

Analysis of Each Option

Option A: Run all analyst dashboard queries as interactive queries and configure ETL jobs to run as batch queries.

While this option correctly identifies the appropriate query priority for each workload type, it doesn't fully address the core problem of resource contention and guaranteed performance. If both interactive dashboard queries and batch ETL jobs (even if run as batch) are competing for the same shared pool of on-demand slots, the interactive queries can still experience inconsistent performance if the system is under heavy load. This approach relies on default on-demand slot allocation, which can be unpredictable under varying loads, and doesn't guarantee dedicated resources for dashboards.

Option B: Implement a custom autoscaling solution using Cloud Run functions to dynamically adjust available BigQuery slots based on time of day.

Building and maintaining a custom autoscaling solution for BigQuery slots using Cloud Run would be overly complex, error-prone, and unnecessary. BigQuery itself provides native mechanisms for managing and scaling slots (reservations, editions) that are designed for this purpose. This option would incur significant development and maintenance overhead without necessarily outperforming BigQuery's built-in capabilities.

Option C: Migrate all ETL jobs to Cloud SQL to reduce BigQuery's workload.

Cloud SQL is a relational database service typically used for transactional workloads, not large-scale analytical data warehousing or complex ETL on petabyte-scale data. Migrating large, complex ETL jobs from BigQuery (a highly scalable, columnar, analytical data warehouse) to Cloud SQL (a row-oriented, transactional database) would be a significant downgrade in capability and performance for ETL. This would likely introduce new performance bottlenecks, increase complexity, and be far more expensive for analytical workloads, fundamentally misunderstanding the purpose of BigQuery for ETL.

Option D: Use BigQuery editions to create an autoscaling reservation with dedicated slots for analyst dashboards, while running ETL jobs with a separate reservation or using the on-demand billing model.

This option provides the most comprehensive, efficient, and BigQuery-native solution. It leverages BigQuery's Editions and reservations to achieve workload isolation, guaranteeing performance for critical dashboards through a dedicated, autoscaling slot reservation. For ETL jobs, using a separate reservation or the on-demand billing model allows for cost optimization while accommodating their tolerance for longer execution times. This approach effectively prevents resource contention and ensures reliable performance for both workload types while managing costs.

Key Takeaways

1. **Workload Isolation:** For diverse BigQuery workloads with different performance and cost requirements, isolating resources is crucial to prevent contention and ensure consistent performance for critical applications.
2. **BigQuery Reservations and Editions:** BigQuery Editions, combined with reservations (especially autoscaling ones), are the recommended native solution for managing processing capacity, guaranteeing performance, and optimizing costs for specific workloads.
3. **On-Demand vs. Reservations:** Understand when to use on-demand billing (pay-per-query, suitable for unpredictable or infrequent batch jobs) versus reservations (committed capacity, ideal for consistent performance needs).
4. **Appropriate Tooling:** Choose the right Google Cloud service for the job; BigQuery is designed for large-scale analytical ETL and warehousing, while Cloud SQL is for transactional databases.

Question: 438

CertyIQ

You need to load a dataset with multiple terabytes of clickstream data into BigQuery. The data arrives each day as compressed JSON files in a Cloud Storage bucket. You need a low-cost, programmatic, and scalable solution to load the data into BigQuery. What should you do?

- A. Create a Cloud Run function to run a Python script to read and parse each JSON file, and use the BigQuery streaming insert API.
- B. Use the BigQuery Data Transfer Service from Cloud Storage.
- C. Use Cloud Data Fusion to create a pipeline to load the JSON files into BigQuery.
- D. Create an external table in BigQuery pointing to the Cloud Storage bucket and run the INSERT INTO ... SELECT * FROM external_table command.

Answer: B

Explanation:**B. Use the BigQuery Data Transfer Service from Cloud Storage.****Explanation of the Correct Answer**

The problem requires a low-cost, programmatic, and scalable solution to load multiple terabytes of daily clickstream data, stored as compressed JSON files in Cloud Storage, into BigQuery. The BigQuery Data Transfer Service from Cloud Storage is specifically designed for this use case. It allows for scheduled, recurring transfers of data from Cloud Storage to BigQuery, leveraging BigQuery's highly optimized and free batch loading capabilities. This makes it very scalable and cost-effective for large, frequent data loads.

Analysis of Each Option

Option A: Create a Cloud Run function to run a Python script to read and parse each JSON file, and use the BigQuery streaming insert API.

While programmatic and scalable for certain use cases, processing terabytes of data daily with custom Python scripts on Cloud Run and using streaming inserts can be inefficient and costly. Streaming inserts are better suited for real-time, smaller data volumes. The overhead of parsing and individual inserts for such a large volume would likely incur significant compute and network costs, making it less cost-effective than native batch loading.

Option B: Use the BigQuery Data Transfer Service from Cloud Storage.

This option is the most suitable. The BigQuery Data Transfer Service is built for large-scale, recurring data transfers from Cloud Storage to BigQuery. It is programmatic (can be configured via API, gcloud, or console), highly scalable as it uses BigQuery's optimized ingestion mechanisms, and cost-effective because BigQuery's batch loading is free, and the service itself has a low operational cost. It efficiently handles compressed JSON files.

Option C: Use Cloud Data Fusion to create a pipeline to load the JSON files into BigQuery.

Cloud Data Fusion is a powerful ETL tool for complex data transformations and orchestrations. While scalable, it is often overkill and more expensive for a straightforward data loading task like this, as it provisions and manages Dataproc clusters. For simply loading compressed JSON files, it would not be the most cost-effective solution.

Option D: Create an external table in BigQuery pointing to the Cloud Storage bucket and run the `INSERT INTO ... SELECT * FROM external_table` command.

This approach is programmatic and scalable. However, querying an external table incurs query costs for every read, including during the `INSERT INTO` operation. For terabytes of daily data, these query costs would accumulate rapidly, making it significantly more expensive than BigQuery's free native batch loading. Additionally, external tables might have performance limitations compared to direct ingestion for very large and frequent loads.

Key Takeaways

For large-scale, recurring batch data loads from Cloud Storage to BigQuery, especially when cost-effectiveness and scalability are critical, the BigQuery Data Transfer Service is the recommended solution. It leverages BigQuery's native, optimized, and free batch ingestion capabilities, making it superior to custom scripting, expensive ETL tools, or external tables that incur query costs.

Question: 439

You need to orchestrate a pipeline with several Google Cloud services: a batch Dataflow job, then a BigQuery query

job followed by a Vertex AI batch prediction. The logic is sequential. You want a lightweight, serverless orchestration solution with minimal operational overhead. What service should you use?

- A. Select Dataproc with Apache Oozie.
- B. Select Cloud Workflows.
- C. Select Compute Engine with cron.
- D. Select Cloud Composer.

Answer: B

Explanation:

B. Select Cloud Workflows.

Explanation of the Correct Answer

Cloud Workflows is the most suitable service for orchestrating a sequential pipeline involving Dataflow, BigQuery, and Vertex AI batch prediction, especially when a lightweight, serverless solution with minimal operational overhead is desired. Cloud Workflows is a fully managed, serverless orchestration service that allows you to define workflows as a series of steps using a declarative YAML or JSON syntax. It excels at calling various Google Cloud APIs in sequence, handling retries, and managing state without requiring any infrastructure management. This aligns perfectly with the requirements for a lightweight, serverless approach with minimal operational overhead.

Analysis of Each Option

Option A: Select Dataproc with Apache Oozie. Dataproc is a managed Apache Hadoop and Spark service, which is powerful for big data but not inherently serverless. Apache Oozie adds complexity and operational overhead. This option is heavy, not truly serverless, and has significant operational overhead, failing to meet the specified requirements.

Option B: Select Cloud Workflows. Cloud Workflows is a fully managed, serverless orchestration service designed for sequential API calls. It's lightweight, requires no infrastructure management, and has minimal operational overhead, making it an ideal fit for orchestrating the described pipeline.

Option C: Select Compute Engine with cron. Compute Engine provides virtual machines (IaaS), which are not serverless and require significant operational effort for management, patching, and scaling. Using cron for scheduling adds to the operational overhead. This option fails on the serverless, lightweight, and minimal operational overhead criteria.

Option D: Select Cloud Composer. Cloud Composer is a fully managed Apache Airflow service. While powerful for complex workflows, it provisions a cluster of VMs and is more "managed" than truly "serverless" in the pay-per-execution sense. For a simple sequential pipeline, it's often considered overkill and introduces more operational overhead and cost compared to Cloud Workflows.

Key Takeaways

For sequential orchestration of Google Cloud services with a strong emphasis on serverless operation, minimal overhead, and lightweight design, Cloud Workflows is the optimal choice. It provides a declarative way to define steps and handles the underlying infrastructure, making it highly efficient for API-driven workflows.

Question: 440

Your team runs a complex analytical query daily that processes terabytes of data. Recently, after running for 20

minutes, the query fails with a "Resources exceeded" error. You need to resolve this issue. What should you do?

- A. Increase your project's BigQuery API request quota.
- B. Increase the maximum table size limit.
- C. Move from BigQuery on-demand to slot reservations.
- D. Analyze the SQL syntax for errors.

Answer: C

Explanation:

C. Move from BigQuery on-demand to slot reservations.

Explanation of the Correct Answer

The problem describes a BigQuery query that fails with a "Resources exceeded" error after running for 20 minutes while processing terabytes of data. This error, especially in the context of large data volumes and long execution times, strongly indicates that the query is hitting the default slot limits or other shared resource constraints under BigQuery's on-demand pricing model. The on-demand model provides shared computational units (slots), and if a query requires more resources than are currently available or allowed by the default limits, it can fail with a "Resources exceeded" error. Moving to slot reservations (flat-rate pricing) allows you to purchase dedicated computational capacity (slots) for your project. These dedicated slots ensure that sufficient resources are always available for your queries, preventing such resource-related failures and providing more predictable performance for complex, resource-intensive analytical queries.

Analysis of Each Option

Option A: Increase your project's BigQuery API request quota.

API request quotas limit the number of API calls a project can make. A "Resources exceeded" error during query execution, after the query has already started and run for a significant time, is typically related to computational resources (CPU, memory, I/O) used during the query's processing, not the number of API calls made to initiate the query. Increasing the API request quota would not address the underlying computational resource bottleneck.

Option B: Increase the maximum table size limit.

BigQuery has very large limits for table sizes, and a query processing terabytes of data is well within these capabilities. The "Resources exceeded" error in this scenario refers to the computational resources needed for query execution, not the storage limits of the tables being read or written. This option is irrelevant to the problem described.

Option C: Move from BigQuery on-demand to slot reservations.

This is the most appropriate solution. BigQuery's on-demand model has default limits on the number of slots (computational units) available to a project. When a complex, data-intensive query processes terabytes of data for an extended period, it can exceed these shared resource limits, leading to a "Resources exceeded" error. Slot reservations provide dedicated computational capacity, ensuring that your queries have the necessary resources to complete successfully without hitting shared resource constraints.

Option D: Analyze the SQL syntax for errors.

The problem states that the query ran for 20 minutes before failing. This implies that the SQL syntax was correct enough for the query to start execution and process data. A SQL syntax error would typically cause an immediate failure at the parsing stage, not a "Resources exceeded" error after a prolonged run time.

Key Takeaways

1. "Resources exceeded" errors in BigQuery, especially for long-running, data-intensive queries, often indicate a bottleneck in computational resources (slots).
2. BigQuery's on-demand pricing model uses shared slots with default limits, which can be insufficient for very large or complex queries.
3. Moving to slot reservations (flat-rate pricing) provides dedicated computational resources, offering a reliable solution for resource-intensive workloads and preventing "Resources exceeded" errors related to slot availability.

Question: 441

CertyIQ

Your retail company is concerned about their BigQuery analytics spend. The company runs several queries that require the use of the same aggregation for the store ID and real-time sales volume. You need to implement the optimal solution that minimizes analytics spend and returns faster results. What should you do?

- A. Leverage partitioning to minimize the number of bytes read.
- B. Create a new table from a CSV file with the repeated aggregation for the other queries to reference for faster processing.
- C. Use join acceleration with primary and foreign keys to increase query joining to live data.
- D. Create a materialized view to minimize repetitive computations.

Answer: D

Explanation:

D. Create a materialized view to minimize repetitive computations.

Explanation of the Correct Answer

The problem asks for the optimal solution to minimize BigQuery analytics spend and return faster results for queries that repeatedly use the same aggregation for store ID and real-time sales volume. The most effective solution for this scenario is to use a materialized view.

A **materialized view** in BigQuery is a pre-computed result set of a query that is stored as a physical table. When the underlying base table data changes, BigQuery automatically and incrementally updates the materialized view. This means that the aggregated data (in this case, for store ID and real-time sales volume) is always up-to-date without manual intervention. When subsequent queries are run that can benefit from this pre-computed aggregation, BigQuery's optimizer can automatically rewrite the query to use the materialized view. This significantly reduces the amount of data scanned and computation performed, leading to faster query results and lower analytics spend.

Analysis of Each Option

Option A: Leverage partitioning to minimize the number of bytes read.

While partitioning is a valuable technique for reducing bytes read and improving query performance by scanning only relevant data segments, it primarily helps when queries filter data based on the partitioning column. It does not directly address the problem of recomputing the same aggregation multiple times. If the aggregation itself is complex and needs to be recomputed every time, partitioning alone won't prevent that recomputation. Therefore, it's not the optimal solution for the specific problem of repetitive aggregations.

Option B: Create a new table from a CSV file with the repeated aggregation for the other queries to reference for faster processing.

This approach involves manually pre-calculating the aggregation, exporting it to a CSV, and then re-importing it into a new BigQuery table. This is a highly inefficient and unsustainable method for "real-time sales volume."

The new table would quickly become stale, requiring frequent manual updates, which is prone to errors, costly in terms of operational overhead, and lacks automation. It is far from an optimal solution for minimizing spend and ensuring data freshness.

Option C: Use join acceleration with primary and foreign keys to increase query joining to live data.

Join acceleration techniques can improve the performance of queries involving joins. While efficient joins are important for overall query performance, this option does not directly address the core issue of repeatedly computing the same aggregation. The problem focuses on optimizing the reuse of a specific aggregation, not primarily on complex joins between multiple tables. Therefore, it's not the most direct or optimal solution for the stated problem.

Option D: Create a materialized view to minimize repetitive computations.

This is the optimal solution. A materialized view pre-computes and stores the results of a query (in this case, the aggregation for store ID and real-time sales volume). BigQuery automatically maintains and incrementally updates the materialized view as the underlying data changes. When subsequent queries need this aggregation, BigQuery can automatically use the pre-computed results from the materialized view, leading to significantly faster query execution and a substantial reduction in analytics spend by avoiding redundant computations. This approach is ideal for frequently queried aggregations on dynamic data.

Key Takeaways

1. **Materialized Views** are powerful tools in BigQuery for optimizing queries that involve repetitive aggregations or complex computations.
2. They reduce analytics spend by minimizing the amount of data scanned and processed.
3. They improve query performance by providing pre-computed results.
4. BigQuery's automatic and incremental updates ensure data freshness without manual intervention, making them suitable for dynamic data like "real-time sales volume."
5. While partitioning and join optimization are valuable, they do not directly solve the problem of repetitive aggregation computations as effectively as materialized views.

Question: 442

CertyIQ

Your company stores vital operational sales data in a BigQuery dataset in us-central1. Your company requires a disaster recovery plan to restore this data to us-east1 with a recovery point objective (RPO) of 24 hours and a recovery time objective (RTO) of 4 hours if us-central1 experiences an outage. You need to implement the disaster recovery plan while keeping costs and complexity to a minimum. What should you do?

- A. Manually export data to a CSV file in a multi-regional Cloud Storage bucket daily and use bq load to restore to us-east1.
- B. Take daily BigQuery table snapshots in us-central1.
- C. Configure BigQuery cross-region dataset replication from us-central1 to us-east1.
- D. Set up continuous queries and Pub/Sub to stream data changes from BigQuery tables in us-central1 to us-east1.

Answer: C

Explanation:

C. Configure BigQuery cross-region dataset replication from us-central1 to us-east1.

Explanation of the Correct Answer

BigQuery cross-region dataset replication is the most suitable solution for this scenario. It is a managed

service designed specifically for disaster recovery and business continuity for BigQuery datasets across different regions. This feature automatically and asynchronously replicates data from a source dataset in one region (us-central1) to a destination dataset in another region (us-east1). This ensures that a copy of the data is available in the target region with a low Recovery Point Objective (RPO) and a very fast Recovery Time Objective (RTO), as the replicated data is immediately queryable in the event of an outage in the primary region. It balances cost-effectiveness with minimal complexity, as it's a built-in BigQuery feature.

Analysis of Each Option

Option A: Manually exporting data daily to a CSV file in a multi-regional Cloud Storage bucket and then using bq load to restore to us-east1 meets the 24-hour RPO but is unlikely to meet the 4-hour RTO. The manual steps involved in exporting, storing, and then loading, especially during a disaster, introduce significant human latency and potential for errors, making a 4-hour RTO very challenging. This approach also adds considerable operational complexity and potential for data consistency issues.

Option B: Taking daily BigQuery table snapshots in us-central1 does not provide cross-region disaster recovery. If us-central1 experiences an outage, the snapshots stored within that region would also be unavailable. This option fails to address the core requirement of restoring data to us-east1 in case of a us-central1 outage.

Option C: Configuring BigQuery cross-region dataset replication from us-central1 to us-east1 is the ideal solution. This managed service provides continuous, asynchronous replication, ensuring a low RPO (well within 24 hours). In a disaster scenario, the replicated dataset in us-east1 is immediately available, allowing for a very fast RTO (easily within 4 hours) without any manual data movement or loading. It offers a good balance of cost and minimal complexity.

Option D: Setting up continuous queries and Pub/Sub to stream data changes from BigQuery tables in us-central1 to us-east1 would achieve a very low RPO and RTO. However, this approach involves building and maintaining a custom streaming pipeline, which significantly increases complexity and cost. It requires multiple services (Pub/Sub, potentially Dataflow/Cloud Functions, BigQuery streaming inserts) and considerable development and operational overhead, which goes against the requirement to keep costs and complexity to a minimum.

Key Takeaways

1. For disaster recovery with specific RPO/RTO requirements, especially across regions, managed services are often the most cost-effective and least complex solutions.
2. BigQuery cross-region dataset replication is designed for this exact use case, offering automated data redundancy with low RPO and RTO.
3. Manual processes or custom-built streaming solutions, while sometimes viable, typically introduce higher operational complexity, greater risk, and potentially higher costs compared to managed replication features.

Question: 443

CertyIQ

Your team has created several BigQuery curated datasets containing anonymized industry benchmark data. You want to make these datasets easily discoverable and accessible for querying by external partner companies within their own Google Cloud projects. You need a secure and scalable solution. What should you do?

- A. Grant the roles/bigquery.dataViewer IAM role to the partner group email addresses on the datasets.
- B. Export the datasets to partner-specific Cloud Storage buckets.
- C. Create authorized views for each dataset and grant access to each partner.

D. Publish the datasets as listings within BigQuery sharing (Analytics Hub).

Answer: D

Explanation:

D. Publish the datasets as listings within BigQuery sharing (Analytics Hub).

Explanation of the Correct Answer

Publishing datasets as listings within BigQuery sharing (Analytics Hub) is the most suitable solution because it directly addresses the requirements for secure, scalable, and discoverable data sharing with external partners, allowing them to query data within their own Google Cloud projects while incurring their own query costs.

Analysis of Each Option

Option A: Grant the roles/bigquery.dataViewer IAM role to the partner group email addresses on the datasets. This approach grants direct access to your project's datasets, leading to potential security risks, scalability issues for managing many partners, and importantly, your project would incur the query costs. It also lacks discoverability.

Option B: Export the datasets to partner-specific Cloud Storage buckets. This option does not allow partners to directly query the data in BigQuery. They would need to ingest the data, which adds complexity, latency, and creates data staleness issues. It's not designed for direct querying of live BigQuery data.

Option C: Create authorized views for each dataset and grant access to each partner. While authorized views offer a layer of security by restricting data access, partners would still be querying views within your project, meaning your project would incur the query costs. It also presents scalability challenges for managing many views and partner access.

Option D: Publish the datasets as listings within BigQuery sharing (Analytics Hub). Analytics Hub is specifically designed for this use case. It allows providers to share data securely and scalably. Partners can discover listings, subscribe, and then query the linked datasets from their own projects, incurring their own query costs. The data itself remains in the provider's project, ensuring control and real-time access.

Key Takeaways

For sharing BigQuery datasets with external partners who need to query the data within their own Google Cloud projects, BigQuery Analytics Hub is the recommended solution. It provides a secure, scalable, and cost-effective mechanism by enabling data discovery, allowing partners to incur their own query costs, and maintaining data control for the provider.

Question: 444

CertyIQ

Your company uses Looker Studio connected to BigQuery for reporting. Users are experiencing slow dashboard load times due to complex queries on a large table. The queries involve aggregations and filtering on several columns. You need to optimize query performance to decrease the dashboard load times. What should you do?

- A. Create a materialized view in BigQuery that pre-calculates the aggregations and filters used in the Looker Studio dashboards.
- B. Configure Looker Studio to use a shorter data refresh interval to ensure fresh data is always displayed.
- C. Use BigQuery BI Engine to accelerate query performance by caching frequently accessed data.
- D. Implement row-level security in BigQuery to restrict data access and reduce the amount of data processed by the queries

Answer: A

Explanation:

A. Create a materialized view in BigQuery that pre-calculates the aggregations and filters used in the Looker Studio dashboards.

Explanation of the Correct Answer

The problem describes slow dashboard load times in Looker Studio due to complex queries involving aggregations and filtering on a large BigQuery table. The most effective solution to optimize query performance in this scenario is to pre-calculate these aggregations and filters. A materialized view in BigQuery serves this exact purpose. By creating a materialized view, the heavy computational work of aggregating and filtering the data is performed once during the view's refresh cycle, rather than every time a dashboard query runs. Looker Studio can then query this pre-computed, smaller, and optimized materialized view, drastically reducing query execution time and improving dashboard load times. This approach directly addresses the core issue of complex, recurring computations.

Analysis of Each Option

Option A: Create a materialized view in BigQuery that pre-calculates the aggregations and filters used in the Looker Studio dashboards.

This option is correct. Materialized views are designed to store the pre-computed results of queries, including aggregations and filters. When Looker Studio queries the materialized view, it retrieves the pre-calculated data, which is much faster than running the complex query against the large base table every time. This directly solves the problem of slow queries involving aggregations and filtering.

Option B: Configure Looker Studio to use a shorter data refresh interval to ensure fresh data is always displayed.

A shorter data refresh interval means Looker Studio will query BigQuery more frequently. If the underlying queries are already slow, increasing the frequency of these slow queries will worsen performance, not improve it. This option would exacerbate the problem.

Option C: Use BigQuery BI Engine to accelerate query performance by caching frequently accessed data.

BigQuery BI Engine is an excellent tool for accelerating query performance by caching data in memory. It can certainly help improve dashboard load times. However, for complex queries involving specific aggregations and filters that are consistently used, a materialized view (Option A) offers a more direct and often more significant performance improvement by pre-computing the results entirely. While BI Engine would be beneficial, a materialized view directly addresses the pre-computation of the complex operations mentioned.

Option D: Implement row-level security in BigQuery to restrict data access and reduce the amount of data processed by the queries

Row-level security (RLS) is a security feature that restricts which rows a user can see. While it reduces the data visible to a user, it doesn't fundamentally optimize the underlying query's computational cost for aggregations across the entire dataset. The query might still need to process a large amount of data before applying the RLS filter. Its primary purpose is data access control, not performance optimization for complex aggregations.

Key Takeaways

For recurring, complex queries involving aggregations and filtering on large datasets, especially when used by dashboards, materialized views are a highly effective optimization strategy. They pre-compute the results, reducing the computational burden at query time. While caching solutions like BI Engine can help, pre-computation directly addresses the complexity of the queries themselves.

Question: 445

You are provided a dataset of customer interactions logs, currently stored as compressed JSON files in Cloud Storage. The data is highly nested and semi-structured and needs to be loaded into BigQuery for training a BigQuery ML model. You want to leverage BigQuery's native capabilities for handling this data type, while still maintaining its nested structure. What should you do?

- A. Load the JSON files directly into a BigQuery table using the JSON data type, leverage JSON functions in the BigQuery ML CREATE MODEL query for feature engineering.
- B. Flatten the JSON data into a wide, denormalized CSV format, then use a batch load job to load the data into BigQuery.
- C. Convert the JSON data into Avro format in Cloud Storage, and load it into BigQuery.
- D. Stream the JSON data directly into a BigQuery table with a STRING column, use JSON_EXTRACT functions in the BigQuery ML CREATE MODEL query.

Answer: A

Explanation:

A. Load the JSON files directly into a BigQuery table using the JSON data type, leverage JSON functions in the BigQuery ML CREATE MODEL query for feature engineering.

Explanation of the Correct Answer

The problem asks to load highly nested and semi-structured JSON data from Cloud Storage into BigQuery for BigQuery ML model training, while leveraging BigQuery's native capabilities and maintaining the nested structure. Option A directly addresses these requirements by utilizing BigQuery's JSON data type. This data type is specifically designed to store and query semi-structured JSON data efficiently, preserving its original hierarchical structure. Furthermore, BigQuery ML allows the use of JSON functions (like JSON_VALUE, JSON_QUERY) directly within the CREATE MODEL query for on-the-fly feature engineering, which is a powerful native capability.

Analysis of Each Option

Option B: Flattening the JSON data into a wide, denormalized CSV format would lead to a loss of the nested structure, which contradicts one of the core requirements. It also introduces additional preprocessing overhead and makes handling evolving schemas difficult.

Option C: While Avro is an excellent format for BigQuery due to its columnar nature and schema evolution support, converting JSON to Avro requires an extra processing step. BigQuery can directly ingest JSON, and the JSON data type is the most direct native solution for raw JSON data, specifically addressing the prompt's emphasis on handling "this data type" (JSON).

Option D: Storing JSON as a STRING column in BigQuery treats the data as opaque text. While JSON_EXTRACT functions can be used, they are generally less performant than using the dedicated JSON data type because BigQuery has to re-parse the string for every extraction. The JSON data type offers optimized storage and querying for JSON structures, making it a more native and efficient solution than a generic STRING column.

Option A: This option correctly identifies that BigQuery can load JSON files directly. The JSON data type in BigQuery is ideal for maintaining the nested structure of semi-structured data. It allows for efficient storage and querying of JSON, and its associated functions can be seamlessly integrated into BigQuery ML CREATE MODEL queries for feature engineering, leveraging BigQuery's native capabilities to their fullest.

Key Takeaways

1. BigQuery's JSON data type is the most effective way to handle semi-structured, nested JSON data while preserving its structure.
2. It allows direct loading of JSON files from Cloud Storage without intermediate conversion steps.
3. JSON functions can be used directly within BigQuery ML queries for flexible and efficient feature engineering.
4. Using the JSON data type is more performant and leverages BigQuery's native optimizations compared to storing JSON as a STRING and parsing it with JSON_EXTRACT.

Question: 446

CertyIQ

You manage your company's BigQuery data warehouse. You need to implement a solution that enables the data science team to modify data for experiments without affecting the original tables, while minimizing additional storage costs. What should you do?

- A. Set up authorized views in a shared dataset that reference the original tables.
- B. Create snapshots of all the tables and restore them for the data science team to use.
- C. Create table clones of all the tables for the data science team to use.
- D. Create a separate dataset with full copies of all the tables for each member of the data science team.

Answer: C

Explanation:

- Option C is correct because table clones in BigQuery are lightweight copies that initially share storage with the base table. This means that only the changes made to the clone consume additional storage, minimizing costs. The data science team can modify the cloned data without affecting the original tables, which meets the requirements of the problem.

- Table clones are designed for scenarios like this, where you need to experiment with data without altering the original source.

Analysis of Each Option

- Option A: Incorrect. Authorized views are read-only, so the data science team would not be able to modify the data for experiments.

- Option B: Incorrect. While snapshots provide a point-in-time copy, restoring them creates new, independent tables, which consumes more storage than clones. Also, snapshots are typically read-only.

- Option C: Correct. Table clones are the most storage-efficient way to allow data modification without affecting the original tables.

- Option D: Incorrect. Creating full copies of the tables for each team member would result in very high storage costs, which contradicts the requirement to minimize storage costs.

Key Takeaways

- Table clones are a cost-effective way to create writable copies of tables in BigQuery for experimentation or development purposes.
- Authorized views are useful for controlling access to data but do not allow data modification.
- Avoid creating full copies of tables when storage costs are a concern.

Correct Answer

- Option C: Create table clones of all the tables for the data science team to use.

Question: 447

CertyIQ

analyze user clickstream data to personalize content recommendations. The data arrives continuously and needs to be processed with low latency, including transformations such as sessionization (grouping clicks by user within a time window) and aggregation of user activity. You need to identify a scalable solution to handle millions of events each second and be resilient to late-arriving data. What should you do?

- A. Use Firebase Realtime Database for ingestion and storage, and Cloud Run functions for processing and analytics.
- B. Use Cloud Storage for ingestion, Dataproc with Apache Spark for batch processing, and BigQuery for storage and analytics.
- C. Use Cloud Data Fusion for ingestion and transformation, and Cloud SQL for storage and analytics.
- D. Use Pub/Sub for ingestion, Dataflow with Apache Beam for processing, and BigQuery for storage and analytics.

Answer: D

Explanation:

- Option D is correct because it uses Pub/Sub for scalable ingestion of high-volume data streams, Dataflow with Apache Beam for powerful stream processing that handles sessionization, aggregation, and late-arriving data, and BigQuery for scalable data warehousing and analytics. This combination is specifically designed for real-time stream processing and meets the requirements of the problem.

Analysis of Each Option

- Option A: Incorrect. Firebase Realtime Database is not designed for high-throughput ingestion and complex analytics at this scale, and Cloud Run would be complex to manage for millions of events per second.
- Option B: Incorrect. Dataproc (Spark) introduces latency due to batch processing, which does not meet the low-latency requirement for real-time personalization.
- Option C: Incorrect. Cloud Data Fusion is more suited for batch-oriented data integration, and Cloud SQL is not scalable enough for the data volume and analytical queries required.
- Option D: Correct. Pub/Sub handles ingestion, Dataflow performs the necessary transformations with low latency and handles late data, and BigQuery provides a scalable analytics platform.

Key Takeaways

- This question highlights the importance of choosing the right tools for real-time data processing.
- Pub/Sub, Dataflow, and BigQuery are a common and effective combination for handling high-volume, real-time data streams.
- Avoid using batch processing tools like Dataproc when low latency is a requirement.
- Be aware of the limitations of databases like Firebase Realtime Database and Cloud SQL when dealing with large-scale data and complex analytics.

Correct Answer

- Option D: Use Pub/Sub for ingestion, Dataflow with Apache Beam for processing, and BigQuery for storage

Question: 448

You are designing BigQuery tables for large volumes of clickstream event data. Your data analyst team will most frequently query by specific event date ranges and filter by the user ID UUID. You want to optimize table structure for query cost and performance. What should you do?

- A. Cluster the table by both the event date and the user ID columns.
- B. Partition the table by the event date column and cluster the table by user ID column.
- C. Create an ingestion-time partitioned table and cluster it by the user ID column.
- D. Partition the table by the user ID column and cluster the table by the event date column.

Answer: B

Explanation:

- The correct approach is to partition by event date and cluster by user ID. Partitioning by event date ensures that queries based on date ranges only scan relevant partitions, significantly reducing the amount of data processed. Clustering by user ID then optimizes filtering within those date partitions, improving the performance of queries that filter by user ID.

- This strategy aligns with best practices for optimizing BigQuery tables for common query patterns.

Analysis of Each Option

- Option A: Clustering by both event date and user ID is not optimal for date range queries. Clustering is most effective on a single column when dealing with date ranges.

- Option B: Partitioning by event date and clustering by user ID is the correct approach. It aligns perfectly with the query patterns, optimizing for both date range queries and user ID filtering.

- Option C: Ingestion-time partitioning is based on when the data is loaded, not the event date. This is not suitable when queries are based on the event date itself.

- Option D: Partitioning by user ID and clustering by event date is the opposite of what's needed. It would force BigQuery to scan all user ID partitions for each date range query, which is inefficient.

Key Takeaways

- Partitioning is most effective for columns frequently used in range filters (like dates).
- Clustering optimizes filtering within partitions.
- Choose partitioning and clustering columns based on the most common query patterns.
- Avoid partitioning on columns with high cardinality if range queries on other columns are more frequent.

Correct Answer

- Option B: Partition the table by the event date column and cluster the table by user ID column

Question: 449

Your organization stores highly personal data in BigQuery and needs to comply with strict data privacy regulations.

You need to ensure that sensitive data values are rendered unreadable whenever an employee leaves the organization. What should you do?

- A. Use customer-managed encryption keys (CMEK) and delete keys when employees leave the organization.
- B. Use column-level access controls with policy tags and revoke viewer permissions when employees leave the organization.
- C. Use AEAD functions and delete keys when employees leave the organization.
- D. Use dynamic data masking and revoke viewer permissions when employees leave the organization.

Answer: C

Explanation:

- The problem requires rendering sensitive data unreadable when an employee leaves the organization. AEAD functions in BigQuery allow encryption and decryption at the row or column level using a key. Deleting the key makes the data unreadable without affecting other data. This directly addresses the problem statement.

- AEAD (Authenticated Encryption with Associated Data) functions provide a granular way to encrypt specific sensitive data.

Analysis of Each Option

- Option A: Customer-managed encryption keys (CMEK) provide control over encryption keys used to protect data at rest. Deleting the key would make the data unreadable, but it would affect all users, not just those who have left the organization.

- Option B: Policy tags are used to classify data and control access. Revoking viewer permissions would prevent access, but it wouldn't render the data unreadable. Users with the correct permissions could still see the raw data.

- Option C: AEAD functions in BigQuery allow you to encrypt and decrypt data at the row or column level using a key. If you delete the key, the data becomes unreadable without affecting other data.

- Option D: Data masking transforms data values based on predefined rules. Revoking viewer permissions doesn't directly relate to the masking rules. While masking can protect data, it doesn't inherently make it unreadable if the masking rule is removed or changed.

Key Takeaways

- AEAD functions offer a granular way to encrypt specific sensitive data.

- Deleting the key used to encrypt the data when an employee leaves effectively renders that data unreadable to anyone without the key.

- CMEK affects all users, while AEAD functions can target specific data.

Correct Answer

- Option C: Use AEAD functions and delete keys when employees leave the organization

Question: 450

CertyIQ

You manage a BigQuery table that is used for critical end-of-month reports. The table is updated weekly with new sales data. You want to prevent data loss and reporting issues if the table is accidentally deleted. What should you do?

- A. Configure the time travel duration on the table to be exactly seven days. In case of deletion, re-create the

deleted table solely from the time travel data.

B. Create a view of the table. In case of deletion, re-create the deleted table from the view and time travel data.

C. Schedule the creation of a new snapshot of the table once a week. In case of deletion, re-create the deleted table using the snapshot and time travel data.

D. Create a clone of the table. In case of deletion, re-create the deleted table by copying the content of the clone.

Answer: C

Explanation:

- Option C is the most robust and cost-effective solution. Snapshots provide a periodic backup, and time travel covers the gap between the snapshot and any potential deletion. If the table is deleted, you can restore it from the most recent snapshot. Time travel can then be used to recover any changes made between the snapshot and the deletion time, if necessary.

Analysis of Each Option

- Option A: BigQuery's time travel feature allows you to query data as it existed at a specific point in the past. Setting the time travel duration to seven days means you can recover the table to its state from up to seven days ago. Since the table is updated weekly, this option ensures you can recover the table to its most recent state before deletion. Recovery is limited to the time travel window. If the deletion occurs and is not noticed within seven days, complete recovery is impossible.

- Option B: A view is a virtual table based on a query. It doesn't store data itself. If the underlying table is deleted, the view becomes useless. Time travel would still be needed to recover the data, but the view itself doesn't aid in the recovery process. A view does not back up the data. It depends on the underlying table.

- Option C: Snapshots are point-in-time copies of a table. Scheduling weekly snapshots provides a regular backup. If the table is deleted, you can restore it from the most recent snapshot. Time travel can then be used to recover any changes made between the snapshot and the deletion time, if necessary. Snapshots are cost-effective for long-term protection. Requires setting up a snapshot schedule. Recovery involves two steps (snapshot restore and potentially time travel).

- Option D: A clone is a writable copy of a table. While it provides a backup, it also incurs storage costs equivalent to the original table. This is less efficient than snapshots for this purpose. Higher storage costs compared to snapshots.

Key Takeaways

- Snapshots combined with time travel provide a balance between cost-effectiveness and data recovery capability.

- Understand the differences between views, snapshots, clones, and time travel in BigQuery.

- Consider the recovery time objective (RTO) and recovery point objective (RPO) when choosing a data protection strategy.

Correct Answer

- Option C: Schedule the creation of a new snapshot of the table once a week. In case of deletion, re-create the deleted table using the snapshot and time travel data.

Question: 451

CertyIQ

You are using Dataflow to build a streaming data pipeline to analyze user website click activity from Pub/Sub. You

need to calculate the number of clicks for each user site visit. A site visit is defined as a period of activity followed by 30 minutes of inactivity for a specific user. What should you do?

- A. Use tumbling windows with a 30 -minute window.
- B. Use hopping windows with a 30 -minute window, and a 1 -minute period.
- C. Use hopping windows with a 30 -minute window, and a 30 -minute period.
- D. Use session windows with a 30 -minute gap duration.

Answer: D

Explanation:

- Session windows are specifically designed to group events based on periods of activity separated by a defined gap of inactivity. In this scenario, a "site visit" is defined by user activity followed by 30 minutes of inactivity. Therefore, using session windows with a 30-minute gap duration perfectly aligns with the problem's requirements. This approach ensures that all clicks within a single site visit are grouped together, and visits are correctly separated based on the inactivity period.

Analysis of Each Option

- Option A: Tumbling windows divide the data stream into fixed, non-overlapping intervals, regardless of user activity. This is incorrect because a single site visit could be split across multiple tumbling windows, or a window might contain parts of multiple visits, failing to accurately represent site visits.
- Option B: Hopping windows with a 30-minute window and a 1-minute period create overlapping windows, analyzing the same data multiple times. This does not directly address the 30-minute inactivity requirement for defining a site visit and is computationally inefficient.
- Option C: Hopping windows with a 30-minute window and a 30-minute period are equivalent to tumbling windows. This option suffers from the same issues as option A, as it does not account for the inactivity period defining a site visit.
- Option D: Session windows group events together until a specified gap of inactivity occurs. A 30-minute gap duration perfectly matches the definition of a "site visit," making this the correct approach.

Key Takeaways

- Session windows are ideal for grouping events based on activity separated by periods of inactivity.
- Tumbling and hopping windows are not suitable for scenarios where event grouping depends on variable periods of inactivity.
- When choosing a windowing strategy, carefully consider how the problem defines event grouping and select the windowing type that best matches those criteria.
- A common mistake is to use fixed-size windows (tumbling or hopping) when session windows are more appropriate for grouping events based on activity patterns.

Correct Answer

- Option D: Use session windows with a 30-minute gap duration.

Question: 452

CertyIQ

Your company runs daily batch query jobs that process large datasets in BigQuery. Recently, you have noticed that some critical reports are delayed, and investigations show that BigQuery jobs are occasionally failing with "rate limit exceeded" errors. This is impacting your business operations and you need to investigate what is causing the

error. What should you do?

- A. Set up custom alerts in Cloud Monitoring for BigQuery query failures and configure notifications to be sent to the operations team.
- B. Review the BigQuery usage quotas in the Google Cloud console to see if the project is hitting any API limits for BigQuery.
- C. Examine the BigQuery job history in the BigQuery admin panel to identify long-running or resource-intensive queries that might be contributing to the issue.
- D. Review the BigQuery audit logs in Cloud Logging for detailed error messages and identify the specific queries causing the rate limit issues.

Answer: D

Explanation:

- The problem is that BigQuery jobs are failing due to "rate limit exceeded" errors, which delays critical reports. The best way to find out what's causing this is to look at the BigQuery audit logs in Cloud Logging. These logs give detailed error messages and show exactly which queries are causing the rate limit issues. This helps in identifying the root cause and taking action to fix it.

Analysis of Each Option

- Option A: Setting up alerts is good for knowing when failures happen, but it doesn't help in finding out **why** the rate limits are being exceeded in the first place. It's a reactive measure, not a diagnostic one.
- Option B: Checking BigQuery usage quotas in the Google Cloud console is a general check. It doesn't give specific details about which queries are causing the problem. You might see you're close to a limit, but not what's contributing the most.
- Option C: Examining the BigQuery job history can help find long-running queries, but it doesn't directly show **why** those queries are hitting rate limits. It's more useful for optimizing performance after you've found the problematic queries.
- Option D: Reviewing the BigQuery audit logs in Cloud Logging is the best first step because it provides detailed information about BigQuery operations. This includes the specific queries that failed, the exact error messages (pinpointing the type of rate limit exceeded), the user or service account that started the query, and timestamps to match with other events.

Key Takeaways

- When dealing with rate limit issues, the most important thing is to identify the specific queries causing the problem.
- Audit logs are a valuable resource for detailed information about what's happening in your BigQuery environment.
- Avoid jumping to reactive measures like setting up alerts before understanding the root cause of the issue.
- Don't rely solely on general checks like quota reviews without investigating the specific queries involved.

Correct Answer

- Option D: Review the BigQuery audit logs in Cloud Logging for detailed error messages and identify the specific queries causing the rate limit issues.

You are developing a fraud detection model using BigQuery ML. You have a raw transaction dataset and need to create new features such as the `average_transaction_amount_last_24_hours` and `time_since_last_transaction`. These features require aggregation and timewindow calculations on the existing data. The goal is to ensure that these features are consistently applied during both model training and prediction without manual intervention. You need to prepare these features efficiently for your model. What should you do?

- A. Use the `TRANSFORM` clause within the `CREATE ML MODEL` statement, leveraging SQL functions for aggregations and time-based calculations.
- B. implement a Cloud Run function that triggers on new transactions, calculates the features, and inserts them into a feature store before model serving.
- C. Export the BigQuery data to Cloud Storage, perform feature engineering using a custom Python script in a Dataflow job, and then re-import the engineered features into BigQuery.
- D. Create a separate BigQuery table containing pre-computed features using complex SQL queries and join this table with the raw data during model training and serving.

Answer: A

Explanation:

- The `TRANSFORM` clause in BigQuery ML is designed for feature engineering within the model definition. It allows defining SQL expressions to create new features directly from the input data. This ensures the same transformations are automatically applied during both training and prediction, maintaining consistency. BigQuery SQL supports window functions and other aggregation functions needed for time-window calculations.

- The `TRANSFORM` clause is the most efficient, consistent, and automated way to prepare features for a BigQuery ML model. It leverages BigQuery's built-in capabilities and avoids unnecessary data movement and external dependencies.

- Example:

```
```sql
```

```
CREATE OR REPLACE MODEL
```

```
`your_project.your_dataset.fraud_model` OPTIONS(model_type='logistic_reg') AS
```

```
SELECT
```

```
*
```

```
AVG(transaction_amount) OVER (ORDER BY transaction_time RANGE BETWEEN INTERVAL 24 HOUR
PRECEDING AND CURRENT ROW) AS avg_transaction_amount_last_24_hours,
```

```
TIMESTAMP_DIFF(CURRENT_TIMESTAMP(), MAX(transaction_time) OVER (), MINUTE) AS
time_since_last_transaction
```

```
FROM
```

```
`your_project.your_dataset.transactions`
```

```
```
```

Analysis of Each Option

- Option A: Correct. The `TRANSFORM` clause in BigQuery ML is specifically designed for feature engineering within the model definition. It allows you to define SQL expressions to create new features directly from the input data. This ensures that the same transformations are automatically applied during both training and prediction, maintaining consistency.

- Option B: Incorrect. While viable, this approach introduces extra complexity and latency. It requires setting up and managing a Cloud Run function, a feature store, and triggers. This adds overhead compared to using BigQuery's built-in capabilities. Also, maintaining consistency between the Cloud Run function's logic and the model's expectations can be challenging.

- Option C: Incorrect. This is the least efficient option. It involves exporting data, running a Dataflow job (which adds complexity and cost), and re-importing the data. This introduces significant overhead and is unnecessary when BigQuery ML can handle the feature engineering directly.

- Option D: Incorrect. This approach can work, but it requires managing a separate table and ensuring that the pre-computed features are kept up-to-date. It also adds complexity to the model training and serving queries, as you need to perform the join. The `TRANSFORM` clause provides a more streamlined and integrated solution.

Key Takeaways

- The `TRANSFORM` clause in BigQuery ML is a powerful tool for feature engineering within the model definition.

- It ensures consistency between training and prediction by applying the same transformations automatically.

- It is more efficient and less complex than using external services like Cloud Run or Dataflow for feature engineering.

- Avoid unnecessary data movement and external dependencies when BigQuery ML can handle the feature engineering directly.

Correct Answer

- Option A: Use the TRANSFORM clause within the CREATE MODEL statement, leveraging SQL functions for aggregations and time-based calculations.

Question: 454

CertyIQ

Your company stores vital operational sales data in a BigQuery dataset in us-central1. Your company requires a disaster recovery plan to restore this data to us-east1 with a recovery point objective (RPO) of 24 hours and a recovery time objective (RTO) of 4 hours if us-central1 experiences an outage. You need to implement the disaster recovery plan while keeping costs and complexity to a minimum. What should you do?

- A. Manually export data to a CSV file in a multi-regional Cloud Storage bucket daily and use bq ls to restore to us-east1.
- B. Configure BigQuery cross-region dataset replication from us-central1 to us-east1.
- C. Take daily BigQuery table snapshots in us-central1.
- D. Set up continuous queries and Pub/Sub to stream data changes from BigQuery tables in us-central1 to us-east1.

Answer: B

Explanation:

- Option B is the correct answer because it leverages BigQuery's built-in cross-region dataset replication feature. This ensures that data is automatically copied from `us-central1` to `us-east1`, providing a low Recovery Point Objective (RPO) and a reasonable Recovery Time Objective (RTO). The automated nature of this solution minimizes complexity and operational overhead, making it ideal for disaster recovery.

- BigQuery cross-region replication is a managed service, meaning Google handles the underlying

infrastructure and maintenance. This reduces the burden on the company's IT staff.

Analysis of Each Option

- Option A: This option is incorrect because manually exporting data to a CSV file and loading it into BigQuery is a slow and labor-intensive process. It would likely exceed the 4-hour RTO and requires manual intervention, increasing the risk of failure.
- Option B: This option is correct because BigQuery cross-region dataset replication automates the process of copying data to a different region. This ensures a low RPO and a reasonable RTO, while also minimizing complexity.
- Option C: This option is incorrect because taking snapshots in the same region (`us-central1`) does not protect against a regional outage. If `us-central1` is unavailable, the snapshots will also be inaccessible.
- Option D: This option is incorrect because setting up continuous queries and Pub/Sub is complex and costly. It requires significant data engineering effort to ensure data consistency and manage the infrastructure.

Key Takeaways

- BigQuery cross-region replication is a powerful feature for disaster recovery, providing automated data replication with low RPO and RTO.
- When evaluating disaster recovery solutions, consider the RPO, RTO, cost, and complexity of each option.
- Avoid manual processes and solutions that do not protect against regional outages.

Correct Answer

- Option B: Configure BigQuery cross-region dataset replication from us-central1 to us-east1.

Question: 455

CertyIQ

You are developing a fraud detection model using BigQuery ML. You have a raw transaction dataset and need to create new features such as the `average_transaction_amount_last_24_hours` and `time_since_last_transaction`. These features require aggregation and timewindow calculations on the existing data. The goal is to ensure that these features are consistently applied during both model training and prediction without manual intervention. You need to prepare these features efficiently for your model. What should you do?

- A. Use the `TRANSFORM` clause within the `CREATE MODEL` statement, leveraging SQL functions for aggregations and time-based calculations.
- B. implement a Cloud Run function that triggers on new transactions, calculates the features, and inserts them into a feature store before model serving.
- C. Export the BigQuery data to Cloud Storage, perform feature engineering using a custom Python script in a Dataflow job, and then re-import the engineered features into BigQuery.
- D. Create a separate BigQuery table containing pre-computed features using complex SQL queries and join this table with the raw data during model training and serving.

Answer: A

Explanation:

- The `TRANSFORM` clause in BigQuery ML is designed for feature engineering within the model definition. It allows defining SQL expressions to create new features directly from the input data. This ensures the same transformations are automatically applied during both training and prediction, maintaining consistency. BigQuery SQL supports window functions and other aggregation functions needed for time-window calculations.

- The `TRANSFORM` clause is the most efficient, consistent, and automated way to prepare features for a BigQuery ML model. It leverages BigQuery's built-in capabilities and avoids unnecessary data movement and external dependencies.

- Example:

```
```sql
```

```
CREATE OR REPLACE MODEL
```

```
`your_project.your_dataset.fraud_model` OPTIONS(model_type='logistic_reg') AS
```

```
SELECT
```

```
*,
```

```
AVG(transaction_amount) OVER (ORDER BY transaction_time RANGE BETWEEN INTERVAL 24 HOUR
PRECEDING AND CURRENT ROW) AS avg_transaction_amount_last_24_hours,
```

```
TIMESTAMP_DIFF(CURRENT_TIMESTAMP(), MAX(transaction_time) OVER (), MINUTE) AS
time_since_last_transaction
```

```
FROM
```

```
`your_project.your_dataset.transactions`
```

```
```
```

Analysis of Each Option

- Option A: Correct. The `TRANSFORM` clause in BigQuery ML is specifically designed for feature engineering within the model definition. It allows you to define SQL expressions to create new features directly from the input data. This ensures that the same transformations are automatically applied during both training and prediction, maintaining consistency.

- Option B: Incorrect. While viable, this approach introduces extra complexity and latency. It requires setting up and managing a Cloud Run function, a feature store, and triggers. This adds overhead compared to using BigQuery's built-in capabilities. Also, maintaining consistency between the Cloud Run function's logic and the model's expectations can be challenging.

- Option C: Incorrect. This is the least efficient option. It involves exporting data, running a Dataflow job (which adds complexity and cost), and re-importing the data. This introduces significant overhead and is unnecessary when BigQuery ML can handle the feature engineering directly.

- Option D: Incorrect. This approach can work, but it requires managing a separate table and ensuring that the pre-computed features are kept up-to-date. It also adds complexity to the model training and serving queries, as you need to perform the join. The `TRANSFORM` clause provides a more streamlined and integrated solution.

Key Takeaways

- The `TRANSFORM` clause in BigQuery ML is a powerful tool for feature engineering within the model definition.

- It ensures consistency between training and prediction by applying the same transformations automatically.

- It is more efficient and less complex than using external services like Cloud Run or Dataflow for feature engineering.

- Avoid unnecessary data movement and external dependencies when BigQuery ML can handle the feature

engineering directly.

Correct Answer

- Option A: Use the TRANSFORM clause within the CREATE MODEL statement, leveraging SQL functions for aggregations and time-based calculations.

Question: 456

CertyIQ

Your team uses Cloud Composer for complex, daily ETL pipelines. The same directed acyclic graph (DAG) needs to run in the development, staging, and production environments, but each environment requires different configurations such as Cloud Storage buckets or BigQuery datasets. You want an integrated solution to manage these configurations without duplicating DAG code. What should you do?

- A. Store configuration files directly in the DAGs folder and use Python to read the environment-specific file.
- B. Create separate Git branches for each environment with hardcoded configurations.
- C. Use Airflow variables/connections to set environment-specific settings.
- D. Create distinct IAM roles for each environment and grant each role the permission to access only that environment.

Answer: C

Explanation:

Correct Answer: C. Use Airflow variables/connections to set environment-specific settings.

Explanation: Airflow variables and connections are the standard, built-in way to parameterize a DAG across multiple environments. By storing environment-specific configuration values (such as bucket names, BigQuery datasets, or API endpoints) as Airflow variables or connections inside each Composer environment, you can keep a single shared DAG codebase. The DAG dynamically reads the variable values at runtime, allowing the same DAG to behave differently depending on whether it runs in dev, staging, or prod — without duplicating code.

Option A: Store configuration files directly in the DAGs folder and use Python to read the file.

This approach is incorrect because storing environment-specific configuration files in the DAGs folder forces you to deploy different files per environment, which leads to config drift and makes automated CI/CD more complex. Cloud Composer syncs the entire DAGs folder, so manually managing environment-specific config files is error-prone and not considered best practice.

Option B: Create separate Git branches for each environment with hardcoded configurations.

This option is incorrect because maintaining separate branches for each environment leads to code duplication, difficult merges, inconsistent deployments, and a high likelihood of diverging DAG logic. A single-source-of-truth DAG combined with environment-specific variables is far more scalable and maintainable.

Option D: Create distinct IAM roles for each environment and grant access only to that environment.

IAM roles manage permissions, not configuration. Even with environment-specific IAM roles, the DAG would still need to know which buckets or datasets to use. IAM alone does not provide the dynamic configuration mechanism needed to avoid duplicating code across environments.

Question: 457

CertyIQ

Your team is building data lake platform on Google Cloud. As a part of data foundation design, you are planning to store all the raw data in Cloud Storage. You are expecting to ingest approximately 25 GB of data a day and your billing department is worried about the increasing cost of storing old data. The current business requirements are: The old data can be deleted anytime. There is no predefined access pattern of the old data. The old data should be available instantly when accessed. There should not be any charges for data retrieval. What should you do to optimize for cost?

•

- A. Create the bucket with the Autoclass storage class feature.
- B. Create an Object Lifecycle Management policy to modify the storage class for data older than 30 days to nearline, 90 days to coldline, and 365 days to archive storage class. Delete old data as needed.
- C. Create an Object Lifecycle Management policy to modify the storage class for data older than 30 days to coldline, 90 days to nearline, and 365 days to archive storage class. Delete old data as needed.
- D. Create an Object Lifecycle Management policy to modify the storage class for data older than 30 days to nearline, 45 days to coldline, and 60 days to archive storage class. Delete old data as needed.

Answer: A

Explanation:

Correct Answer: A. Create the bucket with the Autoclass storage class feature. Explanation: Autoclass is designed specifically for situations where access patterns are unknown and storage must remain instantly available without retrieval fees. Autoclass automatically moves objects between Standard, Nearline, Coldline, and Archive while waiving retrieval fees and early-deletion fees. This meets all business requirements: old data remains instantly accessible, costs decrease automatically as data ages, and retrieval of archived objects is free. Since the old data can be deleted anytime and there is no predictable access pattern, Autoclass provides the best cost optimization without operational overhead. Option B: Lifecycle to Nearline → Coldline → Archive This option is incorrect because Nearline, Coldline, and Archive all charge retrieval fees and some impose minimum storage durations. Since the requirement states that old data must have no retrieval charges and must be available instantly, this tiering strategy does not meet the business constraints. Option C: Lifecycle to Coldline → Nearline → Archive This option is also incorrect because it suffers from the same issues: Nearline, Coldline, and Archive classes all impose retrieval costs and minimum storage durations. Switching between these tiers violates the requirement that retrieval be free and instant for old data. Option D: Aggressive lifecycle transitions after 30/45/60 days This option is incorrect because it places data into colder storage even faster, increasing the likelihood of retrieval costs and minimum retention charges. Since the business explicitly requires no retrieval fees and instant access, this lifecycle configuration is not suitable.

Question: 458

CertyIQ

You are administering BigQuery dataset that uses a customer-managed encryption key (CMEK). You need to share the dataset with a partner organization that does not have access to your CMEK. What should you do?

•

- A. Copy the tables you need to share to a dataset without CMEKs. Create a BigQuery sharing (Analytics Hub) listing for this dataset.
- B. Export the tables to parquet files to a Cloud Storage bucket and grant the storageinsights.viewer role on the bucket to the partner organization.
- C. Create an authorized view that contains the CMEK to decrypt the data when accessed.
- D. Provide the partner organization a copy of your CMEKs to decrypt the data.

Answer: A

Explanation:

- A. Copy the tables you need to share to a dataset without CMEKs. Create a BigQuery sharing (Analytics Hub)

listing for this dataset. This is the correct answer because a partner organization cannot query any BigQuery dataset encrypted with your CMEK unless they have decrypt permissions on that CMEK. Since the partner does not and should not have access to your key, the only secure and compliant method is to copy the data into a dataset that uses Google-managed encryption instead of CMEK. After copying, you can safely publish the data through Analytics Hub, which is designed for external data sharing. This approach keeps your CMEK-protected data secure while enabling the partner to access the shared dataset.

B. Export the tables to parquet files to a Cloud Storage bucket and grant the `storageinsights.viewer` role on the bucket to the partner organization. This option is incorrect because exporting tables does not automatically change the encryption. The exported objects still remain protected by the CMEK unless rewritten with a different encryption key. Furthermore, the `storageinsights.viewer` role does not grant the partner the ability to actually read the objects, making this approach both functionally and security-wise invalid for sharing data.

C. Create an authorized view that contains the CMEK to decrypt the data when accessed. This option is incorrect because authorized views do not override CMEK restrictions. Even if you create an authorized view, the querying user must still have decrypt access on the CMEK. Since the partner organization lacks that permission, they will not be able to query any view or table derived from the CMEK-encrypted dataset.

D. Provide the partner organization a copy of your CMEKs to decrypt the data. This option is completely incorrect and violates every security best practice. Sharing your encryption keys with an external organization exposes all data protected by that key, breaks compliance requirements, and creates unacceptable security risks. Encryption keys must always remain private to the owning organization.

Question: 459

CertyIQ

You have one BigQuery dataset which includes customers' street addresses. You want to retrieve all occurrences of street addresses from the dataset. What should you do?

- A. Create a deep inspection job on each table in your dataset with Cloud Data Loss Prevention and create an inspection template that includes the `STREET_ADDRESS` infoType.
- B. Create a de-identification job in Cloud Data Loss Prevention and use the masking transformation.
- C. Create a discovery scan configuration on your organization with Cloud Data Loss Prevention and create an inspection template that includes the `STREET_ADDRESS` infoType.
- D. Write a SQL query in BigQuery by using `REGEXP_CONTAINS` on all tables in your dataset to find rows where the word "street" appears.

Answer: A

Explanation:

A. Create a deep inspection job on each table in your dataset with Cloud Data Loss Prevention and create an inspection template that includes the `STREET_ADDRESS` infoType. This is the correct answer because a DLP deep inspection job is designed to scan all rows and columns in BigQuery tables to detect specific sensitive infoTypes, including `STREET_ADDRESS`. Deep inspection provides exact matches, supports structured data, and is the recommended method when you explicitly want to find all occurrences of sensitive data such as addresses inside BigQuery datasets. You can configure the job with an inspection template and run it across each table to extract all detected addresses.

B. Create a de-identification job in Cloud Data Loss Prevention and use the masking transformation. This is incorrect because a de-identification job is for transforming or masking sensitive data — not for locating sensitive data. You cannot retrieve or analyze all street addresses using de-identification alone. Masking hides data; it does not scan and return where the data exists.

C. Create a discovery scan configuration on your organization with Cloud Data Loss Prevention and create an inspection template that includes the `STREET_ADDRESS` infoType. This option is incorrect because discovery scans provide metadata-level classification, inventory, and monitoring. They detect the presence of sensitive data but do not return each individual occurrence or record. Discovery scans cannot retrieve all specific street address occurrences; they only summarize data classifications.

D. Write a SQL query in BigQuery using `REGEXP_CONTAINS` on all tables to find rows where the word "street" appears. This is

incorrect because street addresses can come in many forms (e.g., “St”, “Ave”, “Rd”, numbered streets, PO Boxes), and relying on a keyword like “street” produces inaccurate and incomplete results. It also cannot detect structured addresses without that keyword, and would miss standardized address formats. It is not a reliable or scalable solution compared to using Cloud DLP.

Question: 460

CertyIQ

You are developing a model to identify the factors that lead to sales conversions for your customers. You have completed processing your data. You want to continue through the model development lifecycle. What should you do next?

•

- A. Monitor your model performance, and make any adjustments needed.
- B. Use your model to run predictions on fresh customer input data.
- C. Test and evaluate your model on your curated data to determine how well the model performs.
- D. Delineate what data will be used for testing and what will be used for training the model.

Answer: C

Explanation:

C. Test and evaluate your model on your curated data to determine how well the model performs. Once data processing is complete, the next step in the model development lifecycle is to evaluate the model using validation/test data. This helps you quantify accuracy, precision, recall, AUC, or other metrics to determine whether the model is ready for deployment or needs improvement. You should not proceed to prediction, monitoring, or training adjustments until you have confidence in the model's performance. A. Monitor your model performance, and make any adjustments needed. This step happens after deployment, not immediately after data processing. Monitoring is part of the post-deployment lifecycle when a live model is running in production and requires retraining or drift detection. B. Use your model to run predictions on fresh customer input data. You should never run a model on fresh production data until after it has been evaluated and validated. Running predictions before evaluation is risky and may produce incorrect or misleading results. D. Delineate what data will be used for testing and what will be used for training the model.

This should already be done during the data preparation stage. If you are “continuing after data processing,” the data splits should already exist. You normally split data into train, validation, and test before training.

Question: 461

CertyIQ

You are planning to load some of your existing on-premises data into BigQuery on Google Cloud. You want to either stream or batch-load data, depending on your use case. Additionally, you want to mask some sensitive data before loading into BigQuery. You need to do this in a programmatic way while keeping costs to a minimum. What should you do?

•

- A. Use Cloud Data Fusion to design your pipeline, use the Cloud DLP plug-in to de-identify data within your pipeline, and then move the data into BigQuery.
- B. Use the BigQuery Data Transfer Service to schedule your migration. After the data is populated in BigQuery, use the connection to the Cloud Data Loss Prevention (Cloud DLP) API to de-identify the necessary data.
- C. Set up Datastream to replicate your on-premise data on BigQuery.
- D. Create your pipeline with Dataflow through the Apache Beam SDK for Python, customizing separate options within your code for streaming, batch processing, and Cloud DLP. Select BigQuery as your data sink.

Answer: D

Explanation:

D. Create your pipeline with Dataflow through the Apache Beam SDK for Python, customizing separate options within your code for streaming, batch processing, and Cloud DLP. Select BigQuery as your data sink. Dataflow + Apache Beam gives full flexibility for both streaming and batch ingestion, which fits your requirement. Beam pipelines can easily integrate with the Cloud DLP API for in-flight data masking before writing to BigQuery. This approach is also cost-efficient, because you pay only for Dataflow usage and API calls — no additional managed ingestion or integration platform is required. It's the recommended method for programmatic, customizable, low-cost ETL/ELT pipelines involving PII masking.

A. Use Cloud Data Fusion with the Cloud DLP plug-in to de-identify data and then load into BigQuery. Cloud Data Fusion does support DLP integration and can load data into BigQuery, but it is a more expensive, fully managed integration platform. Since the question emphasizes keeping costs to a minimum and building a programmatic solution, Data Fusion is not the best fit.

B. Use the BigQuery Data Transfer Service, then apply Cloud DLP after data is in BigQuery. BigQuery Data Transfer Service does not mask data before loading, and applying DLP after loading into BigQuery means sensitive data reaches the warehouse unmasked — which violates the requirement. Also, BDTs does not support streaming ingestion.

C. Set up Datastream to replicate on-premise data to BigQuery. Datastream is mainly used for change data capture (CDC) from Oracle/MySQL/PostgreSQL, and it does not support masking or transformation before loading. It also cannot handle generic files or custom data formats. This option does not meet your masking or flexibility requirements.

Question: 462**CertyIQ**

You are the data governance team and are implementing security requirements to deploy resources. You need to ensure that resources are limited to only the europe-west3 region. You want to follow Google-recommended practices. What should you do?

-

- A. Create a Cloud Run function to monitor all resources created and automatically destroy the ones created outside the europe-west3 region.
- B. Deploy resources with Terraform and implement a variable validation rule to ensure that the region is set to the europe-west3 region for all resources.
- C. Set the constraints/gcp.resourceLocations organization policy constraint to in: europe-west3-locations.
- D. Set the constraints/gcp.resourceLocations organization policy constraint to in: eu-locations.

Answer: C**Explanation:**

C. Set the constraints/gcp.resourceLocations organization policy constraint to in: europe-west3-locations.

This option correctly uses the organization policy constraints/gcp.resourceLocations, which is Google's recommended way to enforce where resources can be deployed. By setting this policy to in: europe-west3-locations, you ensure that only resources created in the europe-west3 region are allowed across your projects. This is proactive, centralized, secure, and fully aligned with governance best practices.

A. Create a Cloud Run function to monitor all resources created and automatically destroy the ones created outside the europe-west3 region. This option ("A") is reactive and unreliable. A Cloud Run function would only remove non-compliant resources after they are created, which violates governance and compliance policies. Google does not recommend automation cleanup tools for enforcing location restrictions — organization policies are the correct mechanism.

B. Deploy resources with Terraform and implement a variable validation rule to ensure that the region is set to the europe-west3 region for all resources. This option ("B") is insufficient because Terraform validation affects only deployments done through Terraform. Resources created via the GCP Console, gcloud CLI, APIs, or other tools would bypass this check. Governance must be enforced at the organization level, not at the IaC tool level.

D. Set the constraints/gcp.resourceLocations organization policy constraint to in: eu-locations. Option "D" is incorrect because eu-locations is not a valid or relevant location group for europe-west3. Using this constraint would not enforce the required regional restriction. Only an

Question: 463

CertyIQ

You have variety of files in Cloud Storage that your data science team wants to use in their models. Currently, users do not have a method to explore, cleanse, and validate the data in Cloud Storage. You are looking for a low code solution that can be used by your data science team to quickly cleanse and explore data within Cloud Storage. What should you do?

- - A. Create an external table in BigQuery and use SQL to transform the data as necessary. Provide the data science team access to the external tables to explore the raw data.
 - B. Provide the data science team access to Dataprep to prepare, validate, and explore the data within Cloud Storage.
 - C. Load the data into BigQuery and use SQL to transform the data as necessary. Provide the data science team access to staging tables to explore the raw data.
 - D. Provide the data science team access to Dataflow to create a pipeline to prepare and validate the raw data and load data into BigQuery for data exploration.

Answer: B

Explanation:

B. Provide the data science team access to Dataprep to prepare, validate, and explore the data within Cloud Storage. This is the ideal solution because Dataprep (now Cloud Dataprep by Trifacta) is a low-code, visual, interactive data preparation tool designed for exploring, cleansing, profiling, and validating data stored in Cloud Storage. It is specifically built for data science and analytics teams who need to transform raw files without writing complex code or pipelines. It provides instant data previews, suggestions, anomaly detection, and easy export options. This is exactly what the question is asking for.

A. Create an external table in BigQuery and use SQL to transform the data as necessary. Option A gives the team SQL-based access but does not provide a low-code interactive data exploration UI. Also, SQL is not ideal for cleansing semi-structured or inconsistent raw files. This fails the requirement for a “low-code solution” and “quick exploration.”

C. Load the data into BigQuery and use SQL to transform the data as necessary. This option (“C”) creates unnecessary BigQuery ingestion costs and still relies completely on SQL. It does not offer the low-code, visual data cleansing interface the data science team needs. It also adds latency due to ingestion steps.

D. Provide the data science team access to Dataflow to create a pipeline. This option (“D”) is high-code and overly complex. Dataflow requires building pipelines, testing, deploying, and maintaining them — not suited for quick ad-hoc exploration and cleansing. It is the opposite of a low-code rapid-exploration tool.

Question: 464

CertyIQ

You are operating a Dataflow streaming pipeline. The pipeline aggregates events from a Pub/Sub subscription source, within a window, and sinks the resulting aggregation to a Cloud Storage bucket. The source has consistent throughput. You want to monitor and alert on behavior of the pipeline with Cloud Monitoring to ensure that it is processing data. What alerts should you create?

- - A. An alert based on an increase of subscription/num_undelivered_messages for the source and a rate of change decrease of instance/storage/used_bytes for the destination.
 - B. An alert based on a decrease of subscription/num_undelivered_messages for the source and a rate of change increase of instance/storage/used_bytes for the destination.
 - C. An alert based on an increase of instance/storage/used_bytes for the source and a rate of change decrease of subscription/num_undelivered_messages for the destination.
 - D. An alert based on a decrease of instance/storage/used_bytes for the source and a rate of change increase of subscription/num_undelivered_messages for the destination.

Answer: A

Explanation:

A. An alert based on an increase of subscription/num_undelivered_messages for the source and a rate-of-change decrease of instance/storage/used_bytes for the destination. This is the correct monitoring strategy. If the Dataflow pipeline stops processing or slows down, the number of undelivered Pub/Sub messages will increase, because messages accumulate in the subscription backlog. At the same time, the Cloud Storage bucket will stop growing, because the downstream sink is no longer being written to. Monitoring these two conditions together ensures that you catch a pipeline stall early and reliably.

B. An alert based on a decrease of subscription/num_undelivered_messages and an increase of instance/storage/used_bytes. This scenario indicates that the pipeline is functioning normally: fewer undelivered messages means consumption is healthy, and increasing Cloud Storage usage means results are being written. This is the opposite of a failure condition.

C. An alert based on an increase of instance/storage/used_bytes for the source and a decrease of subscription/num_undelivered_messages. This option confuses metrics: Cloud Storage is the destination, not the source. Also, decreasing undelivered messages means the pipeline is processing correctly, not failing.

D. An alert based on a decrease of instance/storage/used_bytes for the source and an increase of subscription/num_undelivered_messages. This mixes source and destination incorrectly — Cloud Storage is not the source. While an increase in undelivered messages is relevant, pairing it with a decrease in Cloud Storage from the "source" is nonsensical based on the actual architecture.

Question: 465

CertyIQ

You have 100 GB data stored in a BigQuery table. This data is outdated and will only be accessed one or two times a year for analytics with SQL. For backup purposes, you want to store this data to be immutable for 3 years. You want to minimize storage costs. What should you do?

•

- A. 1. Create a BigQuery table clone. 2. Query the clone when you need to perform analytics.
- B. 1. Create a BigQuery table snapshot. 2. Restore the snapshot when you need to perform analytics.
- C. 1. Perform a BigQuery export to a Cloud Storage bucket with archive storage class. 2. Enable versioning on the bucket. 3. Create a BigQuery external table on the exported files.
- D. 1. Perform a BigQuery export to a Cloud Storage bucket with archive storage class. 2. Set a locked retention policy on the bucket. 3. Create a BigQuery external table on the exported files.

Answer: D

Explanation:

D. 1. Perform a BigQuery export to a Cloud Storage bucket with archive storage class. 2. Set a locked retention policy on the bucket. 3. Create a BigQuery external table on the exported files. This is the correct solution because it minimizes storage costs while ensuring immutability for 3 years. Cloud Storage archive class provides the lowest cost for data accessed rarely, and a locked retention policy ensures the exported files cannot be deleted or modified before the retention period ends. Creating a BigQuery external table allows you to query the archived data directly without re-importing it into BigQuery, providing a cost-efficient way to perform occasional analytics.

A. 1. Create a BigQuery table clone. 2. Query the clone when you need to perform analytics. This option is incorrect because table clones reside in BigQuery and are charged at full storage cost, even if the data is rarely accessed. Clones are not the lowest-cost storage solution for long-term, infrequently accessed data.

B. 1. Create a BigQuery table snapshot. 2. Restore the snapshot when you need to perform analytics. This option is incorrect because snapshots are still stored in BigQuery and incur standard BigQuery storage costs. They are useful for short-term backups or rollback, but not for 3-year immutable archival at minimal cost.

C. 1. Perform a BigQuery export to a Cloud Storage bucket with archive storage class. 2. Enable versioning on the bucket. 3. Create a BigQuery external table on the exported files. This option is partially correct, but enabling versioning alone does not enforce immutability. Versions can

be deleted or overwritten by users with the right permissions. Since the requirement is data must be immutable for 3 years, a locked retention policy is required instead of just versioning.

Question: 466

CertyIQ

You have table that contains millions of rows of sales data, partitioned by date. Various applications and users query this data many times a minute. The query requires aggregating values by using AVG, MAX, and SUM, and does not require joining to other tables. The required aggregations are only computed over the past year of data, though you need to retain full historical data in the base tables. You want to ensure that the query results always include the latest data from the tables, while also reducing computation cost, maintenance overhead, and duration. What should you do?

•

- A. Create a view to aggregate the base table data. Include a filter clause to specify the last year of partitions.
- B. Create a materialized view to aggregate the base table data. Include a filter clause to specify the last one year of partitions.
- C. Create a new table that aggregates the base table data. Include a filter clause to specify the last year of partitions. Set up a scheduled query to recreate the new table every hour.
- D. Create a materialized view to aggregate the base table data. Configure a partition expiration on the base table to retain only the last one year of partitions.

Answer: B

Explanation:

B. Create a materialized view to aggregate the base table data. Include a filter clause to specify the last one year of partitions. This is correct because a materialized view precomputes aggregations and stores them internally, reducing computation cost and query duration for frequent queries. By filtering only the last year of partitions, the view focuses on the data needed, while the base table retains full historical data. Materialized views in BigQuery automatically refresh to include the latest data, which ensures queries always return up-to-date results without requiring manual scheduling or complex maintenance.

A. Create a view to aggregate the base table data. Include a filter clause to specify the last year of partitions. This is incorrect because a standard view does not precompute data. Every time the view is queried, BigQuery scans the base table and recomputes aggregations, which increases query cost and latency. It also provides no performance benefit for high-frequency queries compared to a materialized view.

C. Create a new table that aggregates the base table data. Include a filter clause to specify the last year of partitions. Set up a scheduled query to recreate the new table every hour. This is incorrect because maintaining a separate aggregated table with scheduled queries adds operational overhead and may introduce delays. It does not automatically include the latest data between runs and requires monitoring to ensure updates run correctly. Materialized views achieve the same result automatically and more efficiently.

D. Create a materialized view to aggregate the base table data. Configure a partition expiration on the base table to retain only the last one year of partitions. This is incorrect because retaining only the last year of partitions in the base table violates the requirement to keep full historical data. While materialized views still improve performance, deleting older partitions from the base table is not acceptable if historical data needs to be preserved.

Question: 467

CertyIQ

You are migrating on-premises data warehouse to BigQuery. One of the upstream data sources resides on a MySQL database that runs in your on-premises data center with no public IP addresses. You want to ensure that the data ingestion into BigQuery is done securely and does not go through the public internet. What should you do?

- A. Use Datastream to replicate data from your on-premises MySQL database to BigQuery. Use Forward-SSH tunnel as the connectivity method to establish a secure tunnel between Datastream and your on-premises MySQL database through a tunnel server in your on-premises data center. Use None as the encryption type when setting up the connection profile in Datastream.

- B. Use Datastream to replicate data from your on-premises MySQL database to BigQuery. Set up Cloud Interconnect between your on-premises data center and Google Cloud. Use Private connectivity as the connectivity method and allocate an IP address range within your VPC network to the Datastream connectivity configuration. Use Server-only as the encryption type when setting up the connection profile in Datastream.
- C. Use Datastream to replicate data from your on-premises MySQL database to BigQuery. Gather Datastream public IP addresses of the Google Cloud region that will be used to set up the stream. Add those IP addresses to the firewall allowlist of your on-premises data center. Use IP Allowlisting as the connectivity method and Server-only as the encryption type when setting up the connection profile in Datastream.
- D. Update your existing on-premises ETL tool to write to BigQuery by using the BigQuery Open Database Connectivity (ODBC) driver. Set up the proxy parameter in the `simba.googlebigqueryodbc.ini` file to point to your data center's NAT gateway.

Answer: B

Explanation:

B. Use Datastream to replicate data from your on-premises MySQL database to BigQuery. Set up Cloud Interconnect between your on-premises data center and Google Cloud. Use Private connectivity as the connectivity method and allocate an IP address range within your VPC network to the Datastream connectivity configuration. Use Server-only as the encryption type when setting up the connection profile in Datastream. This is correct because Datastream supports real-time replication from MySQL to BigQuery, and Private connectivity via Cloud Interconnect ensures that the data does not traverse the public internet. By allocating an IP range within your VPC, Datastream can securely connect to your on-premises database over the private link. Using Server-only encryption ensures secure transmission without exposing encryption keys unnecessarily.

A. Use Datastream with Forward-SSH tunnel and None encryption Option A is incorrect because using Forward-SSH tunnel requires a tunnel server and is complex to manage at scale. Additionally, selecting None as the encryption type would transmit data in plaintext, which violates security best practices. This does not meet the requirement for secure, private ingestion.

C. Use Datastream public IPs with IP allowlisting and Server-only encryption Option C is incorrect because it relies on Datastream public IPs, meaning traffic still flows over the public internet, which contradicts the requirement to avoid public internet paths. IP allowlisting does not provide private connectivity. Although Server-only encryption secures the data in transit, the data still traverses public networks.

D. Use an on-premises ETL tool with BigQuery ODBC driver via NAT proxy Option D is incorrect because using ODBC through a NAT gateway is not a secure or scalable method for continuous replication. It also requires opening network paths and may expose credentials or data to the public internet, which violates the security requirement. Datastream with private connectivity is the recommended approach for cloud-native, secure replication.

Question: 468

CertyIQ

You have data analyst team member who needs to analyze data by using BigQuery. The data analyst wants to create a data pipeline that would load 200 CSV files with an average size of 15MB from a Cloud Storage bucket into BigQuery daily. The data needs to be ingested and transformed before being accessed in BigQuery for analysis. You need to recommend a fully managed, no-code solution for the data analyst. What should you do?

- A. Create a pipeline by using BigQuery pipelines and schedule it to load the data into BigQuery daily.
- B. Create a Cloud Run function and schedule it to run daily using Cloud Scheduler to load the data into BigQuery.
- C. Build a custom Apache Beam pipeline and run it on Dataflow to load the file from Cloud Storage to BigQuery and schedule it to run daily using Cloud Composer.
- D. Use the BigQuery Data Transfer Service to load files from Cloud Storage to BigQuery, create a BigQuery job which transforms the data using BigQuery SQL and schedule it to run daily.

Answer: D

Explanation:

D. Use the BigQuery Data Transfer Service to load files from Cloud Storage to BigQuery, create a BigQuery job which transforms the data using BigQuery SQL and schedule it to run daily. This is the best solution because BigQuery Data Transfer Service (DTS) can natively ingest CSV files from Cloud Storage into BigQuery in a fully managed, no-code way. After loading, a scheduled BigQuery SQL job can perform the required transformations. This approach avoids custom code, is fully managed, and allows the data analyst to easily schedule daily ingestion and transformation without managing infrastructure.

A. Create a pipeline by using BigQuery pipelines and schedule it to load the data into BigQuery daily. This option is incorrect because BigQuery pipelines are not a fully managed, no-code solution for loading files from Cloud Storage. It generally requires SQL or additional orchestration and does not provide built-in, automated ingestion from multiple CSV files.

B. Create a Cloud Run function and schedule it to run daily using Cloud Scheduler to load the data into BigQuery. This option is incorrect because it involves writing and maintaining custom code, which is not a no-code solution. While it can technically accomplish the ingestion, it introduces operational overhead, monitoring, and scalability concerns for 200 CSV files daily.

C. Build a custom Apache Beam pipeline and run it on Dataflow to load the file from Cloud Storage to BigQuery and schedule it to run daily using Cloud Composer. This option is incorrect because it is high-code and intended for complex transformations or streaming pipelines. For the described use case (daily batch of CSV files and simple transformations), it is overkill and not a no-code, fully managed solution.

Question: 469

CertyIQ

You monitor and optimize the BigQuery instance your team. You notice that a particular daily report that uses a large JOIN operation is consistently slow. You want to examine the query's execution plan to identify potential performance bottlenecks within the JOIN as quickly as possible. What should you do?

•

- A. Review the BigQuery audit logs in Cloud Logging.
- B. Run a query on the INFORMATION_SCHEMA.JOBS_BY_PROJECT view filtering by the job_ID and analyze total_bytes_processed.
- C. Use the bq query --dry_run command to review the estimated number of bytes read and review query syntax.
- D. Leverage BigQuery's Query History view and analyze the execution graph.

Answer: D

Explanation:

D. Leverage BigQuery's Query History view and analyze the execution graph. This is correct because Query History in the BigQuery console provides a detailed visual execution plan of queries, including JOIN operations. The execution graph shows stages, slot allocation, data shuffling, and execution times, allowing you to identify performance bottlenecks quickly. This is the most direct and actionable way to optimize a slow query without relying on indirect metrics.

A. Review the BigQuery audit logs in Cloud Logging. This is incorrect because audit logs record metadata about query execution (who ran it, when, cost), but they do not provide detailed execution plans or insights into JOIN performance. Logs are useful for auditing but not for query optimization.

B. Run a query on the INFORMATION_SCHEMA.JOBS_BY_PROJECT view filtering by the job_ID and analyze total_bytes_processed. This is incorrect because INFORMATION_SCHEMA.JOBS_BY_PROJECT provides job metadata such as total bytes processed, execution time, and user info. While it helps monitor cost and usage, it does not give a detailed execution graph to analyze JOIN bottlenecks.

C. Use the bq query --dry_run command to review estimated bytes read and query syntax. This is incorrect because a dry run only estimates the number of bytes a query would read and validates syntax. It does not execute the query or provide insights into execution stages, JOIN performance, or actual bottlenecks.

Thank you

Thank you for being so interested in the premium exam material.
I'm glad to hear that you found it informative and helpful.

If you have any feedback or thoughts on the bumps, I would love to hear them.
Your insights can help me improve our writing and better understand our readers.

Best of Luck

You have worked hard to get to this point, and you are well-prepared for the exam
Keep your head up, stay positive, and go show that exam what you're made of!

Feedback

More Papers



Future is Secured

100% Pass Guarantee



24/7 Customer Support

Mail us - certyiqofficial@gmail.com



Free Updates

Lifetime Free Updates!

Total: **469 Questions**

Link: <https://certyiq.com/papers/google/professional-data-engineer>